

The Java Full-Stack Guide

One system. Every boundary.

Java, Spring, data, identity, distributed systems and delivery, connected through the fictional IntegrationHub.

28 / 28

Java 21+

00

Chapters available

Example baseline

Start with the system

Browser / Edge / Service / Data / Events / UI

Offline edition. Diagrams are embedded as vector graphics. Java lab execution status is stated in each chapter. Code is copyable; syntax highlighting is unavailable. IntegrationHub is fictional; no interview outcome is guaranteed.

Chapter Directory

28 of 28 chapters available. Planned destinations reserve stable links; they contain no completed explanations.

00	The System Before the Details	Available	22	Distributed Systems Theory	Available
01	Core Java and the JVM	Available	11	Object Design, Patterns and LLD	Available
02	Concurrency and the Memory Model	Available	12	Twelve System Design Case Studies	Available
21	Networking and Operating Systems	Available	27	Fintech and Payments Engineering	Available
03	Spring and Spring Boot Internals	Available	13	Data Integration and iPaaS	Available
04	Hibernate and JPA	Available	14	Docker and Container Internals	Available
06	Databases and Durable State	Available	15	Kubernetes and Production Workloads	Available
05	APIs and Real-Time Communication	Available	16	Build, Ship and Roll Back	Available
07	Messaging, Batch and Streams	Available	17	Cloud and Infrastructure	Available
26	Data Platform Basics	Available	18	Observability and Reliability	Available
08	Security, Identity and Data Protection	Available	23	JVM Performance Engineering	Available
09	The Browser and the Java Backend	Available	19	Testing and Quality	Available
10	System Design Foundations	Available	24	IntegrationHub in Motion	Available
25	Architecture Styles and Domain Design	Available	20	Appendix: Interview Toolkit and Index	Available

oo / The System Before the Details

The Java Full-Stack Guide

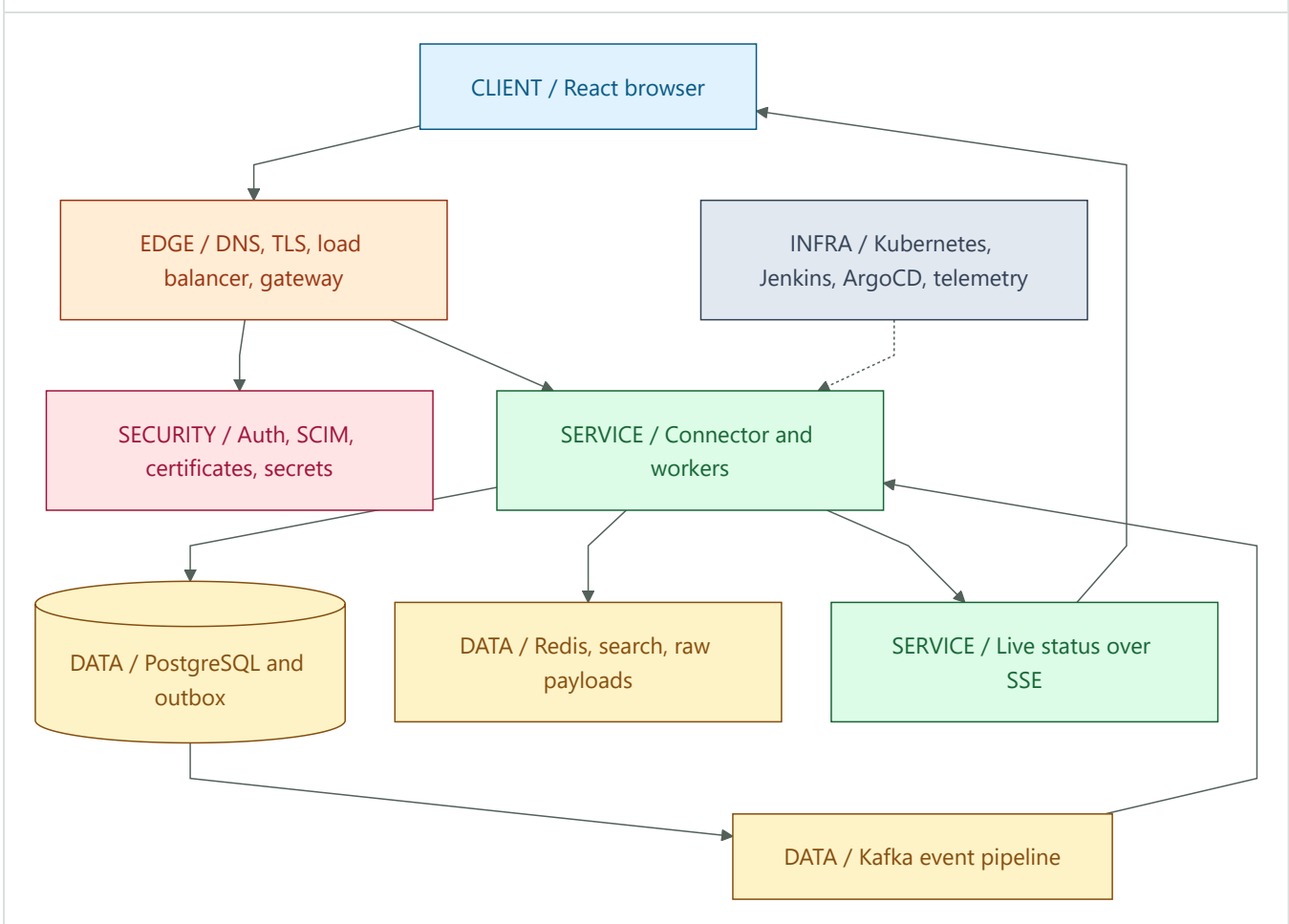
A connected reference for Java backend and full-stack interviews. IntegrationHub, the system throughout this guide, is fictional. Its architecture is a teaching device, not a description of any employer or client.

Version assumptions. Java 21 is the runnable-example baseline; later Java releases will be distinguished where relevant. Spring Boot 4.0.x and Spring Cloud 2025.1.x are the provisional framework families: the [official compatibility table](#) was consulted on 2026-10-05 and lists that pairing. Chapter 03 will cover migration differences from Boot 3.x. PostgreSQL 17, Kafka 4.x, and Kubernetes 1.34 are illustrative infrastructure baselines, not recommendations to deploy an old patch. This chapter does not require those runtimes.

[VERIFY: exact patches, support windows, and dependency/plugin compatibility must be rechecked against Spring's compatibility table, the Spring Boot system requirements, and the PostgreSQL, Kafka, Kubernetes and JDK vendor release documentation before executable examples are pinned.]

Big Picture

Diagram 00.1



The arrows show responsibility and data movement, not a requirement that every component be deployed separately. In particular, the identity service is not contacted for every request when a service can validate a signed access token using trusted keys and the required claim checks.

What You Will Be Able to Explain

- Follow a request from a browser through the network, an authorization boundary, a database commit, a message, and a UI update.
- Say what is durable at each point, who owns a decision, and which outcomes are still uncertain after a timeout.
- Separate a database transaction from a business workflow and from message delivery guarantees.
- Choose the next chapter from a concrete interview gap instead of memorizing a disconnected tool list.

- Explain one mechanism, one trade-off, and one failure case before reaching for product names.

[?] INTERVIEW

An interviewer can change the framework or cloud provider without changing the underlying question. Be able to identify the invariant: what must remain true when a request is duplicated, a process pauses, or a connection breaks?

How to Read This Guide

Start with the request journey below. For each unfamiliar boundary, follow its chapter link. A chapter should leave you able to explain the mechanism aloud, predict a failure, and justify a design choice. Definitions alone do not establish that understanding.

There are three useful passes. First, read each big picture and its IntegrationHub placement. Second, work through traces, runnable examples, and failure cases. Third, answer the interview corner without looking, then compare your answer with the model answer. For code, change an assumption and rerun the test; for design, change a constraint and reconsider the decision.

Recommended reading order: 00, 01, 02, 21, 03, 04, 06, 05, 07, 26, 08, 09, 10, 25, 22, 11, 12, 27, 13, 14, 15, 16, 17, 18, 23, 19, 24, then 20.

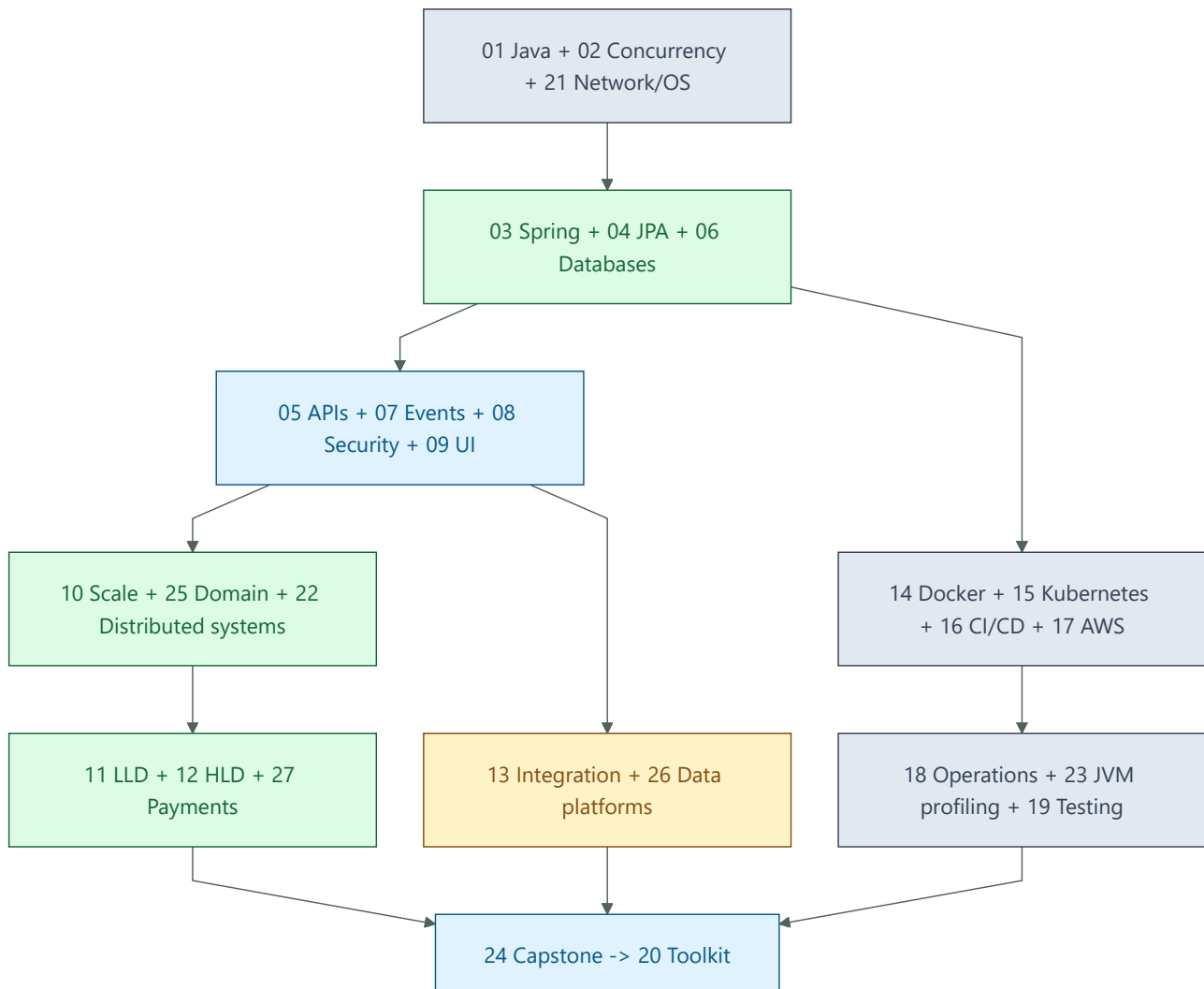
Chapter numbers are permanent identifiers, not prerequisites. The order puts networking before framework request handling, databases before API durability promises, architecture before case studies, and JVM profiling after operational diagnosis. The capstone is the last narrative chapter; the interview toolkit is the final appendix.

Reading route	Use when	Avoid when	Start here
Connected foundation	You can write Spring code but cannot trace what happens underneath	You are trying to locate one precise failure during an incident	00 -> 01 -> 02 -> 21 -> 03
Production diagnosis	You have a symptom and can identify its boundary	You are guessing the failing layer from a dashboard color	18 Operations, then 23, 06, or 21
Design interview	You can reason about APIs and persistence and need to defend decisions	You are substituting memorized diagrams for requirements	10 Foundations -> 25 -> 22 -> 11/12 -> 27
Revision	You have already worked the mechanisms and need retrieval practice	You have not yet understood the basic request journey	Cheat sheets; 20 Toolkit, planned

This guide intentionally excludes DSA/LeetCode, ML/AI engineering, mobile, Spark/Hadoop, blockchain, embedded systems, and hard real-time systems. It cannot guarantee passing any interview. A hiring process may independently assess algorithms, communication, or domain experience that a reference document cannot substitute for.

Chapter Dependency Map

Diagram 00.2



These are learning dependencies, not software imports. Testing is shown near operations for revision, but examples are tested from the first executable chapter. Use the [chapter directory](#) for individual chapter links and publication status.

IntegrationHub: One System, Explicit Boundaries

A tenant connects a source application, maps fields, starts a bulk synchronization, and watches progress. Work can continue after the browser closes. Different tenants must not access one another's credentials, payloads, or results. A retry must not silently turn one requested job into two.

[P] PRODUCTION

Where this lives in IntegrationHub. The same sync job connects browser behavior, Spring request handling, durable state, event delivery, and production diagnosis. Later chapters deepen those mechanisms without inventing a different system.

What Owns What?

Component	Responsibility	Must not quietly become
React UI	Render state, start jobs, reconnect live progress	The authority for tenant membership or durable completion
Gateway	Route requests, enforce coarse limits, handle the browser/session boundary	The only authorization check in the system
Auth/Token and SCIM	Authenticate, issue/validate identity contracts, provision authorized tenant users	A justification for trusting arbitrary tenant headers
Connector service	Authorize a sync, validate its contract, record acceptance, execute connector policies	A transaction spanning every external system
Certificate/secret service	Resolve authorized secret references and coordinate rotation	A store of plaintext secrets in job events or logs
PostgreSQL	Own operational jobs, keys, checkpoints, and outbox records	A shared free-for-all where every service writes every table
Kafka	Retain and distribute events by partition	Automatic exactly-once effects in an external API
Redis / Elasticsearch / MongoDB	Cache and fan-out / search projection / optional raw payload retention	Three interchangeable sources of truth
Kubernetes and delivery	Run and replace workloads from approved artifacts	A guarantee that a healthy process is producing correct business results

The initial teaching design has one write region and a shared-schema tenant model inside each service's owned data. `tenant_id` participates in authorization, lookup predicates, and relevant uniqueness constraints. A job ID is not proof that the caller may access it. Defense-in-depth database controls are covered in 06 and 08.

A connector's configuration stores a secret reference, not the secret itself. The worker obtains the secret through an authorized path. Raw source payloads are optional, retained only for a stated purpose and duration. Durable progress belongs to a job; the live connection is merely one view of it.

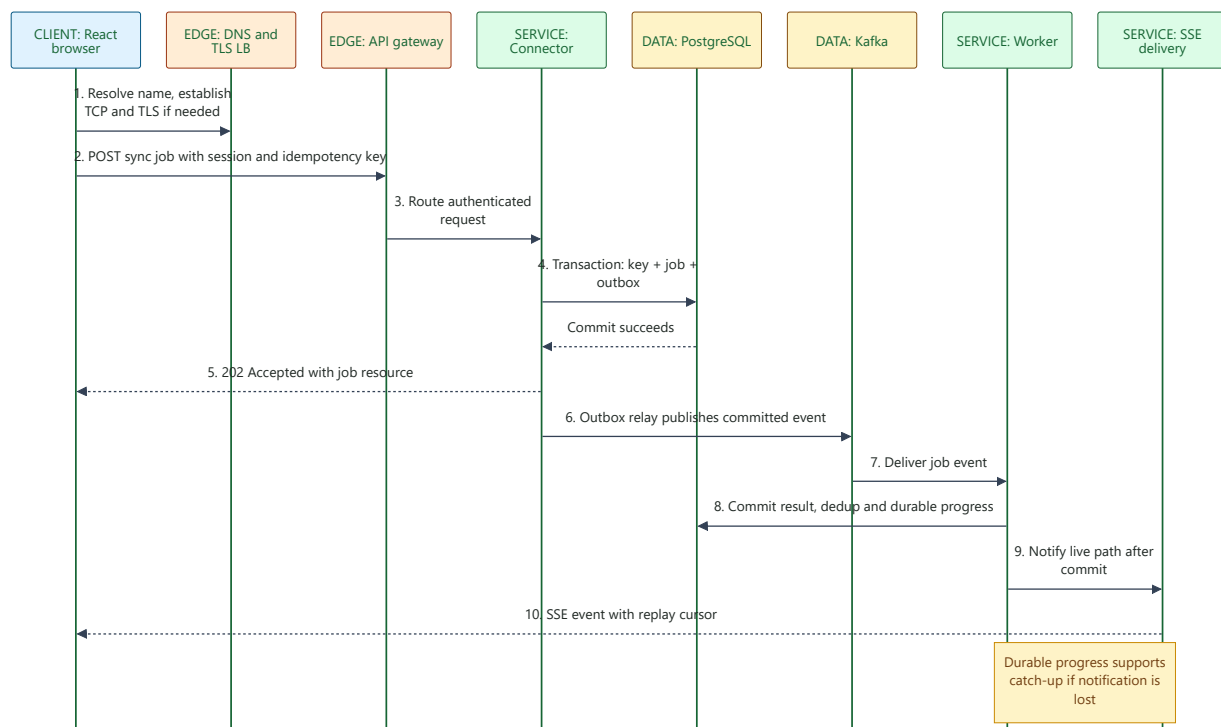
[D] DECISION

This guide includes microservices to expose network and ownership boundaries. A real small team should consider a modular monolith first. The [architecture chapter](#) compares use and avoid conditions; adding Kafka, Redis, and Kubernetes does not by itself improve a design.

Request Journey: From Click to Durable Progress

Scenario: an authenticated user starts a sync. This is an illustrative successful path with explicit failure boundaries, not a benchmark or a production trace. The public connection in this trace uses HTTPS over TCP; HTTP/3 uses QUIC instead and is covered in [21](#). Existing connections and DNS caches may skip setup work.

Diagram 00.3



1. Resolve and Connect

The browser needs an address for the requested hostname. [DNS resolution and caching](#) may involve browser, OS, and recursive resolver caches; a request does not necessarily reach an authoritative server. If there is no reusable connection, the client establishes [TCP](#) and then negotiates [TLS](#). Certificate verification checks the intended server identity through the browser's trust configuration; encryption without authentication would not establish whom the client reached.

The [load balancer](#) selects a healthy upstream. Public TLS termination does not automatically encrypt the next hop. The [network chapter](#) explains L4 versus L7 behavior, connection reuse, and separate connect/read timeouts. The [cloud chapter](#) maps those roles onto a VPC and ALB.

2. Carry Identity, Not an Untrusted Tenant Claim

The earlier login used [OAuth authorization code with PKCE and OIDC](#). For this walkthrough the browser uses a same-origin secure `HttpOnly` session cookie at the gateway/BFF boundary; the gateway can forward an access token internally. [CSRF protections](#) apply to state-changing cookie-authenticated requests. [OIDC](#) establishes identity; the service still authorizes an action on a specific tenant resource.

The [React client](#) sends `POST /v1/sync-jobs` with a stable [idempotency key](#) for this logical attempt. Double-clicking or retrying after a lost response must not create an unrelated key. The server scopes the key by tenant and operation, records a digest of the request, and rejects reuse for different content.

3. Route, Validate, Authorize

The [gateway](#) routes the request and applies a coarse [rate limit](#). At the connector service, [Spring Security, MVC, validation and proxies](#) lead to business logic that checks permission for the connector and tenant. Token signature validation is only one check; issuer, audience, lifetime, and the actual authorization policy also matter.

The request runs on a [Java execution model](#) and consumes finite resources: threads or virtual threads, sockets, and [database connections](#). Virtual threads do not make a ten-connection database pool capable of ten thousand simultaneous queries. The [JVM](#) and [OS](#) chapters identify what is actually scheduled and allocated.

4. Make Acceptance Durable

Inside one [database transaction](#), the connector records the idempotency result, job, and [outbox event](#). A tenant-scoped unique constraint arbitrates concurrent duplicate requests. Application code alone cannot safely decide uniqueness with an unprotected check-then-insert.

[JPA flushing](#) is not synonymous with a committed transaction. The server must not claim durable acceptance merely because an object was added to a persistence context. Once commit succeeds, a later process crash must not erase the accepted job within the stated database durability and recovery assumptions.

[M] MECHANISM

The outbox closes one specific gap: persisting a job and recording the intent to publish happen atomically in one database. It does not atomically commit a PostgreSQL transaction and a Kafka transaction. The relay may publish an event again after a crash; consumer idempotency remains necessary.

5. Return Acceptance, Not Completion

The [HTTP response](#) is `202 Accepted` with a job resource the caller can query. The sync may still fail later. If the server commits and the response is lost, the client's timeout means **unknown outcome**, not **the server did nothing**. A retry with the same key recovers the prior logical result under the API's retention contract.

The stored acceptance result must not be held forever without a policy. Retention must cover the documented client retry window and business risk; an expired key may no longer deduplicate an old request. Payments require stronger reconciliation reasoning, explored separately in [27](#).

6. Publish Committed Work

The outbox relay reads committed work and publishes to [Kafka](#). It waits for the configured broker acknowledgement before recording progress, but a failure between publish and that record can still create a duplicate. [Log-based CDC](#) is another relay implementation, not permission to discard idempotency.

Partitioning by a stable job key can preserve order for that job within a partition. It does not establish global order across jobs. Kafka offsets are positions in partitions, not a universal business progress counter. [Distributed ordering and consistency](#) give the vocabulary to explain that boundary precisely.

7. Execute with Bounded Failure

A worker consumes the event, resolves its [connector strategy](#), obtains authorized [credentials](#), and calls a source or destination application. [Integration techniques](#) handle pagination, mapping, checkpoints, quarantine, and bulk APIs. [Resilience policies](#) bound timeouts and concurrency, use backoff with jitter, and avoid retrying every failure indiscriminately.

An external timeout may hide a successful side effect. The worker uses the provider's idempotency contract or a reconciliation path; blindly issuing another non-idempotent operation is not safe. A circuit breaker stops selected calls temporarily, but it does not undo the calls already made or guarantee another system's recovery.

8. Commit Effects Before Advancing Consumption

For database-local effects, the worker stores its deduplication record, business update, and durable progress in one transaction. The [consumer offset](#) advances only after the intended durable processing boundary. A crash after the database commit and before offset commit produces redelivery; the deduplication contract makes that replay harmless for the covered effects.

[Elasticsearch projections](#) and [Redis caches](#) may lag the authority. A missing search result is not proof the job was never accepted. [Analytics feeds](#) are separate derived consumers; a warehouse delay should not block operational acceptance without an explicit business reason.

9. Deliver Live Progress without Making the Socket the Database

The live path reads committed progress and sends [SSE](#) events as `text/event-stream`. Proxies need appropriate buffering and timeout configuration. After disconnection, an event ID and `Last-Event-ID` can identify a replay cursor; they do not manufacture a replay store. A retention gap requires an explicit reset/snapshot path.

With multiple SSE nodes, [pub/sub fan-out](#) routes live events to interested connections. Redis Pub/Sub is not a durable event history. Kafka consumers in one group load-balance partitions; they do not deliver every event to every node. Slow clients need bounded buffers and a disconnect/catch-up policy rather than unlimited memory growth. [The UI](#) handles reconnects and ignores stale progress without inferring success from silence.

10. Make the Whole Journey Observable

[OpenTelemetry trace context](#) connects HTTP spans to asynchronous work through message metadata; asynchronous work may require appropriate span links. [Prometheus metrics](#), [Loki logs](#), and [Grafana views](#) answer different questions. Use logs/traces for per-job investigation;

unbounded job and tenant metric labels can cause excessive cardinality.

Jenkins and ArgoCD deliver the service image to [Kubernetes](#), but readiness only establishes the configured readiness condition. Test the business path with [integration and end-to-end checks](#). If memory or latency deteriorates, follow evidence into [JVM profiling](#) instead of changing GC flags at random.

The Boundaries an Interviewer Will Probe

Failure point	What the caller knows	Safe design direction	Deep explanation
Response lost after commit	Outcome is uncertain	Reuse the idempotency key and retrieve the accepted job	05 APIs
Relay publishes, then crashes	Publication may be repeated	Idempotent consumer with correctly scoped durable deduplication	07 Messaging
Worker commits, offset does not	Event may be delivered again	Keep covered effects and deduplication atomic	06 Databases , 07 Messaging
External API times out	External side effect may have happened	Provider idempotency or reconciliation; do not infer failure from timeout	27 Payments
SSE process is replaced	Connection is gone; job need not be lost	Reconnect, replay or retrieve a current snapshot	05 APIs , 15 Kubernetes
Cache/search lags	Derived view is stale	Read authority when correctness requires it; monitor projection lag	06 Databases

[!] TRAP

"Kafka gives exactly once, so our payment cannot duplicate" crosses several unrelated guarantees. Name the covered producers, broker transactions, consumer processing, database commit, and external effects. Then show the crash boundary where your assertion could fail.

A Repeatable Interview Answer

Use this structure for a mechanism question: **purpose -> boundary -> steps -> invariant -> failure -> trade-off -> evidence**. For a design problem, start with requirements and assumptions before using the structure. Do not assert a number unless it is measured or derived from named assumptions.

For example, when asked why the connector uses an outbox: first explain the need to durably accept a job even if the broker is temporarily unavailable. Identify the database transaction boundary. Trace the job and outbox inserts, commit, publish, and relay checkpoint. State the invariant that an accepted job has a recorded publication intent. Name the duplicate-publication crash window. Admit relay lag and operational complexity. Describe a test that crashes the relay after publish and verifies idempotent replay.

That answer is stronger than saying "outbox guarantees consistency" because it limits the guarantee and gives a way to falsify the design. It also leads naturally to [testing](#), [distributed guarantees](#), and [operational alerts](#).

Concept Index: Explained Here

Concept	Explanation
Connected learning and scope	How to read
Learning prerequisites	Dependency map
IntegrationHub and ownership boundaries	System responsibilities
Operational source of truth versus derived views	System responsibilities
End-to-end request journey	Ten-step trace
Acceptance versus completion; timeout uncertainty	Request journey, steps 4-5
Cross-system failure boundaries	Failure matrix
Interview answer structure	Purpose through evidence

The [coverage index](#) routes all registered terms to their owning chapters and, where available, their explanatory sections. Use the [chapter directory](#) for current publication status. A planned destination is a reservation, not completed reference material. The final global index is reviewed in Chapter 20 after all narrative chapters exist.

Related Chapters

The [chapter directory](#) links all 28 chapter IDs. Every related chapter destination includes a return link to this master map; the build verifies these reciprocal relationships. The [capstone](#) will return to this same system after the detailed chapters. It has not been written in this edition.

RELATED CHAPTERS / RECIPROCAL LINKS

[01](#) [Core Java and the JVM](#)

[02](#) [Concurrency and the Memory Model](#)

[21](#) [Networking and Operating Systems](#)

[03](#) [Spring and Spring Boot Internals](#)

04	Hibernate and JPA	27	Fintech and Payments Engineering
06	Databases and Durable State	13	Data Integration and iPaaS
05	APIs and Real-Time Communication	14	Docker and Container Internals
07	Messaging, Batch and Streams	15	Kubernetes and Production Workloads
26	Data Platform Basics	16	Build, Ship and Roll Back
08	Security, Identity and Data Protection	17	Cloud and Infrastructure
09	The Browser and the Java Backend	18	Observability and Reliability
10	System Design Foundations	23	JVM Performance Engineering
25	Architecture Styles and Domain Design	19	Testing and Quality
22	Distributed Systems Theory	24	IntegrationHub in Motion
11	Object Design, Patterns and LLD	20	Appendix: Interview Toolkit and Index
12	Twelve System Design Case Studies		

One-Page Cheat Sheet

The path: browser -> DNS/TCP/TLS -> load balancer/gateway -> service -> PostgreSQL job + outbox commit -> Kafka -> idempotent worker -> durable progress -> SSE -> browser.

The ownership: UI renders; gateway routes; identity establishes credentials; service authorizes and decides; database owns operational truth; broker delivers; derived stores project; telemetry explains; deployment replaces processes.

The invariants: authorize tenant access on every path; accept durably before claiming acceptance; scope idempotency keys and deduplication records; do not leak secrets; bound work and buffers; assume processes and connections can disappear.

The distinctions: flush is not commit; accepted is not completed; timeout is not proof of no effect; replay is not automatic correctness; encryption is not authorization; healthy Pod is not correct workflow; Kafka offset is not global order; live update is not durable storage.

The interview order: clarify requirements -> state assumptions -> draw ownership boundaries -> trace the successful path -> crash it at a boundary -> explain recovery -> defend a trade-off -> name a validating test or metric.

Find the mechanism: Java 01; concurrency 02; network/OS 21; Spring 03; JPA 04; SQL/stores 06; API/SSE 05; events 07; security 08; UI 09; scale 10; domains 25; distribution 22; LLD 11; HLD 12; payments 27; integration 13; delivery 14-17; operations 18; profiling 23; tests 19.

Interview Corner

Basic: What Does `202 Accepted` Promise?

It reports acceptance for processing, not completed work. In this API's stronger application contract, acceptance is returned after the job and publication intent commit. The caller follows a job resource for the eventual result. HTTP status alone does not implement durability.

Internals: Why Not Write the Database and Then Publish Directly?

A crash between those operations can leave committed work with no publication intent that a relay can find. Recording the intent in the same local transaction closes that gap. Publishing remains asynchronous and can duplicate; consumer correctness is a separate contract.

Trace/Debug: The Browser Timed Out, but a Job Appeared. Is That a Bug?

Not necessarily. The database may have committed before the response was lost. Correlate the tenant-scoped key and job with traces and durable records. Retry with the same key according to the API contract; a newly generated key may create another job. Also check whether the timeout budget is appropriate.

Scenario: A User Sees Another Tenant's Progress. What Do You Check?

Treat it as an authorization incident, not a rendering glitch. Check the session-to-tenant authorization decision, job lookup predicates, SSE subscription authorization, replay-store queries, and fan-out channel isolation. Contain the exposure, preserve safe audit evidence, and add cross-tenant negative tests. Do not log additional sensitive payloads while investigating.

Scenario: Should Every Service Use Every Technology in This Diagram?

No. Keep a component only for a defined need and an ownership/operational model the team can support. Redis must have a cache, quota, or fan-out purpose; search must justify its projection; Kafka must justify asynchronous retention and consumption; a modular monolith may be sufficient. The guide shows boundaries to teach them, not a minimum production shopping list.

Follow-Up: What Have You Actually Measured Here?

No throughput, latency, or availability measurement is asserted. This chapter is a conceptual trace. The publication build and validation results are recorded separately in the review log; later performance exercises must state hardware, workload, version, measurement method, and actual output.

01 / Core Java and the JVM

From source code to a running connector. This chapter explains the runtime and language contracts behind IntegrationHub's Java services. IntegrationHub is fictional. Its deployment, workload and incidents here are teaching scenarios, not measured production evidence.

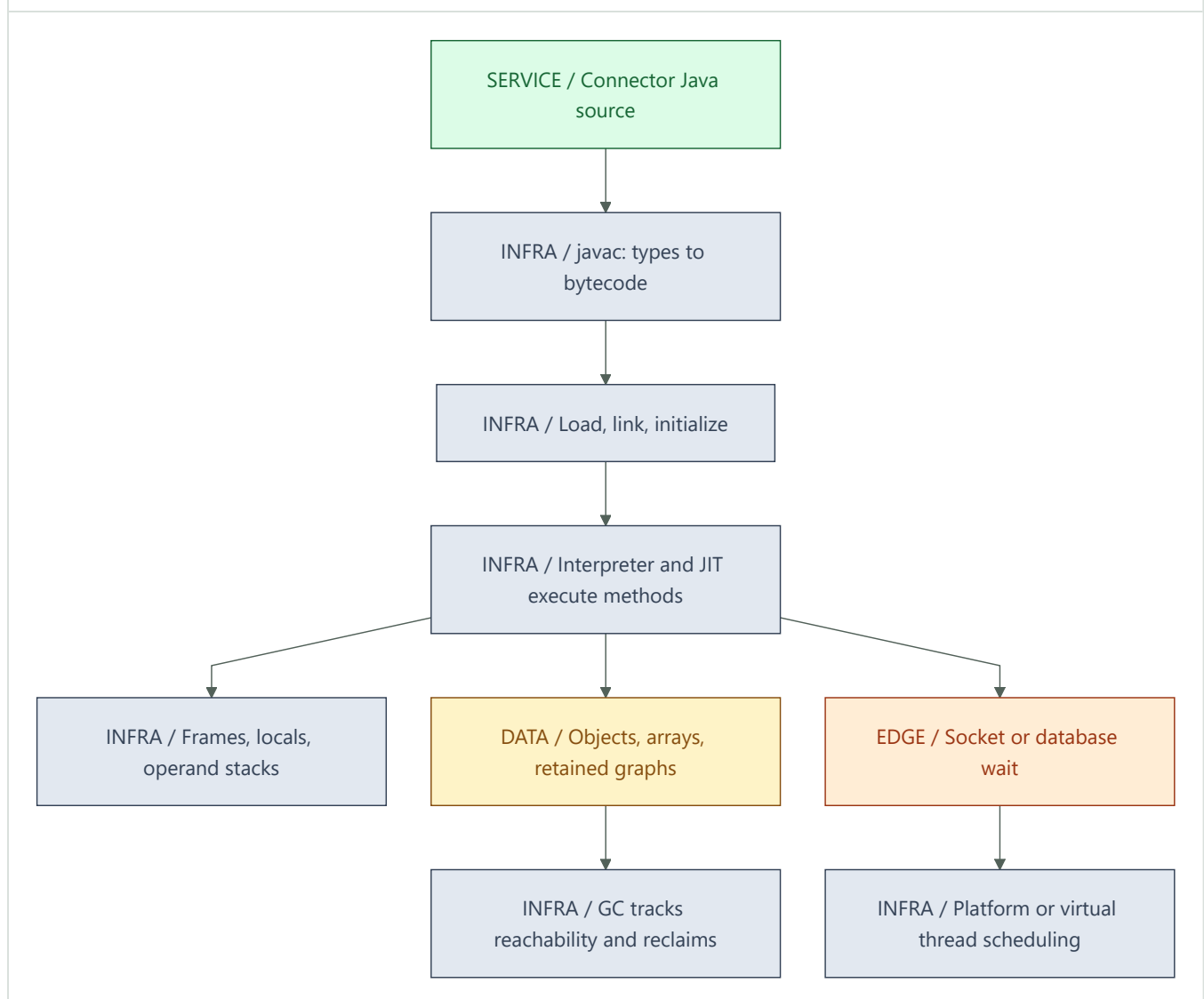
Version assumptions: examples target Java 21 without preview features, using an OpenJDK/HotSpot mental model where implementation details matter. Java 25 is the latest LTS in the Oracle roadmap consulted on 2026-10-05; LTS is a vendor support commitment, not a different language specification. The reference system retains Spring Boot 4.0.x / Spring Cloud 2025.1.x, Kubernetes 1.34, PostgreSQL 17 and Kafka 4.x as illustrative families from Chapter 00. None is required or executed for this chapter; it introduces no new Spring dependency pairing. The family-level Spring compatibility check is recorded in [00](#).

[VERIFY: before using these examples in a service, check the chosen JDK vendor's current patch/support policy and the exact Spring Boot system requirements and Spring Cloud compatibility matrix. Java 25 LTS status was checked against the Oracle roadmap; the runnable source deliberately remains Java 21.]

Execution status: NOT EXECUTED. No JDK compiler was found on PATH, in JAVA_HOME, or in the common installed-JDK locations checked. Two complete source labs and an offline runner are provided under the guide's `code/01-java-jvm/` directory. Compilation, runtime assertions, GC observations and throughput are not claimed. The runner recorded the missing-JDK condition. Mermaid diagrams are rendered as embedded SVG in the published guide; that successful rendering is separate from Java execution.

Big Picture

Diagram 01.1



The diagram separates compilation, runtime execution, memory lifetime and waiting. These interact, but solving one does not solve the others: a faster collector cannot repair an unbounded cache, and virtual threads cannot fix a CPU-heavy field-mapping algorithm.

What You Will Be Able to Explain

- Trace a connector from source compilation through class loading, initialization and method execution.
- Distinguish language/JVM guarantees from HotSpot choices, and heap pressure from total process-memory pressure.
- Explain HashMap lookup, equality, resize and collision handling without treating implementation thresholds as API guarantees.

- Use generic bounds, resource ownership, immutable value carriers and Streams without hiding unsafe casts or side effects.
- Explain records, sealed hierarchies and pattern matching as design tools rather than syntax trivia.
- Describe what virtual threads release while waiting, what they still consume, and what changed after Java 21.

[P] PRODUCTION

Where this lives in IntegrationHub. The [request journey](#) reaches a Java connector that validates tenant-scoped keys, loads connector implementations, transforms a batch and waits on external I/O. [Spring proxies](#) build on runtime types; [JPA](#) retains entity graphs; [concurrency](#) determines safe sharing; [container limits](#) constrain the entire process. Use [23](#) for actual profiling rather than guessing from these diagrams.

1. Java 8 to the Current LTS: Keep Three Versions Separate

A service has a **source/API target**, a **build JDK**, and a **runtime JDK**. A newer compiler can target an older supported release with `--release`, which constrains language and platform API use as well as class-file targeting. It cannot make an independently compiled third-party library compatible with that target. Maven or Gradle toolchains and CI must agree with the deployment image; their mechanics belong in [16](#).

Why it matters: a developer may compile a connector on a newer JDK while the image still runs an older one. The class loader can reject an unsupported class-file version before the request handler runs. Separately, a dependency compiled against a method absent at runtime can fail with a linkage error even if your own source looks compatible.

Release milestone	Useful capabilities to recognize	Use when	Avoid when
8	Lambdas, functional interfaces, Streams, default interface methods, <code>java.time</code>	Reading existing enterprise code and reasoning about its contracts	Assuming modern record or virtual-thread syntax is available
11	LTS migration context; standard HTTP client; local-variable type inference arrived in 10	Maintaining services pinned to a compatible supported distribution	Treating <code>var</code> as dynamic typing or overlooking module-era access restrictions
17	LTS baseline including records, finalized in 16, and sealed classes, finalized in 17	Modeling values and controlled hierarchies in supported services	Assuming Java 21 pattern-switch and virtual-thread APIs are present
21	Virtual threads, record patterns, pattern matching for switch, sequenced collections	This guide's no-preview example baseline	Assuming all preview features in 21 are stable APIs

Release milestone	Useful capabilities to recognize	Use when	Avoid when
25	Current LTS context; includes the earlier Java 24 monitor-pinning change	Planning an upgrade with library, agent and runtime validation	Treating a new LTS as a reason to skip regression or load testing

[VERIFY: release-specific feature claims should be checked against JEPs 286, 395, 409, 431, 440, 441 and 444 and the chosen release documentation. A feature being available does not mean its related preview APIs are stable; this chapter uses no preview APIs.]

Mechanism: compiler checks source -> emits bytecode and metadata -> runtime validates the class format -> resolves referenced types/members -> executes under that runtime's implementation. Changing the runtime can change GC/JIT behavior without changing the Java source contract. The big-picture diagram shows these separate stages.

Trap: `var` infers a static type from an initializer; it does not allow the variable to accept unrelated types later. A runtime upgrade also does not enable newer source syntax when the compiler target remains 21.

Interview check: "We compiled with Java 25. Can we deploy on 21?" Answer: only if the application and all runtime dependencies are compatible with 21, the application was compiled to that target, and the deployed behavior was tested. A successful build on 25 alone proves none of those together.

2. JVM Architecture: What Executes the Method?

The **JDK** provides development tools such as the compiler alongside runtime components. The **JVM** is the abstract execution machine specified by the JVM Specification, with implementations such as HotSpot. A runtime image supplies the VM and required libraries; modern packaging does not require a separately installed desktop JRE. A JAR is an archive of classes/resources, not a prestarted JVM.

In IntegrationHub, a connector's field-mapping method begins as bytecode. An interpreter can execute that bytecode, while HotSpot gathers execution profiles and may compile frequently executed code into machine code. Calls may be inlined; optimizations depend on assumptions about observed types and control flow. If an assumption stops holding, deoptimization can return execution to a less optimized form while preserving Java semantics.

The point is not "Java is interpreted" versus "Java is compiled." Both can participate during one process lifetime. A tiny loop timed at startup may measure loading, compilation, allocation or timer effects instead of the steady-state operation. Benchmark methodology belongs in 23; this chapter presents no timings.

Method Invocation Trace

- A caller evaluates arguments. Java passes their values, including reference values.
- Method resolution establishes the referenced member; virtual dispatch selects an implementation based on the actual receiver where applicable.
- The abstract execution model creates a frame with local-variable storage and an operand stack. These are distinct from the collection class named Stack.
- Bytecode computes values, invokes methods and may allocate objects. Physical implementation can optimize away some allocations/frames while preserving observable behavior.
- Normal return passes a result to the caller; exceptional completion searches for a matching handler and unwinds as needed.

The big-picture diagram's execution/stack/heap branches are this trace. Inlining does not mean source methods cease to exist semantically. The runtime retains information needed for deoptimization and diagnostics, although a sampled profile is not a complete history of every invocation.

[M] MECHANISM

Pass-by-value, including references. A method receives a copy of the caller's reference. It can mutate an object both references identify, but assigning its parameter to another object does not reassign the caller's variable. `CoreJavaLab.changeReference` checks both behaviors. Returning a mutable object can still leak ownership even though parameter passing is by value.

Use when / avoid when: use the JVM model to identify which lifecycle or memory boundary failed. Avoid deriving performance numbers or physical object layouts from the abstract frame model. A Java reference is not a contract exposing a fixed memory address.

Production case: a newly rolled-out Pod has different latency from a warmed process. Separate startup/class loading, dependency initialization, JIT warm-up and actual downstream waits. Do not disable the JIT or multiply replicas based on a single short startup sample.

Interview check: "Why does warm-up matter?" Answer: the runtime may load classes, collect profiles and compile/optimize code over time. A representative measurement needs a controlled workload and measurement phase, not simply an arbitrary sleep.

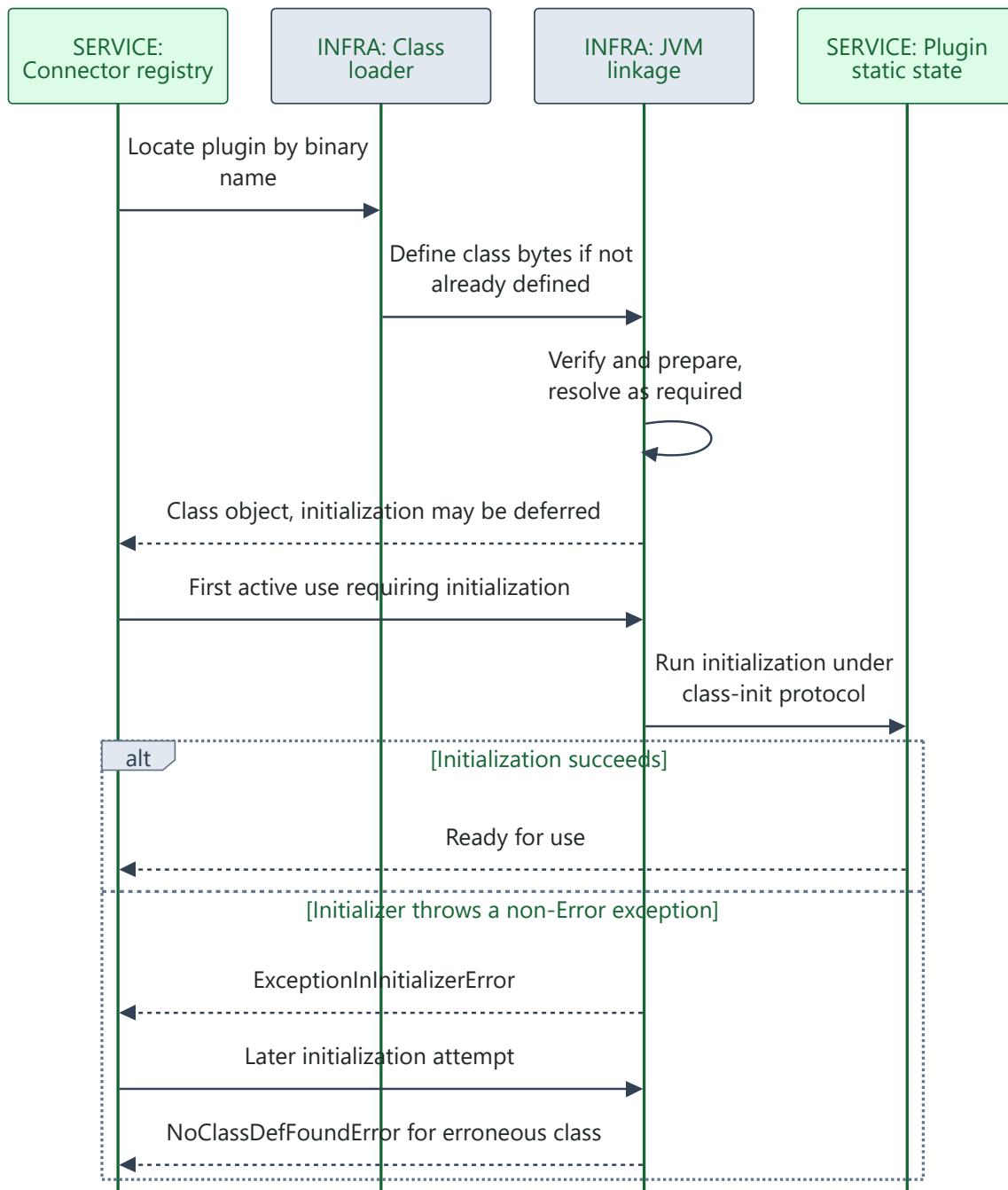
3. Loading, Linking and Initialization Are Different Events

Why they exist: Java can discover and load code at runtime while still checking format, types, access and linkage. The runtime must distinguish merely knowing a class from executing that class's initialization side effects.

Loading obtains a binary representation and creates a runtime class. **Linking** includes verification, preparation and resolution. Verification checks constraints on the class and bytecode; preparation creates static fields with default values. Resolution translates symbolic references into runtime entities and may happen lazily. **Initialization** runs the class initialization procedure, including its static initialization logic. Constant-variable rules are special; do not reduce the complete specification to "every static read runs a static block."

Runtime class identity includes the **binary name and defining class loader**. Two plugin loaders can define different types with the same printed name. A `ClassCastException` between apparently identical names is therefore possible. The default `ClassLoader` delegation model generally checks parents before defining a class itself; custom frameworks may use different delegation policies. Do not claim that every loader in every environment follows an unconditional parent-first law.

Diagram 01.2



The JVM coordinates initialization across threads. A thread can wait for another thread to finish initialization; slow or cyclic initialization dependencies can obstruct startup. Initializing a class also initializes its superclass and specified superinterfaces as required by the specification, not every interface it happens to reference.

Calling `Class.forName` with initialization disabled demonstrates loading without requesting initialization. Creating an instance or invoking an appropriate static method requires initialization. Reading an inlined constant may not. Do not put network access in static

initializers: failure can poison the class's initialization state for that defining loader, and retrying the business method will not reset it.

A Complete Initialization Probe

NOT EXECUTED. The following complete Java 21 program matches `code/01-java-jvm/LoadingLab.java`. The guide checks source synchronization; that is not Java compilation. Run it in a fresh JVM. The success text is source code, not claimed terminal output.

java

```
import java.util.ArrayList;
import java.util.List;

public final class LoadingLab {
    private static final List<String> events = new ArrayList<>();

    static final class Plugin {
        static {
            events.add("initialized");
        }
    }

    static final class BrokenPlugin {
        static final int CONFIG = readConfig();

        private static int readConfig() {
            throw new IllegalStateException("missing teaching configuration");
        }
    }

    public static void main(String[] args) throws Exception {
        ClassLoader loader = LoadingLab.class.getClassLoader();
        Class<?> loaded = Class.forName("LoadingLab$Plugin", false, loader);
        if (!events.isEmpty()) {
            throw new AssertionError("Loading alone must not initialize Plugin");
        }
        Class<?> initialized = Class.forName("LoadingLab$Plugin", true, loader);
        if (loaded != initialized || !events.equals(List.of("initialized"))) {
            throw new AssertionError("One class identity and one initialization");
        }
        Class.forName("LoadingLab$Plugin", true, loader);
        if (events.size() != 1) {
            throw new AssertionError("Initialization must not repeat");
        }
        try {
            Class.forName("LoadingLab$BrokenPlugin", true, loader);
            throw new AssertionError("First initialization must fail");
        } catch (ExceptionInInitializerError failure) {
            if (!(failure.getCause() instanceof IllegalStateException)) {
                throw new AssertionError("Original failure cause is preserved", failure);
            }
        }
        try {
            Class.forName("LoadingLab$BrokenPlugin", true, loader);
            throw new AssertionError("Erroneous class must not initialize again");
        } catch (NoClassDefFoundError expected) {
            System.out.println("LoadingLab: all checks passed");
        }
    }
}
```

Read the Failure Precisely

Symptom	Mechanism to investigate	First evidence
ClassNotFoundException	Explicit dynamic lookup cannot locate a requested class	Requested name, actual runtime classpath/module configuration, loader
NoClassDefFoundError	Runtime needs a definition that cannot be made usable; can also mean earlier initialization failed	Earliest causal exception, not only the last retry's stack
NoSuchMethodError	Caller and runtime dependency disagree on a member signature	Dependency convergence, deployed artifact, runtime library versions
UnsupportedClassVersionError	Runtime does not support the class-file version	Build target and every deployed dependency's bytecode target
Same-name ClassCastException	Distinct defining loaders or genuinely different types	Both Class objects and defining loaders

Use when / avoid when: dynamic loading is useful for real plugin isolation or discovery; avoid custom class loaders just to implement a Strategy pattern. IntegrationHub can start with dependency-injected connectors and a factory. [11](#) explains that design; [03](#) explains proxy/lifecycle effects.

Interview check: "The class exists in the JAR. Why NoClassDefFoundError?" Answer: existence is insufficient. Inspect the loader/dependency path and the first initialization failure. An erroneous class can fail later uses even though its bytes are present.

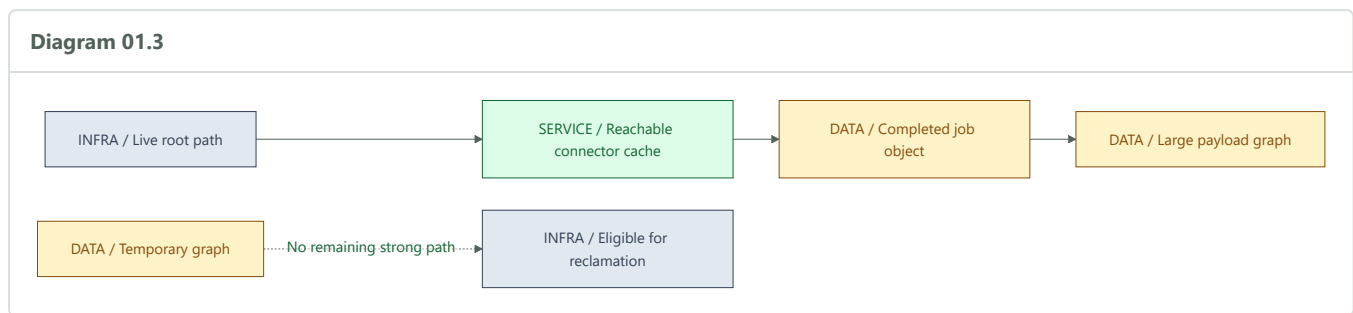
4. Memory and GC: Follow the Reference, Not Just the Allocation

Why automatic memory management exists: code should not manually free every object or access freed storage. The runtime finds objects that are no longer reachable under its reference-processing rules and can reclaim their memory. This does not define when a socket is closed or how much work the service should admit.

The JVM specification defines shared heap and method-area concepts plus per-thread execution areas. HotSpot implements class metadata largely in native-memory Metaspace; that is not a second Java object heap. Class mirror objects and ordinary referenced application data still occupy heap. Platform-thread stacks, JIT code cache, direct buffers and other native allocations also consume process memory.

Heap limit is not process limit. Reserving address space, committing memory and resident memory are different observations. Setting the maximum heap close to a Kubernetes container limit leaves insufficient space for nonheap/native usage and can result in process termination.

The application need not receive a catchable `OutOfMemoryError` first. 15 owns `OOMKilled` diagnosis; 23 owns memory measurements.



This is a reachability sketch, not a promise of immediate reclamation. A completed job can remain strongly reachable through a cache, listener, queue, static registry or thread-local value. The collector is correct to retain it even when the business no longer needs it: that mismatch is a common Java memory leak.

Allocation-to-Reclamation Trace

- The request constructs a payload/DTO graph; the runtime allocates storage unless optimization can eliminate the physical allocation while preserving semantics.
- Locals, fields and container entries keep references. Losing one local reference does not remove all other paths.
- A tracing collector discovers live objects from its root model and handles mutations during concurrent phases with implementation-specific barriers/protocols.
- Unreachable objects become reclaimable. Depending on the collector, survivors may be copied or relocated and references updated, or space reclaimed differently.
- New allocations use reclaimed capacity. An object being eligible does not promise a particular collection time or a returned OS page.

Generational hypothesis: many objects die young, so generational collectors can focus effort on young allocations while retaining older survivors. References from older to younger objects require tracking; otherwise a young-only collection could incorrectly overlook a live path. Region-based designs such as G1 are not a literal permanently contiguous "three boxes" drawing of Eden, survivor and old space.

Pause versus concurrency: concurrent work overlaps application execution; it does not imply the collector has no pauses or free CPU cost. Throughput, latency and memory overhead are different objectives. Start with a supported default, obtain GC/allocation evidence, then consider a change. No universal best heap size or pause flag follows from this chapter.

Choice or technique	Use when	Avoid when
Strong references and explicit bounded caches	Data must remain available while owned; size/expiry/eviction policy is explicit	Retaining every completed sync or its full raw payload indefinitely
Weak references	Association must not by itself keep a referent alive	Implementing an authoritative business store or predictable cache policy
G1 starting point	A balanced, supported baseline fits the deployment and measured goals	Treating defaults as proof of sufficient latency under any workload
Evaluate ZGC	Measured pause objectives justify evaluating its resource trade-offs	Assuming lower pause time means faster business operations or zero overhead
Try-with-resources	A resource has deterministic close ownership	Waiting for GC to release scarce sockets or database connections

[VERIFY: collector availability/defaults, heap ergonomics, object layout and container-awareness behavior vary by JDK vendor, release and platform. Check the selected runtime's GC tuning guide and actual configuration; do not reuse Java 21 tuning flags blindly on Java 25.]

Traps: setting a local variable to null is rarely a principled fix for an owning collection retaining the same object. Soft references are not a predictable cache-size policy. Finalization is not a resource-lifetime strategy; use explicit ownership and close. String interning can share canonical string instances, but is not a blanket optimization for unbounded tenant data.

Production trace: heap occupancy after comparable collections trends upward while completed-job cache cardinality grows. Capture the relevant telemetry and a permitted heap analysis, find retaining paths/dominators, inspect cache eviction, and bound retained history. High allocation rate with stable retained heap is a different problem. Do not infer a leak from one high-utilization sample or forcibly trigger GC as the only test.

Interview check: "Can GC collect a cycle?" Yes, a cycle with no retaining path under the collector's reachability rules can be reclaimed; tracing GC is not simple reference counting. "Why is the Pod out of memory when heap looks fine?" Check native/direct/thread/code/metadata usage and the cgroup limit as well as heap, with appropriate measurements.

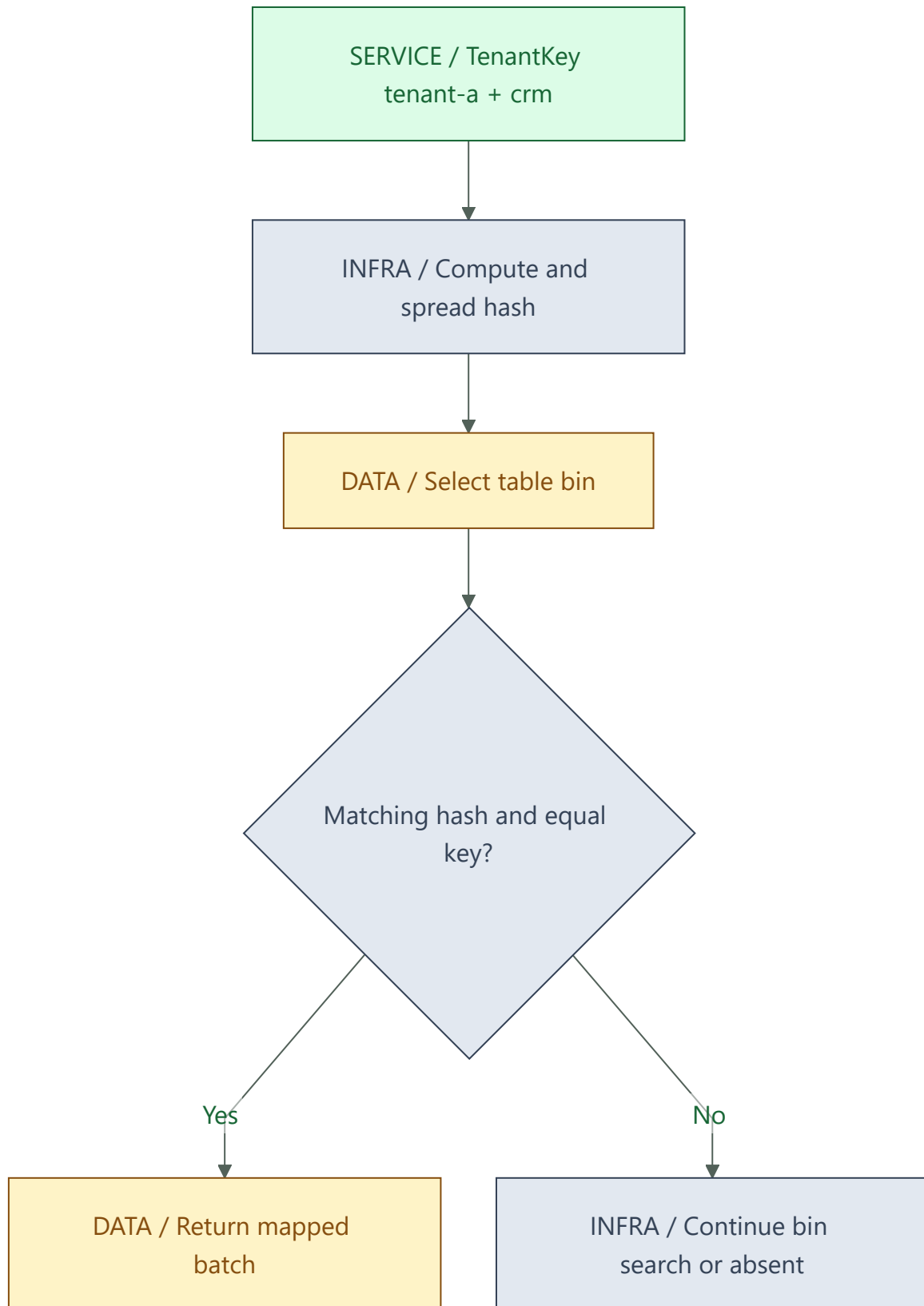
5. Collections Internals: Equality Is Part of Your Data Model

IntegrationHub needs a batch's ordered record IDs, a connector lookup table and perhaps a bounded work queue. Choose their contracts first: order, uniqueness, identity, lookup and mutation patterns, ownership and concurrency. A collection class name alone does not establish safe sharing.

HashMap: Trace a Lookup

The key's hash helps select a bin; equality distinguishes keys within that search. Equal keys must have equal hash codes; unequal keys can collide. A null return may mean no mapping or a mapped null value for a HashMap, so use the appropriate presence contract rather than assuming all maps disallow null.

Diagram 01.4



In OpenJDK's implementation the table length is a power of two; a spread hash selects a bin via a mask. Nodes store a hash, key, value and linkage. An insertion either replaces a matching key's value or adds an entry. As size exceeds a capacity/load threshold, resizing allocates a larger table and redistributes entries. This costs work and temporary capacity; repeatedly growing large per-request maps can increase allocation pressure.

Tree bins mitigate heavy collisions under qualifying conditions. The common interview shorthand "at eight entries it always becomes a tree" is not an API guarantee and omits capacity and insertion-path details. Do not promise an unconditional logarithmic worst case for arbitrary malicious/non-comparable keys merely because tree bins exist. Sensible keys and bounded input still matter.

[VERIFY: spread hashing, power-of-two sizing, tree-bin thresholds, resize behavior and tree-bin lookup details are OpenJDK implementation facts, not Map guarantees. Check HashMap.java in the exact JDK source tag when an interview asks for numerical thresholds or implementation complexity.]

Mutable-Key Failure, Step by Step

A key enters a map with one equality/hash-relevant state. It is then mutated. A subsequent lookup computes the new state-dependent hash and may search a different bin or fail a stored-hash match. The entry can still be present during iteration but no longer be found by ordinary lookup. This is why merely synchronizing a mutable key does not make it a good key: the identity contract itself changed.

CoreJavaLab uses a validated immutable TenantKey record with tenant and connector ID. A new equal key finds the entry; a different tenant does not. Its CollisionKey intentionally gives two unequal values the same hash to prove collision is not equality, not to recommend constant production hashes.

For entity keys, consider lifecycle carefully: a generated database ID appearing after insertion can change equals/hashCode behavior. 04 owns entity identity and persistence-specific choices. Record auto-generated equality is convenient only when its components describe the identity you intend.

Other Structures and Their Costs

Structure	Mechanism	Use when	Avoid when
ArrayList	Resizable reference array; constant-time indexed access, amortized append, shifts for middle changes	Ordered batches with scans/appends/indexing	Assuming every insertion is cheap or resizing has no allocation cost

Structure	Mechanism	Use when	Avoid when
LinkedList	Linked nodes; traversal to an index; local relink after locating a node	A measured need fits its deque/list contract	Choosing it for "fast insertion" while repeatedly searching by index
ArrayDeque	Resizable circular array for ends	In-memory stack or FIFO under single-owner/external coordination	Needing null elements, bounded blocking admission, or concurrent safety without coordination
HashMap / HashSet	Hash-bin lookup; HashSet delegates membership storage to a map	Lookup/uniqueness without sorted order	Depending on iteration order or mutating key identity
LinkedHashMap	Hash lookup plus linked encounter order; optional access order	Stable iteration or a carefully implemented local eviction policy	Assuming access-order reads are structurally read-only or thread-safe
TreeMap / TreeSet	Ordered tree governed by comparison	Sorted/range operations	A comparator inconsistent with intended key identity
ConcurrentHashMap	Concurrent access with atomic per-key operations	Shared lookup/update with a suitable per-key contract	Assuming several separate calls or keys form one transaction

HashMap has no promised iteration order. A fail-fast iterator's `ConcurrentModificationException` is best-effort bug detection, not a concurrency protocol. Wrapping a collection as unmodifiable prevents mutations through that view; it does not necessarily take a snapshot or deeply freeze elements. `List.copyOf` gives an unmodifiable shallow snapshot and rejects null elements. Mutable element state still needs ownership discipline.

Production case: a tenant ID is omitted from a cache key, so different tenants with the same connector ID share an entry. This is a data-model/security failure, not a collision bug. Equal strings are supposed to compare equal; include the complete authorized scope in the key and validate cross-tenant negative cases in [19](#).

Interview check: "Why not ConcurrentHashMap everywhere?" It has a concurrency contract and costs, not automatic business atomicity. A thread-confined map needs no concurrent coordination; a check-then-act across multiple shared entries still needs a larger design. Detailed concurrent collections belong in [02](#).

6. Generics: Compile-Time Constraints, Not Magic Runtime Containers

Why they exist: a connector pipeline should not require unchecked casts at every transform. Generics express relationships between input and output types and reject many invalid programs before execution. They also improve API contracts for callers.

Generic types are normally invariant: a list of Integer is not a list of Number, because a writable Number list could accept a Double. Wildcards describe constrained views. A producer with an upper bound can yield values usable as that bound; a consumer with a lower bound can accept values of that lower-bound type. This is **producer extends, consumer super**, relative to how this operation uses the parameter, not a label permanently attached to a collection.

Diagram 01.5



CoreJavaLab.copyNumbers declares a descriptive type parameter bounded by Number, an `extends` source and a `super` destination. In its call, Integer elements flow to a Number list without an unchecked cast. From an upper-bounded wildcard list, adding an arbitrary Number is unsafe because the actual list might be Integer-only. From a lower-bounded view, reading gives only the safely known Object type unless further information is available. A wildcard is not general permission to mix types.

Erasure Trace

- The compiler checks type arguments and constraints at call sites.
- It emits erased method/type representations, retaining generic signature metadata where specified; "erased" does not mean every trace of generics disappears from class files.
- It inserts casts where needed and can generate bridge methods to preserve overriding behavior after erasure.
- Runtime objects do not generally distinguish a list instantiated with String from one instantiated with Integer by reified element type. Arrays, in contrast, have runtime component-type checks.
- Raw types or unchecked boundaries can pollute a collection; the failure may surface later at a compiler-inserted cast, far from the unsafe insertion.

Use when / avoid when: use a generic API when it preserves a meaningful relationship across values, such as a typed mapper. Use a named domain type when it clarifies a concrete business contract. Avoid elaborate wildcard signatures that make a simple API harder to consume, and avoid suppressing warnings without isolating and justifying the cast.

Trap: `List<?>` is not the same contract as `List<Object>`. The former can represent a list of an unknown element type; the latter promises callers can add any Object. Likewise, primitive type arguments are not available in ordinary Java 21 generics; boxing adds objects/indirection and null-unboxing risks. Specialized primitive Streams can avoid boxing for suitable numeric operations.

Production case: raw payload mapping puts a numeric identifier into a collection assumed to contain strings. Serialization fails later or silently coerces the wrong shape. Keep untrusted JSON/schema validation at the [integration boundary](#), not hidden behind a generic cast. Generics validate Java typing, not a remote system's data.

Interview check: "Does type erasure mean no runtime casts?" No. The compiler often inserts casts at typed retrieval sites; erasure is one reason heap pollution can be detected late.

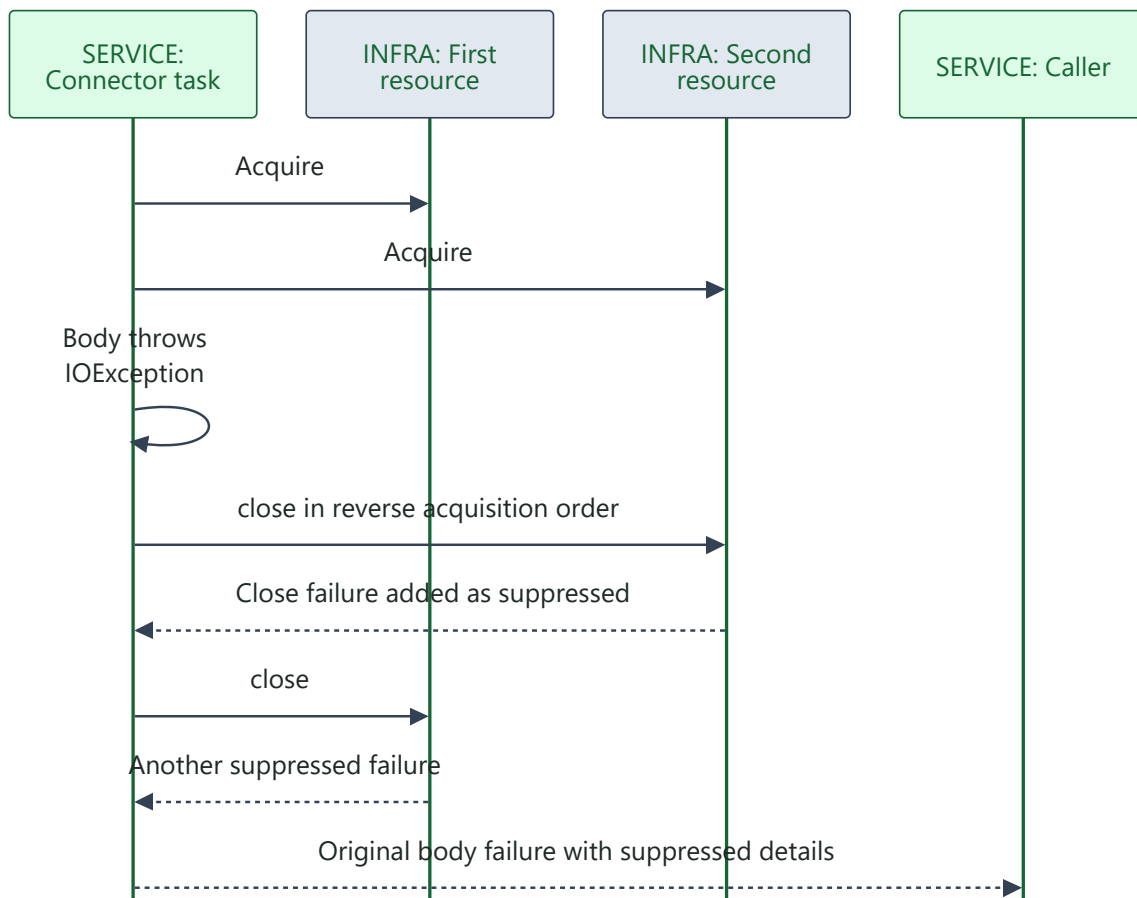
7. Exceptions and Resource Ownership

Why exceptions exist: a method must signal failure without pretending it produced a valid value. The caller needs both the failure category and a preserved causal path. Java distinguishes checked exceptions, unchecked `RuntimeExceptions` and `Errors`; checking is a compiler rule about handling/declaring, not a reliable taxonomy of retryable versus permanent failures.

Checked exceptions can make a recoverable condition explicit at an API boundary. Unchecked exceptions can express invalid arguments or a domain/framework failure contract without declarations everywhere. An `Error` generally signals a condition ordinary business code should not casually catch and continue from. Do not turn every `Throwable` into "try again."

Body Failure Plus Close Failure

Diagram 01.6



Try-with-resources closes successfully initialized resources in reverse order. If the body already failed, subsequent close failures are suppressed on the primary exception. If acquisition of a later resource fails, previously acquired resources still close. If no body failure exists, a close failure can become the primary failure. Preserve this information when translating exceptions.

CoreJavaLab.resourceFailures constructs two `AutoCloseable` resources whose close methods deliberately throw. It checks the body exception, close order and suppressed exceptions. These are simulated resources, not real file descriptors; the test isolates language behavior.

Policy	Use when	Avoid when
Propagate a checked exception	The caller can meaningfully choose recovery and the contract should require attention	Declaring generic Exception across every layer without semantic value
Translate with the original cause	A lower-level failure must cross a domain/API boundary	Replacing the stack/cause with only the message text
Try-with-resources	This scope owns an <code>AutoCloseable</code> lifetime	Closing a shared resource that this scope merely borrowed

Policy	Use when	Avoid when
Return an explicit domain outcome	Rejection is an expected business alternative	Encoding transport failure as an apparently successful empty result
Catch at a task/request boundary	You can map/report failure and complete that unit coherently	Logging and continuing with corrupted or partially initialized state

Interrupts: blocking APIs can report interruption through `InterruptedException` and clear the interrupt status. Propagate the exception when your contract allows it. When translating it and not rethrowing it directly, usually restore the status so higher-level cancellation can observe it. Whether to stop, retry or translate is a policy decision; swallowing interruption in an endless loop defeats shutdown.

Traps: a return or throw in finally can mask an earlier result or exception. Logging the same failure at every layer creates noise without extra evidence. Catching `Exception` does not catch `Error`; catching `Throwable` and ignoring it is rarely a recovery strategy. Retryability depends on the operation and external-effect uncertainty, not merely the exception superclass.

Production case: a connector catches an I/O failure and returns an empty batch, advancing a checkpoint as if there were no source records. Silent data loss follows. Keep "source returned no records" separate from "source could not be read" and connect error policy to [batch restartability](#) and [API error contracts](#).

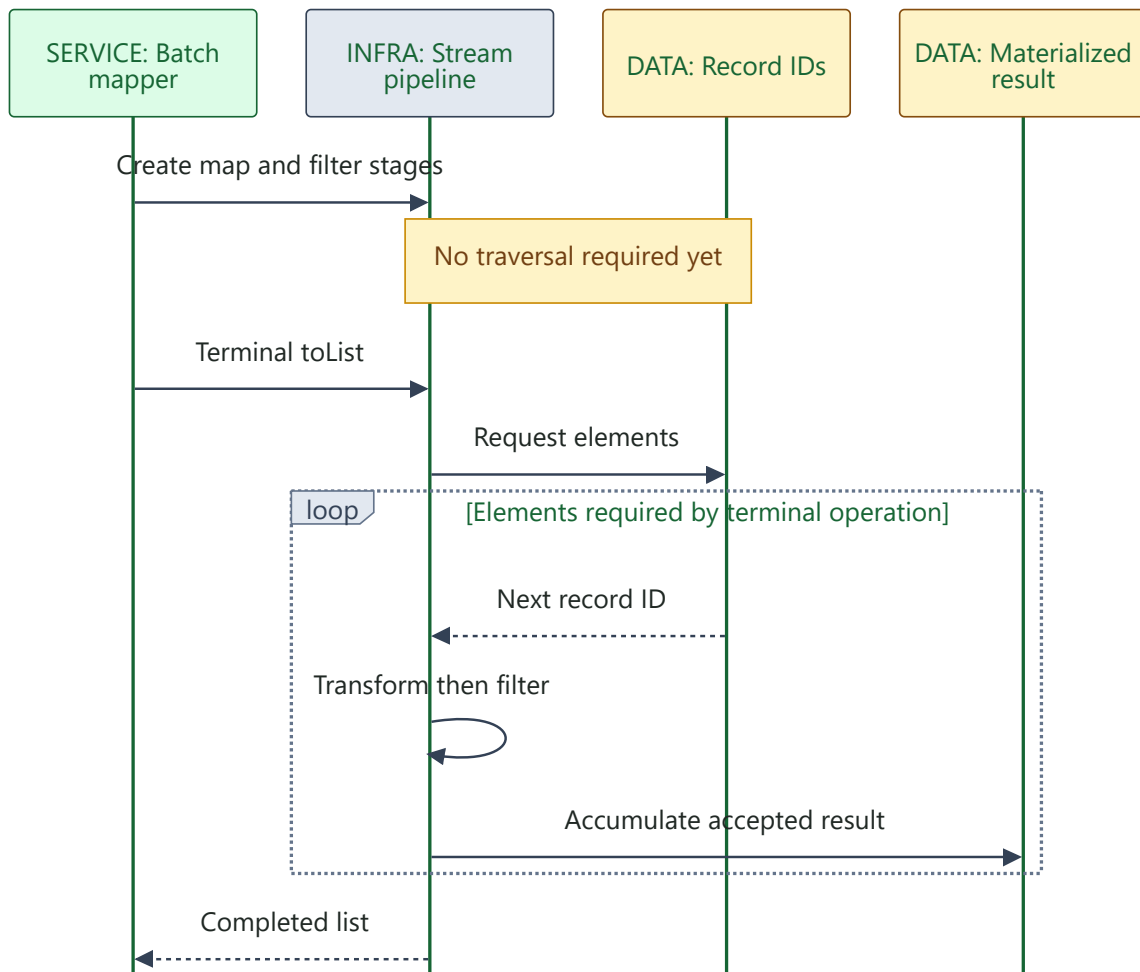
Interview check: "Why is a close failure missing from the main message?" It may be suppressed on a primary failure. Inspect `getSuppressed()` and preserve causes; do not infer that close succeeded from the top-level message alone.

8. Functional Java and Streams: A Description of Work Is Not Work Done

Functional interfaces describe a single abstract operation, such as transforming, testing or consuming a value. A lambda supplies its implementation; a method reference is another way to express a compatible operation. Captured local variables must be final or effectively final, but the objects those references identify may still be mutable. A captured `ArrayList` is not made thread-safe by appearing in a lambda.

Streams describe a pipeline over a source. They do not store another copy of the collection by definition. Intermediate operations typically build a lazy pipeline; a terminal operation drives traversal. Stateless stages such as `map/filter` can be fused, while stateful operations such as `sorted` or `distinct` may need buffering/bookkeeping. Short-circuiting operations may consume only part of a source.

Diagram 01.7



An important nuance: the implementation can eliminate work when it can derive the result without executing a stage, within the Stream contract. A count on a known-size source can make a debugging peek misleading. Side effects in intermediate stages are therefore not a reliable business protocol. Use explicit operations and contracts for required effects, not "this lambda ought to run."

CoreJavaLab.genericsAndStreams uses a deliberately sequential diagnostic map with a tiny side-effect list to observe traversal with toList. The chapter does not generalize that trace to optimized count or parallel execution. The same method checks single-use behavior, an unmodifiable result and duplicate-key failure in toMap.

A Pipeline Walkthrough

For three synthetic record IDs, strip surrounding spaces, discard empty IDs, normalize the remainder with an explicit locale, then materialize. The domain policy determines whether normalization is valid: external IDs are often case-sensitive, so IntegrationHub must not

uppercase real identifiers just because the teaching probe does. The test's uppercase conversion is a visible transformation, not a connector rule.

For grouping, use a collector that owns its accumulation/combination instead of mutating an external shared map from a parallel lambda. Reductions require an appropriate identity and associative combination when partitioning work; subtraction is not a valid interchangeable associative combiner. A merge function for duplicate keys is a business decision, not an arbitrary way to silence an exception.

Technique	Use when	Avoid when
Plain loop	Control flow, early exit, checked failures or required effects are clearer	Repeating a declarative transform more obscurely out of habit
Sequential Stream	A bounded transformation/filter/group pipeline is readable and testable	Hiding retries, transactions or remote side effects inside map/peek
Parallel Stream	Sufficient CPU-bound, splittable work with valid reduction and measured benefit	Blocking connector HTTP calls, thread-local transaction assumptions, or shared mutable accumulation
Stream.toList	An unmodifiable result matches the API	Assuming you can append, or assuming deep element immutability
Collect to a specified collection	The caller needs a chosen mutable collection contract	Relying on an unspecified collector implementation type
Optional return	Absence is a legitimate result	Replacing error information with empty, or blindly calling get

[VERIFY: parallel Stream scheduling/pool interactions, optimization behavior and collection-return contracts must be checked against the chosen JDK's Stream API and implementation. Do not treat the common-pool execution pattern as a universal caller-controlled executor contract.]

Resource trap: collection-backed Streams generally do not own external resources, but an I/O-backed Stream can. Close an owned `Files.lines` stream deterministically. A terminal operation does not universally mean the underlying I/O resource has been closed. Materializing all rows can also defeat streaming's memory benefit; bound batches and checkpoints in [13](#).

Production case: `parallelStream` invokes blocking provider calls, consumes shared execution capacity, loses assumed thread-local context and overloads a rate-limited provider. Choose an explicit concurrency/admission policy rather than a one-word parallelization change. [02](#) and [10](#) cover the execution and resilience boundaries.

Interview check: "Why did peek not log every record?" The pipeline may not have been consumed, may short-circuit, or may be optimized so the stage is unnecessary. Logging via peek is not proof of complete business processing.

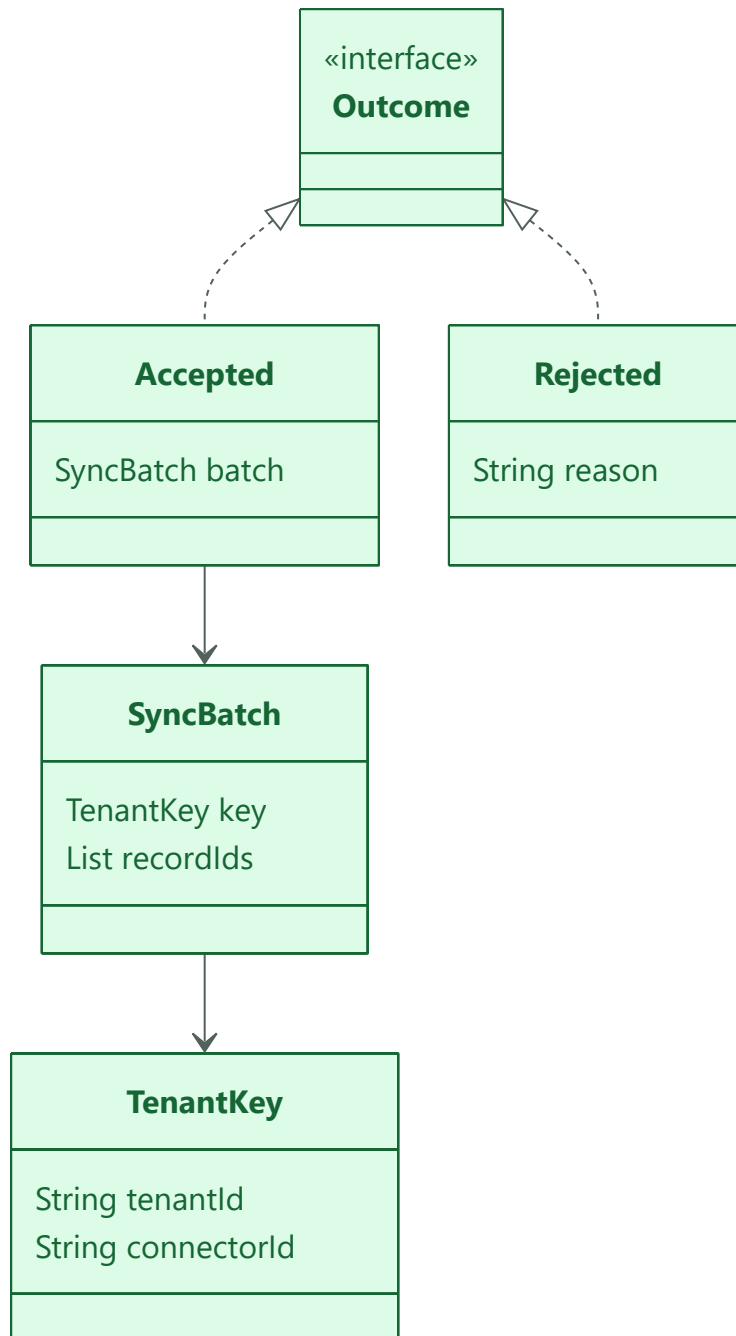
9. Records, Sealed Types and Pattern Matching

Why these features exist: a batch envelope is data with an invariant; a processing outcome is a constrained alternative; reading that alternative should not require repeated casts. These are different modeling needs, even though their syntax often appears together.

A record defines a transparent carrier with component fields, accessors and generated value-oriented members. It is implicitly final and its component fields are final. A compact constructor can validate and defensively transform constructor arguments. This is **shallow immutability** unless component values themselves have the required immutability/ownership behavior. A record containing a mutable list can expose mutations; an array component also has array equality semantics unless explicitly handled.

CoreJavaLab.SyncBatch uses List.copyOf so callers cannot mutate its list through either the original list or the returned list reference. Strings in that list are immutable, making this particular element contract simple. If the list held mutable payload objects, the copy would not freeze them. Constructor validation rejects bad IDs before a key enters a map.

Diagram 01.8



This is a small Java model, not the persistence schema or job-lifecycle state machine. Sealed **Outcome** permits **Accepted** and **Rejected**; the record implementations are final. A sealed hierarchy limits direct implementations according to the language's package/module rules. Permitted subtypes must choose final, sealed or non-sealed behavior as applicable; a non-sealed branch intentionally reopens extension.

Pattern matching tests and binds a value with a type/shape. In the lab's `describe` method, a switch covers both permitted outcomes and record patterns bind their components. Adding another direct alternative forces a useful exhaustiveness review at compilation in this closed

setup. Pattern matching does not validate arbitrary business invariants or make a remote JSON schema automatically compatible.

Null remains a design decision. The lab rejects a null outcome explicitly before switching. If an API intentionally accepts null, handle that contract explicitly; an exhaustive list of non-null alternatives is not permission to ignore null. Dominated cases and guards also affect which pattern can match; clearer ordering beats clever patterns.

Model	Use when	Avoid when
Record value carrier	Components define the data, equality and exposed contract	Mutable identity/lifecycle or representation hiding is essential
Ordinary class	Encapsulation, controlled mutation or identity needs custom behavior	Writing boilerplate solely to reproduce a simple value carrier
Sealed hierarchy	A bounded set of domain alternatives enables exhaustive reasoning	Third-party connector implementations must extend an open SPI freely
Open interface with implementations	Plugin extensibility is part of the contract	You require the compiler to know every implementation
Pattern switch	Alternatives and their handling read clearly together	Hiding unchecked assumptions about null, invariants or external schema evolution

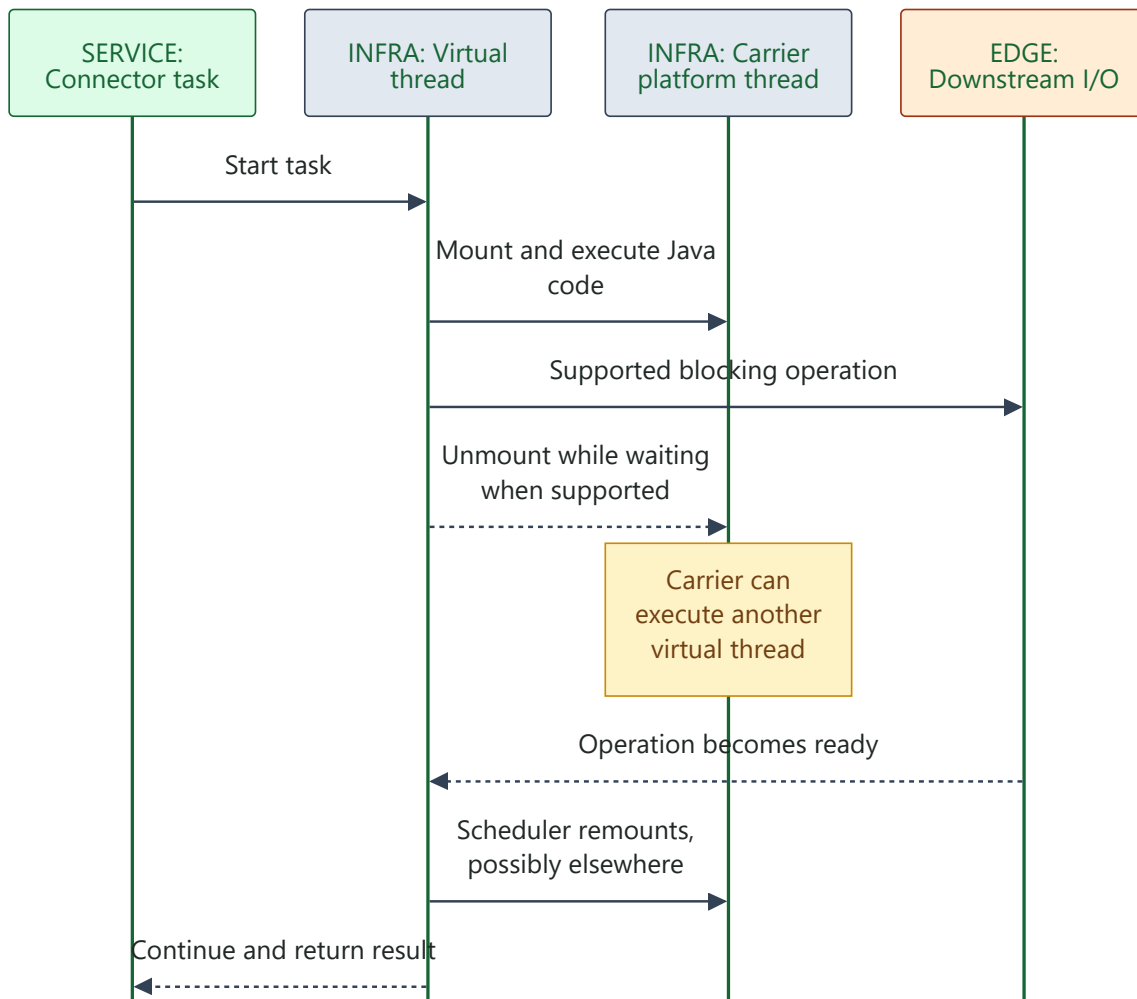
Production traps: generated toString can expose sensitive component values, so do not place raw secrets in convenient DTOs and log them. A record is not automatically a suitable JPA entity; entity proxies and lifecycle constraints need separate analysis in [04](#). Sealing IntegrationHub's connector SPI prevents the extension model it was meant to support; seal internal outcomes instead, and keep connector strategies open where required.

Interview check: "Is every record a good HashMap key?" No. Component equality/hash stability must fit the intended identity. A mutable component can still make a record unsafe as a key, and a secret-bearing record can create logging risks through generated members.

10. Virtual Threads: Cheap Waiting, Not Unlimited Work

Why they exist: synchronous request code is easy to reason about, but dedicating an expensive OS-backed platform thread to every mostly-waiting request can limit concurrency. A virtual thread is a Java Thread managed by the runtime and scheduled onto carrier platform threads. The OS schedules those carriers; it does not directly schedule each Java virtual thread.

Diagram 01.9



Trace the Resources

- The executor creates a virtual thread for the task, rather than borrowing one from a virtual-thread pool.
- While running CPU instructions it needs a carrier and CPU time. Virtual does not mean running without a processor.
- Many supported blocking JDK operations park/unmount the virtual thread while awaiting readiness; not every native/library call has identical behavior.
- Its stack/state and referenced request objects still occupy memory. Cheap waiting can make excessive admitted work consume substantial heap.
- When readiness arrives, the scheduler makes it runnable and mounts it again. A different carrier does not mean a different logical Thread identity.
- Completion/failure must still be observed. Executors do not turn submitted failures into automatically handled business outcomes.

CoreJavaLab.virtualThreadIdentity submits one task and checks Thread.currentThread().isVirtual through a Future with a bounded get wait. It cancels in cleanup and closes the executor. This demonstrates an API/identity contract only; it does not test I/O unmounting, pinning, throughput, or production cancellation. ExecutorService.close can wait for tasks; a timed Future.get alone is not a guarantee that all later cleanup returns within the same deadline.

Java 21 Versus Later Runtimes

In Java 21, a virtual thread performing a blocking operation while holding a synchronized monitor can pin its carrier; some native/foreign-call situations also prevent unmounting. Long frequent pinning can undermine scalability even though mutual exclusion remains correct. Short in-memory synchronized sections are not automatically a reason for a mass lock rewrite.

JEP 491, delivered in Java 24 and inherited by Java 25, changed monitor handling so synchronized constructs no longer cause that same ordinary monitor-pinning limitation. Native callbacks and other specified cases can remain problematic; it is not a promise that all blocking code now releases carriers. The Java 21 diagnostic property jdk.tracePinnedThreads was removed as part of that change, so copying an old diagnosis command can mislead.

[VERIFY: pinning and diagnostic claims are version-sensitive. Check JEP 444 for Java 21 and JEP 491 for the Java 24 change, plus the exact deployed vendor build and library/native-call path. Both JEPs were consulted for this chapter; no pinning experiment was executed.]

[!] TRAP

A virtual thread does not create a new database connection, downstream quota, CPU core or durable transaction. IntegrationHub must bound admitted work and concurrent resource use. A semaphore can limit active calls but an unbounded number of tasks waiting on that semaphore can still exhaust memory. Admission limits and deadlines belong outside the scarce-resource boundary too.

Execution model	Use when	Avoid when
Virtual thread per task	Many independent blocking-style tasks spend time waiting and libraries cooperate	CPU throughput is the bottleneck or unlimited admission hides a queue in memory
Bounded platform-thread executor	CPU work or legacy/library constraints require controlled execution	One blocked task per thread makes the chosen capacity insufficient for the workload
Reactive/event-loop model	End-to-end nonblocking pipelines and streaming/backpressure contracts are valuable	Blocking on the event loop or paying complexity without a requirement

Do not pool virtual threads merely to make them scarce; limit the resource explicitly. Do not create an expensive per-thread database connection/cache for every virtual thread via `ThreadLocal`. Thread-local values belong to the logical virtual thread, not whichever carrier runs it; propagation of request/transaction/trace context to new tasks still needs an explicit policy.

Production case: after enabling virtual threads, the service accepts much more waiting work, yet throughput stays bounded by a downstream quota and heap rises. Inspect in-flight requests, queue/admission limits, retained request size, database pool wait and remote latency. The solution is not necessarily more carriers. 02, 05 and 23 provide the deeper execution, reactive and sizing tools.

Interview check: "Will virtual threads make JSON transformation faster?" Not by themselves. They improve how compatible waiting work consumes OS threads; CPU work still needs CPU and may need a different bounded execution strategy.

Lab Contracts and Evidence

The files below accompany this chapter in `docs/java-fs-guide/code/01-java-jvm/`. They require only an existing JDK 21+, not Maven downloads, Spring or a running database. The runner accepts an existing JDK path and compiles with the Java 21 release target. It records PASS, FAIL or NOT EXECUTED; in this environment the recorded result is **NOT EXECUTED**.

Lab	Contract checks present in source	What it does not prove
CoreJavaLab.java / valuesAndCollections	Defensive snapshot, mutation rejection, record equality, tenant-scoped key lookup, deliberate hash collision, pass-by-value, sealed result handling	Security authorization, distributed idempotency, map performance or shared-state concurrency
CoreJavaLab.java / genericsAndStreams	Variance transfer, laziness, sequential diagnostic traversal, single-use stream, result mutation rejection, duplicate-key collector failure	Parallel safety or scheduling, bulk-sync memory bounds, external schema validity
CoreJavaLab.java / resourceFailures	Reverse close order, primary and suppressed exceptions	Real socket, file or connection-pool cleanup
CoreJavaLab.java / virtualThreadIdentity	Virtual-thread identity through observed Future result	Pinning, I/O unmounting, scheduling fairness, bounded production latency
LoadingLab.java	Loading versus initialization; once-only initialization; first failure versus erroneous-class reuse	Plugin isolation, all linkage failures, class-loader leaks or concurrent initialization deadlock

No expected console transcript is presented as an observation. The chapter's inline LoadingLab listing must stay byte-equivalent after newline normalization to its runnable source. Application checks remain unexecuted until a JDK is available. GC timing and JIT behavior require dedicated measured experiments rather than assertions that rely on nondeterministic collection timing.

Explained-Here Index and Sources

Concept	Owning section
Java release/API/runtime targeting	Version boundaries
JVM architecture, JIT and pass-by-value	Runtime execution
Class loaders, linking and initialization failure	Class lifecycle
JVM memory, GC, reachability and retained graphs	Memory and GC
Collections, equals/hashCode and map internals	Collections
Generics, variance, erasure and heap pollution	Generic contracts
Exceptions, suppression and resource ownership	Failure contracts
Lambdas, functional interfaces, Streams and Optional	Functional pipelines
Records, sealed types and pattern matching	Value and alternative modeling
Virtual threads, carriers and pinning	Waiting and resource bounds
Runnable source and execution limitations	Lab contracts

Primary references consulted for volatile/current claims: [Oracle Java support roadmap](#), [JVMS 21 loading/linking/initialization](#), [JEP 444 virtual threads](#), and [JEP 491 monitor pinning](#). Further authoritative references for the mechanisms are the [JVMS runtime areas](#), [Java 21 API](#), [JLS generics/types](#) and [JLS exceptions and resource statements](#). The latter are reading references, not a claim that every page was fetched or every platform-specific behavior executed during this review.

Related Chapters

Return to the [master request journey](#), then continue to [02 Concurrency](#). Runtime types feed [03 Spring](#); equality and retention affect [04 JPA](#). Waiting crosses [21 Network/OS](#); nonblocking alternatives are in [05 APIs](#). Apply the patterns in [11 LLD](#), prove them with [19 Testing](#), and measure them in [23 JVM performance](#). Generated reciprocal links also cover the other explicitly referenced chapters.

RELATED CHAPTERS / RECIPROCAL LINKS

- | | |
|--|--|
| 00 The System Before the Details | 23 JVM Performance Engineering |
| 03 Spring and Spring Boot Internals | 16 Build, Ship and Roll Back |
| 04 Hibernate and JPA | 11 Object Design, Patterns and LLD |
| 02 Concurrency and the Memory Model | 19 Testing and Quality |
| 15 Kubernetes and Production Workloads | 13 Data Integration and iPaaS |

07	Messaging, Batch and Streams
05	APIs and Real-Time Communication
10	System Design Foundations
21	Networking and Operating Systems

06	Databases and Durable State
14	Docker and Container Internals
24	IntegrationHub in Motion
20	Appendix: Interview Toolkit and Index

One-Page Cheat Sheet

Execution: source/API target, build JDK and runtime JDK are separate. Bytecode can be interpreted and JIT-compiled. Optimizations must preserve language behavior; warm-up is not a benchmark methodology.

Classes: load bytes -> verify/prepare -> resolve as needed -> initialize on required use. Class identity includes defining loader. An initial non-Error initializer failure can surface as `ExceptionInInitializerError`; later use can fail with `NoClassDefFoundError`.

Memory: heap is not total process memory. Follow retaining paths; unreachable cycles can be collected. GC does not deterministically close sockets. Large allocation rate and a growing retained live set are different observations.

Values and collections: references are passed by value. Equal keys require equal hashes; collisions need not be equal. Keep key identity stable and tenant-scoped. `HashMap` is unordered and not a synchronization mechanism. `ArrayList` is a strong default for bounded batches; choose other structures by contract.

Types: generics are invariant; producer extends, consumer super. Erasure can leave signature metadata and inserted casts. Raw types can move a failure away from its cause. Records are shallowly immutable; seal closed alternatives, not an intentionally open connector SPI.

Failures and Streams: preserve causes/suppressed exceptions; close owned resources; respect interruption. Streams are lazy and single-use; terminal operations drive consumption. Avoid required side effects in intermediate stages and blocking I/O in uncontrolled parallel pipelines.

Threads: virtual threads share carriers while compatible waits suspend; they still consume memory and CPU when running. Bound admission, database use and remote concurrency. Java 21 monitor pinning differs from Java 24/25. No Java lab was executed in this edition.

Interview Corner

Basic: How Are JDK, JVM and Bytecode Related?

The JDK supplies development tools and runtime components. `javac` emits class files containing bytecode and metadata. A JVM loads/links/initializes the classes and executes them, possibly using both interpretation and compiled machine code. Portability still depends on compatible bytecode, libraries and native/runtime requirements.

Internals: Why Can Two Classes with the Same Name Be Incompatible?

Their defining class loaders may differ. Runtime identity is not the printed binary name alone. Inspect the actual Class objects and defining loaders before blaming a compiler bug.

Trace/Debug: A Connector Static Initializer Failed Once; Why Do Retries Look Different?

The initial non-Error failure can be wrapped in `ExceptionInInitializerError`. That runtime class is then erroneous; later initialization attempts can yield `NoClassDefFoundError`. Find the original cause, fix configuration/initialization design, and establish a fresh valid class lifecycle; retrying a business call is not a class reset.

Scenario: Should a Plugin Registry Download Secrets in a Static Block?

No. Static initialization couples type readiness to an external dependency and its failure semantics. Use an explicit lifecycle with timeout, authorization, retry policy and observable readiness appropriate to the service. A factory or injected registry is usually clearer than an implicit startup side effect.

Basic: Is Java Pass-by-Reference?

No. It passes values, including object-reference values. Reassigning a parameter changes that local reference; mutating the shared object can be visible through the caller's reference. Visibility across threads still needs the concurrency contract from Chapter 02.

Internals: Why Can HashMap Find One of Two Keys with the Same Hash?

Hashing narrows the search; equality identifies the matching key. Collision handling retains distinct unequal keys. A poor hash affects distribution/work, not permission to treat unequal keys as the same mapping.

Trace/Debug: A Key Appears in Iteration but get Returns Null. What Do You Check?

Check whether fields used by equals/hashCode changed after insertion, whether the lookup key uses the same identity policy, and whether a null mapping is legitimate. Also check unsafe concurrent access. Do not immediately blame resizing or a hash collision.

Scenario: Which Key Prevents Tenant Cache Mix-Ups?

A stable key containing every dimension of the authorized lookup scope, such as tenant and connector ID. That prevents accidental key aliasing but is not itself authorization. Enforce tenant permission on every access path and test equal connector IDs across different tenants.

Basic: Can GC Fix a Memory Leak?

It reclaims objects that are reclaimable under reachability/reference rules. It cannot infer that the business no longer wants objects still held by an unbounded cache or listener. Remove or bound the retaining ownership after collecting evidence.

Internals: Why Can Heap Fit Yet the Container Be Killed?

The container accounts for more than Java heap. Native/direct allocations, metadata, platform-thread stacks, code cache and other process/runtime memory need headroom. Examine container and process evidence rather than setting maximum heap equal to the container limit.

Trace/Debug: Does a Rising Heap Graph Prove a Leak?

No. Compare live-set/retention behavior across comparable workload and collection conditions, and inspect retaining paths if warranted. Allocation bursts, delayed collection and cache warm-up can raise occupancy without an ever-growing retained graph. Avoid claims from one sample.

Basic: Why Is List of Integer Not List of Number?

A writable Number list could accept Double and break an Integer list's contract. Use an upper-bounded view when only reading number values and a lower-bounded view for accepting specific values. The appropriate variance depends on the operation.

Internals: How Can a Generic List Throw `ClassCastException` on Retrieval?

Raw types or unchecked code may have inserted a value violating the assumed element type. Erasure means that insertion may not check the full generic contract; a compiler-inserted cast at retrieval reveals the pollution. Trace backward to the unchecked boundary.

Trace/Debug: Both the Work and close Failed. Which Exception Wins?

With try-with-resources, the body failure remains primary and close failures are suppressed, with successfully acquired resources closed in reverse order. Preserve causes and suppressed details when adapting the failure to the domain/API.

Scenario: Is `IOException` Always Retryable?

No. Retryability depends on the operation, failure cause, deadlines and whether a remote effect might already have happened. Retrying a non-idempotent write after an uncertain timeout can duplicate effects. Exception inheritance is not an idempotency guarantee.

Basic: Why Did a Stream Do Nothing?

There may be no terminal operation. Even with one, short-circuiting and permitted optimizations can skip stages/elements. A pipeline describes processing; it is not a queue of guaranteed callback invocations. Inspect the terminal contract.

Internals: Why Can Parallel Reduction Give the Wrong Answer?

The identity, accumulator or combiner may violate reduction requirements, including associativity, or shared mutable state may race. Splitting and combining work must preserve the intended result. A sequential run looking correct does not prove the parallel contract.

Scenario: Should a Connector Use `parallelStream` for HTTP Calls?

Not as a default. Blocking calls, provider limits, retries, deadlines and context propagation require an explicit execution/admission policy. Consider bounded concurrency with a suitable executor or a nonblocking pipeline, and measure the real workload.

Basic: Does a Record Freeze a List Component?

No. Its field reference is final, but the referenced list or elements may mutate. A defensive unmodifiable snapshot protects the list structure; deep immutability requires an element contract too. Validate nulls and other invariants deliberately.

Internals: Why Pair a Sealed Hierarchy with Pattern Switch?

The compiler can reason about a controlled set of alternatives and check coverage in a suitable exhaustive switch. Records can expose components to patterns. Null, non-sealed branches and external schema evolution still require explicit decisions.

Scenario: Should the Connector Interface Be Sealed?

Only if the allowed implementations really are closed. For an extensible connector SPI, an open interface supports plugins; seal internal outcomes or commands whose alternatives the application owns. Match the model to extension requirements.

Basic: Is a Virtual Thread a Faster OS Thread?

No. It is a Java thread the runtime schedules on carrier platform threads. Compatible waiting can release a carrier; running code still uses CPU. It improves concurrency economics for suitable workloads, not the instruction speed of a computation.

Trace/Debug: A Virtual-Thread Service Has Low CPU and Growing Latency. What Next?

Inspect downstream wait, pool acquisition, admission/backlog, retained in-flight data and version-appropriate pinning evidence. Name the actual runtime before repeating Java 21 monitor advice on Java 25. Low CPU alone does not prove a scheduler defect.

Scenario: Does More Waiting Capacity Mean We Can Remove Limits?

No. Each task retains state and competes for finite downstream capacity. Bound admission as well as active calls, apply deadlines, observe completion and cancel appropriately. A semaphore in front of an unlimited population of waiting tasks is not a bounded system.

02 / Concurrency and the Memory Model

Correct shared state, bounded work, observable completion. Concurrency lets IntegrationHub overlap independent sync jobs and I/O waits. It also creates races over configuration, quotas, progress and shutdown. IntegrationHub is fictional; all schedules, capacities and incidents in this chapter are teaching examples, not observed production behavior.

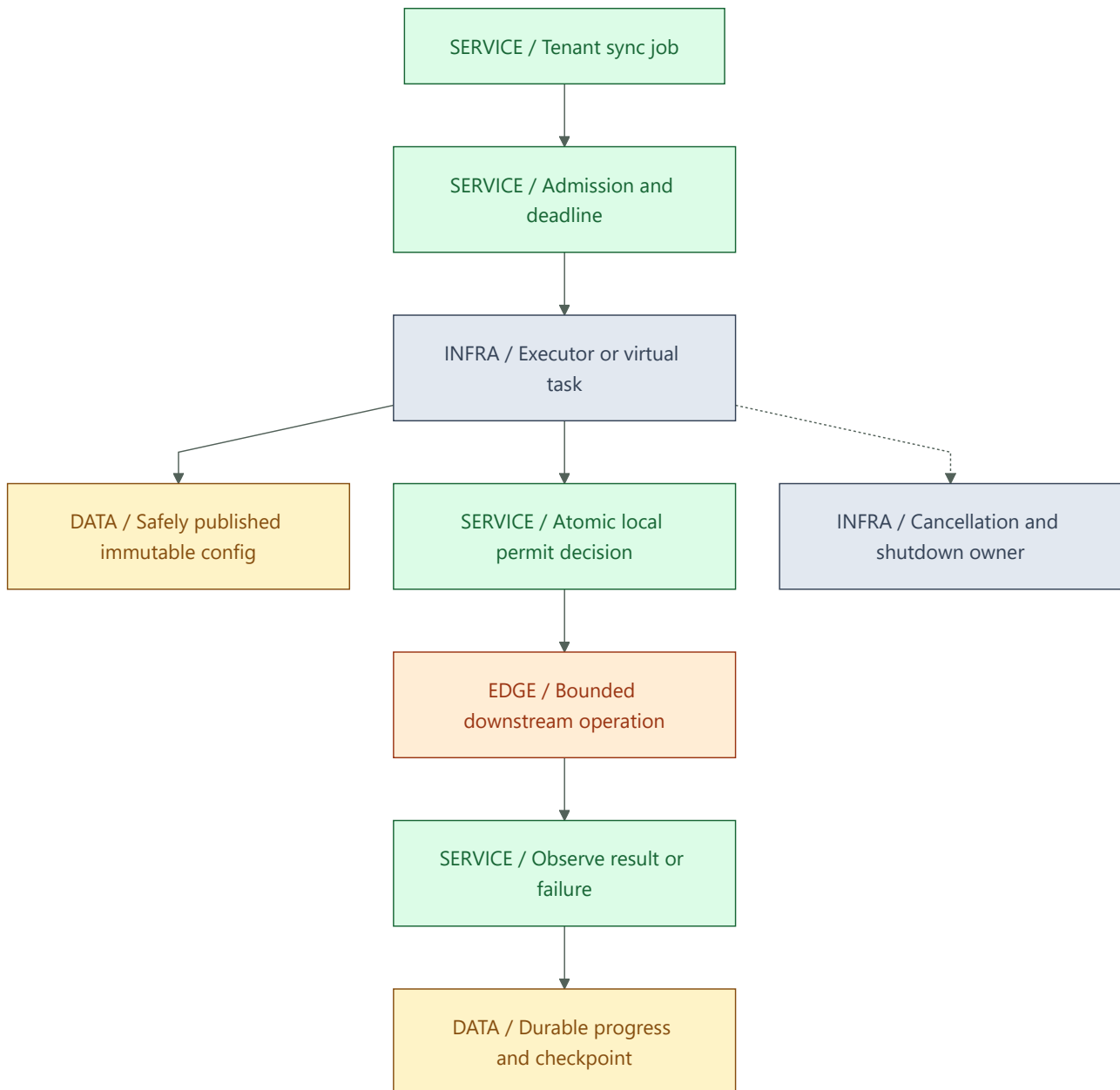
Version assumptions: Java 21 without preview features is the source baseline. Java 25 remains the current-LTS comparison checked in Chapter 01; monitor-pinning changes are distinguished where relevant. The surrounding reference design still assumes illustrative Spring Boot 4.0.x / Spring Cloud 2025.1.x, Kubernetes 1.34, PostgreSQL 17 and Kafka 4.x. No framework or infrastructure runtime is required by these standalone labs; this chapter introduces no new Spring pairing. The prior family-level compatibility check is documented in [00](#).

[VERIFY: confirm the deployed JDK vendor/build and framework patch compatibility against the JDK release notes, Spring Boot system requirements and Spring Cloud compatibility table. Java 21 source compatibility does not validate a production executor configuration or third-party driver's cancellation behavior.]

Execution status: NOT EXECUTED. The local runner found no compiler through PATH/JAVA_HOME and recorded that result. Both complete Java labs are under `code/02-concurrency/` ; no Java dependencies or JDK were downloaded. Compilation, runtime assertions, stress behavior, pinning and performance remain unverified. Mermaid diagrams are rendered as embedded SVG in the published guide; Java execution remains a separate, unverified requirement.

Big Picture

Diagram 02.1



One successful run is not a proof of thread safety. Name the shared state, the invariant, the operation that changes it atomically, and the ordering that makes it visible. Then explain what happens when admission fails, waiting is interrupted, or a worker never returns.

What You Will Be Able to Explain

- Separate visibility, ordering, atomicity, safety and liveness, and identify the happens-before relationship for a handoff.

- Choose confinement, immutable snapshots, monitors, atomics or locks from the invariant rather than familiarity with an API.
- Trace `ThreadPoolExecutor` submission, worker growth, queueing, rejection, result handling and shutdown.
- Compose `CompletableFuture` stages without hidden blocking, dropped failures or imaginary cancellation guarantees.
- Use concurrent collections without assuming multi-call transactions or consistent whole-map snapshots.
- Diagnose deadlocks and pool starvation, and bound virtual-thread work without confusing local permits with distributed quotas.

[P] PRODUCTION

Where this lives in IntegrationHub. A connector worker shares read-mostly configuration, enforces local outbound concurrency and reports progress while many jobs run. The [durable request journey](#) remains authoritative: an in-memory lock does not replace [database constraints](#), [idempotent event processing](#), or [distributed coordination](#). [Spring](#) may schedule or proxy work, but its beans still obey Java concurrency contracts.

1. The JMM: Which Observations Are Legal?

The Java Memory Model defines allowed observations of shared-memory operations across threads. It lets compilers and processors optimize while constraining correctly synchronized programs. It is not a tutorial instruction to "flush all caches" every time a keyword appears. Hardware coherence alone does not establish a Java-level synchronization contract.

Visibility asks whether one thread can observe another's updates. **Ordering** asks which effects must precede which observations. **Atomicity** asks whether a relevant operation can be interleaved or partially observed. **Safety** means an invariant is not violated; **liveness** means required progress is possible. A perfectly visible counter can still lose updates, and a mutually exclusive critical section can still deadlock.

Two conflicting accesses to the same variable, at least one a write, constitute a data race when they are not ordered by happens-before. A broader business race can also occur among individually thread-safe operations: checking a concurrent map and later inserting is not one indivisible reservation. Conversely, making something run in a single thread can eliminate the need for shared-memory coordination if ownership really stays confined.

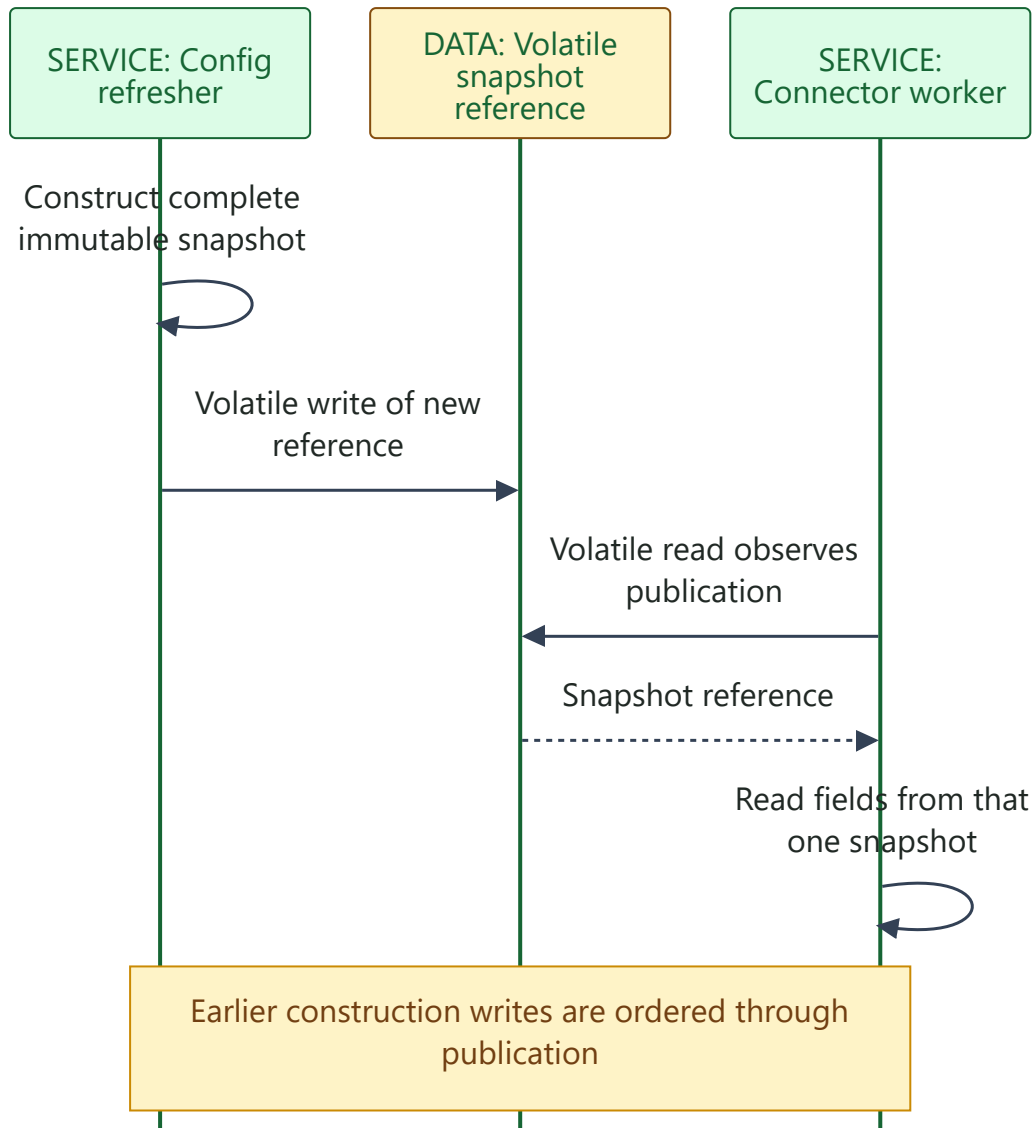
Construct a Happens-Before Chain

Happens-before is a partial order, not wall-clock ordering. Within one thread, earlier actions in program order happen-before later actions. Synchronization adds edges between threads; transitivity combines them. If initialization happens-before publication and publication happens-before a reader's access, the relevant earlier writes are ordered before that access. A timestamp in a log is not such an edge.

Handoff	Ordering you can use	Boundary to remember
Unlock then later lock of the same monitor	Writes before release are ordered before reads after acquisition	Locking a different object provides no matching monitor handoff
Volatile write then subsequent read of that same volatile variable	Prior writes are published through the volatile synchronization edge	It does not make a group of later mutations atomic
Thread.start	Actions before start are ordered before actions in the started thread	Calling run directly is an ordinary method call, not a new thread
Successful Thread.join observing termination	Joined thread's actions are ordered before the joiner's subsequent actions	A timed join that returns while the target is alive is not proof of termination
Task submission and Future.get	Pre-submission actions precede task actions, which precede actions following result retrieval	Mutating captured state after submission races unless separately coordinated
BlockingQueue handoff	Actions before placing an element precede subsequent access/removal by another thread	Mutating the transferred object afterward requires another ownership protocol
CountDownLatch countDown and successful await	Pre-countDown actions precede actions after the corresponding await completes	Timeout/interrupt is not successful completion of the wait

These are contract summaries, not an exhaustive formal definition. The [JLS memory-model chapter](#) and [concurrent package memory-consistency documentation](#) are the primary references. For correctly synchronized programs, the JMM provides the familiar sequentially consistent reasoning model; a data race removes that general assurance, not all language guarantees.

Diagram 02.2



Safe Publication Is Not Permanent Thread Safety

Safe publication makes a constructed object visible through a valid handoff. If another thread later mutates it, those mutations need their own contract. Returning a mutable `ArrayList` from a safely published singleton does not make the list safe to share. An immutable object can be shared much more simply, provided its construction and references do not leak inappropriately.

Final fields have special initialization-safety semantics for correctly constructed objects. Do not let this escape during construction through a listener registration, overridable method or started task. Final does not make referenced objects deeply immutable, and it is not a general mechanism for promptly communicating a replacement reference. Prefer an obvious publication mechanism instead of depending on subtleties to justify a racy reference field.

Use when / avoid when: use a happens-before proof for each state handoff. Avoid "it usually sees the change," sleeps, yields, log statements, or an assumed CPU memory barrier as a correctness argument. Logging can perturb timing and may add synchronization, so a race disappearing under logging is a clue, not a fix.

IntegrationHub failure: a configuration refresher mutates a shared map in place while workers read it. Workers can see inconsistent policy fields even if a flag is volatile. Publish a validated immutable snapshot, or guard all participating state with one appropriate lock. Read the snapshot reference once per logical decision; two separate reads may select two different configurations.

Interview checks: basic: visibility is not atomicity. Internals: identify the exact publication edge and transitive chain. Debug: find the first unsynchronized mutation after publication. Scenario: replacing an immutable policy object is simpler than coordinating many mutable fields independently.

2. A Complete Snapshot Publication Example

NOT EXECUTED. The complete listing matches `code/02-concurrency/SnapshotLab.java`. It is a finite teaching probe, not a production spin-wait design. The reader may start before or after publication; correctness comes from the contracts, not which schedule this one execution happens to take. No console output is asserted here.

java

```
import java.util.Map;
import java.util.Objects;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public final class SnapshotLab {
    record Limits(Map<String, Integer> byTenant) {
        Limits {
            byTenant = Map.copyOf(byTenant);
            if (byTenant.values().stream().anyMatch(limit -> limit < 1)) {
                throw new IllegalArgumentException("Limits must be positive");
            }
        }
    }

    static final class Registry {
        private volatile Limits current = new Limits(Map.of());

        void replace(Limits next) {
            current = Objects.requireNonNull(next);
        }

        Limits snapshot() {
            return current;
        }
    }

    public static void main(String[] args) throws Exception {
```

```

var registry = new Registry();
var oldSnapshot = registry.snapshot();
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    var reader = executor.submit() -> {
        long deadline = System.nanoTime() + TimeUnit.SECONDS.toNanos(5);
        while (System.nanoTime() - deadline < 0) {
            Limits observed = registry.snapshot();
            if (observed.byTenant().containsKey("tenant-a")) {
                return observed.byTenant().get("tenant-a");
            }
            if (Thread.currentThread().isInterrupted()) {
                throw new InterruptedException("Reader cancelled");
            }
            Thread.onSpinWait();
        }
        throw new AssertionError("Reader deadline exceeded");
    });
    try {
        registry.replace(new Limits(Map.of("tenant-a", 3)));
        if (reader.get(10, TimeUnit.SECONDS) != 3) {
            throw new AssertionError("Published snapshot contents");
        }
        if (!oldSnapshot.byTenant().isEmpty()) {
            throw new AssertionError("Prior snapshot must remain unchanged");
        }
    } finally {
        reader.cancel(true);
    }
}
System.out.println("SnapshotLab: all checks passed");
}

```

Trace: the Registry initially owns an empty snapshot. The writer validates a replacement, copies the map into an unmodifiable representation, and assigns the volatile reference. The reader takes one local reference and inspects only that snapshot. The previously obtained snapshot stays unchanged. String keys and Integer values keep the element contract simple; mutable values would need deeper ownership reasoning.

The main thread obtains the reader's result through `Future.get`, establishing the result-observation boundary. The five- and ten-second limits are test failure guards, not latency targets or measured performance. `Thread.onSpinWait` is only a hint; the repeated volatile read supplies the ordering contract. A production waiter should generally use a blocking coordination mechanism or request-driven snapshot reads.

Trap: this is not a test that proves removing volatile always causes visible corruption. Final-field semantics, scheduling and runtime behavior can make many incorrect variants appear to pass. Use the JMM argument as the proof obligation and tests as supporting evidence. Compare-and-set or a suitable lock is needed if multiple writers must update from the current value without losing each other's changes; simple replacement deliberately allows the latest published assignment to replace the prior policy.

3. synchronized and volatile: Match the Size of the Invariant

Why monitors exist: several related reads/writes may need to behave as one protected operation. Acquiring an object's monitor provides mutual exclusion against other code acquiring that same monitor; releasing and reacquiring it provides memory ordering. A synchronized instance method uses the receiver's monitor, while a static synchronized method uses the corresponding Class object's monitor. These are not the same lock.

Mechanism: acquire monitor -> inspect guarded state -> validate/update the invariant -> release even on exceptional method/block exit. Java monitors are reentrant: the owning thread can acquire the same monitor again. Reentrancy avoids self-deadlock for that case; it does not prevent cycles involving different locks or resources. Synchronizing on one instance does not coordinate another instance or another Pod.

ConcurrencyLab.Quota guards a check-and-decrement under one synchronized method. Starting with one available unit, two reservations can yield only one success. The entire invariant is inside the monitor; reading remaining outside that monitor and then calling a separately synchronized decrement would reopen a race.

Volatile is different: a volatile read/write is an ordered access to that variable; it does not hold a critical section around surrounding code. Increment is read, compute, write, so two threads can both read the old value and overwrite one another even when the field is volatile.

Step in a constructed schedule	Worker A	Worker B	Shared counter
Start	No work yet	No work yet	0
Read	Copies 0 locally	Copies 0 locally	0
Compute	Computes 1	Computes 1	0
Write	Writes 1	Later writes 1	1

This is an allowed interleaving, not measured output. ConcurrencyLab deliberately splits the increment and uses a barrier after both reads so the schedule is reproducible if the test runs. It demonstrates why ordering each access is not enough; it is not a claim that a normal ++ expression always loses exactly one update.

Tool	Use when	Avoid when
Thread confinement	One task/thread owns mutable state until a defined handoff	A reference secretly escapes to another task or callback
Immutable snapshot with volatile reference	Independent readers need a coherent replaceable configuration	Writers must perform a coordinated read-modify-write without replacement conflicts

Tool	Use when	Avoid when
Volatile flag	One state transition/notification is sufficient and cancellation can be observed	The worker is stuck in I/O that never checks it, or several fields must update atomically
synchronized	A clear multi-field invariant fits a small, well-defined critical section	A global lock spans slow network calls or uncontrolled callbacks
Atomic variable	A single state value has an appropriate atomic operation	Updating several independent atomics is incorrectly treated as one transaction

wait/notify mechanics: a thread must own the object's monitor to call wait/notify. wait releases that monitor while waiting and reacquires it before returning or throwing. notify/notifyAll do not transfer ownership immediately; the notifying thread still holds the monitor until it releases it. Wait in a loop that checks the predicate because another thread may consume the condition and wakeups can be spurious. Sleep does not release held monitors. Prefer BlockingQueue or other higher-level coordination where it matches the problem.

Production failure: a synchronized connector cache performs a remote token refresh while holding a global monitor. Every unrelated tenant lookup now waits behind the provider. Move remote work out of the shared critical section, but preserve duplicate-refresh coordination with a per-key in-flight result or carefully scoped state machine. Merely deleting synchronized trades serialization for races.

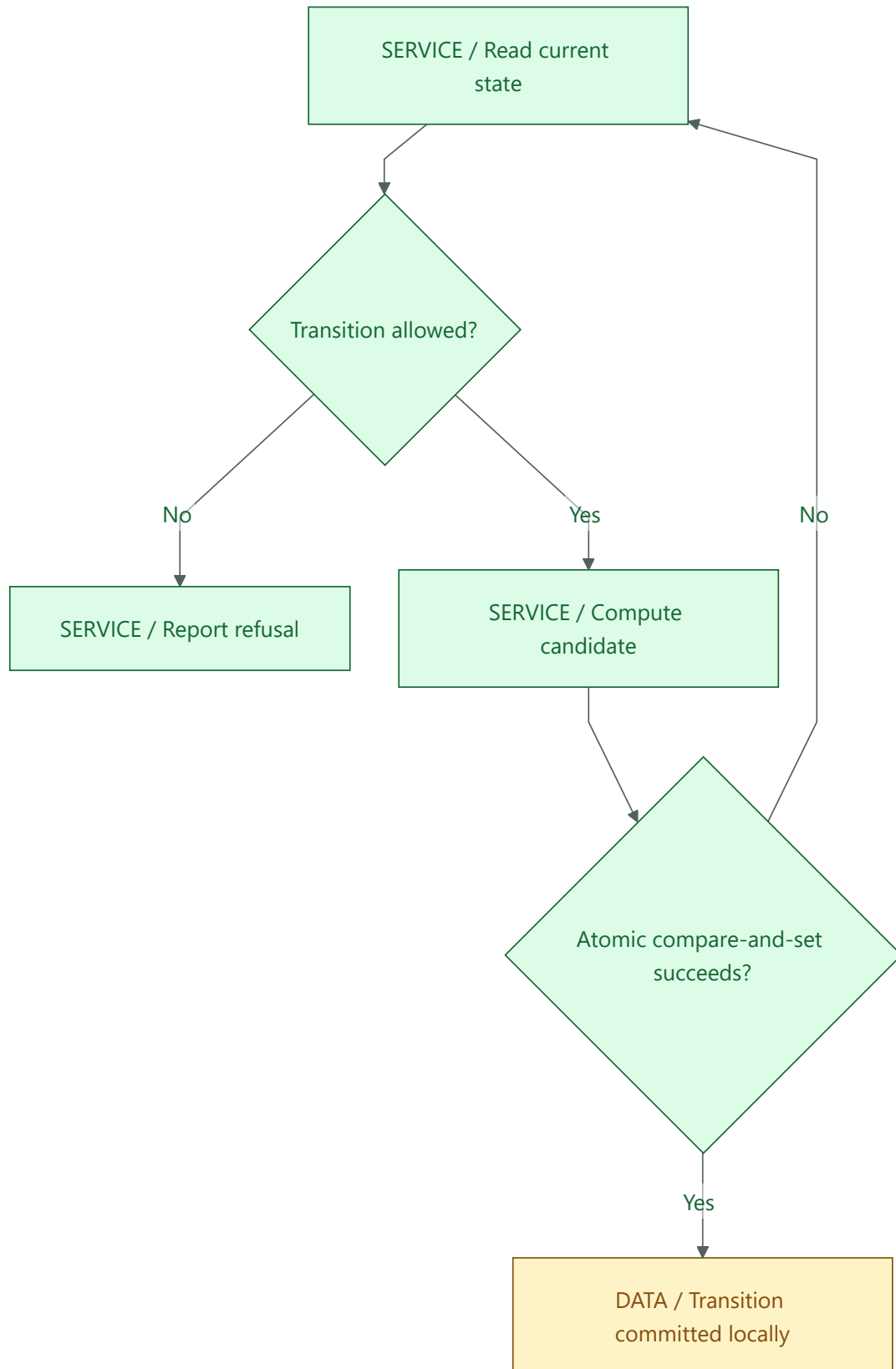
Interview checks: basic: volatile is not a lock. Internals: name the monitor used by instance versus static methods. Debug: inspect whether all accesses use the same guard. Scenario: protect the complete reservation, not separately the check and mutation.

4. Atomics, CAS and Contention

Why atomics exist: simple shared counters and state transitions should not always require a manually managed lock. AtomicInteger.incrementAndGet provides one atomic update, not the split read/write from the preceding schedule. The relevant operation has a point at which it takes effect; separate get and set calls do not become one atomic transaction because the object is atomic.

Compare-and-set tries to replace an expected value with a new value only if the comparison still holds at the atomic operation. A retry loop reads current state, computes a candidate, attempts CAS, and recomputes if another writer won. The application must define what to do when the state no longer permits the operation, such as a quota already exhausted.

Diagram 02.3



CAS can avoid blocking on a monitor, but retries under contention still consume CPU. Lock-free does not mean wait-free, starvation-free, fair or automatically faster. A lock-free algorithm's progress property concerns system-wide progress; a particular participant may repeatedly lose. Do not infer algorithm-wide progress guarantees solely from one atomic instruction.

The retry-side-effect trap: an update function supplied to an atomic update operation may run more than once before an update succeeds. Keep it free of external side effects. Calling a payment provider, appending an audit event, or publishing to Kafka in that function can duplicate effects even though only one local state change wins.

ABA: a reference/value changes from one state to another and back, so a comparison succeeds although relevant intervening history occurred. A version stamp or a different ownership design can encode that history where it matters. Not every counter use has an ABA requirement; explain the invariant before adding stamped references. Java GC reduces certain reclamation hazards compared with manual memory management but does not erase logical ABA concerns.

Primitive	Use when	Avoid when
AtomicInteger / AtomicLong	Exact atomic single-value update or compare-and-set fits the invariant	Check-one-field/update-another must be atomic together
AtomicReference to immutable state	A complete state transition can replace one immutable value	Expensive retries or object-graph copies outweigh clarity; external effects are inside the update
LongAdder	Highly contended aggregate statistics can tolerate a non-atomic concurrent sum snapshot	Exact quota, sequence generation or money decisions require one current value
Lock-protected state	Multiple fields must change together and the contract is clearer under a guard	You hold the guard while doing slow unrelated work

LongAdder spreads contention across internal state rather than treating every increment as an update to one hot cell. Its sum is not an atomic snapshot during concurrent updates. That is useful for telemetry counters, not a safe admission test such as "sum is below the tenant limit, therefore reserve." False sharing can also make logically independent hot fields contend for cache resources; diagnose it with appropriate profiling rather than guessing a padding recipe.

[VERIFY: atomic access modes, weak-CAS spurious-failure rules, VarHandle memory effects and LongAdder internals must be checked against the selected JDK's atomic/VarHandle API and exact source. This chapter's labs use ordinary strong atomic operations, not hand-written relaxed-memory algorithms or claimed hardware instruction mappings.]

IntegrationHub failure: several local atomics count started, completed and failed work. Reading them separately can yield a combination that never existed together. For a coherent progress snapshot, define one synchronized/immutable aggregate or accept and document approximate telemetry. Durable business completion still belongs to 06 and 07.

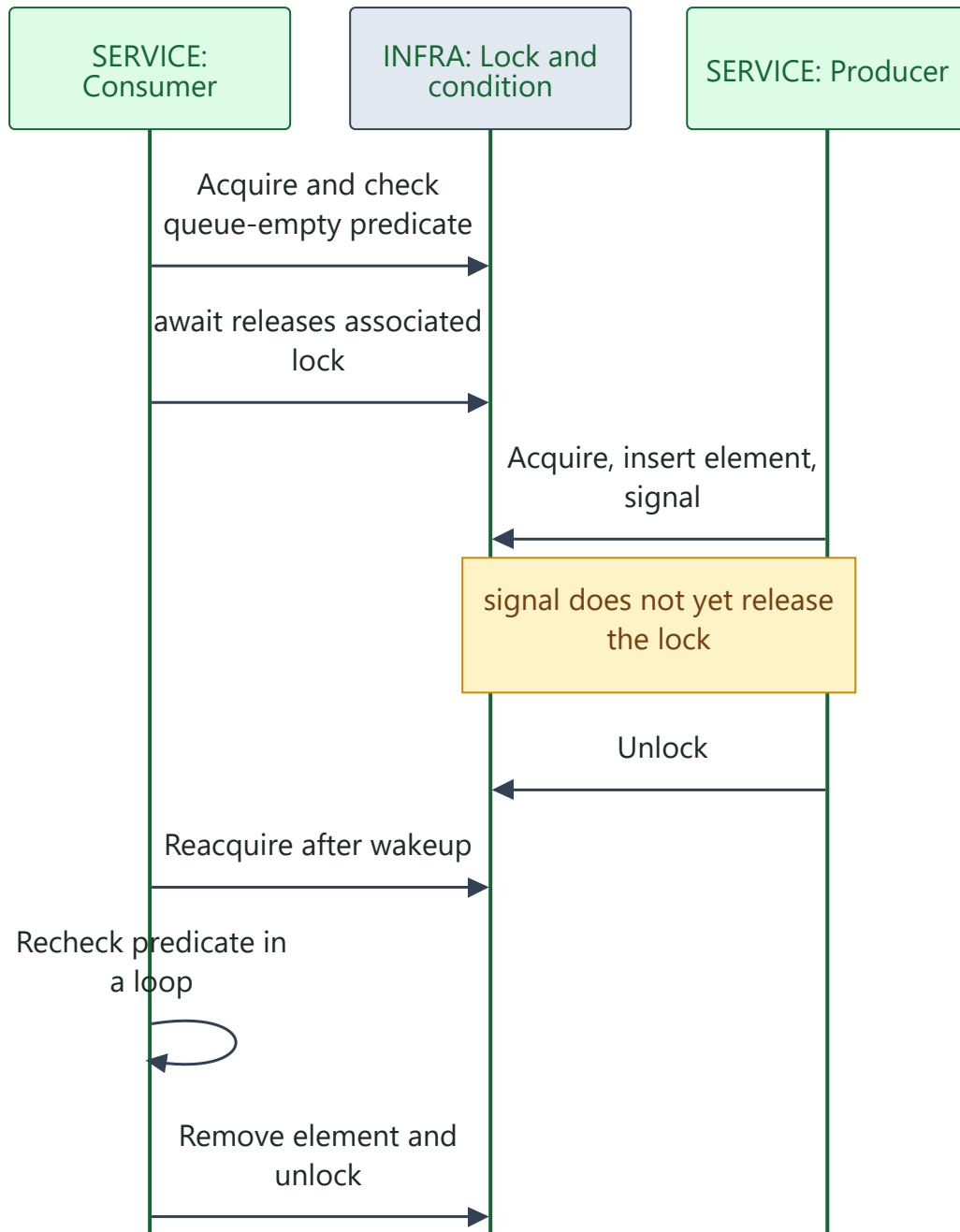
Interview checks: basic: atomic get plus atomic set is not atomic increment. Internals: successful CAS is the transition's commit point, not a durable database commit. Debug: high CPU can be retry contention. Scenario: choose LongAdder for statistics, not precise permit accounting.

5. Explicit Locks, Conditions and Coordination

ReentrantLock provides explicit acquisition policies beyond intrinsic monitors: interruptible acquisition, timed attempts and optional fairness. Release in finally after successful acquisition; failing to acquire must not lead to an unconditional unlock. A lock is still a local coordination object, not a database lock or cluster-wide lease.

In OpenJDK, many synchronizers build on AbstractQueuedSynchronizer. At a high level, state transitions decide whether acquisition succeeds; unsuccessful contenders enqueue and may park; release can make a successor eligible to retry. Waking does not mean the invariant is now available forever. The details differ across locks and releases; memorize the state/queue/park relationship before memorizing private fields.

Diagram 02.4



Conditions separate waiting sets associated with a lock, such as not-empty and not-full. `await` releases the associated lock while waiting and reacquires it before normal continuation. Another thread signals after changing the predicate while holding that lock. Always recheck in a loop; signal is not a remembered business event or a queued message. A signal before anyone waits need not be saved for a future waiter.

Coordination tool	Use when	Avoid when
<code>ReentrantLock</code>	Timed/interruptible acquisition, conditions or a justified fairness choice is required	Adding lock/unlock boilerplate without a need beyond synchronized

Coordination tool	Use when	Avoid when
ReentrantReadWriteLock	Measured read-heavy access to a mutable structure benefits from concurrent readers	Tiny reads or frequent writes make overhead/delays worse; attempting an unsafe read-to-write upgrade
StampedLock optimistic read	Small copied observations can be validated and retried safely	Reentrant calling patterns, unvalidated use of read data or complex object traversal during concurrent mutation
Semaphore	Limit concurrent users of a resource without exclusive ownership of a data structure	Treating a permit as a durable/distributed quota or releasing one that was never acquired
CountDownLatch	One-time completion of a known number of events	You need to reset the same latch for another round
CyclicBarrier / Phaser	Parties coordinate repeated or phased work	Parties can disappear without a defined failure/deregistration policy
BlockingQueue	A producer-consumer handoff and optional capacity bound fit directly	Building an extra wait/notify protocol around it unnecessarily

A fair lock reduces certain forms of acquisition bargaining but cannot guarantee OS scheduling fairness or a hard response-time bound. Untimed tryLock can have different fairness behavior from queued/timed acquisition. Read-write locks are not universally faster; lock upgrading can deadlock if readers retain their read locks while waiting for exclusive access. Optimistic read is not permission to use torn logical state before stamp validation.

[VERIFY: fairness exceptions, condition interrupt behavior, StampedLock guarantees and AQS internals are API/implementation-specific. Check the Java 21 lock/condition documentation and the deployed JDK source before attributing a wait state, spinning policy or performance advantage to a particular lock.]

Production case: a connector acquires a semaphore permit, then an exception path returns without releasing it. Capacity declines until all jobs wait. Scope release to a finally block after confirmed acquisition, account for cancellation before acquisition, and monitor wait duration plus admitted/active work. Conversely, releasing after an interrupted failed acquire invents capacity and violates the bound. ConcurrencyLab's interrupted waiter checks that no permit appears from a failed acquisition.

Interview checks: basic: a semaphore limits concurrent access, not rate per time interval. Internals: await temporarily releases the associated lock. Debug: a missed predicate check can turn a wakeup into an invalid dequeue. Scenario: prefer an existing bounded queue over a custom condition protocol unless requirements justify one.

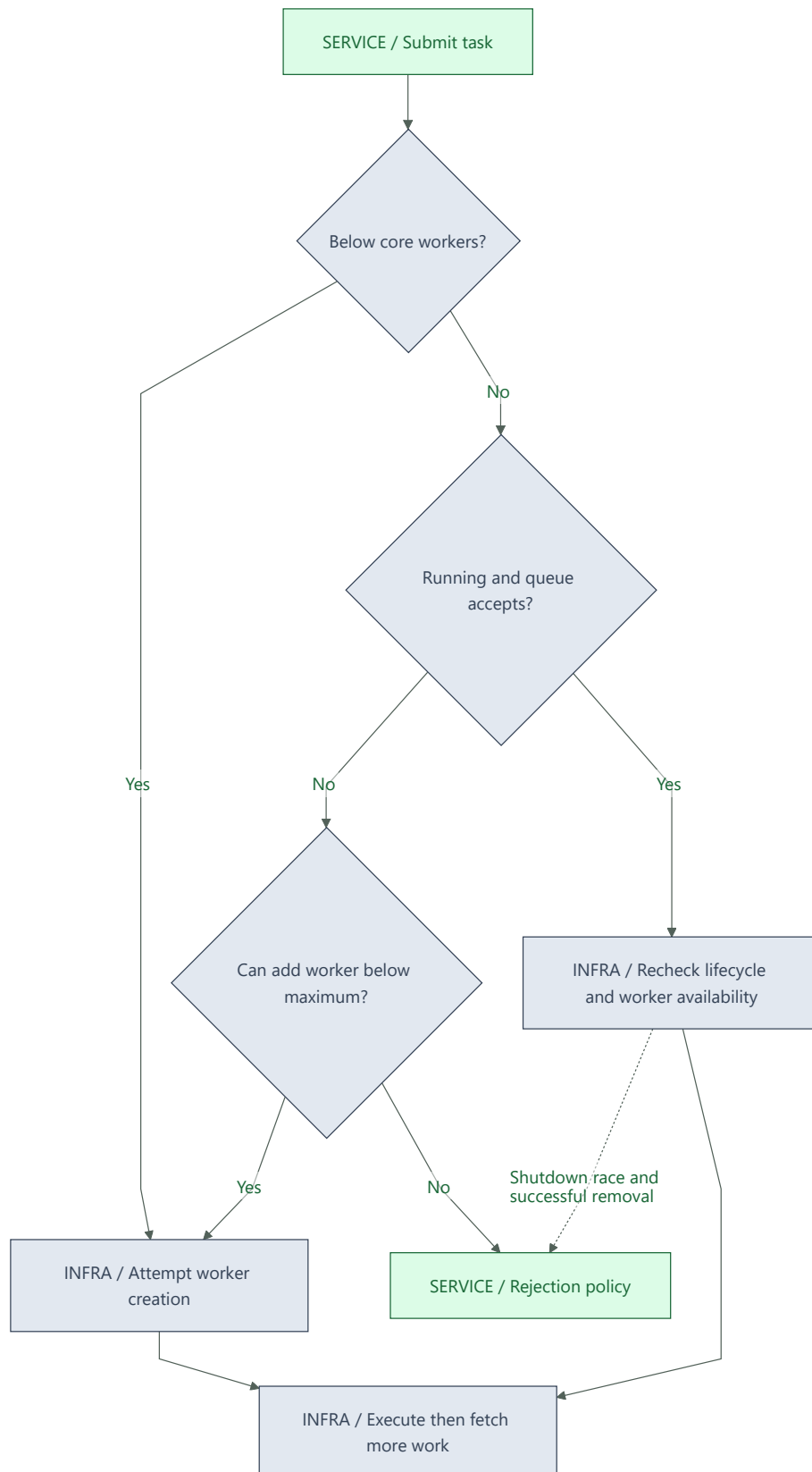
6. ThreadPoolExecutor: Admission Policy Before Thread Count

Why executors exist: separate task submission from thread creation, reuse platform threads, observe results and own shutdown. ThreadPoolExecutor adds explicit workers, a work queue, thread factory and rejection policy. Choosing only `maximumPoolSize` ignores much of its behavior.

Submission Trace

Under ordinary running conditions, if fewer than `corePoolSize` workers exist, attempt to create a worker for the new task. Otherwise try to enqueue it. If queueing fails, attempt to add a worker up to `maximumPoolSize`. If the executor cannot accept it, invoke the rejection handler. The real implementation rechecks lifecycle and worker conditions around races; a successful queue offer alone does not mean shutdown can be ignored.

Diagram 02.5



The diagram summarizes the successful worker-creation branch; thread creation can itself fail. It does not promise a worker for every submit or omit rejection on shutdown as a real possibility. Below core size refers to worker count, not just workers currently executing useful CPU instructions. Core workers are normally created on demand unless prestarted.

A Deliberately Small Saturation Trace

Assume core=1, maximum=2, queue capacity=1. Hold the first task behind a latch so it cannot finish. A second task queues; the pool does not grow immediately merely because maximum is two. The third task cannot queue, so an additional worker can start it. Hold that worker too. A fourth task meets both a full queue and the maximum worker count and is rejected by AbortPolicy. Release the latch, then observe every accepted result.

ConcurrencyLab.queueThenGrowThenReject arranges exactly those conditions with startup latches and bounded waits. These capacities are test fixtures, not recommended production values. The assertions are not yet executed in this environment.

Queue/pool choice	Use when	Avoid when
Bounded ArrayBlockingQueue with finite workers	Memory/backlog and rejection must be explicit	Capacity is picked without retained-task size, deadline and arrival/service-rate reasoning
Unbounded LinkedBlockingQueue	A separately enforced workload bound truly limits submissions	Expecting maximumPoolSize to rescue growth beyond core or absorb sustained overload safely
SynchronousQueue	Direct handoff without buffered task storage fits the admission policy	Unbounded worker growth is allowed under a sustained blocking workload
Fixed-thread factory executor	Simple finite lab or genuinely bounded producer	Assuming its fixed thread count also bounds its default task queue
Separate pools by work class	Slow blocking tasks must not exhaust unrelated CPU/control work	Creating a pool per request and multiplying resources without lifecycle ownership

A standard fixed thread pool uses an unbounded work queue; its thread count bounds running tasks, not waiting work. A cached pool can grow platform threads aggressively. The labs submit only a fixed tiny number of tasks to their helper pools; do not transplant those factories into an unbounded request path without an admission design.

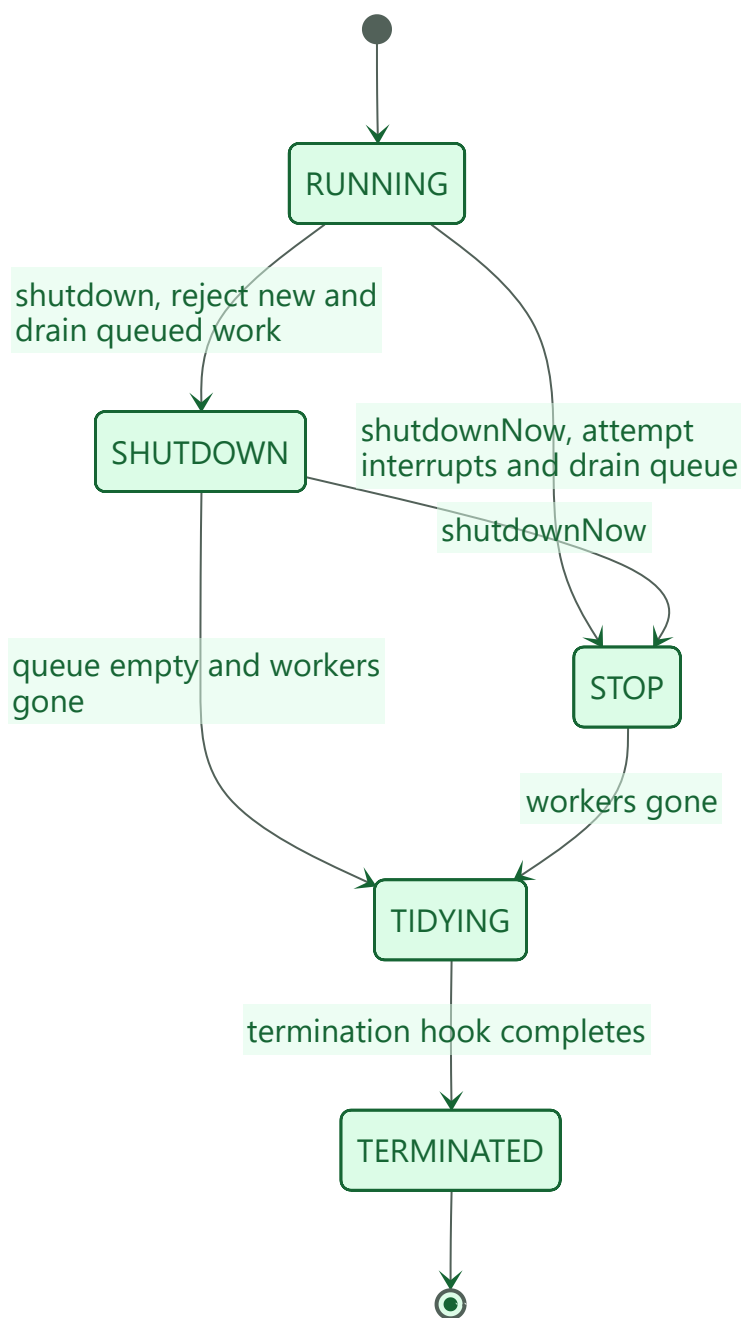
Worker and Lifecycle Internals

A worker runs an initial task or repeatedly retrieves another from the queue. Retrieval can wait indefinitely or with keep-alive semantics depending on pool configuration. Excess idle workers may retire; core timeout can be enabled under its documented conditions. Hooks run

before/after tasks; hook failures can themselves damage worker availability. Approximate active-count statistics are observability signals, not a reservation API.

OpenJDK's implementation combines lifecycle state and worker count in an atomic control field so updates can coordinate decisions about worker creation and shutdown. Its states conceptually progress through RUNNING, SHUTDOWN or STOP, TIDYING and TERMINATED. The principle is an atomic lifecycle protocol with race rechecks, not the numerical bit layout of a private field.

Diagram 02.6



[VERIFY: the queue-before-growth and shutdown contracts were checked in the Java 21 ThreadPoolExecutor API. The combined control field, worker loop and internal state encoding must be checked in the exact OpenJDK/vendor source tag; they are not an ExecutorService implementation requirement.]

Rejection Is Part of the API Contract

Policy	Use when	Avoid when
AbortPolicy	Caller must explicitly handle refused work	Unhandled rejection becomes an unexplained request failure or drops durable work ownership
CallerRunsPolicy	Running on the submitter is safe and slowing that submitter is desired	Submitter is an event loop, holds a lock, has a strict latency budget, or rejection follows shutdown
DiscardPolicy	Completion truly is not relied on and deliberate loss is acceptable	A caller waits for a submitted Future or the task represents accepted business work
Custom reject/defer policy	You have a bounded, observable alternative and precise task ownership	Handler silently pushes work into another unbounded queue

CallerRunsPolicy runs on the caller only when the executor is not shut down; on shutdown it discards. A discarded FutureTask can leave a returned Future incomplete if nobody cancels/completes it. Silent loss is incompatible with IntegrationHub claiming that durable work has been successfully handed off. For accepted jobs, preserve the durable retry/recovery path from 07, rather than equating local submission with durable completion.

execute, submit and Shutdown

An uncaught task failure passed through execute can terminate a worker and reach its uncaught-exception handling. submit wraps work in a Future-bearing task that captures computational failure; ignoring the Future can hide failure. An afterExecute hook's Throwable alone may therefore miss submit failures. Observe results or implement a deliberate completion/reporting policy, preserving exception causes.

shutdown stops acceptance and lets previously accepted work drain; it does not wait for completion. shutdownNow attempts interruption, removes queued tasks and returns those that never began, but cannot force arbitrary code to stop. Removed tasks still need application ownership decisions; do not assume every associated Future automatically completes. awaitTermination returns a status you must inspect. AutoCloseable executor cleanup can wait; it is not automatically bounded by a previous Future timeout.

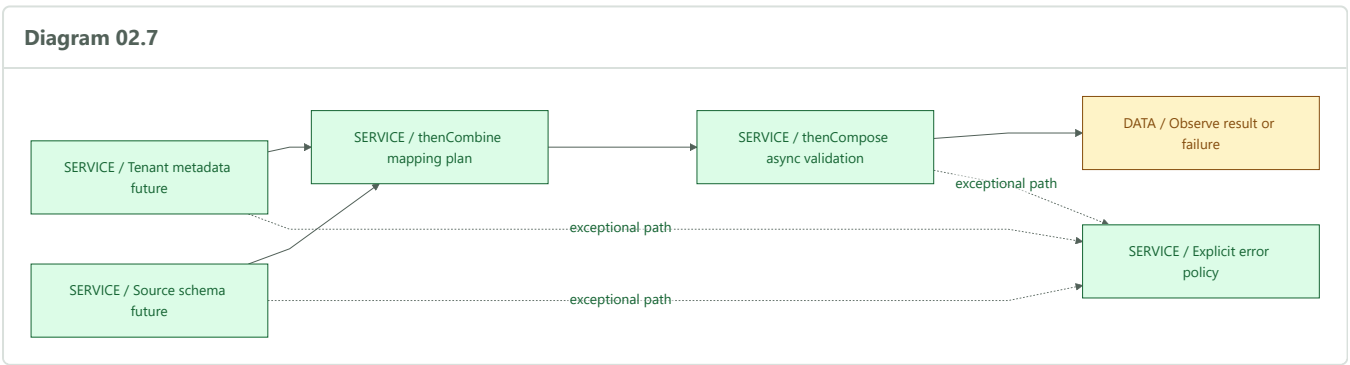
Production incident: CPU is modest while queue age and retained heap rise. Workers may be blocked on remote calls or database acquisition; raising maximum does nothing when an unbounded queue always accepts tasks. Inspect queue type, core/max, active tasks' stacks, queue age, cancellation and downstream budgets. The sizing workflow is in 23, not a universal thread-count formula.

Interview checks: basic: fixed threads do not imply bounded backlog. Internals: trace core -> queue -> maximum -> rejection. Debug: look for captured submit failures. Scenario: make saturation and shutdown visible to callers and durable workers.

7. CompletableFuture: Dependency Graphs, Not Cancellation Magic

Why it exists: represent an eventual result and compose work that depends on it without forcing the caller to block after every step. A CompletableFuture is a completion object; it does not always own the computation that might complete it. Manual completion, callbacks and remote requests can all feed a future.

For IntegrationHub, two independent metadata reads may run concurrently, then combine into a validated mapping plan. A later asynchronous action needs that plan; compose it. Waiting for the first result before submitting the second removes that overlap; submitting both first and then observing their results does not inherently serialize their execution.



Operation	Use when	Avoid when
thenApply	Transform one successful value into another ordinary value	Callback actually returns asynchronous work and you accidentally create nested futures
thenCompose	Next operation returns a CompletionStage depending on the first result	Work is independent and could have been started earlier
thenCombine	Two independent successful results are needed together	One task depends on the other's result before it can start

Operation	Use when	Avoid when
allOf	Need a stage representing completion of a set	Expecting a typed result list, immediate fail-fast behavior, or sibling cancellation
anyOf	First completion, including possible failure, is the desired policy	Expecting first successful result or automatic cancellation of losers
handle	Transform either success or failure to a new outcome	Silently converting real data loss into a normal-looking empty batch
exceptionally	A specific failure can legitimately recover to a value	Every exception is swallowed under a blanket default
whenComplete	Observe/log completion without intentional recovery	Assuming an exception thrown by the callback cannot affect the returned stage

allOf normally waits for all supplied futures to complete; it has a Void result, so retrieve individual results after appropriate completion/error handling. It is not a task scope that automatically cancels siblings on failure. anyOf is first completion, not "first nonexceptional answer." A business policy for hedged requests needs explicit loser cleanup, deadlines and idempotency.

Which Thread Runs the Callback?

A non-async continuation may run on the thread completing the upstream stage or a caller participating in completion; when the value is already available, the registration call can run the action inline. Do not put an expensive/blocking continuation there assuming it has its own worker.

Async methods without an explicit executor use the default asynchronous facility. For ordinary CompletableFuture this is typically the common ForkJoinPool, with the documented low-parallelism fallback; subclasses may override the instance default. Supplying an executor to supplyAsync does not permanently bind all later async stages to that executor. Specify the intended executor at execution boundaries that require one. Even an Executor contract does not universally guarantee a separate thread; its implementation matters.

Timeout, Cancellation and the Underlying Effect

Future.get with a timeout limits the caller's wait. It does not itself cancel the task. CompletableFuture.orTimeout exceptionally completes that same future after the timeout if still incomplete; completeOnTimeout gives that same future a fallback value. Neither guarantees the underlying HTTP/database operation stopped. Because these mutate the future's completion state, applying different caller timeouts to one shared future can affect other observers; a caller-specific derived view needs deliberate design.

`CompletableFuture.cancel` treats cancellation as exceptional completion; its `mayInterruptIfRunning` parameter has no effect in that implementation. Contrast a `Future` returned by executor submission that may attempt to interrupt its running task when cancellation allows it. Even there, interruption is cooperative and a cancelled/done `Future` is not proof that the external operation never happened or the task released every resource.

`get` exposes checked interruption/execution-failure handling; `join` exposes unchecked completion failure and does not provide the same interruptible waiting API. Neither is an invitation to block a scarce worker on another task queued to that same exhausted executor. Compose dependencies or separate execution capacity deliberately.

[VERIFY: `CompletableFuture` completion, default-executor and cancellation policies were checked against the Java 21 API. Check the exact completion-stage implementation and HTTP/JDBC client contracts before relying on timeout propagation, interruption, exceptional-result selection or callback execution context.]

Context trap: thread-local transaction/security/logging context does not automatically follow an arbitrary continuation onto another worker. Capture only the immutable authorized metadata you need, propagate it through supported mechanisms, and restore/clear task context in finally according to nesting rules. Never share one `EntityManager` across arbitrary concurrent tasks; 03, 04 and 18 own the framework and tracing contracts.

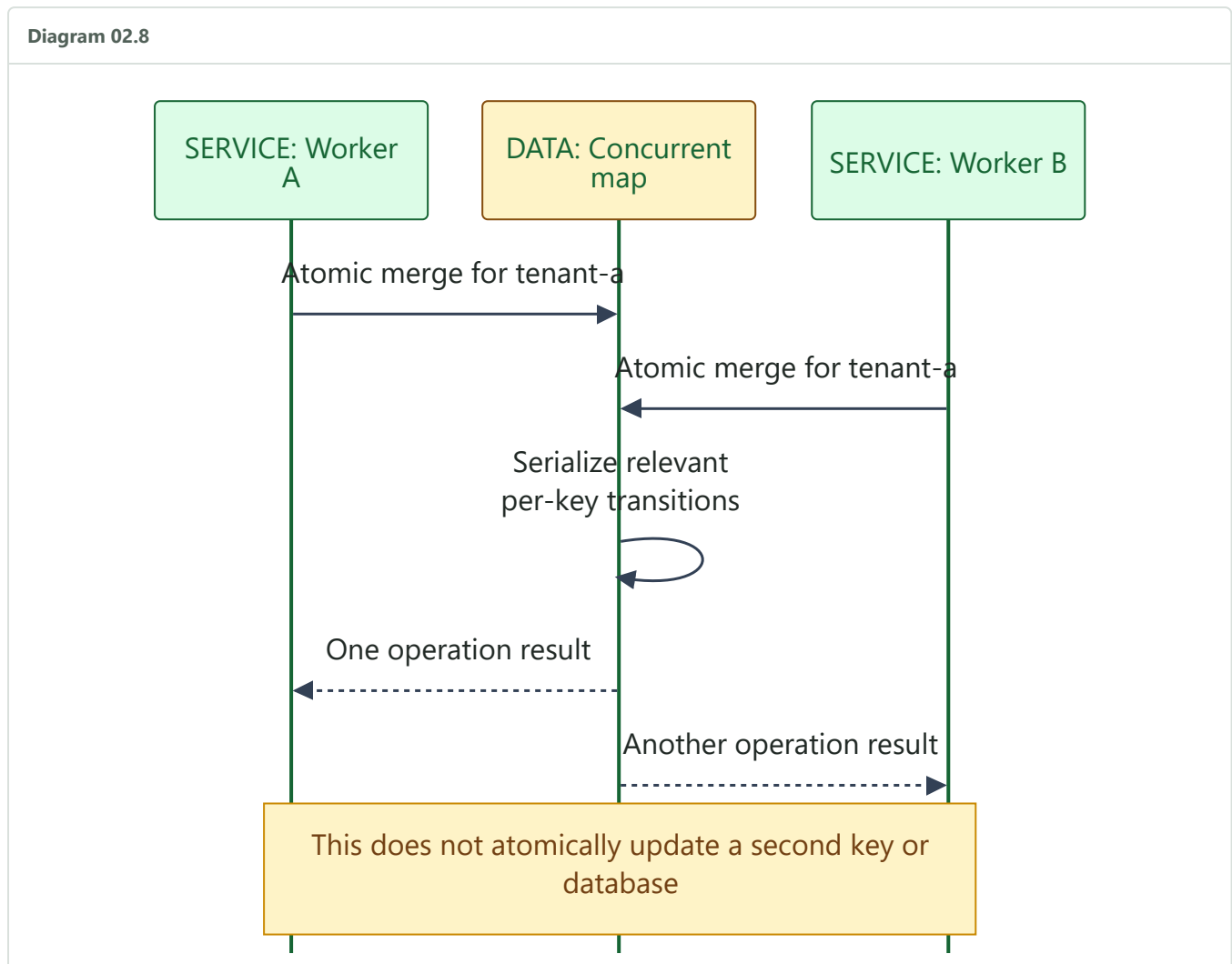
Production case: a timeout fallback reports "zero rows," while the underlying connector is still writing records. A retry overlaps it and duplicates effects. Separate deadline reporting, actual task cancellation and external-effect reconciliation. 05, 07 and 27 explain why unknown outcome is not confirmed failure.

Interview checks: basic: compose flattens a dependent stage; combine joins independent values. Internals: specify the executor per async boundary. Debug: inspect the future that was actually timed out, and the separately owned task. Scenario: cancellation needs task/resource ownership, not just a future flag.

8. Concurrent Collections: One Safe Operation Is Not a Transaction

Why they exist: reusable structures provide documented coordination instead of requiring every caller to guard a `HashMap` or queue manually. They are still precise tools with limits. Thread safety of a container does not recursively protect each stored value or establish consistency across multiple calls.

ConcurrentHashMap supports concurrent access and atomic per-key methods such as `putIfAbsent`, `compute` and `merge` under their specified contracts. A `containsKey` followed by `put` is not one operation. A non-null retrieval reporting an update has the documented per-key visibility relationship; iteration and aggregate observations are not generally a globally locked snapshot.



The old shorthand "ConcurrentHashMap always uses a fixed set of segment locks" is not a sound description of current OpenJDK implementations. The useful interview explanation is finer-grained coordination, atomic per-key contracts and generally nonblocking retrievals, with implementation-specific CAS/bin locking and resize coordination underneath. Do not claim the entire structure is lock-free or that every read yields the latest wall-clock state of the whole map.

[VERIFY: ConcurrentHashMap implementation details and concurrent-collection iterator/aggregate guarantees must be checked against the exact JDK API and source tag. Per-key atomicity is an API-level reasoning tool; segment/bin layouts and resize algorithms are not universal concurrent-map contracts.]

Keep computation callbacks short. Blocking on a remote service inside `computeIfAbsent` can hold up conflicting updates and entangle lock/order dependencies. The callback is not a durable exactly-once event handler. A mapping can be removed, a computation can fail, or another process can make its own call. Cache-in-flight-future designs also need failure eviction, cancellation and bounded retention, not just a clever map method.

Collection	Use when	Avoid when
<code>ConcurrentHashMap</code>	Concurrent lookup and atomic per-key mutations	Multi-key transactions, null key/value semantics, or mutable value safety are assumed
<code>ConcurrentSkipListMap</code>	Concurrent sorted/range access is required	Unordered lookup is sufficient and the extra ordering cost is unjustified
<code>CopyOnWriteArrayList</code>	Small read-mostly listener/config lists benefit from snapshot iteration	Frequent writes or large arrays cause copying and retained snapshots
Bounded <code>BlockingQueue</code>	Producers/consumers need coordination and explicit waiting capacity	Capacity is unbounded, or full-queue handling ignores request deadlines
<code>ConcurrentLinkedQueue</code>	Nonblocking unbounded queue semantics fit an independently bounded workload	The queue is expected to enforce backpressure on its own
Synchronized collection wrapper	A simple single-guard policy fits	Compound iteration/multiple operations are assumed atomic without taking the correct guard

`ConcurrentHashMap` iterators are weakly consistent, while `CopyOnWriteArrayList` iterators see a snapshot of its backing array at creation. Neither means mutable elements are frozen. A reader retaining a copy-on-write iterator can also retain an old array and its references. 01 explains ownership and memory retention.

Queue semantics matter: `offer` can fail immediately, `put` can wait, and a timed `offer` can fail after waiting. An executor typically uses `offer` for admission; naming its queue `BlockingQueue` does not mean submit blocks until room appears. Account for the waiting producer population as well as queue contents. A blocking queue establishes a handoff for preceding writes, not protection against a producer continuing to mutate an element after enqueueing it.

Production incident: two workers independently check that no job is marked running, then each puts a marker. Using a concurrent map protected each individual operation but did not protect the decision. Replace the compound sequence with an appropriate atomic local operation, and still enforce durable job ownership across replicas at the [database](#) or [distributed coordination](#) boundary.

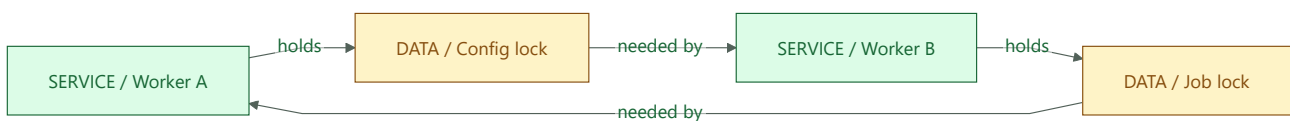
Interview checks: basic: concurrent container does not imply concurrent elements. Internals: explain atomic per-key methods and weakly consistent traversal. Debug: look for separate get/check/put sequences. Scenario: an in-memory duplicate-suppression map is not cross-Pod

idempotency.

9. Deadlock, Starvation and Cancellation

Deadlock is a cycle of dependencies where the participants cannot progress. The classic conditions include exclusive resources, holding while waiting, lack of forced resource preemption and circular wait. Resources can be monitors, explicit locks, executor slots, connection-pool entries or application promises. A JVM monitor detector does not necessarily know about the whole business dependency graph.

Diagram 02.9



Trace a Lock-Order Cycle

Worker A acquires the configuration lock, then requests the job lock. Worker B acquired the job lock and requests the configuration lock. If each retains its first lock while awaiting the second, neither can release the resource the other needs through normal completion. A consistent global acquisition order breaks this cycle when all paths obey it, including callbacks and exceptional paths. Lock reentrancy does not help because ownership is split across different threads.

Thread-Pool Starvation Deadlock

An executor has all its workers occupied by parent tasks. Each parent submits a child to the same executor and waits for the child's result. The children remain queued because no worker is free. There may be no monitor cycle at all. A larger pool can defer the symptom until concurrency grows; it does not repair the dependency policy. Compose instead of blocking, execute suitable child work directly, or use deliberately separate resources while still bounding admission.

Starvation means a participant cannot make progress because other work repeatedly wins resources. **Livelock** means participants keep acting but fail to advance, for example repeatedly backing off and retrying in synchrony. Excessively aggressive CAS retries, unfair acquisition and competing retry loops can create operational symptoms without a deadlock cycle.

Technique	Use when	Avoid when
Consistent resource order	Multiple exclusive resources must be acquired	Hidden callbacks acquire resources in a conflicting order
Short lock scope and no external calls under lock	Local data mutation is separable from slow work	Releasing the lock breaks an invariant without a reservation/version recheck
Timed/interruptible acquisition	You can abort/release/retry with a bounded policy	Timeout simply starts another retry while held resources leak
Nonblocking stage composition	Dependent work otherwise exhausts worker slots	Computation still blocks internally on an exhausted external pool
Bulkheads and admission bounds	One slow dependency must not occupy all service capacity	The alternative queue or waiting population is unbounded

Interruption Is a Request, Not a Kill Signal

`Thread.interrupt` sets interrupt state or triggers documented interruption behavior in certain blocking operations. It does not safely stop arbitrary code at any instruction. Code should propagate `InterruptedException` when possible; when translating or terminating at a boundary, preserve the intended cancellation signal, often by restoring interrupt status. Do not catch interruption, log it and continue indefinitely.

`Thread.interrupted` checks and clears the current thread's status; `isInterrupted` checks without clearing. Some waits throw `InterruptedException` and clear status. Acquiring an intrinsic monitor is not an interruptible lock acquisition API; use an appropriate explicit lock if abortable acquisition is required. Socket/driver behavior is API-specific, and remote side effects can survive local cancellation.

`ConcurrencyLab` interrupts a virtual-thread semaphore waiter and verifies that the path ends without creating a permit. Its latch signals entry into the path, not proof that the OS observed it blocked; interrupt-before-acquire and interrupt-during-acquire are both legitimate schedules. No lab intentionally leaves a permanent deadlock running.

Diagnose Before Changing Lock Types

Capture a permitted thread dump and relevant traces/queue metrics, then identify what each waiting task owns and awaits. Java `BLOCKED` usually describes monitor acquisition; parked explicit-lock waits often appear `WAITING` or `TIMED_WAITING`. `RUNNABLE` does not alone prove CPU consumption: native/socket states can require additional evidence. Match repeated observations with CPU, lock contention and downstream metrics rather than diagnosing from one label.

[VERIFY: deadlock detection and thread-dump visibility differ for platform and virtual threads and across JDK tools/releases. Check ThreadMXBean, jcmd thread-dump documentation and JFR event support for the deployed version; an empty monitor-deadlock report does not prove the absence of pool, resource or virtual-thread deadlocks.]

Production incident: shutdown waits forever because a task swallowed interruption while holding a permit. Trace task ownership, cancellation response and permit release; repair the protocol and add a finite shutdown test. Do not use unsafe forced thread termination as a substitute. Operational capture and incident response belong in [18](#); dump/profiler interpretation in [23](#).

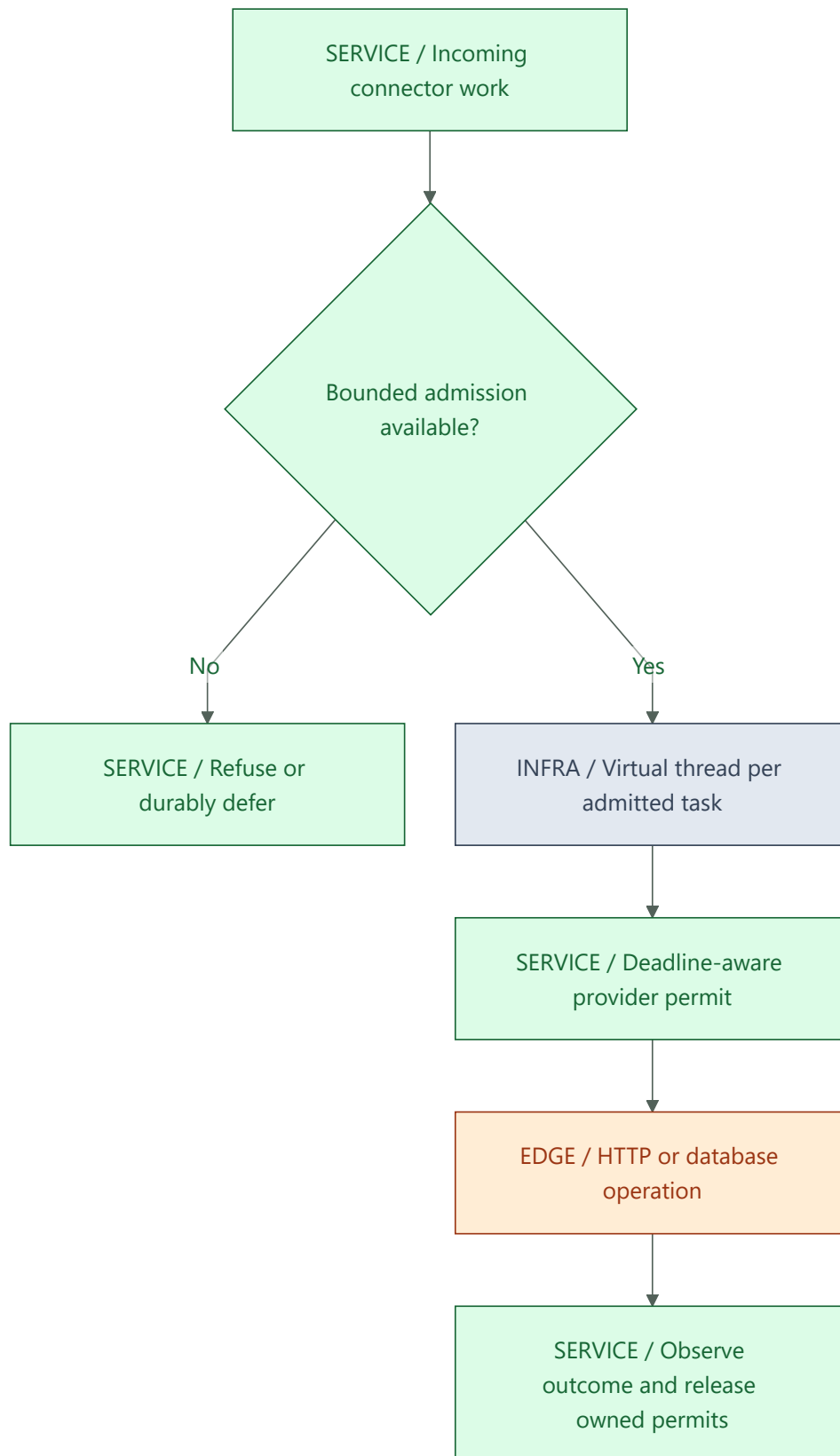
Interview checks: basic: low CPU does not rule out a severe concurrency incident. Internals: draw held-resource/wait dependencies. Debug: examine pool starvation even with no monitor cycle. Scenario: timeouts need cleanup and terminal outcomes, not an unlimited retry loop.

10. Virtual Threads and Bounded Connector Work

[01](#) explains virtual threads, carriers and Java 21 versus Java 24/25 pinning. Here the question is policy: how much work may enter, which scarce resource does it consume, who cancels it and what result survives a process failure? Moving the same unbounded workload to cheaper threads is not a capacity plan.

Use one virtual thread per task where blocking-style code and libraries fit. Do not pool virtual threads to simulate a fixed worker pool. Apply admission limits and resource-specific concurrency limits explicitly. A semaphore can bound active provider calls, while a separate bounded admission boundary controls how many pending requests, task objects, payloads and waiters exist.

Diagram 02.10



The graph's cleanup path must also cover failed permit acquisition and rejected task creation without releasing unowned permits or leaking already acquired admission. Each permit should have an explicit owner and release point. A concurrency limit of ten per Pod becomes up to forty active calls across four Pods if all are independently saturated; this arithmetic is an assumption-based illustration, not a provider quota claim. Distributed rate/quota enforcement is a separate design in [10](#) and [22](#).

Deadlines should cover admission waiting, queue waiting, resource acquisition and remote work, not restart the full budget at every stage. Use monotonic elapsed-time reasoning within one process; wall-clock timestamps have a separate role for audit and cross-system time. Every retry consumes the same user/business deadline unless the contract explicitly creates a new attempt.

Context and Task Lifetime

Virtual thread-local values belong to that logical thread, not its transient carrier. Creating a new task does not inherently propagate every framework context correctly. A database connection per virtual thread can overwhelm a finite pool, and a large thread-local cache multiplied by many tasks can exhaust heap. Pass tenant/job identity explicitly or use a supported context mechanism; preserve authorization and log-redaction rules from [08](#).

Virtual threads are daemon threads; do not let required work depend on the process staying alive after its other lifecycle owners finish. Observe task completion through owned executors/futures or another managed scope. Executor close can wait for work and is not a hard shutdown deadline. Structured concurrency aims to make related task lifetimes explicit, but API maturity/version support must not be inferred from virtual threads being final.

[VERIFY: Java 21 virtual-thread contracts, the Java 24 monitor-pinning change, current structured-concurrency/scoped-value API status, and framework context support must be checked against JEP 444, JEP 491 and the selected release/framework documentation. These labs use final Java 21 APIs only and do not test pinning or preview structured-concurrency APIs.]

Workload decision	Use when	Avoid when
Virtual-thread-per-task blocking flow	Independent waiting dominates and all scarce resources are bounded	CPU work is expected to become faster merely by changing the thread type
Bounded CPU executor	Parsing/compression/transformation needs controlled CPU parallelism	Pool tasks synchronously wait for children queued to the same exhausted pool
End-to-end nonblocking/reactive flow	Backpressure and streaming composition are real requirements	Blocking libraries run on the event loop or cancellation is assumed to undo remote effects

Workload decision	Use when	Avoid when
Durable queue and worker recovery	Work must outlive browser disconnects or Pod restarts	An in-memory future is used as the only record of accepted work

Production case: virtual-thread count and heap rise while provider throughput stays flat. Look at admission, timeout budgets, permit wait, pool wait, retries and retained payload size. Raising a carrier setting is not the default answer. A virtual-thread application has the same business invariants and distributed uncertainty as a platform-thread application.

Interview checks: basic: cheap threads do not make unlimited work safe. Internals: a permit limits a named resource, not all task memory. Debug: inspect waiters outside the queue. Scenario: keep durable job acceptance separate from the lifetime of an in-memory task.

11. Test a Contract, Not a Lucky Schedule

The complete sources are SnapshotLab.java and ConcurrencyLab.java under `code/02-concurrency/`. The Windows PowerShell wrapper run.ps1 accepts an already installed JDK via JdkHome, compiles with the Java 21 release target and records actual output in execution.json. Its executed preflight currently says NOT EXECUTED. A publication check verifies the displayed SnapshotLab matches its file; it does not compile Java.

Probe in source	Controlled condition and assertion	Limit of the evidence
Snapshot publication	Immutable replacement; reader takes one reference; old snapshot unchanged	Passing once is not proof of every JMM execution or a reliable way to expose a missing volatile
Split volatile increment	Barrier places both reads before either write; checks one retained increment	Constructed lost-update schedule, not a probabilistic ++ benchmark
Atomic counter	Two finite tasks increment and results are observed	Supports one operation contract, not arbitrary multi-variable atomicity
Monitor reservation	Two callers compete for one unit; exactly one succeeds	Local instance only, not replica-wide quota enforcement
Queue/grow/reject	Latches keep two workers occupied and one task queued; fourth is rejected	Safety deadlines can fail on a severely delayed machine; no performance conclusion
Futures and map merge	Explicit executor, combination/composition/recovery, per-key atomic aggregation	No real network cancellation, transaction propagation or whole-map snapshot proof
Interrupted permit wait	Task observes interruption and exits; permit count stays unchanged	Does not prove every external blocking API responds to interrupts

Use latches/barriers to arrange the interleaving relevant to the bug. Observe every Future, bound waits and release blocked workers in finally so a failed assertion does not strand the suite. Thread.sleep guesses at timing, and a test that succeeds only on the author's laptop is weak evidence. A barrier/latch itself creates ordering, so do not accidentally use it to hide the unsynchronized handoff you meant to investigate.

For subtle JMM outcomes, use an appropriate stress/litmus framework such as jcstress when tooling is available, alongside a specification-based argument. For throughput/latency use workload-driven measurement and JMH where suitable, not a stopwatch around a single run. Neither framework was downloaded or executed here. A source review and a test plan cannot be reported as a Java test pass.

One Interview Scenario, End to End

Prompt: a tenant starts many sync jobs; provider calls become slow; progress counters disagree; shutdown fails to drain. Begin with three separate hypotheses instead of changing every thread pool.

- **Inconsistent progress:** locate the invariant. Several independent atomic counters may need a coherent snapshot, while durable progress must come from committed business state.
- **Growing latency:** inspect admission/backlog, worker blocking, permit/connection wait and retry load. A bounded executor is insufficient if tasks fan out into an unbounded population of futures or virtual threads.
- **Failed shutdown:** stop accepting new work, preserve durable ownership of queued jobs, request cancellation, release owned resources, and observe completion within the declared shutdown budget. Follow up on tasks that ignored interruption.

Then explain which fix is local Java coordination and which belongs to [messaging](#), [database transactions](#), [resilience](#), or [Kubernetes termination](#). A thread-safe component can still participate in an incorrect distributed workflow.

Explained-Here Index and Sources

Concept	Explanation
JMM, visibility, ordering, happens-before	Memory contracts
Safe publication and immutable replacement	Complete snapshot example
synchronized, volatile, monitor ownership, wait/notify	Invariant scope
Atoms, CAS, ABA, LongAdder	Atomic transitions
Locks, conditions, AQS, semaphore, latch and barrier	Coordination

Concept	Explanation
ThreadPoolExecutor, queues, rejection, shutdown	Executor lifecycle
CompletableFuture, composition, timeout and cancellation	Completion graphs
Concurrent collections and handoff	Collection boundaries
Deadlock, starvation, interruption and diagnosis	Progress failures
Virtual threads, admission and context	Bounded connector work
Deterministic probes and execution status	Testing contracts

Primary pages consulted for this chapter: [ThreadPoolExecutor Java 21 API](#) and [CompletableFuture Java 21 API](#). The JLS and concurrent-package references in the JMM section are authoritative reading references, not claims of new runtime evidence. [JEP 444](#) and [JEP 491](#) were consulted during Chapter 01 and are reused with their version boundaries intact. Exact vendor implementation and unexecuted lab behavior remain explicit verification items.

Related Chapters

Start from [00's request journey](#) and [01's JVM model](#). Apply task/context behavior in [03 Spring](#), [04 JPA](#) and [05 reactive/real-time APIs](#). Keep local synchronization distinct from [06 database isolation](#), [07 delivery semantics](#) and [22 distributed locks](#). [21 Network/OS](#) explains blocking at the system boundary; [19 Testing](#) and [23 JVM performance](#) deepen validation. Every related destination has a generated return link.

RELATED CHAPTERS / RECIPROCAL LINKS

00 The System Before the Details	27 Fintech and Payments Engineering
01 Core Java and the JVM	10 System Design Foundations
06 Databases and Durable State	08 Security, Identity and Data Protection
07 Messaging, Batch and Streams	15 Kubernetes and Production Workloads
22 Distributed Systems Theory	21 Networking and Operating Systems
03 Spring and Spring Boot Internals	19 Testing and Quality
23 JVM Performance Engineering	09 The Browser and the Java Backend
04 Hibernate and JPA	25 Architecture Styles and Domain Design
18 Observability and Reliability	11 Object Design, Patterns and LLD
05 APIs and Real-Time Communication	24 IntegrationHub in Motion
	20 Appendix: Interview Toolkit and Index

One-Page Cheat Sheet

Start with the invariant: shared mutable state needs a documented owner or coordination rule. Visibility is not atomicity; thread safety is not liveness; local atomicity is not durable/distributed correctness.

Happens-before: program order plus a matching synchronization edge plus transitivity. Same-monitor unlock/lock, volatile write/read, start/join and documented executor/queue/latch handoffs are useful edges. Sleep, timestamps and "the other thread probably finished" are not proofs.

Choose deliberately: confinement first; immutable snapshot for read-mostly replacement; synchronized for a small multi-field invariant; atomics for suitable single-value transitions; explicit locks when timed/interruptible/condition policies are required. LongAdder is for aggregate statistics, not exact quotas.

Executor path: core worker -> queue -> growth toward maximum -> rejection. An unbounded queue can prevent growth beyond core while consuming heap. Observe submit results. CallerRuns can run on a dangerous caller and discards after shutdown. Shutdown is a lifecycle protocol, not forced termination.

Futures: apply transforms; compose flattens a dependency; combine joins independent results. Specify execution boundaries. allOf is not typed result collection or automatic sibling cancellation. A timeout/cancelled future does not prove its task or remote effect stopped.

Collections and waits: per-key atomic methods do not form multi-key transactions. Concurrent containers do not protect mutable values. Await the predicate in a loop; release only acquired permits; respect interruption. Draw resource cycles, including worker slots and database connections.

Virtual threads: one per task where appropriate; bound admission and scarce resources, not a virtual-thread pool. Context and lifetime need owners. Java 21 pinning advice is version-specific. Both chapter labs are NOT EXECUTED in this environment.

Interview Corner

Basic: What Is the Difference Between Visibility and Atomicity?

Visibility concerns observing writes across threads under the memory contract. Atomicity concerns an operation taking effect without relevant interleaving. Volatile can order counter reads/writes while a read-modify-write still loses an update; a suitable atomic increment or shared lock protects that operation.

Internals: Show a Happens-Before Chain for a Published Snapshot.

Construct and validate the snapshot, then assign its volatile reference. The reader observes it through that same volatile field, then reads the snapshot's fields. Program-order and volatile synchronization edges compose transitively. Do not mutate the snapshot afterward without a new contract.

Trace/Debug: The Code Works After Adding Logging. Is It Fixed?

No. Logging can change schedules and add synchronization. Identify the actual shared-variable accesses and missing ordering/invariant. Build a contract-based repair and an appropriate test instead of keeping logging as an accidental memory barrier.

Scenario: Can final Fields Replace Safe Publication Everywhere?

No. Correct construction gives special final-field guarantees, not a universal replacement-update or later-mutation protocol. Avoid constructor escape and choose a clear handoff. Final references do not deeply freeze mutable collections or signal that a refreshed reference will be observed promptly.

Basic: Which Object Does synchronized Lock?

An instance method locks its receiver; a static method locks its Class object; a synchronized block locks its specified object. Different instances or different guard objects do not protect the same invariant merely because methods have the same name.

Internals: What Happens During wait and notify?

wait requires monitor ownership, releases that monitor while waiting, and reacquires it before continuation. notify makes a waiter eligible but does not release the notifier's monitor. Recheck the predicate in a loop; notification is not guaranteed ownership of an available item.

Trace/Debug: Why Did a Volatile Counter Lose an Increment?

The compound operation read the old value, computed a new one, then wrote it. Another thread could do the same before either write. Volatile orders accesses but does not make the compound action indivisible. Use the atomic operation matching the invariant.

Scenario: Is AtomicInteger Enough for a Tenant Quota?

Only if the full local decision can be expressed atomically, such as a valid CAS reservation. Separate get/check/decrement is racy; several Pods still have separate counters. A cluster-wide quota requires an appropriate shared authority and failure policy.

Basic: AtomicLong or LongAdder?

AtomicLong supports exact single-value atomic operations. LongAdder targets contended aggregate updates with a sum that is not an atomic concurrent snapshot. Use the latter for suitable metrics, not sequence generation or an exact capacity decision.

Internals: Why Must an Atomic Update Function Avoid Side Effects?

It may be evaluated multiple times while the operation retries contention. A provider call or event publish inside it can repeat even if one final local update succeeds. Keep candidate computation pure and coordinate external effects with the proper durable protocol.

Trace/Debug: A Fair Lock Still Has Slow Requests. Why?

Acquisition policy cannot guarantee OS scheduling or bound the work while the lock is held. Some acquisition methods have fairness exceptions. Inspect hold time, queueing, external calls under lock and workload imbalance before assuming the flag guarantees latency.

Scenario: Why Not Always Use a Read-Write Lock?

It adds coordination overhead and writer/read interactions that may outweigh benefits for short operations or frequent writes. Measure a suitable read-heavy workload, and do not attempt a read-to-write upgrade while retaining incompatible read ownership. Immutable snapshots may simplify the design.

Basic: Why Does maximumPoolSize Not Increase My Throughput?

With a queue that keeps accepting work after core workers are present, the usual path queues rather than creating noncore workers. Check core size and queue policy, then the actual bottleneck; increasing workers can still overwhelm downstream resources.

Internals: Trace core=1, maximum=2, Queue Capacity=1.

Hold the first task running; second queues; third cannot queue and starts a second worker if creation succeeds; hold it too; fourth is rejected under AbortPolicy. When workers can finish concurrently the observed path can differ, which is why the lab explicitly holds them.

Trace/Debug: Why Did submit Not Log the Task Exception?

Its Future-bearing task captured the exception. Observe the Future or a completion/error-reporting policy; an uncaught-exception handler or afterExecute Throwable alone may not see captured failures. Preserve the underlying cause and avoid silently dropping the result.

Scenario: Is CallerRunsPolicy Always Safe Backpressure?

No. It runs the task on the submitting thread during saturation, which may be an event loop or a lock-owning caller. After shutdown it discards instead. Decide whether caller execution is safe and whether every accepted/refused task has an observable outcome.

Basic: thenCompose Versus thenCombine?

Compose starts/joins a stage that depends on the earlier result and flattens the nesting. Combine waits for two independent successful results and derives a value. Choose from the dependency graph, not which method name sounds more asynchronous.

Internals: Does supplyAsync with My Executor Bind Later Stages?

No. Later non-async stages may execute inline on completion-related threads; later async stages without explicit executors use their default facility. Provide the intended executor at each relevant boundary and do not assume context propagates automatically.

Trace/Debug: orTimeout Fired, but the Database Query Continued. Is That Possible?

Yes. It completed the future exceptionally; it did not necessarily stop the computation or query. Trace the underlying task handle, driver cancellation and deadlines separately. A remote write may also have happened despite the local timeout.

Scenario: Does allOf Implement Fail-Fast Sibling Cancellation?

No. It represents completion of all the supplied futures and does not automatically cancel the others on failure. Define the group policy, collect/translate results deliberately and own loser/failed-task cleanup. Do not confuse a completion graph with a managed task lifetime.

Basic: Is a ConcurrentHashMap of ArrayLists Fully Thread-Safe?

No. Map operations protect their mapping contracts, not arbitrary concurrent mutation of stored lists. Use immutable values, a suitable value-level concurrent structure, or a shared guard that matches the whole invariant.

Internals: Is computeIfAbsent a Durable Exactly-Once Function?

No. Its coordination is local and tied to a mapping operation under documented conditions. Failure, removal, retries or another process can cause computation again. Do not place non-idempotent external effects in it and call that distributed deduplication.

Trace/Debug: No Monitor Deadlock Was Found, Yet Every Request Is Stuck.

Inspect worker slots, child futures, connection pools and permits. Parents blocking on children queued to the same full executor can deadlock without a monitor cycle. Virtual-thread diagnostic coverage is also version/tool-specific. Build the full wait-for graph.

Scenario: Can shutdownNow Guarantee Termination?

No. It attempts to interrupt running work and removes queued tasks; uncooperative work can continue. Inspect the termination result, preserve ownership of never-started jobs, and use documented resource cancellation. Forced unsafe thread stopping is not a correct fallback.

Basic: Do Virtual Threads Remove the Need for Admission Limits?

No. Each task retains objects and competes for finite external capacity. A million cheap waiters can still exhaust memory or miss every useful deadline. Bound the entire admitted population and each scarce resource separately.

Internals: Is a Semaphore a Rate Limiter?

It limits concurrent permit holders, not a count per time interval. Faster calls can produce a higher rate with the same concurrency. Provider rate limits need a time-aware policy; replica-wide limits additionally need shared coordination or partitioned budgets.

Trace/Debug: Cancellation Increased availablePermits Unexpectedly. Why?

Check whether cleanup released a permit after an interrupted or timed-out acquire that never succeeded. Track acquisition success/ownership and release exactly once. Cancellation is a path through the same resource accounting, not permission to reset counts blindly.

Scenario: How Would You Prove This Code Is Safe?

State the invariant, all access paths, ownership/guard rules, ordering edges, linearization points and failure/cancellation cleanup. Add arranged-interleaving tests and stress tests appropriate to the claim. Passing one run, source review or a clean editor is not a proof of all schedules, and these labs have not yet run locally.

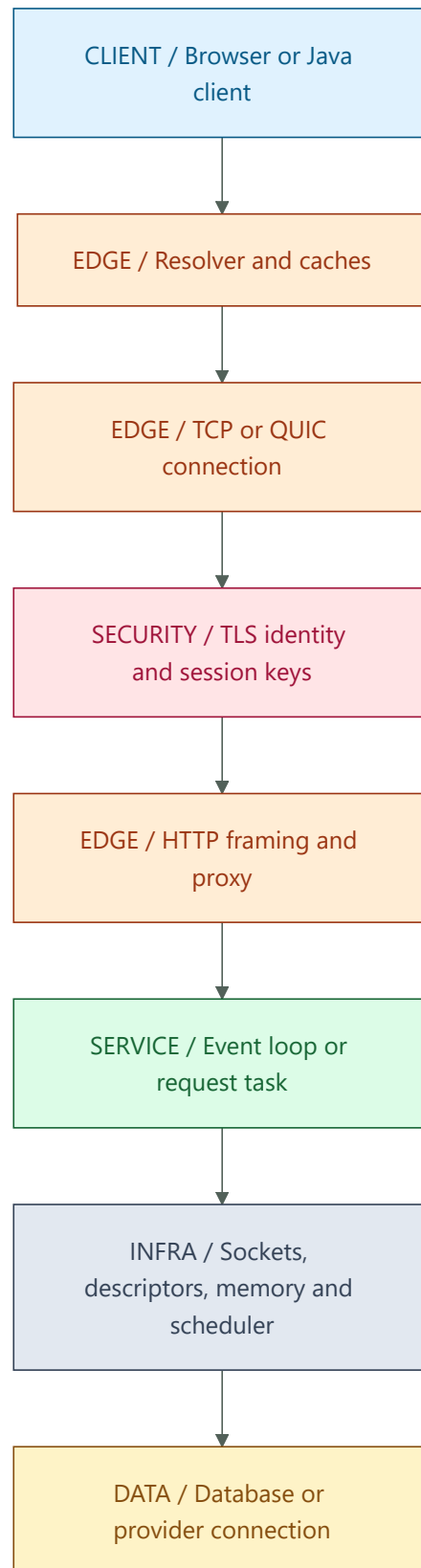
21 / Networking and Operating Systems

A socket is a stream of bytes carried by several independent state machines. DNS chooses an address, routing finds a path, TCP or QUIC manages transport, TLS establishes protected peer communication, and HTTP frames application messages. IntegrationHub is fictional; this chapter follows that request path without treating a timeout as proof that no remote effect occurred.

Version assumptions: Linux networking/OS concepts, Java 21 and modern HTTP/TLS implementations. Spring Boot 4.0.x / Cloud 2025.1.x, PostgreSQL 17, Kafka 4.x and Kubernetes 1.34 remain the guide baseline. The actual workspace is Windows. The saved Node-compatible lab exercised only local stream decoding and loopback HTTP; Linux commands, Java NIO, DNS and TLS experiments were NOT EXECUTED.

Big Picture

Diagram 21.1



What You Will Be Able to Explain

- Trace DNS resolution and caching, including JVM-specific stale-address behavior.
- Explain TCP setup, byte ordering, flow/congestion control, half-close, TIME_WAIT and keep-alive.
- Compare HTTP/1.1, HTTP/2 and HTTP/3 multiplexing and head-of-line blocking.
- Explain TLS 1.2/1.3 handshake differences, certificate verification, SNI and mTLS boundaries.
- Diagnose connect/read/pool timeouts, NAT/port exhaustion and L4/L7 proxy behavior.
- Distinguish processes, threads, virtual memory, page cache, file descriptors and cgroups.
- Explain blocking/nonblocking I/O, readiness APIs, Java NIO/Netty and conditional zero-copy paths.
- Choose a bounded diagnostic command that can distinguish the suspected failure.

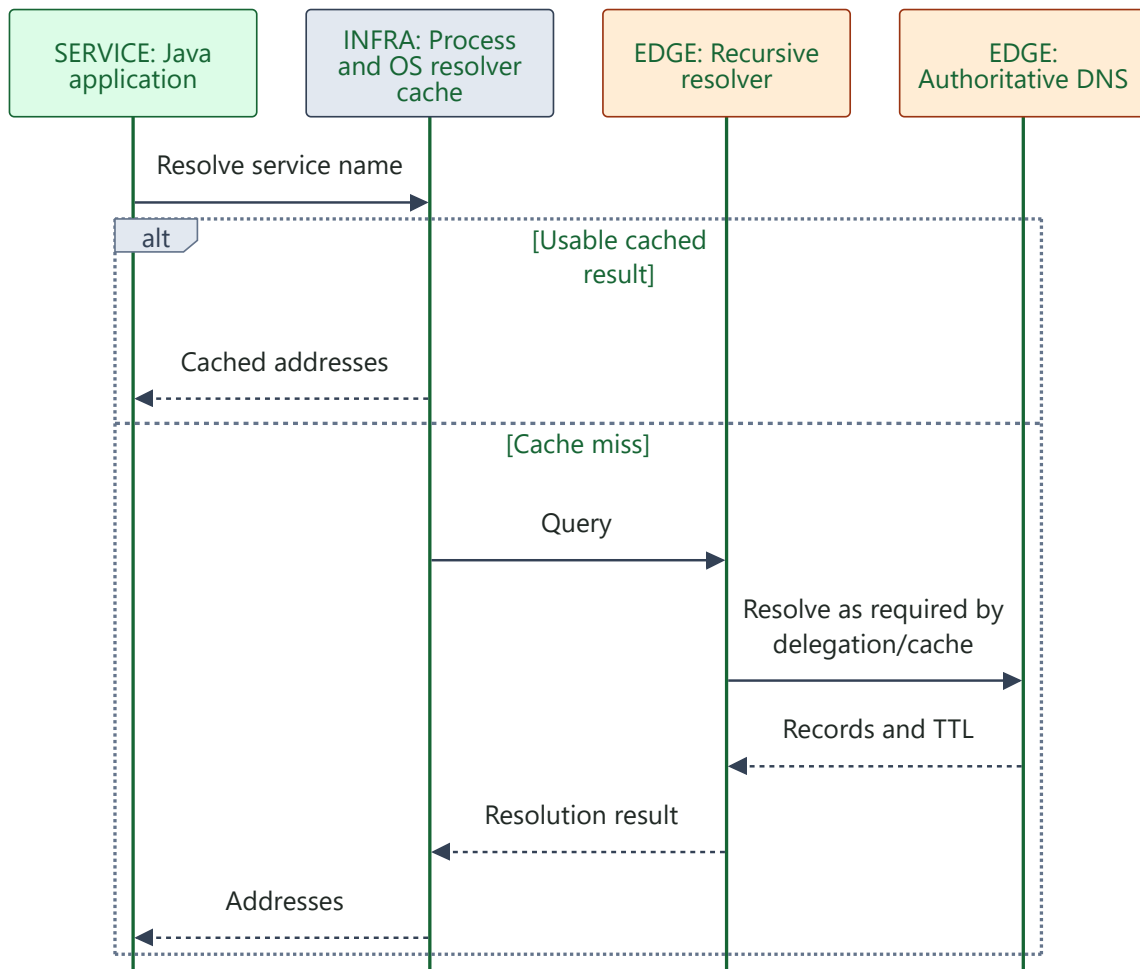
[P] PRODUCTION

Where this lives in IntegrationHub. The 00 request journey uses these layers before reaching 03's controller. 05 depends on incremental streaming through proxies, 08 owns trust and authorization, and 14/15 change namespace and resource boundaries. 18 correlates transport evidence with user impact.

1. DNS, TTL and Address Selection

An application commonly asks a resolver for address records; caches may exist in the process, operating system, local resolver and recursive infrastructure. The recursive resolver follows DNS delegation as needed and returns records subject to TTL and policy. A lookup's returned address is not a live health check, and a cached negative result can keep a newly available name appearing absent.

Diagram 21.2



DNS-based load balancing returns addresses or routing answers; clients decide when to resolve and which address to use. Existing pooled connections can outlive a TTL, so changing DNS does not instantly redirect traffic. Failover also needs to establish which backend is authorized to write; changing an address does not fence an old writer.

Java `InetAddress` caching has runtime/security-property policy, including positive and negative cache behavior. Do not assume a generic `-D` option changes every security property or that OS TTL behavior equals JVM behavior. Inspect the chosen JDK configuration and client connection-pool lifecycle. Resolve once at startup and retaining that address forever can defeat a carefully configured DNS failover.

Resolution policy	Use when	Avoid when
Bounded caching	Reduce resolver latency/load while permitting change	TTL is assumed to close existing connections
Service discovery with explicit endpoints	Application policy needs health/metadata beyond DNS	Discovery metadata is assumed to route requests itself

Resolution policy	Use when	Avoid when
Negative caching	Repeated absent names should not overload DNS	Long negative lifetimes hide newly created services

[VERIFY: verify JVM positive/negative DNS cache security properties, resolver behavior, client address selection and pool refresh against the selected JDK/OS/client. No DNS failover or cache experiment ran.]

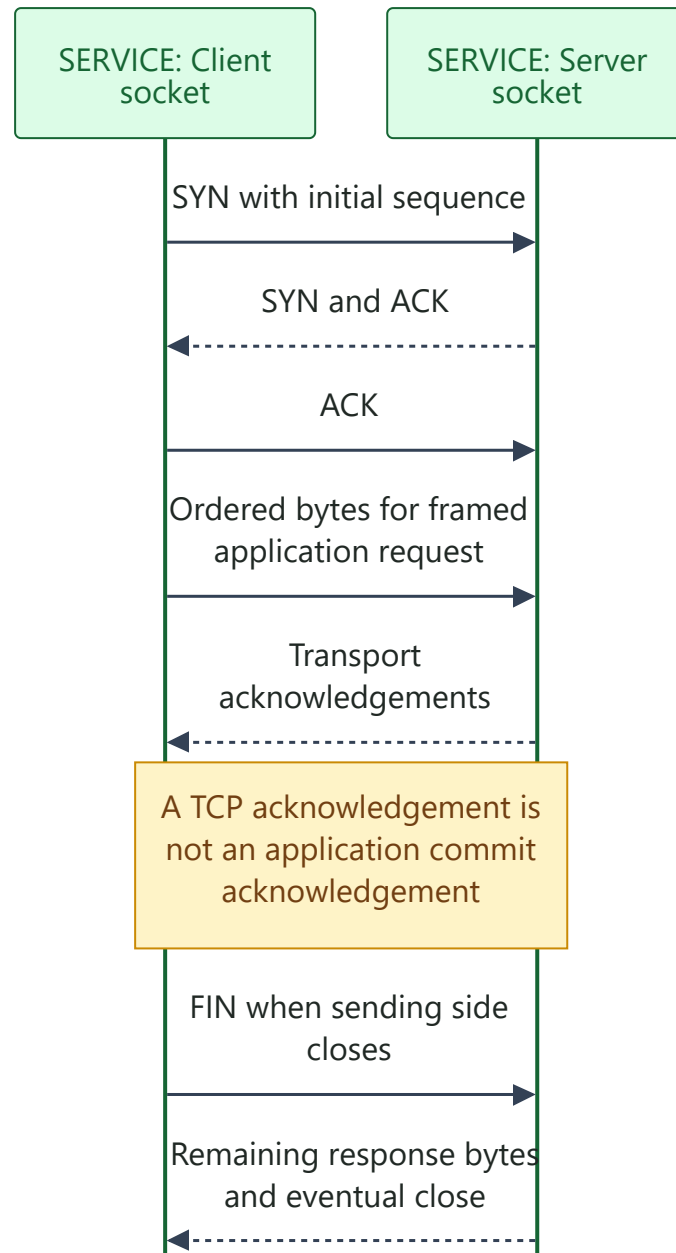
Interview checks: basic: DNS returns naming information, not guaranteed health. Internals: multiple caches and existing connections affect change propagation. Debug: compare process resolution and actual remote socket address. Scenario: coordinate DNS with connection and write-authority policy.

2. TCP Setup, Byte Streams and Close

TCP provides an ordered reliable byte stream between endpoints under its failure model. A connection is identified by protocol and endpoint addresses/ports; the familiar handshake exchanges SYN, SYN-ACK and ACK to establish initial sequence state. Connect success establishes transport readiness, not authentication, HTTP validity or successful business processing.

Application write calls do not define read or packet boundaries. Two writes can be read together, one write can require several reads, and a multibyte character can cross a buffer boundary. Frame messages using the actual protocol or a maintained parser. Do not parse JSON, HTTP or SSE by assuming each received chunk is one complete message.

Diagram 21.3



Sequence numbers and acknowledgements track bytes. Retransmission can recover missing segments while preserving the stream abstraction. A successful write may mean bytes entered a local buffer, not that the peer application consumed them. A timeout after sending therefore leaves a remote mutation's outcome unknown. [05](#) explains the idempotency response.

TCP supports closing one direction before the other. End-of-stream is not the same as a reset or a zero-byte nonblocking read. `TIME_WAIT` commonly occurs on the active closer and protects connection reuse against delayed segments under TCP's rules. It is not simply a leaked application socket, and eliminating it indiscriminately can break assumptions. High connection churn can still consume practical port/state capacity and deserves investigation.

Interview checks: basic: TCP orders bytes, not messages. Internals: local write and remote application completion differ. Debug: identify reset, EOF and timeout separately. Scenario: preserve request identity when transport failure leaves the effect uncertain.

3. Flow Control, Congestion and Keep-Alive

Flow control protects the receiving endpoint from more data than its advertised buffer/window can accept. Congestion control adapts sending to conditions along the network path. Receiver window and congestion window are different constraints; a slow-reading application can reduce progress even on a healthy uncongested path.

TCP algorithms increase and reduce outstanding traffic based on acknowledgements and congestion signals under the selected implementation. Slow start, loss recovery, retransmission timeouts and congestion avoidance influence throughput and latency. The details vary by algorithm and OS; do not infer one universal ramp-up duration or that every delay is packet loss.

TCP keepalive, HTTP connection reuse and application heartbeats are different mechanisms. TCP probes can help detect some dead idle peers under configured timing; HTTP keep-alive commonly means reusing a connection; an SSE comment heartbeat can keep an intermediary from treating the application stream as idle. None is an instant perfect failure detector.

Mechanism	Use when	Avoid when
Receive flow control	Receiver buffers/application need bounded input	It is confused with end-to-end application demand semantics
Congestion control	Shared paths need adaptive sending	Application throughput is assumed independent of path conditions
Connection reuse	Handshake and port churn should be reduced	Stale pooled connections have no timeout/recovery policy
Application heartbeat	A protocol needs bounded liveness/idle behavior	It is taken as proof a peer can complete business work

[VERIFY: confirm TCP congestion algorithm, keepalive, retransmission, TIME_WAIT and socket-buffer behavior for the actual kernel/network path. The loopback lab is not a packet trace or congestion test.]

Interview checks: basic: flow and congestion control protect different resources. Internals: transport and application liveness differ. Debug: inspect whether the receiver is reading. Scenario: reuse connections with bounded stale-connection recovery.

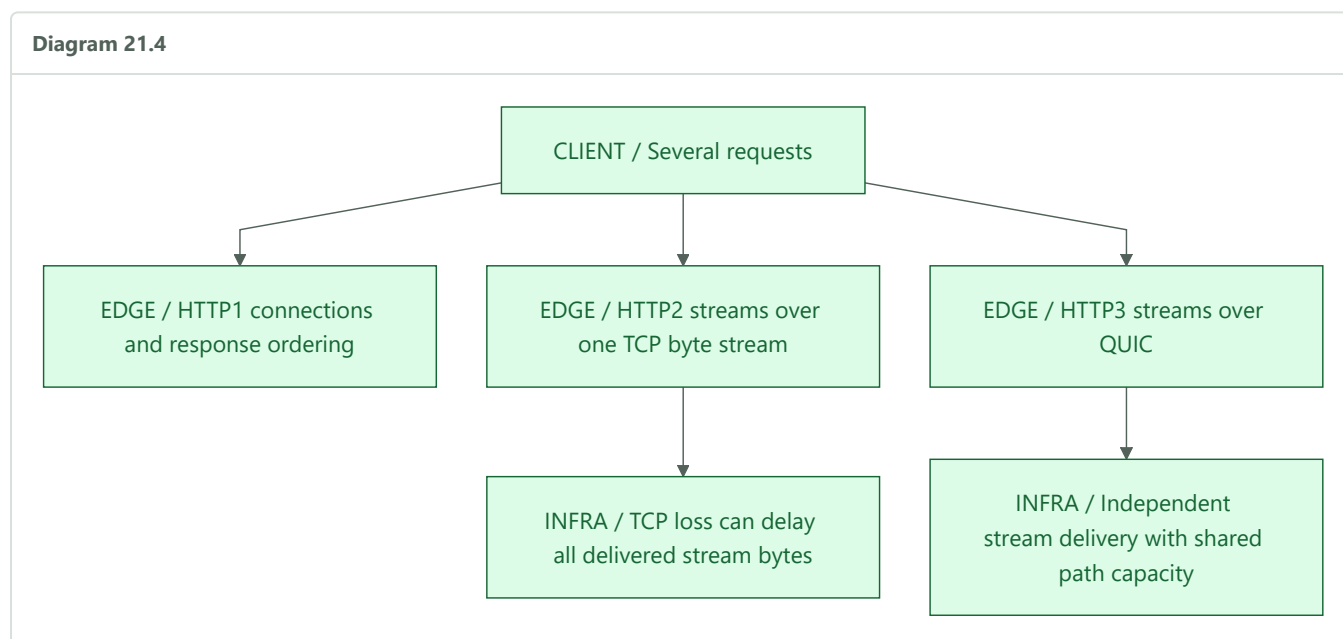
4. HTTP/1.1, HTTP/2 and HTTP/3

HTTP/1.1 commonly reuses connections for multiple requests. Pipelined responses are ordered, which can cause application-level head-of-line blocking; clients often use several connections instead. HTTP message framing uses defined lengths, transfer coding and status/method rules, not connection read boundaries.

HTTP/2 multiplexes independent streams over a connection and has framing and header compression. Responses on different streams can progress independently at the HTTP layer, but they share the underlying ordered TCP byte stream. Loss delaying delivery of required TCP bytes can stall delivery across those streams even though their HTTP messages are independent.

HTTP/3 runs over QUIC, which uses UDP transport while implementing secure reliable streams and congestion control. Independent stream delivery avoids the same TCP-wide transport ordering blockage, but streams still share connection/path resources and can have higher-level dependencies. QUIC is not unreliable fire-and-forget HTTP, and it does not eliminate congestion or every form of head-of-line delay.

Diagram 21.4



Multiplexing changes pool sizing and server concurrency, not business idempotency. One connection can carry many active requests, so connection count is not request count. Flow-control windows, proxy support and backend translation can create bottlenecks. A front door accepting HTTP/2 may forward HTTP/1.1 to the application; inspect each hop rather than assuming one protocol end to end.

Protocol	Use when	Avoid when
HTTP/1.1	Broad compatibility and ordinary request patterns fit	Many concurrent streams are forced through a small serialized connection set
HTTP/2	Multiplexing and supported infrastructure help	TCP-wide transport delays are assumed impossible
HTTP/3	Supported clients/edge and path behavior justify it	UDP restrictions or operational tooling gaps are ignored

[VERIFY: confirm HTTP/2 and HTTP/3 support, QUIC/UDP reachability, flow-control and proxy translation behavior for the deployed clients and edge. No protocol negotiation or packet capture ran.]

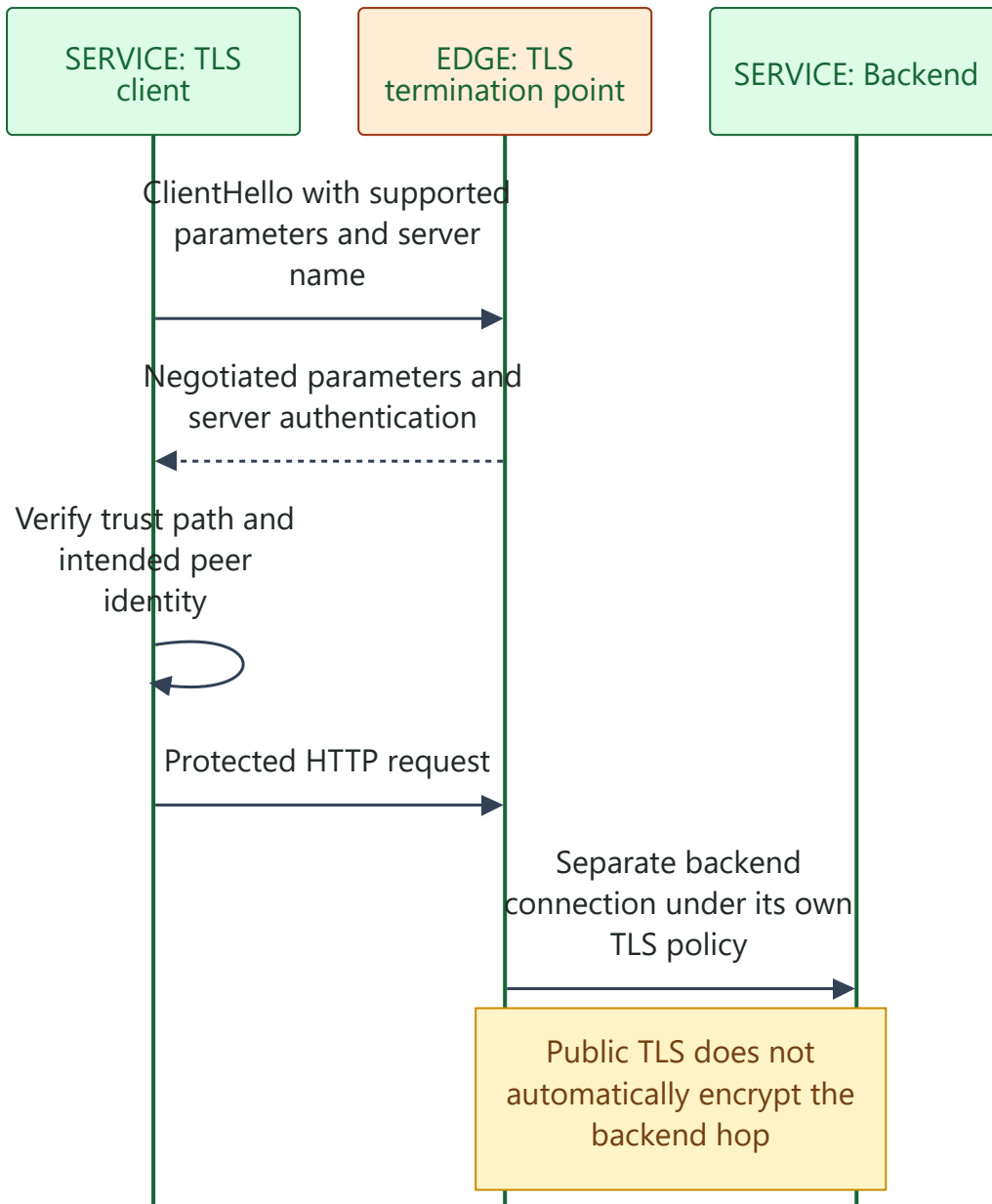
Interview checks: basic: HTTP/2 streams still share TCP. Internals: QUIC supplies reliability rather than discarding it. Debug: inspect protocol per hop. Scenario: size concurrency and deadlines independently of connection count.

5. TLS Handshake, Certificates, SNI and mTLS

TLS protects transport with peer authentication, key establishment and encrypted integrity-protected records under configuration. A typical full TLS 1.3 handshake reduces round-trip work compared with a typical full TLS 1.2 handshake, but DNS, transport setup, resumption and network conditions determine actual latency. Do not quote one total connection time as a protocol constant.

Certificate validation includes a path to an accepted trust anchor, validity, intended identity such as hostname/SAN, and usage/policy checks. A trusted signature alone is not enough if the certificate belongs to a different host. SNI allows the client to indicate the intended hostname during connection setup for routing/certificate selection; it is not tenant authorization.

Diagram 21.5



mTLS adds client certificate authentication under the server's policy. Map that identity to permitted actions and resources; certificate possession does not authorize every tenant operation. Keep trust rotation and application credential policy coordinated but distinct.

Session resumption can reduce handshake work. TLS 1.3 early data/0-RTT has replay considerations; do not allow unsafe state-changing actions solely because the data arrived on an encrypted transport. Use the protocol/application's supported anti-replay and idempotency policy. 08 owns detailed identity and key lifecycle.

[VERIFY: confirm TLS 1.2/1.3 negotiation, resumption/early-data policy, certificate verification, SNI and mTLS behavior for the actual JVM, browser and proxy. No handshake timing or certificate-validation experiment ran.]

Interview checks: basic: encryption and peer identity are separate checks. Internals: termination creates a new trust/transport boundary. Debug: inspect hostname and trust configuration, not only expiry. Scenario: early data needs replay-safe application policy.

6. L4/L7 Proxies, Pools and Timeout Budgets

L4 balancing routes transport connections; L7 reverse proxies understand supported application protocols and may terminate TLS, buffer bodies, retry requests or rewrite headers. These behaviors are operationally significant. A streaming response buffered by a proxy can appear hung even though the backend writes regularly. A proxy retry can repeat a mutation after the backend already committed.

Connection pools reuse expensive connections and bound active resource use under their configuration. Distinguish waiting for a pooled lease from establishing a new connection and from waiting for response data. A client can time out waiting for a pool while no new SYN is ever sent. Increasing connect timeout does not fix that queue.

Read timeout often bounds inactivity between reads, not total operation duration; a peer sending a byte periodically can keep it alive under such a policy. Total deadlines bound the whole logical attempt or request, including retries where designed. Document semantics in the actual client, then divide the parent's remaining budget across stages and cleanup.

Deadline/limit	Use when	Avoid when
Pool acquisition timeout	Waiting callers must be bounded	It is misdiagnosed as remote network failure
Connect timeout	New connection setup must not wait indefinitely	It is assumed to cover a reused connection's response
Read/inactivity timeout	Stalled peers need detection	Slow trickle responses are assumed bounded in total duration
Parent deadline	End-to-end work/retries need a total budget	Independent timeouts sum far beyond user patience

Interview checks: basic: timeout names have different boundaries. Internals: a pool wait can occur before any network attempt. Debug: locate where the timer started. Scenario: retry only repeatable work within one parent budget.

7. NAT, Ports and Connection Churn

Outbound connections use local addresses/ephemeral ports and remote endpoint tuples. NAT and connection-tracking infrastructure maintain mappings under capacity and timeout policies. High churn, many connections to a small destination set, idle retention or leaked sockets can exhaust practical port/state limits. There is no single universal connections-per-host number independent of addresses, destinations, protocol and implementation.

TIME_WAIT accumulation can be a normal consequence of active connection closure, while CLOSE_WAIT often indicates the local application has not completed closing after the peer did. Interpret states with process ownership and traffic pattern, not as automatic proof of a leak. Reuse connections and close resources deliberately; do not disable protocol safeguards as the first response.

A cluster's source-NAT topology can concentrate many Pods behind a smaller egress address set. Adding application replicas can therefore worsen egress pressure rather than increase usable capacity. Inspect node/NAT metrics and actual remote destinations. Server listen backlog, file descriptors and client pools are separate limits that can produce similar connection symptoms.

Interview checks: basic: endpoint tuple and translation state matter. Internals: churn consumes state after calls finish. Debug: separate TIME_WAIT from application close leaks. Scenario: model aggregate egress, not only each Pod's local pool.

8. Processes, Threads and Context Switching

A process owns an address-space/resource context; threads execute within a process and share memory while retaining execution state such as stacks and registers. Shared memory needs synchronization under the language/runtime model. Operating-system thread scheduling does not by itself establish Java happens-before relationships from [02](#).

Context switches move execution among runnable tasks and can involve register/state changes and cache effects. More runnable threads do not create more CPU cores. Oversubscription can increase scheduling overhead and latency; blocked I/O and CPU work need different concurrency reasoning. Java virtual threads are runtime-managed logical threads scheduled onto carriers, not one new OS thread per task.

Priorities, affinity and scheduler implementation can affect behavior but should not be used to paper over a broken application ownership model. A process can be CPU-throttled by cgroups while host-wide CPU appears available. Inspect the namespace/cgroup where the workload actually runs before interpreting host metrics.

Execution model	Use when	Avoid when
Process isolation	Separate address spaces and ownership are needed	Process boundaries are assumed to remove all shared-resource contention
Threads	Shared in-process work benefits from low-overhead coordination	Mutable state is shared without a synchronization contract
Virtual threads	Blocking task structure fits cheap waiting	CPU or downstream resource limits are assumed to disappear

Interview checks: basic: threads share a process's memory. Internals: scheduling is not language-level synchronization. Debug: inspect runnable work and cgroup throttling. Scenario: bound downstream resources even with cheap threads.

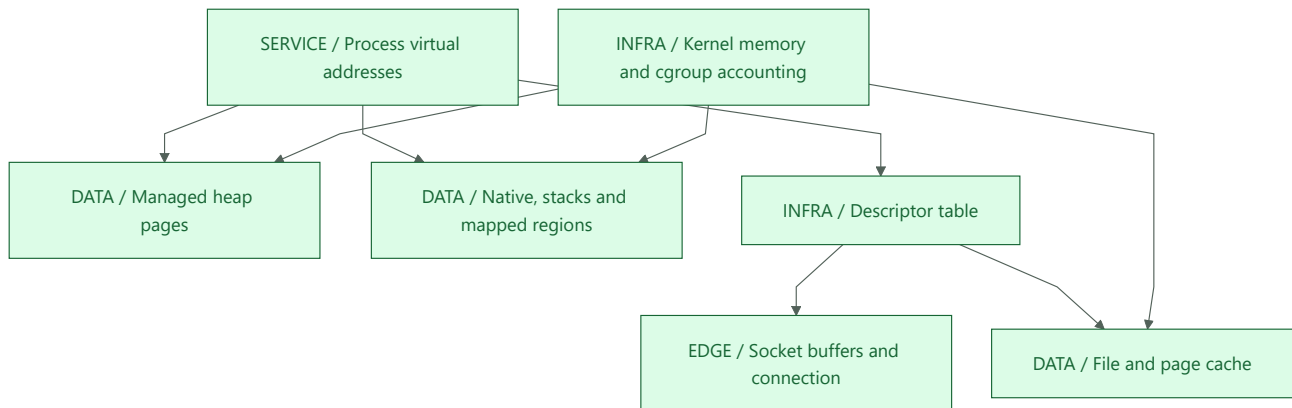
9. Virtual Memory, Page Cache and Descriptors

Virtual memory maps process addresses onto physical memory or other backing under OS policy. A large virtual address reservation is not the same as resident physical memory. Page faults can involve ordinary mapping/zeroing work or storage I/O depending on the situation. RSS, heap committed/used and virtual size answer different questions.

The page cache retains file-backed data to avoid repeated storage access. Warm file reads can be fast without the application holding the entire file in Java heap. Dirty pages eventually need writeback; a successful buffered write does not necessarily imply power-loss durability. Databases implement logging and flush protocols to establish stronger guarantees, as [06](#) explains.

File descriptors identify open resources such as files, sockets and pipes within a process. Descriptor limits and leaks can cause too-many-open-files failures even when CPU and heap are fine. Closing the high-level Java resource must actually release the underlying descriptor; abandoned streams or response bodies can hold both descriptors and pool capacity.

Diagram 21.6



Container namespaces change what processes see; cgroups change resource accounting/control. A container's process view may hide the host PID needed by a host profiler. Limits can differ between host, container and process configuration. 14 and 15 own packaging and scheduling implications.

[VERIFY: confirm Linux virtual/resident memory accounting, page-cache/writeback, descriptor limits and cgroup behavior for the actual kernel/container runtime. No Linux resource or descriptor experiment ran in this Windows workspace.]

Interview checks: basic: virtual size, RSS and heap differ. Internals: cached file data can sit outside Java heap. Debug: inspect open descriptors and unclosed response bodies. Scenario: capture evidence from the correct process/namespaces.

10. Blocking, Readiness, NIO and Netty

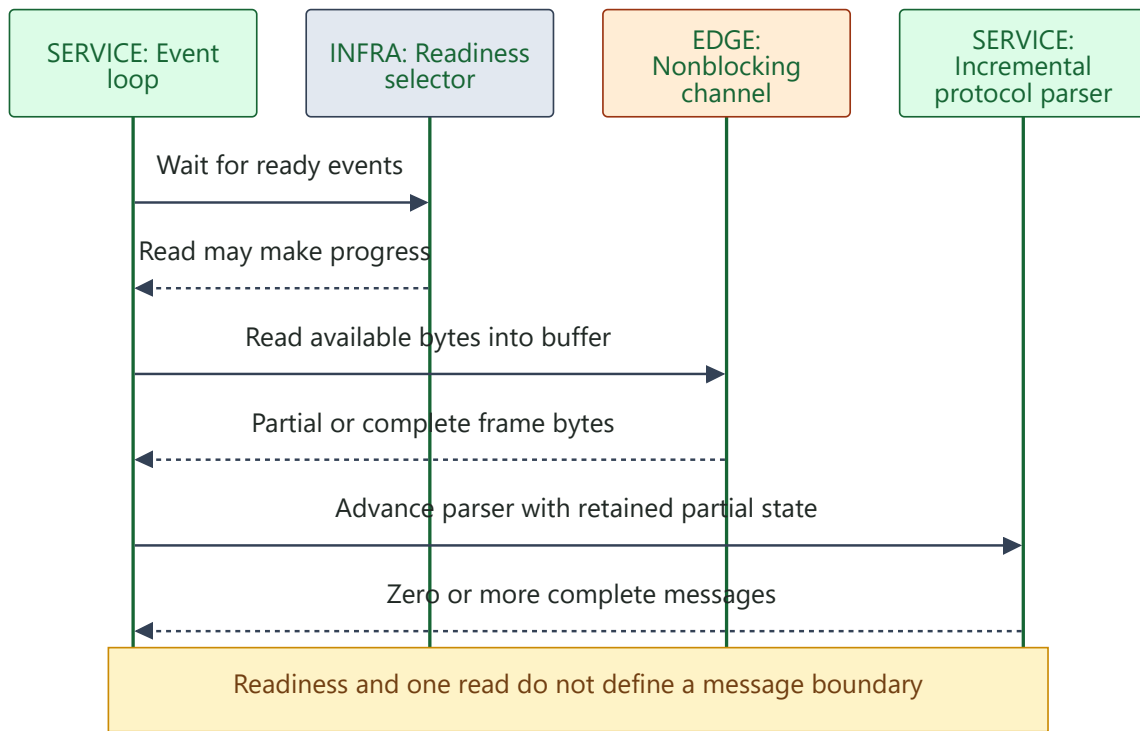
A blocking operation waits until progress or failure under its contract. A nonblocking socket operation can return without transferring all requested bytes. Readiness APIs tell the application which operations may make progress; readiness is not completion of a full message or a guarantee another thread has not consumed the ready data first.

select and poll inspect sets of descriptors with different APIs/limits and scanning behavior. epoll maintains interest in descriptors and returns ready events under configured semantics. It does not run all application handlers in parallel or remove the need for correct buffering, close handling and backpressure. Edge-triggered and level-triggered handling have different obligations; use the framework's documented event model.

Java NIO uses channels, buffers and selectors for supported nonblocking operations. A read can return positive bytes, zero when no progress is made, or end-of-stream according to the channel contract. A write can consume only part of a buffer. ByteBuffer position/limit and

flip/compact control which bytes are produced, consumed or preserved; confusing them can drop partial frames.

Diagram 21.7



Netty combines event loops, channels, pipelines and buffer abstractions. A channel's handlers commonly execute on its assigned event-loop thread under configuration. Blocking a handler can delay many channels served by that loop. Offload necessary blocking work to a bounded executor and preserve response ordering, cancellation and context. Reference-counted buffers need correct ownership/release under the selected APIs; leaking direct buffers can grow process memory outside heap.

Register interest in write readiness only while output remains pending, according to the framework contract; a continuously writable socket can otherwise drive a busy loop. Bound queued outbound data for slow readers. Nonblocking I/O changes where waiting lives, not the finite bandwidth or memory available.

Zero-copy is shorthand for optimized paths that avoid some user-space copies, such as suitable file-to-socket transfer. It is not zero CPU, zero kernel work or a guarantee for every TLS/compression path. Java transfer APIs may use platform optimizations or fallbacks; measure the actual path rather than relying on the name.

[VERIFY: verify Java NIO selector/channel semantics, Netty event-loop and buffer ownership, epoll modes and zero-copy/TLS support against the selected OS/JDK/library. No Java NIO, Netty or zero-copy benchmark ran.]

Interview checks: basic: readiness is not message completion. Internals: partial buffers survive across events. Debug: look for blocking handlers or permanent write interest. Scenario: preserve bounded queues and buffer ownership when offloading work.

11. Diagnostic Toolbox and Safe Scope

Choose a command that separates hypotheses and record the host/container context. The examples below are Linux-oriented unless otherwise stated, require the named tools and are NOT EXECUTED here. Read-only inspection can reveal credentials, addresses, process arguments or customer data; authorization and redaction still apply. Packet capture and tracing can require elevated capabilities and impose overhead.

Tool/example	Use when	Avoid when
curl -v with an approved URL and a bounded timeout	Inspect DNS/connect/TLS/HTTP behavior from one client path	Verbose output containing authorization/cookies is shared unredacted
dig for an approved hostname	Compare resolver records and TTL behavior	Resolver output is assumed to describe a JVM's cached connection
ss -lntp and ss -s	Inspect listeners, socket states and aggregate connection pressure	Missing process detail under permissions is interpreted as no owner
tcpdump with approved interface/host filter and packet count	A scoped packet observation can distinguish retransmission/reset behavior	Broad production capture runs without authority, privacy or stop bounds
top in bounded batch mode	Identify process/thread CPU and memory candidates	One instantaneous sample is treated as a complete profile
vmstat	Compare runnable work, memory and system activity over a defined interval	Counters are interpreted without OS/version/time-window context
iostat	Investigate storage latency/utilization signals	Device averages alone are used to identify a Java call site
strace with an approved PID and bounded capture	Observe selected syscalls/waits to test an I/O hypothesis	Attaching to sensitive or busy production processes is assumed harmless

Do not equate curl success from an administrator host with success from a Pod using another DNS, route, proxy, truststore or role. Likewise, encrypted packet capture generally cannot show an application payload without additional sensitive instrumentation; metadata can still be useful. Thread/CPU profiles and OS traces answer different questions and should be correlated, not substituted blindly.

[D] DECISION

Localize the failure before changing a setting. Name resolution, pool wait, connect, TLS, HTTP status, application processing and return-path streaming are separate stages. The right diagnostic is the one that can disprove the current stage hypothesis.

12. Executed Stream Lab

`code/21-network-os/StreamLab.mjs` uses built-in `StringDecoder`, `readline` and `HTTP` libraries rather than implementing a protocol parser. Its Windows PowerShell runner is `& './docs/java-fs-guide/code/21-network-os/run.ps1'` from workspace root. It binds one temporary HTTP server to 127.0.0.1 on an ephemeral port, uses a bounded client deadline and closes all server connections afterward.

The actual three PASS checks show that a UTF-8 decoder preserves a character split across input buffers, line framing preserves records split across chunks, and the HTTP client assembles JSON from two server write calls. The test does not assert that each write becomes a separate TCP packet/read; the OS is free to coalesce or segment them. It measures no latency or bandwidth and contacts no external host.

[M] MECHANISM

Protocol state must survive buffer boundaries. The decoder/parser remembers incomplete input until enough bytes arrive. This is why a maintained framing implementation is safer than treating each read callback as a complete application message.

[!] TRAP

A transport acknowledgement or cancelled client wait does not settle a remote transaction. Preserve logical request identity and resolve the outcome at the service authority instead of assuming the network exception undid the work.

[?] INTERVIEW

Explain one connection from name to resource. Then identify which timeout, identity check, queue or kernel limit can fail at each step. That is more useful than listing socket flags without a causal model.

Explained-Here Index and Sources

Concepts	Explanation
DNS	Resolution and caches
TCP, ports and TIME_WAIT	Transport lifecycle
Keep-alive	Flow and liveness
HTTP/1.1, HTTP/2, HTTP/3 and QUIC	Framing and multiplexing
TLS handshake and SNI	Protected transport
L4 vs L7, reverse proxy, connect/read timeouts	Proxy and pool budgets
NAT and port pressure	Connection churn
Processes, threads, context switch	Execution
Virtual memory, page cache, file descriptors	OS resources
Blocking I/O, NIO, Netty, event loop, select/poll/epoll, zero-copy	Readiness and buffers
Linux, curl, dig, ss, tcpdump, top, vmstat, iostat, strace	Toolbox

Verification destinations: [TCP RFC 9293](#), [TLS 1.3 RFC 8446](#), [HTTP/2 RFC 9113](#), [HTTP/3 RFC 9114](#), [Java NIO](#) and [Netty documentation](#). These are verification pointers; no protocol trace is fabricated.

Related Chapters

Use [00](#), [01 JVM](#), [02 concurrency](#), [03 web stack](#), [05 streaming](#), [06 durability](#), [08 security](#), [10 deadlines](#), [14 containers](#), [15 cluster networking](#), [18 diagnostics](#), [22 failures](#) and [23 profiling](#). Generated links are reciprocal.

RELATED CHAPTERS / RECIPROCAL LINKS

00 The System Before the Details	18 Observability and Reliability
01 Core Java and the JVM	06 Databases and Durable State
02 Concurrency and the Memory Model	10 System Design Foundations
03 Spring and Spring Boot Internals	22 Distributed Systems Theory
05 APIs and Real-Time Communication	23 JVM Performance Engineering
08 Security, Identity and Data Protection	09 The Browser and the Java Backend
14 Docker and Container Internals	17 Cloud and Infrastructure
15 Kubernetes and Production Workloads	24 IntegrationHub in Motion
	20 Appendix: Interview Toolkit and Index

One-Page Cheat Sheet

Names/connections: DNS caching and existing pools both delay address changes. TCP orders bytes, not messages. Connect/write/transport acknowledgement do not prove application commit. TIME_WAIT is protocol state, not automatically a leak.

Transport: flow control protects the receiver, congestion control the path. HTTP/2 multiplexes over TCP and retains transport-wide loss effects; HTTP/3 uses reliable QUIC streams but still shares path resources. Keepalive, HTTP reuse and application heartbeat differ.

Trust/deadlines: validate certificate identity and trust, not only a signature. SNI routes, mTLS authenticates peers, services authorize resources. Pool acquisition, connect, read inactivity and total deadline cover different waits.

OS: processes isolate address spaces, threads share memory, context switching does not create capacity. Virtual size, RSS and heap differ. Page cache and native buffers matter. Close descriptors and response bodies deliberately.

I/O: selectors report readiness, reads/writes can be partial, parsers retain state and event loops must not block. Zero-copy is path-dependent. Three stream/loopback checks passed; Linux, DNS, TLS and Java NIO experiments remain unexecuted.

Interview Corner

Basic: Why Can One Write Require Several Reads?

TCP exposes bytes without preserving application write boundaries. Framing and decoding must retain partial input and may produce zero, one or several complete messages per read.

Internals: Flow Control Versus Congestion Control?

Receiver capacity versus network-path conditions. Both constrain sending but diagnose different bottlenecks; a slow reader can stall an otherwise healthy path.

Trace/Debug: DNS Changed but Traffic Still Uses the Old Server.

Inspect JVM/OS/resolver caches and long-lived pooled connections. TTL changes do not close existing sockets, and failover also needs write-authority control.

Scenario: A Client Times Out After Posting a Job.

The server may have committed. Retry/query using the same logical identity under the API contract rather than creating a fresh request and assuming the old effect vanished.

Basic: Does HTTP/2 Eliminate All Head-of-Line Blocking?

No. Streams are independent at the HTTP layer but share an ordered TCP transport. HTTP/3 changes that transport relationship but still shares congestion and other resources.

Internals: What Does a Selector Event Mean?

An operation may make progress under the readiness model. It does not guarantee a full message, a full write or exclusive ownership after another thread acts.

Trace/Debug: Read Timeout Never Fires During a Slow Response.

The configured timer may measure inactivity between bytes, not total duration. A trickle can reset it; add an appropriate total deadline and inspect client semantics.

Scenario: An Event Loop Calls a Blocking Database Driver.

One blocked loop can delay many connections. Use the appropriate blocking stack or bounded offload while preserving context, ordering and backpressure; more channels do not fix it.

Basic: Virtual Memory Versus RSS?

Address-space mappings/reservations versus resident physical pages under the OS accounting model. Neither is identical to Java heap used.

Internals: Why Can NAT Limit a Scaled-Out Service?

Many workloads can share egress addresses and translation state. Connection churn and destination concentration may exhaust practical capacity even as application replicas increase.

Trace/Debug: Too Many Open Files With Normal Heap.

Inspect descriptors, unclosed sockets/files/response bodies, process limits and namespace context. Descriptor exhaustion is a different resource failure from heap growth.

Scenario: Which Tool Do You Run First?

Choose the one that distinguishes the suspected stage: resolver lookup, bounded HTTP client trace, socket state, CPU/storage metric or authorized syscall/packet capture. Do not begin with a broad intrusive capture without a hypothesis.

03 / Spring and Spring Boot Internals

Follow the object, the proxy and the request. IntegrationHub's connector is not transactional or secure because its source contains familiar annotations. The relevant infrastructure must exist, the request must traverse it, and the selected policies must match the business invariant. IntegrationHub is fictional; incidents and traces below are explanatory scenarios, not company architecture or measured runtime evidence.

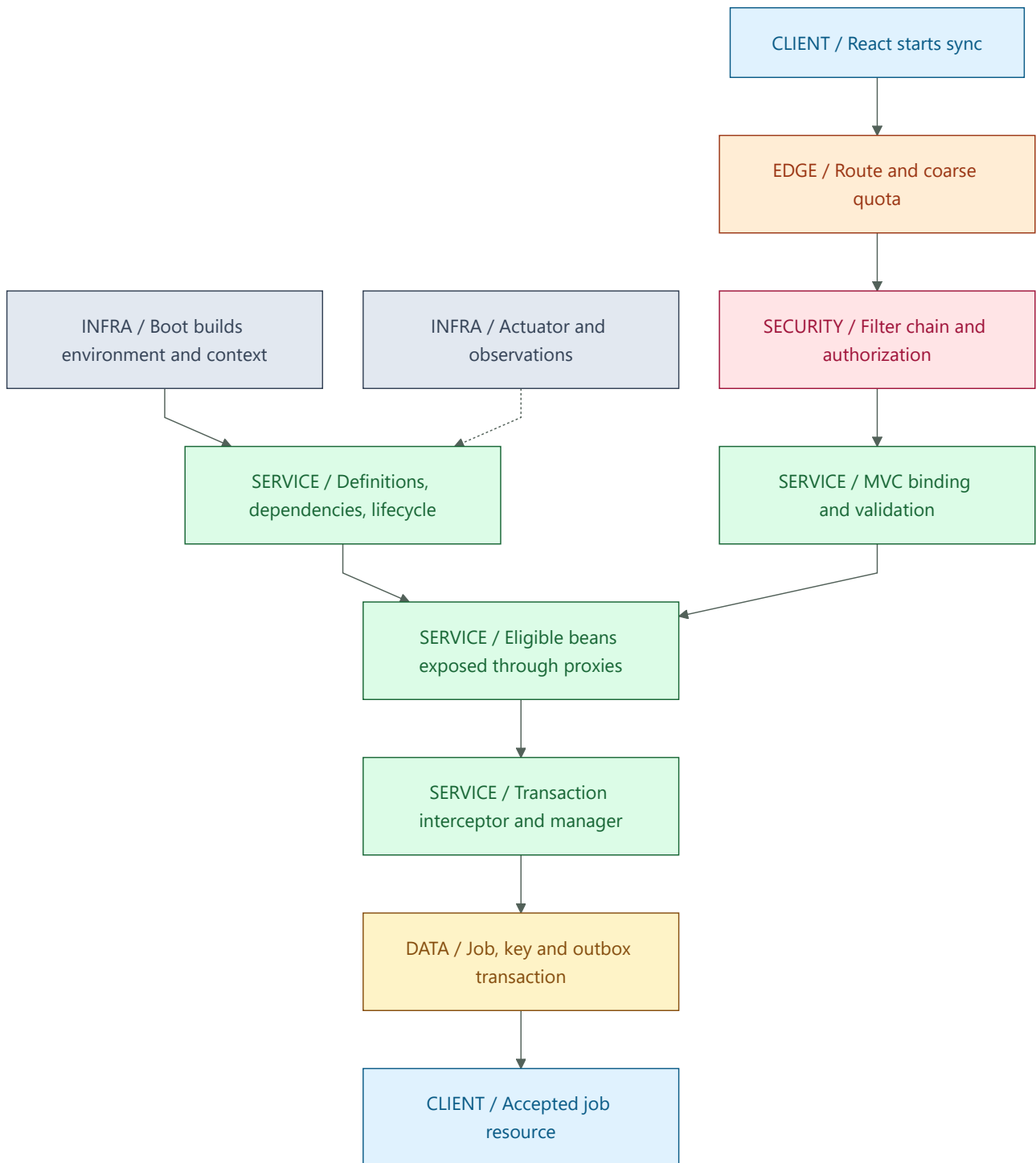
Version assumptions: Java 21, Spring Boot 4.0.x and Spring Framework 7.0.x are the primary baseline; Spring Cloud 2025.1.x is the compatible release-train family. The official Cloud matrix was rechecked on 2026-10-06 and lists 2025.1.x with Boot 4.0.x, plus Boot 4.1.x starting with Cloud 2025.1.2. This chapter stays on Boot 4.0 rather than silently adopting the latest feature line. The Boot 4.0 system-requirements page currently describes 4.0.8 with Java 17 minimum and Framework 7.0.9 or above. Java 21 fits that documented range. The lab pins Boot parent 4.0.8 and Cloud BOM 2025.1.3; importing a BOM is not running Cloud services. Kubernetes 1.34, PostgreSQL 17 and Kafka 4.x remain illustrative IntegrationHub families. The lab uses H2 only for isolated JDBC transaction behavior, not as evidence about PostgreSQL.

[VERIFY: Boot 4.0.8 and Cloud BOM 2025.1.3 are teaching pins selected after checking the official system requirements and compatibility matrix, not a latest-security-patch recommendation. Recheck the exact artifacts, support windows, JDK range, build plugins and transitive dependencies before execution or deployment; no Maven resolution was possible here.]

Execution status: NOT EXECUTED. Java, Maven and a local Maven repository are unavailable. The offline-only runner recorded this; no Java dependencies were downloaded. Complete Spring proxy and container/transaction labs are supplied in `code/03-spring/`. Compilation and all Spring/H2 assertions remain unverified. MVC, validation, Security, Actuator, Gateway, Config, Feign and discovery are explained but not exercised by those labs. Mermaid diagrams are rendered as embedded SVG; syntax highlighting remains unavailable.

Big Picture

Diagram 03.1



The top path prepares objects; the request path uses them. Initialization is not an HTTP request, dependency injection is not authorization, and a proxy is not a database. Keeping those boundaries separate makes unfamiliar startup and production failures much easier to trace.

What You Will Be Able to Explain

- Describe how bean definitions become initialized objects, and distinguish definition processors from instance processors.
- Explain JDK versus class-based proxies, self-invocation, configuration enhancement and advice ordering.
- Diagnose auto-configuration using conditions and dependency evidence rather than adding annotations at random.
- Trace a servlet request through filters, MVC argument resolution, validation, controller, service proxy and response conversion.
- Reason about transaction boundaries, rollback-only state, propagation and the limits of thread-bound transactions.
- Place authentication, authorization, Actuator exposure and probes at the right boundaries.
- Choose Gateway, Config, Feign and discovery intentionally, including what Kubernetes can already provide.

[P] PRODUCTION

Where this lives in IntegrationHub. The [master journey](#) enters a connector endpoint whose injected service is a managed proxy. [Java type/runtime rules](#) constrain that proxy, and [concurrency rules](#) constrain singleton state and task handoffs. The acceptance transaction owns local durable writes, while [07's outbox](#) owns later publication. [08](#) deepens identity and [15](#) deepens runtime networking; neither makes a missing service authorization check acceptable.

1. IoC: The Container Owns Construction, Not Your Invariants

Why it exists: a connector service should depend on a token provider and a job store without hard-coding their construction at every call site. Inversion of control moves object assembly and lifecycle policy to a container. Dependency injection is the mechanism by which collaborators are supplied. This supports replaceable implementations and explicit tests; it does not make a poorly chosen dependency graph a good design.

A bean definition describes how to create and configure an object: type or factory method, scope, dependencies, qualifiers, lifecycle callbacks and other metadata. It is not yet the object. Definitions can come from annotated configuration, component scanning, imports or programmatic registration. The container resolves references and creates instances when the relevant scope/laziness rules require them.

From Definition to Dependency

- Register the service definition and its collaborators, for example a connector registry, secret-reference resolver and job repository.
- Select the constructor/factory method and resolve its arguments against candidate definitions/instances.
- Use type, qualifiers and primary-candidate rules as applicable; ambiguity is an error to understand, not permission to select a random bean.
- Construct the object and complete managed population/initialization before exposing the intended reference to consumers.
- Inject the exposed reference, which may be a proxy rather than the raw object that was constructed.

Constructor injection makes required dependencies visible and permits final fields. It allows a plain unit test to construct the class without starting the container. Setter injection can fit genuinely optional/reconfigurable dependencies; field injection hides requirements and encourages tests coupled to reflection or container startup. A single constructor usually needs no redundant Autowired annotation under modern Spring conventions; do not infer that every constructor overload is automatically unambiguous.

Assembly choice	Use when	Avoid when
Constructor injection	Collaborators are required and should be fixed for the object's lifetime	You mistake a very long constructor for a container problem rather than excessive responsibilities
Bean factory method	You need explicit construction or integrate a third-party class	Factory methods hide unmanaged secondary objects requiring their own lifecycle
Component scanning	Application-owned components follow a clear package boundary	Broad scanning unintentionally registers test/client configuration or misses a sibling module
ObjectProvider or scoped proxy	Lookup must be deferred or cross an actual scope boundary	Deferred lookup merely hides a circular architecture dependency
Direct new	The object is a plain value/helper with no managed requirements	You expect transaction, async, validation or other container advice on the unmanaged instance

Scope is not thread safety. Singleton means one shared bean instance for that definition in the relevant container, not one object per Java class across the JVM. Two contexts or two definitions can produce different instances. A singleton service may receive many concurrent requests; request data must not live in mutable instance fields without a deliberate ownership model. [02](#) explains the required coordination.

Prototype means new instance on each relevant container retrieval, not a fresh object every time any method runs. Injecting a prototype into a singleton normally resolves that dependency at injection time, so the singleton can keep one prototype instance. If fresh lookup

per operation is required, use an explicit provider/factory. Request/session scopes need the appropriate web context and valid scope lifetime; passing a request-scoped object to an unowned background task is not safe by default.

Production trap: a tenant-specific credential field is stored on a singleton connector bean. The next request overwrites it while another task is using it, potentially crossing tenant boundaries. Keep immutable connector definitions shared and authorized per-call context local. Injection fixes wiring, not confidentiality or [data protection](#).

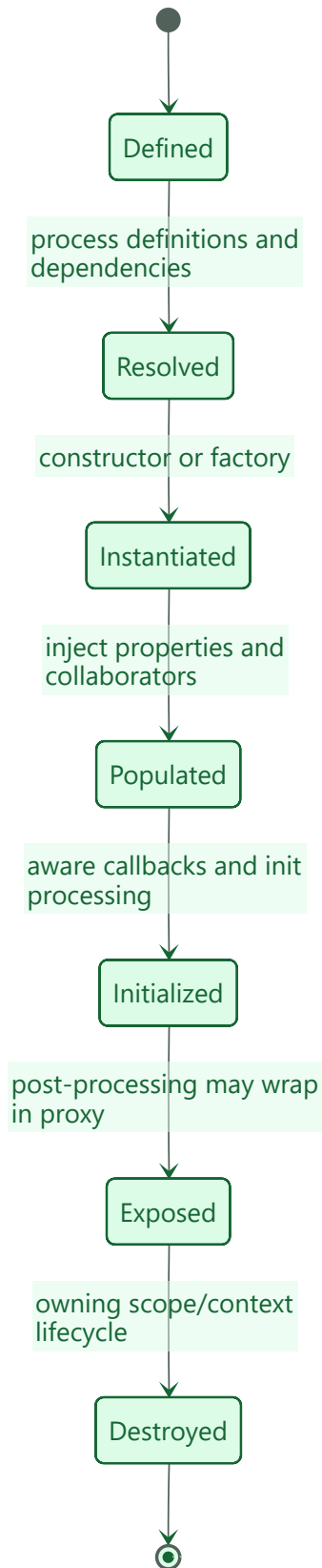
Interview checks: basic: bean definition versus instance. Internals: dependency resolution can supply a proxy. Debug: inspect candidates, qualifiers and scan boundaries. Scenario: a constructor cycle usually indicates responsibilities that need separation; do not reflexively enable circular references.

2. Bean Lifecycle: Definitions First, Instances Later

Why lifecycle hooks exist: resources may need initialization after dependencies are available and cleanup when their owning context closes. Framework extensions also need points to inspect definitions and wrap instances. A mental model that collapses all of these into "Spring calls the constructor" misses where much of Spring's behavior is installed.

BeanFactoryPostProcessor operates on definition-level/container metadata before ordinary application beans are created; BeanDefinitionRegistryPostProcessor additionally contributes definitions. Configuration-class processing is an important example. BeanPostProcessor works around instance creation/initialization; specialized variants participate in instantiation and dependency population. Auto-proxy infrastructure uses these extension points to expose eligible objects with advice.

Diagram 03.2



This is a normal-path model, not the exact callback list for every bean. Special infrastructure beans, `FactoryBeans`, lazy instances, early references and scope proxies add branches. Dependency injection itself can be implemented through specialized post-processors; it is not always a single universal step between two fixed callbacks.

For a typical initialized bean, relevant `Aware` callbacks provide container context, initialization processing handles `PostConstruct`, `InitializingBean.afterPropertiesSet` and a configured `init` method as applicable, then after-initialization post-processing can expose the advised object. When using multiple initialization mechanisms, understand their order and avoid performing the same expensive action twice. Constructors must establish a valid object; lifecycle hooks are not a reason to publish partially initialized state.

Initialization Is a Poor Place for Remote Work

A `PostConstruct` method runs as part of creating the target bean, before callers use the fully exposed proxy in the usual way. A self-call from that initialization method should not be expected to trigger transactional advice. A remote provider outage during startup can block or fail context creation; readiness and retry policy need a conscious lifecycle design, not a network call hidden inside a constructor.

For startup work, distinguish lightweight local validation, asynchronous warm-up, migration, a one-time administrative job and essential initialization that must prevent serving. `SmartLifecycle`, runners and application events have different timing/ownership semantics. A runner finishing is not a distributed leader-election guarantee, and starting the same runner on every Pod can duplicate work. 22 owns singleton-job coordination.

Cleanup callbacks apply to objects whose lifecycle the container owns. Singleton destruction on context close does not mean prototype instances are automatically tracked and destroyed after arbitrary use. Request-scoped cleanup also does not make detached background tasks safe. A process kill or node failure can skip normal shutdown entirely; durability cannot depend solely on `PreDestroy`.

[VERIFY: exact callback order, early-reference behavior, circular-reference policy, scoped-proxy behavior and AOT/native constraints depend on the Spring/Boot version and bean kind. Check the selected Framework lifecycle/post-processor documentation and test the actual context; the chapter diagram is a normal-path model, not a universal callback trace.]

Lifecycle decision	Use when	Avoid when
Local init validation	Invalid configuration should fail before serving	Slow external calls obscure whether startup or a dependency is unhealthy

Lifecycle decision	Use when	Avoid when
Managed start/stop component	The component owns a background resource with explicit ordering	A raw unmanaged thread survives context close
Explicit job execution	Work needs retries, durable state or single-owner coordination	Every replica's startup implicitly runs the same destructive job
Destroy callback	Best-effort orderly cleanup of owned resources	It is treated as the only mechanism to preserve accepted business work

Lab connection: ContainerTransactionLab holds a LifecycleProbe, checks its configured open callback and per-context singleton identity, closes the context, then checks close was invoked. It does not demonstrate every lifecycle callback or Spring Boot startup event. All checks remain unexecuted.

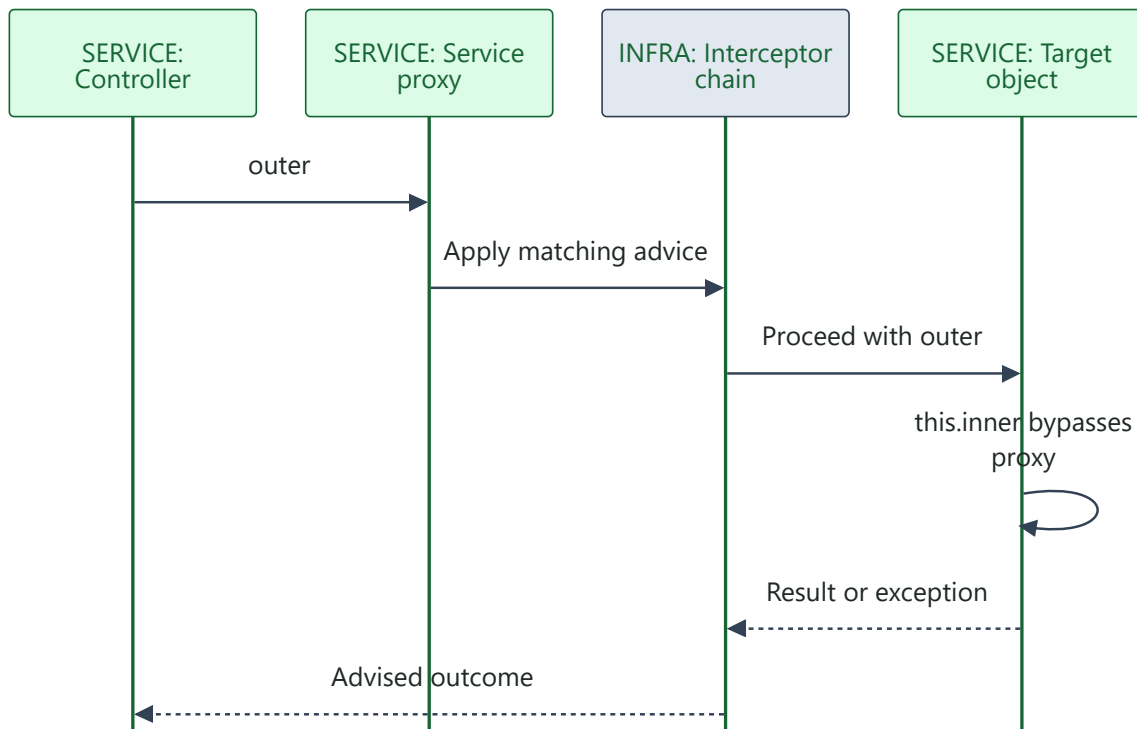
Interview checks: basic: post-processors are not all interchangeable. Internals: definition processing precedes ordinary instance work. Debug: a bean can exist without the proxy behavior you expected if it was created too early or outside management. Scenario: choose an explicit startup policy rather than relying on annotations inside initialization.

3. AOP Proxies: Did the Call Cross the Boundary?

Why AOP exists: transaction interception, auditing and other cross-cutting behavior should not be manually repeated in every business method. A pointcut selects relevant executions, advice describes the extra behavior, and an advisor associates the selection with the advice. Spring's usual AOP model intercepts method calls through a proxy; it is not arbitrary interception of every field access or every internal method call.

The client holds a proxy. On a selected invocation, the proxy runs an interceptor chain around the target call. Before/after/around behavior depends on the advice; around advice can change whether, how often, or with which result the target executes. Advice ordering therefore has business consequences. Retry outside a transaction can create a fresh transaction per attempt; retry inside one already-rollback-only transaction may not produce the intended recovery.

Diagram 03.3



JDK and Class-Based Proxies

JDK dynamic proxies implement interfaces; consumers should use the exposed interface contract. Class-based proxies create a subclass of the target class. For that mechanism, final classes cannot be subclassed, and final/private methods cannot be overridden for advice. These restrictions should not be misstated as "a final target can never be used behind any Spring proxy": a JDK interface proxy can delegate to a final implementation.

Core Spring and Boot can select different defaults depending on configuration. Explicit proxy settings and framework version matter, so inspect the actual bean type and AopUtils diagnostics rather than reciting one unconditional default. Native-image/AOT environments impose additional constraints beyond the ordinary JVM teaching model.

Mechanism	Use when	Avoid when
JDK interface proxy	Consumers need a defined interface contract	Code insists on casting the exposed proxy to a concrete implementation
Class-based proxy	The class/method contract supports subclass interception	Final/private/inaccessible methods are assumed interceptable
Separate collaborating bean	A cross-cutting boundary must be crossed during a workflow	Splitting classes solely to hide a still-confused business transaction boundary
Programmatic transaction template	An explicit lexical boundary is clearer than proxy interception	Repeating low-level transaction code without a reason

Mechanism	Use when	Avoid when
AspectJ weaving	Requirements genuinely need bytecode-level join points	Introducing operational/build complexity merely to avoid understanding self-invocation

Self-invocation: once execution is inside the target, `this.inner` is a direct target call, not another call through the caller's proxy. It bypasses advice on `inner` in the usual proxy model. A class-based proxy does not automatically fix this. Move the advised operation to a collaborator, make the external method the correct boundary, or choose explicit programmatic control. Self-injection/`currentProxy` approaches add coupling and are rarely the clearest default.

[VERIFY: proxy defaults, method visibility support and per-bean proxy controls differ across Framework/Boot releases. Official proxy documentation was consulted; the labs deliberately choose interface proxying and class-based transaction proxying explicitly. Do not apply class-proxy final-method restrictions to all interface proxy calls or assume weaving behaves like proxies.]

Complete Proxy Boundary Probe

NOT EXECUTED. This complete class matches the saved `ProxyBoundaryLab.java` in the Maven lab. It uses real Spring `ProxyFactory`, not a homemade simulation. Its success string is code, not observed output. The target is final to demonstrate the interface-proxy distinction; the code does not request a class-based proxy.

java

```
package guide.spring;

import java.util.ArrayList;
import java.util.List;
import org.aopalliance.intercept.MethodInterceptor;
import org.springframework.aop.framework.ProxyFactory;

public final class ProxyBoundaryLab {
    public interface Operations {
        void outer();
        void inner();
    }

    public static final class Target implements Operations {
        private final List<String> events;

        Target(List<String> events) {
            this.events = events;
        }

        public void outer() {
            events.add("target:outer");
            inner();
        }

        public void inner() {
            events.add("target:inner");
        }
    }
}
```

```

    }

    public static void main(String[] args) {
        var events = new ArrayList<String>();
        var factory = new ProxyFactory(new Target(events));
        factory.setInterfaces(Operations.class);
        factory.addAdvice((MethodInterceptor) invocation -> {
            events.add("advice:" + invocation.getMethod().getName());
            return invocation.proceed();
        });
        Operations proxy = (Operations) factory.getProxy();
        proxy.outer();
        if (!events.equals(List.of("advice:outer", "target:outer", "target:inner"))) {
            throw new AssertionError("Self-invocation must bypass this proxy's advice: " + events);
        }
        events.clear();
        proxy.inner();
        if (!events.equals(List.of("advice:inner", "target:inner"))) {
            throw new AssertionError("External invocation must cross the proxy: " + events);
        }
        System.out.println("ProxyBoundaryLab: all checks passed");
    }
}

```

The expected event sequences are assertions in unexecuted source, not a terminal transcript. They identify exactly what would disconfirm the explanation: inner advice appearing during target self-invocation or missing during a direct external proxy call under this setup.

Configuration Enhancement Is a Separate Concern

Full Configuration classes can be enhanced so calls between Bean methods resolve managed beans rather than naively creating unrelated instances. With `proxyBeanMethods=false`, ordinary method calls between factory methods are ordinary Java calls; use method parameters for dependencies instead. This optimization/control is distinct from transaction/AOP proxying of the beans the configuration creates. The transaction lab disables configuration-method proxying but explicitly enables class-based transaction proxies, demonstrating that these are different decisions.

Production failure: a connector's public method calls a private annotated method and assumes its retry/transaction rule applies. The annotation is present but the invocation boundary never existed. Trace the actual exposed reference, interception eligibility and call path before changing isolation levels or increasing retries.

Interview checks: basic: annotation metadata is not executable interception by itself. Internals: advisors run on calls through the proxy. Debug: new, self-calls and ineligible methods are common missing boundaries. Scenario: choose the business boundary first, then make the framework call path match it.

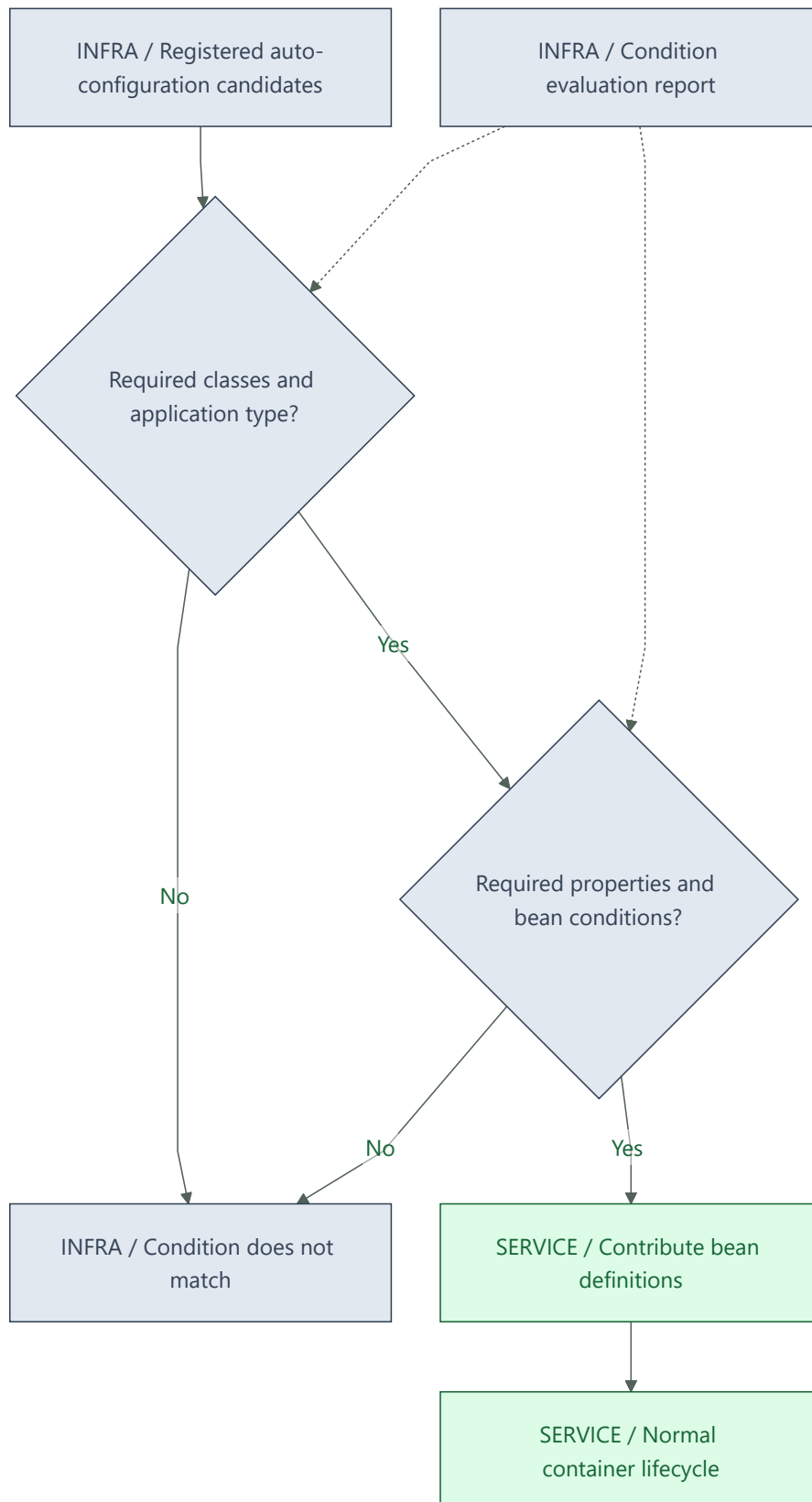
4. Boot Startup and Conditional Auto-Configuration

Why Boot exists: common applications otherwise repeat dependency selection, infrastructure configuration, runtime packaging and operational setup. Boot layers conventions and conditional configuration over Spring; it does not replace the IoC container or invent a separate transaction model.

SpringBootApplication combines Boot configuration, auto-configuration enablement and component scanning. SpringApplication prepares an Environment and application context, loads source definitions, refreshes the context, and coordinates the application lifecycle. Embedded-web-server startup, runners, events and availability transitions are part of that lifecycle, not one promise that every possible external dependency is ready when the constructor returns.

Auto-configuration candidates come from registered metadata. In current Boot, AutoConfiguration.imports is the standard registration mechanism for auto-configuration classes. Conditions decide whether particular configuration/bean contributions apply: classes on the classpath, web-application kind, properties and already registered bean candidates are common inputs. Do not describe this as scanning every library class for anything that looks useful.

Diagram 03.4



Backing Off Is Conditional, Not Magical Override

For example, an application-defined DataSource can make the corresponding missing-bean condition fail, so Boot's default contribution backs off. This is different from two definitions colliding under a global override setting. Not every custom bean disables every related auto-configuration; inspect the exact condition and declared type. An overly vague Bean return type can make type-based conditions or resolution harder to reason about.

Starter dependencies make a coherent set of libraries easy to select; auto-configuration reacts to the available capabilities. A BOM manages dependency versions; it does not add the corresponding runtime libraries to the application. A starter also does not prove a feature is enabled under every property/profile combination.

Diagnostic trace: identify the missing or unexpected bean -> inspect active profiles and effective properties -> inspect resolved dependencies and application type -> read the condition evaluation report -> inspect user-defined competing beans/exclusions. Boot's debug mode can emit the report; an Actuator conditions endpoint is a separate, access-controlled observation surface. Neither should justify exposing internal configuration publicly.

[VERIFY: Boot 4 modularization, starter names, auto-configuration packages and condition definitions differ from Boot 3 examples. Check the pinned Boot reference and condition report; official Boot 4.0 auto-configuration documentation was consulted, but no Boot application or report was executed here.]

Configuration approach	Use when	Avoid when
Boot's matching defaults	They satisfy the service contract and can be verified	Defaults are assumed without inspecting profiles/dependencies
Application Bean replacing a default	You need a specific implementation and understand backoff conditions	Expecting one custom bean to disable all security/data infrastructure
Targeted exclusion	A known auto-configuration must not apply	Broad exclusions hide a missing dependency or break neighboring capabilities
Explicit framework configuration	A small lab or unusual integration needs precise control	Rebuilding every Boot convention without a requirement

Migration checkpoint: Boot 3 commonly pairs with Framework 6 and the corresponding Cloud train, while this chapter uses Boot 4/Framework 7. Jakarta package migration from older enterprise APIs, changed module/starter packaging and removed deprecated APIs are different classes of issue. Not every javax package moved: javax.sql.DataSource is still a Java SE API and is deliberately used in the lab. Do not perform a blind global javax-to-jakarta replacement.

Production failure: adding an unrelated starter changes the classpath enough to activate an unintended infrastructure path. Startup now fails or uses a different bean. Capture the dependency change and condition report, then choose an explicit supported configuration rather than downgrading random transitive JARs.

Interview checks: basic: Boot is Spring plus conventions/integration, not another DI model. Internals: conditions contribute definitions. Debug: backoff differs from override. Scenario: prove which configuration applies using the effective environment and report.

5. External Configuration: Typed Values and Controlled Change

Why it exists: the same image should run with different endpoints, capacities and secret references without rebuilding application code. Environment aggregates property sources, while binding maps external values into application configuration. Configuration is an input with validation and authorization needs, not a harmless collection of strings.

ConfigurationProperties is useful for a coherent group such as connector timeouts, concurrency and retry limits. It supports typed binding, including values such as Duration when configured appropriately, and validation when the validation infrastructure/annotations are in place. Register the properties type through the supported scanning/enablement mechanism. A record-style properties object can make immutable startup configuration clear.

Value is convenient for an isolated value or expression, but many scattered placeholders obscure the service's configuration contract. Neither annotation implies live updates to every existing object. Resolving a value at construction time is different from reading the Environment again during an operation, and neither alone safely replaces a live connection pool.

Resolution Trace

- Assemble property sources from configured locations and inputs.
- Determine active profiles and imports according to Boot's config-data rules.
- Resolve a property's winning source under the documented precedence rules.
- Convert and bind it to the declared target type.
- Validate constraints and cross-field invariants before the component uses it.
- Record a nonsecret version/origin for diagnosis, not raw secret material.

Command-line inputs, environment variables and files can have different precedence, and early bootstrap properties are read at particular phases. A late property source cannot necessarily change an earlier logging/bootstrap decision. Profile documents add another dimension: verify

the active configuration, not just a checked-in YAML file. Never assume the mounted file is the value currently inside every bean.

Technique	Use when	Avoid when
Typed configuration properties	A coherent, validated configuration contract has several related fields	Invalid cross-field combinations are accepted until the first request
Isolated Value injection	One simple value is genuinely local	Scattered injection hides a large shared config schema
Immutable rollout configuration	Reproducibility and coordinated replacement matter	Restart is used blindly where a documented safe live-update mechanism is required
Controlled live refresh	Specific beans support safe rebind/recreation and change is audited	A property update is assumed to atomically switch all processes/resources
Secret references	Application needs a credential through an authorized secret path	Plain secrets enter Git, environment dumps, error messages or full HTTP logs

[VERIFY: property-source precedence, relaxed binding, constructor binding, validation activation and config-data imports vary with Boot/Cloud version and application setup. Consult the pinned external-configuration reference and inspect effective origins; no configuration binding or live-refresh test ran in this lab.]

IntegrationHub failure: a concurrency limit refresh lowers the intended bound while many jobs already hold permits. Replacing a number does not settle ownership of existing permits or implement a safe transition. Define how old work drains and when the new policy applies; [02](#) owns atomic snapshots and resource accounting. Config refresh is a business change, not merely an HTTP management call.

Interview checks: basic: property source versus bound object. Internals: conversion/validation occur at binding boundaries. Debug: find the actual winning property origin. Scenario: distinguish restart-safe configuration from stateful resources requiring coordinated refresh.

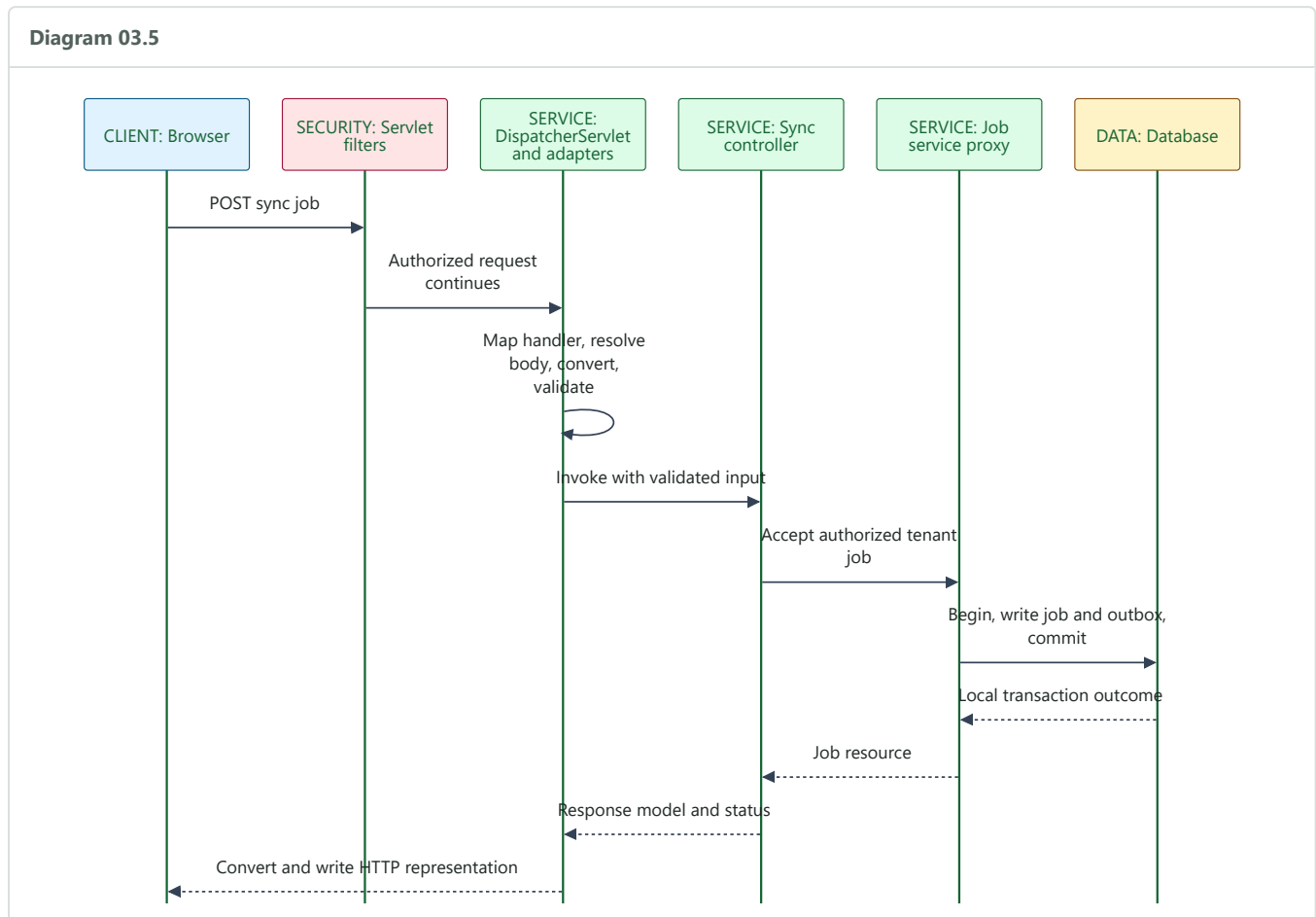
6. MVC Request Lifecycle: Before and After the Controller

Why MVC infrastructure exists: routing, argument conversion, validation, exception translation and response representation are recurring concerns. A controller should translate an HTTP contract into an application call, not parse sockets, authenticate every token and manually serialize every failure.

This section traces the servlet MVC stack, not WebFlux. A servlet container accepts the request and runs its filter chain, including applicable security and other filters. DispatcherServlet is the front controller for MVC dispatch. HandlerMapping locates the handler and related

interceptors; HandlerAdapter knows how to invoke it. Argument resolvers obtain path/query/header/principal/body values, conversion and message converters build Java arguments, and validation can reject them before the method body runs.

Diagram 03.5



Return-value handlers and message converters produce the response, for example serializing a response body rather than resolving a view. RestController combines controller semantics with response-body handling; it is not a separate HTTP server. A ResponseEntity expresses status, headers and body, not a transaction or authorization guarantee.

Exception Boundaries Matter

HandlerExceptionResolver infrastructure can translate applicable MVC exceptions; ControllerAdvice/ExceptionHandler centralizes application-specific mappings. A failure earlier in the servlet/security chain may never reach that MVC mechanism. A response already committed or partially streamed cannot always be replaced with a clean error document. Distinguish where the failure happened before assuming one global annotation catches every request failure.

A transaction can commit successfully inside the service proxy, then response serialization can fail later. The client sees a failure or loses the response, but the job may already be durable. Conversely, flushing an entity before returning is not necessarily commit. This is the same unknown-outcome boundary from 00, with API idempotency in 05 and persistence mechanics in 04.

Filters surround servlet-level processing and can reject requests before handler selection. MVC interceptors act around MVC handler invocation and see handler-related context; they are not substitutes for the Security filter chain. Argument resolvers populate controller parameters; service AOP wraps managed method calls. These extension points overlap in capabilities but operate at different boundaries.

Extension point	Use when	Avoid when
Servlet filter	Concern spans servlet processing before/after dispatch	You need a selected handler but assume mapping has already happened
MVC interceptor	Handler-aware MVC behavior is required	Treating it as complete security coverage for all dispatches/endpoints
Argument resolver	A well-defined controller parameter should be derived consistently	Hiding untrusted tenant selection as an already authorized identity
ControllerAdvice	Translate applicable MVC/application errors consistently	Expecting it to handle every filter failure or rewrite a committed stream
Service advice	A managed application-method boundary owns the concern	HTTP-specific parsing is mixed into every business method

Async request trap: MVC async processing can release a servlet thread while work continues, and later dispatch/completion callbacks have their own context and error behavior. It is not automatically a reactive end-to-end pipeline or a transaction spanning arbitrary new threads. 02 and 05 explain execution and stream lifetimes.

Production incident: the controller breakpoint is never hit and the response is unauthorized, unsupported media type or invalid input. Inspect filter/security decisions, handler mapping, converter selection and binding/validation first. A service bug cannot explain a path that never reached the service.

Interview checks: basic: DispatcherServlet dispatches; the container accepts network traffic. Internals: mapping, adaptation, argument resolution and conversion are separate steps. Debug: locate the exception boundary. Scenario: a post-commit serialization failure still needs an idempotent client retry contract.

7. Validation: Shape, Business Rules and Database Invariants

Why validation exists: reject invalid input early with a stable error contract, while preserving authoritative checks under concurrency. Bean Validation can express local constraints such as nonblank connector IDs or bounded batch size. It does not prove the caller owns that connector, that the provider supports the requested mapping, or that another request has not already used an idempotency key.

Valid triggers cascaded validation at supported boundaries; Validated enables relevant Spring validation/group behavior. The exact MVC or method-validation path depends on annotations, configuration and version. A constraint annotation on an arbitrary object does nothing unless some validator actually invokes it. Nested DTOs and container elements need the appropriate constraints/cascade rules.

Three Checks, Three Authorities

- **Request shape:** can input be parsed and converted, and do local field/nested constraints hold? Perform this before unnecessary work.
- **Business/security policy:** is the authenticated principal allowed to operate on the resolved tenant resource, and does the requested operation make sense now? The service owns this decision.
- **Durable concurrency invariant:** can the database transaction insert this tenant-scoped key/job without violating uniqueness or another required invariant? The database remains the authority when requests race.

The MVC sequence's validation stage covers the first boundary; the service and database stages cover the others. Keeping one giant validator responsible for every database/remote check can produce expensive duplicated queries and time-of-check/time-of-use races. For example, two validated requests can both see an unused key before either inserts it; a unique constraint plus defined conflict handling is still necessary.

Validation mechanism	Use when	Avoid when
Bean Validation on DTOs	Local input constraints are deterministic and cheap	It is mistaken for authentication, authorization or serializable access
Cross-field constraint	Several input fields have a stable relationship	A remote provider call is hidden in a field validator without timeout/cost policy
Service-level validation	Domain decisions need authorized current context	Every caller can bypass it by calling a lower-level write method

Validation mechanism	Use when	Avoid when
Database constraint	An invariant must survive concurrent writers	Relying only on a prior existence check
Error translation	Clients need stable codes and field paths	Returning raw exception messages, SQL or secrets

[VERIFY: MVC method-validation activation, exception types, validation groups and proxy-based service validation differ across Framework versions and annotation placement. Check the pinned MVC/Bean Validation integration documentation; no controller, validator or error-handler integration test was executed.]

Production trap: validation error text echoes a secret or a rejected raw payload. Return a bounded, sanitized error model and a safe correlation identifier; retain only approved diagnostics. 08 owns data handling, and 19 should include malformed input, cross-tenant access and concurrent duplicate-request cases.

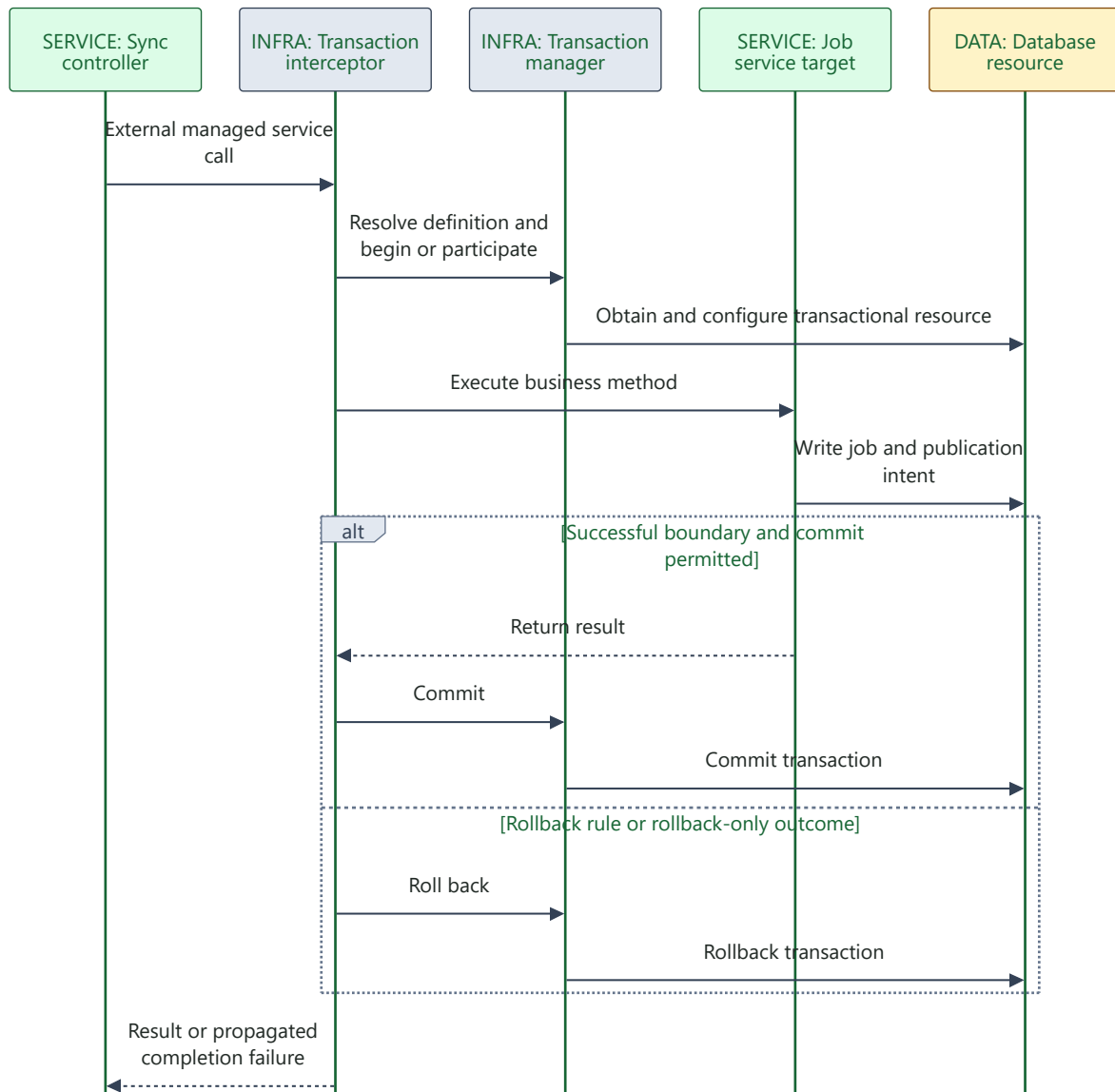
Interview checks: basic: constraints require an invocation boundary. Internals: binding/conversion may fail before validation. Debug: verify cascades and groups. Scenario: early validation and a database uniqueness constraint solve different problems.

8. Transactional: Metadata, Interceptor, Manager, Resource

Why declarative transactions exist: a service operation often needs a local group of database changes to commit or roll back together. Annotation metadata describes the intended policy; a proxy interceptor discovers it, a transaction manager applies it, and participating resource access must use that manager's resource integration. The annotation is not a database feature by itself.

For IntegrationHub acceptance, job, idempotency record and outbox intent should commit in the same local transaction. The later Kafka publish is outside that database commit. 07 explains the durable relay/duplicate-delivery boundary; using Transactional on a Java method does not turn PostgreSQL plus an HTTP provider plus Kafka into one atomic operation.

Diagram 03.6



Thread Binding Is Not Global Scope

With imperative transaction management, relevant resources/synchronizations are commonly bound to the executing thread through Spring's transaction infrastructure. `JdbcTemplate` participates through its resource utilities instead of casually opening an unrelated connection. A new thread or arbitrary async continuation does not automatically inherit that transaction. Reactive transaction management uses a different context/lifetime model; do not apply the thread-bound story unchanged to a Publisher.

The correct manager matters. A JDBC manager for one `DataSource` does not automatically coordinate another `DataSource` or an external system. When multiple managers exist, select/qualify deliberately. Different repositories may participate through different resource integrations; inspect the actual stack before attributing everything to a single annotation.

Defaults and Rollback Rules

The documented default propagation is **REQUIRED** and isolation is **DEFAULT**. The default exception rule rolls back on **RuntimeException** and **Error**, not ordinary checked exceptions; explicit rules and newer global rollback configuration can change that. Never say "checked exceptions can never roll back" or "all Spring exceptions always roll back." A caught exception that never reaches the interceptor can also produce a different outcome than an uncaught one, unless the transaction was independently marked rollback-only.

readOnly is a hint/policy input whose enforcement depends on the transaction manager, database and persistence provider. It is not a universal write-prevention security boundary. A configured transaction timeout also does not reliably interrupt arbitrary remote I/O or bound every later cleanup step. Keep database work bounded and avoid holding connections/locks while waiting for slow providers.

Propagation choice	Use when	Avoid when
REQUIRED	Service calls should join one local unit if it exists, otherwise start one	Assuming inner success/failure commits independently
REQUIRES_NEW	A deliberately independent transaction must suspend an outer one	It accidentally breaks business atomicity or exhausts a pool while outer resources remain held
NESTED	Supported resource/manager savepoint semantics fit partial rollback inside one physical transaction	Assuming every JPA/driver/manager supports it or that it commits independently
SUPPORTS / NOT_SUPPORTED	The operation's contract intentionally allows participation or suspends it	The resulting no-transaction path performs writes that needed atomicity
MANDATORY / NEVER	You want to enforce presence or absence of an existing transaction	These are used without understanding callers' boundaries

REQUIRED and the Rollback-Only Surprise

An outer transactional facade calls another bean's transactional method. The inner call participates in the same physical transaction and throws a rollback-triggering exception through its interceptor. The outer code catches the exception and returns normally, but the shared transaction may already be marked rollback-only. When the outer interceptor attempts commit, **UnexpectedRollbackException** reports that it could not commit as the caller expected. Catching an exception does not undo the transaction manager's state.

ContainerTransactionLab deliberately exercises this with a facade and a separate **JobService** proxy. It also contrasts an external **insertThenFail** call, which should roll back, with a nontransactional **selfInvokeThenFail** call that invokes the annotated method directly on the target. Under the lab's **JDBC/H2** setup, that self-call bypasses advice and the insert can auto-commit before the exception. This is a negative fixture, not a recommended service design.

Diagnosing a Missing or Wrong Transaction

- Was this object obtained from the intended context, or created directly with new?
- Did the caller invoke the exposed proxy, or a target/self/private/final path ineligible for that mechanism?
- Did the advice resolve the intended transaction manager and resource?
- Did an existing outer transaction change the effective propagation/isolation behavior?
- Was the exception thrown through the interceptor, translated, caught, or followed by rollback-only state?
- Did another thread, connection or external service perform work outside the local boundary?
- Did the database commit fail after the method body returned, or did response conversion fail after commit?

[VERIFY: transactional method visibility, global rollback defaults, reactive cancellation semantics, savepoint support and timeout/read-only enforcement vary by Framework, transaction manager and resource. Official annotation/proxy documentation was consulted; the H2 lab is unexecuted and cannot establish PostgreSQL behavior, production pooling or cross-resource atomicity.]

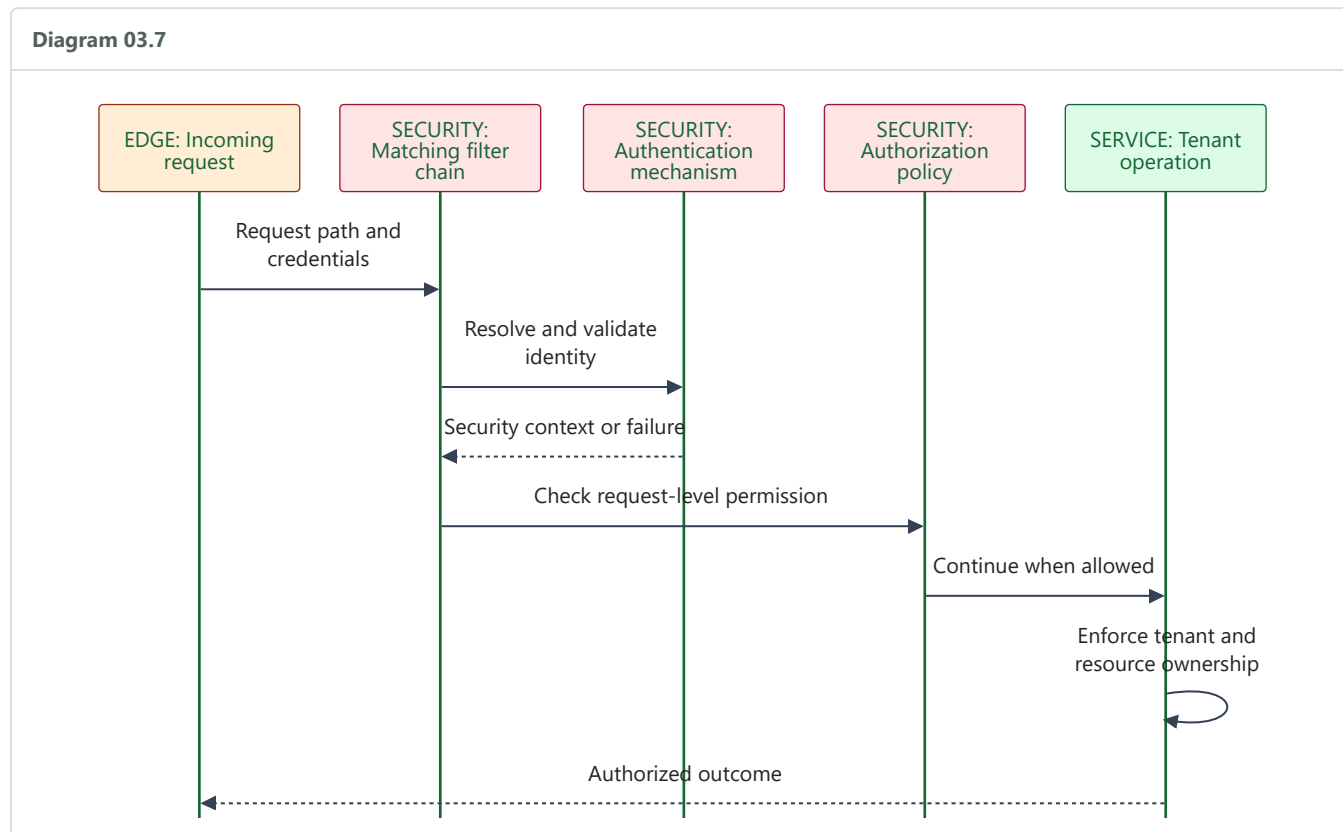
Production failure: `REQUIRES_NEW` is added to make audit records survive a failed outer operation, but each caller holds its outer connection while waiting for another. Pool starvation can follow under load if capacity cannot satisfy the nested demand. Make the independent-commit requirement explicit, size/test resources, and consider a durable event design where appropriate. [02](#), [06](#) and [23](#) connect those effects.

Interview checks: basic: annotation plus interception plus manager plus participating resource. Internals: logical `REQUIRED` scopes can share one physical transaction. Debug: distinguish missing proxy from rollback-only participation. Scenario: never put a remote payment effect inside a local transaction and claim it will be undone by rollback.

9. Spring Security: Select the Chain, Then Authorize the Resource

Why it exists: authentication and access policy should be enforced consistently before protected work, not reimplemented ad hoc in controllers. On the servlet stack, a container-level `DelegatingFilterProxy` delegates to Spring-managed security infrastructure; `FilterChainProxy` selects the applicable `SecurityFilterChain`. With multiple chains, matching/order determines which chain handles the request, rather than every chain being concatenated blindly.

Authentication produces an authenticated principal/authorities under a configured mechanism. Authorization asks whether that principal may perform this operation. A verified JWT signature alone does not answer tenant/resource authorization; issuer, audience, time claims and authority mapping need their configured checks, and the service must enforce ownership for the requested connector/job.



A broad earlier chain can shadow a more specific one. A gap with no applicable chain can leave a request outside expected protection depending on configuration. Request-level matchers and method-level authorization are different enforcement points; method authorization also needs its infrastructure enabled and an eligible call path. Proxy self-invocation can matter there too. Inspect the selected chain and actual authentication rather than asserting that declaring one bean secures every route.

The IntegrationHub Browser Boundary

The reference browser flow uses a same-origin secure HttpOnly session at the gateway/BFF boundary, with internal access-token handling as appropriate. Cookie-authenticated state-changing requests need CSRF protection. A stateless internal bearer API may have a different CSRF analysis, but a broad "REST means disable CSRF" rule is unsafe. CORS controls browser cross-origin access; it is not authentication or a rule restricting all nonbrowser callers.

Do not accept an arbitrary tenant header just because the gateway sent other useful headers. Strip/rebuild trusted identity metadata at the boundary, authenticate the hop where required, and authorize at the service. SecurityContext and logging/tracing context need supported propagation when work crosses thread/executor boundaries; 02 explains why ThreadLocal does not follow arbitrary future composition automatically.

Enforcement choice	Use when	Avoid when
Request-level rules	Paths/methods define a useful first access boundary	Fine-grained resource ownership is assumed from a path pattern alone
Method authorization	Application operations need reusable policy independent of one controller	Infrastructure is not enabled or a bypassed method call is assumed intercepted
JWT resource-server validation	Service receives tokens for its intended audience and trust model	Signature-only checks or user-controlled authority/tenant claims are blindly trusted
Session/cookie boundary	Browser session lifecycle is intentionally server-managed	CSRF, cookie scope, logout/revocation and cross-origin behavior are ignored

[VERIFY: SecurityFilterChain defaults/order, matcher APIs, method-security activation, JWT claim validation and management-endpoint security differ by Spring Security/Boot version and configuration. Check the pinned servlet security architecture and test both allowed and denied paths; no authentication or authorization integration test ran in these labs.]

Production incident: a management-path rule is assumed to protect Actuator, but the actual management context/port or selected chain differs. An internal endpoint becomes accessible more broadly than intended. Treat management networking and authentication as part of the attack surface and validate them from outside the process, not only through unit tests of a controller.

Interview checks: basic: authentication is not authorization. Internals: first applicable chain and its filters determine the servlet path. Debug: distinguish filter failures from MVC advice failures. Scenario: test cross-tenant denial even for a correctly authenticated token.

10. Actuator: Operational Visibility Is an Access-Control Decision

Why it exists: operators need standard health, metrics and diagnostic surfaces without adding custom debugging controllers. Actuator contributes endpoints and integrations; what is enabled, accessible, exposed over HTTP/JMX and authorized are separate concerns. A dependency on Actuator is not a decision to publish every endpoint to the internet.

Health can aggregate indicators; metrics expose measurements through appropriate registries/endpoints; diagnostic surfaces can reveal conditions, mappings or configuration. These can disclose topology, secret-adjacent data or resource usage even when some values are sanitized. A heap dump is especially sensitive and expensive. Minimize exposure, protect it, audit access and bound diagnostic collection.

Liveness and Readiness Are Different Questions

Liveness asks whether restarting this process is an appropriate response to its state. Readiness asks whether it should receive traffic now. Putting a shared database outage into every Pod's liveness check can create a restart storm without fixing the database. Readiness dependency policy also requires judgment: marking every replica unready during a shared outage may be correct for one service contract and harmful for another.

A management port responding proves that its handler path works, not that the main application connector is healthy. Where supported and appropriate, expose/check relevant probes through the serving path as well. A successful health endpoint does not verify the entire job/outbox/consumer/SSE business workflow. 15 covers probes; 18 covers SLIs, alerting and synthetic/business checks.

Operational surface	Use when	Avoid when
Minimal health endpoint	Infrastructure needs a narrow availability signal	Detailed dependency names/secrets are exposed unnecessarily
Metrics registry endpoint	A secured monitoring path consumes bounded-cardinality signals	Tenant/job IDs become unbounded label dimensions
Conditions/configuration diagnostics	Authorized troubleshooting needs configuration evidence	They are left broadly exposed after an incident
Heap/thread diagnostics	A permitted investigation needs process evidence	Uncontrolled public access or collection without resource/PII review

[VERIFY: Actuator endpoint access/exposure defaults, property names, health groups, probe integration and security backoff vary across Boot versions. Verify the pinned operational reference and actual network paths; no Actuator endpoint or Kubernetes probe was executed.]

Production trap: adding a custom SecurityFilterChain changes the assumptions behind Boot's management security defaults. Re-test management access explicitly. A default that protected an endpoint before customization is not evidence for the customized deployment.

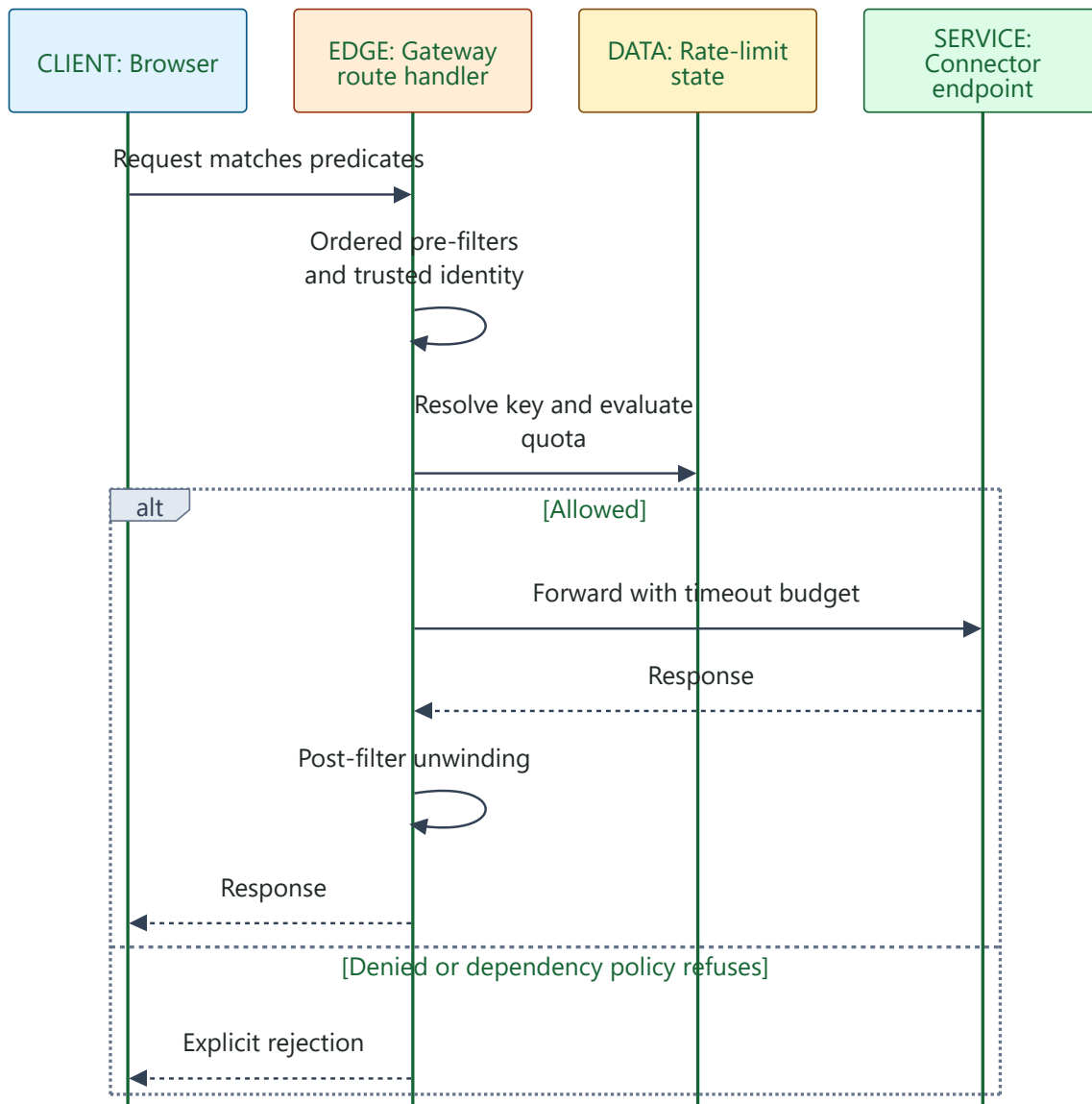
Interview checks: basic: enablement, exposure and authorization are separate. Internals: health indicators and business success measure different things. Debug: management-port success can hide serving-path failure. Scenario: do not use shared-dependency liveness checks as a substitute for graceful degradation.

11. Spring Cloud Gateway: Match, Filter, Route

Why it exists: edge routing, coarse authentication/quotas and request policy benefit from a consistent boundary. A route associates matching predicates with a destination and filters. Predicates select requests; filters modify or short-circuit processing before/after forwarding. This does not make the gateway the owner of every service's business authorization or transaction.

Gateway has distinct server variants, including WebFlux and MVC arrangements in the relevant ecosystem. Do not mix their APIs/property examples or assume the same runtime behavior. In the reactive server, blocking JDBC or synchronous Feign work on an event loop is dangerous. Choose the stack deliberately and use a compatible client/execution model.

Diagram 03.8



Filter ordering matters. A high-priority filter's pre-work can happen early and post-work late as the chain unwinds. A routing or response-body filter cannot assume a body is freely reusable: buffering, streaming, size limits and data-buffer ownership matter in a reactive stack. Retrying a large upload or rewriting an SSE response can consume memory or break streaming semantics. [05](#) owns real-time protocol constraints.

Rate Limiting Needs a Trusted Key and Failure Policy

A quota filter may use a Redis-backed token-bucket implementation for shared decisions across gateway replicas. The key resolver must derive the intended authorized scope, not accept any header as a tenant identity. Decide how anonymous/empty-key requests behave, how provider outages affect quota checks, and whether rejection is retryable under the API contract. A local limit multiplied across replicas is not a global guarantee.

Coarse ingress quota, provider-specific outbound rate, and concurrent in-flight calls are different limits. A gateway request rate cannot by itself stop a single admitted bulk job from issuing too many provider calls. [10](#) connects token buckets, retries and bulkheads; [02](#) distinguishes concurrency permits from rate per interval.

Edge choice	Use when	Avoid when
Gateway route predicate/filter	Shared application-edge policy is genuinely required	The gateway accumulates every domain decision and direct service access bypasses security
Redis-backed rate decision	Replicas need coordinated coarse quota state	Redis failure behavior, trusted keys and hot-key load are undefined
Infrastructure ingress/load balancer	Standard TLS/routing/traffic distribution is sufficient	It is assumed to implement domain authorization or exactly-once processing
Dedicated application endpoint	The operation needs service-owned state/invariants	A complex domain workflow is disguised as a routing filter

[VERIFY: Gateway variant, starter names, route/filter configuration namespaces, filter ordering and rate-limiter defaults are release-specific. Check the pinned Spring Cloud Gateway documentation for the chosen WebFlux or MVC server; no route, Redis limiter or streaming proxy was executed.]

Production failure: retries exist at browser, gateway, Feign and worker levels. One logical sync trigger becomes many attempts, exhausting a downstream quota and amplifying uncertain writes. Give retries explicit ownership, share deadline budgets and require the operation's idempotency contract. A gateway retry cannot infer that a timed-out downstream request made no change.

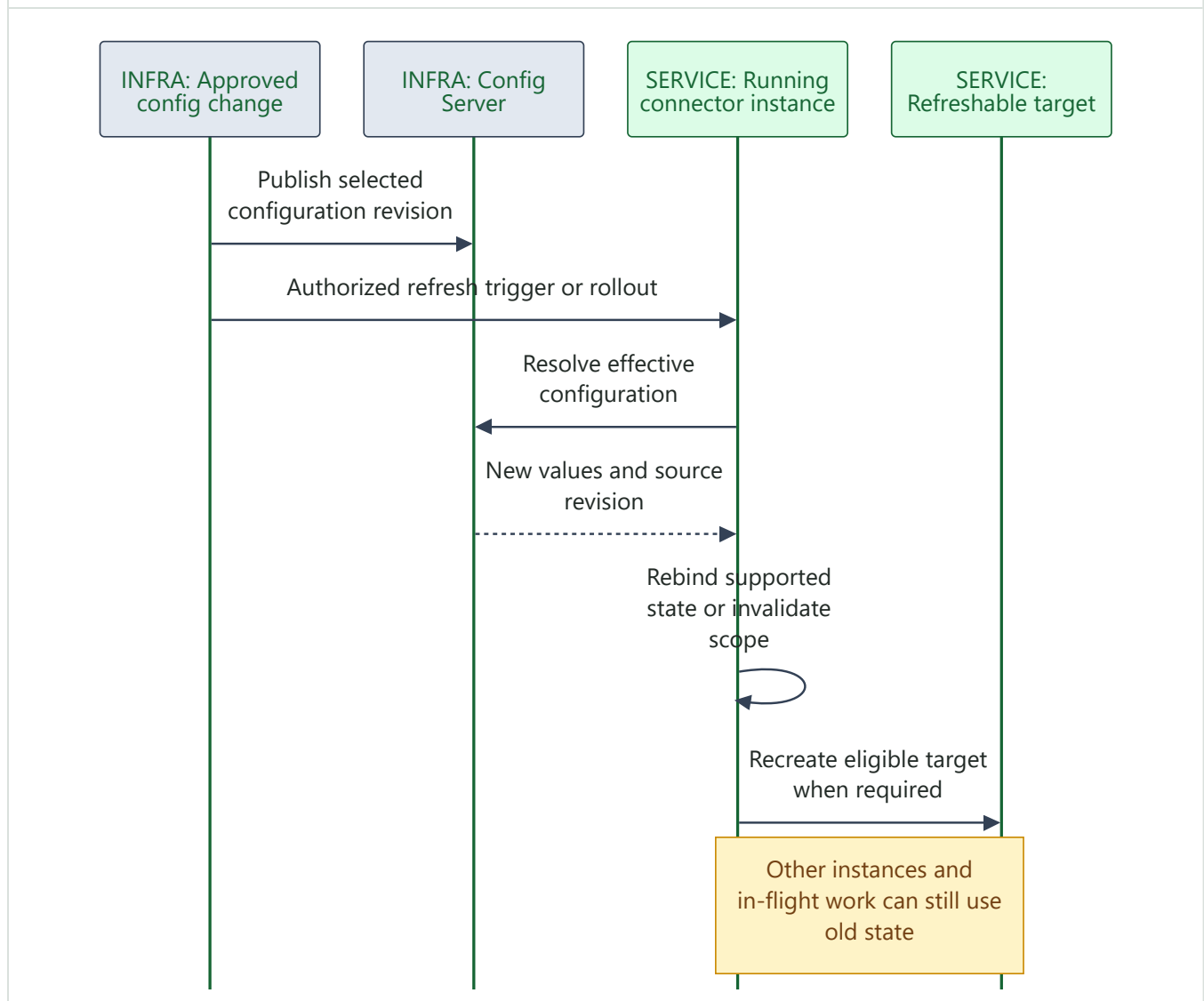
Interview checks: basic: predicates match; filters apply behavior. Internals: chain order affects pre/post execution. Debug: check path rewriting, trusted headers, timeouts and body buffering. Scenario: central quotas complement, not replace, tenant checks and outbound provider limits.

12. Config Server and Refresh: Distribution Is Not Atomic Change

Why it exists: many services may need versioned external configuration, environment/profile selection and consistent retrieval from a central source. Config Server provides a configuration API over a backend such as Git; clients resolve/import configuration into their Environment according to the configured protocol. A Git commit is not an automatic simultaneous change in all running services.

Modern config-data imports differ from legacy bootstrap patterns. Decide whether remote config is mandatory for startup or optional, and understand the effect on outage behavior. An optional import can allow startup with local defaults; that is dangerous when the missing remote value is safety-critical. Secure config retrieval and backend access; treating the config repository as a secret store requires a separate security analysis.

Diagram 03.9



RefreshScope commonly works through a proxy around a lazily initialized target whose cached instance can be invalidated/recreated. Not every bean is refreshable, and changing an Environment entry is not proof that a constructor-injected field changed. Some immutable objects, client resources and AOT/native arrangements have restrictions. Requests in flight may still hold/use old state, so safe refresh requires resource ownership and rollout reasoning.

Spring Cloud Bus can distribute refresh-related events using messaging when configured; it does not turn an asynchronous fleet into an atomic transaction. Protect refresh endpoints, audit revision changes and observe effective configuration per instance. Treat failed/bad

refresh as a release incident, with a rollback path and compatibility window.

Configuration distribution	Use when	Avoid when
Config Server with versioned backend	Central application-aware/profile-aware config retrieval is needed	It duplicates simpler platform delivery without an ownership/security benefit
Refreshable targeted bean	Recreating that resource is supported and in-flight behavior is defined	Updating unrelated state assumes a fleet-wide atomic switch
Immutable deployment rollout	Reproducible config/image combinations and controlled rollback matter	The deployment must change safe live parameters too frequently for this policy
Kubernetes ConfigMap/Secret delivery	Platform-native configuration distribution fits	Mounted/env values are assumed automatically rebound into all Spring beans

[VERIFY: Config Client import/bootstrap behavior, refresh/rebind support, RefreshScope restrictions, Bus integration and management endpoint exposure vary by Cloud/Boot version. Check the selected Config/Common documentation and test the specific bean/resource; no Config Server or refresh cycle was executed.]

Production failure: an old connector instance uses one schema mapping while a refreshed peer uses another against the same checkpoint format. Correctly delivered configuration still breaks the workflow. Version the mapping/checkpoint contract and coordinate compatible transitions through [13](#), [16](#) and the later capstone.

Interview checks: basic: retrieval, binding and resource recreation are separate. Internals: a refresh proxy does not mutate every reference already held by callers. Debug: inspect effective revision per instance. Scenario: prefer a controlled rollout when a live mixed-version window is unsafe.

13. OpenFeign: A Declarative Interface Still Makes a Network Call

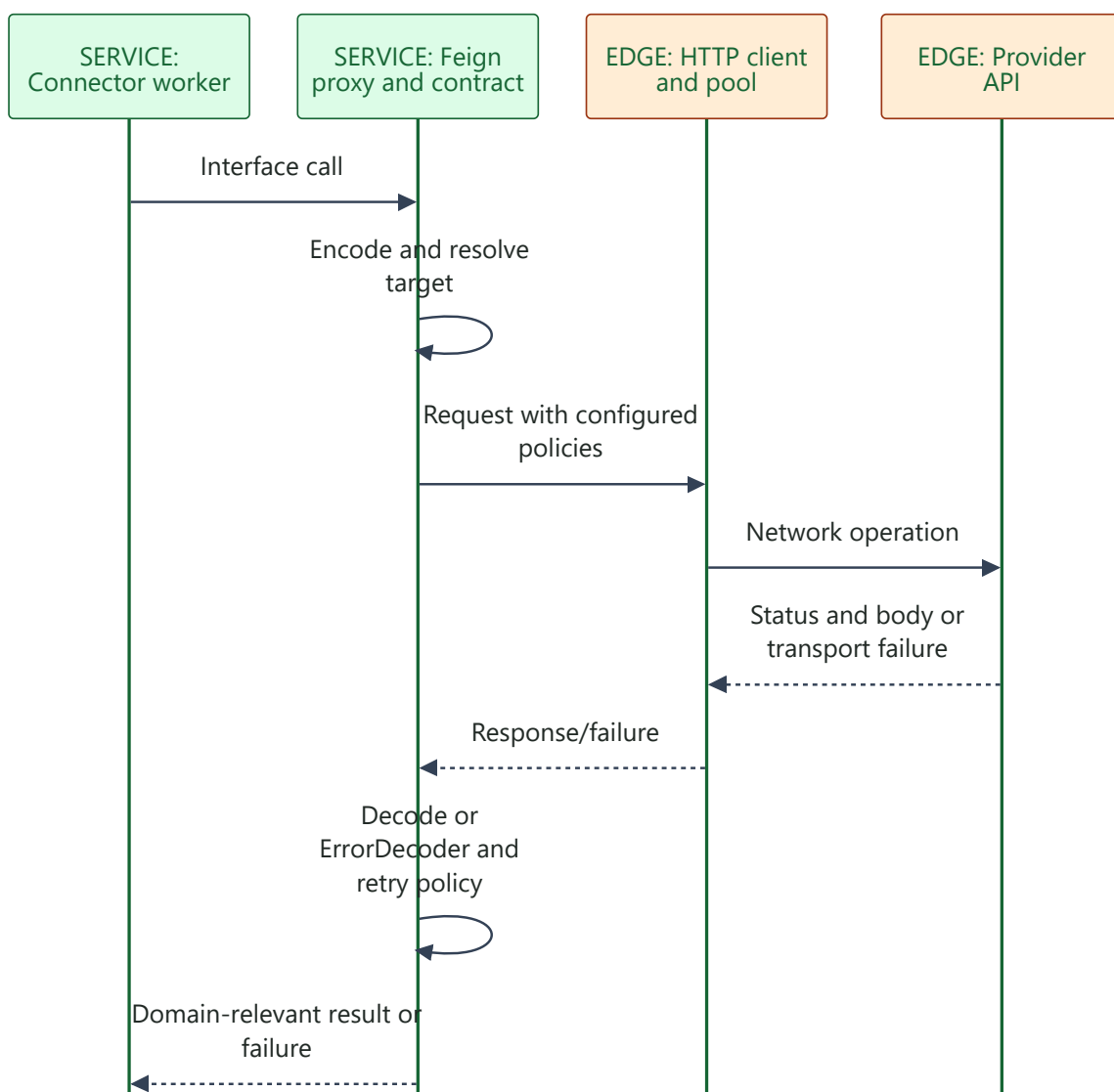
Why it exists: named client interfaces can centralize remote HTTP contracts and client configuration. Spring Cloud integrates the interface contract, encoder/decoder, interceptors, underlying client, error handling and optional load-balancing/circuit-breaker features. A locally typed interface does not make the remote server trustworthy, available or transactionally attached to the caller.

Invocation Trace

- The injected client proxy receives an interface method call and interprets its supported annotations/contract metadata.

- Arguments become path/query/header/body data through the selected encoding rules.
- Resolve a fixed URL or a logical service name through the configured destination/load-balancing path.
- Acquire/connect through the underlying HTTP client's pool and send the request under defined timeouts.
- Decode an acceptable response or invoke error handling for non-success responses under the configured contract.
- Apply any explicit retry/circuit/fallback policy and return a value or propagate a meaningful failure.

Diagram 03.10



An explicitly supplied URL generally bypasses the logical-name client load-balancing resolution described by Spring Cloud OpenFeign. If name-based load balancing is required, the appropriate LoadBalancer dependency/configuration must exist; declaring a client name is not

the same as discovering instances successfully. Avoid sharing a server controller interface indiscriminately with clients: coupling annotation quirks and response models can make independent contract evolution harder.

Timeouts and Retries Need One Owner

Connect timeout concerns establishing a connection under the client's semantics; read/response timeout concerns waiting for response data. DNS, TLS, pool-acquisition waits and overall deadlines can be separate or library-dependent. Do not claim that one `connectTimeout` property bounds the entire request, or that a read timeout means the remote handler did nothing. [21](#) provides the network trace.

The consulted Spring Cloud OpenFeign reference creates a `Retryer.NEVER_RETRY` bean by default, unlike assumptions often copied from core Feign behavior. A `RetryableException` alone does not mean retries occur if the configured retryer refuses them. Choose which failures and operations may retry, cap attempts under a shared deadline, apply backoff/jitter where appropriate, and coordinate with gateway/worker retries.

`ErrorDecoder` translates relevant HTTP error responses into exceptions; transport failures may follow a different path. A fallback should express a valid degraded business result, not convert failed data retrieval into an empty successful batch that advances a checkpoint. Full Feign logging can expose authorization headers and bodies; keep logs minimized/redacted and consistent with [08](#).

Client policy	Use when	Avoid when
Per-client timeout/pool configuration	Providers have distinct latency and capacity contracts	Global defaults accidentally constrain every client identically
<code>ErrorDecoder</code> with typed failures	Caller needs a stable classification of provider HTTP errors	All errors become retryable or sensitive bodies enter exception text
Explicit retry policy	Operation is safe to repeat under a bounded deadline	Several layers multiply attempts or external effects are uncertain
Circuit-breaker integration	Sustained failures require a defined open/half-open policy	Merely declaring Feign is assumed to install/enable the whole policy
Fallback	A genuinely correct degraded result exists	Failure is disguised as successful empty data or authorized absence

Spring Cloud OpenFeign is described by its current reference as feature-complete, with Spring HTTP Service Clients suggested for new evolution, including blocking/reactive choices. That is not a command to rewrite every working client immediately. Existing code still needs sound

timeout, pool, retry and contract testing. Evaluate migration against required features and supported versions rather than assuming declarative HTTP clients are interchangeable.

[VERIFY: OpenFeign defaults, client selection, Retryer.NEVER_RETRY, ErrorDecoder behavior, URL/load-balancer resolution, refresh support and feature-complete status are release-sensitive. The official reference was consulted; underlying HTTP-client timeout/cancellation semantics and every remote invocation remain untested.]

Production incident: a custom client bean is recreated frequently, creating extra connection pools instead of reusing managed resources. Another common path shares one undersized pool across slow providers and unrelated calls. Inspect client bean lifetime, pool acquisition, active connections, eviction and shutdown. A declarative interface has not removed those network resources.

Interview checks: basic: Feign is still an HTTP client call. Internals: contract, encoder/decoder, client and policy layers are separate. Debug: locate pool wait versus connect/read failure. Scenario: a timeout needs idempotency/reconciliation reasoning, not a blanket retry annotation.

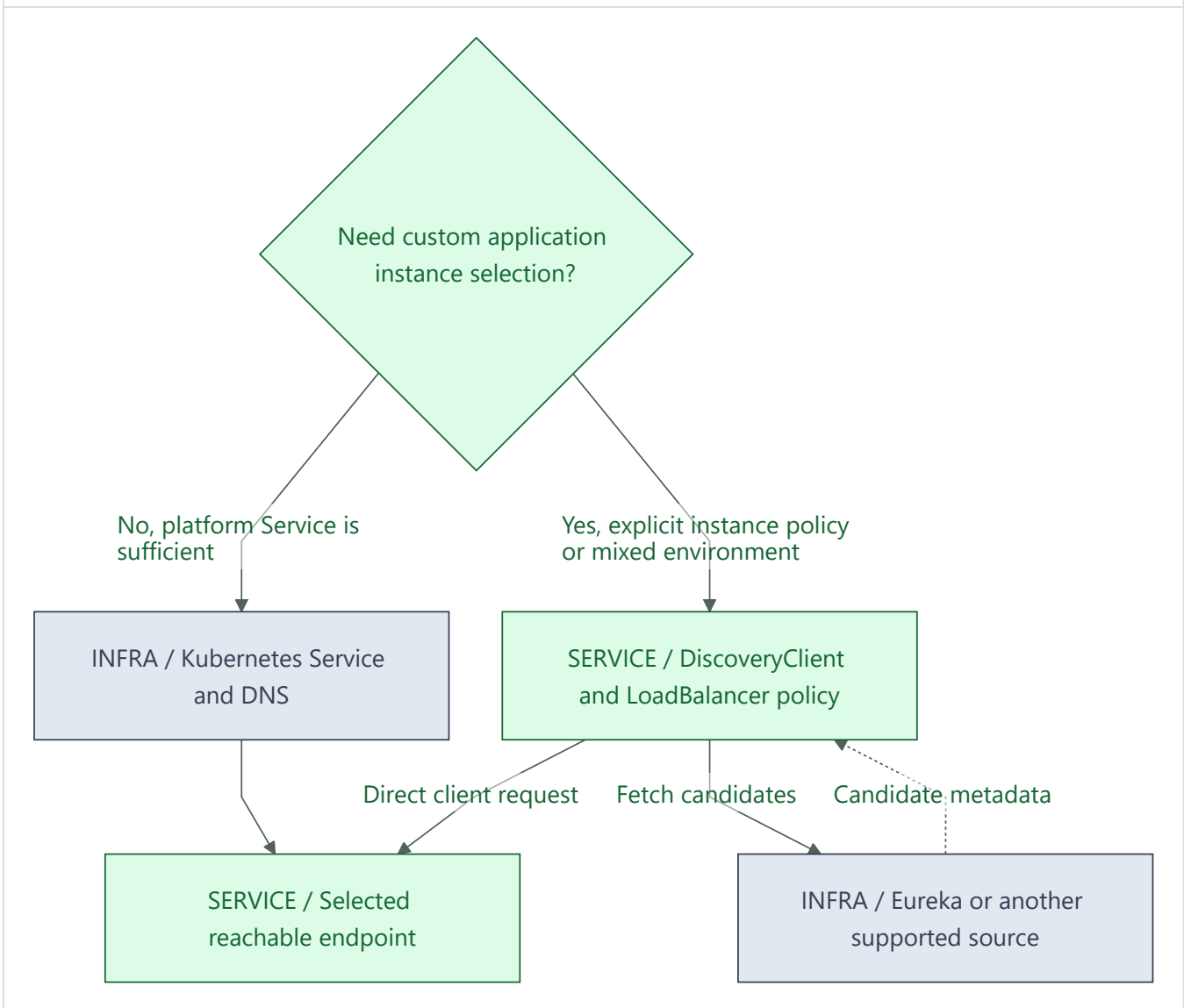
14. Discovery and Load Balancing: What Kubernetes Makes Unnecessary

Why discovery exists: callers need a stable way to find healthy/relevant destinations while instances move. A registry and a load balancer solve different steps: discovery supplies candidates; selection chooses where to send work. Neither guarantees that the chosen instance survives until the request completes.

Eureka is an application-level registry with client registration/renewal and client-side discovery behavior. It fits some mixed or non-Kubernetes environments, but lease expiry, registry availability, stale caches and rollout timing are part of its operational contract. A discovered address can already be stale; clients still need timeouts and safe failure handling.

In Kubernetes, Services and DNS supply stable discovery/routing abstractions, backed by current endpoints and readiness-related behavior. A normal ClusterIP service path typically distributes connections through the cluster's data plane, not through Spring choosing a Pod on every application call. A headless Service exposes addresses for clients that need different discovery behavior. Exact routing depends on the cluster implementation and protocol.

Diagram 03.11



Client-Side Versus Server-Side Selection

With client-side load balancing, the client obtains an instance set and applies its selection policy before making the request. The registry supplies metadata; it is not the HTTP forwarding hop. With an infrastructure/server-side proxy or virtual service address, the client sends to that address and the infrastructure chooses a backend according to its data-plane behavior. Both have availability and observability costs; simply adding both can create unnecessary indirection or confusing locality policies.

Persistent HTTP connections complicate the shorthand "round robin every request." A connection established through an L4 service path can continue reaching the same backend. HTTP/2 multiplexes requests over a connection, so traffic distribution depends on client connection behavior and any L7 proxy, not just the number of Pods. DNS caching and headless discovery introduce additional staleness considerations explained in 21.

Capability	Kubernetes may already supply	Use an additional Spring/Cloud component when	Avoid when
Basic in-cluster discovery	Service naming and DNS	App-level instance metadata/selection or mixed infrastructure requires it	Eureka duplicates the same endpoint directory without a justified policy
Traffic distribution	Service data plane and/or ingress/L7 proxy	Client-side selection has a defined routing/locality requirement	Two load-balancing layers conflict or make diagnosis opaque
Configuration delivery	ConfigMaps, Secrets, volumes and rollout inputs	Config Server adds needed profile/version/backend behavior across environments	It is kept solely because all microservices are assumed to need it
Restart and replica management	Controllers, probes and rollout machinery	Application lifecycle still must own safe work drain and recovery	Platform restarts are mistaken for business retries or transaction recovery
Edge routing/TLS	Ingress, Gateway API implementations or external load balancers	Application-edge filters/token/session/quotas require a custom gateway	Two gateways repeat policy without clear responsibility

Kubernetes does **not** make application authorization, provider-specific retries, idempotency, database invariants, circuit-breaker policy or a good API contract unnecessary. A service mesh can add transport policy and telemetry, but it does not know every domain-side effect. Conversely, not every Spring service on Kubernetes needs Eureka, Config Server and a custom gateway merely to be called a microservice.

[VERIFY: Eureka lease/cache behavior, Spring Cloud LoadBalancer selection/caching and Kubernetes DNS/Service/endpoint routing depend on release and deployment. Check the actual client and cluster data plane; no registry, Kubernetes service or multi-Pod load-distribution experiment was run.]

Production failure: scaling from two to six Pods does not evenly spread traffic because clients retain a small number of long-lived connections. Measure per-backend load and connection behavior, inspect the routing layer, and choose a compatible client/proxy strategy. Do not conclude that Spring ignored the replica count or claim a universal round-robin guarantee.

Interview checks: basic: discovery and selection are separate. Internals: a Service can balance connections rather than each logical request. Debug: inspect stale endpoints, readiness and connection reuse. Scenario: begin with the platform's simplest adequate path and justify every extra registry/proxy.

15. Labs, Review Boundaries and a Debugging Routine

The Maven lab is a small Framework application, not a Boot web application. It uses Boot/Cloud dependency management to state an explicit compatibility baseline, but imports only the Framework/JDBC/H2 pieces needed for its mechanisms. No Cloud feature becomes tested because its BOM is imported.

Source	Assertions present	Not established
ProxyBoundaryLab.java	Interface-proxy advice on external calls; self-invocation bypass	Boot default proxy selection, weaving, method security or HTTP routing
ContainerTransactionLab.java	Singleton identity, init/destroy, explicit class proxy, transaction-active check, external commit/rollback, self-call bypass, rollback-only participation	PostgreSQL isolation, production pool behavior, MVC/Security/Actuator, Gateway/Config/Feign or distributed consistency
pom.xml and offline runner	Fixed parent/BOM/plugin declarations; offline invocation; explicit PASS/FAIL/NOT EXECUTED reporting	Dependency availability/resolution, Java compilation or runtime success without tools/cache

The runner's preflight actually reported NOT EXECUTED. The POM was parsed as XML, which checks structure well-formedness rather than Maven resolution. The chapter's full inline Java listing is checked against the saved file. These checks cannot substitute for a JDK, Maven and real Spring/H2 execution. No fake console transcript, request response or SQL trace is included.

Trace an Annotation That Appears to Do Nothing

- Identify the exact concern: lifecycle, transaction, caching, async, validation or authorization. Similar-looking annotations use different infrastructure.
- Confirm registration in the intended context and that required infrastructure/dependencies are active.
- Inspect the exposed object and invocation path: managed proxy, unmanaged new, self-call, eligible method and advice order.
- Inspect effective configuration, qualifiers/managers and condition evaluation rather than only source annotations.
- Follow resource boundaries: thread, database connection, HTTP call, broker publication and response serialization.
- Add a negative-path test that distinguishes the claim, such as a write followed by rollback-triggering failure, a denied tenant request or an unmatched security chain.

Tests should scale with the claim. A pure unit test checks domain decisions; a context test checks wiring/proxy behavior; an MVC test checks conversion/errors; an authenticated integration test checks security; a database-backed test checks actual persistence semantics;

deployment tests check gateway/config/discovery behavior. [19](#) details test slices and Testcontainers. A happy-path mocked repository test cannot prove rollback.

Explained-Here Index and Sources

Concept	Explanation
IoC, injection, scopes and bean definitions	Container ownership
Bean lifecycle and post-processors	Lifecycle
Proxies, AOP, self-invocation and configuration enhancement	Proxy boundary
Boot startup, starters and auto-configuration	Conditional configuration
External configuration and typed binding	Configuration
DispatcherServlet, MVC conversion and errors	Request lifecycle
Validation, authorization and durable constraints	Validation boundaries
Transactional, managers, propagation and rollback-only	Transaction mechanism
SecurityFilterChain and tenant authorization	Spring Security
Actuator, endpoint exposure and probes	Operational surfaces
Gateway routes, predicates, filters and rate limits	Gateway
Config Server and refresh	Configuration distribution
OpenFeign timeouts, decoding and retry policy	Remote clients
Eureka, Kubernetes discovery and load balancing	Discovery decisions
Runnable source and execution limitations	Labs and diagnosis

Primary references consulted in this chapter: [Spring Cloud compatibility](#), [Boot 4.0 system requirements](#), [Framework proxy mechanisms](#), [transaction annotations](#), [Boot auto-configuration](#), and [OpenFeign reference](#). Additional owning documentation for unexecuted integrations: [Spring Security servlet architecture](#), [Spring Cloud Gateway](#), [Spring Cloud Config](#) and [Boot Actuator reference](#). Listing an owning reference does not claim every page was fetched or every integration ran.

Related Chapters

Return to [00 IntegrationHub](#); use [01 Java/JVM](#) and [02 Concurrency](#) beneath these mechanisms. Continue to [04 JPA](#), then the [06 database](#) and [05 API contracts](#). [07 Messaging](#), [08 Security](#), [10 Resilience](#), [15 Kubernetes](#), [18 Operations](#), [19 Testing](#) and [21 Network/OS](#) deepen the

boundaries. Generated reciprocal links cover every explicitly referenced destination.

RELATED CHAPTERS / RECIPROCAL LINKS

00	The System Before the Details	23	JVM Performance Engineering
01	Core Java and the JVM	18	Observability and Reliability
02	Concurrency and the Memory Model	10	System Design Foundations
21	Networking and Operating Systems	13	Data Integration and iPaaS
07	Messaging, Batch and Streams	16	Build, Ship and Roll Back
08	Security, Identity and Data Protection	26	Data Platform Basics
15	Kubernetes and Production Workloads	25	Architecture Styles and Domain Design
22	Distributed Systems Theory	11	Object Design, Patterns and LLD
05	APIs and Real-Time Communication	12	Twelve System Design Case Studies
04	Hibernate and JPA	27	Fintech and Payments Engineering
19	Testing and Quality	24	IntegrationHub in Motion
06	Databases and Durable State	20	Appendix: Interview Toolkit and Index

One-Page Cheat Sheet

Container: definitions describe construction; managed instances have lifecycle and scope. Constructor injection exposes required dependencies. Singleton is per definition/context, not automatically thread-safe. Definition processors and instance processors are different extension points.

Proxy: client -> interceptor chain -> target. Target self-calls bypass usual proxy advice. JDK proxies expose interfaces; subclass proxies cannot override final/private methods. Configuration-method enhancement is separate from a service's transaction proxy.

Boot: starters supply libraries, BOMs manage versions, conditions contribute infrastructure definitions. A custom bean may trigger backoff, not a universal override. Check the condition report and effective properties. Boot 3 and Boot 4 package/starter examples are not interchangeable.

HTTP: servlet filters/security -> MVC mapping/adapter -> argument conversion/validation -> controller -> service proxy -> response conversion. Earlier filter failures and already committed responses are not universally repairable by ControllerAdvice. Tenant authorization belongs in the service contract too.

Transaction: metadata + intercepted call + chosen manager + participating resource. REQUIRED can share a rollback-only transaction; REQUIRES_NEW can need another connection. Read-only/timeout are not universal enforcement. A response failure after commit leaves an uncertain client outcome; use the idempotency contract.

Operations: secure Actuator access separately from endpoint enablement/exposure. Liveness is not readiness; a healthy management port is not a completed business workflow. Keep metrics bounded and sensitive diagnostics restricted.

Cloud: Gateway matches/filters/routes; Config retrieves/rebinds/recreates eligible state; Feign still performs real HTTP and defaults to no retry in the consulted Spring Cloud reference. Kubernetes often supplies basic discovery/distribution, not business authorization or idempotency. Both Spring labs are NOT EXECUTED.

Interview Corner

Basic: What Does the IoC Container Actually Manage?

It registers definitions, resolves dependencies, constructs/configures objects and applies relevant scope/lifecycle/post-processing. It does not automatically make every new object managed or every singleton thread-safe. Business invariants remain the application's responsibility.

Internals: BeanFactoryPostProcessor Versus BeanPostProcessor?

The former works on definition/container metadata before ordinary instance creation; the latter works around managed instances and can contribute wrapping such as proxies. Specialized variants add earlier hooks. Mixing them up obscures why a definition changed versus why the injected object is a proxy.

Trace/Debug: Why Did Injecting a Prototype Not Create One per Request?

The singleton's dependency was normally resolved when it was injected. That reference can be reused. If each operation requires a new instance, use an explicit provider/factory or appropriate scope boundary and define who cleans up the resource.

Scenario: Where Should Tenant-Specific Request State Live?

In an authorized request/task context or local value passed through the operation, not an unsynchronized mutable singleton field. Shared connector configuration can be immutable; credentials and job decisions still need per-call scope and access checks.

Basic: Why Does Transactional on a Private Self-Called Method Fail to Help?

The ordinary call does not cross the exposed proxy, and private methods are not eligible for subclass overriding. The annotation alone does not create an interception boundary. Put the transaction at a suitable externally invoked managed method or use deliberate programmatic control.

Internals: Can a Final Class Be Behind a Spring Proxy?

An interface-based JDK proxy can delegate to a final implementation. A class-based proxy cannot subclass a final class. State the proxy mechanism before applying the final/private restrictions; inspect the actual exposed type and configuration.

Trace/Debug: Why Does a New Service Instance Bypass AOP?

Constructing the target directly does not apply the container's auto-proxy infrastructure. The production caller may receive a managed proxy while a unit test receives a plain object. Separate domain tests from framework-boundary tests so both claims are explicit.

Scenario: Retry Outside or Inside the Transaction?

It depends on the invariant, but retrying a local failed transaction usually requires a fresh transaction per attempt and safe repeatability. If retries happen inside one rollback-only transaction, later attempts cannot simply restore its commit capability. Also account for remote effects that database rollback cannot undo.

Basic: What Is Auto-Configuration?

Registered candidate configuration contributes bean definitions when its classpath, environment, application-type and bean conditions match. It is not arbitrary classpath scanning or a second IoC container. User configuration can cause specific defaults to back off.

Internals: Does Importing a Cloud BOM Enable Gateway?

No. A BOM supplies version management, not runtime libraries, route definitions or a running server. Add only required dependencies and configuration, verify compatibility and test the behavior. The chapter lab imports the BOM but does not run any Cloud service.

Trace/Debug: An Infrastructure Bean Disappeared After a Change. What First?

Check resolved dependencies, active profiles, effective properties, exclusions and competing beans, then read the condition evaluation report. Determine whether the definition was never contributed, failed during creation or was replaced/backed off. Do not begin by enabling global bean overriding.

Scenario: Is It Safe to Copy Boot 3 Auto-Configuration Imports into Boot 4?

Not without checking the targeted modules/packages and supported API. Boot 4 reorganized some infrastructure. Match the exact dependency line, consult its reference and compile/test. A blanket javax-to-jakarta replacement is also wrong for Java SE types such as DataSource.

Basic: What Happens Before the Controller Method Runs?

The container's filters, relevant security chain, MVC handler mapping/adaptation, argument resolution, conversion and validation can all act first. A rejected request may never call the controller. Trace the boundary at which it stopped.

Internals: Can ControllerAdvice Handle Every Authentication Failure?

No. Servlet/security filters can fail or reject before MVC handling, and their error paths are configured separately. ControllerAdvice applies to relevant MVC exception resolution, not every component in the HTTP stack or a response already committed.

Trace/Debug: Why Did Two Validated Requests Create a Duplicate-Key Error?

Both can pass a pre-insert existence check before either transaction commits. Validation did not serialize the writes. Keep the database constraint as authority and translate its conflict into the documented tenant-scoped idempotency/API outcome.

Scenario: The Service Committed but Response Serialization Failed. What Now?

Do not assume the transaction rolled back. The client has an uncertain outcome and should retry/query under the same logical idempotency contract. Server-side response handling cannot undo an already committed transaction merely by returning a different status.

Basic: What Makes a Method Transactional at Runtime?

Eligible metadata must be consumed by active transaction infrastructure; the call must traverse the intended proxy or weaving boundary; the chosen manager must control the resources being used. A directly constructed annotated class alone supplies none of that interception.

Internals: Why Does Catching an Inner Failure Still Produce UnexpectedRollbackException?

The inner REQUIRED call may have marked the shared transaction rollback-only when its exception crossed its interceptor. The outer catch does not clear that state. The later outer commit attempt reports that it could not commit as the caller expected.

Trace/Debug: Why Did a Checked Exception Commit?

Under the usual default rollback rules, ordinary checked exceptions do not trigger rollback. Check explicit method rules and any global rollback configuration as well as whether the exception reached the interceptor. Do not generalize the default to all Spring applications.

Scenario: Can REQUIRES_NEW Fix Every Transaction Problem?

No. It creates an independent transaction where supported, can consume additional resources while the outer transaction is suspended, and can violate the intended all-or-nothing business outcome. Choose it for a deliberate independent-commit requirement and test pool capacity/failure behavior.

Basic: Authentication Versus Tenant Authorization?

Authentication establishes a principal under configured trust checks. Authorization decides whether that principal can act on this tenant/resource. A valid token or successful gateway check does not automatically authorize any job ID supplied by the caller.

Internals: How Do Multiple SecurityFilterChains Interact?

Request matching and order select an applicable chain; they are not all concatenated into one universal rule set. A broad earlier match can shadow another chain. Validate the actual path, including management endpoints and deliberate fallbacks.

Trace/Debug: Actuator Is Healthy but Users Cannot Reach the API. Why?

The management path may use a different port, context or dependency policy. It can be responsive while the serving connector, route or business dependency fails. Check the main traffic path and appropriate readiness/business signals, not just one health response.

Scenario: Should a Database Outage Fail Liveness on Every Pod?

Usually not as an automatic rule: restarting application Pods does not repair a shared database and can amplify the outage. Choose liveness for restart-worthy process failures; design readiness and degradation from the service contract.

Basic: Gateway Predicate Versus Filter?

A predicate participates in selecting a route; a filter changes, surrounds or short-circuits processing. Order affects pre/post behavior. Neither substitutes for service-owned authorization or business invariants.

Internals: Why Can Config Refresh Leave Mixed Behavior?

Retrieval, rebinding and recreation are separate; only eligible targets refresh, in-flight references may keep old state, and other instances may update later. A fleet refresh is not an atomic distributed commit. Version and observe the configuration transition.

Trace/Debug: Why Didn't Feign Retry a RetryableException?

The configured retry policy may be `NEVER_RETRY`, which is the consulted Spring Cloud default. Inspect the actual client, `ErrorDecoder`, retryer and surrounding resilience layers. The exception type alone does not force a retry or make repeating the operation safe.

Scenario: Should Every Kubernetes Service Register with Eureka?

No. Kubernetes Service/DNS may already satisfy discovery needs. Add an application registry only for a concrete policy or mixed-environment requirement. Keep client timeouts and endpoint-staleness handling regardless of registry choice.

Basic: Does a Kubernetes Service Balance Every HTTP Request Equally?

Not universally. L4 connection selection plus persistent connections or HTTP/2 can concentrate many requests on one backend. A client-side selector or L7 proxy has different behavior. Inspect the actual network and connection layer.

Internals: Why Are Connect and Read Timeouts Not the Whole Deadline?

DNS, pool acquisition, TLS, retries and application processing can consume time outside or differently within those settings, depending on the underlying client. Define an overall deadline and verify the library's precise timeout/cancellation contract.

Trace/Debug: How Would You Test a Transaction Claim?

Invoke the managed bean through its real proxy against an appropriate database, perform a write, trigger a relevant failure, then inspect durable state from a proper boundary. A mocked repository call count cannot prove commit/rollback, and these supplied labs have not yet run.

Scenario: Which Cloud Components Would You Keep for IntegrationHub?

Keep each only for a named responsibility: gateway for required edge policy, Config Server if its distribution/version behavior adds value, declarative clients with explicit timeout/retry ownership, and discovery only beyond what the platform already provides. Preserve service authorization, durable outbox/idempotency and observability independently of that choice.

04 / Hibernate and JPA

An object graph is not a database transaction. IntegrationHub accepts jobs as durable facts, loads connector configuration to execute them, and updates progress under concurrent workers. JPA gives that Java code a persistence contract; Hibernate implements it with mapping, identity management, dirty checking and SQL generation. IntegrationHub is fictional, and this chapter makes no claim about a real employer's schema or performance.

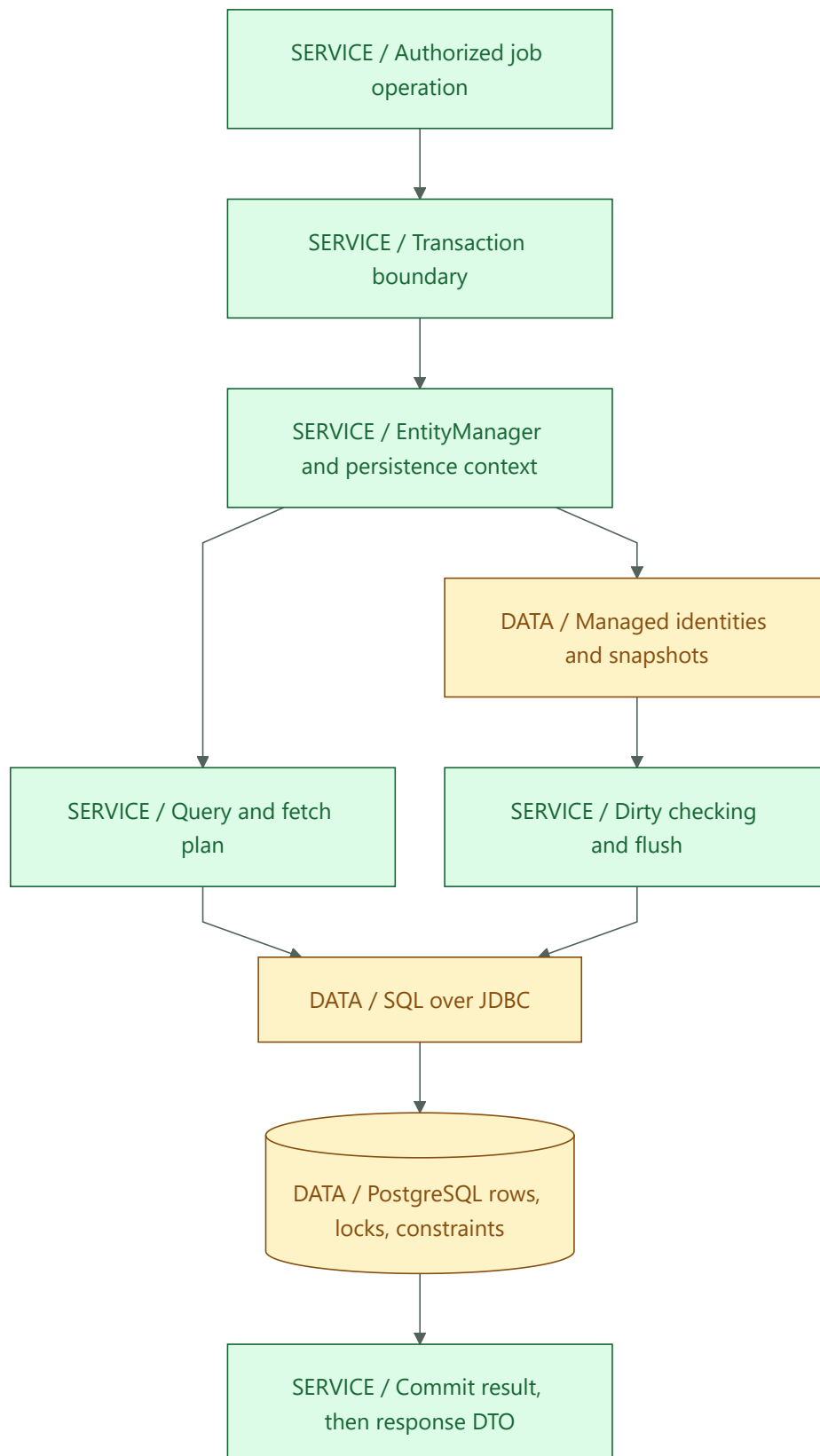
Version assumptions: Java 21, Jakarta Persistence 3.2 and Hibernate ORM 7.2.x under the guide's Boot 4.0.8 parent. The Boot 4.0 managed-dependency table consulted on 2026-10-06 lists jakarta.persistence-api 3.2.0, hibernate-core 7.2.24.Final, Spring Data JPA 4.0.7 and H2 2.4.240. These are documentation observations, not resolved local artifacts. Spring Cloud 2025.1.x remains the compatibility family checked in 03; this standalone lab needs no Cloud library. PostgreSQL 17 is the reference operational database; Kafka 4.x and Kubernetes 1.34 remain surrounding illustrative baselines. H2 is only the lab database.

[VERIFY: recheck the exact Boot-managed Hibernate/Jakarta Persistence/Spring Data versions, provider support and database driver/dialect against the selected artifacts before execution. The published Boot dependency table was consulted, but no Maven resolution or PostgreSQL compatibility test ran.]

Execution status: NOT EXECUTED. Java/Maven and cached ORM dependencies remain unavailable; approval to download Mermaid was limited to diagram dependencies. The offline runner records this explicitly. Complete entity and probe sources are under `code/04-jpa/` ; compilation, Hibernate/H2 assertions and SQL counts are unverified. Diagrams in this edition are rendered locally to embedded SVG, including earlier chapters; syntax highlighting remains unavailable.

Big Picture

Diagram 04.1



The persistence context coordinates Java identities and pending work. The database still owns relational constraints, transaction isolation and durability. A method named `save` is not necessarily an immediate `INSERT`, and `flush` is not `commit`. Those distinctions explain many bugs that appear mysterious when only repository interfaces are visible.

What You Will Be Able to Explain

- Trace new, managed, detached and removed entity states and distinguish `persist` from `merge`.
- Explain first-level identity, dirty checking, flush triggers and transaction completion without claiming every query uses the cache.
- Diagnose lazy-loading failures and N+1 from the actual access pattern and SQL, not from annotation names alone.
- Choose fetch joins, entity graphs, projections or batched loading with pagination and cardinality in mind.
- Explain optimistic versions and pessimistic locks, including what neither protects across unrelated rows or external effects.
- Map associations with clear ownership, cascade and deletion semantics, and avoid mutable identity and tenant-boundary mistakes.
- Distinguish Spring Data convenience from the underlying JPA/provider/database contracts.

[P] PRODUCTION

Where this lives in IntegrationHub. 03's `service transaction` persists a tenant-scoped job and outbox intent; it does not publish to Kafka atomically. 07 owns relay and consumer idempotency. A worker reads connector configuration and updates progress with a version check where appropriate. 06 owns isolation, indexes and SQL plans; 08 owns tenant authorization and secret protection. Returning a JPA entity directly to 09's UI can leak both schema and unexpected lazy queries.

1. JPA, Hibernate, Spring Data and the Database

Why ORM exists: applications often manipulate related domain objects, while relational databases store rows and constraints. An ORM maps between these models and tracks persistence work. It reduces repetitive mapping, but it cannot remove the need to understand joins, transactions, row counts and database cost.

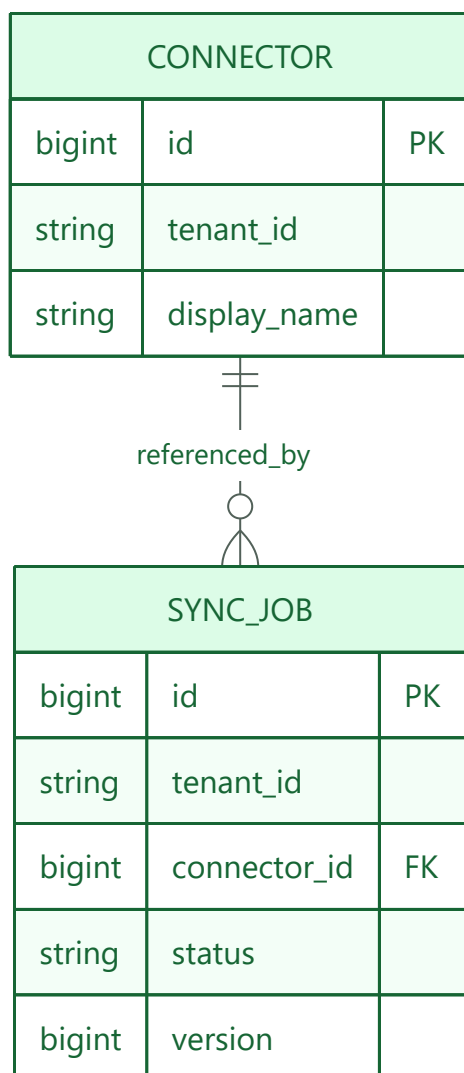
JPA, now Jakarta Persistence, specifies APIs and semantics such as `EntityManager`, entity state, queries and locking. Hibernate is a provider implementing those contracts with extra behavior and features. Spring Data JPA creates repository implementations and integrates common

operations; Spring transaction infrastructure supplies service-level boundaries. JDBC/driver/database layers ultimately execute statements. These are cooperating abstractions, not interchangeable names for the same thing.

EntityManagerFactory is a long-lived factory for a persistence unit. An EntityManager controls a persistence context and is not intended for concurrent application use by unrelated threads. A Spring-injected shared EntityManager reference is commonly a proxy that delegates to an appropriate contextual EntityManager; that proxy does not make one underlying context safe to share freely across tasks.

The Minimal Teaching Model

Diagram 04.2



This is a deliberately small mapping for the lab, not the whole IntegrationHub model. Every fixture job references one connector. Production also needs idempotency, outbox, progress and checkpoint structures from 00. A foreign key to connector_id alone does not prove that both rows have the same tenant. Application authorization and suitable schema-level tenant invariants must complement the relationship.

Access approach	Use when	Avoid when
JPA entity model	Unit-of-work updates and domain relationships fit managed objects	Bulk/reporting operations are forced through enormous managed graphs
DTO/scalar query	A read endpoint needs a bounded, specific shape	You assume it establishes a managed updateable entity graph
JDBC or SQL-oriented access	SQL control, bulk operations or provider-independent query behavior is important	It silently bypasses version/callback/cache contracts expected by ORM writers
Spring Data repository	Standard lookup/query patterns benefit from generated infrastructure	Method names substitute for checking SQL, authorization and transaction scope

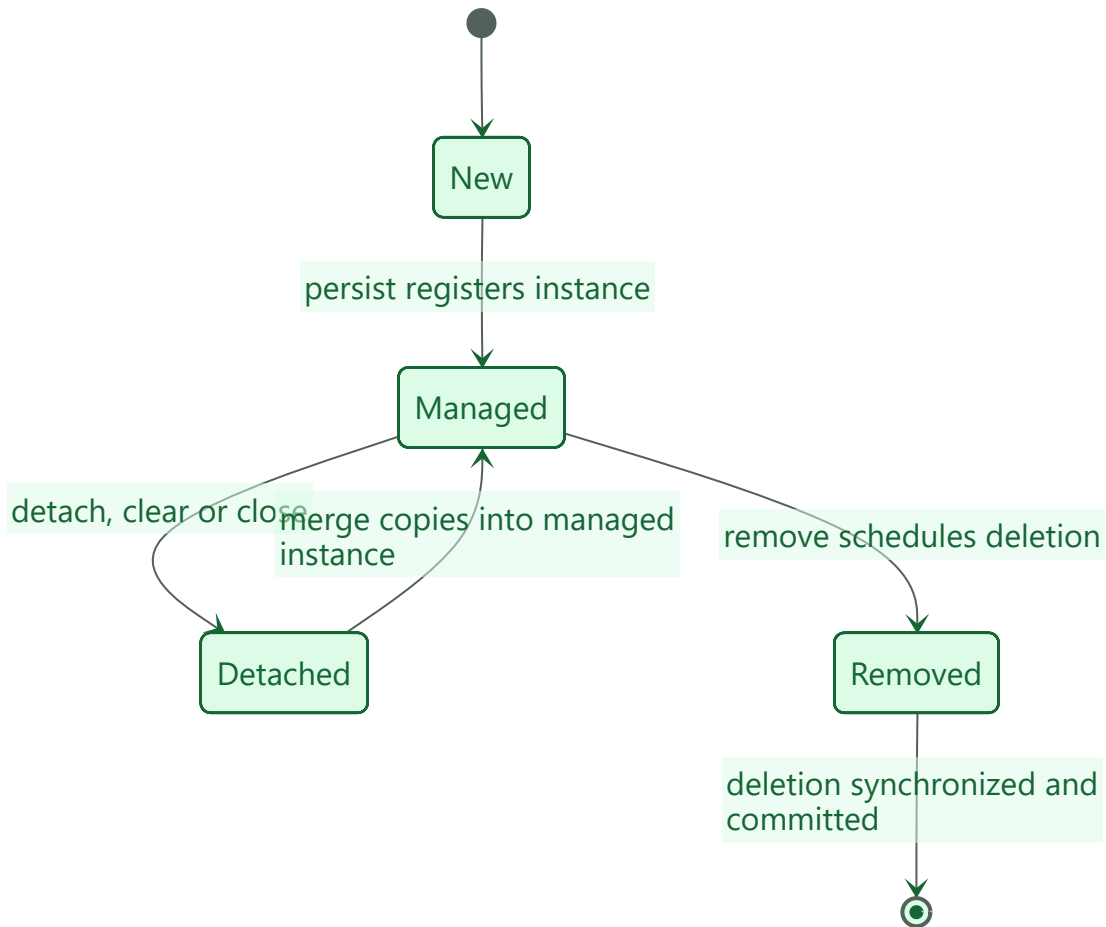
Production trap: parallel tasks share an EntityManager and its managed entities because the service singleton already has a repository field. The repository proxy may be shareable, but the underlying mutable entity graph/context is not a concurrent data structure. Pass IDs or immutable commands, authorize each unit, and define transaction ownership per execution path. See 02.

Interview checks: basic: JPA is the contract and Hibernate is a provider. Internals: context identity is distinct from database state. Debug: inspect the actual provider, manager and transaction boundary. Scenario: use SQL directly when it better matches a bulk/read workload, while preserving the surrounding invariants explicitly.

2. Entity States: Track the Instance You Actually Hold

Why state matters: the provider needs to know which objects participate in change tracking and which rows they represent. A Java object with an ID is not necessarily managed, and an object that was managed earlier may no longer participate in any current unit of work.

Diagram 04.3



The merge arrow describes copying into a managed instance, not magically changing the original detached object into that instance. Deletion timing depends on flush/transaction behavior; the final state in this sketch is an outcome, not a specification that the old Java object has vanished from memory. Rollback is not drawn as an inverse arrow because it does not reliably restore your entire Java object graph to its former field values.

- **New/transient:** created in application code, not associated with a persistence context. It may even have an assigned ID; that alone does not establish management.
- **Managed:** associated with a context under a persistent identity. The provider can track changes and synchronize them when the relevant conditions apply.
- **Detached:** previously persistent identity/state is now outside the context. Mutating it changes only that Java object until an explicit operation copies/applies state through a new managed boundary.
- **Removed:** managed removal has been requested. SQL deletion is coordinated with flush, constraints and transaction completion; remove is not a Java memory-free operation.

persist Versus merge

`persist` makes a suitable new instance managed. It does not promise that the `INSERT` must wait until `commit`; identifier strategies and provider work can cause earlier SQL. `merge` takes the input's state and copies it to a managed instance for that identity, returning the managed result. The original detached input normally remains detached. If the argument is already managed, the result/behavior follows that state; do not promise a different Java reference for every possible merge call.

In JpaLab, a context loads a job and closes, producing a detached input. Another context merges it, then checks that the returned object is managed and the detached input is not. That assertion targets this specific detached-input scenario. Passing a removed entity or an invalid/stale graph has different error behavior; merge is not a universal "make anything valid" command.

Operation	Use when	Avoid when
<code>persist</code>	A new entity should join this unit of work	Detached state is passed indiscriminately as if it were always new
<code>merge</code>	A deliberately accepted detached representation must be reconciled	Untrusted API fields overwrite managed state wholesale
<code>find</code>	Need managed state for a known identity, subject to authorization	ID possession is treated as permission to access another tenant's row
<code>getReference</code>	An association can be represented by identity without requiring state immediately	You assume it validates row existence immediately or eliminates later SQL
<code>detach / clear</code>	The unit of work intentionally releases managed state	Pending changes are discarded accidentally or detached mutation is expected to persist
<code>refresh</code>	Existing managed state should be reloaded from its authority	It overwrites intentional unflushed edits or is treated as a universal lock

Production failure: an API deserializes an entity from a client and calls `merge`. Fields omitted by the client, including relationships or status, can overwrite state unexpectedly; the client may also supply an unauthorized version/tenant/association. Prefer a narrow command DTO, load the authorized entity, and apply only allowed changes inside the service boundary. 03 separates validation from authorization and durable constraints.

Interview checks: basic: contains tests management in a particular context. Internals: `merge` copies state and returns the managed instance. Debug: inspect where `clear/close` detached the object. Scenario: load-and-apply can be safer than blindly merging a web payload.

3. Persistence Context: Identity Map and Unit of Work

For a persistent identity within one context, there is a unique managed entity instance. Repeated find calls for the same identity can return that same Java reference without another entity load. This is the first-level cache/identity-map idea. It enables coherent in-memory updates and dirty tracking within that unit of work.

It does not mean every query is answered without SQL. A JPQL query may still execute to determine matching rows; hydration can then reuse already-managed instances. Query results can therefore reflect a mixture of query membership from the database and field state already held in the context unless the operation's refresh/locking semantics say otherwise. Do not use an old long-lived context as a universal live view of concurrent database changes.

Dirty Checking Trace

- Load or persist an entity into the context, with identity and provider tracking metadata.
- Mutate its mapped state through the application's intended domain operation.
- At synchronization, the provider detects relevant changes. Snapshot comparisons and bytecode-enhanced tracking are implementation techniques, not the same thing as a manual save on every setter.
- Schedule/execute the required SQL, including version predicates where configured.
- On successful transaction completion, the database commits; on failure, the application must handle rollback and context/object-state consequences.

The big-picture diagram's identity/snapshot -> flush -> SQL path is this mechanism. A managed entity changed in a transaction ordinarily does not need a redundant Spring Data save call merely to activate dirty checking. Detached entities do not gain that behavior by calling a setter with the same method name.

Memory cost: the context retains managed objects and tracking state. Reading a huge data set and keeping every entity managed increases heap and flush/dirty-checking work. For batch processing, bound chunks, flush intentionally, then clear where it fits the algorithm. Clearing before needed synchronization can lose pending changes; clearing afterward means subsequent references are detached. [07](#) covers restartable batch contracts and [23](#) covers measurement.

Transaction scope is not universally context scope. Application-managed contexts can span multiple transactions; extended contexts and request-bound arrangements also exist. Spring's usual transaction-scoped usage is a common model, not the only valid JPA setup. A transaction ending and a context closing are different lifecycle events, so describe the actual arrangement before declaring all objects detached at every commit.

Use when / avoid when: use a short, explicit unit of work for coherent domain changes. Avoid a globally shared EntityManager, an unbounded context or entity sharing across threads. If a conflict/rollback occurs, re-evaluate from a clean transaction/context rather than assuming the mutated Java objects automatically reverted.

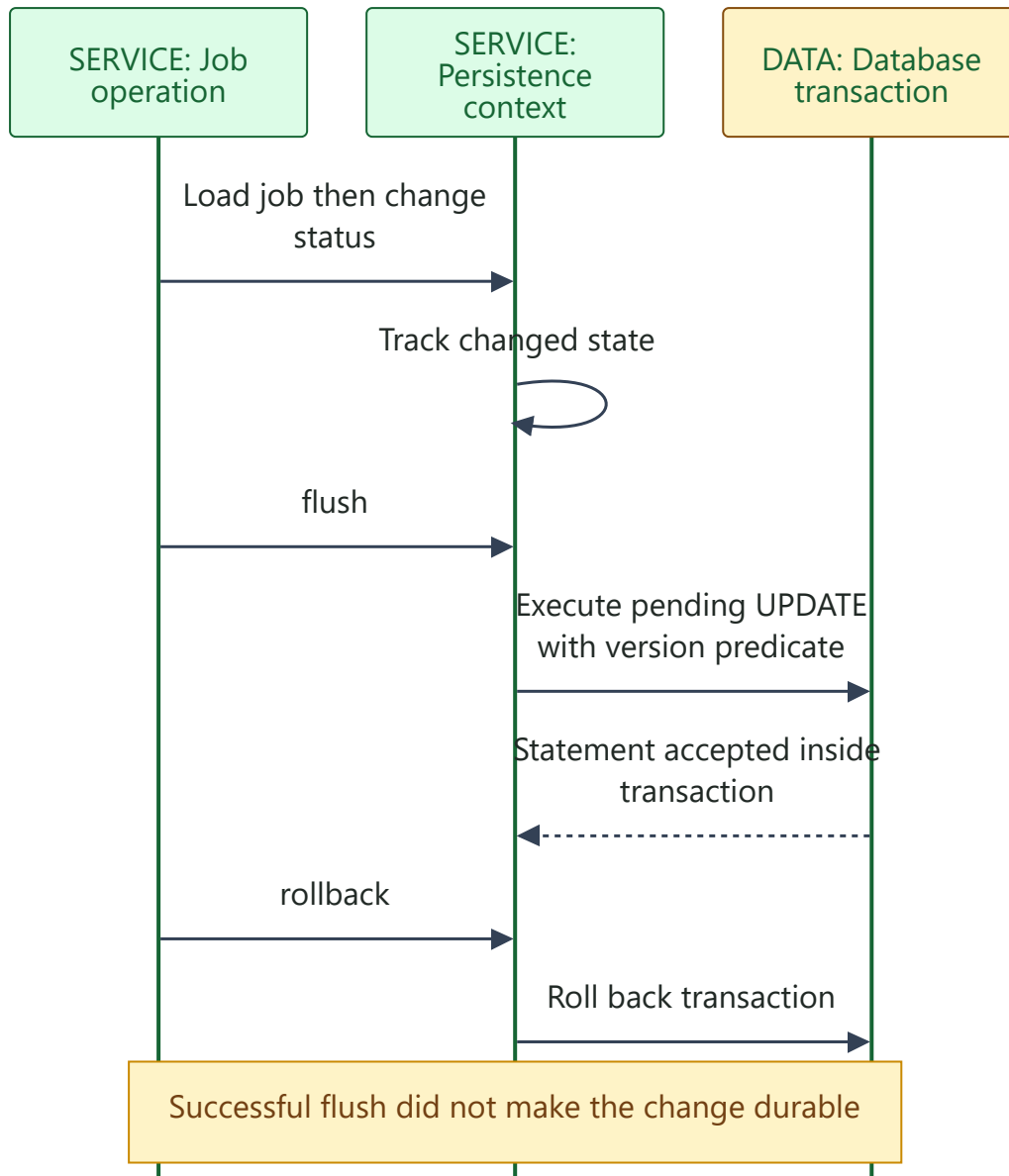
[VERIFY: dirty-checking implementation, enhancement, read-only optimizations and persistence-context lifecycle vary by Hibernate/Framework configuration. Check the selected provider documentation and actual SQL/memory behavior; no dirty-checking or batch-memory experiment ran locally.]

Interview checks: basic: first-level cache is context-local. Internals: a query can hit SQL yet reuse an existing managed instance. Debug: stale fields may come from a retained context rather than a failed database commit. Scenario: batch size must bound managed state as well as the number of submitted tasks.

4. Flush Is SQL Synchronization, Not Durable Completion

Why flushing exists: the unit of work can accumulate related changes and schedule SQL coherently rather than requiring every field write to execute a statement immediately. A flush synchronizes relevant pending persistence changes with the database in the current transaction context. Commit is the separate durability boundary.

Diagram 04.4



JpaLab updates status, explicitly flushes, reads that row through a native query in the same transaction, then rolls back. A fresh context checks that the committed state did not change. This checks flush versus commit, not cross-transaction visibility before commit. No output or SQL count from that probe has been observed here.

Flush Triggers and Modes

Flush can occur explicitly, during transaction completion, or before a query where the mode/provider must ensure the required query semantics. AUTO does not mean "always flush before every query"; COMMIT does not promise that the provider never emits SQL earlier. Query-space analysis, native-query integration, resource strategy and Hibernate-specific flush modes add nuance.

An ID generated through an identity column may require an INSERT earlier than expected to obtain the identifier. Sequence-based allocation may obtain identifiers separately and leave more opportunity for batching. Neither means that the resulting row is committed. SQL ordering can also differ from the order of individual Java method calls, so inspect statement/constraint behavior rather than assuming setters are a transaction log.

Synchronization choice	Use when	Avoid when
Normal transaction completion	Default unit-of-work synchronization fits the service	You assume method-body return already proves the later commit succeeded
Explicit flush	Detect a constraint conflict before later steps, or bound a deliberate batch	A flush is treated as commit or added after every operation without measuring batching cost
Clear after a controlled batch	Previously synchronized managed objects should be released	Callers keep using detached objects as though tracking continues
Bulk JPQL/SQL update	Many rows need a set-based change	Managed state, callbacks, version checks and caches are assumed to update automatically

Bulk mutation can bypass normal per-entity dirty checking and leave managed instances stale. Flush relevant pending work before the bulk operation if needed, then clear/refresh under a deliberate policy. Automatic flush/clear options on repository modifying queries have trade-offs: blindly clearing can discard other pending work. Bulk updates also do not automatically apply the normal version increment/check to every affected entity; encode the required concurrency policy explicitly.

[VERIFY: AUTO/COMMIT and Hibernate-specific flush behavior, native-query synchronization, SQL ordering, identifier timing and bulk-mutation semantics require the pinned provider/driver configuration. The lab is unexecuted; statement-shape explanations are not actual SQL traces or a guarantee that one annotation produces one statement.]

Production failure: a service calls `saveAndFlush`, publishes an event directly, then its transaction rolls back. The event describes a job that is not durable. The fix is the [transactional outbox](#), not adding more flush calls. Likewise, a committed job followed by HTTP response failure needs [API idempotency](#), not an assumption that the exception undid the commit.

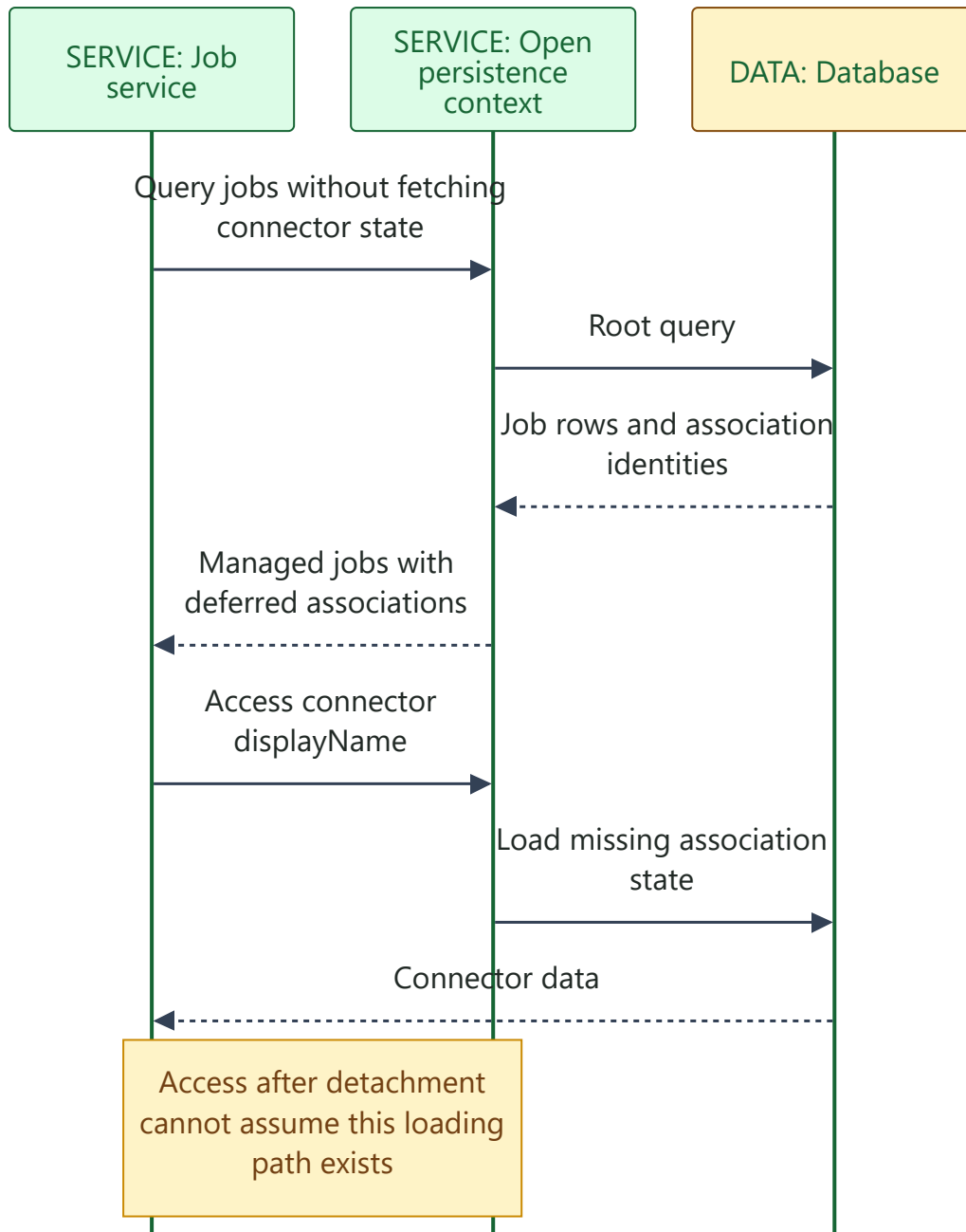
Interview checks: basic: flush sends work; commit settles the transaction. Internals: query semantics and identifier generation influence timing. Debug: find the first actual constraint/commit failure. Scenario: external publication must not be based on an uncommitted flush.

5. Lazy Loading: Deferred Access Has a Resource Lifetime

Why it exists: loading every reachable association for every use case can be expensive or unbounded. Lazy loading defers an association's state until needed, often through proxies or enhanced access interception. Fetch configuration is a requirement/hint contract with provider-specific realization; it is not a promise of a particular join layout.

The familiar JPA defaults generally make to-one associations eager and to-many associations lazy; always inspect the actual mapping. Eager does not mean "one joined SELECT," and lazy is not a portable guarantee that no earlier fetching ever occurs. Enhancement, one-to-one ownership and proxyable types can affect realization. Use a use-case-specific fetch plan rather than assuming defaults optimize the endpoint.

Diagram 04.5



If the object is detached and a required association was not initialized, Hibernate can throw `LazyInitializationException`. The precise behavior depends on configuration, but creating hidden temporary loading contexts is not a substitute for a coherent service boundary. A reference proxy can also defer missing-row failure until access; `getReference` is not a cheap authorization/existence check.

Open Session in View Is a Trade-Off

Keeping persistence access available during web rendering can make entity serialization trigger additional queries after the business transaction ended. It may hide lazy-loading errors while spreading data access into response generation and creating mixed-time observations. Turning it off can expose the missing fetch plan; turning it on is not inherently an N+1 solution. Inspect the actual Boot configuration rather than treating a default as an API guarantee.

For IntegrationHub, load the authorized data needed by an endpoint inside a defined service/query boundary, then return a DTO. SSE progress events should carry an intentional immutable view, not a managed entity whose fields trigger database access long after the originating operation. 05 and 09 define those HTTP/UI contracts.

Choice	Use when	Avoid when
Lazy association plus explicit fetch plan	Use cases need different bounded parts of the model	Access is deferred until uncontrolled serialization/background work
Eager mapping	The relationship genuinely belongs in essentially every entity use	It is added globally to silence one lazy-loading failure
Service-created DTO	API shape and query cost should be explicit	Sensitive persistence fields are copied blindly
Open context during rendering	A consciously accepted architecture owns the query/lifetime costs	It hides accidental queries, resource retention or mixed snapshots

[VERIFY: lazy/eager realization, proxy/enhancement requirements, to-one defaults and Open Session in View behavior must be checked against Jakarta Persistence, the exact Hibernate mapping and Boot settings. No web serialization or lazy-detachment failure test ran in this environment.]

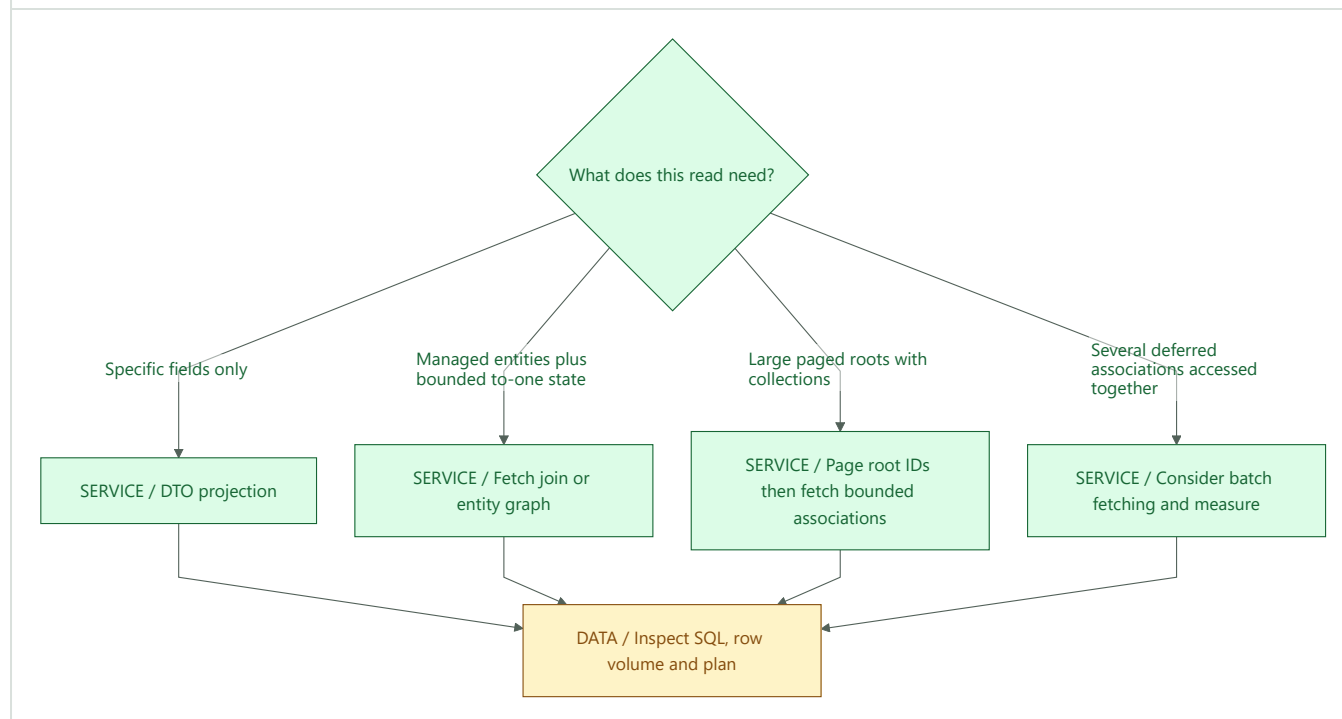
Interview checks: basic: deferred loading still needs a usable context/resource path. Internals: eager is not equivalent to join fetch. Debug: locate the first association access, including debugger/toString/JSON paths. Scenario: fetch the response contract deliberately instead of making every relationship eager.

6. N+1 and Fetch Plans: Count Work, Not Annotations

The mechanism: first query a set of jobs. Then iterate the result and read an unfetched connector for each job. With distinct missing connectors and no relevant batching/cache hits, this can produce a root query plus one query per connector. The actual count depends on shared identities, fetch strategy, provider optimization and cache state, so "one plus N" is a failure pattern, not a universal statement count.

JpaLab creates three jobs with three distinct connectors, disables L2/query caching and batch fetching, then measures prepared statements for lazy traversal and a to-one fetch join in separate fresh contexts. Its assertions require the fetch-join fixture to use one statement and the unfetched fixture to use more. Those are expected conditions in unexecuted source, not benchmark results. A warm context could conceal the problem if the test did not isolate it.

Diagram 04.6



Fetch Join, Entity Graph, Projection, Batch

A normal join can filter rows without instructing the ORM to initialize the association as a fetch join does. A fetch join changes the loading plan for that query. An entity graph describes attributes to fetch according to graph semantics; fetchgraph and loadgraph have different treatment of unspecified attributes, and neither promises exactly one SQL statement for every provider/mapping.

A projection selects only the data needed by the read contract; it avoids loading a large updateable graph when that is unnecessary. Be careful with nested/interface projections and provider behavior: a convenient projection interface is not proof of minimal SQL. Batch fetching groups deferred loads, often using multiple IDs, reducing round trips without necessarily reducing all transferred data.

Fetch technique	Use when	Avoid when
To-one fetch join	A bounded query needs related to-one state immediately	You confuse authorization filtering with association initialization

Fetch technique	Use when	Avoid when
Collection fetch join	A bounded root set needs a limited collection	Large fan-out, collection pagination or multiple independent collections multiply rows
Entity graph	A reusable attribute fetch plan clarifies a use case	Graph semantics are assumed to dictate one universal SQL join
DTO projection	The endpoint is a read model with a precise shape	Callers expect to mutate the result and have entity dirty checking persist it
Batch/subselect fetching	Deferred access groups fit the provider's measured strategy	It is treated as a fix for inherently unbounded graph traversal
Two-step root paging	Page membership must be stable while associations are fetched separately	Ordering, concurrent changes or memory bounds between steps are ignored

The Collection-Pagination Trap

A root with several children yields several joined rows. Limiting joined rows is not the same as limiting distinct root entities. Providers may reject, warn, or perform in-memory handling for problematic collection-fetch pagination. That can turn a small requested page into a huge database transfer. Inspect the actual SQL and configured warning/failure behavior, and prefer a bounded root-ID page followed by a deliberate association fetch when appropriate.

Fetching two to-many collections can create a Cartesian multiplication of child combinations. Hibernate can also reject certain multiple-bag fetch arrangements. Replacing a List with a Set merely to silence a fetch error can change domain semantics while leaving row explosion unresolved. Choose a query plan from cardinality and use-case needs, not from whichever annotation suppresses an exception.

[VERIFY: Hibernate fetch joins, entity graph semantics, batch/subselect fetching, duplicate-root handling, multiple-bag restrictions and pagination warnings/failure settings are provider/version-specific. Verify actual SQL and row volume on the selected database; no fixture count, query plan or load result is measured here.]

Production diagnosis: capture sanitized SQL or appropriate query instrumentation for one representative endpoint, identify repeated association loads, then check returned row volume and EXPLAIN, not just round-trip count. One giant join may be worse than a few bounded queries. Include both cold and warm cache cases and realistic relationship cardinality. [06](#) and [19](#) deepen those checks.

Interview checks: basic: N+1 follows access pattern. Internals: join versus fetch join matters. Debug: caches can mask a bad test fixture. Scenario: fix paged collection reads without loading every child for every tenant.

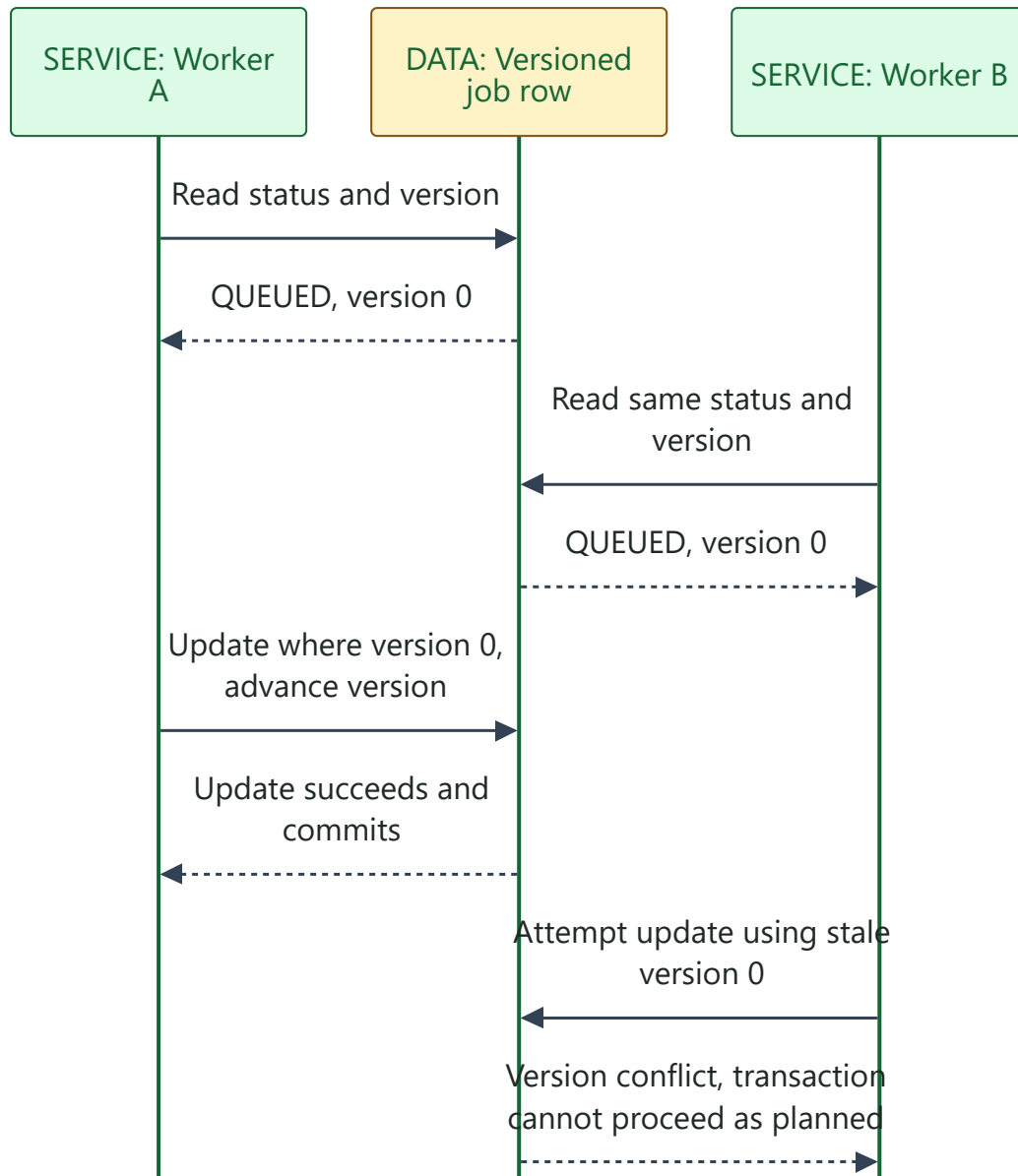
7. Locking: Detect Stale Writes or Coordinate Access

Why it exists: separate workers can read the same job state and attempt conflicting transitions. Transactional does not by itself specify how the application detects lost updates or protects a multi-row invariant. Choose the invariant first, then use version checks, database locks, constraints or isolation as required.

Optimistic Versioning

A Version attribute lets the provider compare the version originally read with the row version at the update/check boundary. Conceptually, an update includes identity plus the expected old version and advances the version on success. If the expected row/version no longer matches, the provider reports an optimistic conflict rather than silently applying a stale overwrite. The generated SQL, timing and exception wrapping depend on the provider and transaction boundary.

Diagram 04.7



Version zero/one in the diagram is an illustrative starting state, not a claim that every provider/version type initializes identically. JpaLab arranges the conflict with two EntityManagers and two transactions on one thread, so both read before the first commits. It then checks for OptimisticLockException in the failure chain. This probes stale state without relying on a lucky concurrent schedule; it does not test high-contention throughput.

On conflict, abandon the failed transaction and retry only when the business operation is safe to recompute against fresh authorized state. Reusing the same stale object in a loop does not resolve a conflict. A retry that repeats a remote charge or publishes an event outside the durable protocol can duplicate effects. 07 and 27 distinguish local conflict recovery from external idempotency.

Pessimistic Locking

A pessimistic lock mode asks the provider/database to acquire an appropriate lock for selected rows/resources. The database can make conflicting operations wait or fail under configured timeout/deadlock rules. The exact SQL, lock scope and behavior vary by database and query plan; the label PESSIMISTIC_READ does not mean an identical lock implementation everywhere.

Hold such locks briefly, inside a clearly owned transaction. Do not lock a row and then wait on a slow provider. Acquiring several locks in inconsistent order can deadlock, and a query that finds no row does not necessarily lock the absence as your uniqueness policy requires. Constraints and suitable isolation remain important.

Concurrency technique	Use when	Avoid when
Version-based optimistic locking	Conflicts are acceptable to detect/recompute and every relevant writer honors the version contract	High contention causes destructive retries or writers bypass version handling
Pessimistic row locking	A short critical transaction benefits from database coordination before modifying state	Slow external calls, long user think time or uncontrolled lock ordering
Unique/check/foreign-key constraints	Relational invariants must survive all writer paths	Application validation is treated as a sufficient substitute
Stronger transaction isolation	The invariant spans observations not protected by one row version	It is assumed to remove all retry/deadlock handling or cost

An entity version does not automatically protect a multi-row total, every related collection change, a bulk SQL writer or a second database. All relevant update paths must participate in the intended contract. Do not let clients assign the server's version field freely; a version supplied as a conditional update token still needs authorization and explicit conflict semantics.

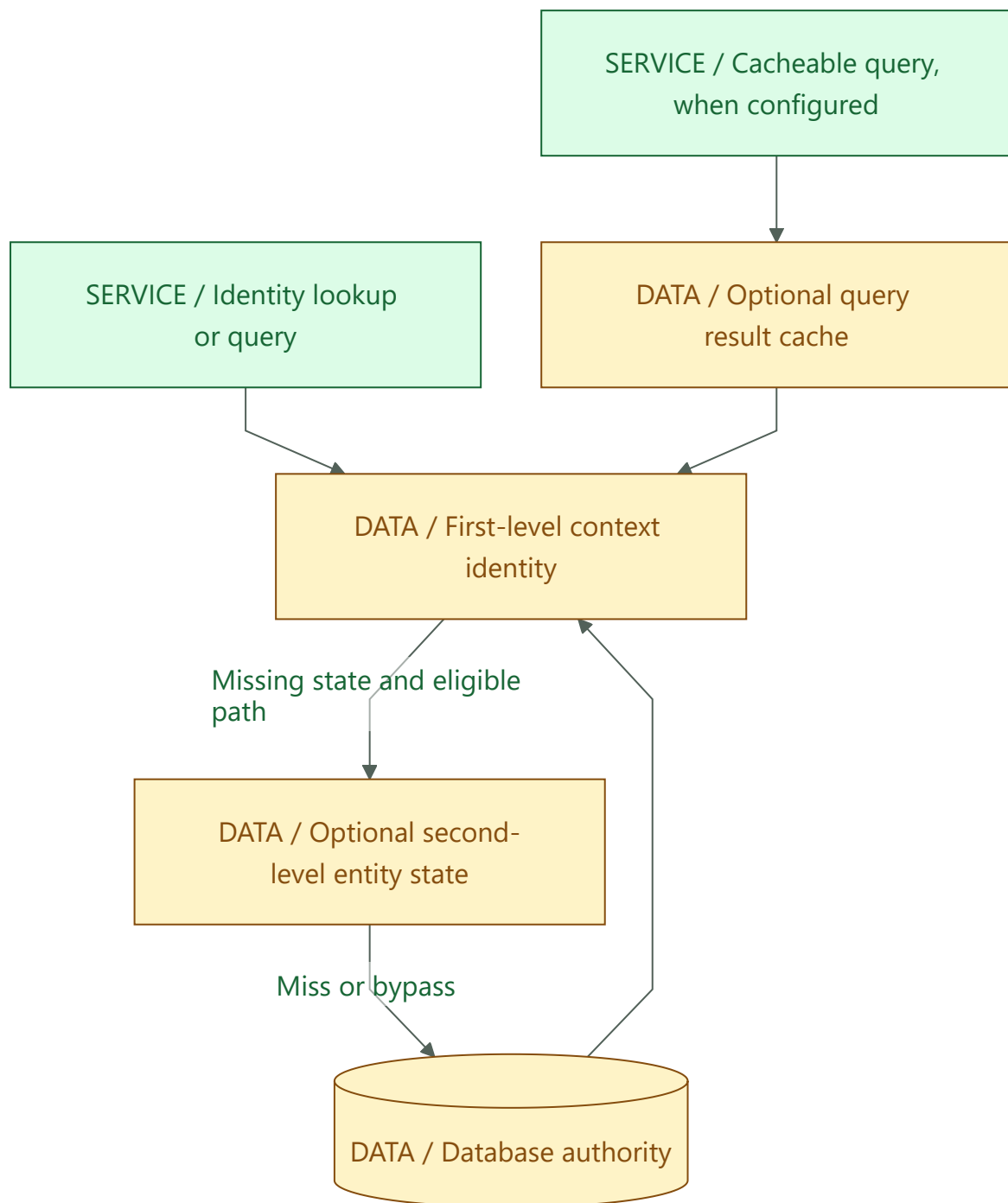
[VERIFY: optimistic-check timing, exception translation, relationship version increments, pessimistic lock scope/timeouts and deadlock behavior depend on provider, mapping and database. H2 is not evidence for PostgreSQL locks or isolation; no pessimistic-lock experiment or PostgreSQL conflict test was executed.]

Interview checks: basic: optimistic detects conflict; pessimistic coordinates via database locks. Internals: expected version participates in the write/check. Debug: inspect every writer, especially bulk/native paths. Scenario: a wallet total or cross-row capacity constraint needs more reasoning than adding Version to one entity.

8. First-Level, Second-Level and Query Caches

Why cache layers exist: repeated identity/state access can avoid repeated work, but each layer has a different lifetime and invalidation contract. Caching must preserve required correctness, not simply lower a query counter in one test.

Diagram 04.8



This is a responsibility map, not a promise that every JPQL query checks every box in this order. First-level identity is tied to a context. A configured second-level cache can share eligible entity/collection state across contexts for a factory/provider setup; it does not share one mutable managed Java instance among every request.

Query caching is a separate feature from entity-state caching. Its representation and interaction with entity state depend on the Hibernate version and cache layout; avoid repeating the universal claim that it "always stores only IDs." Query eligibility, parameters, invalidation metadata, regions and cache provider matter. A cache hit also does not imply that every required association is initialized without SQL.

Layer/policy	Use when	Avoid when
First-level identity/context	One unit of work needs consistent managed identity	Long-lived contexts become stale, large or shared across threads
L2 read-mostly entity caching	Repeated access and a suitable provider/concurrency policy justify it	Frequently changing progress state or external writers make invalidation too risky
Query cache	Measured repeatable query shapes and invalidation behavior show value	High-cardinality parameters/churning tables destroy hit rate or complicate correctness
Application DTO cache	The API tolerates explicit TTL/invalidation semantics	It silently becomes the authority for permissions or committed financial state

Hibernate knows about updates performed through its own coordinated paths, but external SQL/jobs/services can bypass relevant cache invalidation. Multi-node deployment adds coherence and eviction/replication policy questions. A cache region key must respect the tenant model; a single shared schema with `tenant_id` is not automatically equivalent to provider-managed multi-tenancy.

In IntegrationHub, rapidly changing job progress is an awkward candidate for casual long-lived caching. Stable connector definitions may be better candidates, but secret rotation and authorization changes still require a defined freshness policy. The [Redis/search discussion in 06](#) applies the same authority-versus-projection distinction at another layer.

[VERIFY: L2/query cache defaults, provider integration, concurrency strategies, cache layout and external-write invalidation are Hibernate-version/configuration specific. Both shared/query caches are explicitly disabled in the lab, so no cache-provider behavior or cluster coherence was tested.]

Production trap: a test reads the same entity twice in the same EntityManager and calls it a successful L2-cache test. That may only demonstrate first-level identity. Isolate contexts, inspect cache statistics and database work, and specify warm/cold/eviction cases before making an L2 claim.

Interview checks: basic: context cache differs from shared state cache. Internals: query cache is not synonymous with entity cache. Debug: identify which cache supplied the observation. Scenario: define what happens when a maintenance job updates the database outside Hibernate.

9. Mapping Pitfalls: Ownership, Cascades and Identity

Why ownership matters: a Java relationship has navigation directions, while a relational foreign key has a concrete storage location. On a typical bidirectional one-to-many/many-to-one mapping, the many-to-one side owns the FK mapping and the collection uses `mappedBy`. Keeping only the inverse collection updated may leave the database relationship unchanged or inconsistent with the in-memory graph.

Use domain helper operations that maintain both directions when the model is bidirectional. Relationship ownership is not the same concept as aggregate/business ownership: it describes which mapping writes the relationship. Choose unidirectional navigation when the reverse collection is not useful; do not create a giant bidirectional graph merely because it looks symmetric.

Complete Lab Entity

NOT EXECUTED. This complete `SyncJob.java` matches the Maven project's source. It compiles as part of that project with Connector and Jakarta dependencies, not as a standalone no-dependency program. The small entity intentionally uses assigned fixture IDs and a plain status string; it is not a production state machine or tenant-security implementation.

java

```
package guide.jpaa;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;
import jakarta.persistence.Version;
import java.util.Objects;

@Entity
@Table(name = "guide_sync_job")
public class SyncJob {
    @Id
    private Long id;

    @Column(name = "tenant_id", nullable = false)
    private String tenantId;

    @Column(nullable = false)
    private String status;
}
```

```

@Version
private Long version;

@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "connector_id", nullable = false)
private Connector connector;

protected SyncJob() {}

public SyncJob(Long id, String tenantId, Connector connector) {
    this.id = Objects.requireNonNull(id);
    this.tenantId = Objects.requireNonNull(tenantId);
    this.connector = Objects.requireNonNull(connector);
    if (!tenantId.equals(connector.getTenantId())) {
        throw new IllegalArgumentException("Tenant mismatch");
    }
    this.status = "QUEUED";
}

public Long getId() { return id; }
public String getStatus() { return status; }
public Long getVersion() { return version; }
public Connector getConnector() { return connector; }
public void setStatus(String status) { this.status = Objects.requireNonNull(status); }
}

```

The annotations are on fields, establishing field access for this mapping; casually mixing field/getter annotations without explicit access rules can create confusing persistence behavior. A no-argument constructor is available for persistence. The constructor's tenant check helps the intended application path, but the database FK only references `connector_id`; external SQL or other creation paths can still violate matching tenant IDs. Production requires authorization plus appropriate schema constraints, not confidence in one constructor.

Cascade Is Not Fetch or SQL ON DELETE

Cascade propagates selected persistence operations through an association: persisting an aggregate can persist its owned children when configured accordingly. It does not mean the association is always fetched, and it is not identical to a database ON DELETE action. `orphanRemoval` expresses deletion of privately owned children removed from the relationship under its contract, not a license to delete shared reference data.

For `IntegrationHub`, many jobs can reference the same connector. Removing a job must not cascade deletion to that shared connector merely because `CascadeType.ALL` is convenient. Conversely, genuinely owned job-item children may have a different lifecycle. Name ownership and retention requirements before choosing cascade/remove rules.

Mapping choice	Use when	Avoid when
Narrow cascades	Only specific operations should propagate to owned children	ALL includes REMOVE toward shared entities

Mapping choice	Use when	Avoid when
Orphan removal	Removing a privately owned child from the relationship means deleting it	Children are shared/reparented without accounting for the deletion semantics
Stable natural/value equality	A genuinely immutable unique business identity fits the entity model	Mutable fields or lazy collections participate in hashCode/equals
Generated ID	Database/provider identity allocation fits persistence and batching needs	Hash identity changes after insertion into a Set and breaks lookup
Explicit join entity	A many-to-many relationship has attributes, ownership or audit needs	A bare association hides lifecycle, uniqueness or deletion rules

Equality traps: managed identity within one context is not an instruction to use every field in equals/hashCode. Generated IDs can be absent before persistence; proxies can complicate naïve class comparisons; mutable or lazy associations can recurse, trigger SQL or change hash buckets. There is no single universal equals implementation for every entity. Select a documented identity contract and test transient, managed, detached and proxied cases. [01](#) explains the underlying Java key rules.

Other common mistakes: storing enums by ordinal can corrupt meaning after reordering values; use an explicit stable storage contract. Money needs explicit numeric scale/currency/rounding rather than an arbitrary floating-point field. Temporal types need an agreed instant/local-time/time-zone model. Unbounded large payloads should not be accidentally pulled into every small job read. Full entity toString/JSON can recurse across bidirectional relationships or expose PII/secrets.

[VERIFY: mapping access rules, ID generation/batching, cascade/orphan-removal behavior, proxy equality, enum/temporal conversion and schema generation need validation against the selected provider and database. The lab is deliberately minimal and create-drop is restricted to its disposable H2 database; no production migration or tenant-constraint enforcement was executed.]

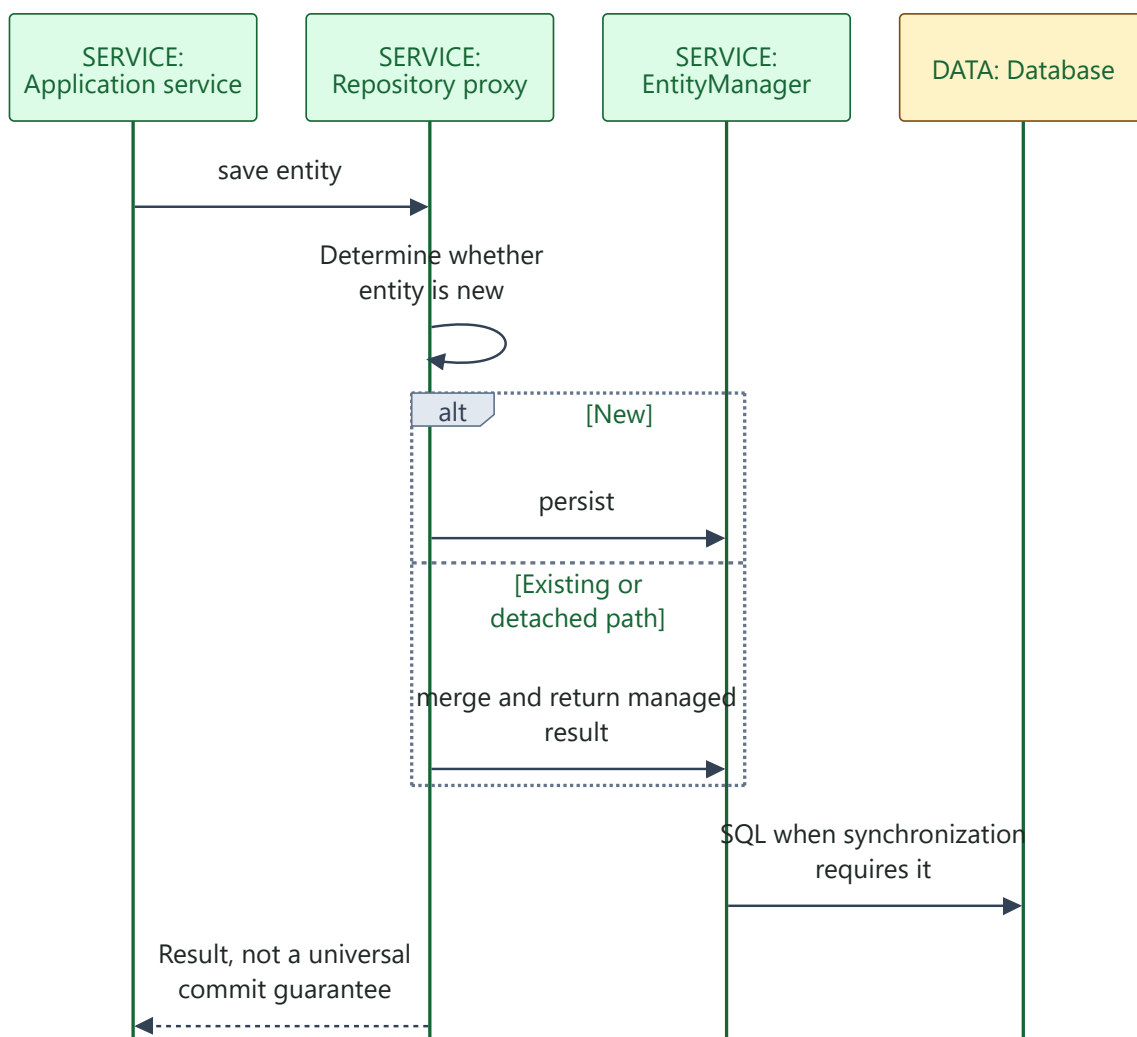
Interview checks: basic: owning side writes the association. Internals: cascade, fetch and database deletion are different policies. Debug: check both sides of a bidirectional relationship and the emitted FK update. Scenario: never cascade job deletion into shared connector configuration by accident.

10. Spring Data JPA: Repository Convenience without Hidden Guarantees

Why it exists: common repository operations and query patterns can be implemented consistently instead of writing repetitive adapter code. A repository proxy delegates to infrastructure such as SimpleJpaRepository and query execution machinery; the EntityManager/provider still decides persistence behavior under the active transaction.

save ordinarily chooses persist for an entity considered new and merge otherwise. Spring Data's new-entity detection can consider nullable version metadata and then identifier state, with Persistable/custom strategies as alternatives. Assigned IDs therefore need careful state detection; non-null ID alone should not be assumed to mean "already in the database" in every setup. Always use the returned entity when merge semantics might apply.

Diagram 04.9



When an outer service transaction exists, repository methods normally participate according to their configuration. Calling several repository operations independently is not a substitute for one service-level unit of work. Do not assume every declared query method has identical transaction/read-only behavior; inspect interface annotations, implementation defaults and the caller. 03 owns proxy and propagation mechanics.

Query Methods and Paging

Derived query names and Query declarations can simplify expression, but generated SQL must still be reviewed. EntityGraph annotations can attach a loading plan. Specifications/criteria compose predicates, not permission to ignore indexes or tenant filtering. A convenient method name does not guarantee efficient joins or a correct authorization boundary.

Page commonly includes total-count information and can require a count query; Slice can answer whether more results exist without an exact total, depending on execution strategy. Offset paging can get increasingly expensive and unstable under concurrent changes; keyset/scroll approaches fit some ordered access patterns. Always define a stable ordering and relevant tie-breaker. 05 covers API contracts and 06 covers SQL access paths.

Repository feature	Use when	Avoid when
save with understood new-state detection	Repository contract intentionally persists/merges the passed state	It is assumed to always issue INSERT immediately or to commit an outer transaction
Derived query	A simple predicate is readable and bounded	Extremely long names hide joins, tenant semantics or performance
Explicit query/projection	The endpoint needs a deliberate data shape and access path	Query strings are untested or accept unsafe dynamic fragments
EntityGraph	The repository method needs an explicit association plan	It is assumed to fix collection pagination automatically
Modifying bulk query	Set-based mutation is intentional	Persistence-context staleness, callbacks and version handling are ignored
Page / Slice / scroll choice	The client needs the selected total/navigation contract	Exact totals and deep offsets are computed needlessly on every request

Spring Data does not make a repository method tenant-safe by existing. Built-in ID lookup can return any matching globally identified row unless the access path enforces authorization/tenant constraints. Prefer narrowly designed service/query paths, and add negative cross-tenant tests rather than relying on untrusted tenant fields in an entity.

[VERIFY: Spring Data JPA new-entity detection, repository transaction defaults, modifying-query flush/clear options, entity-graph execution and paging/scroll support must be checked against the selected version. The standalone JPA lab does not instantiate Spring Data repositories or execute any repository integration tests.]

Production failure: a service ignores save's returned managed copy, mutates the old detached instance again, and expects another update at commit. The second mutation may never be tracked. Either operate on a managed entity loaded in the unit of work or use the returned managed instance intentionally; do not rely on a method name to hide state transitions.

Interview checks: basic: save can select persist or merge. Internals: new-state detection includes more than one simplistic ID rule. Debug: inspect the actual returned instance and outer transaction. Scenario: query shape, count cost and tenant isolation must be tested even behind a repository interface.

11. Test What the Persistence Layer Actually Does

The supplied lab is deliberately single-process and standalone. Its persistence unit lists the two entity classes explicitly; it uses an in-memory H2 database and create-drop only for that disposable fixture. There is no PostgreSQL process, Spring Data proxy, L2 provider, web request or Kafka publisher. It imports managed ORM dependencies through Boot's parent, not a running Boot application.

Probe	Intended assertion	Evidence limit
Managed identity	Two finds for one ID in one context return the same instance	Not proof that every query skips SQL
Dirty flush and rollback	Changed state reaches SQL inside a transaction, then disappears from committed state after rollback	Not a cross-session isolation or durability benchmark
Detached merge	Returned copy is managed, original detached input is not	Does not validate untrusted merge safety or all cascade graphs
Optimistic conflict	Two contexts read one version; later stale write fails	Not a pessimistic-lock, multi-row invariant or contention-throughput test
Fetch-plan comparison	Cold-context lazy traversal prepares more statements than the to-one fetch-join fixture	Expected source assertions only; no measured counts available

NOT EXECUTED: the runner's preflight was executed and reported missing Java/Maven/cached dependencies. The POM and persistence XML were parsed for well-formedness, not validated through artifact resolution or a JPA provider. The complete inline SyncJob listing is checked against its saved source, which is not compilation. No success output, SQL trace, statement count or benchmark is fabricated.

For real tests, use the production database family where lock/isolation/dialect behavior matters. Reset state between cases, control transaction boundaries and cache warmth, and inspect outcomes from independent contexts where appropriate. A test transaction that automatically rolls back around the entire method can hide commit-time failures or make detached/serialization behavior unlike production. [19](#) discusses integration tests and Testcontainers once tools are available.

A Repeatable Diagnosis

- State the failing contract: wrong data, missed write, stale update, extra queries, latency, memory or tenant exposure.
- Record the transaction and persistence-context lifetimes, including thread and web serialization boundaries.
- Inspect actual SQL, parameters only under a safe data policy, statement counts and row volume. Confirm which cache/fetch settings were active.
- Identify entity states, association ownership and every relevant writer, including bulk/native paths.
- Use database evidence for blocking, isolation and query plans, not only Java annotations.
- Fix the owning boundary and add a test that fails for the specific former mistake, without claiming broader guarantees than it exercises.

This keeps ORM reasoning connected to the full [IntegrationHub journey](#): Java identity helps one unit of work; the database settles local facts; messaging transports durable intent; retries and UI reconnects must respect those boundaries.

Explained-Here Index and Sources

Concept	Explanation
JPA versus Hibernate versus Spring Data	Ownership boundaries
Entity states, persist, merge, detach and refresh	Lifecycle
Persistence context, first-level identity and dirty checking	Unit of work
Flush, commit, query synchronization and bulk updates	SQL synchronization
Lazy loading, proxies and response lifetime	Deferred access
N+1, fetch joins, entity graphs and pagination	Fetch plans
Version checks and pessimistic locking	Concurrency
L1, L2 and query caching	Cache boundaries
Association ownership, cascade, orphan removal and identity	Mapping

Concept	Explanation
Spring Data save, queries, paging and transactions	Repositories
Lab scope and actual execution status	Testing

The [Boot 4.0 dependency table](#) was consulted for the selected version family. Authoritative mechanism references: [Jakarta Persistence 3.2 specification](#), [Hibernate ORM 7.2 user guide](#), and [Spring Data JPA persistence](#). The latter are owning references for further checks, not claims that this environment resolved artifacts or ran their examples.

Related Chapters

Use [01 Java identity/collections](#), [02 concurrency](#) and [03 service transactions](#) beneath the ORM model. Continue through [06 databases](#) and [05 API design](#) in the recommended reading order. [07 messaging](#), [08 security](#), [09 frontend contracts](#), [19 testing](#), [23 profiling](#) and [27 payments](#) deepen the touched boundaries. Return to [00](#) for the whole request. Generated Related chapters links are reciprocal.

RELATED CHAPTERS / RECIPROCAL LINKS

00 The System Before the Details	09 The Browser and the Java Backend
01 Core Java and the JVM	23 JVM Performance Engineering
02 Concurrency and the Memory Model	05 APIs and Real-Time Communication
03 Spring and Spring Boot Internals	19 Testing and Quality
07 Messaging, Batch and Streams	27 Fintech and Payments Engineering
06 Databases and Durable State	25 Architecture Styles and Domain Design
08 Security, Identity and Data Protection	24 IntegrationHub in Motion
	20 Appendix: Interview Toolkit and Index

One-Page Cheat Sheet

Layers: JPA defines contracts; Hibernate implements them; Spring Data supplies repository infrastructure; Spring manages service transaction boundaries; the database owns constraints/isolation/durability.

States: new is not managed merely because it has an ID. persist registers a new instance; merge copies detached state into a managed result. Use the result deliberately. clear/close/detach release management, not Java references. Rollback does not rewind every object field.

Context: one managed instance per persistent identity per context. Queries may still execute SQL. Managed mutation can be dirty-checked without another save. Bound managed graphs in batch work; never share one EntityManager across unrelated concurrent tasks.

Timing: flush synchronizes SQL; commit settles the transaction. AUTO, native queries and ID generation affect SQL timing. Bulk mutation can bypass managed-state/version/callback expectations. External events need durable publication intent, not a flush assumption.

Reads: lazy state needs a valid loading boundary. Eager is not one join. N+1 follows access pattern. Choose bounded fetch plans or DTOs; beware collection pagination and Cartesian row explosion. Measure cold/warm SQL and row volume.

Concurrency/cache: Version detects covered stale writes, not every multi-row or external conflict. Pessimistic locks are database-specific and should be short-lived. L1, L2 and query caches have different lifetimes; external writers can invalidate assumptions.

Mapping/repositories: owner writes the FK; cascade is not fetch or ON DELETE. Avoid REMOVE toward shared connector data, mutable entity hashes and untrusted entity merge. Repository save can persist or merge and is not a commit guarantee. The Java/Hibernate lab remains NOT EXECUTED.

Interview Corner

Basic: JPA, Hibernate and Spring Data Are What, Exactly?

JPA is the persistence API/specification, Hibernate a provider, and Spring Data JPA repository infrastructure over that persistence layer. None replaces database transaction/isolation rules. Identify which layer owns the observed behavior before changing settings.

Internals: Why Can Two find Calls Return the Same Object?

The persistence context maintains managed identity for a persistent key. It can reuse the same instance, often without another identity load. That is not proof that arbitrary JPQL queries are served without SQL or that other contexts share the same Java object.

Trace/Debug: A Setter Ran but No UPDATE Appeared. What Do You Check?

Check whether the instance is managed in the expected context, the property is mapped, a meaningful dirty change exists, the transaction/synchronization boundary was reached, and read-only/provider settings apply. Detached mutation alone is only an in-memory change.

Scenario: Can You Send a Managed Entity to Several Executor Tasks?

Not as a default concurrency strategy. EntityManager and its mutable managed graph are not general shared concurrent structures. Pass identifiers or immutable data and give each authorized unit explicit context/transaction ownership.

Basic: persist Versus merge?

Persist associates a suitable new instance with the context. Merge copies accepted input state into a managed instance and returns it; detached input normally remains detached. State, cascades and version checks affect behavior, so merge is not a universal repair command.

Internals: Why Should save's Return Value Matter?

Spring Data can choose merge, whose returned managed instance may differ from detached input. Mutating the old object afterward may not be tracked. Prefer a managed load-and-apply operation or deliberately continue with the returned instance.

Trace/Debug: saveAndFlush Succeeded but the Row Was Not Durable. How?

Flush synchronized work inside the transaction; a later rollback or failed commit can prevent durability. Trace the outer transaction rather than assuming repository method return commits it. A generated identifier is not proof of committed insertion either.

Scenario: When Would You Explicitly Flush?

When an operation needs to surface a constraint failure at a controlled point or synchronize a bounded batch before clearing, with an understood transaction boundary. Do not flush after every setter or publish an external event solely because flush succeeded.

Basic: What Causes LazyInitializationException?

An unfetched lazy value is accessed without a usable provider context/loading path under the configured model. Identify the access site and lifetime. Making the entire domain eager may hide the exception while causing much larger query costs.

Internals: Does EAGER Mean JOIN FETCH?

No. Eager declares required fetching behavior, not one universal SQL strategy. A provider can satisfy it through additional selects. A query-specific fetch join/entity graph or DTO plan must be evaluated by actual SQL and cardinality.

Trace/Debug: Why Did Your N+1 Test Pass but Production Still Had Extra Queries?

The test may have reused a warm persistence context/cache, used fewer distinct associations, or failed to access the same fields serialization touches. Use fresh contexts, representative cardinality and explicit traversal/response assertions, then inspect statements and rows.

Scenario: How Would You Page Jobs with Many Items?

Define a stable root ordering/page, then fetch a bounded set of required associations or project the needed data. Avoid treating a collection join's limited rows as a correct root page. Check provider pagination behavior, count cost and concurrent-change semantics.

Basic: What Does Version Protect?

Covered optimistic writes/checks compare expected version with current state so stale updates fail. It does not automatically protect every related row, external writer, bulk operation or remote effect. Every relevant path must respect the contract.

Internals: Why Can an Optimistic Failure Appear at Commit?

Dirty SQL/version checks may be deferred until flush during transaction completion. The method body can finish before synchronization detects conflict. Observe the full transaction result and handle translated/wrapped exceptions at the appropriate boundary.

Trace/Debug: Why Is Retrying the Same Detached Object Not Enough?

Its version/state may still be stale, and the failed transaction/context is not a clean retry scope. Reload authorized state in a new unit, recompute a safe operation and preserve idempotency for external effects. Do not retry indiscriminately.

Scenario: Optimistic or Pessimistic Locking for a Contended Job Transition?

Start with the invariant and conflict rate. Optimistic checks fit recomputable conflicts; a short pessimistic lock can coordinate before modifying, but can block/deadlock. Database constraints/isolation may still be required, and neither justifies holding a lock across slow provider I/O.

Basic: First-Level Versus Second-Level Cache?

First-level identity belongs to one persistence context. Optional L2 state can be shared across contexts under provider/cache policy; it does not hand every request the same managed object. Test with separate contexts when claiming an L2 benefit.

Internals: Is Query Cache Just the Same as Entity Cache?

No. It caches eligible query-result information under its own representation/invalidation rules; entity state and association access can involve other layers. Hibernate versions/layouts differ, so do not promise that query caching always stores only IDs or eliminates every SQL access.

Trace/Debug: A Maintenance SQL Job Changed a Row but the App Sees Old Data. Why?

Check the current context and any L2/query/application caches, not just database replication. External writers can bypass ORM invalidation. Determine the freshness contract, refresh/evict deliberately and fix the update/invalidation protocol where needed.

Scenario: Should Every Job/Progress Entity Be L2-Cached?

No. High update churn and freshness requirements can make it costly or incorrect under a weak invalidation design. Measure a suitable read-mostly candidate instead and account for tenant isolation, secret rotation and external writes.

Basic: What Does mappedBy Mean?

It identifies the other side that owns the relationship mapping; it does not create a second FK-writing authority. Maintain both in-memory directions when the domain is bidirectional, and verify the owning side is updated.

Internals: Cascade REMOVE Versus orphanRemoval?

Cascade REMOVE propagates a removal operation through a configured relationship. Orphan removal handles the lifecycle of privately owned children removed from that relationship under its contract. Neither equals fetch strategy or automatically matches database ON DELETE behavior.

Trace/Debug: Why Did Removing a Job Delete Shared Connector Data?

Inspect cascades and database FK deletion rules. An indiscriminate cascade toward shared reference data can propagate a delete that business ownership never intended. Narrow the policy, repair data carefully and add retention/deletion tests.

Scenario: How Do You Make Entity Access Tenant-Safe?

Authorize the operation, scope queries and keys correctly, validate association ownership, and enforce suitable database invariants. A tenant field, constructor check or repository interface alone cannot secure every path. Test cross-tenant IDs, associations and direct/bulk write paths explicitly.

o6 / Databases and Durable State

The database is where a claim becomes a fact. Everything else in IntegrationHub is a cache, a retry or an opinion. A job exists because a row committed; a tenant's progress is real because the write-ahead log survived a restart. This chapter is about the machinery that turns an application's intent into durable, consistent, queryable state, and about the specific ways that machinery disappoints people who treat it as a black box behind a repository interface. IntegrationHub is fictional and no schema, volume or latency figure here describes a real employer's system.

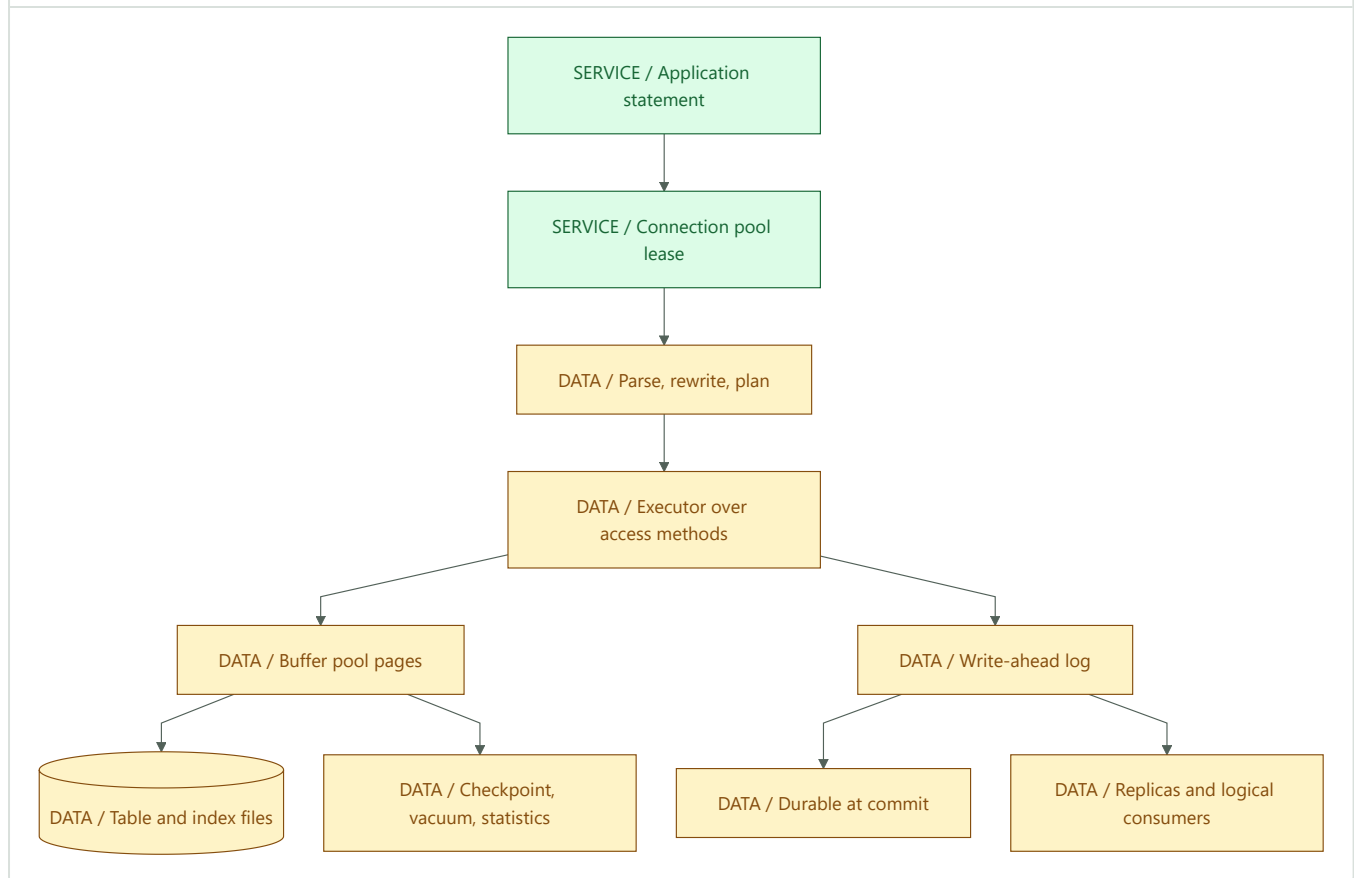
Version assumptions: PostgreSQL 17 is the reference operational database, with MySQL 8.4 as the comparison point. Redis 7.x, MongoDB 8.x and Elasticsearch 9.x appear as complementary stores rather than primary authority. Flyway and Liquibase are discussed at mechanism level. The surrounding stack is unchanged from 03: Java 21, Spring Boot 4.0.8, Kafka 4.x and Kubernetes 1.34.

[VERIFY: recheck PostgreSQL 17, MySQL 8.4, Redis, MongoDB and Elasticsearch version-specific behavior, along with the Boot-managed driver and pool versions, against current release documentation. No engine was installed, no driver resolved and no release note retrieved in this environment.]

Execution status: NOT EXECUTED. No JDK, Maven, database engine or client is available, and download authorization covered diagram dependencies only. The two labs under `code/06-databases/` are complete Java 21 sources with explicit assertions, and they model isolation and pooling behavior in memory. They have not been compiled or run. No SQL statement, query plan, row count, timing, lock wait or replication lag in this chapter is an observation. Diagrams render locally to embedded SVG; syntax highlighting remains unavailable.

Big Picture

Diagram 06.1



Three facts in that diagram explain most database surprises. Commit durability comes from the log, not from writing table files, so a committed row may still live only in memory pages plus a flushed log record. The planner chooses the access path from statistics, so the same SQL can behave completely differently next month. And the connection pool sits in front of everything, which means application concurrency is bounded long before the database's own limits are reached.

What You Will Be Able to Explain

- Describe how a statement travels from the pool through planning and execution to pages, the log and durability.
- Explain B-tree and LSM storage, buffer pool behavior and write-ahead logging well enough to predict their cost profiles.
- Design indexes from the predicates, ordering and selectivity of actual queries, and say why an index can be ignored.
- Read a plan critically, including the difference between estimated and actual rows, and know what that difference implies.

- Distinguish isolation levels by the anomalies they prevent, and explain MVCC, write skew and first-committer-wins.
- Diagnose lock waits and deadlocks from the access order rather than from the application's method names.
- Explain vacuum, bloat, statistics and checkpoints as operational requirements rather than background magic.
- Size and reason about a connection pool, including why increasing it often makes things worse.
- Compare streaming and logical replication, partitioning and sharding, and the consistency each one costs.
- Run a zero-downtime schema change with expand and contract, and state a real backup and recovery objective.

[P] PRODUCTION

Where this lives in IntegrationHub. PostgreSQL is the authority for jobs, idempotency records, outbox rows and progress. [04](#) generates most of the statements, [05](#) depends on the unique constraint behind idempotency keys and the index behind keyset pagination, and [07](#) relies on the outbox committing in the same local transaction as the business row. [08](#) owns tenant isolation and encryption at rest, [15](#) and [17](#) own where the instance runs, and [18](#) owns the backup verification that makes the word "durable" meaningful.

1. SQL and the Relational Model

Why it exists: a relational database separates what you want from how it is obtained. You declare a result set; the engine chooses the access path, join order and algorithms. That indirection is what lets the same query survive a hundredfold growth in data by changing an index rather than application code, and it is also why performance reasoning requires understanding the planner rather than only the SQL text.

The model's useful discipline is that constraints live with the data. A unique constraint, a foreign key, a check constraint and a not-null column are enforced for every writer, including the maintenance script someone runs at midnight and the import path nobody remembered. Application-level validation as described in [03](#) improves error messages; it does not protect an invariant, because it only runs on the paths that call it.

Declarative Queries and Set Thinking

- **A query describes a result, not a loop.** Filtering, joining, grouping and ordering are set operations the engine can reorder and optimize. Fetching rows and filtering them in Java moves work to the slowest place and discards the index.
- **NULL is not a value, it is the absence of one.** Comparisons with NULL yield unknown, which silently removes rows from predicates and changes the meaning of NOT IN. This is a frequent source of quietly wrong results rather than errors.
- **Joins are not loops over objects.** Cardinality is the thing to reason about: how many rows enter, how many survive each predicate, and how many come out. A join that multiplies rows will multiply everything downstream.
- **Aggregation and window functions** can replace whole layers of application code, and they execute next to the data rather than across a network.

Modeling choice	Use when	Avoid when
Normalized tables with constraints	Correctness matters and the data has genuine relationships	A read path needs a wide denormalized shape and the cost is measured, not assumed
Deliberate denormalization	A specific hot read justifies duplicated data with a stated update rule	It is a default, so invariants now live in several places with no owner
JSON or document column	Genuinely variable connector configuration with no fixed shape	It becomes a schemaless dumping ground that constraints and indexes cannot see
Surrogate key plus natural unique constraint	Stable identity with a business uniqueness rule enforced by the database	The natural key is enforced only by application code

For IntegrationHub, connector configuration is a reasonable JSON column because each connector type has a different shape, while tenant identity, job status, timestamps and the idempotency key are ordinary typed columns with constraints. The decision rule is whether the database needs to enforce or index the field; if it does, it is a column.

[M] MECHANISM

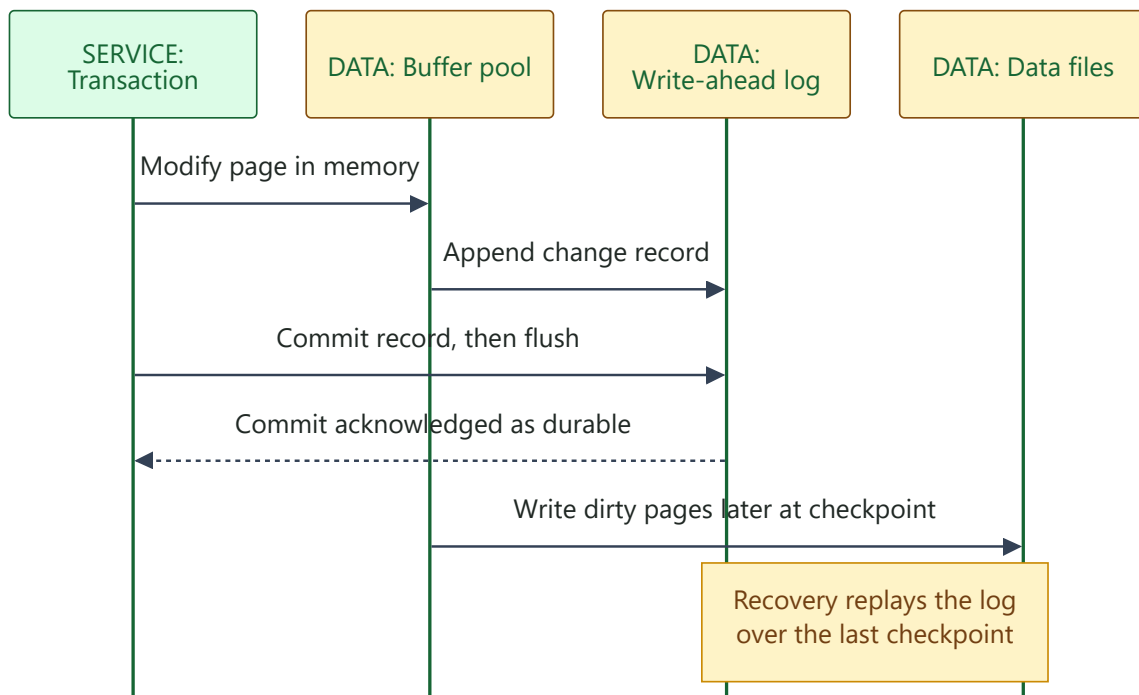
Constraints are the only rules that apply to every writer. Application checks, ORM callbacks and service validation run on the paths that invoke them. A unique index, a check constraint and a foreign key run for the bulk import, the console session and the three-year-old script nobody owns.

Interview checks: basic: SQL is declarative, so the planner chooses the access path. Internals: constraints are enforced below the application. Debug: unexpected row loss often traces to NULL semantics. Scenario: put a field in a column when the database must enforce or index it.

2. Storage Engines: B-Trees, LSM Trees, Buffer Pools and the Log

Why this matters: every performance characteristic you care about comes from how the engine stores and durably records data. Knowing the two dominant structures and the write-ahead protocol turns surprising behavior into predictable behavior.

Diagram 06.2

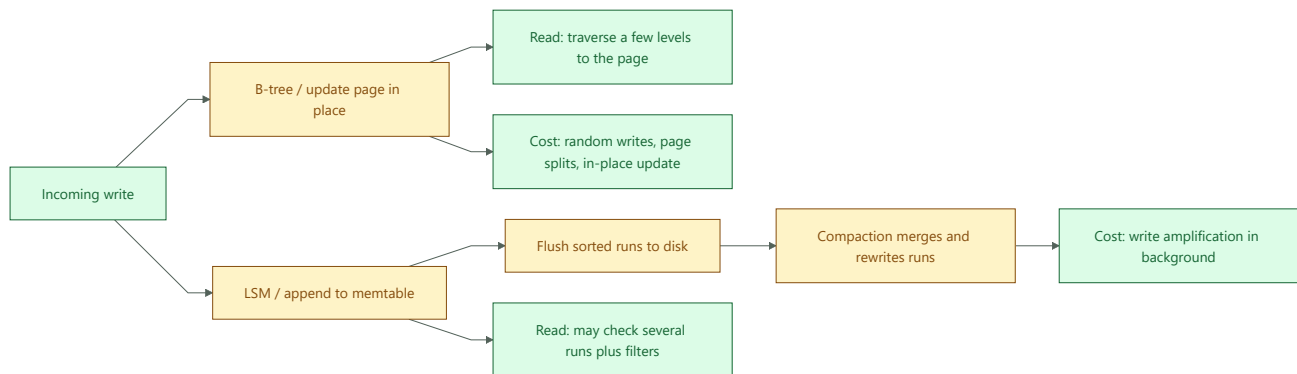


Write-ahead logging is the protocol that makes commit cheap and crash recovery possible. The change record reaches durable storage before the modified page does, so a crash can be repaired by replaying the log from the last checkpoint. The consequences are practical: commit latency is dominated by log flush, a long gap between checkpoints means a longer recovery, and the log is also the stream that replication consumes.

The buffer pool is the engine's page cache. Reads come from memory when the page is resident and from storage when it is not, and writes land in memory and are flushed later. Its hit ratio is the single most influential number in read performance, which is why working-set size relative to memory matters more than total database size. An engine-level cache is also not the same as an application cache: it is coherent with the data it serves, which an application cache in [10](#) is not.

B-Tree Versus LSM Tree

Diagram 06.3



A **B-tree** keeps sorted pages and updates them in place, giving predictable point lookups and efficient ordered range scans. Its costs are random write patterns, page splits under random key insertion and, in some engines, an extra write of the full page for crash safety. PostgreSQL, MySQL's InnoDB and most relational engines are B-tree based.

An **LSM tree** buffers writes in memory, flushes sorted runs, and merges them in the background. This converts random writes into sequential ones, which suits write-heavy workloads, at the cost of reads that may consult several runs and of compaction work that competes for I/O. Many key-value and wide-column stores use it. Neither structure is better in general; they trade read amplification against write amplification.

Property	B-tree engines	LSM engines
Point read	Predictable, few levels	May consult multiple runs, mitigated by filters and caches
Range scan in key order	Natural strength	Requires merging runs
Write-heavy ingest	Random page updates and splits	Sequential flushes, strong fit
Background cost	Checkpoints, vacuum or purge, page splits	Compaction with write amplification and I/O spikes
Space behavior	Fragmentation and bloat from updates	Obsolete versions persist until compaction

[VERIFY: full-page-write behavior, checkpoint tuning, compaction strategies and buffer-pool defaults are engine- and version-specific. Confirm against PostgreSQL 17 and MySQL 8.4 documentation; no engine was installed or measured in this environment.]

Interview checks: basic: the log makes commits durable, not the data files. Internals: buffer-pool residency governs read cost. Debug: a slow commit usually points at log flush or storage latency. Scenario: write-heavy ingest is where LSM designs earn their compaction cost.

3. Indexes: Access Paths, Not Decoration

Why they exist: without an index the engine must examine every candidate row. An index is an ordered structure that lets the engine find the relevant rows directly, and it is the main lever you control. It is also a cost: every index must be maintained on write, occupies space and must stay in memory to be fast.

Design from the query, not from the column. A composite index serves predicates in its leading-column order, so an index on tenant then status then created-at helps a query filtering on tenant and status and ordering by created-at, and does nothing for a query filtering on status alone. The ordering within the index can also satisfy an ORDER BY, removing a sort, which is often the larger win.

What Makes an Index Usable

- **Leading-column match.** Predicates must line up with the index's leading columns for the engine to seek rather than scan.
- **Selectivity.** An index that matches most rows may be slower than a sequential scan, which is why a planner can rationally ignore it.
- **Sargable predicates.** Wrapping the column in a function or applying an implicit type conversion can prevent index use; an expression index exists precisely for the cases where you must transform the column.
- **Covering.** When the index contains every column the query needs, the engine can answer from the index alone, avoiding the lookup into the table.
- **Write cost.** Each additional index slows inserts, updates and deletes and consumes cache memory. Unused indexes are pure cost.

Index kind	Use when	Avoid when
Composite B-tree	Multi-column predicates plus an ordering that matches	Columns are listed in a convenient rather than a selective order
Partial or filtered	A small hot subset such as jobs still running	The filter excludes the predicate the query actually uses
Unique	An invariant must hold for every writer, such as an idempotency key	It is added later without handling the duplicates already present
Expression	Queries legitimately filter on a transformation of the column	It hides a modeling problem better fixed by storing the value
Full-text or specialized	Search semantics the B-tree cannot express	A dedicated search system in 26 is the real requirement

For IntegrationHub, the keyset pagination contract in 05 requires an index on tenant plus the total ordering key, because the cursor's seek is only cheap if the index supports it. The idempotency contract requires a unique index on tenant, operation and key. The outbox relay in 07 wants a partial index over unpublished rows, because that set stays small while the table grows.

Production trap: an index is added to fix one slow query, and nothing is ever removed. A year later the table carries a dozen indexes, write latency has doubled, and cache memory is consumed by structures no query uses. Review index usage statistics periodically and treat removal as a normal, reversible change.

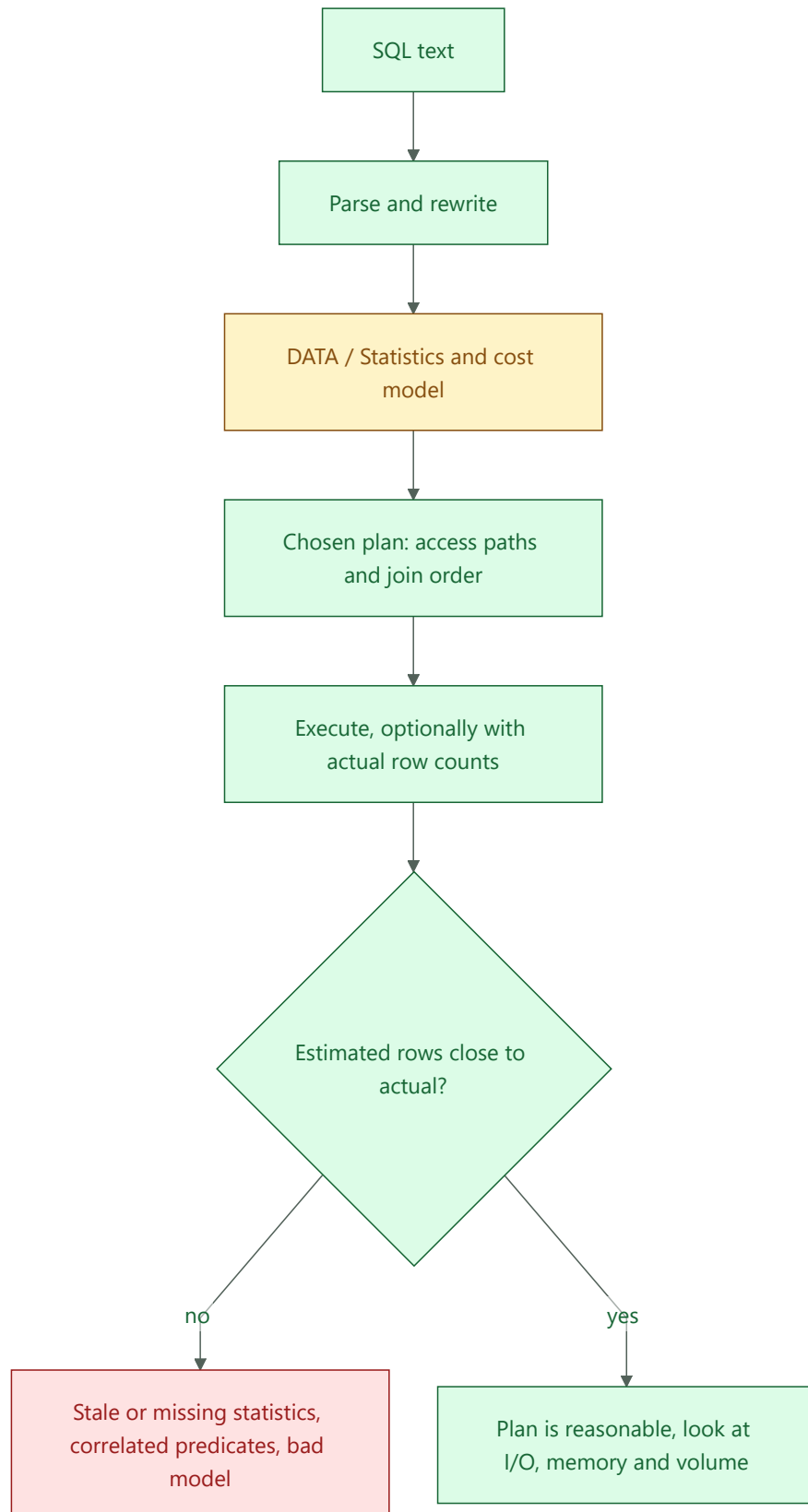
[VERIFY: index-only scan requirements, partial-index matching rules, implicit-conversion behavior and index maintenance costs differ between PostgreSQL and MySQL and across versions. Verify with real plans on representative data; no index was created or measured here.]

Interview checks: basic: an index is an access path with a write cost. Internals: leading-column order and selectivity decide usability. Debug: a function around the column often explains an ignored index. Scenario: design the composite index from the predicate plus the ordering.

4. Reading Plans: EXPLAIN and the Estimate Gap

Why plans matter: the planner picks the access path from statistics about your data. When the plan is wrong, the SQL is usually fine and the statistics or the model are not. Reading a plan is how you stop guessing.

Diagram 06.4



Plain EXPLAIN shows the plan the engine would choose, including estimates. Running it with actual execution adds the rows each node really produced and the time it took, and that comparison is the most valuable signal available. A node estimating ten rows and producing a hundred thousand explains a nested-loop join that should have been a hash join, and the fix is upstream of the join choice.

- **Scan nodes** tell you whether an index was used and how. A sequential scan is not automatically wrong; on a small or unselective query it is the right choice.
- **Join nodes** reveal the algorithm. Nested loops are excellent for few outer rows and catastrophic for many; hash joins need memory; merge joins need sorted input.
- **Sort and aggregate nodes** show whether work spilled to disk, which usually means a memory setting or a volume you did not expect.
- **Row counts** are the diagnosis. Timing tells you where it hurts; the estimate gap tells you why.

The planner's choices change with data volume and distribution, so a plan verified on a small test database proves little about production. Test with representative volume and skew, including the tenant whose data is ten times larger than everyone else's, because that tenant is who will find the nested loop.

Symptom in a plan	Likely cause	Reasonable response
Large estimate-versus-actual gap	Stale or missing statistics, correlated columns	Refresh statistics, consider extended statistics, reconsider the model
Sequential scan on a large filtered query	No usable index, or an unsargable predicate	Add or fix the index, remove the function around the column
Nested loop over many outer rows	Underestimated cardinality upstream	Fix the estimate rather than forcing a join method
Sort spilling to disk	Working memory smaller than the sort	Reduce the sorted set, or revisit the memory setting with care
Index used but still slow	Low selectivity, or expensive lookups back into the table	Make the index covering, or narrow the predicate

[VERIFY: plan node names, EXPLAIN option syntax, statistics collection behavior and planner cost parameters differ by engine and version. Confirm against PostgreSQL 17 and MySQL 8.4 before relying on specific option names; no plan was captured in this environment.]

Interview checks: basic: EXPLAIN shows the chosen access path. Internals: the estimate-versus-actual gap diagnoses the plan. Debug: compare row counts before timings. Scenario: validate plans at representative volume and skew, not on a seeded test database.

5. Transactions and What Durability Actually Promises

Why they exist: a business operation usually spans several rows, and partially applied work is worse than no work. A transaction groups statements so they commit together or not at all, and the engine maintains that guarantee across concurrency and crashes.

The familiar acronym is worth stating precisely because each letter is often overclaimed.

Atomicity means the group applies entirely or not at all. **Consistency** means declared constraints hold at transaction boundaries; it does not mean your business invariants are automatically correct. **Isolation** is a configurable spectrum, not a single guarantee, which the next section covers. **Durability** means a committed transaction survives a crash of that instance, assuming the storage honored its flush, and says nothing about surviving the loss of the machine or the data centre.

That last point is the one that costs companies money. Durability is a local promise about one instance's log. Surviving disk loss requires replication; surviving a bad deployment or a mistaken DELETE requires backups and point-in-time recovery; surviving a region failure requires a multi-region design with its own consistency costs. 17 and 22 own those layers.

- **Keep transactions short.** A transaction holds resources and, under MVCC, keeps old row versions alive. Long transactions are the most common cause of bloat and lock waits alike.
- **Never do remote I/O inside one.** An HTTP call to a connector inside an open transaction means a slow third party holds your database resources. 03 places that boundary in the service layer.
- **A transaction is not a distributed transaction.** Committing a row and publishing to Kafka are two systems; the outbox pattern in 07 exists because a single local transaction cannot cover both.
- **Commit is the boundary that matters.** As 04 stresses, a successful flush inside a transaction is not durability.

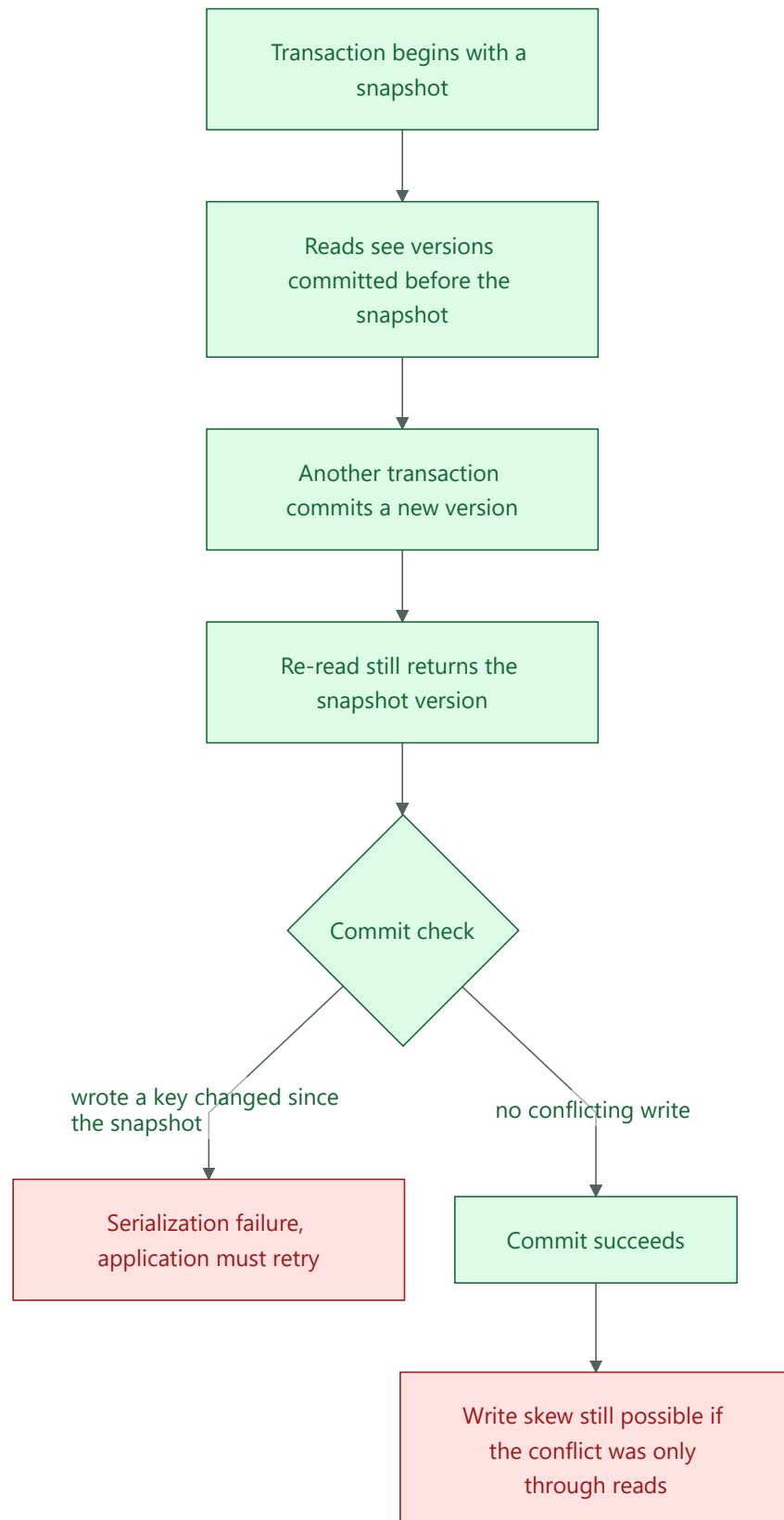
Interview checks: basic: atomic, consistent at boundaries, isolated by level, durable locally. Internals: durability is a log-flush promise about one instance. Debug: long-running transactions explain both lock waits and version retention. Scenario: cross-system atomicity requires an outbox, not a bigger transaction.

6. Isolation Levels, MVCC and the Anomalies That Remain

Why levels exist: perfect isolation means transactions behave as if they ran one at a time, which costs concurrency. Isolation levels are a negotiated trade: each one prevents a specific set of anomalies and permits the rest. Choosing a level is therefore choosing which anomalies your application must handle itself.

Multiversion concurrency control is how most modern engines deliver read isolation without blocking writers. A write creates a new version of a row rather than overwriting it, and each transaction reads the versions visible to its snapshot. Readers do not block writers and writers do not block readers, which is a large practical win. The price is that obsolete versions accumulate and must be cleaned up, which is the subject of the maintenance section.

Diagram 06.5



The Anomalies, Named Precisely

- **Dirty read:** seeing another transaction's uncommitted change. Prevented at read committed and above; MVCC engines commonly do not expose it at all.
- **Non-repeatable read:** re-reading one row and getting a different committed value. Permitted at read committed, prevented by snapshot-based repeatable read.
- **Phantom read:** re-running a range query and finding new rows that match. Classically permitted at repeatable read; engines differ substantially here, which is why the level name alone is not a specification.
- **Lost update:** two transactions read a value, each computes from it, and one result overwrites the other. This is the anomaly that unprotected read-modify-write produces, and the one application developers hit most often.
- **Write skew:** two transactions read an overlapping set, each writes a different row, both commit, and a cross-row invariant breaks. Snapshot isolation permits it, because first-committer-wins only examines written keys.

MvccLab models exactly this progression. A snapshot read stays stable while another transaction commits. An unprotected read-modify-write loses an update. First-committer-wins aborts the stale writer instead, so the application retries from a fresh snapshot. Write skew commits under snapshot isolation because the two transactions wrote different keys. Adding a read-set check turns that write skew into a serialization failure. The lab is a teaching model and has not been executed; it is not a reimplementation of any engine's visibility rules.

Level	Prevents	Still possible	Use when
Read committed	Dirty reads	Non-repeatable reads, phantoms, lost updates from read-modify-write	Ordinary short statements where each statement's own view is enough
Repeatable read or snapshot	Dirty and non-repeatable reads, and under MVCC much phantom behavior	Write skew, and serialization failures you must retry	Multi-statement reads that need a consistent view
Serializable	Anomalies, by definition of the level	Nothing in theory; in practice more aborts and more retry code	Cross-row invariants that cannot be expressed as a constraint
Explicit locking or a constraint	Exactly the conflict you name	Everything you did not name	A specific invariant with a clear owner

The critical corollary: any level above read committed can fail a transaction at commit time with a serialization error. That is not a bug, it is the contract. Code that uses these levels must have a retry path that re-reads from a fresh snapshot, must make that retry safe for external effects as 05 requires, and must bound its retries.

[?] INTERVIEW

Answer isolation questions with anomalies, not level names. Say which anomaly the scenario describes, which level or mechanism prevents it, and what the application must then handle, such as a retry path for serialization failures. Naming lost update and write skew precisely, and distinguishing them, is what separates a memorized answer from an operational one.

[!] TRAP

Isolation level names are not portable specifications. Two engines labelled repeatable read can permit different anomalies, and some engines implement a level more strictly than the standard requires. Verify the behavior of your engine and version rather than reasoning from the name.

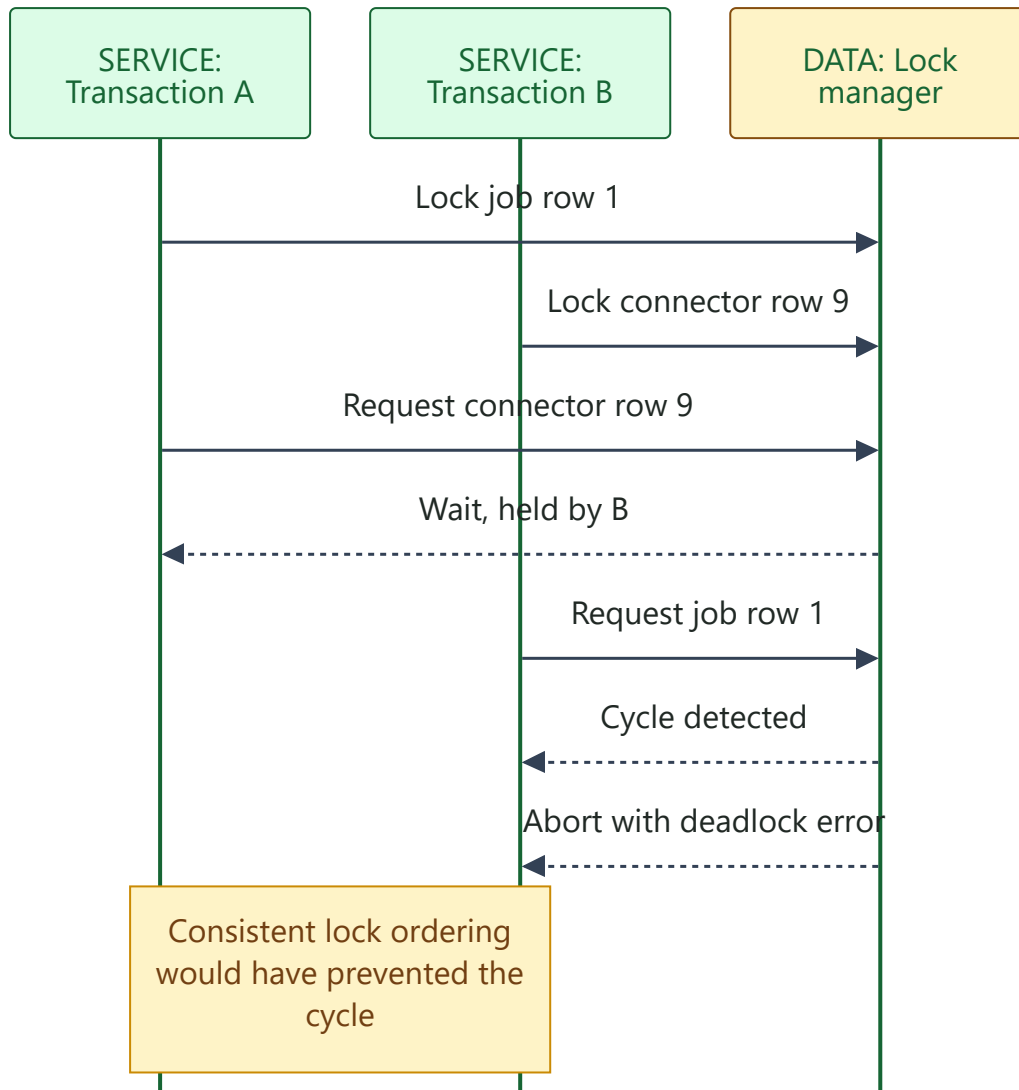
[VERIFY: the exact anomalies permitted at each level in PostgreSQL 17 and MySQL 8.4, PostgreSQL's serializable snapshot isolation behavior, and phantom handling under each engine's repeatable read must be checked against their documentation. No engine behavior was executed or observed here.]

Interview checks: basic: levels are defined by the anomalies they prevent. Internals: MVCC gives non-blocking reads at the cost of version retention. Debug: a lost update means a read-modify-write without a conditional write or a strong enough level. Scenario: write skew needs serializable, a lock, or a constraint that makes the invariant explicit.

7. Locks, Waits and Deadlocks

Why locks remain: MVCC removes read-write blocking, not write-write conflict. Two transactions changing the same row must be ordered, and the engine does that with row locks. Schema changes, maintenance and explicit statements take heavier locks, which is where a quick change becomes an outage.

Diagram 06.6



A **deadlock** is a cycle of waiters, and the engine resolves it by aborting a victim. The application sees an error it must handle; the database has already protected itself. The durable fix is almost always ordering: acquire rows in a consistent order, keep transactions short, and avoid touching more rows than the operation needs. Retrying without changing the ordering simply reproduces the cycle under load.

- **Row lock waits** usually mean a hot row. A per-tenant counter updated by every job is a design problem, not a tuning problem; consider per-partition counters or an aggregate computed from an append-only table.
- **Lock queues cascade.** A blocked transaction can hold locks that block others, so one slow statement becomes a widening wait tree. Statement and lock timeouts turn that into a bounded failure.
- **Explicit row locking** such as a select-for-update is appropriate for short coordination, and dangerous when held across slow work or remote calls.

- **Schema locks are the ones that cause incidents.** An operation requiring an exclusive table lock queues behind running transactions and then blocks every new one, so a change that takes milliseconds can stall a table for the length of the longest open transaction.
- **Advisory locks** coordinate application-level work such as a singleton job, and need the same care about holder lifetime.

Production failure: a migration adds a constraint during business hours. The statement waits behind a long-running analytics query, and every subsequent query queues behind the waiting migration. The outage is caused by lock queueing, not by the migration's own duration. Use a lock timeout, apply changes when long transactions are absent, and prefer the online variants of operations where the engine offers them.

[VERIFY: lock modes, deadlock detection behavior, lock-timeout settings and which DDL operations take blocking locks are strongly engine- and version-specific, including concurrent index creation and constraint validation. Confirm against your engine's documentation; no lock or deadlock was reproduced in this environment.]

Interview checks: basic: deadlock is a wait cycle resolved by aborting a victim. Internals: consistent access ordering prevents it, retries alone do not. Debug: find the blocking statement and its transaction age before tuning anything. Scenario: DDL during peak traffic needs a lock timeout and an awareness of the queue behind it.

8. Vacuum, Bloat, Statistics and Checkpoints

Why maintenance exists: MVCC's obsolete row versions, free space management, planner statistics and the checkpoint cadence all need periodic work. Treating that work as invisible is how a healthy database slowly becomes a slow one.

Bloat is space occupied by dead versions and partially used pages. It grows with update and delete churn, it inflates table and index size, and because the buffer pool caches pages rather than live rows, it reduces effective cache efficiency. In PostgreSQL, vacuum reclaims dead versions for reuse and updates visibility information; in InnoDB, purge threads perform the analogous cleanup against the undo log.

The thing that blocks cleanup is the oldest transaction still able to see those versions. A long-running read transaction, an abandoned session with an open transaction, or a stalled replication slot can hold the horizon back and let bloat grow regardless of how aggressively cleanup runs. That is why "vacuum is not keeping up" is usually a question about who is holding an old snapshot.

Maintenance concern	Symptom	Direction of fix
Bloat from churn	Table and index size growing faster than live rows	Find the oldest open transaction or stalled slot; tune cleanup aggressiveness
Stale statistics	Plans degrade after bulk loads or large deletes	Analyze after bulk changes; review automatic thresholds for large tables
Checkpoint spikes	Periodic I/O stalls and latency bumps	Spread checkpoint writes; revisit the interval against recovery objectives
Index bloat	Index much larger than its data warrants	Rebuild with the concurrent variant where available; remove unused indexes
Transaction-id wraparound pressure	Engine warnings and eventually protective shutdown	Treat as an urgent operational issue, not a tuning preference

Statistics deserve equal attention. After a bulk load, a mass delete or a sudden distribution change, the planner is working from a description of data that no longer exists, and the resulting plan can be orders of magnitude worse. A migration or import pipeline should refresh statistics as a step, not hope that automatic collection notices in time.

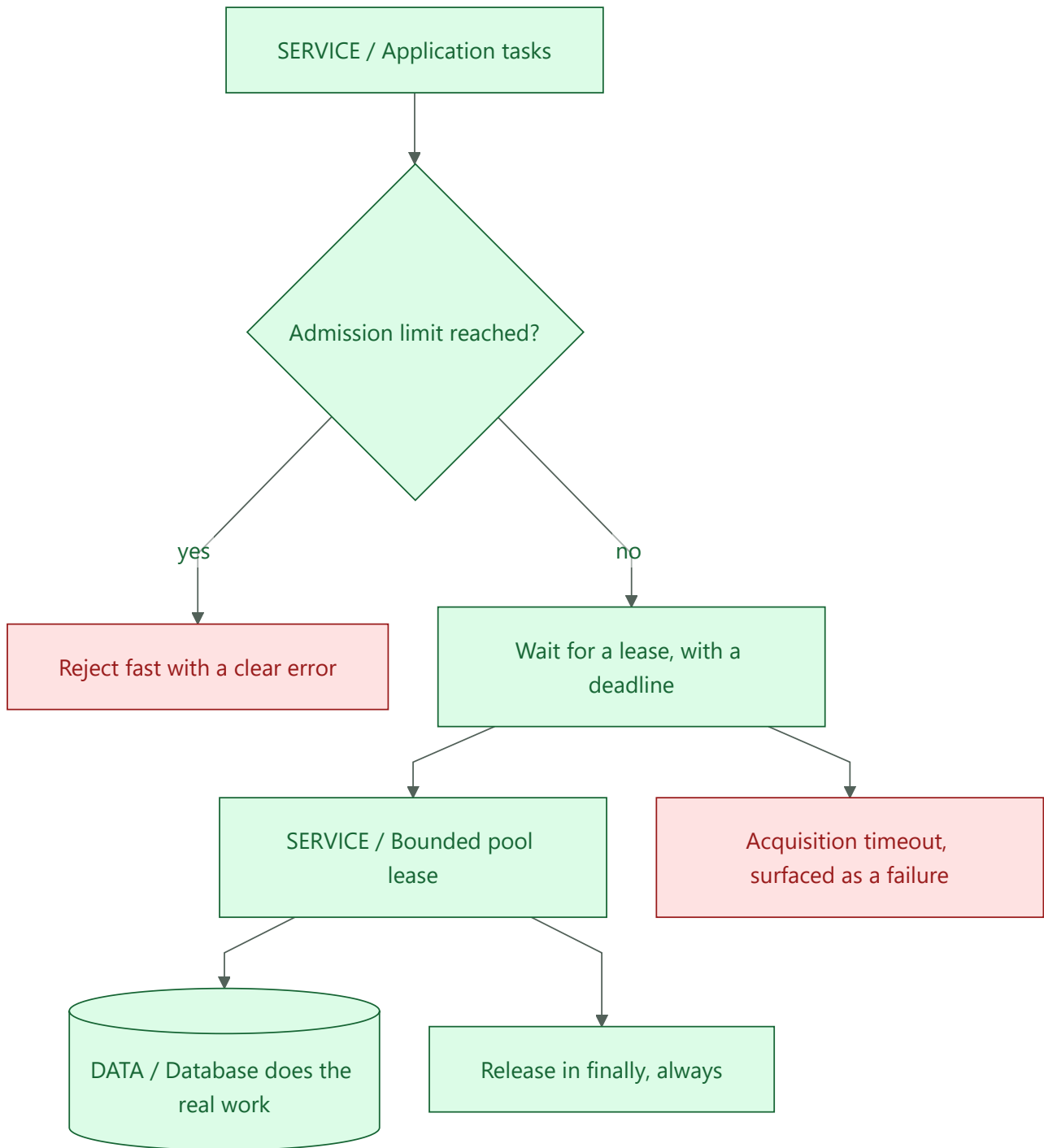
[VERIFY: autovacuum thresholds and cost settings, freeze and wraparound behavior, InnoDB purge, checkpoint tuning and statistics collection defaults are version-specific and interact with workload. Confirm against PostgreSQL 17 and MySQL 8.4 documentation and with your own monitoring; nothing was measured here.]

Interview checks: basic: MVCC creates versions that must be cleaned up. Internals: the oldest snapshot holds back cleanup. Debug: check the oldest transaction and replication slots before blaming the cleanup process. Scenario: refresh statistics as an explicit step in bulk pipelines.

9. Connection Pooling: A Limit, Not Extra Capacity

Why pools exist: establishing a connection is expensive, and every open connection costs the database memory and scheduling. A pool reuses a bounded set of connections, which is the correct design. The misunderstanding is that enlarging it produces more capacity; past the point where the database's CPUs and disks are busy, more concurrent work mostly adds contention and queueing.

Diagram 06.7



`ConnectionPoolLab` models the properties that matter: pool size caps concurrent database work, acquisition fails on a deadline rather than blocking forever, a leaked lease is permanent capacity loss, releasing in a `finally` block preserves capacity when a statement fails, and callers past an admission limit are rejected rather than queued without bound. It is an in-memory model with no driver or database, and it has not been executed.

- **Separate the limits.** Pool size bounds concurrent users of the database; an admission limit bounds how many callers may wait. Without the second, an outage becomes an unbounded backlog, exactly as 02 argues for executors.
- **Always set an acquisition timeout.** A caller waiting indefinitely for a connection converts a database problem into a thread-exhaustion problem upstream.
- **Leaks are permanent.** A connection borrowed and never returned reduces capacity for the process lifetime. Leak detection exists because this failure is silent until saturation.
- **Count every pool.** Several application instances each with a pool, plus workers, plus migration tooling, sum to the database's connection limit. Size against the total, not per service.
- **Virtual threads do not change the arithmetic.** Cheap threads mean more tasks can wait for a connection, not that the database can serve more of them. See 01.

Production failure: latency rises, so the pool is enlarged from twenty to two hundred. The database now has far more concurrent sessions competing for the same CPUs and buffers, context switching and lock contention increase, and median latency gets worse while the symptom looks the same. The real fix was the missing index that made each query ten times more expensive than it needed to be.

[VERIFY: pool defaults, leak-detection behavior, timeout semantics and recommended sizing formulas differ between pool implementations and engines, and sizing guidance should be derived from measurement rather than a formula. Confirm against the selected pool's documentation; no pool was run or measured here.]

Interview checks: basic: a pool bounds concurrency and reuses connections. Internals: admission and concurrency are two separate limits. Debug: acquisition timeouts point at slow queries or leaks, not at pool size alone. Scenario: fix the query before enlarging the pool.

10. Replication: Copies With a Delay

Why it exists: a single instance is a single point of failure and a single pool of read capacity. Replication maintains copies for failover, read scaling and feeding other systems. Every form of it introduces a delay, and most replication incidents are really incidents about that delay being assumed away.

Streaming (physical) replication ships the write-ahead log to replicas that apply it, producing a byte-level copy of the primary. It is the standard mechanism for high availability and read replicas. Replicas are read-only, follow the primary's exact structure, and lag by the time it takes to ship and apply records.

Logical replication decodes the log into row-level change events for selected tables and publishes them to consumers that need not be identical databases. It enables selective replication, cross-version movement and change data capture into Kafka or a search index, as 07 and 26 describe. Its cost is a replication slot whose backlog retains log on the primary if the consumer stalls, which is a common cause of a primary unexpectedly running out of disk.

Mechanism	Use when	Avoid when
Synchronous replica	A committed transaction must exist on another instance before acknowledgement	Commit latency and availability coupling are not acceptable for that workload
Asynchronous replica	Failover capability and read scaling with tolerable lag	Reads require the user's own write to be visible immediately
Logical replication or CDC	Feeding search indexes, analytics or other services from durable changes	Nobody monitors slot lag and the retained log fills the primary's disk
Reading from a replica	Analytics, exports, reporting and other lag-tolerant reads	A read-after-write path such as showing the job a user just created

Read-after-write is the trap. A user creates a job, the UI immediately reads from a replica, and the row is not there yet. The result is a support ticket about data loss that is actually replication lag. Route reads that must observe the caller's own write to the primary, or use the engine's mechanism for waiting until a replica has caught up to a known position. 09 and 22 cover the consistency vocabulary for this.

Synchronous replication is not free availability. Requiring an acknowledgement from another instance adds its latency to every commit and couples your availability to that instance's health. It is the right choice when losing a committed transaction is unacceptable, and it must be chosen deliberately with a failure plan for the replica being unavailable.

[VERIFY: synchronous commit modes, replica consistency options, slot management, failover tooling and lag monitoring differ by engine, version and managed-service provider. Confirm against your deployment; no replication topology was configured or observed here.]

Interview checks: basic: replicas lag, including by small but nonzero amounts. Internals: physical replication ships log records, logical replication decodes row changes. Debug: a missing just-created row is replication lag far more often than a lost write. Scenario: never serve read-after-write from an asynchronous replica.

11. Partitioning and Sharding

Why they exist: tables and workloads eventually outgrow convenient maintenance on a single machine. Partitioning splits one logical table into physical pieces within one database; sharding spreads data across independent databases. They solve different problems and have very different costs.

Partitioning helps when it matches the access pattern. Range partitioning by time lets old partitions be detached and dropped instantly instead of deleting millions of rows, and lets queries with a time predicate skip irrelevant partitions entirely. It does not help a query that must touch every partition, and it adds planning overhead and maintenance obligations such as creating future partitions before they are needed.

Sharding is a distributed-systems decision disguised as a database decision. Once data lives in independent databases, cross-shard queries, cross-shard transactions, global uniqueness and rebalancing all become application concerns. The shard key becomes the most consequential and least reversible choice in the system, and a poorly chosen one produces hot shards that defeat the entire exercise.

Technique	Use when	Avoid when
Time-range partitioning	Retention and time-scoped queries dominate, as with job history	Queries rarely filter by time and every partition is scanned anyway
Partitioning by tenant	A few very large tenants need isolation of maintenance and plans	Thousands of small tenants create thousands of partitions to manage
Read replicas first	Read capacity is the constraint and lag is acceptable	Write throughput or data volume is the actual limit
Sharding	A single instance genuinely cannot hold the data or the write rate	Vertical scaling, archiving, indexing and caching have not been exhausted

[D] DECISION

Exhaust the reversible options before the irreversible one. Indexing, archiving, vertical scaling, read replicas and time partitioning can all be undone in an afternoon. A shard key cannot. Reach for sharding only when a single instance genuinely cannot hold the data or absorb the write rate, and say which measurement established that.

The ordered path matters. Fix the queries and indexes, archive what retention does not require, scale the instance, add read replicas, partition by time, and only then consider sharding. For IntegrationHub, tenant identity is the natural shard key because it already scopes authorization

and almost every query, but the honest assessment is that archiving and partitioning solve the problem long before sharding is justified. 10 and 12 treat the distributed design that follows.

[VERIFY: partition pruning behavior, partition-wise joins, limits on partition counts, foreign-key and unique-constraint support across partitions, and online repartitioning differ substantially by engine and version. Confirm before designing a partitioning scheme; nothing was created or measured here.]

Interview checks: basic: partitioning is within one database, sharding spans several. Internals: partitioning pays off through pruning and cheap retention. Debug: a hot shard usually traces to an uneven shard key. Scenario: exhaust the cheaper options in order before sharding.

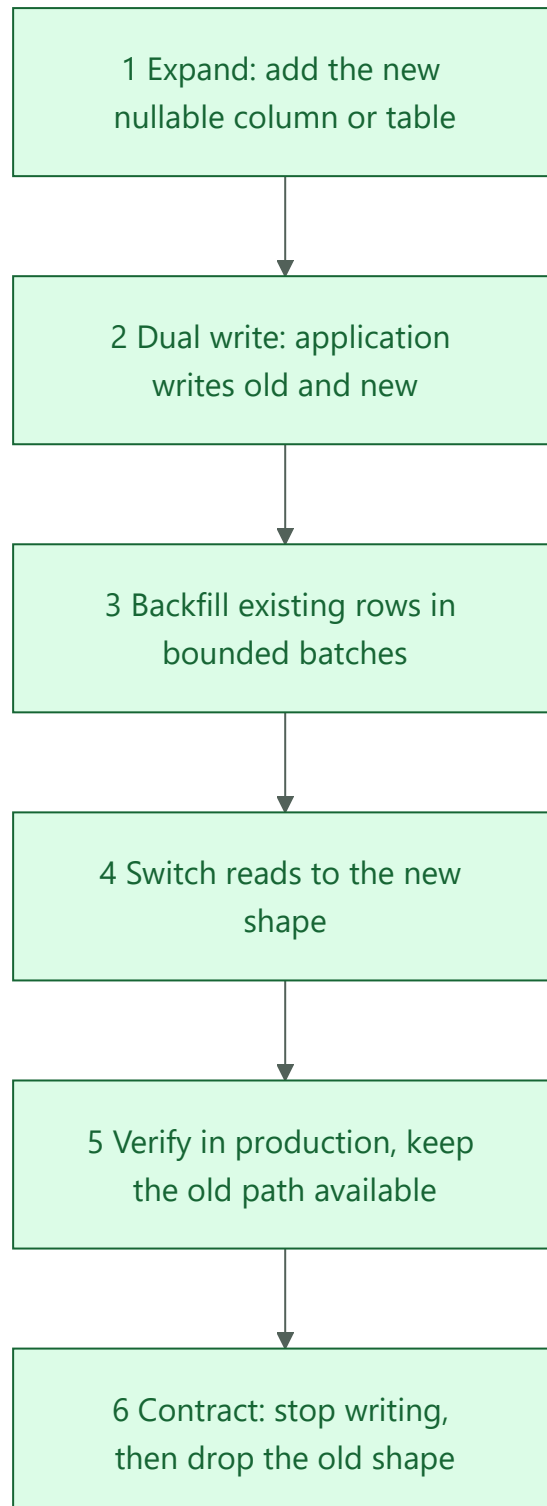
12. Migrations and Zero-Downtime Schema Change

Why migration tooling exists: schema changes must be versioned, ordered, repeatable across environments and visible in review, exactly like application code. Running SQL by hand into production is how environments drift until nobody can reproduce an incident.

Flyway applies ordered, versioned SQL scripts and records what ran in a schema-history table; its model is simple, SQL-centric and easy to reason about. **Liquibase** describes changes as changesets in several formats, supports preconditions, contexts, labels and rollback definitions, and offers more abstraction at the cost of more concepts. Both solve the same core problem: a recorded, ordered, idempotent application of schema change. Choose one and apply it consistently, including for the production-like environments where you rehearse.

Expand and Contract

Diagram 06.8



The reason for the sequence is that the application and the schema deploy at different moments, and both the old and the new version of the code run at the same time during a rolling release. Every intermediate state must therefore be compatible with both versions. A

rename executed as a single statement breaks that rule, which is why a rename becomes add, dual-write, backfill, switch, and drop, spread over multiple releases.

- **Add columns as nullable, or with a default the engine can apply cheaply.** Verify how your engine handles adding a column with a default on a large table before assuming it is instant.
- **Backfill in bounded batches with commits between them.** One enormous update holds locks, generates a huge amount of log and blocks cleanup.
- **Create indexes with the concurrent or online variant** where the engine offers one, and expect it to take longer and to be able to fail and leave an invalid index behind.
- **Validate constraints separately from adding them** where the engine supports a not-valid-then-validate sequence, so the blocking step is short.
- **Dropping is a separate, later release.** Keep the old shape until you are certain no running code reads it, because reverting a deploy must not require reverting a schema.

Rollback deserves honesty. Many schema changes cannot be rolled back without data loss, so the real safety mechanism is that every intermediate state is compatible with the previous application version. A rollback plan that says "revert the migration" for a dropped column is not a plan. [16](#) covers the release process this implies.

[VERIFY: engine-specific behavior for adding columns with defaults, concurrent index creation, not-valid constraint validation, and the exact locking of each DDL statement must be confirmed per engine and version before planning an online change. No migration was executed in this environment.]

Interview checks: basic: migrations are versioned, ordered and recorded. Internals: rolling deploys mean two code versions share one schema. Debug: a failed concurrent index can leave an invalid index requiring cleanup. Scenario: a rename is a multi-release expand-and-contract sequence.

13. Backups, Point-in-Time Recovery and Restore Testing

Why this is the most important section operationally: replication protects against instance failure, not against a mistaken DELETE, a bad migration or a corrupted write, because replicas faithfully replicate the mistake. Backups are the only defence against logical damage, and an untested backup is a hypothesis.

State the objectives in numbers. The **recovery point objective** is how much data you accept losing, which determines backup frequency and log archiving. The **recovery time objective** is how long restoration may take, which determines the backup format, storage location and

rehearsal. "We take nightly backups" is not an objective; it is a schedule that implies losing up to a day.

Point-in-time recovery combines a base backup with archived write-ahead log, so you can restore to a moment just before the damaging statement. It is the mechanism that turns "we deleted the wrong tenant's jobs at 14:32" into a recoverable event. It requires continuous log archiving, enough retention to cover your detection window, and a rehearsed procedure.

Protection	Covers	Does not cover
Replica	Instance or disk failure, planned failover	Mistaken deletes, bad migrations, logical corruption, which replicate faithfully
Periodic full backup	Loss of the dataset, with loss up to the backup age	Anything written after the backup, unless logs are archived
Continuous log archive plus base backup	Point-in-time recovery to just before an incident	Damage detected after log retention has expired
Logical export	Selective restore, version migration, single-table recovery	Large-database recovery time objectives, usually far too slow
Snapshot of a cloud volume	Fast whole-instance restoration	Correctness, unless the snapshot is consistent and has been tested

Test the restore, on a schedule, into a separate environment. Measure how long it takes, verify row counts and application startup against the restored data, and record the result. Restore rehearsal is also how you discover that the backup excluded a tablespace, that the encryption key is not available in the recovery environment, or that recovery takes nine hours against a four-hour objective. 18 owns that exercise and the runbook it produces.

Backups also carry every data-protection obligation the production database has. They contain personal data, they must be encrypted, their access must be audited, and retention has to satisfy both deletion requirements and recovery requirements, which can conflict. 08 owns that tension.

[VERIFY: base-backup tooling, log-archiving configuration, retention mechanics and managed-service restore behavior vary by engine, version and provider, and restore duration is specific to your data volume and storage. Confirm and measure in your environment; no backup or restore was performed here.]

Interview checks: basic: replication is not backup. Internals: point-in-time recovery needs a base backup plus archived log. Debug: an untested backup is an assumption, so rehearse restores. Scenario: state recovery point and time objectives in numbers and verify them.

14. Choosing a Store: PostgreSQL, MySQL, MongoDB, Redis, Elasticsearch

Why the comparison matters: the useful question is never "which database is best" but "which store owns which fact". IntegrationHub's answer is that PostgreSQL owns authority and everything else is derived, which keeps the consistency reasoning simple and makes every other store disposable and rebuildable.

Store	Genuine strength	Use when	Avoid when
PostgreSQL	Rich SQL, strong constraints, extensible types, serializable isolation available	Transactional authority for jobs, idempotency, outbox and progress	A workload is genuinely a cache, a full-text search engine or an analytics warehouse
MySQL with InnoDB	Mature, widely operated, strong replication ecosystem	The organization's operational expertise and tooling already live there	A specific PostgreSQL feature is central to the design
MongoDB	Flexible documents, horizontal scaling model	Genuinely variable documents with access patterns designed around them	It is chosen to avoid schema design, leaving invariants with no enforcement
Redis	In-memory data structures, very low latency	Caching, rate-limit counters, ephemeral coordination and short-lived state	It becomes the authority for data you cannot afford to lose
Elasticsearch	Inverted-index search, relevance, aggregation over text	Search over job and log data fed from a durable source	It is treated as a system of record rather than a derived index

The rule that keeps this manageable is the authority-versus-derived distinction from 00. A derived store may lag, may be rebuilt from the authority, and must never be the only place a fact exists. If you cannot rebuild it, it is not derived, and it now carries the backup, consistency and durability obligations of an authority whether you planned for that or not.

Polyglot persistence has a real cost that is easy to underestimate: every store needs backups, monitoring, upgrades, security review, capacity planning and on-call expertise. Two stores is more than twice the operational surface of one. Add a store when a workload genuinely cannot be served by the existing one, and write down who rebuilds it when it is lost.

[VERIFY: feature claims for MongoDB, Redis and Elasticsearch, including their durability and transaction options, change across versions and deployment modes. Confirm against current documentation before relying on any of them; no store was installed or exercised in this environment.]

Interview checks: basic: choose a store per workload, not per fashion. Internals: authority versus derived decides consistency obligations. Debug: a derived store that cannot be rebuilt has quietly become an authority. Scenario: every added store multiplies operational work.

15. Labs and Execution Status

Two self-contained Java sources live under `code/06-databases/`, with `run.ps1`, `README.md` and `execution.json`. The workspace command is `& ' ./docs/java-fs-guide/code/06-databases/run.ps1'`, and an already-installed JDK can be supplied through the optional `-JdkHome` parameter. No downloads are attempted and no database is required, because neither lab touches one.

java

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

/** Chapter 06 lab: what a snapshot shows, and which anomalies a snapshot alone does not prevent. */
public final class MvccLab {

    /**
     * How a transaction is checked at commit time.
     * LAST_WRITE_WINS models an application doing read-modify-write with no protection at all.
     * SNAPSHOT models first-committer-wins on written keys, which is what snapshot isolation gives you.
     * SNAPSHOT_WITH_READ_CHECK additionally rejects a commit whose read set moved, approximating
     * serializable snapshot isolation. It is a teaching model, not PostgreSQL's implementation.
     */
    enum Mode { LAST_WRITE_WINS, SNAPSHOT, SNAPSHOT_WITH_READ_CHECK }

    record Version(long value, long commitTime) { }

    static final class AbortedException extends RuntimeException {
        AbortedException(String reason) {
            super(reason);
        }
    }

    /** A tiny multi-version store. Each key keeps every committed version with its commit time. */
    static final class Store {
        private final Map<String, List<Version>> history = new HashMap<>();
        private long clock;

        Store seed(String key, long value) {
            history.computeIfAbsent(key, ignored -> new ArrayList<>()).add(new Version(value, ++clock));
            return this;
        }

        long now() {
            return clock;
        }

        /** The value a transaction started at snapshotTime is entitled to see. */
        long valueAt(String key, long snapshotTime) {
            List<Version> versions = history.get(key);
            if (versions == null) {
                throw new IllegalArgumentException("unknown key " + key);
            }
        }
    }
}
```

```

        long visible = Long.MIN_VALUE;
        for (Version version : versions) {
            if (version.commitTime() <= snapshotTime) {
                visible = version.value();
            }
        }
        if (visible == Long.MIN_VALUE) {
            throw new IllegalStateException("no version visible to this snapshot");
        }
        return visible;
    }

    /** The commit time of the newest version of a key, used to detect concurrent change. */
    long lastChange(String key) {
        List<Version> versions = history.get(key);
        return versions.get(versions.size() - 1).commitTime();
    }

    Transaction begin(Mode mode) {
        return new Transaction(this, mode, clock);
    }

    private void append(Map<String, Long> writes) {
        long commitTime = ++clock;
        for (Map.Entry<String, Long> write : writes.entrySet()) {
            history.computeIfAbsent(write.getKey(), ignored -> new ArrayList<>())
                .add(new Version(write.getValue(), commitTime));
        }
    }
}

static final class Transaction {
    private final Store store;
    private final Mode mode;
    private final long snapshotTime;
    private final Set<String> readSet = new HashSet<>();
    private final Map<String, Long> writes = new HashMap<>();
    private boolean finished;

    Transaction(Store store, Mode mode, long snapshotTime) {
        this.store = store;
        this.mode = mode;
        this.snapshotTime = snapshotTime;
    }

    long read(String key) {
        readSet.add(key);
        if (writes.containsKey(key)) {
            return writes.get(key);
        }
        return mode == Mode.LAST_WRITE_WINS ? store.valueAt(key, store.now()) : store.valueAt(key, snapshotTime);
    }

    void write(String key, long value) {
        writes.put(key, value);
    }

    void commit() {
        if (finished) {
            throw new IllegalStateException("transaction already finished");
        }
        finished = true;
        if (mode != Mode.LAST_WRITE_WINS) {
            for (String key : writes.keySet()) {
                if (store.lastChange(key) > snapshotTime) {
                    throw new AbortedException("write-write conflict on " + key);
                }
            }
        }
        if (mode == Mode.SNAPSHOT_WITH_READ_CHECK) {
            for (String key : readSet) {
                if (store.lastChange(key) > snapshotTime) {

```

```

        throw new AbortedException("read set changed under this snapshot: " + key);
    }
}
}
store.append(writes);
}
}

static void check(boolean condition, String description) {
    if (!condition) {
        throw new AssertionError(description);
    }
}

static void snapshotReadIsStableWhileAnotherTransactionCommits() {
    Store store = new Store().seed("job-1.processed", 100);
    Transaction reader = store.begin(Mode.SNAPSHOT);
    check(reader.read("job-1.processed") == 100, "the reader starts from the committed value");

    Transaction writer = store.begin(Mode.SNAPSHOT);
    writer.write("job-1.processed", 150);
    writer.commit();

    check(reader.read("job-1.processed") == 100, "a snapshot does not move because someone else committed");
    check(store.valueAt("job-1.processed", store.now()) == 150, "a transaction starting now sees the new value");
}

static void readModifyWriteWithoutProtectionLosesAnUpdate() {
    Store store = new Store().seed("connector-7.quota", 100);
    Transaction first = store.begin(Mode.LAST_WRITE_WINS);
    Transaction second = store.begin(Mode.LAST_WRITE_WINS);

    long firstRead = first.read("connector-7.quota");
    long secondRead = second.read("connector-7.quota");
    check(firstRead == 100 && secondRead == 100, "both transactions read the same starting value");

    first.write("connector-7.quota", firstRead - 10);
    first.commit();
    second.write("connector-7.quota", secondRead - 10);
    second.commit();

    check(store.valueAt("connector-7.quota", store.now()) == 90, "two decrements of ten produced one decrement");
}

static void snapshotIsolationAbortsTheSecondWriterInstead() {
    Store store = new Store().seed("connector-7.quota", 100);
    Transaction first = store.begin(Mode.SNAPSHOT);
    Transaction second = store.begin(Mode.SNAPSHOT);

    first.write("connector-7.quota", first.read("connector-7.quota") - 10);
    first.commit();

    second.write("connector-7.quota", second.read("connector-7.quota") - 10);
    boolean aborted = false;
    try {
        second.commit();
    } catch (AbortedException expected) {
        aborted = true;
    }
    check(aborted, "first committer wins, so the stale writer must be told to retry");
    check(store.valueAt("connector-7.quota", store.now()) == 90, "the losing write was not applied");

    Transaction retry = store.begin(Mode.SNAPSHOT);
    retry.write("connector-7.quota", retry.read("connector-7.quota") - 10);
    retry.commit();
    check(store.valueAt("connector-7.quota", store.now()) == 80, "the retry recomputes from a fresh snapshot");
}

static void writeSkewCommitsUnderSnapshotIsolation() {
    // Invariant the application believes it has: at least one connector stays enabled.
    Store store = new Store().seed("connector-a.enabled", 1).seed("connector-b.enabled", 1);
    Transaction first = store.begin(Mode.SNAPSHOT);

```

```

Transaction second = store.begin(Mode.SNAPSHOT);

check(first.read("connector-a.enabled") + first.read("connector-b.enabled") >= 2, "first sees two enabled");
check(second.read("connector-a.enabled") + second.read("connector-b.enabled") >= 2, "second sees two enabled");

first.write("connector-a.enabled", 0);
second.write("connector-b.enabled", 0);
first.commit();
second.commit();

long remaining = store.valueAt("connector-a.enabled", store.now()) + store.valueAt("connector-b.enabled", store.now());
check(remaining == 0, "both commits succeeded because they wrote different keys, and the invariant broke");
}

static void readSetCheckingRejectsTheSecondWriteSkewCommit() {
    Store store = new Store().seed("connector-a.enabled", 1).seed("connector-b.enabled", 1);
    Transaction first = store.begin(Mode.SNAPSHOT_WITH_READ_CHECK);
    Transaction second = store.begin(Mode.SNAPSHOT_WITH_READ_CHECK);

    first.read("connector-a.enabled");
    first.read("connector-b.enabled");
    second.read("connector-a.enabled");
    second.read("connector-b.enabled");

    first.write("connector-a.enabled", 0);
    first.commit();

    second.write("connector-b.enabled", 0);
    boolean aborted = false;
    try {
        second.commit();
    } catch (AbortedException expected) {
        aborted = true;
    }
    check(aborted, "checking the read set is what turns write skew into a serialization failure");
    long remaining = store.valueAt("connector-a.enabled", store.now()) + store.valueAt("connector-b.enabled", store.now());
    check(remaining == 1, "the invariant survived because the application must retry, not because writes are magic");
}

public static void main(String[] args) {
    snapshotReadIsStableWhileAnotherTransactionCommits();
    readModifyWriteWithoutProtectionLosesAnUpdate();
    snapshotIsolationAbortsTheSecondWriterInstead();
    writeSkewCommitsUnderSnapshotIsolation();
    readSetCheckingRejectsTheSecondWriteSkewCommit();
    System.out.println("MvccLab checks completed");
}
}

```

Lab	What its assertions cover	What it does not claim
MvccLab	Snapshot stability under a concurrent commit, lost update without protection, first-committer-wins abort and retry, write skew under snapshot isolation, and read-set checking turning write skew into a serialization failure	Not PostgreSQL, MySQL or any engine's visibility rules, predicate locking, phantom handling or performance
ConnectionPoolLab	Pool size caps concurrency, acquisition fails on a deadline, a leaked lease is permanent capacity loss, release in <code>finally</code> preserves capacity, and an admission limit rejects rather than queueing without bound	Not a JDBC, driver or database test; no throughput, latency or sizing guidance is measured

NOT EXECUTED: the runner's preflight ran and reported that no JDK compiler is reachable through PATH or JAVA_HOME, and downloads are not authorized. Nothing was compiled, no assertion was evaluated and no output was produced. The inline `MvccLab` listing is checked against its saved source, which verifies synchronization rather than correctness.

What a real environment should add, once tools exist: the isolation scenarios run against an actual PostgreSQL with two sessions, so the engine's own behavior at each level is observed rather than modelled; plans captured with actual row counts on representative data; a lock and deadlock reproduction; a backfill rehearsal measuring lock duration and log volume; and a timed restore from backup. 19 covers how those become repeatable tests.

A Repeatable Diagnosis

- State the failing property: wrong result, slow query, lock wait, growing table, saturated pool, stale replica read or lost data.
- Separate the layer. Query cost is a plan and index question; wrong concurrent results are an isolation question; waits are a locking question; growth is a cleanup question; saturation is usually an upstream query cost question.
- For slow queries, capture the plan with actual rows at representative volume and compare estimates against actuals before changing anything.
- For concurrency bugs, write down the exact interleaving and name the anomaly. Lost update and write skew need different fixes.
- For waits, identify the blocking transaction and its age, then look at access ordering rather than at the victim statement.
- For growth, find the oldest open transaction or stalled replication slot before tuning cleanup settings.
- For pool saturation, measure query duration first. A pool is almost never the cause; it is where the cost becomes visible.
- For missing rows after a write, check which instance served the read before suspecting data loss.
- Fix the owning layer, add the constraint or index that makes the mistake impossible, and write a test that fails for that specific case.

Explained-Here Index and Sources

Concept	Explanation
Declarative queries, constraints and modeling	Relational model
B-trees, LSM trees, buffer pool and write-ahead logging	Storage engines
Access paths, composite and partial indexes, covering reads	Indexes

Concept	Explanation
Plans, estimates versus actual rows, join choices	Reading plans
Atomicity, durability scope and transaction discipline	Transactions
Isolation levels, MVCC, lost update and write skew	Isolation
Row locks, lock queues, deadlocks and DDL locking	Locks
Vacuum, bloat, statistics and checkpoints	Maintenance
Pool sizing, admission limits, timeouts and leaks	Connection pooling
Streaming and logical replication, lag and read-after-write	Replication
Partition pruning, retention and the shard-key decision	Partitioning and sharding
Flyway, Liquibase and expand-and-contract releases	Migrations
Recovery objectives, point-in-time recovery and restore testing	Backups
PostgreSQL, MySQL, MongoDB, Redis and Elasticsearch roles	Choosing a store
Lab scope and actual execution status	Labs

Owning references for further checking: the [PostgreSQL 17 documentation](#), particularly its transaction isolation and routine maintenance chapters; the [MySQL 8.4 reference manual](#); the [Flyway documentation](#) and [Liquibase documentation](#). These are where to verify the mechanisms described above; citing them is not a claim that this environment retrieved or executed them.

Related Chapters

This chapter sits under [04 persistence](#) and behind [05 API contracts](#) and [03 service transactions](#), with [01 Java](#) and [02 concurrency](#) underneath. [07 messaging](#) depends on the outbox committing here, [26 data platform](#) consumes changes from the log, and [08 security](#) owns tenant isolation and encryption. [10 system design](#), [12 case studies](#), [15 Kubernetes](#), [16 delivery](#), [17 cloud](#), [18 operations](#), [19 testing](#), [22 distributed systems](#) and [09 frontend consistency](#) extend the boundaries touched here. Return to [00](#) for the whole request journey. Generated Related chapters links are reciprocal.

RELATED CHAPTERS / RECIPROCAL LINKS

00 The System Before the Details	05 APIs and Real-Time Communication
02 Concurrency and the Memory Model	07 Messaging, Batch and Streams
21 Networking and Operating Systems	08 Security, Identity and Data Protection
03 Spring and Spring Boot Internals	15 Kubernetes and Production Workloads
04 Hibernate and JPA	17 Cloud and Infrastructure

18	Observability and Reliability
10	System Design Foundations
26	Data Platform Basics
22	Distributed Systems Theory
01	Core Java and the JVM
09	The Browser and the Java Backend
12	Twelve System Design Case Studies

16	Build, Ship and Roll Back
19	Testing and Quality
27	Fintech and Payments Engineering
13	Data Integration and iPaaS
23	JVM Performance Engineering
24	IntegrationHub in Motion
20	Appendix: Interview Toolkit and Index

One-Page Cheat Sheet

Model: SQL is declarative and the planner picks the path. Constraints are the only rules every writer obeys. NULL is unknown, not a value. Use a column when the database must enforce or index the field.

Storage: the write-ahead log makes commit durable; data files are written later at checkpoints. Buffer-pool residency governs read cost. B-trees update in place and favor ordered reads; LSM trees append and compact, favoring write-heavy ingest.

Indexes: design from predicates plus ordering. Leading columns and selectivity decide usability; functions and implicit conversions defeat it; covering indexes avoid table lookups. Every index costs writes, memory and space, so remove unused ones.

Plans: read EXPLAIN with actual rows. The estimate-versus-actual gap is the diagnosis; timing only says where. Nested loops over many rows mean a bad estimate upstream. Verify at representative volume and skew.

Transactions: atomic, constraint-consistent at boundaries, isolated by level, durable on one instance. Keep them short, never wrap remote I/O, and use an outbox for cross-system atomicity.

Isolation: read committed permits lost update through read-modify-write. Snapshot isolation permits write skew. Serializable aborts instead, so you need a retry path. Level names are not portable; verify your engine. MVCC gives non-blocking reads and creates versions to clean up.

Locks: writers still conflict. Deadlocks are wait cycles resolved by aborting a victim, prevented by consistent access ordering. Lock timeouts bound the damage. DDL queueing behind a long transaction blocks everything after it.

Maintenance: bloat follows churn, and the oldest open transaction or stalled slot holds cleanup back. Refresh statistics after bulk changes. Checkpoint cadence trades steady I/O against recovery time.

Pooling: the pool is a limit, not capacity. Separate admission from concurrency, always set an acquisition timeout, release in `finally`, count every pool against the server limit, and fix the slow query before enlarging anything.

Scale and safety: replicas lag, so never serve read-after-write from an async replica. Logical slots retain log and can fill the primary's disk. Partition before sharding, and pick the shard key last and carefully. Migrations are expand, dual-write, backfill, switch, contract. Replication is not

backup; point-in-time recovery needs archived log; an untested restore is a hypothesis. The Java labs remain NOT EXECUTED.

Interview Corner

Basic: What Does a Write-Ahead Log Give You?

Durability and crash recovery. The change record is flushed before the modified data page, so a crash is repaired by replaying the log from the last checkpoint. It also makes commit cheap, because only the log must be flushed, and it is the stream replication consumes.

Internals: Why Is the Buffer Pool Hit Ratio So Influential?

Because it decides whether a read is a memory access or a storage access, which differ by orders of magnitude. What matters is the working set relative to memory, not total database size, which is also why bloat hurts twice: it wastes space and it wastes cache.

Trace/Debug: Commits Suddenly Became Slow. Where Do You Look First?

At log flush and storage latency, then at whether synchronous replication is waiting on a replica, then at checkpoint activity. Commit cost is dominated by making the log durable, so the investigation starts there rather than at the query.

Scenario: When Is an LSM-Based Store a Better Fit Than a B-Tree?

When ingest is write-heavy and random, because appending and compacting converts random writes into sequential ones. You accept background compaction cost and potentially higher read amplification, so it is a poor trade for read-heavy ordered access.

Basic: Why Might the Database Ignore Your Index?

Because the planner estimates a scan is cheaper, usually because the predicate is unselective, or because the index is unusable: wrong leading column, a function or implicit conversion around the column, or statistics that misdescribe the data.

Internals: What Makes a Composite Index Usable for a Query?

The query's predicates must match the index's leading columns in order, and the index ordering can additionally satisfy an ORDER BY. An index on tenant, status, created-at serves a tenant-and-status query ordered by creation time, and does nothing for a status-only query.

Trace/Debug: A Query Is Slow Only in Production. What Is Your Sequence?

Capture the plan with actual rows there, compare estimates against actuals, check statistics freshness and data skew, confirm the index exists and is usable, and check whether the working set fits in memory. Volume and distribution, not the SQL text, are usually the difference.

Scenario: Which Indexes Does a Keyset-Paginated Job List Need?

One supporting the tenant filter plus the total ordering key used by the cursor, so the seek is an index operation rather than a scan and sort. If the response needs only indexed columns, making it covering avoids the table lookups entirely.

Basic: What Does Each Letter of ACID Actually Promise?

Atomicity: all or nothing. Consistency: declared constraints hold at boundaries, not that your business logic is correct. Isolation: a configurable level, not one guarantee. Durability: a committed transaction survives a crash of that instance, not loss of the machine or a mistaken delete.

Internals: What Is MVCC and What Does It Cost?

Writers create new row versions instead of overwriting, and each transaction reads versions visible to its snapshot, so readers and writers do not block each other. The cost is that obsolete versions accumulate and must be cleaned up, and the oldest live snapshot delays that cleanup.

Trace/Debug: Two Users Updated a Counter and One Change Vanished. Name and Fix It.

That is a lost update from read-modify-write. Fix it with a conditional update that includes the expected value or version, an atomic in-database update, an appropriate isolation level with a retry path, or an explicit row lock for the duration of the read and write.

Scenario: An Invariant Spans Two Rows and Keeps Breaking. What Do You Do?

That is write skew, which snapshot isolation permits because the transactions write different rows. Options are serializable isolation with a retry path, explicit locking of the rows that carry the invariant, or restructuring so a single constraint or single row expresses the rule.

Basic: What Causes a Deadlock and How Does the Database Respond?

Two or more transactions wait on each other in a cycle. The engine detects the cycle and aborts a victim, which surfaces to the application as an error to handle. The durable fix is consistent access ordering plus shorter transactions.

Internals: Why Can a Quick DDL Statement Stall an Entire Table?

Because it needs a lock that conflicts with running transactions, so it waits, and new queries queue behind the waiting lock request. The duration of the outage is set by the longest open transaction, not by the DDL's own execution time.

Trace/Debug: A Table Is Growing Far Faster Than Its Live Rows. Why?

Bloat from update and delete churn whose dead versions cannot be reclaimed. Look for the oldest open transaction, an idle-in-transaction session, or a stalled replication slot holding the visibility horizon back, before changing cleanup settings.

Scenario: How Would You Add a Unique Constraint to a Large Live Table?

Check for and resolve existing duplicates, build the index with the engine's concurrent or online variant, validate separately from adding where that is supported, and use a lock timeout so the change fails fast instead of queueing. Expect it to take time and to be retryable.

Basic: Why Not Simply Make the Connection Pool Much Larger?

Because the pool does not add database capacity. Past the point where the server's CPUs and storage are saturated, more concurrent sessions add contention, context switching and lock conflicts, so latency usually gets worse while the symptom looks unchanged.

Internals: What Two Limits Should a Pool Express?

Concurrency, which bounds simultaneous users of the database, and admission, which bounds how many callers may queue for one. Without the second, a slow database turns into an unbounded backlog of waiting work upstream.

Trace/Debug: Connection Acquisition Timeouts Appeared Overnight. What Changed?

Usually query duration, because connections are held longer. Look for a plan change after a data-volume shift or statistics refresh, a new long transaction, a leak that has reduced effective capacity, or more application instances sharing the same server limit.

Scenario: Do Virtual Threads Let You Shrink or Remove the Pool?

No. Cheap threads mean more tasks can wait for a connection; they do not make the database serve more work. The pool remains the limiting resource, and removing the bound simply moves saturation into the database itself.

Basic: Is a Read Replica a Backup?

No. A replica faithfully reproduces mistaken deletes, bad migrations and corruption. It protects against instance and disk failure. Protection against logical damage requires backups and point-in-time recovery, tested by actually restoring.

Internals: Streaming Versus Logical Replication?

Streaming replication ships write-ahead log records to produce a byte-identical replica, used for failover and read scaling. Logical replication decodes the log into row-level changes for selected tables, enabling selective and cross-system consumption at the cost of a slot whose backlog retains log on the primary.

Trace/Debug: A User Says the Job They Just Created Is Missing. What Do You Check?

Which instance served the read. An asynchronous replica may not have applied the change yet, so the row genuinely is not there on that instance. Route read-after-write to the primary or wait for the replica to reach the known position before suspecting data loss.

Scenario: Describe a Zero-Downtime Column Rename.

Add the new column, write to both from the application, backfill existing rows in bounded batches, switch reads to the new column, verify in production, then stop writing the old column and drop it in a later release. Every intermediate state must work with both deployed application versions.

Basic: What Are RPO and RTO?

The recovery point objective is how much data you accept losing, which sets backup frequency and log archiving. The recovery time objective is how long restoration may take, which sets backup format, storage and rehearsal. Both are numbers you commit to, not schedules.

Internals: How Does Point-in-Time Recovery Work?

Restore a base backup, then replay archived write-ahead log up to a chosen moment just before the damaging change. It requires continuous log archiving and enough retention to cover the time between the damage and its detection.

Trace/Debug: You Need to Restore and Discover It Takes Nine Hours Against a Four-Hour Objective. What Went Wrong?

The restore was never rehearsed, so the objective was an assumption. Measure restore duration regularly in a separate environment, then change the backup format, storage location or instance strategy until the measured time meets the stated objective.

Scenario: When Do You Add a Second Kind of Store?

When a workload genuinely cannot be served by the existing one, for instance text relevance or very low-latency counters. Keep the relational database as authority, treat the new store as derived and rebuildable, and write down who rebuilds it and how, because every store adds backups, monitoring, upgrades and on-call surface.

05 / REST, API Design and Real-Time Communication

A response is a promise about state, not a description of code. IntegrationHub exposes an HTTP surface where tenants create sync jobs, poll or stream progress, and receive callbacks when work finishes. The network between those parties is unreliable in both directions, so an API design is mostly a set of decisions about retries, ordering, truncation and unknown outcomes. IntegrationHub is fictional; nothing here describes a real employer's endpoints, limits or SLAs.

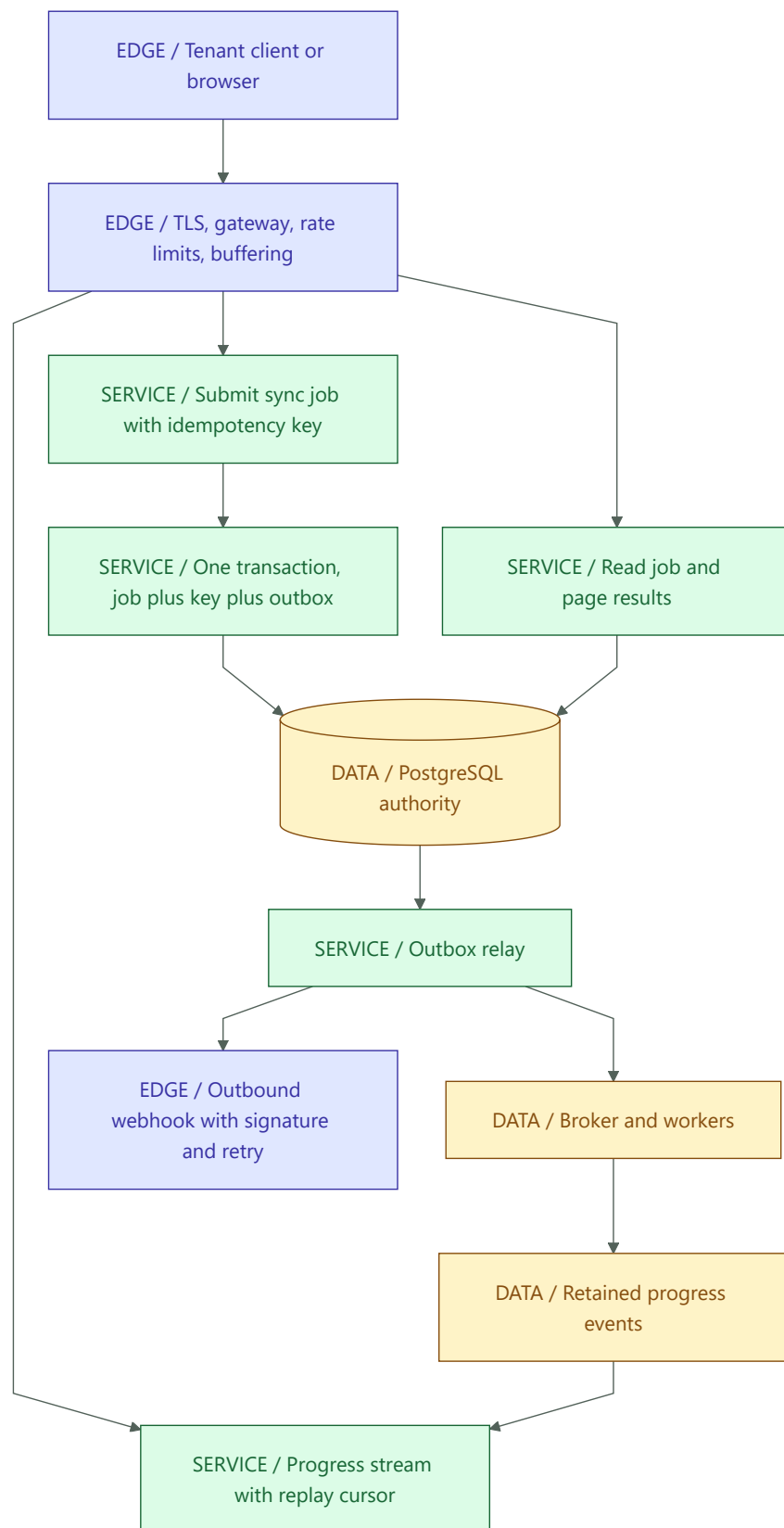
Version assumptions: Java 21 with Spring Boot 4.0.8 and Spring Framework 7.0.x, as established in [03](#); Spring Cloud 2025.1.x remains the compatibility family. HTTP/1.1 and HTTP/2 are the reference transports, with HTTP/3 treated as an infrastructure option rather than an application API change. Reactor 3.8.x travels with that Boot family, gRPC and Protobuf are discussed at specification level, and Kafka 4.x, PostgreSQL 17 and Kubernetes 1.34 remain the surrounding illustrative baselines.

[VERIFY: recheck the Boot-managed Reactor, Jackson, springdoc/OpenAPI and gRPC artifact versions, plus the HTTP semantics RFC numbers cited below, against the actual dependency table and the current IETF documents before publication. No artifact resolution, specification diff or protocol trace was performed in this environment.]

Execution status: NOT EXECUTED. No JDK, Maven or cached dependency set is available, and the earlier download approval covered diagram dependencies only. The two labs under `code/05-apis-realtime/` are complete, self-contained sources with explicit assertions; they have not been compiled or run, and no HTTP trace, latency figure, throughput number or broker output in this chapter is an observation. Diagrams render locally to embedded SVG; syntax highlighting remains unavailable.

Big Picture

Diagram 05.1



Three request shapes dominate: a write that must survive retries, a read that must stay bounded, and a stream that must survive disconnection. Each one fails differently. The write risks duplicate work, the read risks unbounded cost and inconsistent paging, and the stream risks missed events and slow consumers. The remaining sections attach a specific contract to each risk rather than a framework feature.

What You Will Be Able to Explain

- Describe a resource model, HTTP method semantics and status selection without reducing REST to "JSON over HTTP".
- Design a stable error model that is machine-readable, safe to expose, and useful to a client deciding whether to retry.
- Choose a versioning strategy and state precisely which changes are compatible under it.
- Explain offset and keyset pagination, their consistency properties and their cost at depth.
- Implement idempotency as a tenant-scoped, content-bound, retention-limited contract and say what it does not cover.
- Compare polling, long polling, SSE and WebSocket by failure mode, infrastructure requirement and recovery story, not by novelty.
- Explain reconnect, replay cursors, backpressure and slow-client handling, including the point where durable state must take over from a buffer.
- Decide between Reactor/WebFlux and blocking code on virtual threads using the actual constraint, not a benchmark slogan.

[P] PRODUCTION

Where this lives in IntegrationHub. 03 owns the controller, validation and transaction boundary; 04 persists the job and 06 owns the index supporting keyset paging and the uniqueness constraint behind idempotency keys. 07 owns the outbox relay, broker delivery and consumer-side duplicate handling. 08 owns authentication, tenant authorization, webhook signatures and the rule that stream credentials never travel in URLs. 09 owns the browser reconnect behavior; 21 explains the TCP, TLS and proxy behavior these streams depend on.

1. Resources, Contracts and HTTP Semantics

Why this matters: HTTP already defines meanings for methods, status codes, conditional requests and caching. Reusing those meanings lets proxies, clients and operators reason about traffic they did not write. Inventing a private convention inside a POST body discards that shared vocabulary and moves every decision into application code.

A resource is a named thing with an identity and a representation, not a database table and not a service method. IntegrationHub exposes connectors, sync jobs, job results and progress streams. A sync job is created by posting a submission to the collection, read individually, listed with filters, and cancelled through a state-changing operation. The underlying implementation may touch several tables and a broker; that is not the client's contract.

Method Semantics Worth Being Precise About

- **Safe** means the method is not expected to request a state change. GET and HEAD are safe; a GET that mutates data breaks caches, prefetchers and crawlers in ways that are hard to diagnose.
- **Idempotent** means that issuing the request multiple times has the same intended effect on server state as issuing it once. GET, HEAD, PUT and DELETE are defined as idempotent; POST is not. Idempotent is a specification property of the method, not a promise that your handler implemented it correctly.
- **Cacheable** depends on method, status and headers. A response is not cacheable merely because it is a GET, and it must never be cacheable when it is tenant-specific and the cache is shared.
- POST is the general-purpose method for operations that do not fit the others. Using POST is not a design failure; pretending a POST is idempotent without a key is.

PUT replaces a representation at a known identity, which is what makes it idempotent: the same body sent twice describes the same final state. PATCH applies a partial modification and is not idempotent in general, because the modification can be relative. DELETE is idempotent in the sense that the resource ends up absent; the second call may legitimately return a different status than the first.

Status group	Use when	Avoid when
200 with body	A completed read or operation has a representation to return	A long-running job is reported as finished because the request was accepted
201 with Location	A new resource identity was created and can be addressed	Creation is only queued and no durable identity exists yet
202 Accepted	Work was durably accepted for later completion, with a status resource to follow	It is used to hide an unknown outcome or an unvalidated request
400 / 422	The request is malformed or semantically invalid under a documented rule	An authorization failure is disguised as a validation problem
401 / 403	Credentials are missing or invalid, or the authenticated principal is not permitted	Tenant isolation leaks through a 404-versus-403 difference that reveals existence
409	A documented conflict, such as a reused idempotency key with different content	Every concurrency problem is flattened into one opaque conflict

Status group	Use when	Avoid when
429	A rate or concurrency limit was exceeded, with retry guidance	The server is actually failing and should report a server-side status
5xx	The server failed and the client may retry a safe or idempotent request	A client's invalid input is reported as a server fault, hiding real error budgets

Representation discipline. The response body is a published contract. Returning a JPA entity serializes schema decisions, may trigger lazy loading outside the transaction as 04 describes, and leaks fields that were never meant to be public. Build a response record deliberately, and keep secrets, internal identifiers and tenant-foreign data out of it by construction rather than by filter.

Richardson's maturity levels and hypermedia controls are worth knowing by name because interviewers ask. In practice, most production APIs stop at resource-plus-method discipline with documented link fields where navigation genuinely varies. Claiming full hypermedia maturity for an API whose clients hardcode URL templates is not an architecture, it is a slide.

[M] MECHANISM

A status code is an instruction to a machine. Clients, proxies and retry libraries branch on the status before any human reads the body. Choosing 200 for a failure because "the body explains it" disables every automated recovery path between the server and the caller.

[VERIFY: method safety, idempotency and cacheability definitions, the exact semantics of 202 and 409, and conditional-request handling should be rechecked against RFC 9110 and RFC 9111 rather than quoted from memory. No specification text was retrieved in this environment.]

Interview checks: basic: safe, idempotent and cacheable are three different properties. Internals: PUT's idempotency comes from full-state replacement. Debug: a mutating GET usually surfaces first as mysterious duplicate effects behind a cache or prefetcher. Scenario: a queued operation deserves 202 plus a status resource, not an optimistic 200.

2. Error Model: Machine-Readable, Safe and Actionable

Why a model is needed: without one, each endpoint improvises. Clients then parse message strings, retry logic becomes guesswork, and a change in wording breaks an integration. A shared error shape lets a caller decide between fix-the-request, retry-later and escalate without reading prose.

RFC 9457 problem details, the successor to RFC 7807, gives a usable baseline: a stable `type` identifier, a short `title`, the `status`, a human-oriented `detail`, and an `instance` for this occurrence. Extension members carry structured specifics such as field-level violations or a retry hint. Spring's `ProblemDetail` support in the Framework 6 and later line makes this the path of least resistance; the important part is the stability of the `type` values, not the library.

text

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{
  "type": "https://api.integrationhub.example/problems/idempotency-key-reuse",
  "title": "Idempotency key reused with different content",
  "status": 409,
  "detail": "Key k-1 was already used for a different request body in this tenant.",
  "instance": "/v1/sync-jobs",
  "correlationId": "01J9Z2F7Q9T2V8",
  "retryable": false
}
```

This listing is an illustrative contract sketch, not captured output. No server produced it here.

What Belongs and What Must Not

- **Belongs:** a stable type URI, the status, a correlation identifier the client can quote in a support request, structured field violations, and an explicit retryability signal where the client cannot infer it from status alone.
- **Does not belong:** stack traces, SQL, internal hostnames, framework class names, upstream vendor error text, or any value from another tenant. 08 treats these as disclosure, not as debugging convenience.
- **Careful with:** validation messages that echo input. Echoing a submitted value back is usually fine; echoing a resolved internal value is not.

Separate the three failure families explicitly. A request-shape failure is the client's to fix and must not be retried unchanged. A conflict describes a state the client can inspect and possibly resolve. A transient failure invites a retry with backoff, and the server should say so through `Retry-After` or a documented extension member rather than leaving the client to guess.

Error style	Use when	Avoid when
Problem details with stable type URIs	Multiple clients and versions must branch on failures programmatically	The type strings are regenerated from class names and change with refactors
Per-field violation list	A form or batch submission has several correctable problems	Each field message leaks a server-side lookup result

Error style	Use when	Avoid when
Single opaque code plus correlation id	Disclosure risk is high and support has access to correlated logs	Clients legitimately need to self-correct and are given nothing to act on
Mapping every failure to 500	Nothing	Always; it destroys error budgets, alerting and client retry logic

Production failure: a gateway timeout between the edge and the service returns 504 with an HTML body from the proxy. Clients parsing JSON throw a parse error and report "invalid response" rather than "upstream timeout", and the incident is diagnosed an hour late. Normalize error representations at the edge so that every failure a client can see has the documented content type. 18 covers correlation and alerting around this.

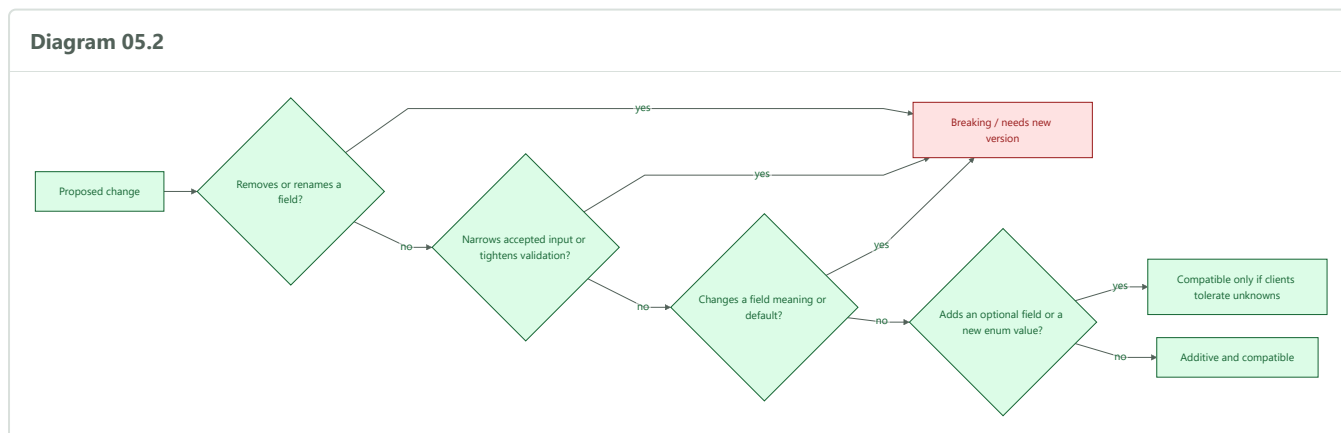
[VERIFY: confirm RFC 9457's obsolescence of RFC 7807, the registered application/problem+json media type behavior and Spring's current ProblemDetail defaults, including whether a given Boot version emits problem details for framework-level errors without explicit configuration.]

Interview checks: basic: errors are a contract, not log output. Internals: type URIs provide stability that message text cannot. Debug: a non-JSON error body usually means a layer in front of the application produced it. Scenario: retryability must be explicit when the status alone is ambiguous.

3. Versioning and Compatibility

Why it exists: once an external client depends on a representation, you no longer control deployment timing. Versioning is the mechanism that lets the server change while existing callers keep working, and its real content is a definition of which changes are safe.

Diagram 05.2



The tolerant-reader branch matters. Adding a field is compatible only for clients that ignore unknown fields; a client with strict deserialization will fail. Adding an enum value is compatible only for clients that have a defined fallback. You cannot assert compatibility for clients you have not specified, so document the reader requirements as part of the contract and treat a strict-schema client as a known constraint.

Strategy	Use when	Avoid when
URI path version, <code>/v1/sync-jobs</code>	External clients, cacheability and routability matter; human-visible version is an advantage	Dozens of resources force a lockstep version bump for one unrelated change
Media-type or header negotiation	The team genuinely operates content negotiation end to end, including caches and gateways	Intermediaries strip or ignore the header and silently serve the default
Query-parameter version	A pragmatic stopgap exists and routing cannot change quickly	It becomes the permanent design and collides with cache keys and logs
Additive-only, no version	A single internal consumer evolves with the producer in the same pipeline	Third parties or mobile clients are in play and cannot be upgraded on demand

Deprecation needs a process, not a note. Announce the replacement, instrument usage per client so you know who still calls the old shape, set a sunset date you actually hold, and return advisory headers during the window. Removing an endpoint whose remaining callers you never measured is how a mobile application fleet breaks for months.

Mobile and webhook consumers deserve special treatment. You cannot force an upgrade on an installed application, and a receiving endpoint you do not own may be maintained by nobody. These consumers set the real minimum support window, which is a product decision with an engineering cost, not an API style preference. 13 covers partner-facing contract management in more depth.

[D] **DECISION**

Pick the version axis from the consumer, not the aesthetic. Public and partner-facing surfaces favor explicit, visible path versions with long support windows. Internal service-to-service calls in one deployment pipeline can often rely on strict additive evolution plus contract tests, because the producer and consumer ship together.

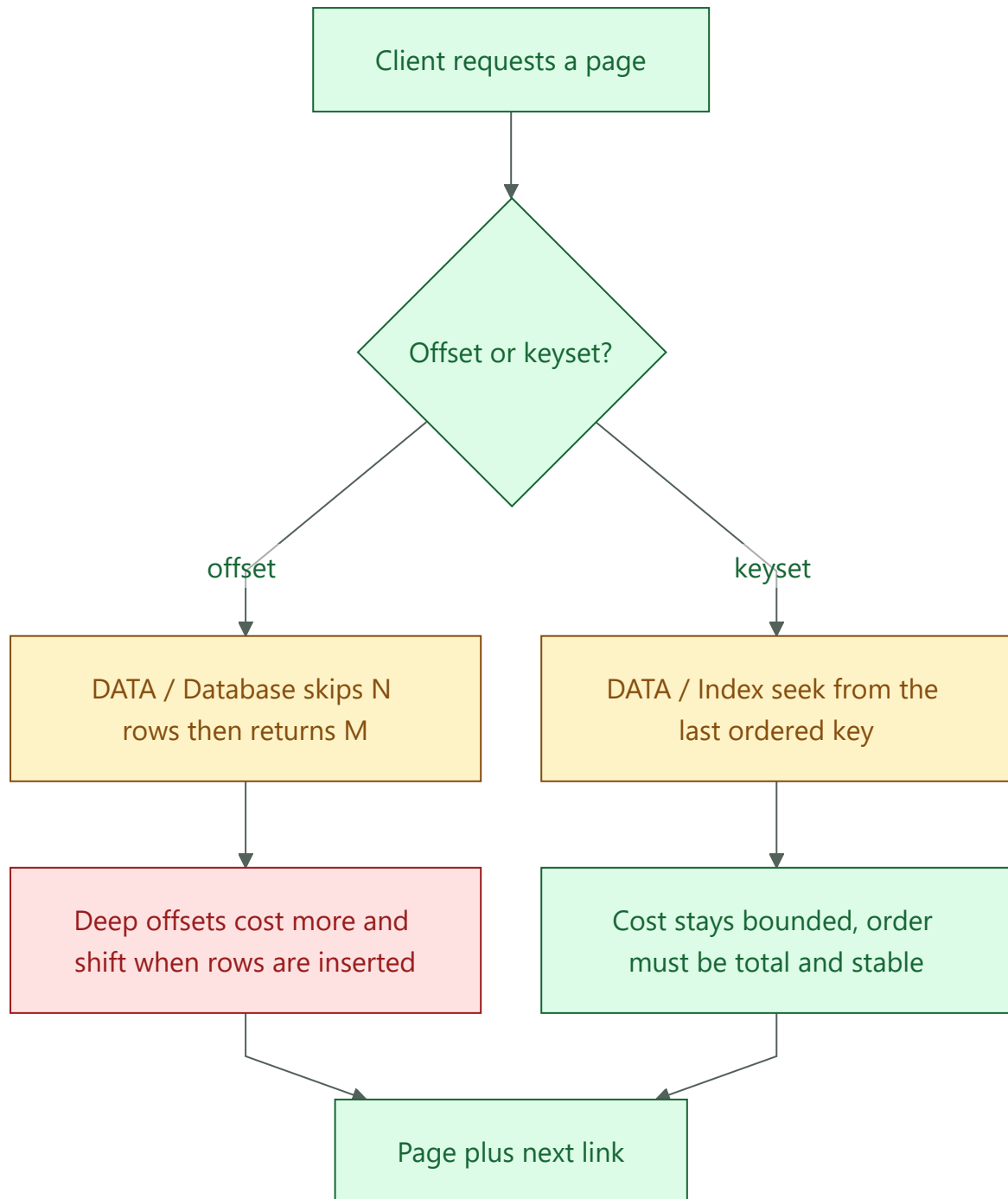
[VERIFY: check the current Sunset and Deprecation header specifications and their adoption status before presenting them as standard practice, and confirm the chosen JSON binding's unknown-field defaults in the selected Boot version.]

Interview checks: basic: additive change is not automatically compatible. Internals: tolerant readers are a precondition, not a given. Debug: per-client usage telemetry is what makes deprecation safe. Scenario: mobile and webhook receivers determine the real support window.

4. Pagination: Bound Cost and State Consistency

Why it exists: a collection endpoint without a bound is an availability risk. One tenant with a large job history can produce a response that exhausts memory, saturates a connection and times out behind the gateway. Pagination makes the cost of a single request predictable; it does not make the total cost of walking the collection small.

Diagram 05.3



Offset pagination is simple and supports jumping to an arbitrary page. Its two costs are real. Deep offsets usually require the database to produce and discard the skipped rows, so page 5,000 is far more expensive than page 1. And because each page is a separate query, concurrent inserts and deletes shift the window: a row can be seen twice or skipped entirely while a client walks the collection.

Keyset pagination orders by a total, stable key and asks for rows after the last key seen. With a matching index, the cost of each page is similar regardless of depth, and the window does not shift under inserts that sort after the cursor. The price is that arbitrary page jumps are gone and the sort key must be genuinely total. Ordering by `created_at` alone is not total when timestamps collide; pair it with the primary key.

Approach	Use when	Avoid when
Offset and limit	Small bounded collections, admin screens, arbitrary page access is required	Deep paging over large tenant histories, or exports that must not skip rows
Keyset cursor	Large collections, continuous scrolling, export or sync walks	A UI genuinely requires page numbers and total counts on huge data sets
Opaque cursor token	You want to change the internal key or add filters without breaking clients	The token is a signed-free plain encoding of internal state a client can forge
Unbounded list	Never for tenant data	Always; add a default and a maximum page size instead

Total counts are a separate decision. An exact count over a large filtered set can cost as much as the page itself, and it is stale the moment it is computed. Offer it as an opt-in, an estimate, or a boolean has-more flag derived from fetching one extra row. 06 covers the index and plan behavior that determines which of these is affordable.

Make the cursor opaque to the client and validate it on arrival. A cursor that is a plain, readable encoding of a primary key invites a client to construct one, which becomes a tenant-isolation question rather than a paging question: the cursor must be interpreted only within the authorized tenant's query scope. 08 owns that boundary.

[VERIFY: deep-offset cost, index-supported keyset behavior and count-query cost are engine- and plan-dependent. Measure on PostgreSQL 17 with representative data volume; no query, plan or timing was executed here.]

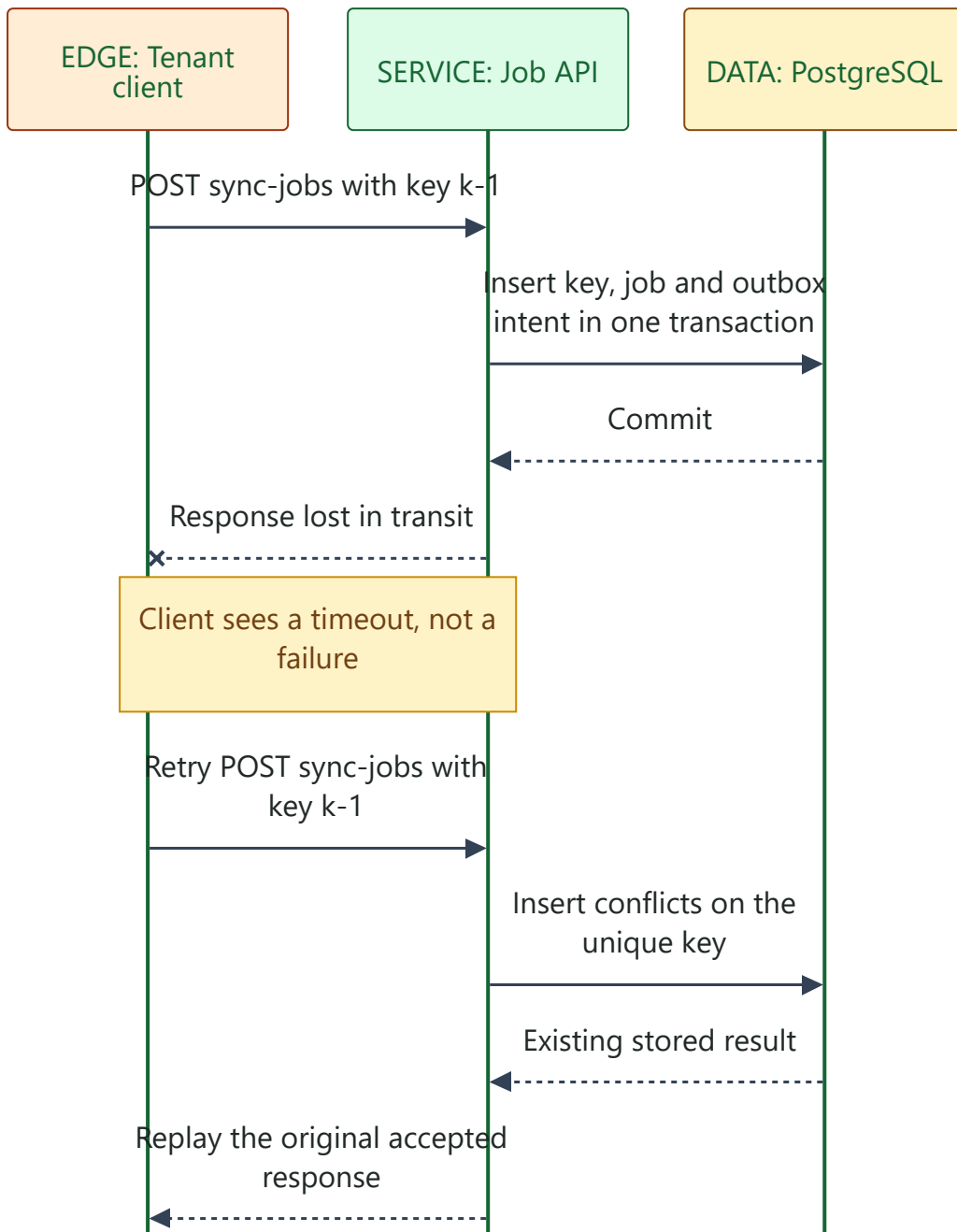
Interview checks: basic: pagination bounds per-request cost. Internals: offset paging shifts under concurrent writes. Debug: duplicated or missing rows during an export usually indicate offset paging plus insert traffic. Scenario: keyset with a total ordering is the default for a sync walk.

5. Idempotency and the Unknown Outcome

Why it exists: when a client's request times out, the client learns nothing about the server. The request may never have arrived, may have been fully processed, or may have been processed with the response lost on the way back. This is the **unknown outcome** defined in 00, and it is

the single most important asymmetry in API design. A client that cannot retry will drop work; a client that retries blindly will duplicate it.

Diagram 05.4



The retry arrives at a server that already committed the work. The unique constraint, not the application's read-then-write check, is what makes the second attempt safe. A handler that first selects by key and then inserts has a race window in which two concurrent retries both find nothing and both create a job.

The Four Parts of the Contract

- **Scope.** The key is meaningful only within a tenant and an operation. A bare client string shared across tenants is a collision and a cross-tenant disclosure risk. The lab encodes this as a tuple of tenant, operation and client key.
- **Content binding.** Store a digest of the request content with the result. If the same key arrives with different content, that is a client bug and deserves 409, not a silent replay of an unrelated job.
- **Stored result.** What is replayed is the recorded outcome of the first accepted request, including its status and body. Recomputing a "similar" response is not a replay and can diverge.
- **Retention.** The guarantee has a lifetime. After the window, the key is unknown and the same request creates new work. Retention must be longer than the client's maximum retry horizon, and it must be stated in the documentation rather than implied.

Accepted is not completed. Committing the job row, the idempotency record and the publication intent in one local transaction establishes that the request was accepted. The connector sync has not run, the broker has not received anything, and the outbox relay in 07 still has to publish. The response should say 202 with a job resource, and the client's success handler must not treat it as finished work.

Mechanism	Use when	Avoid when
Unique constraint on tenant plus operation plus key	The write is a single local transaction against the authority database	The check is implemented as a select followed by an insert in application code
Natural idempotency through PUT at a client-chosen identity	The client can safely choose the identity and the operation is a replacement	Server-side identity generation or side effects make replacement meaningless
Consumer-side deduplication in the worker	Broker delivery is at-least-once and the effect must happen once	It is used as a substitute for an API-level key, leaving the HTTP layer duplicating work
Client-side request hashing only	A best-effort local guard against double submission in a UI	It is presented as a correctness guarantee across devices, tabs or processes

[!] TRAP

Idempotency does not make the operation exactly once. It bounds duplicate acceptance at one boundary for one retention window. Downstream publication, consumer processing and any external connector call each need their own duplicate-handling story, and a network timeout against a third-party system remains an unknown outcome for that call.

The `IdempotencyLab` source under `code/05-apis-realtime/` asserts exactly these boundaries: an identical retry replays one job identity, a reused key with different content is a conflict, two tenants using the same client string are two independent requests, expiry ends the guarantee, and eight concurrent duplicates collapse to one acceptance because the reservation is atomic rather than read-then-write. Those are source-level assertions. The lab has not been compiled or executed.

[VERIFY: idempotency-key header conventions are still being standardized in the IETF; check the current draft status before describing a header name as a standard. Also confirm the chosen store's constraint behavior and expiry semantics for the production deployment.]

Interview checks: basic: a timeout is not evidence of no effect. Internals: the unique constraint closes the concurrent-retry race. Debug: duplicate jobs with distinct ids usually mean the key was unscoped, expired or checked before insertion. Scenario: 202 with a status resource is the honest answer for queued work.

6. Rate Limiting and Outbound Webhooks

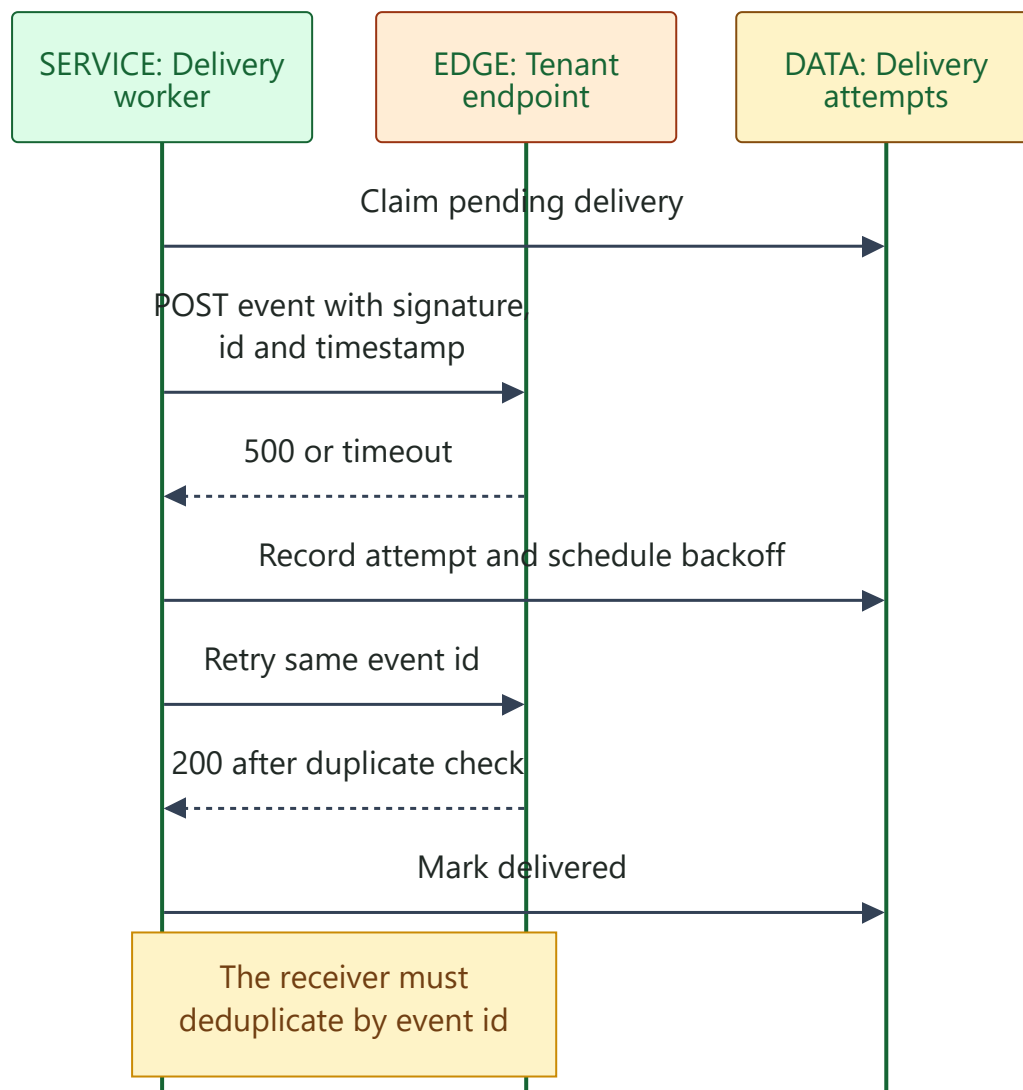
Why limits exist: capacity is shared. Without a limit, one tenant's retry storm or export loop consumes the pool that every other tenant depends on. Rate limiting is an availability control first and a commercial control second, and it belongs where it can protect the resource, which is usually the gateway rather than the application.

Distinguish the counters. A **rate limit** bounds requests per unit of time; a **concurrency limit** bounds simultaneous in-flight work; a **quota** bounds consumption over a billing period. They fail differently: a rate limit smooths bursts, a concurrency limit protects a connection pool or thread budget, and a quota is an accounting decision. 02 distinguishes admission from concurrency control in the same way for in-process work.

- Return 429 with retry guidance, and make the guidance specific enough that a well-behaved client stops guessing.
- Communicate the limit through documented response headers so clients can self-pace before they are rejected.
- Expect retries to synchronize if every client backs off by the same fixed interval. Jitter is what prevents a coordinated second wave.
- Enforce per tenant and per credential, not only per source address. Shared network egress makes address-based limits both leaky and unfair.

Webhooks Are Someone Else's API, Called by You

Diagram 05.5



An outbound webhook inverts the trust relationship: your service becomes a client of an endpoint you do not operate, which may be slow, unavailable, or in an untrusted network location. The delivery contract therefore needs an event identifier the receiver can deduplicate on, a signature with a timestamp so the receiver can verify origin and reject replays, a bounded timeout, a backoff schedule with a terminal state, and a way for the tenant to replay missed events from retained history.

Delivery concern	Correct handling	Common mistake
Duplicate deliveries	Stable event id plus receiver-side deduplication	Assuming one successful POST per event

Delivery concern	Correct handling	Common mistake
Ordering	Per-subject sequence numbers; receivers tolerate reordering	Assuming the receiver observes global event order
Authentication	Signature over the raw body with a timestamp and rotation support	A shared secret in a query string, or no verification at all
Slow receiver	Bounded timeout, backoff, circuit breaking, dead-letter state	Unbounded retries that saturate the delivery pool
Destination safety	Validated and allow-listed destinations	Posting to any tenant-supplied URL, enabling server-side request forgery

The last row is a security control, not a convenience check. A tenant-supplied callback URL pointing at an internal address or a cloud metadata endpoint turns your delivery worker into a proxy for internal network access. 08 and 17 own that validation; it must happen at delivery time rather than only at registration, because DNS can change between the two.

Production failure: a tenant endpoint starts responding in thirty seconds instead of two hundred milliseconds. Delivery workers block, the pool fills, and deliveries to every other tenant stall behind one slow receiver. The fix is isolation and bounded timeouts per destination, not more workers. 18 covers the bulkhead and circuit-breaker patterns that contain it.

[VERIFY: webhook signature schemes, timestamp tolerance windows and standard rate-limit header names vary by vendor and draft specification. Confirm the chosen scheme and header set against current references before publishing them as the IntegrationHub contract.]

Interview checks: basic: 429 is a client-visible limit signal, not a server failure. Internals: synchronized backoff without jitter creates retry waves. Debug: a stalled delivery queue often traces to one slow destination. Scenario: tenant-supplied URLs require destination validation at send time.

7. OpenAPI, Swagger and Contract Discipline

Why a specification artifact matters: a written contract that is generated from, or validated against, the running service is the only documentation that stays true. Prose descriptions drift within weeks. The specification is also machine input: client generation, mock servers, contract tests and gateway validation all consume it.

Be precise with the names, because interviewers use them as a quick filter. **OpenAPI** is the specification for describing HTTP APIs; 3.1 aligns with JSON Schema. **Swagger** is the older name of the specification plus the current tool family, most visibly Swagger UI. Saying "we use

Swagger" usually means "we render an OpenAPI document with Swagger UI", and knowing the difference signals that you have read the specification rather than only the dependency name.

Approach	Use when	Avoid when
Design-first, specification is the source	Multiple teams or external partners build against the contract before implementation	Nobody validates that the implementation still matches the committed document
Code-first with generated specification	A single team owns both sides and annotations stay close to handlers	The generated document is published without review and leaks internal shapes
Generated clients from the specification	Many consumers and frequent additive changes	Regeneration churn becomes the only reason a consumer redeploys
Gateway request validation from the specification	Rejecting malformed input early protects downstream services	It is treated as a replacement for server-side validation and authorization

Documentation exposure is an access decision. An interactive explorer that reaches production data, pre-filled with a working token, is a security surface. Expose the specification where it belongs, and keep examples synthetic. 08 covers the authentication boundary; 03 covers the Actuator and management-endpoint equivalent of the same mistake.

Contract tests close the loop that a document alone cannot. A consumer-driven contract test asserts that the producer still satisfies the expectations a specific consumer relies on, which is narrower and more useful than asserting that a schema file parses. 19 covers how those tests run in the pipeline.

Interview checks: basic: OpenAPI is the specification, Swagger is the tool family and former name. Internals: 3.1 aligns with JSON Schema. Debug: drift appears first in examples and error shapes. Scenario: publish the document, restrict the interactive explorer, and back both with contract tests.

8. gRPC and GraphQL: Different Problems, Not Upgrades

Why alternatives exist: REST over JSON optimizes for ubiquity and cacheability. Some problems care more about payload efficiency and strict schemas, and others care about client-controlled response shapes. Choosing a protocol is choosing which constraint you accept, not climbing a ladder of sophistication.

gRPC uses HTTP/2 transport with Protobuf messages and a generated service contract in each language. It offers compact binary payloads, bidirectional streaming, deadlines that propagate, and schema evolution rules enforced by field numbers. The costs are real: browsers cannot speak native gRPC without a proxy layer, payloads are not human-readable in a log, and HTTP caches and generic gateways cannot reason about the messages.

GraphQL gives the client control over which fields a response contains, resolving over-fetching and under-fetching for UIs with varied needs. The cost moves to the server: query cost analysis, depth and complexity limits, the N+1 problem between resolvers, caching that no longer follows URL semantics, and an authorization check that must apply at the field level rather than the endpoint level.

Protocol	Use when	Avoid when
REST over JSON	Public or partner-facing surfaces, cacheable reads, broad client diversity	Extremely chatty internal calls where payload size dominates cost
gRPC	Internal service-to-service calls where schema, deadlines and efficiency matter	Browsers are direct consumers, or no team owns the schema registry and proxy layer
GraphQL	A UI aggregates many sources and field needs vary strongly per screen	Field-level authorization, cost limits and resolver batching are not budgeted work
Messaging instead of a synchronous call	The work is asynchronous and the caller does not need the result inline	A request genuinely needs a bounded synchronous answer; see 07

For IntegrationHub, the tenant-facing API stays REST because it is public, cacheable and consumed by diverse clients including partner integrations. An internal connector-execution call between the job service and the worker fleet is a reasonable gRPC candidate because both ends are owned, the schema is stable, and deadline propagation helps. GraphQL would only be justified if the console in [09](#) needed highly varied field selections across many sources, which it does not.

[VERIFY: gRPC-Web and browser support details, GraphQL cost-analysis tooling and the current Spring support for both should be rechecked against their project documentation. No gRPC or GraphQL runtime was exercised in this environment.]

Interview checks: basic: gRPC is schema-first binary over HTTP/2, GraphQL is a client-shaped query layer. Internals: GraphQL moves cost control and authorization to the field level. Debug: resolver N+1 mirrors the ORM N+1 in [04](#). Scenario: protocol choice follows ownership and client diversity.

9. Polling and Long Polling

Why polling persists: it is the only progress mechanism that requires nothing beyond a plain GET. It works through every proxy, survives every disconnection trivially, needs no server-side connection state, and scales horizontally without sticky routing. Dismissing it is a common and expensive mistake.

Simple polling asks for the status resource on an interval. Its two inefficiencies are the latency floor equal to the interval and the request volume when many clients poll frequently. Both are controllable: use conditional requests with `ETag` or `If-Modified-Since` so unchanged state returns 304 with no body, and let the server suggest an interval that widens as a job sits idle.

Long polling holds the request open until state changes or a timeout expires, then returns. It reduces latency without a new protocol, but it converts a stateless request into a held connection, which means timeouts at every intermediary, a connection consumed per waiting client, and a careful contract about what happens to events that occur between the response and the next request. That gap is why long polling still needs a cursor.

Mechanism	Use when	Avoid when
Interval polling with conditional GET	Progress granularity of seconds is acceptable, infrastructure is unknown	Sub-second updates are a product requirement for many concurrent viewers
Adaptive interval	Jobs vary from seconds to hours and clients should not poll a stalled job hard	The interval is client-chosen only and ignored by misbehaving clients
Long polling	Lower latency is needed and streaming is blocked by intermediaries	Held connections exceed the server or proxy connection budget
Streaming push	Many updates per job and the latency requirement is genuinely sub-second	The gain is assumed rather than measured against a polling baseline

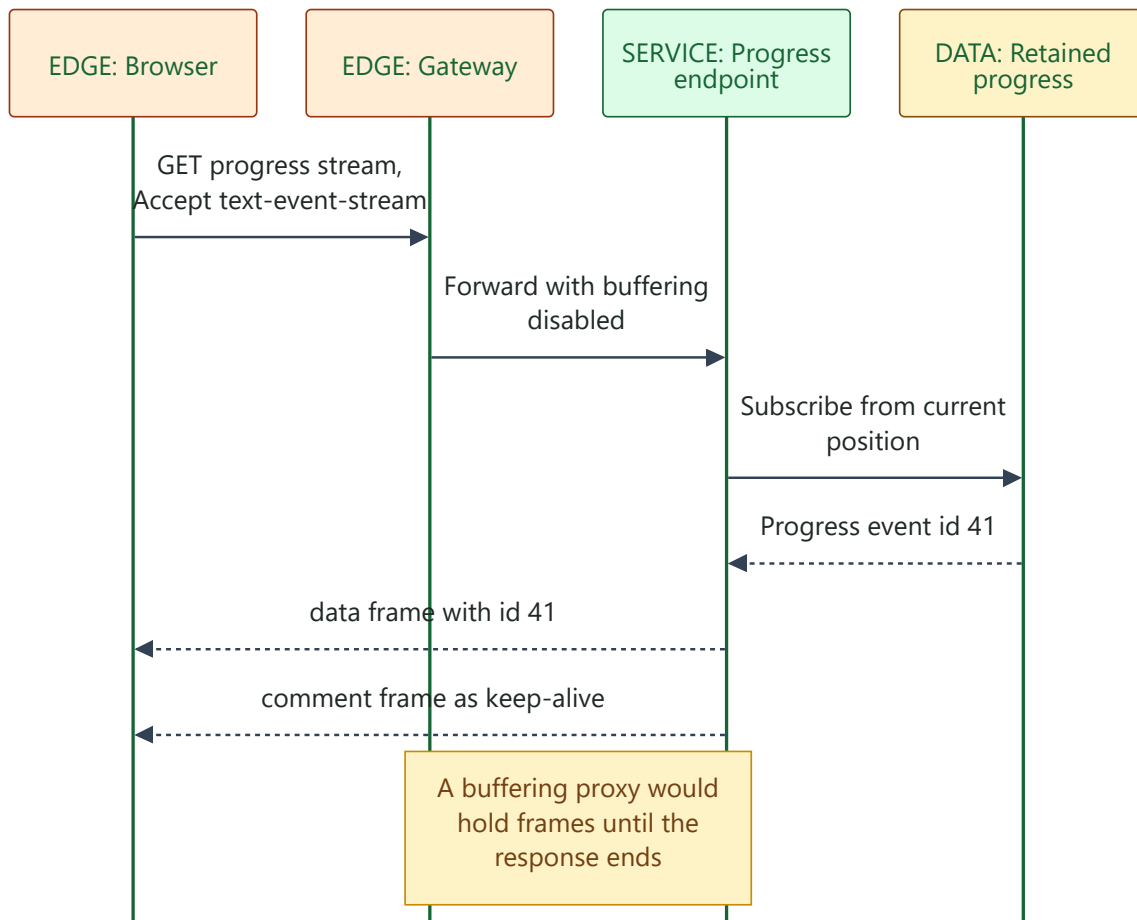
For IntegrationHub, polling the job resource is the documented baseline that every API client can rely on, and the progress stream is the optimization. Keeping the polling path authoritative means a client behind a hostile proxy degrades to a slower experience rather than a broken one, and the status endpoint remains the recovery authority for any stream that falls behind retention.

Interview checks: basic: polling trades latency for simplicity and resilience. Internals: conditional requests remove the body cost, not the request cost. Debug: a polling storm usually traces to a fixed short interval with no backoff on idle jobs. Scenario: keep a polling fallback whenever intermediaries are outside your control.

10. Server-Sent Events

Why SSE fits progress: job progress is one-directional, text-shaped and ordered. SSE is a long-lived HTTP response in which the server writes `text/event-stream` frames. It keeps HTTP semantics, works with ordinary authentication and gateways, defines automatic reconnection in the browser, and carries an event identifier that the client sends back on reconnect. For a server-to-client progress feed, it is usually the smallest adequate mechanism.

Diagram 05.6



The Infrastructure Requirements Are the Hard Part

- **Response buffering must be disabled** on every intermediary in the path. A proxy that buffers for efficiency will accumulate frames and deliver them when the response completes, which for a long-lived stream means never. The symptom is a stream that works locally and delivers nothing through the gateway.
- **Idle timeouts must be longer than the keep-alive interval**, and the server must emit a periodic comment frame so that idle connections are not reaped. Load balancers, service meshes and browsers each have their own timeout.
- **Compression and chunking behavior** must not defeat incremental flushing. A content encoder that waits for more input before emitting a block delays every frame.
- **Connection budget is per client and per tier.** Under HTTP/1.1 a browser's per-origin connection limit is small, and a long-lived stream consumes one of them; HTTP/2 multiplexing changes the arithmetic but not the server-side cost of holding state.

Authentication must not move into the URL. The browser's native `EventSource` cannot set arbitrary request headers, which tempts teams to put a token in the query string where it lands in proxy logs, browser history and referrer data. The guide's documented position, set in 00, is

a same-origin secure `HttpOnly` session cookie through the gateway or BFF, with state-changing requests separately protected against cross-site request forgery. A fetch-based streaming client can send an `Authorization` header instead, at the cost of implementing reconnection yourself. 09 covers that client choice.

SSE aspect	Reality	Frequent misconception
Direction	Server to client only	That it is a lightweight bidirectional channel
Transport	Ordinary HTTP response, works with HTTP authentication and gateways	That it needs a protocol upgrade like WebSocket
Reconnection	Built into <code>EventSource</code> , with <code>Last-Event-ID</code> resent automatically	That reconnection also restores events the server no longer retains
Payload	UTF-8 text frames; binary needs encoding	That arbitrary binary can be streamed directly
Scaling	Each stream holds server-side state and a connection	That it is free because it is "just HTTP"

[VERIFY: EventSource header limitations, default reconnection timing, per-origin connection limits under HTTP/1.1 versus HTTP/2, and proxy buffering defaults vary by browser, server and gateway. Confirm against current browser documentation and your gateway configuration; no browser or proxy behavior was measured here.]

Interview checks: basic: SSE is one-directional text over a long-lived HTTP response. Internals: proxy buffering and idle timeouts decide whether it works in production. Debug: frames arriving only at stream end means something in the path buffered. Scenario: never carry a bearer token in a stream URL.

11. WebSocket and STOMP

Why WebSocket exists: some interactions are genuinely bidirectional and low-latency, such as collaborative editing, interactive terminals or trading interfaces. WebSocket starts as an HTTP request with an upgrade, then becomes a full-duplex frame-oriented connection that is no longer shaped like request and response.

That upgrade is the source of most of its operational cost. Intermediaries must support the upgrade and forward it, the connection is long-lived and often sticky, and the usual HTTP tooling stops helping: status codes, caching, conditional requests and ordinary request logging no longer describe what is happening on the socket. Authorization also changes shape, because it must apply per message or per subscription rather than once per request, and the connection may outlive the credential that established it.

STOMP is a simple text-based messaging protocol frequently layered over WebSocket, and Spring provides a messaging abstraction for it with destinations, subscriptions and a broker relay. It adds useful structure, but it also adds a second routing model inside your application, and a simple in-memory broker does not survive multiple instances without an external relay. Choosing STOMP means choosing to operate that topology.

Choice	Use when	Avoid when
SSE	Server-to-client updates, HTTP semantics, straightforward reconnection	Clients must send frequent, low-latency messages upstream
WebSocket	Genuine bidirectional interaction with sub-second expectations	Only progress updates are needed and the infrastructure cost is not justified
WebSocket plus STOMP	Destination-based pub/sub with multiple topics per connection	A single stream per job would do, or no one owns the broker relay
Plain HTTP request for upstream messages	Client-to-server actions are occasional commands	Upstream volume is high and per-request overhead dominates

For IntegrationHub, progress is server-to-client and the console's upstream actions are occasional commands such as cancel or retry. Those are ordinary authorized POST requests. Adding WebSocket would buy a bidirectional channel the product does not need while adding sticky routing, per-message authorization and a broker relay to operate. That is the decision the chapter recommends, and it would change if a genuinely interactive feature appeared.

[D] **DECISION**

Choose the weakest mechanism that meets the requirement. Polling, then SSE, then WebSocket is an increasing order of capability and of operational obligation. Moving up the list should be justified by a specific latency or directionality requirement, not by the expectation that a richer transport is more modern.

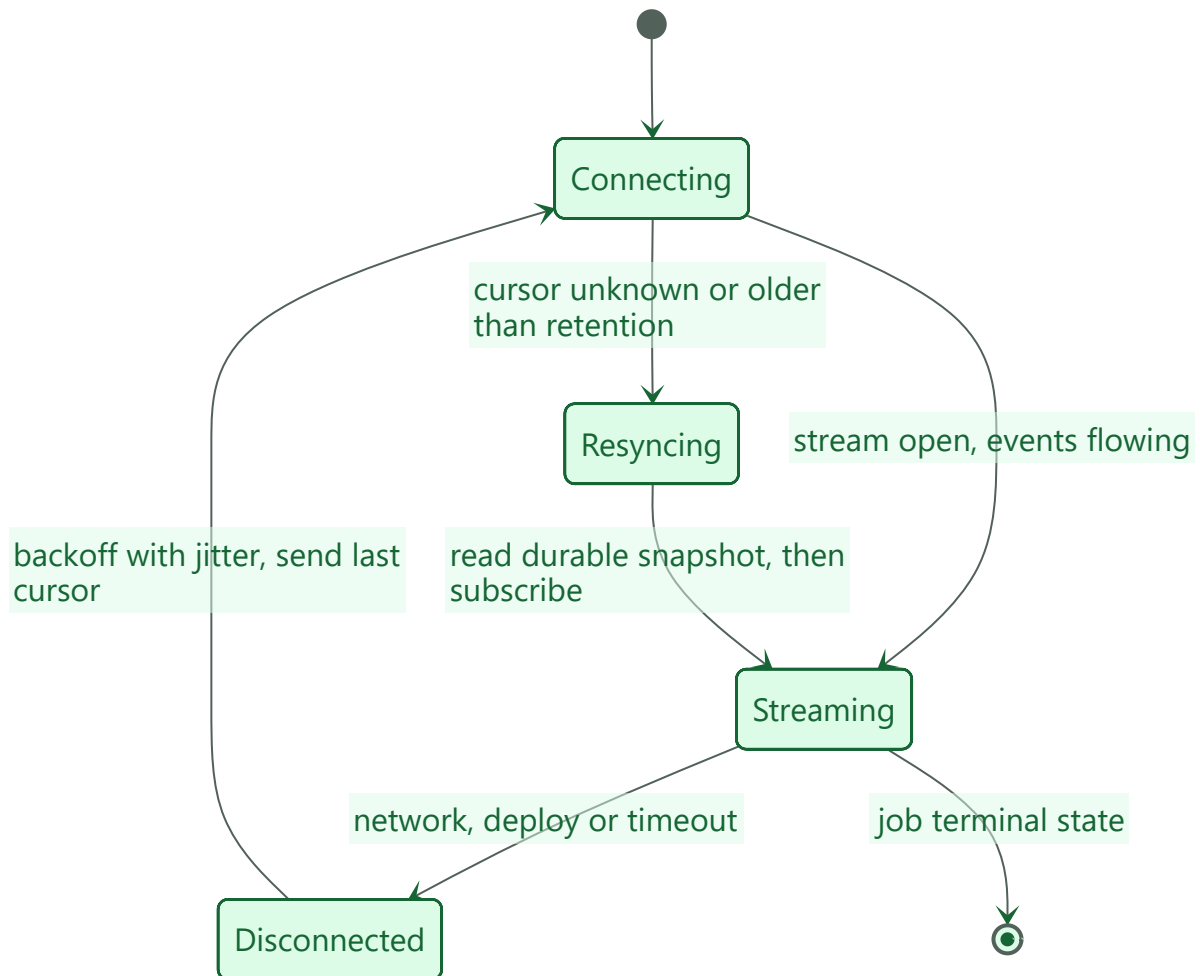
[VERIFY: confirm the current Spring WebSocket and STOMP support, broker relay requirements and the status of the Framework's messaging abstractions in the selected Boot version before recommending a topology.]

Interview checks: basic: WebSocket is bidirectional after an HTTP upgrade. Internals: per-message authorization and sticky routing are the hidden costs. Debug: upgrade failures usually sit in an intermediary that does not forward the upgrade. Scenario: one-directional progress rarely justifies a socket.

12. Reconnect, Cursors and Replay

Why this is the real work: every long-lived connection ends. Deploys, idle timeouts, mobile network changes, laptop sleep and proxy restarts all drop streams routinely. A streaming feature is therefore mostly a resumption design, and the question that matters is what the client is guaranteed when it comes back.

Diagram 05.7



SSE gives the client half of this for free. The server emits an `id` on each event, the browser remembers the last one, and on reconnect it sends `Last-Event-ID`. What the specification does not give you is the server side: the decision about how much history to retain, what to do when the cursor is older than retention, and how to reject a cursor that cannot be valid. Those are application contracts, and they are what the `ReplayCursorLab` makes explicit.

- **Cursor inside retention:** send exactly the events after the cursor, in order, with no gaps. This is the only case that is a pure replay.

- **Cursor equal to the current position:** a valid resume that replays nothing. Treating "no missed events" as an error is a common bug.
- **Cursor older than retention:** the buffer cannot satisfy it. The client must re-read the durable job state and resume from the position that snapshot reports. Silently starting from the oldest retained event would present a gap as continuity.
- **No cursor:** a first connection, which is a snapshot-then-subscribe, not a replay.
- **Cursor ahead of the stream:** impossible for this job. It indicates a different job, a restarted sequence or a forged value, and it must be rejected rather than treated as caught up.

java

```
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Deque;
import java.util.List;
import java.util.OptionalLong;

/** Chapter 05 lab: what a Last-Event-ID cursor can and cannot recover after a dropped stream. */
public final class ReplayCursorLab {

    record ProgressEvent(long id, String status, int processed) { }

    sealed interface Resume permits Replay, SnapshotRequired { }

    /** The server could satisfy the cursor entirely from retained history. */
    record Replay(List<ProgressEvent> missed) implements Resume { }

    /** The cursor is unknown or older than retention, so the client must re-read durable state first. */
    record SnapshotRequired(String reason) implements Resume { }

    /** A bounded per-job retention buffer. Retention is a capacity decision, not an unlimited log. */
    static final class ProgressStream {
        private final Deque<ProgressEvent> retained = new ArrayDeque<>();
        private final int retentionSize;
        private long nextId;
        private int processedTotal;

        ProgressStream(int retentionSize) {
            this.retentionSize = retentionSize;
        }

        ProgressEvent publish(String status, int batchSize) {
            processedTotal += batchSize;
            ProgressEvent event = new ProgressEvent(++nextId, status, processedTotal);
            retained.addLast(event);
            while (retained.size() > retentionSize) {
                retained.removeFirst();
            }
            return event;
        }
    }

    /** The durable projection the snapshot endpoint would read, independent of the event buffer. */
    ProgressEvent snapshot() {
        return new ProgressEvent(nextId, nextId == 0 ? "PENDING" : "RUNNING", processedTotal);
    }

    Resume resume(OptionalLong lastEventId) {
        if (lastEventId.isEmpty()) {
            return new SnapshotRequired("no cursor supplied");
        }
        long cursor = lastEventId.getAsLong();
        if (cursor > nextId) {
            return new SnapshotRequired("cursor ahead of the stream, likely a different job or restarted sequence");
        }
    }
}
```

```

        long oldestRetained = retained.isEmpty() ? nextId + 1 : retained.getFirst().id();
        if (cursor + 1 < oldestRetained) {
            return new SnapshotRequired("cursor older than retention");
        }
        List<ProgressEvent> missed = new ArrayList<>();
        for (ProgressEvent event : retained) {
            if (event.id() > cursor) {
                missed.add(event);
            }
        }
        return new Replay(missed);
    }
}

static void check(boolean condition, String description) {
    if (!condition) {
        throw new AssertionError(description);
    }
}

static ProgressStream streamWith(int retention, int events) {
    ProgressStream stream = new ProgressStream(retention);
    for (int index = 0; index < events; index++) {
        stream.publish("RUNNING", 10);
    }
    return stream;
}

static void cursorInsideRetentionReplaysOnlyMissedEvents() {
    ProgressStream stream = streamWith(5, 5);
    Resume resume = stream.resume(OptionalLong.of(3));
    check(resume instanceof Replay, "a cursor inside retention is replayable");
    List<ProgressEvent> missed = ((Replay) resume).missed();
    check(missed.size() == 2, "only events after the cursor are resent");
    check(missed.get(0).id() == 4 && missed.get(1).id() == 5, "replay is ordered and gapless");
    check(missed.get(1).processed() == 50, "the last replayed event carries the cumulative figure");
}

static void currentCursorReplaysNothing() {
    ProgressStream stream = streamWith(5, 5);
    Resume resume = stream.resume(OptionalLong.of(5));
    check(resume instanceof Replay, "an up-to-date cursor is still a valid resume");
    check(((Replay) resume).missed().isEmpty(), "nothing is missed, so nothing is resent");
}

static void cursorOlderThanRetentionFallsBackToSnapshot() {
    ProgressStream stream = streamWith(3, 10);
    Resume resume = stream.resume(OptionalLong.of(2));
    check(resume instanceof SnapshotRequired, "retention is bounded, so old cursors cannot be honoured");
    ProgressEvent snapshot = stream.snapshot();
    check(snapshot.id() == 10, "the snapshot carries the position the client should resume from");
    check(snapshot.processed() == 100, "durable state, not the event buffer, is the recovery authority");
}

static void missingCursorRequiresSnapshotFirst() {
    ProgressStream stream = streamWith(5, 4);
    check(stream.resume(OptionalLong.empty()) instanceof SnapshotRequired, "a first connection is not a replay");
}

static void cursorAheadOfStreamIsRejected() {
    ProgressStream stream = streamWith(5, 4);
    Resume resume = stream.resume(OptionalLong.of(99));
    check(resume instanceof SnapshotRequired, "an impossible cursor must not be treated as caught up");
}

public static void main(String[] args) {
    cursorInsideRetentionReplaysOnlyMissedEvents();
    currentCursorReplaysNothing();
    cursorOlderThanRetentionFallsBackToSnapshot();
    missingCursorRequiresSnapshotFirst();
}

```

```
        cursorAheadOfStreamIsRejected();
        System.out.println("ReplayCursorLab checks completed");
    }
}
```

This listing is the saved lab source, checked for exact synchronization by the publication test. That check is not compilation, and the file has not been executed. The in-memory deque stands in for whatever retains progress in production, which might be a broker topic, a database table or a cache with an eviction policy. The important property is that retention is bounded and that the durable job state, not the buffer, is the recovery authority.

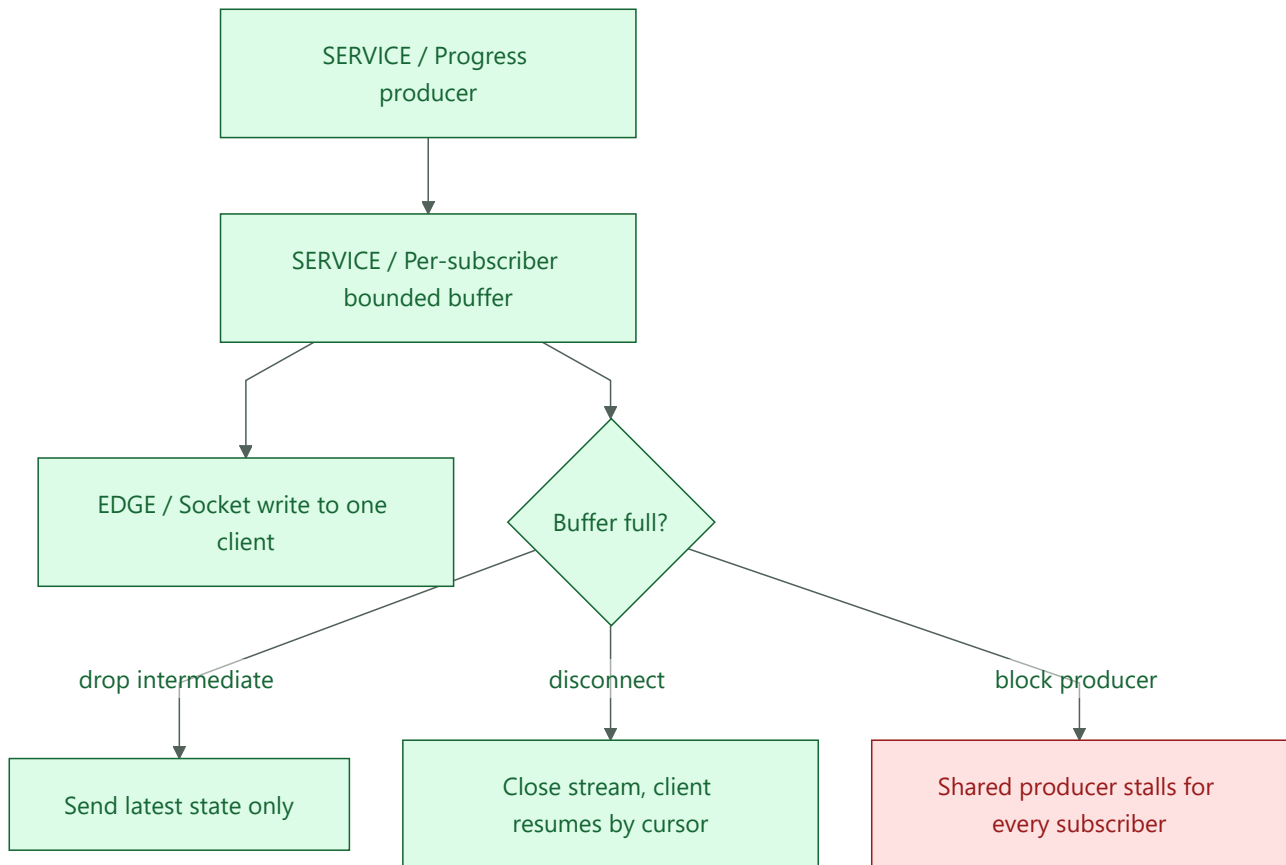
Reconnect politely. Exponential backoff with jitter and a maximum interval prevents a deploy from being followed by a synchronized reconnection surge from every connected client. A client that reconnects instantly on every failure turns a brief restart into a sustained load event. [09](#) covers the browser-side implementation and [18](#) covers the deployment pattern that drops the connections in the first place.

Interview checks: basic: `Last-Event-ID` is a request for retained history, not a guarantee of it. Internals: retention bounds what any cursor can recover. Debug: a silent gap after reconnect usually means an out-of-retention cursor was served from the oldest buffered event. Scenario: fall back to a durable snapshot, then subscribe.

13. Slow Clients and Backpressure

Why a slow client is a server problem: when the server produces events faster than one client consumes them, something must absorb the difference. If nothing is designed to, the buffer is wherever the default happens to be: socket buffers, framework queues or heap. The result is memory growth driven by the slowest consumer, which is the opposite of an isolation property.

Diagram 05.8



Three strategies exist and each is a trade. **Coalescing** suits progress because intermediate percentages are disposable: send the latest state and skip the ones nobody saw, which keeps memory bounded and loses nothing the user needed. **Disconnecting** a hopeless subscriber is honest when every event matters, because the client resumes by cursor and the server stops paying for it. **Blocking the producer** is almost always wrong for a shared stream, since one slow client then throttles everyone.

- Make every per-subscriber buffer explicitly bounded. An unbounded queue is a deferred outage, as 02 argues for executors.
- Decide the overflow policy per stream type. Disposable progress coalesces; an audit or billing feed must not silently drop and should fall back to durable replay.
- Apply write timeouts. A client that has stopped reading but has not closed the connection can hold resources indefinitely.
- Measure subscriber count, buffer occupancy and drop or disconnect counts. Without these, slow-consumer problems appear as unexplained heap growth.

Reactive libraries give this a formal shape. Reactor's `Flux` with a request-based protocol lets a subscriber signal demand, and operators such as the buffering and sampling family make the overflow decision explicit at the point where it happens. That is a genuine strength: the policy is

in the code rather than in whichever buffer happened to grow. It is not a magic absence of the problem, because the policy still has to be chosen.

[VERIFY: per-subscriber buffering behavior, default overflow strategies and write-timeout settings differ across Spring MVC SSE emitters, WebFlux and the servlet container. Confirm the defaults for the selected stack; no load test or memory measurement was performed here.]

Interview checks: basic: backpressure is the absence of an unbounded buffer. Internals: the overflow policy must be chosen per stream semantics. Debug: heap growth correlated with subscriber count points at per-subscriber queues. Scenario: coalesce progress, disconnect-and-replay for feeds where every event matters.

14. Reactor, WebFlux and Virtual Threads

Why the comparison matters: both approaches address the same constraint, which is that a thread blocked on I/O is an expensive way to wait. They differ in what they ask of your code. This is a frequent interview topic and an even more frequent source of unjustified rewrites.

Reactor and WebFlux express work as a pipeline of publishers with explicit demand signalling, executed by a small event-loop thread pool. The gains are a low thread count under high connection concurrency and first-class operators for streaming, composition, timeouts and backpressure. The costs are a programming model that is viral through the call stack, debugging where stack traces no longer describe causation, and the rule that one blocking call on an event-loop thread can stall many unrelated requests.

Virtual threads, as introduced in Java 21 and covered in [01](#) and [02](#), keep ordinary blocking code and sequential control flow while making a blocked thread cheap. Debuggers, profilers, thread dumps, try-with-resources and plain stack traces keep working. They do not provide backpressure, streaming operators or demand signalling, and they do not fix a saturated downstream dependency or a connection pool that is still the real limit.

Constraint	Reactor and WebFlux	Blocking code on virtual threads
Very high concurrent connection count with little per-request CPU	Strong fit, small thread footprint	Workable, but each task still needs its stack and scheduling
Streaming with explicit demand and overflow policy	First-class, operators make the policy visible	Must be built by hand with bounded queues
Existing blocking drivers and libraries	Requires offloading to a bounded scheduler, easy to get wrong	Natural fit, no model change

Constraint	Reactor and WebFlux	Blocking code on virtual threads
Team familiarity and debuggability	Steep; traces and operator semantics need real expertise	Familiar; ordinary tooling applies
Bounded downstream resource such as a connection pool	Still the limiting factor	Still the limiting factor, and easier to exhaust by accident

[!] TRAP

One blocking call on an event-loop thread is not a local slowdown. It occupies a thread that serves many connections, so a single synchronous lookup inside a reactive chain can degrade unrelated requests. Blocking-call detection in development and explicit offloading to a bounded scheduler are not optional extras in a reactive codebase.

The decision rule the guide applies: keep the blocking model unless a specific, measured constraint demands otherwise, because virtual threads removed the most common reason teams reached for reactive code. Reach for Reactor when you genuinely need streaming composition with demand signalling, or when the surrounding ecosystem is already reactive end to end. Do not mix the models casually in one request path; the boundary between them is where blocked event-loop threads and lost context appear. [23](#) covers how to measure before choosing, and [03](#) covers the MVC and WebFlux stack decision.

[VERIFY: virtual-thread support in the selected Boot version, pinning behavior differences between Java 21 and JEP 491 in Java 24 and 25, and current WebFlux defaults all require confirmation against release documentation. No benchmark, thread dump or load comparison was produced in this environment.]

Interview checks: basic: both target wasted waiting, with different programming models. Internals: event-loop threads are shared, so blocking them is not local. Debug: a reactive pipeline with a hidden blocking call shows latency on unrelated endpoints. Scenario: prefer the simpler model unless streaming or demand signalling is genuinely required.

15. Labs and Execution Status

Two self-contained Java sources live under `code/05-apis-realtime/`, with `run.ps1`, `README.md` and `execution.json`. The workspace command is `& ' ./docs/java-fs-guide/code/05-apis-realtime/run.ps1 '`, and an already-installed JDK can be supplied through the optional `-JdkHome` parameter. No downloads are attempted.

Lab	What its assertions cover	What it does not claim
IdempotencyLab	Scoped keys, content binding, replay of a stored result, conflict on key reuse, tenant isolation, retention expiry, and one acceptance among eight concurrent duplicates	Not an HTTP, database-constraint or exactly-once delivery test; the store is an in-memory stand-in
ReplayCursorLab	Cursor inside retention replays only missed events, an up-to-date cursor replays nothing, an out-of-retention cursor requires a snapshot, a missing cursor is not a replay, and an impossible cursor is rejected	Not an SSE, proxy, browser or network test; retention is a fixed-size deque, not a broker

NOT EXECUTED: the runner's preflight ran and reported that no JDK compiler is reachable through `PATH` or `JAVA_HOME`, and downloads are not authorized. Nothing was compiled, no assertion was evaluated, and no output was produced. The inline `ReplayCursorLab` listing above is checked against its saved source, which verifies synchronization rather than correctness.

What a real test environment should add: contract tests against the published OpenAPI document, an integration test that submits the same idempotency key concurrently against a real unique constraint, a proxy-in-the-path test that would catch SSE buffering, and a slow-consumer test that asserts bounded memory. 19 covers those once the toolchain exists.

A Repeatable Diagnosis

- State the failing contract precisely: duplicate work, wrong status, stale page, missed event, stalled stream, or unbounded memory.
- Identify which boundary owns it. Duplicates are an idempotency or consumer-deduplication question; missed events are a retention and cursor question; stalled streams are usually an intermediary.
- Reproduce with the real path included. A stream that works against localhost proves nothing about the gateway, and that is where buffering lives.
- Check the status and headers before the body. A non-JSON error body or an unexpected 504 identifies the layer that produced it.
- For duplicates, compare the scoping tuple and the retention window against the client's retry horizon before changing handler code.
- For memory, correlate heap growth with subscriber count and buffer occupancy, not with total request rate.
- Fix the owning contract and add a test that fails for that specific mistake, without claiming a broader guarantee than it exercises.

Explained-Here Index and Sources

Concept	Explanation
Resources, methods, status selection and representations	HTTP semantics
Problem details and safe error contracts	Error model
Compatible change, version axes and deprecation	Versioning
Offset, keyset cursors, counts and bounded cost	Pagination
Scoped keys, content binding, retention and unknown outcomes	Idempotency
Rate limits, quotas, signatures and delivery retries	Limits and webhooks
OpenAPI, Swagger tooling and contract discipline	Documentation
gRPC and GraphQL trade-offs	Protocol alternatives
Interval polling, conditional requests and long polling	Polling
Event streams, buffering and keep-alive requirements	SSE
Full-duplex sockets, STOMP destinations and their costs	WebSocket
Last-Event-ID, retention limits and snapshot fallback	Reconnect and replay
Bounded buffers, coalescing and overflow policy	Backpressure
Reactive pipelines versus blocking code on cheap threads	Reactor and virtual threads
Lab scope and actual execution status	Labs

Owning references for further checking: [RFC 9110 HTTP semantics](#), [RFC 9457 problem details](#), the [HTML living standard server-sent events section](#), [OpenAPI 3.1](#) and the [Spring Framework web documentation](#). These are where to verify the mechanisms described above; citing them is not a claim that this environment retrieved or executed them.

Related Chapters

Build on [03 web layer and transactions](#) and [04 persistence boundaries](#), with [01 Java](#) and [02 concurrency](#) underneath. [06 databases](#) owns the constraints and indexes this chapter relies on, and [07 messaging](#) owns everything after acceptance. [08 security](#), [09 frontend contracts](#), [13 partner integration](#), [18 operations](#), [19 testing](#), [21 network and OS](#), [22 distributed systems](#), [23 performance](#) and [17 cloud](#) extend the boundaries touched here. Return to [00](#) for the whole request journey. Generated Related chapters links are reciprocal.

RELATED CHAPTERS / RECIPROCAL LINKS

00	The System Before the Details	13	Data Integration and iPaaS
01	Core Java and the JVM	17	Cloud and Infrastructure
02	Concurrency and the Memory Model	19	Testing and Quality
21	Networking and Operating Systems	23	JVM Performance Engineering
03	Spring and Spring Boot Internals	22	Distributed Systems Theory
04	Hibernate and JPA	10	System Design Foundations
06	Databases and Durable State	25	Architecture Styles and Domain Design
07	Messaging, Batch and Streams	11	Object Design, Patterns and LLD
08	Security, Identity and Data Protection	12	Twelve System Design Case Studies
09	The Browser and the Java Backend	27	Fintech and Payments Engineering
18	Observability and Reliability	24	IntegrationHub in Motion
		20	Appendix: Interview Toolkit and Index

One-Page Cheat Sheet

Semantics: safe, idempotent and cacheable are three separate properties. PUT replaces, PATCH modifies, POST is the general case. 202 means durably accepted, not completed. Never flatten failures into 200 or 500.

Errors: publish a stable problem-details shape with type URIs, a correlation identifier and explicit retryability. Never expose stack traces, SQL, internal hosts or other tenants' data. Normalize edge-produced errors so clients always get the documented content type.

Versioning: additive change is compatible only for tolerant readers. Removals, renames, narrowed input and changed meanings are breaking. Measure per-client usage before deprecating. Mobile and webhook receivers set the real support window.

Pagination: always bound page size. Offset paging shifts under concurrent writes and costs more with depth. Keyset needs a total, stable ordering and an index. Exact counts are an opt-in cost. Cursors are opaque and validated within the tenant scope.

Idempotency: scope the key by tenant and operation, bind it to content, store the result, and state retention. Enforce with a unique constraint, not select-then-insert. A timeout is an unknown outcome. Acceptance is not completion, and this is not exactly-once.

Limits and webhooks: 429 with guidance, jittered backoff, per-tenant enforcement. Outbound deliveries need stable event ids, signatures with timestamps, bounded timeouts, backoff with a terminal state, replay, and destination validation against internal addresses.

Transports: polling is the resilient baseline and deserves conditional requests. SSE is one-directional text over HTTP and fails on proxy buffering and idle timeouts. WebSocket buys bidirectionality at the cost of upgrades, stickiness and per-message authorization. Choose the weakest mechanism that meets the requirement.

Streams: `Last-Event-ID` requests retained history and does not guarantee it. Handle cursor-in-retention, caught-up, out-of-retention, absent and impossible cursors distinctly. Bound every per-subscriber buffer, coalesce disposable progress, disconnect hopeless subscribers, and never let one slow client block a shared producer. Reactor offers explicit demand; virtual threads keep blocking code cheap; neither fixes a saturated downstream. The Java labs remain NOT EXECUTED.

Interview Corner

Basic: What Makes an API RESTful Beyond Returning JSON?

Resource identity, uniform method semantics, stateless requests, and meaningful use of status codes, conditional requests and caching. JSON is a representation format, not an architectural style. Most production APIs stop short of full hypermedia and should say so honestly.

Internals: Why Is PUT Idempotent but POST Is Not?

PUT describes the complete intended state at a known identity, so repeating it converges on the same result. POST is a general submission whose effect may be cumulative, such as appending a new job. Idempotency is a property of the defined method semantics, not of your handler.

Trace/Debug: A GET Started Causing Duplicate Side Effects. Where Do You Look?

Look for a handler that mutates state on a safe method. Caches, prefetchers, link scanners and retry middleware all repeat GETs freely, so the duplicates often appear without any client change. Move the mutation to POST and audit for other safe-method writes.

Scenario: The Job Takes Ten Minutes. What Does the Submission Return?

202 Accepted with a Location pointing at a job status resource, after the job and its idempotency record are durably committed. The body carries the job identity and current state. The client then polls the status resource or subscribes to the progress stream.

Basic: What Should an Error Response Contain?

A stable machine-readable type, the status, a human-readable summary, a correlation identifier, structured field violations where relevant, and an explicit retryability signal when status alone is ambiguous. No stack traces, SQL, internal hostnames or cross-tenant values.

Internals: Why Prefer a Type URI Over an Error Message?

Message text is written for humans and changes freely during refactors and translations. A type URI is a committed identifier that clients can branch on and that survives rewording. It also gives you a place to document the condition and its remedy.

Trace/Debug: Clients Report "Invalid JSON" During an Incident. What Is Likely Happening?

An intermediary is producing the error rather than the application, typically an HTML gateway timeout or error page. Check the status and content type before parsing. Normalize edge errors into the documented problem format so this failure mode stops being ambiguous.

Scenario: How Do You Signal That a Failure Is Worth Retrying?

Use the status correctly first, add `Retry-After` for limits and transient server conditions, and include an explicit retryability member when the distinction is not inferable. Document the backoff expectation, because a retry policy you did not specify will be invented by the client.

Basic: Which API Changes Are Backward Compatible?

Adding an optional field or a new endpoint is usually compatible for tolerant readers. Removing or renaming fields, tightening validation, changing a field's meaning or default, and changing status semantics are breaking. Adding an enum value is breaking for clients without a fallback.

Internals: Why Is a Tolerant Reader a Precondition Rather Than an Assumption?

Compatibility depends on the consumer's deserialization behavior, which you do not control. A strict-schema client fails on an unknown field. Document the reader requirement as part of the contract, and treat known strict consumers as a constraint on what counts as additive.

Trace/Debug: How Do You Know It Is Safe to Remove Version One?

Instrument usage per client and credential, not just aggregate call counts. Announce the replacement, publish a sunset date, return advisory headers, and watch the per-client series fall to zero. Absence of complaints is not evidence of absence of callers.

Scenario: Path Versions or Header Negotiation?

For a public or partner-facing API, path versions, because they are visible, routable, cacheable and easy to support. Header negotiation is defensible when the whole path, including caches and gateways, genuinely implements it. The failure mode is an intermediary that drops the header and serves the default.

Basic: Why Is Deep Offset Pagination Expensive?

The engine generally has to produce and discard the skipped rows before returning the page, so cost grows with depth. Concurrent inserts and deletes also shift the window between pages, which can duplicate or skip rows during a walk.

Internals: What Does Keyset Pagination Require to Be Correct?

A total, stable ordering, an index that supports the seek, and a cursor that encodes every column in that ordering. Ordering by a timestamp alone is not total when values collide, so pair it with a unique key. Filters must be part of the cursor contract or results shift.

Trace/Debug: An Export Produced Duplicate and Missing Rows. What Happened?

Almost certainly offset pagination over a table receiving concurrent inserts or deletes. The page boundaries moved between requests. Switch to a keyset walk over a stable ordering, or snapshot the data set, and verify with a run under concurrent write load.

Scenario: The UI Wants Total Pages for a Large Tenant History. What Do You Offer?

Explain that an exact count over a large filtered set is expensive and immediately stale. Offer a has-more indicator derived from fetching one extra row, an estimated count, or an opt-in exact count with a documented cost. Keep the default cheap.

Basic: What Problem Does an Idempotency Key Solve?

It lets a client retry a non-idempotent operation after an unknown outcome without creating duplicate work. The server recognizes the repeated logical request and replays the recorded result instead of performing the operation again.

Internals: Why Is a Unique Constraint Better Than Checking Before Inserting?

Select-then-insert has a race window: two concurrent retries can both find nothing and both proceed. A unique constraint makes the reservation atomic at the authority, so exactly one attempt wins and the other observes the stored result. The lab models this with an atomic compute rather than a read followed by a write.

Trace/Debug: Duplicate Jobs Appeared Despite Idempotency Keys. What Do You Check?

Check the key's scope, whether the retry reused the same key at all, whether the retention window had expired before the retry, whether the check happened before the insert, and whether a second path such as a bulk import bypasses the key entirely.

Scenario: Does Idempotency Give You Exactly-Once Processing?

No. It bounds duplicate acceptance at the API boundary for one retention window. Broker delivery is still at-least-once, the worker needs its own deduplication, and a timed-out call to an external connector remains an unknown outcome that only that system's idempotency contract can resolve.

Basic: When Would You Choose Polling Over Streaming?

When updates are infrequent, the latency requirement is seconds rather than milliseconds, intermediaries are outside your control, or client diversity is high. Polling with conditional requests is cheap, stateless and recovers from disconnection with no design work.

Internals: Why Does SSE Deliver Nothing Through Some Gateways?

Because an intermediary is buffering the response. Buffering is a reasonable default for ordinary responses and fatal for a stream that never ends. Disabling buffering on the stream path, plus idle timeouts longer than the keep-alive interval, is what makes it work.

Trace/Debug: A Stream Works Locally and Fails in Production. What Is Your Sequence?

Reproduce through the real path, not localhost. Check for buffering, idle timeouts, compression that defers flushes, and per-origin connection limits. Confirm keep-alive frames are emitted and reaching the client, then check whether the gateway terminated the response at a fixed duration.

Scenario: Progress Updates Only. SSE or WebSocket?

SSE. The data is one-directional text, reconnection and the event cursor are built in, and ordinary HTTP authentication and infrastructure apply. WebSocket adds an upgrade, sticky routing and per-message authorization for a bidirectionality the feature does not need.

Basic: What Does Last-Event-ID Actually Guarantee?

That the client told the server which event it last saw. Whether the server can satisfy that cursor depends entirely on retention. Outside the retained window, the only correct response is to require a fresh snapshot of durable state.

Internals: What Are the Distinct Cursor Cases a Resume Handler Must Cover?

Inside retention, replay exactly the missed events in order. Equal to the current position, replay nothing and keep streaming. Older than retention, require a snapshot. Absent, treat as a first connection. Ahead of the stream, reject as impossible rather than treating it as caught up.

Trace/Debug: Heap Grows With Connected Users but Not With Request Rate. Why?

Per-subscriber buffers. Each slow or stalled consumer accumulates undelivered events, so memory tracks subscriber count and buffer occupancy rather than throughput. Bound the buffers, choose an overflow policy, add write timeouts, and emit metrics for drops and disconnections.

Scenario: One Client Cannot Keep Up With a Shared Progress Stream. What Do You Do?

Coalesce to the latest state, since intermediate progress values are disposable, and bound the buffer. If every event matters, disconnect that subscriber and let it resume by cursor. Never block the shared producer, because that converts one slow client into a global stall.

07 / Messaging, Batch and Streams

Moving a message is not the same as completing its effect. IntegrationHub has already committed a sync job when the API returns 202. Publication may still be pending, a worker may repeat an attempt, and the browser may miss a progress notification. This chapter connects those gaps with durable intent, explicit ownership and restartable work. IntegrationHub is fictional; its design is an instructional example, not a description of a real company's deployment.

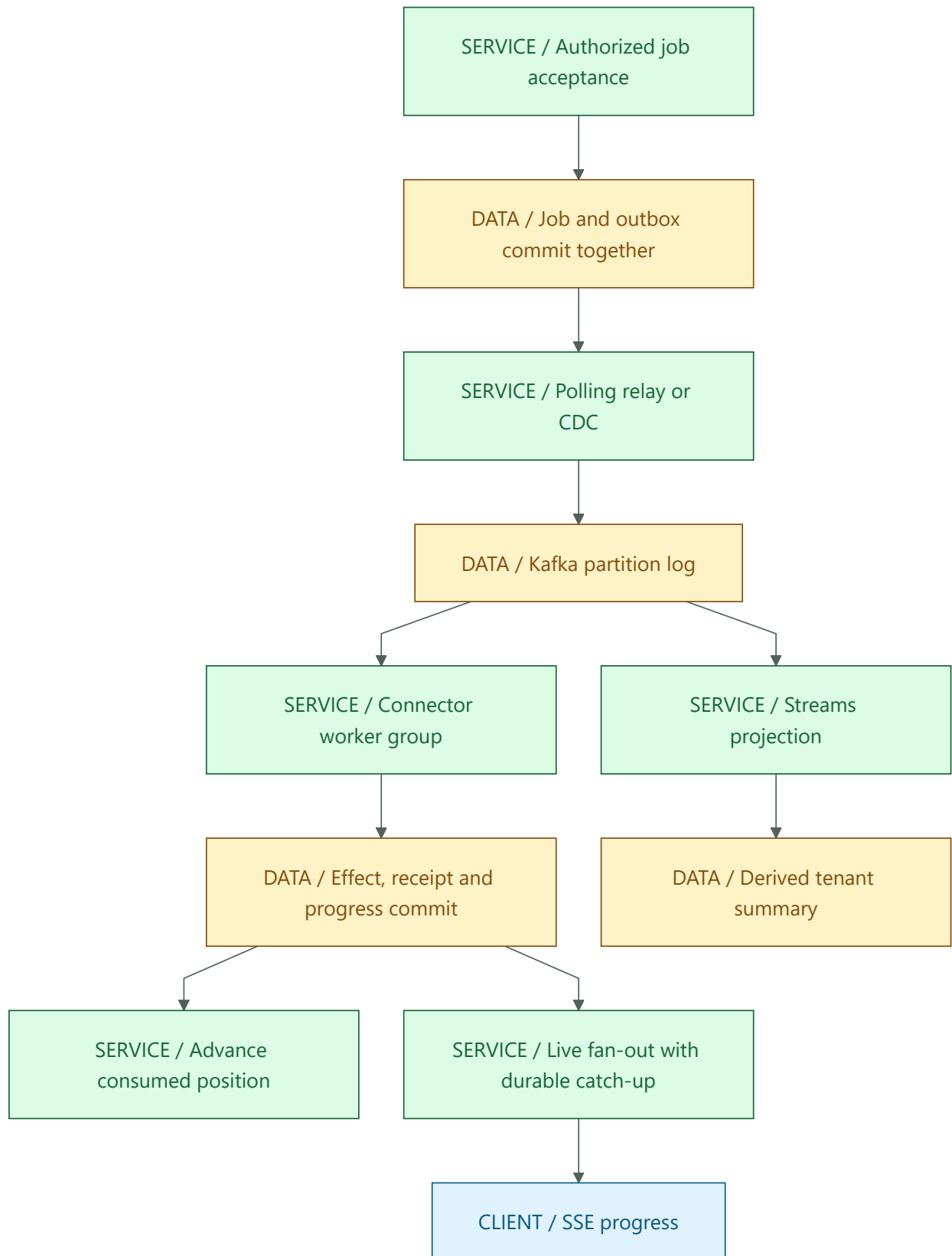
Version assumptions: Java 21, the previously checked Spring Boot 4.0.x / Spring Cloud 2025.1.x family, PostgreSQL 17, Kafka 4.x and Kubernetes 1.34. Kafka 4.0 documentation is the versioned mechanism reference here, not a recommendation to install an old patch. Spring Batch's current reference describes the 6.x generation; its builder and repository APIs must not be mixed indiscriminately with older tutorials. RabbitMQ acknowledgements are discussed for AMQP 0-9-1; RabbitMQ Streams is a distinct option, not the queue model described below. Debezium's stable documentation is a moving reference.

[VERIFY: check the actual Boot-managed Spring Kafka, Kafka client and Spring Batch versions, broker compatibility, RabbitMQ queue policies and Debezium connector compatibility before choosing artifacts. Kafka 4.0 Streams, current Batch domain, RabbitMQ confirms and Debezium outbox-router documentation were consulted, but dependencies were not resolved and no broker or framework ran.]

Execution status: NOT EXECUTED. The local runner checked for an existing JDK and found no compiler. Java and dependency downloads remain unauthorized. The complete Java 21 DeliveryLab under `code/07-messaging/` models four failure scenarios without a broker or database. Its assertions have not run. There are no fabricated broker logs, throughput results, SQL traces or framework outputs in this chapter.

Big Picture

Diagram 07.1



The first transaction establishes accepted work, the consumer transaction establishes a local business effect, and the offset records how far the consumer can safely resume. They are separate facts. Streaming projections and browser notifications are downstream views; their delivery does not define whether the underlying job committed.

What You Will Be Able to Explain

- Trace a Kafka record through partition selection, broker acknowledgement, consumer ownership, business commit and offset advancement.
- Contrast Kafka's retained log with RabbitMQ queues without pretending either product has only one messaging model.
- State what at-most-once, at-least-once and exactly-once mean at a named boundary.
- Implement the transaction boundary behind an idempotent database consumer and explain why external API effects need a different contract.
- Compare polling outboxes with log-based CDC, including duplicates, ordering, cleanup and replication-slot failures.
- Design a saga with compensations and unresolved outcomes rather than imaginary distributed rollback.
- Separate event sourcing from CQRS, and justify when neither is worth its operational cost.
- Explain KStream, KTable, windows, local state, repartitioning and the scope of Kafka Streams transactions.
- Design a restartable Spring Batch job with stable input identity, checkpoints, bounded skip/retry policies and disjoint partitions.

[P] PRODUCTION

Where this lives in IntegrationHub. 03's service transaction, 04's persistence layer and 06's database commit the job with publication intent. 05 owns HTTP retry identity and SSE recovery; this chapter owns the next delivery boundaries. 13 supplies connector input contracts, 26 consumes analytical changes, and 18 watches event age, failed work and recovery. Tenant authorization and payload protection remain 08's responsibility even on internal topics.

1. Commands, Events, Batch and Streams

Messaging decouples when one participant can produce work from when another can execute it. That is useful for burst absorption, independent deployments and replay. It also replaces an immediate answer with another operational obligation: someone must track unfinished work and tell the caller what eventually happened.

A **command** asks an owner to do something and can be rejected: StartSync is a request. An **event** reports an accepted fact: SyncRequested or SyncCompleted. A fact may lead several independent consumers to act, but those actions are not part of the original fact's transaction. Do not call an event Completed if the only thing known is that a command was placed in a queue.

Event-driven describes what triggers work. **Batch** describes bounded input and a completion boundary. **Stream processing** describes incremental computation over continuing input, often with state and time. A file-arrival event can launch a batch job; a stream can continuously aggregate events emitted by batches. These are compatible dimensions, not three competing brands of middleware.

Approach	Use when	Avoid when
Synchronous call	A caller needs a bounded answer now and can handle dependency failure	Long work keeps an HTTP connection and database transaction open
Event-driven worker	Durable accepted work can finish asynchronously	Nobody owns completion status, retries or unresolved work
Batch	A named input set has a finite reconciliation or import result	The product needs continuous low-latency updates from unbounded input
Stateful stream	Updates must continuously maintain aggregates, joins or alerts	A periodic SQL query already meets freshness and cost requirements

For IntegrationHub, a tenant submits a command over HTTP, the database records acceptance, an outbox event dispatches it, and a worker may execute a bounded import as a batch. A separate stream maintains operational summaries. The same business journey uses all four mechanisms without giving them the same guarantee.

Interview checks: basic: command requests, event records. Internals: publication and reaction are different transactions. Debug: a job stuck at accepted needs an owner at each downstream boundary. Scenario: a nightly reconciliation is still batch even if a message launches it.

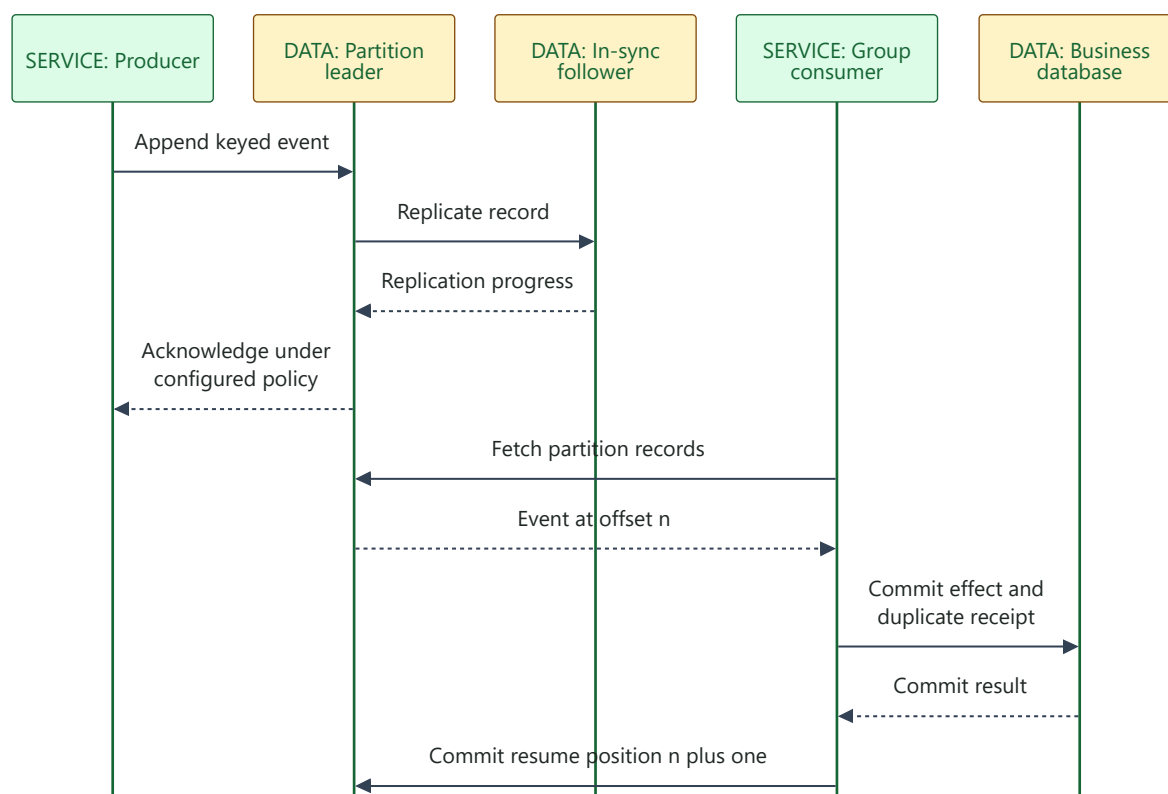
2. Kafka: Partitions, Replication and Consumer Ownership

Kafka retains records in ordered partition logs. A record has a topic, partition and offset; the offset is a position within that partition, not a global sequence or business event identity. Consuming a record does not delete it. Retention or compaction governs how long it remains available, so different consumer groups can process the same history independently.

The producer serializes a key and value, selects a partition, batches records and sends them to the partition leader. Followers replicate the leader's log. The producer receives an acknowledgement according to its configuration; failure to receive one leaves the producer uncertain whether an append occurred. Producer retries and idempotence address this protocol-level uncertainty, not every possible duplicate application submission.

`acks=all` works with the in-sync replica set and the configured minimum in-sync replicas. It does not mean every replica ever assigned to the partition is online and has acknowledged. A replication factor names intended copies; a minimum ISR requirement determines when the broker should refuse a write rather than accept it with too little replication. Availability and durability are a deliberate trade during a failure.

Diagram 07.2



For ordinary partition-assigned consumer groups, one consumer owns a partition at a time within a group. Extra consumers beyond the available partitions cannot increase this group's partition parallelism. Another group receives its own assignments and offsets, which is how a search projector and a job worker can both read the same event without stealing it from each other. This explanation is not the newer share-group consumption model.

A rebalance transfers ownership after membership or assignment changes. The old owner may still have application work in progress, so merely changing the broker assignment does not undo that work. Stop handing revoked work to new tasks, coordinate in-flight completion and

use idempotent effects or fencing for stale work. Never share a `KafkaConsumer` across arbitrary worker threads as if it were a concurrent queue.

Offset commitment follows the completed prefix. If offsets 10 and 12 finished but 11 is still running, committing position 13 can skip 11 after a crash. A parallel consumer must track per-partition completion and advance only across a contiguous completed prefix. Committing an offset says nothing about an unrelated transaction unless the application arranged that relationship.

Choice	Use when	Avoid when
Key by tenant and job	Per-job partition order matches the invariant	A single hot job's throughput requirement exceeds one ordered lane
Separate consumer group per function	Workers, projections and auditing independently need the event	A new group is created accidentally on every restart and reprocesses history
More partitions	Measured partition parallelism is the limit	A downstream database or provider quota is already saturated
Log compaction	A keyed latest-state changelog can rebuild a table	Every historical transition must remain recoverable forever

Compaction is asynchronous. Multiple values for a key can coexist until cleaning, and tombstone retention matters to consumers that have been offline. It is not immediate deduplication or a substitute for an audit retention policy. Time/size retention can remove records before a lagging consumer reaches them; replay is conditional on retained data.

[VERIFY: verify producer idempotence constraints, `acks/min.insync.replicas` interactions, unclean leader election, compaction/tombstone retention, group protocol and rebalance configuration against the selected Kafka 4.x broker and client. This section describes ordinary partition-assigned groups, not share groups; no failover or rebalance test ran.]

Interview checks: basic: order belongs to a partition. Internals: producer acknowledgement is not consumer completion. Debug: adding consumers cannot fix a hot partition. Scenario: parallel processing must commit the completed prefix, not the largest offset observed.

3. RabbitMQ: Routing, Confirms and Acknowledgements

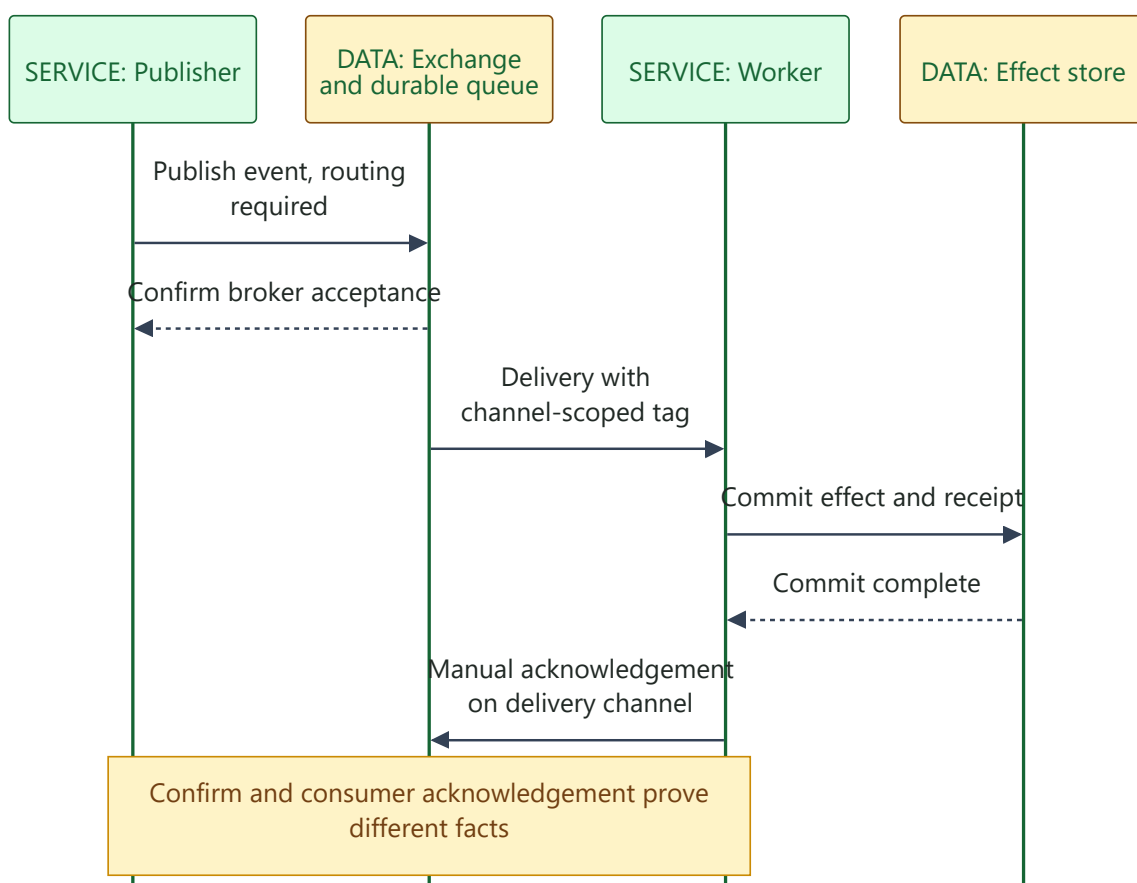
RabbitMQ's AMQP queue model separates routing from consumption. Producers publish to an exchange; bindings route messages to queues; consumers compete for deliveries from a queue. A direct exchange routes by exact binding keys, a topic exchange uses patterns, and a

fanout exchange routes to its bound queues. Independent subscribers normally need independent queues. Several consumers on one queue divide work rather than each receiving every message.

The publisher's confirm answers whether the broker accepted the publication under the queue's durability rules. A consumer acknowledgement says the consumer considers a delivery handled. The official confirms guide explicitly treats them as independent. A publisher confirm cannot establish that a worker wrote PostgreSQL successfully, and an application writing bytes to a socket has not yet received a publisher confirm.

An unroutable publication can be confirmed even though no queue received it. When routing is required, handle returned mandatory publications or a deliberate alternative-routing policy as well as confirms. Queue durability, message persistence, replication and confirms must be designed together; a durable queue name alone does not prove that a particular message survived a crash.

Diagram 07.3



Manual acknowledgement allows redelivery when a consumer disconnects with unfinished work. Automatic acknowledgement trades away that recovery point. Delivery tags are scoped to a channel and must be acknowledged on the channel that supplied them; they are not stable

deduplication IDs. Use an application event ID that survives reconnection.

Prefetch bounds outstanding unacknowledged deliveries under the configured consumer/channel scope. It prevents one fast network reader from building an unlimited local backlog, but does not bound every queue in the application. An executor added after delivery still needs its own admission policy. A high prefetch can also concentrate work on a slow consumer; tune from payload size, service time and capacity rather than copying a universal number.

Model	Use when	Avoid when
RabbitMQ queues	Routing and competing task consumers fit the work	You assume acknowledged queue deliveries remain an indefinitely replayable log
Kafka partition log	Independent groups need retained replay and partitioned processing	Topic partitioning and retention cost buy no value for the task
RabbitMQ stream model	Retained-log semantics fit an existing RabbitMQ platform	Queue assumptions are copied without checking the stream protocol
Manual ack plus bounded prefetch	Consumer effects must survive redelivery safely	Acknowledgement happens when work is merely submitted to an executor

[VERIFY: confirm RabbitMQ queue type, persistence/confirm behavior, mandatory returns, prefetch scope, delivery limits and dead-letter guarantees against the deployed version. The current 4.3 confirms page was consulted; no RabbitMQ deployment or channel test was executed.]

Interview checks: basic: exchanges route and queues hold deliveries. Internals: confirms and consumer acknowledgements are orthogonal. Debug: a confirmed but missing message can be unroutable. Scenario: stable event identity belongs in the message, not the delivery tag.

4. Delivery Semantics and the Crash Matrix

The labels at-most-once and at-least-once describe a boundary and a failure model. A system can be at-least-once from broker to consumer but at-most-once for an external effect if it acknowledges before the provider call. Always say what is being counted: broker records, handler invocations, database updates or customer-visible actions.

At-most-once accepts loss to avoid repeated delivery. At-least-once retries unfinished or uncertain work and therefore permits duplicates, subject to retained data and eventual recovery. Exactly-once effects require atomic state transitions or idempotency at the effect's authority. They do not require that every instruction physically execute only once.

Crash boundary	Durable fact left behind	Recovery obligation
Before acceptance transaction commits	No accepted job or outbox event	Retry the request under its API key
After acceptance commit, before relay runs	Job plus publication intent	Relay must resume unpublished intent
After publish, before relay records success	Broker may already hold the event	Republish same event ID; consumer handles duplicates
Before consumer database commit	No effect or durable receipt	Redeliver and retry the whole local unit
After consumer commit, before offset or ack	Effect and receipt already exist	Redelivery observes receipt and does not repeat effect
After ack, before business effect	Broker considers work handled, effect missing	Lost work; change the acknowledgement order
After provider effect, before response arrives	External outcome unknown	Query, reconcile or retry with the same provider key

The forbidden shortcut is to solve the duplicate window by moving the acknowledgement earlier. That exchanges visible duplicates for silent loss. IntegrationHub commits local effects first, acknowledges afterward and makes redelivery safe. It cannot atomically commit an HTTP provider's internal state with its own PostgreSQL transaction, so provider-side keys and reconciliation remain necessary.

[M] MECHANISM

The safe gap is after the effect and before the acknowledgement. A crash there repeats delivery but can be neutralized by an atomic receipt. Reversing the order creates a gap where the broker has forgotten work that never happened.

Guarantee	Use when	Avoid when
At-most-once	Loss is explicitly acceptable for disposable samples	Accepted jobs or money movement can disappear
At-least-once plus idempotent effect	A local authority can atomically recognize repeated intent	Duplicate detection is an in-memory cache that vanishes on restart
Kafka transactional processing	Input offsets and Kafka outputs form the unit	An unrelated database write is assumed to join the Kafka transaction

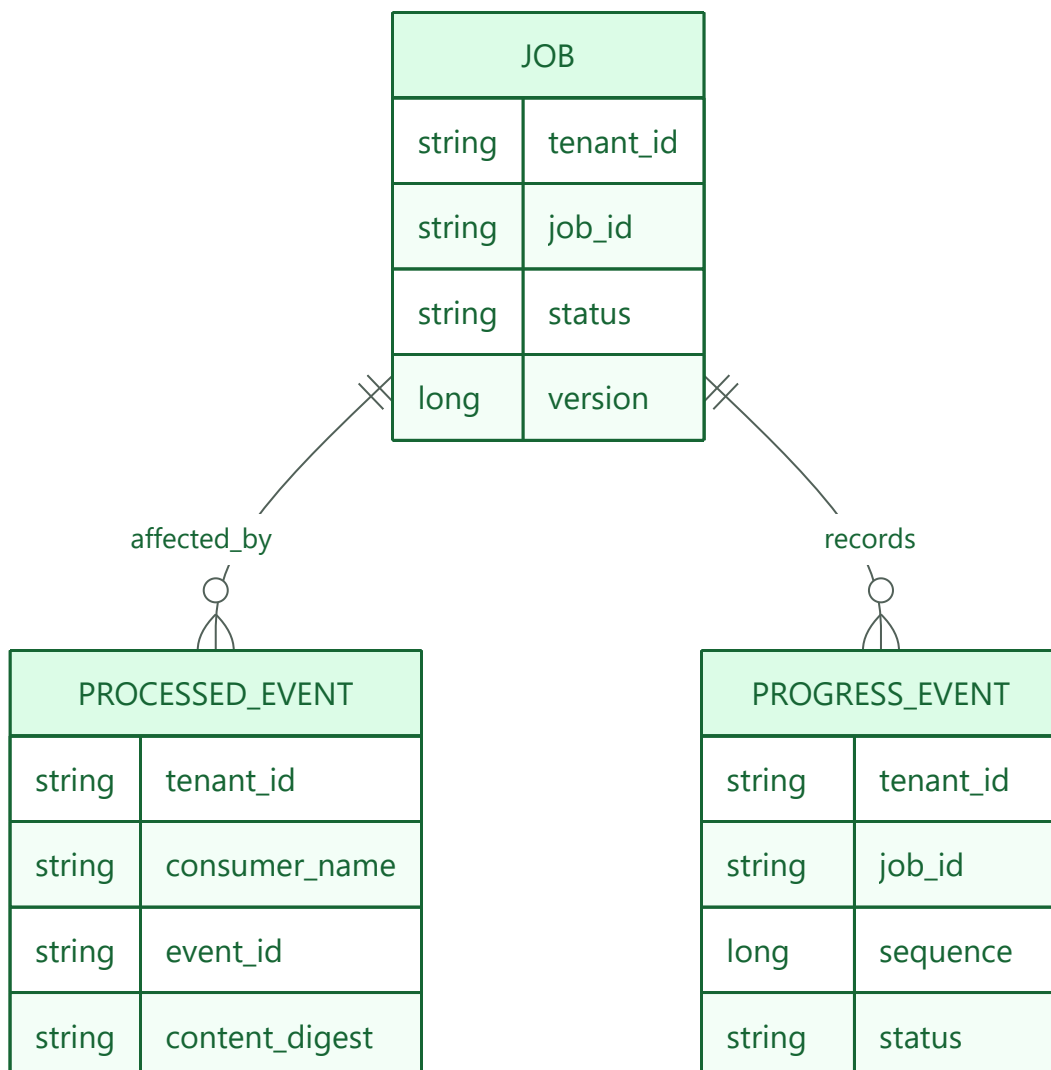
Interview checks: basic: repeated delivery does not have to mean repeated effect. Internals: enumerate the commit/ack crash gap. Debug: missing work often comes from early acknowledgement. Scenario: name the external authority before claiming exactly-once.

5. Idempotent Consumers and Ordering

A durable idempotent consumer keys a receipt by tenant, consumer function and event identity. Consumer function matters: the search projection must still process an event already handled by the progress worker. The receipt and local business effect must commit together, under a unique constraint that arbitrates concurrent duplicates.

The application opens a transaction, tries to reserve the scoped event ID, applies the valid state change, writes progress or next publication intent and commits. A duplicate key means inspect the prior receipt, including a content binding when required, rather than apply the effect again. If anything fails before commit, the reservation must roll back too. Marking a receipt in a separate first transaction can suppress work that never completed.

Diagram 07.4



The drawing omits physical keys for readability; the schema needs explicit tenant-scoped constraints. An event receipt is not an authorization decision. Validate topic access, event provenance, tenant ownership, schema and allowed state transition before mutating state. Never trust a tenant header merely because the message came from an internal broker.

Retention is part of duplicate safety. If receipts expire before retained messages, a replay of old history can repeat previously committed effects. Either retain identities for the replay horizon, make the operation naturally idempotent, or rebuild a separate projection with a new consumer identity. A replay into a fresh search index is different from replaying customer notifications or external mutations.

Ordering requires more than a key. A consistent tenant/job key routes related events to one partition while the partitioner and partition count remain compatible. Multiple producers can still append logically later events before earlier ones. Increasing partition count can change where a key maps. Concurrent handlers and retry topics can reorder completion even when the log order is correct.

Give events an aggregate version where order affects validity. A receiver can reject or park a gap, ignore an obsolete version when the operation is a monotonic state replacement, or reconcile from authority. Do not discard a gap blindly for noncommutative deltas. Adding 5 and then subtracting 5 may commute numerically, but validating a quota or sending a terminal notification usually does not.

Strategy	Use when	Avoid when
Atomic receipt plus effect	Local transactional writes face duplicates	Receipt is committed separately or deleted before possible replay
Versioned state replacement	Newer authoritative state supersedes older notifications	Intermediate deltas or transitions must all be applied
Serial processing per key	Ordered noncommutative changes are essential	Unrelated tenants are forced through one global bottleneck
Park gaps and reconcile	Missing earlier state makes current application unsafe	Parking has no timeout, owner or repair process

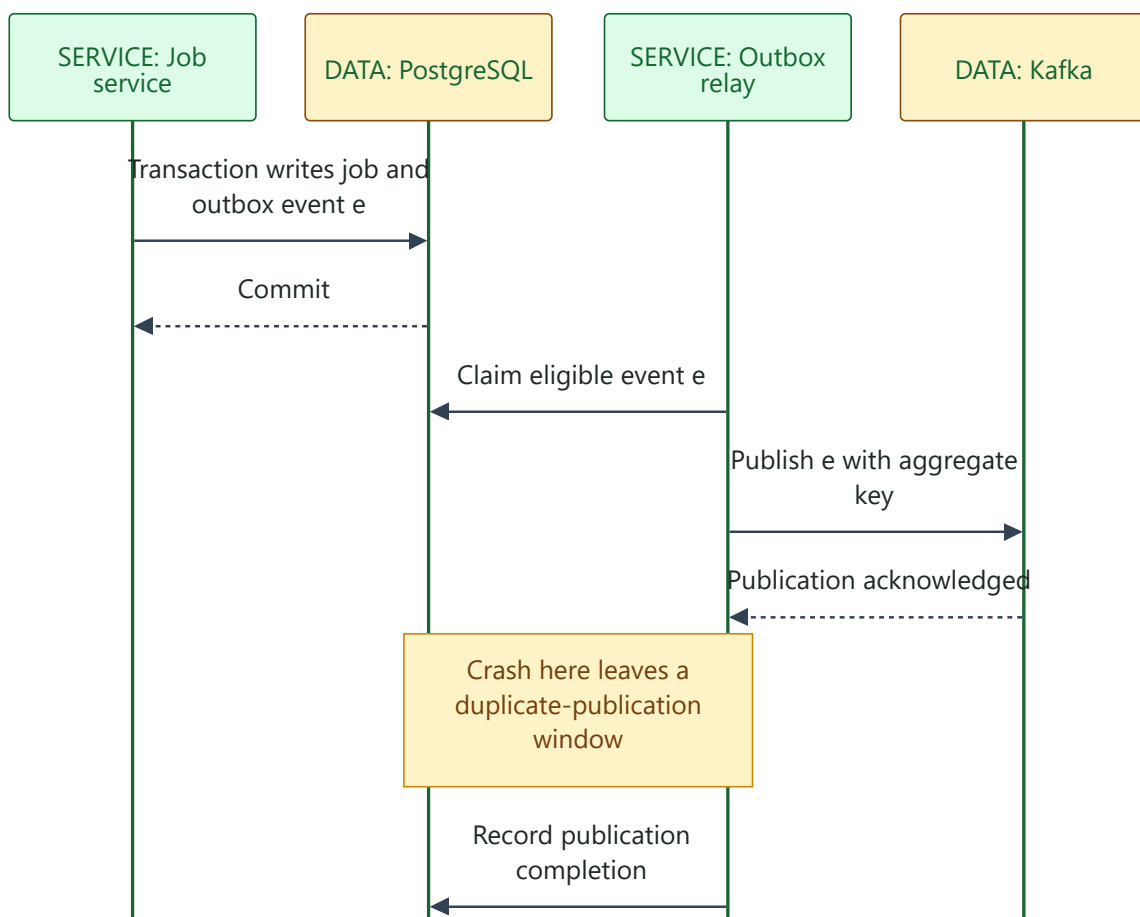
Interview checks: basic: receipt scope includes the consumer. Internals: uniqueness and effect share one transaction. Debug: receipts without effects expose a split commit. Scenario: event ordering, handler ordering and effect ordering are separate questions.

6. Transactional Outbox and Log-Based CDC

The dual-write problem is simple: a service updates PostgreSQL and publishes Kafka, but neither ordinary local transaction covers both. Publish first and a rollback can create a nonexistent fact. Commit first and a crash can leave a real fact without notification. The outbox replaces the second write with publication intent in the same database transaction as the business update.

A polling relay repeatedly discovers eligible rows, claims a bounded set, publishes with stable event IDs, waits for the relevant broker result and marks progress. Claiming prevents uncontrolled duplicate simultaneous work but cannot remove the publish/mark crash gap. Use short claims or leases with a recovery rule; do not hold a database transaction across arbitrary broker delays by accident.

Diagram 07.5



Auto-increment IDs are not universal commit-order cursors. One transaction can allocate a lower ID, stall and commit after a transaction with a higher ID. A relay that permanently advances a high-water mark past the uncommitted lower ID can miss it. Poll an explicit

unpublished state with a correct claim protocol, or use a committed-log position rather than assuming identity allocation establishes commit order.

CDC changes the relay, not the need for intent. Debezium can read committed database log changes and apply an outbox event router to explicit outbox inserts. The consulted router documentation uses event ID for identity and aggregate ID as the message key. This reduces polling and preserves a domain publication contract without broadcasting every internal table change as a public event.

Raw table CDC is useful for replication and projections, but a row update is not automatically a domain event. Renaming a column should not accidentally change a partner's business contract. The outbox allows the service to choose event type, schema version and safe payload while CDC handles extraction. Do not simultaneously run an uncoordinated polling publisher and CDC publisher over the same records during migration.

CDC introduces a real operational system: source permissions, replication slots, connector offsets, snapshots, schema history, serialization, routing and recovery. A stopped consumer can retain WAL until the primary runs out of disk. Deleting a slot to free space can destroy the resume position and require a resnapshot. The database may recover while the downstream projection remains behind; monitor both.

Relay choice	Use when	Avoid when
Polling outbox	Volume is modest and an application-owned relay is easier to operate	Poll scans, latency or claim contention dominate
CDC over explicit outbox	Reliable log extraction and domain events justify connector operations	No team owns slots, offsets, snapshots or schema compatibility
Raw-table CDC	A derived view intentionally follows storage changes	External business contracts become coupled to private tables
Direct database-plus-broker dual write	Only if loss/inconsistency is explicitly acceptable	Both writes are presented as one atomic accepted fact

[VERIFY: check Debezium outbox-router insert/update/delete handling, snapshot behavior, connector offset recovery, publication ordering and PostgreSQL slot retention against pinned connector versions. The stable router reference was consulted; no CDC connector, cleanup policy or migration was executed.]

Interview checks: basic: the outbox persists intent with the fact. Internals: CDC can publish the outbox without replacing it. Debug: a growing slot can endanger the primary. Scenario: a monotonically allocated ID is not proof of commit order.

7. Retries, Dead Letters and Replay

Retries recover transient failures, but repeated work consumes capacity and can prolong an outage. Classify failures before choosing a policy. A temporary connection loss may be retryable; an unsupported schema version may require deployment; a permission failure may need operator action; an invalid business transition may be terminal. Retrying them all indefinitely destroys the distinction.

A bounded retry policy specifies attempts, elapsed-time budget, backoff with jitter, destination isolation and what happens at exhaustion. The retry must preserve event identity. Creating a new business event ID for every delivery attempt defeats duplicate detection and makes one logical operation appear to be many independent ones.

A DLQ or quarantine is a durable record of unresolved processing, not proof of success. Store original identity and source position, sanitized failure class, attempt metadata and a repair/replay decision. Keep payload access controlled; dead-letter topics often contain the most malformed and sensitive input in the system. Replay needs authorization and an audit trail, with a preview of which side effects will repeat.

Moving a poison record aside and advancing the main offset changes ordering semantics. If later events require that record's effect, they cannot safely continue just because the queue has somewhere to put it. For independent events a retry topic can preserve overall throughput; for ordered aggregates, park that key or partition and provide a recovery path.

Policy	Use when	Avoid when
Short in-place retry	A transient issue fits the poll/deadline budget	Long blocking causes consumer reassignment or stalls unrelated work
Delayed retry destination	Independent events can overtake the failed event	Strict aggregate ordering is assumed unchanged
Quarantine with repair	Bad input or missing support needs a human or deployment	It becomes a silent sink with no owner or age alert
Stop the affected ordered lane	Proceeding could corrupt an invariant	A global halt is used for isolated tenant failures

[VERIFY: validate Spring Kafka retry and dead-letter publication acknowledgement behavior, offset handling on failed recovery, and RabbitMQ dead-letter delivery safety for the chosen queue policies. A configured DLQ is not automatically an atomic transfer; no fault-injection test of either framework ran.]

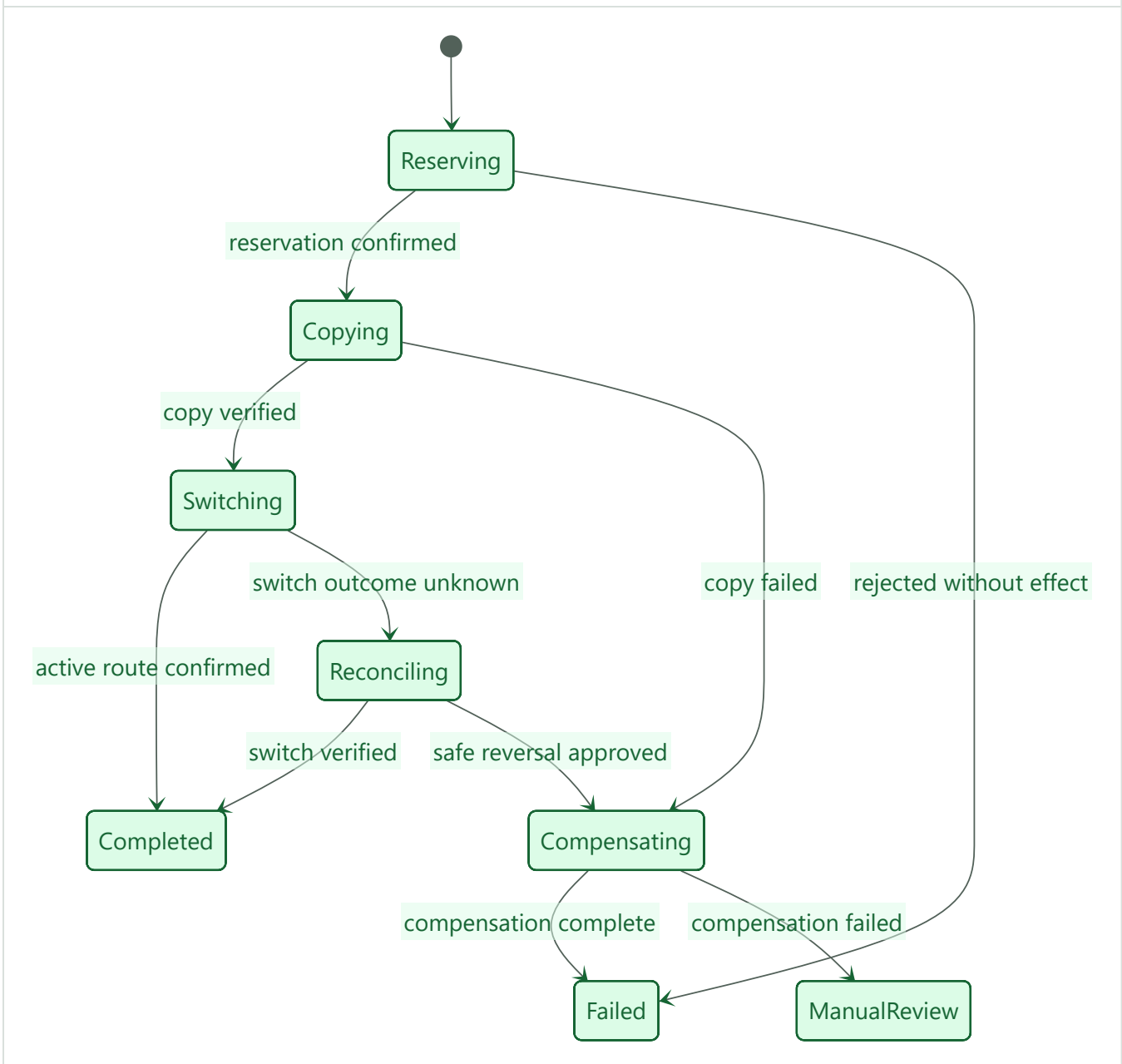
Interview checks: basic: DLQ means unresolved, not completed. Internals: the transfer to a recovery destination has a failure boundary. Debug: rapid requeue loops are often misclassified permanent failures. Scenario: replay only after defining ordering and duplicate effects.

8. Sagas: Local Transactions With Business Compensation

A saga coordinates a business operation across owners without pretending there is one database transaction. Each participant commits a local step and emits the result; the workflow continues or requests a compensating business action. Compensation is not time travel. Revoking a temporary reservation may undo its business meaning, but it does not unsend a notification or erase an audit record.

For a connector migration, IntegrationHub might reserve capacity, prepare a destination, copy data and switch active routing. Failure before switching may allow temporary capacity release and destination cleanup. Failure after switching may require a controlled reverse migration or manual intervention, not a blind delete. The workflow must define which steps are reversible and which are points of no simple return.

Diagram 07.6



An orchestrator owns durable workflow state, correlates replies, issues commands and schedules timeout/recovery. Choreography lets participants react to events without one central workflow component. Neither removes coordination complexity. Choreography distributes it across handlers; orchestration concentrates it in an explicit state machine with an operational owner.

Messages need saga ID, step identity and attempt correlation. A stale reply to an earlier attempt must not advance the current state. Step execution and compensation are themselves idempotent, and a timeout creates an unknown outcome until the participant can be queried or reconciled. Saga state changes and next commands should use local transactions and outbox publication just like ordinary domain changes.

Approach	Use when	Avoid when
Orchestration	Many steps, deadlines and compensation paths need visible ownership	The orchestrator takes over participants' internal domain rules
Choreography	Few reactions remain understandable and independently useful	Event chains hide cycles, deadlines and responsibility for completion
One local transaction	The invariant belongs to one owner and store	Services are split solely to demonstrate a saga

Sagas do not provide isolation across their whole lifetime. Other work can see intermediate states. Model reservations, pending status, escrow or semantic locks where needed; a compensation later does not prevent another transaction from acting on an intermediate fact now. [22](#) compares this with two-phase commit, while [27](#) gives money-specific examples.

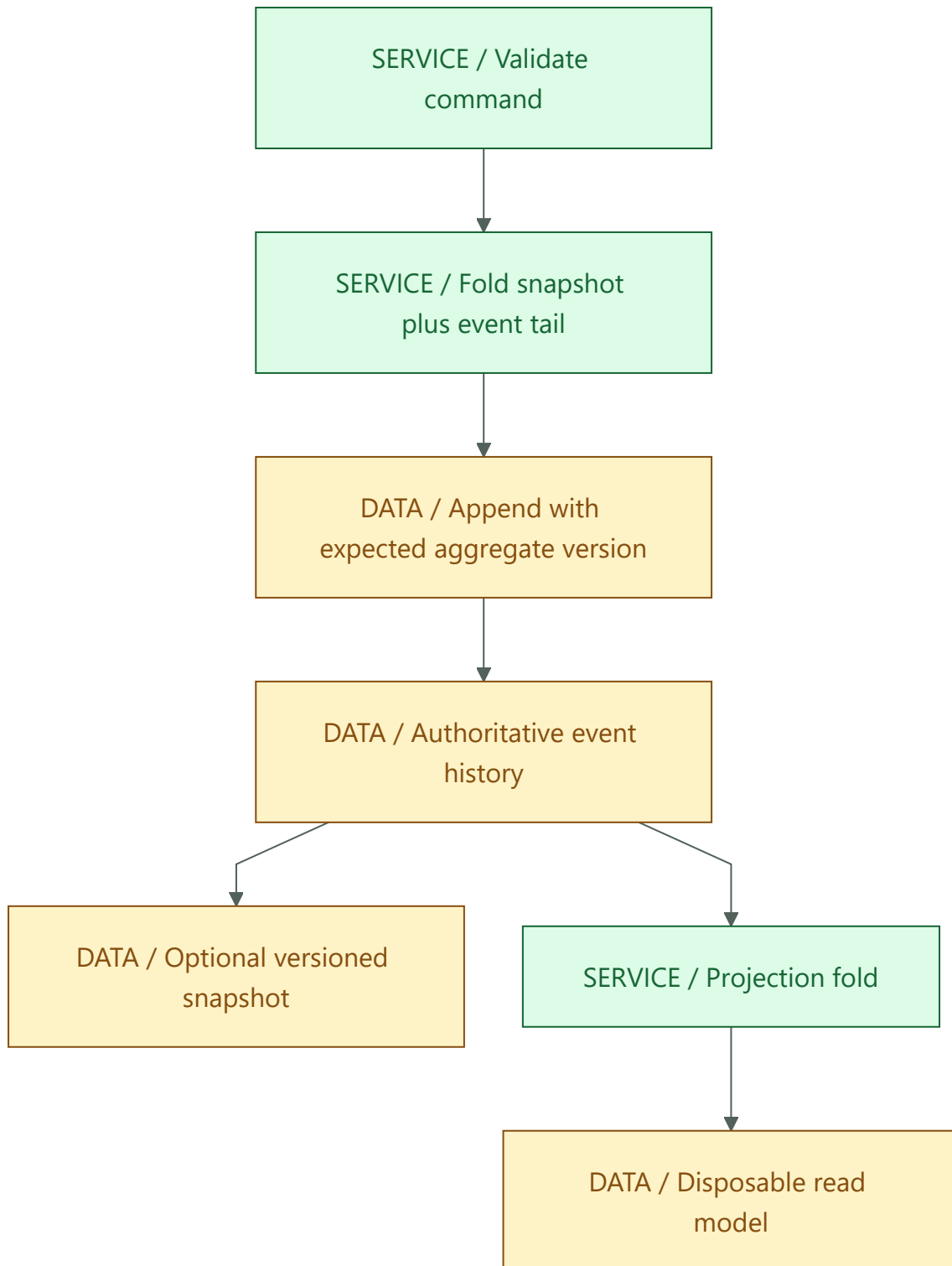
Interview checks: basic: compensation is a business operation. Internals: intermediate states are visible. Debug: a timed-out participant may still have succeeded. Scenario: durable reconciliation belongs in the state machine, not in an exception log.

9. Event Sourcing: History Becomes Authority

Ordinary event-driven services store current business state and publish selected changes. Event sourcing makes the accepted event history the authoritative representation. Current aggregate state is obtained by folding those events; a snapshot accelerates the fold but is not a replacement for history unless the retention contract deliberately changes.

For a connector configuration, a command handler loads the aggregate's events or a snapshot plus tail, checks the command against current state, derives new events and appends them with an expected aggregate version. The expected-version check prevents two commands that both read version 8 from both appending incompatible version-9 transitions. After one succeeds, the other must reload and re-evaluate; blindly retrying the append preserves the wrong decision.

Diagram 07.7



Events record accepted facts, not current implementation objects. Include aggregate identity, event identity, version and enough information to interpret the fact later. A historical amount or decision cannot depend on looking up today's exchange rate or today's mutable connector rules during replay. Persist relevant inputs or accepted results, and make the reconstruction rules explicit.

Schema evolution lasts as long as history. Removing an old event class does not remove the old bytes. Version readers, upcasters or explicit migration strategies must reconstruct old histories under new code. Test representative historical streams and snapshots; a test that creates all events with today's classes never exercises compatibility.

Reconstruction must not repeat external side effects. Folding EmailRequested should reconstruct the fact that a request existed, not send another email whenever an aggregate loads. Keep side-effect dispatch separate, with its own durable identity. Replaying a projection into a fresh table is different from executing a workflow against live providers.

An append-only log complicates data deletion and secret rotation. Avoid storing secrets and unnecessary personal data in long-lived immutable events; use controlled references and retention strategies. Whether deletion, anonymization or cryptographic erasure satisfies an obligation is a security/legal decision, not a guarantee supplied by event sourcing. 08 owns that review.

Choice	Use when	Avoid when
Event sourcing	Historical decisions, temporal reconstruction and auditability justify permanent event governance	Ordinary CRUD and audit records meet the requirement
Current state plus outbox	Transactional writes and integration events are enough	Complete history reconstruction is promised but transitions were never retained
Snapshots	Replaying long aggregate histories costs too much	Snapshot state/version is assumed compatible after arbitrary code changes

IntegrationHub does not event-source every job by default. A job table, progress events and audit trail are easier to query, repair and retain for the baseline. A later domain that truly needs reconstruction may make a different decision; event sourcing is a persistence choice, not a prerequisite for using Kafka.

Interview checks: basic: authoritative events distinguish event sourcing from event publication. Internals: expected-version append protects aggregate decisions. Debug: replay invoking provider calls reveals a mixed responsibility. Scenario: retain the simpler current-state model unless history is a first-class requirement.

10. CQRS and Projection Recovery

CQRS separates the write model from read models. It can be as small as different command and query objects over one database, or as large as independent materialized stores. It does not require separate services, asynchronous delivery, event sourcing or Kafka. Those are optional implementation choices with additional failure modes.

IntegrationHub writes jobs under transactional invariants but serves a search screen from a denormalized projection. A projector reads events, applies an idempotent update and checkpoints its progress. The projection is eventually consistent, so a just-created job may not appear immediately. The UI can use the write-side status resource until the projection reaches a known version, rather than claiming that missing search results mean failed acceptance.

Projection rebuild is a controlled migration. Choose the input history or a consistent snapshot plus change position, create a new destination, replay with bounded resources, catch up, compare invariants and switch read traffic. Keeping the old destination briefly makes rollback possible. Clearing the live index and hoping replay finishes before users notice is not an availability plan.

The input must be sufficient. A compacted topic containing only latest values cannot reconstruct every historical analytical result. An expired retention window cannot rebuild a projection that depends on lost deltas. A database snapshot taken at one position and a change stream resumed after a later position has a gap; resumed too early, it has overlap that needs idempotent application.

Read/write separation	Use when	Avoid when
Separate query DTOs over one store	Read shape differs but strong freshness and simple operation matter	Extra services are added with no distinct scale or ownership need
Asynchronous materialized view	Expensive reads and search can tolerate defined lag	The read decides a write invariant using stale projected state
Event-sourced aggregate plus projections	Full reconstruction is genuinely required	CQRS is used as an excuse to mandate event sourcing everywhere

Interview checks: basic: CQRS and event sourcing are independent. Internals: projection recovery needs a complete input and checkpoint contract. Debug: projection lag is not source data loss. Scenario: enforce command invariants at the write authority, not a stale search view.

11. Kafka Streams: Records, Tables and Topology

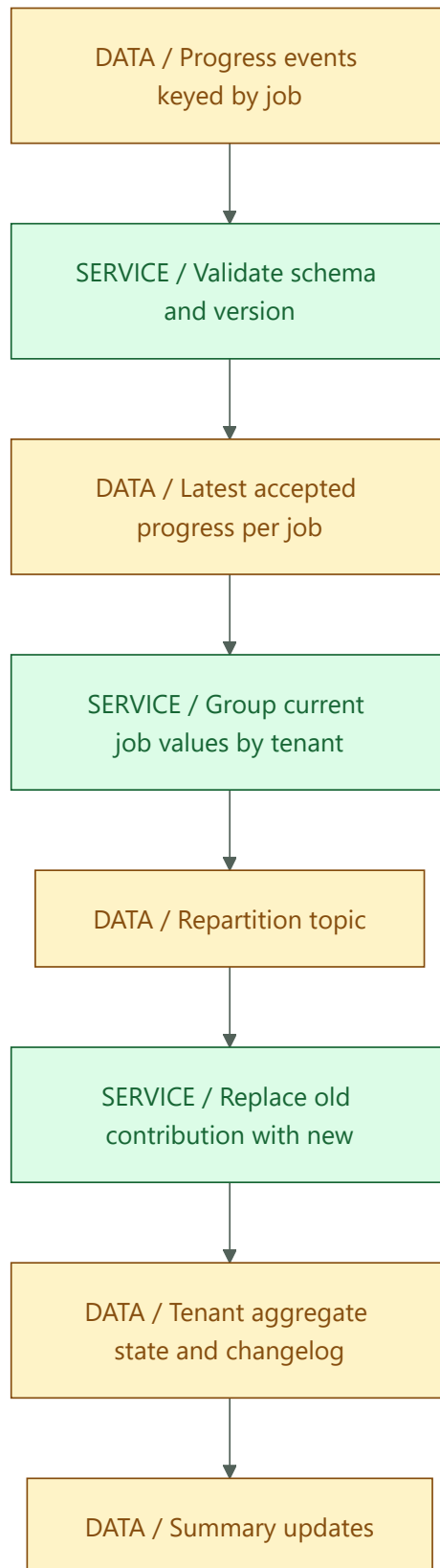
Kafka Streams is a library embedded in an application, not a separate broker-side computation engine. Instances sharing an application identity cooperate over partition-based tasks. The topology describes sources, transformations, stateful operations and sinks; deployment still needs CPU, memory, disk, monitoring and a shutdown/rebalance policy.

A **KStream** treats each record as an occurrence. If two records for a job report processed totals 10 and 20, summing those records yields 30, which is wrong when they were cumulative values. A **KTable** treats keyed updates as replacement state. The latest total becomes 20; downstream

table aggregates must account for replacement rather than counting every update as an independent fact.

Stateless operations such as filter and a value-only mapping need no remembered history for their result. Aggregates, deduplication, joins and windows need state. Changing a key is not itself sufficient to colocate records for later keyed computation: records must be redistributed so all values for the new key reach the correct task.

Diagram 07.8



The topology deliberately models cumulative progress as current keyed state. Updates that arrive with older application versions need an explicit policy before becoming the latest table value. A conventional unversioned KTable does not automatically know which business version is authoritative merely because a payload contains a version field.

Stream-table enrichment normally uses table state available under the selected join/store semantics when the stream event is processed. That is not automatically a historical as-of join against the configuration valid when the event occurred. Versioned stores and timestamp-aware joins can address particular cases, but need retained versions and tested late-update semantics. Never casually attach today's tenant classification to yesterday's billing events and call it historical truth.

Abstraction	Use when	Avoid when
KStream	Every record is an occurrence such as a completed item	Cumulative snapshots are summed as independent deltas
KTable	Each key represents replaceable current state	Every historical transition must be kept by the table alone
GlobalKTable	A reasonably small reference table should be available on every instance	Replicating a large dataset to every process is unaffordable
Stateless pipeline	Each record can be transformed independently	Hidden local mutable maps quietly create nonrecoverable state

Interview checks: basic: record occurrence and table replacement differ. Internals: grouping by a changed key requires repartitioning where locality is needed. Debug: totals doubling after progress updates often reflect the wrong stream/table interpretation. Scenario: define as-of enrichment before choosing a join.

12. Windows, Event Time and Late Data

Unbounded input needs a rule for when records belong together. Event time describes when something occurred at the source; processing time describes when this application handles it. Ingestion time records arrival at the broker. A delayed upload can have all three far apart. A dashboard about yesterday's failures should not silently move them into today's total merely because replay happened today.

Tumbling windows divide time into nonoverlapping intervals. Hopping windows overlap and can count one event in several intervals intentionally. Session windows group activity separated by an inactivity gap and may merge when a late event connects sessions. Sliding-window semantics depend on the operation and library; do not substitute a hand-waved timer for the documented boundary rules.

Kafka Streams uses record timestamps and data-driven stream time for window processing. A grace period allows bounded out-of-order arrival relative to window closure. It is not simply a wall-clock sleep after each record, and an idle input may not advance stream time. A far-future timestamp can prematurely make legitimate records appear late, so timestamp validation and monitoring are part of correctness.

Window policy	Use when	Avoid when
Tumbling	Disjoint reporting periods match the business question	A rolling interval is required instead
Hopping	Overlapping rolling summaries are deliberate	Summing all windows is assumed to count each event once
Session	Activity bursts define sessions better than fixed boundaries	Late merges cannot be represented in downstream updates
Larger grace	Delayed arrivals are expected and state cost is acceptable	Lower latency and bounded storage requirements are ignored

Outputs may be provisional updates, not final immutable facts. Downstream consumers need replacement semantics or versioned results. Suppressing intermediate results can wait for closure, but costs buffer capacity and depends on time advancing. If a record arrives after grace, choose whether it is discarded with metrics, quarantined for reconciliation or included by an offline correction process. A dropped late record cannot be recovered by declaring the processor exactly-once.

[VERIFY: confirm Kafka Streams window boundary, grace, suppression, stream-time advancement and versioned-store join behavior against the exact 4.x API and topology. The 4.0 core-concepts reference was consulted; no late-event or timestamp-skew experiment ran.]

Interview checks: basic: event time and processing time answer different questions. Internals: grace uses stream-time semantics, not an ordinary timer. Debug: future timestamps can make good records late. Scenario: communicate whether a window result is provisional or final.

13. State Stores, Repartitioning and Exactly-Once

A state store belongs to stream tasks and may use local persistent storage or memory. Changelog topics enable recovery when configured appropriately. Losing a pod can mean rebuilding local state from its changelog before useful processing resumes; that restoration time belongs in the recovery objective. A standby copy reduces some recovery work but consumes resources and is not a replacement for retained changelog data.

Repartition topics shuffle records after a key change so keyed operations execute with the required locality. They add network traffic, serialization, storage and another recovery boundary. Joining partitioned sources often needs compatible partitioning. A topology that appears to be a few map/group/join calls can create substantial internal traffic, so inspect its described topology and internal topics rather than counting lines of DSL.

The consulted Kafka Streams reference describes exactly-once processing as atomic progress across input offsets, Kafka outputs and recoverable state-store updates. Configure the supported transaction mode, such as `exactly_once_v2` in the referenced generation, and ensure downstream readers honor committed records. Aborted processing can physically run again while only one committed result remains visible under the defined model.

An HTTP request performed inside a processing callback is not rolled back by aborting a Kafka transaction. Neither is an unrelated JDBC commit. Integrate such effects through a separate idempotent consumer/outbox boundary or a sink-specific coordination contract. The transactional guarantee is valuable precisely because its scope is concrete; extending it rhetorically to every side effect makes the design wrong.

[!] TRAP

Exactly-once is not a repair for duplicate business input. Two separately committed source records with different identities can both legitimately be processed once. If they represent the same business intent, deduplication must use that intent's identity; broker transactions cannot infer it.

State/guarantee choice	Use when	Avoid when
Recoverable local store	Low-latency keyed state belongs to partitioned tasks	The local disk is treated as the only durable copy
Kafka transaction mode	Kafka input/state/output atomicity matters	HTTP or database effects are assumed included
Stateless recomputation	Transformation is cheap and history retained	Aggregation state is discarded without a rebuild budget
Separate external-effect consumer	External systems need their own retry/deduplication contract	A callback performs non-idempotent side effects during replay

Changing application identity, processor names or state schemas can change internal-topic ownership and restore behavior. Deploy topology migrations as data migrations with a compatibility and rollback plan. A reset utility run against a live application can trigger broad reprocessing; it is not a harmless way to clear an error.

[VERIFY: validate processing.guarantee, transactional IDs/fencing, read_committed consumers, state changelog configuration, standby recovery and topology migration compatibility against the selected Kafka Streams release. Documentation supports the Kafka-scoped guarantee; broker transactions and restore behavior were not tested locally.]

Interview checks: basic: state stores need restoration, not just disk. Internals: repartitioning changes record locality. Debug: a pod that is healthy but restoring state may contribute no processing capacity yet. Scenario: external effects require a separate correctness boundary.

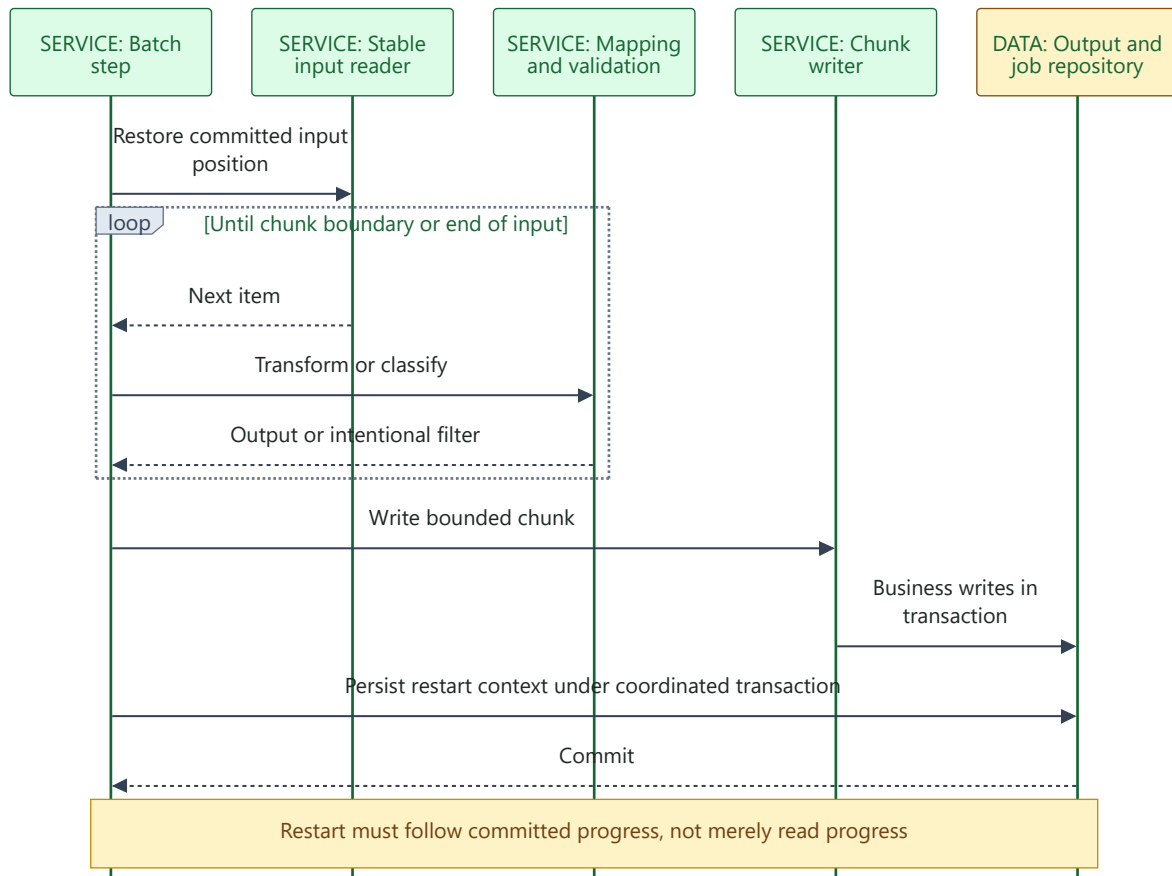
14. Spring Batch: Job Identity and Chunk Transactions

Spring Batch makes finite processing operationally explicit. A Job defines a workflow, Steps define its phases, and execution records describe attempts. A JobInstance identifies a logical run through job name and identifying parameters; a JobExecution is one attempt at that instance. Restarting a failed instance creates another execution, not a new business input set.

A timestamp added as an identifying parameter on every launch creates a new instance. That can be useful for intentionally distinct runs, but it is not a way to restart the failed file import from yesterday. Use stable identity such as tenant, immutable object version and mapping version when those define the input contract. Job identity does not select the rows by itself; the reader must actually enforce that selection.

In chunk processing, an ItemReader supplies items, an optional ItemProcessor transforms or filters them, and an ItemWriter writes a chunk. At a commit boundary, business output and restart metadata must be coordinated under the chosen transaction arrangement. A Tasklet step can perform other work without fitting this item-oriented pattern; not every Step necessarily has a reader/processor/writer trio.

Diagram 07.9



The JobRepository persists execution identity, statuses and checkpoint metadata. An ExecutionContext stores restartable state for a job or step. Store compact, serializable positions and input fingerprints, not a whole transient object graph or a live database connection. A reader that claims restart support must reopen the same input and interpret its saved position consistently.

Larger chunks can amortize commit overhead but increase memory, lock duration and rollback/replay cost. Smaller chunks reduce rework but add transaction overhead. Filtering an item intentionally is not the same as skipping an exception; record those counts separately because they answer different reconciliation questions.

Batch decision	Use when	Avoid when
Stable identifying parameters	A failed attempt must restart the same logical input	Every retry changes identity and begins again
Chunk-oriented step	Bounded item transformations fit repeated commits	One indivisible operation is artificially split into misleading items
Tasklet step	A phase is a controlled operation such as preparing a manifest	Unbounded work hides inside one enormous transaction

Batch decision	Use when	Avoid when
Persistent JobRepository	Restart and operational history must survive processes	In-memory metadata is presented as durable recovery

[VERIFY: check Spring Batch 6.x job/repository configuration, builders, transaction managers, execution-context serialization and restart APIs against the Boot-managed version. The current domain reference was consulted; DeliveryLab is not a Spring Batch execution and no Batch schema was created.]

Interview checks: basic: instance is logical identity, execution is an attempt. Internals: a chunk checkpoint describes committed progress. Debug: changing identifying parameters can defeat restart. Scenario: verify input identity along with the saved cursor.

15. Restartability, Skip/Retry and Partitioning

Restart is a protocol between input, output and metadata, not a boolean on a job definition. Suppose a writer commits output in one database and the JobRepository records position in another. A crash between those commits can repeat output or skip it depending on ordering. Use a coordinated transaction where supported, or idempotent output and a recovery design that explicitly tolerates overlap. Framework metadata alone cannot make two resources atomic.

A stable file reader needs the immutable file identity and position. A database reader needs a stable selection, ordering and cursor, not offset pagination over a table changing during the run. An API reader needs a provider-supported cursor or snapshot contract and an answer for expired tokens. A job that restarts at line 20,000 in a changed file has not restarted the same job.

Retry transient failures with bounded attempts and elapsed time. Re-run the whole affected transaction after rollback when required; retrying one statement inside an already failed transaction is not generally sufficient. External writes may already have occurred, so a retryable writer must use their idempotency protocol. Business validation failures ordinarily belong in quarantine or explicit rejection, not transient retry.

Skip is a business decision: which records may be omitted, who approves the threshold, and how are missing effects reconciled? A pipeline that completes after skipping half a tenant's data should not show an unqualified successful sync. Preserve sanitized error records with source identity, make skip thresholds explicit, and surface a completed-with-errors result if the contract permits partial completion.

Partitioning splits finite input into disjoint ranges or manifests, each with its own checkpoint. A coordinator tracks completion of those partitions. Parallelism is bounded by the database, provider quota, connection pool and repository contention. Do not share a stateful non-thread-safe reader across workers merely because it is a Spring bean. Distinguish partitioning input from remote chunking, where a manager distributes chunks to remote writers and must handle delivery and completion coordination.

Recovery choice	Use when	Avoid when
Retry	Failure is transient and repeated effects are safe	Validation defects or unknown external outcomes are blindly repeated
Skip plus quarantine	Product rules permit partial completion with reconciliation	A green status conceals missing data
Fail and resume	Further work is unsafe or a threshold was exceeded	Operators must infer the failed input from logs alone
Disjoint partitions	Work is independently addressable with durable per-partition progress	Ranges overlap or a shared cursor races across threads

[VERIFY: verify Spring Batch retry/skip defaults, rollback classification, reader thread safety, partition restart behavior and JobRepository/business transaction coordination for the selected release and resources. No framework fault-injection, remote partitioning or API-cursor restart ran.]

Interview checks: basic: restart needs stable input. Internals: output and checkpoint must agree across crashes. Debug: missing rows can be a moving input set, not a bad counter. Scenario: partial success requires an explicit data-quality contract.

16. IntegrationHub Operations and Lab Review

A useful messaging dashboard follows work age through the whole pipeline. Monitor the oldest unpublished outbox item, relay errors, consumer lag by partition, age of oldest unfinished business work, retry/quarantine age, state-store restoration, and batch checkpoint advancement. Low broker lag with a full application executor means work moved into memory, not that it completed. Count accepted, completed, rejected and unresolved work under a reconcilable identity model.

One Kafka consumer group is not a broadcast mechanism to every SSE node. In the baseline, worker events update durable progress, then a live backplane routes notifications to nodes with interested subscribers. Redis Pub/Sub can serve ephemeral live fan-out but supplies no durable replay. A dropped notification is recovered from the durable progress cursor as 05 specifies. Sticky routing may reduce routing complexity but does not survive a node loss or provide missing history.

[D] DECISION

Separate work distribution from notification fan-out. A worker group divides durable processing. Browser nodes need notifications for their local subscribers, possibly on several nodes for one job. Use routing or broadcast deliberately, and keep recovery in the durable store.

Trace context crosses publication and consumption as explicit metadata; request-thread context does not automatically follow a record through a broker. Use safe correlation IDs, bounded metric labels and sanitized failures. Event IDs and tenant IDs are useful for restricted diagnostics but can create unbounded time series if used as every metric's label. 18 owns the full telemetry design.

DeliveryLab: Four Arranged Failures

The complete saved source is `code/07-messaging/DeliveryLab.java`. The Windows PowerShell runner is `& './docs/java-fs-guide/code/07-messaging/run.ps1'`, from the workspace root; it requires no administrator rights and performs no download. It accepts `-JdkHome` for an existing Java 21-or-later installation. The actual compiler preflight reported NOT EXECUTED.

The model copies candidate receipts and totals, then replaces one immutable State under synchronization. Throwing before that replacement leaves receipts, totals and checkpoint unchanged; throwing after it models a committed effect with a lost acknowledgement. This is a local teaching commit point, not disk persistence, SQL isolation or a broker transaction. The process does not actually crash, and none of its assertions has been evaluated here.

Probe	Intended assertion	Not demonstrated
Failure before local commit	Receipt, effect and checkpoint all remain absent; retry applies once	PostgreSQL rollback or JDBC transaction wiring
Failure after commit, before ack	Broker position is unchanged; redelivery does not repeat the effect	Kafka offset commit, RabbitMQ redelivery or process recovery
Scope and content binding	Tenant and consumer identities differ; changed content is rejected without advancing	Authorization, cryptographic digests or distributed uniqueness
Chunk restart	Failed chunk leaves old position; committed prefix selects remaining input	Spring Batch metadata, reader reopening or two-database coordination

The single checkpoint models one ordered lane. It is not a vector of offsets for multiple partitions, and its callers intentionally supply monotonic positions. Do not generalize the toy API into production code without lifecycle, retention, validation, storage and concurrency tests.

The complete state copies also make it unsuitable as a large in-memory store; their purpose is to expose the commit boundary clearly.

Real validation should kill processes before and after database commits, disconnect broker acknowledgements, force a rebalance during slow processing, replay duplicate identities concurrently against a unique constraint, fail dead-letter publication, restore stream state, and restart a batch against immutable input. Assert durable effects, not merely handler invocation counts. 19 owns that environment once dependencies are available.

[?] INTERVIEW

A precise answer names five things: event identity, ordering scope, commit point, acknowledgement point and recovery owner. If any one is missing, a claim about reliable messaging is probably hiding its hardest failure case.

Explained-Here Index and Sources

Concept	Explanation
Commands, events, batch and streaming	Processing models
Kafka partitions, groups, offsets and replication	Kafka mechanics
RabbitMQ routing, confirms, ack and prefetch	Queue mechanics
At-most-once, at-least-once and effect guarantees	Crash boundaries
Idempotent consumer and business ordering	Receipts and versions
Outbox, CDC and Debezium	Durable publication
DLQ, retry and replay	Recovery policies
Saga and compensation	Workflow state
Event sourcing	Authoritative history
CQRS	Read models and rebuilding
Kafka Streams, KStream and KTable	Topology semantics
Windowing and late data	Time semantics
State store, repartitioning and exactly-once	State and transactions
Spring Batch, chunk and job repository	Batch identity
Restartability, skip/retry and partitioning	Batch recovery

Consulted on 2026-10-06: [Kafka 4.0 Streams core concepts and processing guarantees](#), [RabbitMQ consumer acknowledgements and publisher confirms](#), [Debezium outbox event router](#), and [Spring Batch domain language](#). The old Kafka documentation fragment returned a redirect script, not substantive semantics text; the versioned Streams page supplied the transaction-scope evidence. Documentation observations are distinct from unexecuted integration tests, and moving references must be pinned before implementation.

Related Chapters

Return to [00's request journey](#). Use [02 concurrency](#), [03 transactions](#), [04 persistence](#), [05 APIs](#) and [SSE](#) and [06 durable state](#) for the underlying boundaries. Continue with [08 security](#), [13 integration](#), [18 operations](#), [19 tests](#), [22 distributed systems](#), [26 analytics](#) and [27 payments](#). Generated Related chapters navigation supplies reciprocal links.

RELATED CHAPTERS / RECIPROCAL LINKS	
00 The System Before the Details	27 Fintech and Payments Engineering
01 Core Java and the JVM	19 Testing and Quality
02 Concurrency and the Memory Model	09 The Browser and the Java Backend
03 Spring and Spring Boot Internals	10 System Design Foundations
04 Hibernate and JPA	25 Architecture Styles and Domain Design
06 Databases and Durable State	11 Object Design, Patterns and LLD
05 APIs and Real-Time Communication	12 Twelve System Design Case Studies
13 Data Integration and iPaaS	14 Docker and Container Internals
26 Data Platform Basics	15 Kubernetes and Production Workloads
18 Observability and Reliability	17 Cloud and Infrastructure
08 Security, Identity and Data Protection	24 IntegrationHub in Motion
22 Distributed Systems Theory	20 Appendix: Interview Toolkit and Index

One-Page Cheat Sheet

Identity and order: event ID is business identity; Kafka offset is a partition position; RabbitMQ delivery tag is channel-scoped. A partition key alone does not guarantee application-version order or ordered effects after concurrent handling.

Commit before ack: atomically commit local effect, scoped receipt and progress; acknowledge afterward. A crash in the gap permits redelivery, so keep it idempotent. Never advance offsets beyond unfinished earlier work.

Publication: job plus outbox is one local transaction. Polling or CDC publishes it later, with possible duplicates. CDC over an outbox preserves domain intent; raw table CDC exposes storage changes. Monitor connector/slot retention and oldest unpublished work.

Recovery: retries preserve identity and have budgets. A DLQ is unresolved work with an owner. Moving a record aside can violate ordering. External uncertain effects need provider idempotency or reconciliation.

Architecture: sagas compensate local commits but do not roll back time or isolate intermediate state. Event sourcing makes events authoritative; CQRS separates write and read models. Neither is mandatory for ordinary messaging.

Streams: KStream records occurrences; KTable replaces keyed state. Repartitioning establishes locality. Windows require event-time, grace and late-data policy. State restoration needs retained changelogs. Kafka exactly-once covers its defined input/state/output boundary, not arbitrary HTTP/JDBC effects.

Batch: job plus identifying parameters defines an instance; executions are attempts. Commit output with checkpoints or provide safe overlap recovery. Stable input, explicit skip/retry policy and disjoint partitions make restart meaningful. The Java lab remains NOT EXECUTED.

Interview Corner

Basic: Is Kafka a Queue?

It can distribute work, but its central model is a retained partition log. Consumer groups own independent positions and consumption does not delete records. Compare that with acknowledged queue deliveries, then discuss the actual product features required rather than relying on the word queue.

Internals: What Does a Producer Acknowledgement Prove?

Broker acceptance under the configured durability policy, not that a consumer completed a business action. For Kafka, explain the leader, ISR and minimum-ISR settings. For RabbitMQ, distinguish confirms, routing returns and consumer acknowledgements.

Trace/Debug: Why Did Adding Consumers Not Reduce Lag?

Check partition count, skew, ownership, restore/rebalance time and downstream capacity. Extra consumers may be idle, or one hot partition may dominate. More workers cannot increase a provider's quota or database capacity.

Scenario: How Do Three Independent Services Read Every Event?

Use distinct Kafka consumer groups for the three functions, with stable identities and their own offsets. In a queue model, route to separate queues. Multiple consumers in one competing group or queue divide work rather than broadcast it.

Basic: Why Can At-Least-Once Delivery Still Be Correct?

Because delivery and effect are different counts. Persisting a receipt with the business effect lets repeated deliveries observe the same accepted result. Name the retention horizon and local authority; an in-memory set is insufficient after restart.

Internals: Why Must the Receipt Share the Business Transaction?

A separately committed receipt can suppress a retry after the effect rolled back. A receipt committed after a separate effect leaves a duplicate-effect window. One transaction with a uniqueness constraint closes both local failure cases.

Trace/Debug: Offset 12 Finished Before 11. What Can Be Committed?

Only the resume position after the contiguous completed prefix. Committing 13 while 11 remains unfinished can lose 11 after a restart. Track completion per partition or retain ordered processing.

Scenario: Can a Kafka Transaction Cover a Provider HTTP Call?

No. The provider owns that effect and does not join the Kafka transaction. Use a stable provider request identity, durable attempt state and reconciliation for unknown outcomes. Reprocessing the Kafka callback may repeat the call.

Basic: What Does the Outbox Fix?

It atomically stores publication intent with business state in one database. A later relay can recover that intent after a crash. It does not make publication single-attempt or remove consumer duplicates.

Internals: Why Use CDC Over an Outbox Instead of Every Business Table?

The explicit outbox preserves domain event types and safe payloads while CDC supplies committed-log extraction. Raw table CDC couples consumers to storage changes and may omit business meaning. Both are useful for different contracts.

Trace/Debug: Why Is a Stopped Connector Filling Primary Disk?

A replication slot or equivalent retained position can prevent old log removal. Inspect connector offsets and retained log age before deleting anything. Resetting the position may require resnapshot and downstream reconciliation.

Scenario: How Do You Replay a Dead-Lettered Event?

Fix or classify the cause, authorize the replay, preserve logical identity, check ordering dependencies and verify duplicate safety. Record who replayed it and the outcome. A blind replay-all button can repeat side effects or reintroduce the same failure storm.

Basic: Is Saga Compensation a Rollback?

No. It is a new business action after earlier local commits became visible. Some actions are reversible, others need remediation or manual review. Compensation can fail and must have durable recovery state too.

Internals: Are Event Sourcing and CQRS the Same?

No. Event sourcing chooses authoritative event history for writes; CQRS separates command and query models. Either can exist without the other, and CQRS can remain within one database and process.

Trace/Debug: Why Did Rebuilding a Projection Send Emails Again?

Replay mixed deterministic state reconstruction with external side effects. Keep projection folds free of such actions and dispatch effects through an independently idempotent protocol. Rebuilding a read model must not impersonate new business intent.

Scenario: When Would You Reject Event Sourcing?

When current state plus an audit trail satisfies requirements and the team cannot justify permanent event-version governance, reconstruction testing and data-retention complexity. Kafka adoption alone is not a reason to event-source a domain.

Basic: KStream Versus KTable?

A KStream treats records as occurrences; a KTable treats updates as replacements for a key. Cumulative progress of 10 then 20 is 20 in the current-state interpretation, not a sum of 30. Decide the meaning before selecting operators.

Internals: Why Does a Grouping Operation Need a Repartition Topic?

After changing keys, related records may still live on different original partitions. Stateful keyed computation requires colocating them. Repartitioning performs that shuffle and adds storage, network and recovery cost.

Trace/Debug: Why Are Valid Events Suddenly Too Late?

Inspect timestamp extraction, malformed future timestamps, stream-time advancement and grace. A replay or skewed timestamp can change when windows close. Increasing wall-clock wait does not necessarily repair event-time semantics.

Scenario: What Does Kafka Streams Exactly-Once Exclude?

It excludes arbitrary external database and HTTP effects, invalid business input, and records deliberately dropped by a lateness policy. Its value is atomic committed Kafka input progress, state recovery and outputs under the configured transaction model.

Basic: JobInstance Versus JobExecution?

The instance identifies one logical job through its name and identifying parameters. Each execution is an attempt, possibly failed and restarted. Changing identifying parameters can create a new instance rather than resume an old one.

Internals: What Makes a Chunk Restartable?

Stable input identity and position, repeatable reader behavior, safe output and a checkpoint that agrees with committed effects. Framework metadata does not automatically coordinate two independent databases or a remote API.

Trace/Debug: Why Did a Restart Miss Source Rows?

Check whether the file changed, the query's offset window moved, the cursor expired, partitions overlapped, or checkpoint advanced before output committed. A saved numeric position is meaningful only against the same input contract.

Scenario: Should One Invalid Row Fail the Entire Import?

That is a business policy, not a universal framework setting. If partial completion is permitted, quarantine with source identity, bounded skip thresholds and visible partial status. Otherwise fail and resume after repair; never hide omissions behind an unqualified success.

26 / Data Platform Basics

Operational truth and analytical usefulness are different contracts. A sync service answers whether one job was accepted. An analytical system answers which connector types failed most often last month, under the definitions and dimensions valid for that report. IntegrationHub is fictional; its analytical platform is a derived view, never the authority for job acceptance or a wallet balance.

Version assumptions: Java 21, PostgreSQL 17 and Kafka 4.x are the guide's reference environment, not locally tested dependencies. Warehouse and table-format features are engine-specific. No Spring Cloud behavior is introduced here; retain the compatibility checks in 03.

Execution status: `code/26-data-platform/WarehouseLab.mjs` has five recorded PASS checks on Node v24.20.0. It models dimensions and facts in memory. No SQL, warehouse, CDC connector, object store, orchestration engine or BI query was executed. All deployment workflows below are design exercises, not command transcripts.

Big Picture

Diagram 26.1



What You Will Be Able to Explain

- Separate transaction-serving workloads from analytical scans without confusing replication with a data platform.
- Choose warehouse, lake or lakehouse from governance and query requirements.
- Trace a change from snapshot to CDC, replay, transformation and publication.
- Declare a fact grain before adding measures and dimension joins.
- Handle late events, historical dimensions, deletes and backfills deliberately.
- Define data quality, lineage and an owner for every published metric.

Where this lives in IntegrationHub. Job outcomes feed tenant reliability reports and connector adoption analysis. PostgreSQL remains authoritative for operational jobs (06); Kafka transports changes (07); connectors own mapping and quarantine (13). Reporting never reads secrets or trusts a tenant filter supplied by the browser. Security and retention belong to 08, and billing reconciliation belongs to 27.

1. OLTP and OLAP: Start with the Question

OLTP serves short transactions over operational entities: create a sync job, claim a batch, update a checkpoint. Useful work is usually selective, indexed and latency-sensitive. Transactions enforce invariants over concurrent changes. **OLAP** serves analytical questions over many records: failure rates by connector, day and subscription tier, or a trend over a year. Scans, grouping, joins and column pruning matter more than updating a single row quickly.

These are workload descriptions, not hard product categories. A relational database can serve both at modest scale. A separate analytical system is justified when large scans, long snapshots or arbitrary BI queries compete with request traffic, or when multiple source systems need shared historical definitions. Moving the query to a read replica can isolate some compute, but replica lag, history, schema coupling and replica-side query costs still require decisions. It does not automatically produce a governed model.

Trace the interference: a dashboard scans a large table, consumes CPU and I/O, evicts useful pages, and holds resources while users wait for operational requests. Adding an index may help one report but increases write and maintenance costs. Separate the workload only after defining its freshness requirement; a daily report rarely needs every row to appear within a second.

Choice	Use when	Avoid when
Indexed queries in OLTP	Small bounded reports fit the operational resource budget	Arbitrary historical scans can exhaust request capacity
Read replica for reports	Same schema and some lag are acceptable	You need independent history, cross-source semantics or guaranteed current balances
Analytical store	Repeated large scans, governed metrics and historical joins justify a pipeline	A simple report would acquire unnecessary ingestion and on-call machinery

IntegrationHub decision: operational progress is served by the job API, not yesterday's reporting mart. A chart says its data cutoff and freshness status. An empty result during ingestion failure must not be labeled zero failures. The SLI discussion in 18 applies: missing

evidence differs from a healthy result.

2. Warehouse, Lake and Lakehouse

A **warehouse** provides managed analytical tables and a query environment organized for business consumption. A **data lake** stores files or objects, often retaining source-shaped data for multiple downstream uses. A **lakehouse** combines lake-style storage with table-management features such as transactional metadata and schema evolution. None of these labels promises correct definitions, low cost or access isolation.

Mechanically, an analytical query needs to discover the right data files or table partitions, select columns, scan relevant data and aggregate it. In a lake, unmanaged file listings alone give little protection against partial loads, duplicate files or conflicting writers. A table format can introduce snapshots and metadata that identify a consistent table state. A query engine still has to support the format and its chosen features. Object storage is not itself a SQL transaction coordinator.

Storage approach	Use when	Avoid when
Warehouse	Teams need governed SQL access and managed analytical operations	Its access patterns or economics do not fit required raw-data uses
Lake	Diverse retained inputs need replay and several consumers	No owner will manage catalog, permissions, file layout and cleanup
Lakehouse	Shared object data needs table snapshots and compatible query engines	The team assumes a format removes operational work or universalizes feature support

[VERIFY: check the selected warehouse, object store, table format and query engine for snapshot isolation, concurrent writes, schema evolution, delete support and catalog interoperability. No warehouse or lakehouse implementation was executed.]

IntegrationHub can start with restricted landing objects and a small warehouse mart. It does not need three different platforms because three names appear in an interview. Separate landing access from curated access: source-shaped payloads may include attributes that reports must never expose. Encryption alone does not implement that separation.

3. ETL, ELT and Streaming Are Different Axes

ETL transforms before loading the analytical destination. **ELT** loads first and transforms there. **Streaming** describes continuous processing rather than a scheduled bounded run; either transformation placement can be used with continuous ingestion. A scheduled micro-batch

and a record-at-a-time processor can produce the same business table under different latency and operating costs.

An ETL flow reads a source manifest, validates records, normalizes units and types, writes a staged result, then publishes a completed load. It can exclude sensitive fields before they reach the destination. An ELT flow lands a controlled source copy, executes versioned transformations near the data, checks quality, then exposes curated tables. It makes reprocessing convenient only when the source copy, transformation version and permissions are actually retained.

Choice	Use when	Avoid when
ETL	Data must be minimized or transformed before the destination	Every change would require rereading an unavailable source
ELT	Governed landing data enables repeatable warehouse transformations	Raw sensitive data cannot be permitted in the destination
Scheduled batch	Minutes or hours of freshness meet the requirement	Decisions genuinely require lower-latency change visibility
Streaming	Continuous updates have explicit business value	The team cannot support replay, ordering, lag and evolving state

[D] **DECISION**

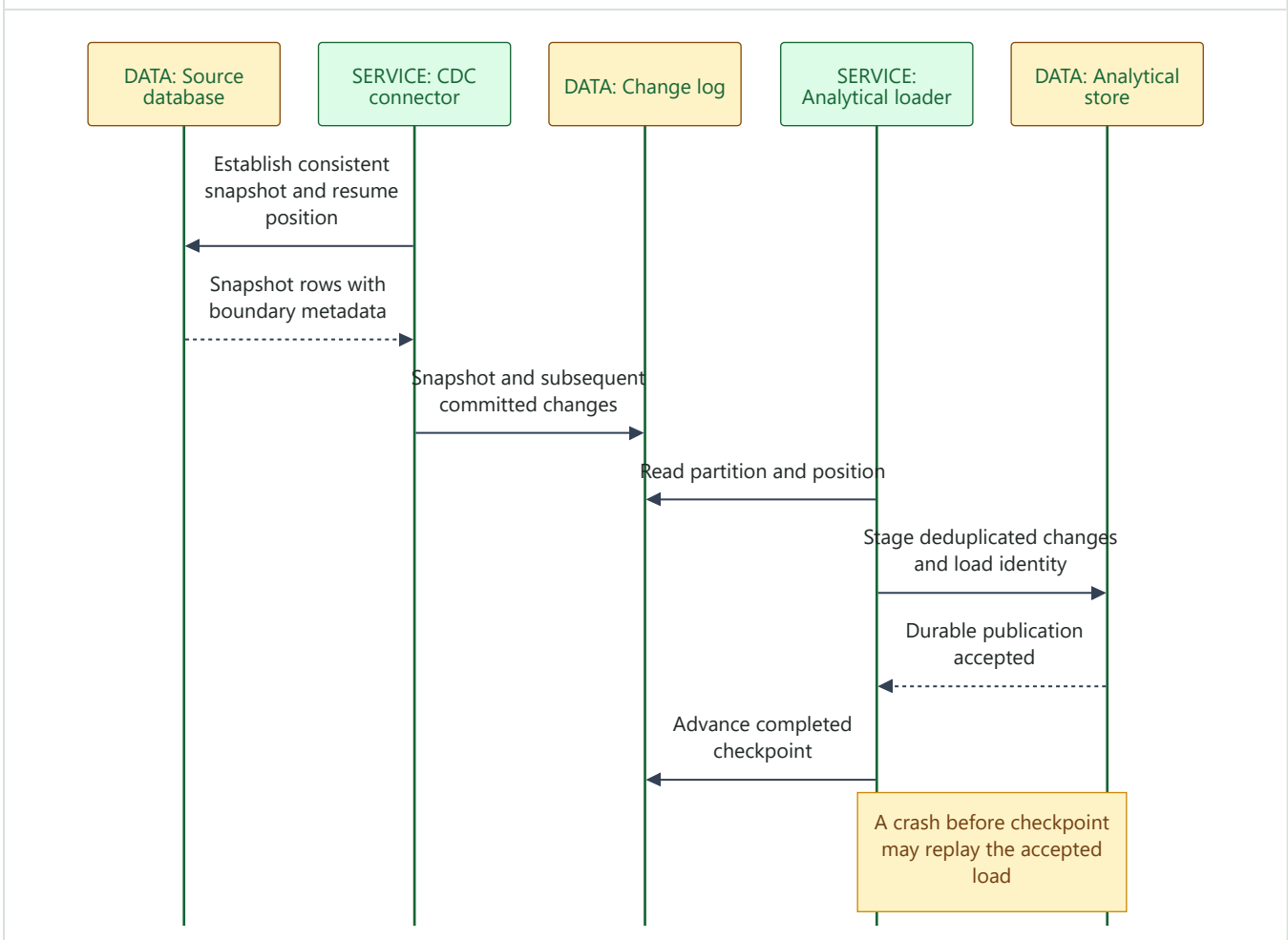
Ask for freshness before choosing technology. IntegrationHub's daily adoption report can be a bounded batch. A near-live operational anomaly feed may justify streaming. Billing close needs completeness and reconciliation, not merely a short ingestion delay. One dataset can support all three through separate published contracts.

For either mode, pin input identity, transformation version and output destination. Write into a staging generation, validate, then switch the published pointer or commit the final table operation. Readers must not see half a daily partition. Restartability is not "run yesterday again"; it means repeating the same logical input does not duplicate its accepted output. See the checkpoint mechanism in [13](#).

4. CDC into Analytics: Snapshot, Changes and Recovery

Analytics CDC turns committed source changes into analytical updates. Log-based CDC can avoid repeated full-table polling, but still needs a consistent starting snapshot, a source position and enough retained log history to resume. The source's log position is not automatically comparable with another source's position or a different Kafka partition offset.

Diagram 26.2



Trace a source update: the source transaction commits; capture reads its change record; transport can redeliver; the loader identifies the entity, source epoch and version; an idempotent merge or batch identity avoids repeated effects; only then does its progress advance. The source database transaction, broker offset commit and warehouse commit are not automatically one transaction. A crash in the gap is expected, so replay must be safe.

For a current-state projection, newer entity versions can replace older ones and old arrivals can be ignored. For a historical change log, discarding an older arrival may lose required history. Store the event occurrence separately if the business needs every transition. An update setting total processed rows to 1,200 is replacement state, not an increment of 1,200; summing every version invents throughput.

[VERIFY: confirm Debezium and the selected database connector's consistent snapshot, source-position, transaction metadata, tombstone, schema-history and recovery semantics for the deployed versions. No CDC snapshot, log-retention or restart test ran.]

Deletes need a contract. A deleted source row may close a dimension's current version, remove a current-state projection, or trigger a governed erasure process. A broker tombstone is a transport representation, not a complete deletion of landing files, historical tables and backups. Retention and privacy decisions must cover every copy, as described in [08](#).

[!] TRAP

A healthy consumer is not evidence of complete data. It may be consuming the wrong topic, silently quarantining a column, ignoring a schema change or starting after a lost log segment. Monitor coverage, rejected records, source-to-target control totals and last complete watermark, not only process uptime.

[?] INTERVIEW

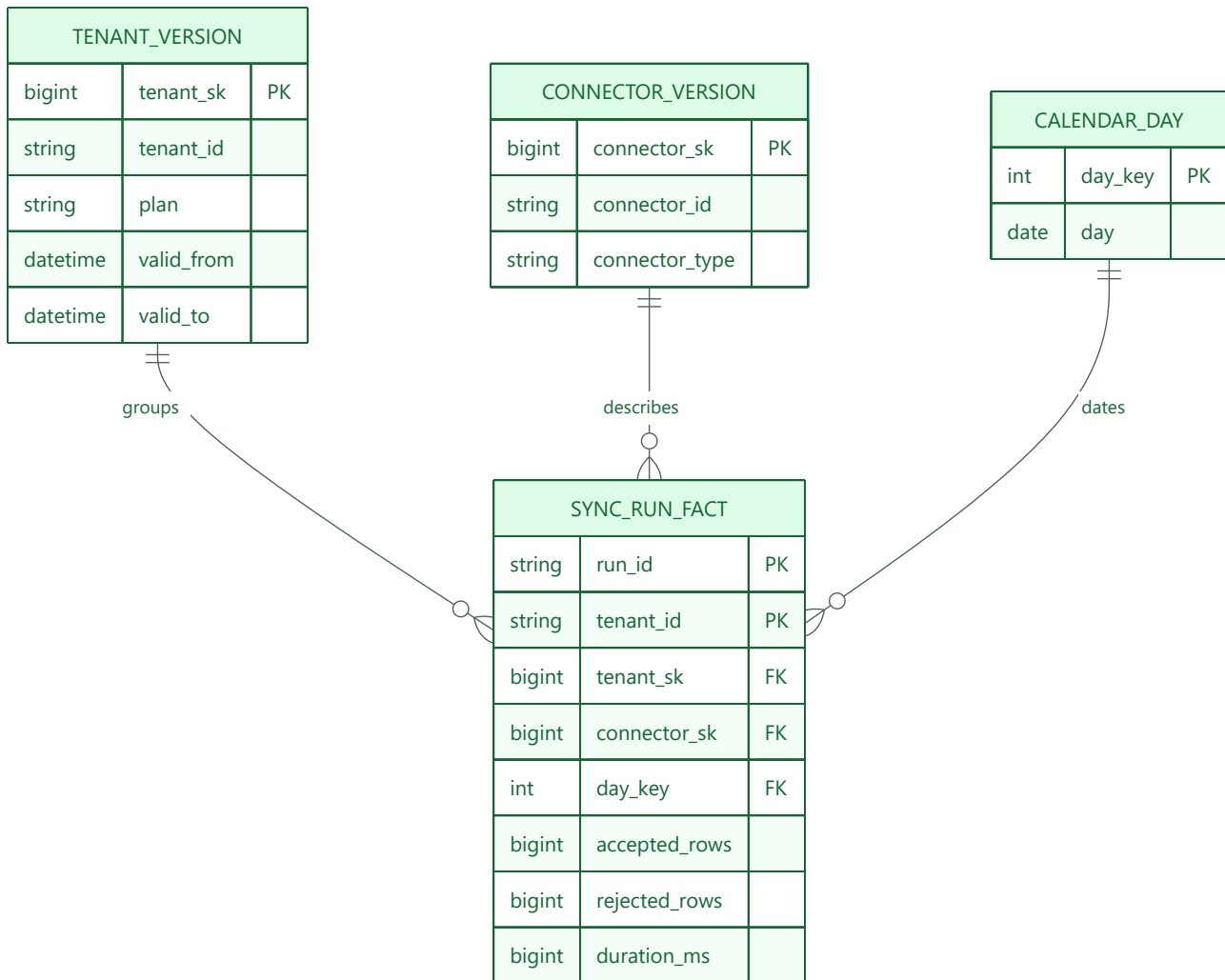
Name the grain before the tool. Explain the business row, its unique identity, measure units, freshness target and replay behavior before selecting a warehouse or streaming product.

If capture falls behind retained source logs, do not invent continuity. Stop claiming completeness, establish a new snapshot generation, reconcile it with the retained analytical history and explicitly hand over to the new stream. Rebuilds need an epoch or namespace so positions from an old stream cannot reject legitimate new input.

5. Star Schema: Declare the Grain First

A **fact table** records measurements at an explicit grain. A **dimension** provides descriptive attributes used to filter and group those measurements. A star schema links the central fact to dimensions. The grain sentence is the most important part: "one row per completed sync run per tenant" differs from "one row per connector per tenant per UTC day." Mixing both in one additive report double-counts.

Diagram 26.3



For the run-grain design, `(tenant_id, run_id)` is the business uniqueness boundary. `tenant_sk` identifies a particular historical tenant dimension version; it does not replace operational tenant authorization. A surrogate key is an analytical row identity, while a natural key identifies the source entity. Maintain both and validate the mapping.

Measures have aggregation rules. Accepted-row counts may be additive across disjoint runs; a daily active-connector count cannot be added across days to get monthly distinct connectors. Balances are often additive across accounts at a point in time, but not across snapshots of the same account. Percentages need their numerators and denominators, not an unweighted average of tenant percentages. A percentile cannot generally be averaged into a fleet percentile.

Avoid join fan-out: if a run has five errors and three tags, joining both detail tables before summing run volume produces fifteen joined rows. Aggregate each detail to the declared grain first, or use an explicitly modeled bridge with the correct allocation rules. A foreign key that matches is not proof that the resulting measure is meaningful.

Historical Dimensions

Type 1 changes overwrite an attribute, suitable when the report intentionally uses the current classification. Type 2 changes close one version and open another with a new surrogate key, suitable when the report must preserve the classification at the time of an event. For half-open intervals, the effective time belongs to `valid_from <= time < valid_to`; an open row has no upper bound. Enforce nonoverlap and define the source of effective time.

Suppose a tenant changes from standard to premium in February. A January fact joined to the current dimension answers "January usage by today's plan." Joined to the January version, it answers "usage by plan at the time." Both are legitimate questions; silently choosing one changes the metric. Late corrections can require splitting historical intervals and restating affected facts, not merely opening a row at processing time.

Dimension policy	Use when	Avoid when
Type 1 overwrite	Current categorization is the intended report meaning	Auditable historical categorization must be preserved
Type 2 versions	Historical attributes affect interpretation	No owner can define effective dates and correction rules
Separate event facts	Every transition is part of the required history	A current-state view alone satisfies the question

[VERIFY: validate merge/upsert atomicity, unique-key enforcement, interval checks and partition publication with the chosen analytical engine. The in-memory fixture does not execute SQL or enforce concurrent historical intervals.]

6. Late Data, Backfills and Metric Finality

Keep event time, ingestion time and processing time distinct. A sync may complete just before midnight, arrive after midnight and be aggregated the next morning. The metric contract must name the timezone and which time selects its day. UTC storage helps unambiguous instants but does not decide the business reporting calendar.

A watermark is a completeness policy, not proof that nothing older will ever arrive. Publish provisional results until the lateness contract closes the window. Late input may cause a correction, an exception workflow or an explicit restatement. "Final" must mean something an owner can defend, especially when a financial report consumes the result.

A safe backfill selects immutable input boundaries, pins the transformation and dimension policy, writes a separate generation, compares row counts and control totals, validates tenant isolation, then replaces the agreed partitions atomically where supported. Live ingestion must

not overwrite that generation with older logic. Establish a handoff boundary and reconcile the overlap. Rollback is switching to a retained valid generation, not hoping the previous files were not deleted.

Hypothetical sizing: 100,000 run facts per day at an assumed 500 bytes each gives 50,000,000 bytes of logical daily input before encoding, metadata, indexing, replicas or retained versions. This is arithmetic, not measured storage. Retaining raw changes, current projections and historical marts can multiply bytes; compression must be measured with representative input rather than asserted from a vendor name.

7. Data Quality Is a Publication Contract

Quality has several independent dimensions: schema validity, uniqueness at grain, referential integrity, completeness, freshness and semantic validity. A perfectly typed negative duration may still be invalid. A complete partition containing duplicated runs may still be wrong. A low-latency table with missing tenants may still be unusable.

At ingestion, verify the envelope, source identity and required fields. During transformation, check currency or unit consistency, supported enum meanings and dimension references. Before publication, reconcile counts and meaningful totals against the selected source boundary. After publication, monitor drift and consumer-facing freshness. Store rejection reason, original identity and mapping version in restricted quarantine; never drop malformed input silently.

[M] MECHANISM

Publish data and its completeness evidence together. A load manifest can bind source ranges, transformation version, row counts, rejection counts, control totals and the output generation. A dashboard reads the published generation and its cutoff. A failed quality gate leaves the last valid generation visible with a stale warning, rather than replacing it with a partial success.

Null is not automatically zero. Zero failures means observed runs had no failures; null might mean not collected, unknown, inapplicable or rejected. A quality rule must preserve the intended distinction. Similarly, a quarantine policy has to say whether a partition with rejected rows may be published as partial, must stop, or may exclude a documented class of optional records.

[VERIFY: validate the chosen orchestration and data-quality tooling's retry, backfill, dependency, alerting and atomic-publication behavior in a sandbox. No scheduler, quality framework or BI freshness alert was executed.]

8. Lineage and the Backend Service Contract

Lineage records how an output was derived: source table or event schema, fields, transformations, versions, runs and destinations. It makes “why did this number change?” answerable. A diagram alone is insufficient if it cannot identify which transformation revision produced a particular partition. Record both dataset-level dependencies and field-level derivation where sensitive fields or disputed measures require it.

Backend services should publish stable identities, tenant scope, occurrence time, entity version and explicit units. A domain event can state business intent better than a raw row change, but events need versioning and retention too. A CDC consumer tied to internal tables must coordinate schema evolution with the owner. An outbox reduces dual-write risk; it does not define the meaning of “successful run.”

The contract names the owner, grain, schema, metric definition, freshness target, replay window, delete semantics and compatibility policy. Consumers need notice before a column changes meaning, even if its SQL type does not change. An integer that used to mean milliseconds and now means seconds is structurally valid and semantically breaking.

Access controls must survive transformations and exports. A shared analytical table requires server-enforced tenant isolation and narrowly scoped service identities. Avoid raw tokens, certificates and payload secrets in landing data. For erasure or retention changes, lineage identifies affected stores; governing deletion from historical copies and backups remains a policy and implementation question, not a promise made by a CDC tombstone.

[VERIFY: confirm retention, erasure, audit and cross-border data obligations with the responsible privacy/legal team and current jurisdiction-specific guidance. Validate access enforcement across landing, curated tables and BI exports; no legal determination or live access audit was performed.]

9. Executable Model and Its Limits

`WarehouseLab.mjs` supplies `DimensionTable` and `FactTable`. Five checks cover ignoring replayed or older positions in a current-state projection, opening a changed dimension version while closing the old one, closing a deleted dimension without erasing prior history,

suppressing repeated fixture grain keys and rejecting noninteger fixture measures. The report is `code/26-data-platform/execution.json` ; the Windows runner is `code/26-data-platform/run.ps1` .

The fixture uses ordered per-key integer positions and supplied observation dates, not a real source log or event-time correction model. It is not a complete Type 2 loader: it has no concurrent writers, transactional checkpoint, durable restart or interval validation. Its fact keys join fixture values with a separator; production keys must be typed tuples or an unambiguous encoding. It ignores repeated fact keys without checking changed content, so a production loader must add content binding or an explicit correction contract. Integer measures describe this test dataset only; exact decimals are valid analytical measures elsewhere.

Recommended unexecuted acceptance tests: crash after analytical commit before offset commit; replay the same range; deliver a late dimension correction; change a source unit without a schema type change; load a fact with no dimension; delete a tenant across every retained copy; compare a backfill with the live projection at the same cutoff. Specify expected invariant, rollback and evidence before running them.

Concepts Explained Here

- [OLTP and OLAP](#): workload isolation and freshness.
- [Warehouse, data lake and lakehouse](#): storage and table-management choices.
- [ETL, ELT and streaming](#): placement versus cadence.
- [Analytics CDC](#): snapshot boundaries, positions, replay and deletes.
- [Star schema](#): facts, grain, measures and historical dimensions.
- [Data quality](#): checks that gate publication.
- [Lineage](#): source-to-output evidence and ownership.

Related Chapters

Use [00 master map](#), [06 databases](#), [07 transport and CDC](#), [08 data protection](#), [13 connectors](#), [18 reliability](#), [19 test boundaries](#), [25 ownership](#), [27 financial reconciliation](#) and [24 the connected journey](#). Generated relationships are reciprocal.

RELATED CHAPTERS / RECIPROCAL LINKS

[00](#) The System Before the Details
[06](#) Databases and Durable State
[07](#) Messaging, Batch and Streams
[03](#) Spring and Spring Boot Internals
[13](#) Data Integration and iPaaS
[08](#) Security, Identity and Data Protection

[27](#) Fintech and Payments Engineering
[18](#) Observability and Reliability
[19](#) Testing and Quality
[25](#) Architecture Styles and Domain Design
[24](#) IntegrationHub in Motion
[12](#) Twelve System Design Case Studies

One-Page Cheat Sheet

Workload: OLTP protects request transactions; OLAP answers scan-heavy historical questions. A replica does not automatically supply historical semantics.

Storage: warehouse for governed analytics, lake for retained diverse objects, lakehouse for table-managed lake data. Product names do not establish correctness.

Pipeline: ETL versus ELT chooses where transformation occurs. Batch versus stream chooses cadence. Both need replay identity, checkpoints and publication boundaries.

Model: declare one fact grain; preserve measure units; distinguish additive counts, distinct counts and snapshots. Dimension versioning changes the question a join answers.

Recovery: retain source positions and epochs; replay idempotently; handle late changes and deletes; backfill into an isolated generation before publishing.

Trust: quality gates, lineage, tenant enforcement and a visible completeness cutoff travel with the data. Five local checks passed; no data-platform runtime was tested.

Interview Corner

Basic: Why Not Run Every Report on PostgreSQL?

You can run bounded reports there. Separate analytics when scans, history or cross-source definitions exceed the operational resource and schema budget. First state freshness and completeness requirements.

Internals: What Does CDC Actually Guarantee?

A specific connector exposes changes under documented snapshot and log semantics. It does not automatically coordinate the analytical commit with its offset. Explain the commit gap and idempotent replay.

Trace/Debug: A Dashboard Doubled After a Restart.

Check whether cumulative values were summed, a batch was appended twice, or joins multiplied fact rows. Compare business grain keys, source ranges and output generation manifests before changing the chart.

Scenario: A Connector's Category Changed Last Month.

Ask whether old reports use current or historical categorization. Choose overwrite or versioned dimensions deliberately, define effective time and handle late corrections without overlapping intervals.

Basic: Warehouse or Lakehouse?

Decide from consumers, governance, existing skills, engine compatibility and cost. A table format adds metadata semantics, not automatic metric quality or universal query compatibility.

Internals: How Is a Percentage Aggregated?

Preserve and aggregate its numerator and denominator at compatible grain. Averaging tenant percentages weights tiny and large tenants equally unless that is the intended metric.

Trace/Debug: The Pipeline Is Green but a Tenant Is Missing.

Inspect source coverage, schema rejection, quarantine and last complete watermark. Process uptime and offset movement do not prove semantic completeness.

Scenario: Rebuild One Year While Live Ingestion Continues.

Pin source and transformation identity, write a separate generation, define an overlap cutoff, reconcile counts and totals, then publish atomically and retain rollback evidence.

Basic: What Is Lineage?

The evidence connecting an output to source datasets, fields, transformations, versions and runs. It supports impact analysis, repair and data-protection work.

Internals: Can a Source Delete Erase the Lake?

No. It is an input signal. Every retained representation needs its own authorized deletion or retention workflow, including derived tables and exports.

Trace/Debug: Why Did a Historical Number Change?

Compare input cutoff, transformation revision, dimension-effective dates, lateness policy and backfill generation. A reproducible report needs all of them, not only the query text.

Scenario: Does the Lab Prove Production Idempotency?

No. It checks a few in-memory transitions and duplicate fixture keys. Durable atomicity, conflicting content, concurrent ingestion and real source positions remain untested.

o8 / Security, Identity and Data Protection

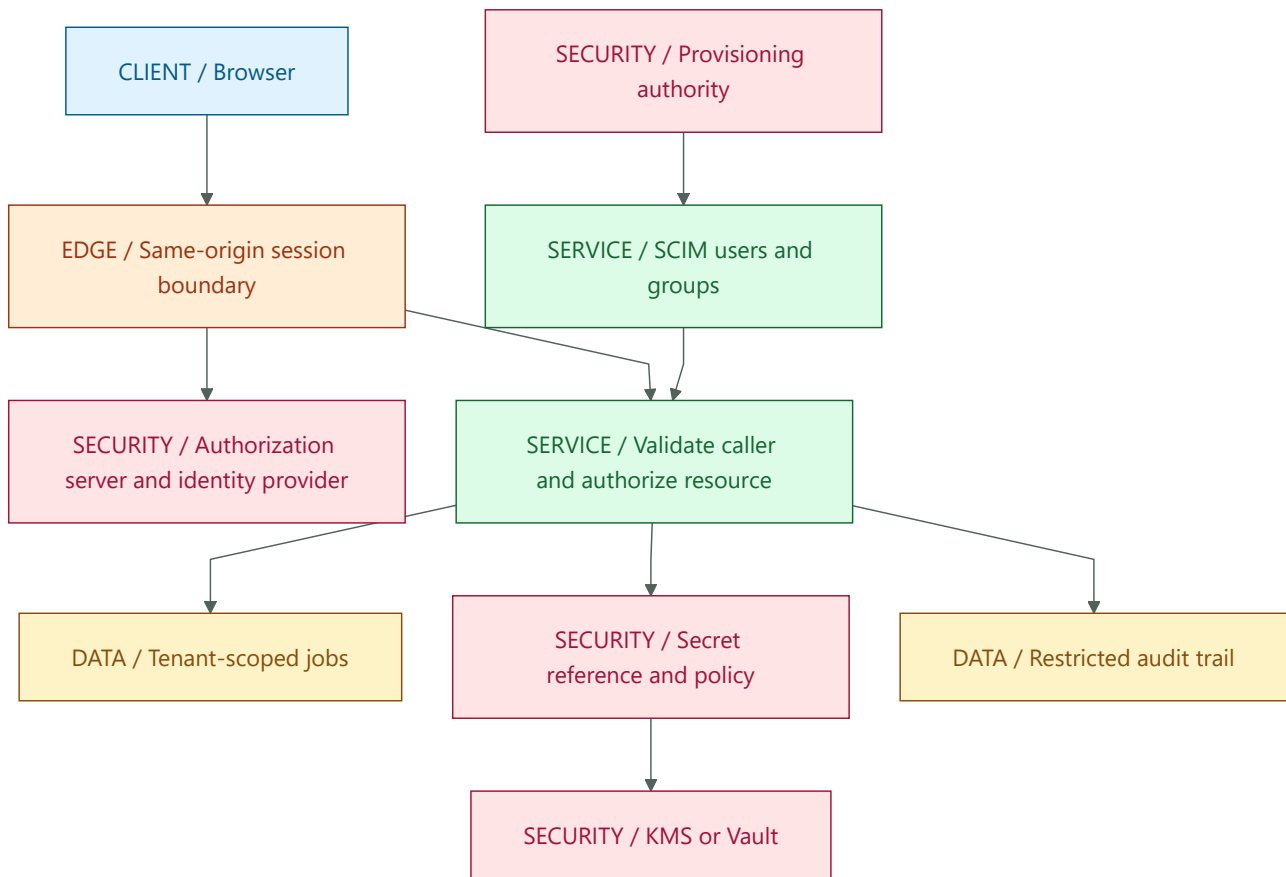
Identity tells you who is asking; authorization decides what that identity may do now. A correctly signed token is not permission to read every tenant's job. A private network is not an authorization boundary. Encryption does not make a badly authorized query safe. IntegrationHub is fictional, and this chapter follows its boundaries without claiming certification or describing a real organization's controls.

Version assumptions: Java 21, Spring Boot 4.0.x with the Spring Cloud 2025.1.x family previously checked in [03](#), PostgreSQL 17, Kafka 4.x and Kubernetes 1.34. OAuth 2.0 with the security guidance in RFC 9700, OIDC Core and TLS 1.3 are mechanism references, not an installed compatibility matrix. Product-specific identity-provider, Spring Security, Vault and KMS settings require deployment verification. No dependencies are downloaded in this chapter.

[VERIFY: check the selected Spring Security/resource-server version, OAuth provider discovery metadata, supported flows and RFC 9700 security guidance before implementation. Boot/Cloud family compatibility was checked previously; no identity provider, resource server or federation test ran here.]

Big Picture

Diagram 08.1



What You Will Be Able to Explain

- Separate authentication, authorization, provisioning, federation and transport identity.
- Trace authorization code with PKCE and distinguish access, ID and refresh tokens.
- Specify JWT validation without treating decoding or signature checking as sufficient.
- Explain SAML enterprise login and SCIM lifecycle management as different protocols.
- Trace certificate validation, keystore/truststore responsibilities, TLS termination and mTLS authorization.
- Apply tenant authorization, CORS/CSRF, secret rotation and OWASP API controls at the owning boundary.
- Explain envelope encryption, Vault leases, tokenization, safe audit records and engineering obligations around personal and payment data.

Where this lives in IntegrationHub. The 03 service layer authorizes each job and connector operation; 06 enforces tenant-scoped constraints; 07 carries authenticated, authorized work without assuming a topic is trusted. 09 owns browser state and logout handling. 17 owns workload identity and network boundaries, and 27 expands payment-specific obligations. Every external secret is referenced, not embedded in an event or browser response.

1. Authentication, Authorization and Tenant Isolation

Authentication establishes a principal from evidence under a trust policy. Authorization evaluates principal, action, resource and context. Provisioning creates and updates identity records and memberships. Auditing records security-relevant decisions. Treating these as one login feature hides failures such as a terminated user retaining a long-lived session or a valid tenant member accessing a different tenant's connector.

Trace a job read: validate the credential, obtain a trusted principal, resolve the requested tenant against that principal's current permissions, query the job within the authorized tenant, and apply operation-specific policy. Do not load by globally unique job ID and assume uniqueness implies access. Do not accept a browser's tenant selector as authority. For cross-tenant support operations, use a separate explicit capability with stronger auditing, not a hidden bypass in the normal query.

RBAC groups permissions into roles; ABAC evaluates attributes such as resource owner, environment or account status. Both need a policy owner and a revocation strategy. A role embedded in a token is a snapshot: removing membership does not rewrite already issued tokens. Choose a bounded token lifetime, live policy checks for sensitive actions, introspection or session invalidation according to the required revocation latency.

Control	Use when	Avoid when
Service resource authorization	Every protected resource access needs a decision	Gateway token validation is treated as sufficient
Database row-level policy	Defense in depth can be consistently configured and tested	Pooled session context leaks the previous tenant
RBAC	Stable job functions map well to permission bundles	Broad roles become an excuse to skip object checks
Attribute policy	Context and resource relationships matter	Rules are duplicated inconsistently across handlers

Production failure: a query scopes reads correctly, but a bulk update or export endpoint omits tenant scope. Test every access path, including background workers and administrator tools, using two tenants whose records have similar identifiers. Authorization is a property of the operation, not of the controller annotation alone.

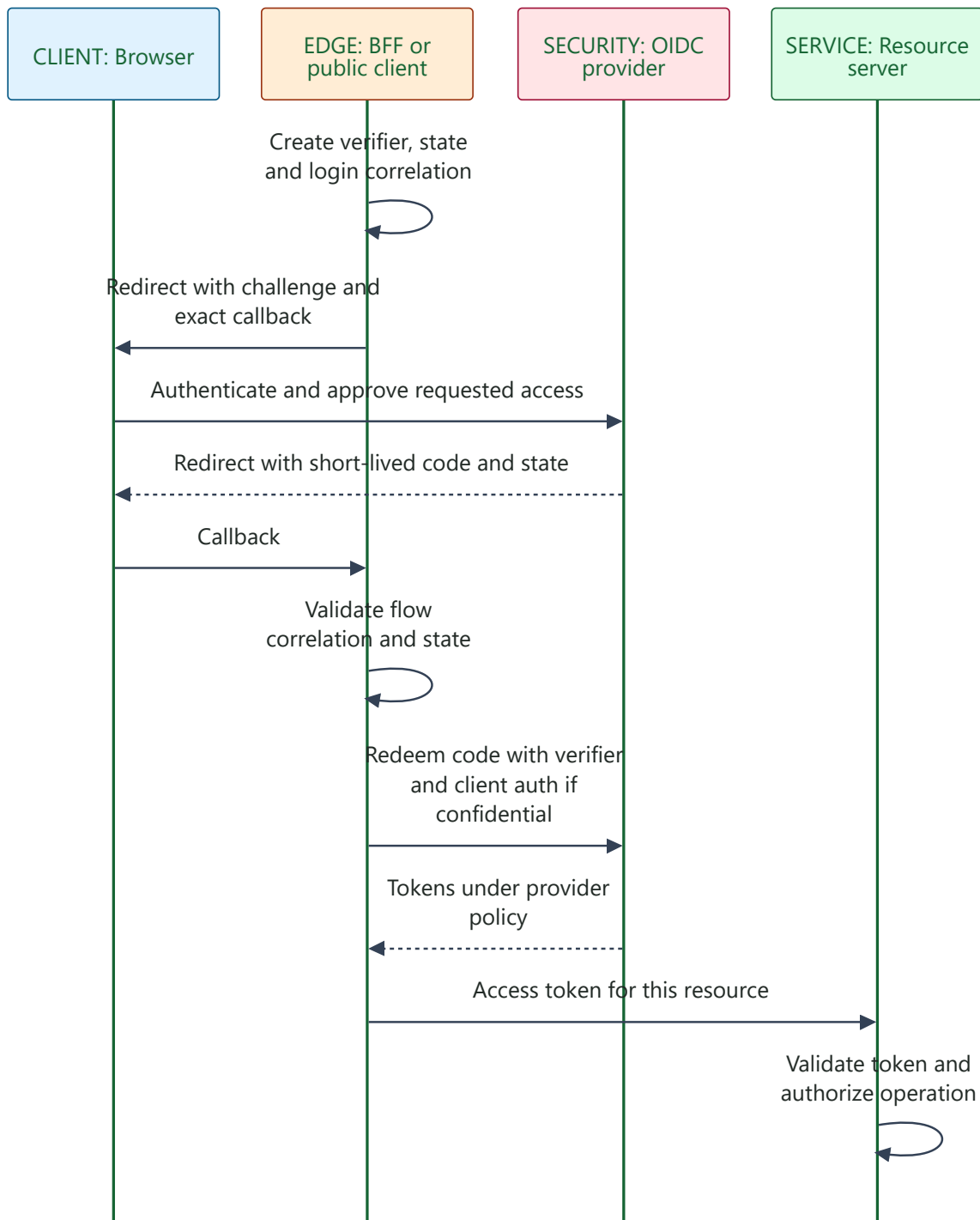
Interview checks: basic: authentication is not authorization. Internals: permissions have freshness. Debug: inspect resource selection and tenant binding. Scenario: a signed tenant claim must still be checked against the requested operation and resource.

2. OAuth 2.0, PKCE and OIDC

OAuth delegates API access. Its roles are resource owner, client, authorization server and resource server. OIDC adds an authentication layer with an ID token and identity claims. The same product can perform several roles, but the contracts differ: an ID token tells the client about an authentication event; an access token is for a resource server. An API should not accept an ID token merely because its signature is valid.

For interactive browser login, authorization code with PKCE keeps the access token out of the front-channel response and binds code redemption to the initiating client flow. The client creates an unpredictable verifier, derives an S256 challenge, and sends the challenge with the authorization request. The server binds the returned code to that challenge. Redemption supplies the original verifier; an intercepted code without it should not be sufficient. PKCE does not authenticate a public SPA as a confidential client, replace redirect-URI validation, or prevent malicious script already running in the page from acting as the user.

Diagram 08.2



`state` correlates a callback to a browser flow and helps prevent login CSRF; OIDC `nonce` binds an ID token to the authentication request where used. Both need unpredictable values, correct storage, one-time processing and checks, not mere presence in a URL. Exact redirect-URI registration limits where authorization responses may go. Issuer validation prevents a response from a different authority being accepted under the wrong configuration.

The IntegrationHub baseline puts a secure HttpOnly session cookie at a same-origin BFF, with tokens held server-side. A pure SPA is a public client: it cannot protect a client secret embedded in JavaScript. It can use code plus PKCE, but token storage, refresh and cross-origin access need an explicit threat model. SameSite and HttpOnly reduce specific risks; they do not make a browser session immune to XSS or CSRF.

Flow	Use when	Avoid when
Authorization code plus PKCE	Interactive user login for modern clients	An embedded secret is used to pretend a SPA is confidential
Client credentials	A workload acts on its own granted authority	It is used to impersonate an interactive user
Device authorization	A constrained device cannot conveniently run normal login	The product has an ordinary secure browser redirect flow already
Legacy implicit/password flows	Understand existing migrations and restrictions	Designing a new application around exposing tokens in redirects or collecting user passwords

Scopes describe delegated permissions but do not necessarily encode all business policy. A token with `jobs:read` must still be restricted to allowed resources and tenants. Audience identifies the intended resource; a token minted for a different API should not become valid just because both APIs trust the same issuer.

Interview checks: basic: OAuth delegates; OIDC authenticates. Internals: PKCE binds code redemption. Debug: compare issuer, callback, state, nonce and verifier across one flow. Scenario: use workload identity/client credentials for service authority, not a fabricated end-user login.

3. JWT Validation and Refresh Lifecycle

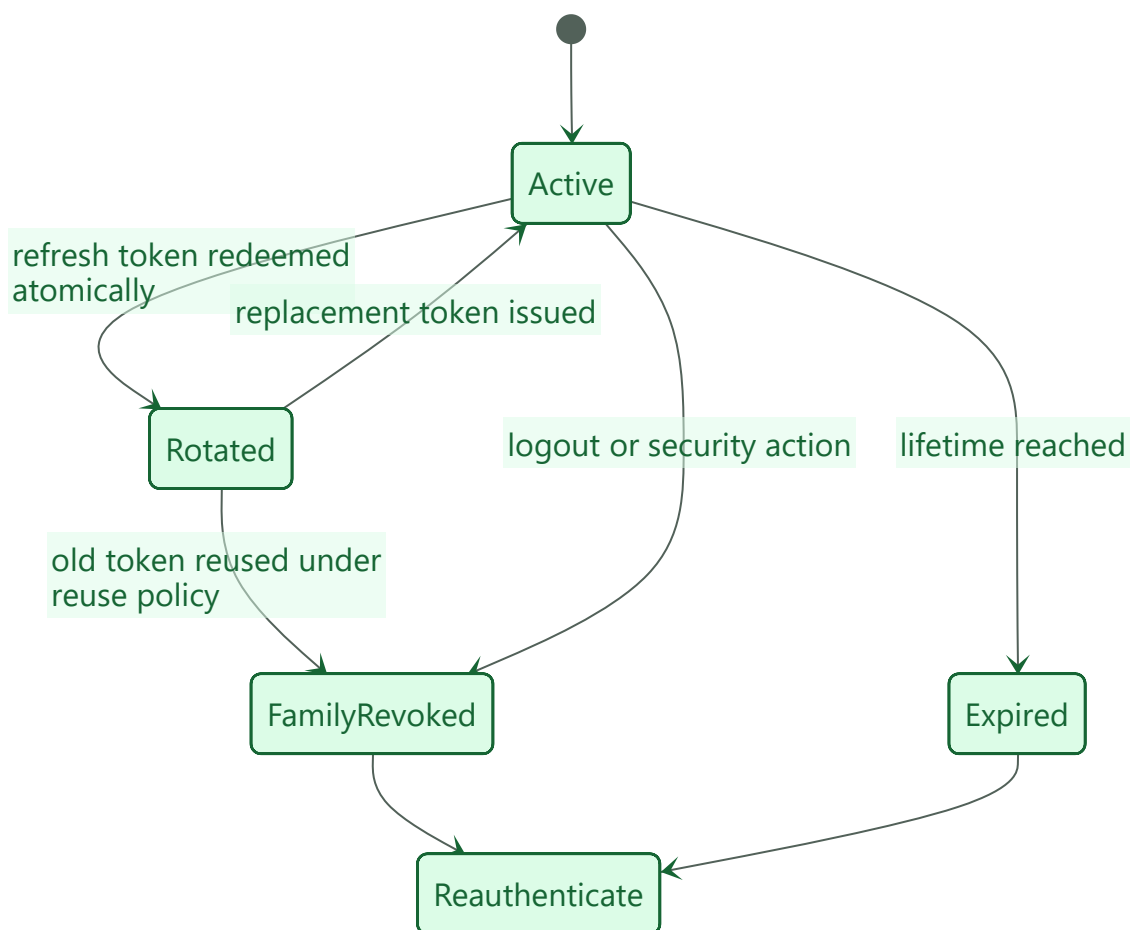
A JWT is a representation, not a security policy. Signed tokens commonly expose their header and payload to anyone holding them; signing is not encryption. Parsing three segments and reading claims establishes no authenticity. Use a maintained JOSE/OIDC library integrated with Spring Security rather than writing a parser, signature verifier or key-discovery client for production.

Validation selects an explicitly trusted issuer and permitted algorithms, obtains keys from trusted configured metadata, verifies the signature, enforces audience and token purpose, and checks temporal claims with bounded clock skew. Application policy may require expiration, subject, authorized party, tenant bindings or other claims beyond what generic parsing requires. Do not let untrusted token fields choose arbitrary remote key URLs or relax algorithm requirements. Treat `kid` as a lookup hint inside trusted key material, not proof of identity.

Key caches need bounded refresh. Rotation introduces a new signing key, publishes its verification key, starts issuing with it, and retains old verification keys while accepted old tokens can still be valid. An unknown key ID may justify a rate-limited refresh, not an unbounded network request for every hostile token. Removing all old keys instantly can log out legitimate callers; retaining a compromised key indefinitely can preserve unauthorized access.

Opaque access tokens move validation toward server-side lookup/introspection. JWT validation can avoid a network call for every request but has revocation/freshness trade-offs. A denylist or live policy check can narrow those trade-offs, at the cost of state and availability coupling. Neither token format automatically supplies revocation, tenant authorization or correct logout.

Diagram 08.3



Refresh rotation must be atomic: consume the old token and establish its replacement under a single family policy. Concurrent tabs can race and make a legitimate retry resemble theft. Coordinate refresh at the client/BFF and define provider-supported retry behavior; do not silently allow arbitrary reuse. Reuse detection is a signal to revoke the affected family and

investigate, not permission to log raw token values. Hash stored refresh handles where the chosen implementation supports it, protect session stores, and bound idle and absolute lifetimes.

[!] TRAP

Valid signature does not mean valid request. Wrong issuer, audience, token purpose, expiration, tenant or resource permission can make an otherwise authentic token unacceptable. Authentication succeeds only under the complete validation policy; authorization is still a separate decision.

[VERIFY: confirm JWT validation defaults, required claims, clock-skew tolerance, JWK cache/rotation behavior, introspection and refresh-family reuse handling against the selected Spring Security and identity-provider versions. No token, concurrent refresh or logout experiment was executed.]

Interview checks: basic: JWT is not encrypted by default. Internals: trusted metadata constrains key selection. Debug: valid signatures with wrong audiences must fail. Scenario: revocation latency is a requirement to choose, not a property inferred from JWT.

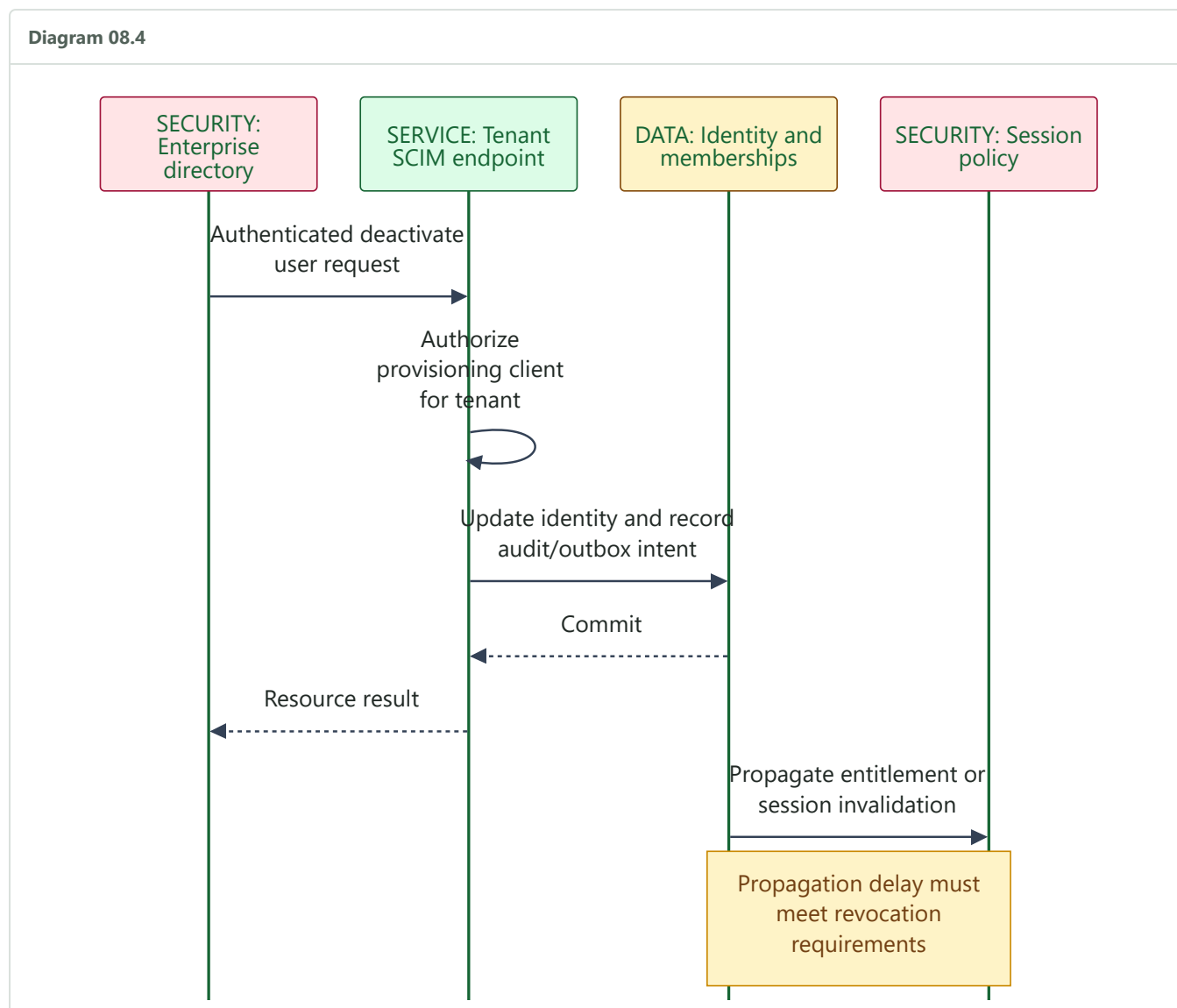
4. SAML, SCIM and Enterprise Identity Providers

SAML commonly connects enterprise identity providers to service providers through signed XML assertions about authentication. The service provider must validate issuer, signature under trusted metadata, audience, recipient/destination, time conditions and request correlation as required by the flow. A signature on one element is not permission to consume an unrelated unsigned element. Use mature SAML/XML security libraries with hardened parsing and documented message validation.

Federation introduces identifier mapping. An email address can change and may be recycled; it is usually a poor durable account key by itself. Bind accounts using a stable provider-specific subject within an issuer/tenant namespace, and make account linking an authenticated operation. Do not automatically join unrelated identities merely because two providers supplied the same display name or unverified email.

SCIM manages identities and memberships: users, groups, updates, deactivation and provisioning synchronization. It does not perform interactive login and does not replace OAuth or SAML. A SCIM request is a privileged API call with its own authentication, tenant boundary,

idempotency and audit requirements. Deprovisioning must reach session/token policy and application entitlements; setting active=false in one directory does not magically terminate every resource-server session.



Okta is an example of an identity-provider platform, not a synonym for any protocol. A provider may support OIDC, SAML, provisioning and lifecycle automation with different configuration surfaces. Keep issuer registration, protocol metadata, stable subject mapping, provisioning credentials and tenant onboarding as separate concerns. A discovery document can describe endpoints; it does not authorize trusting every issuer a user types into a form.

Mechanism	Use when	Avoid when
OIDC federation	Modern clients need standard authentication over OAuth flows	An ID token is forwarded as an arbitrary API bearer credential
SAML federation	An enterprise integration requires established SAML SSO	XML assertions are parsed with hand-written signature selection

Mechanism	Use when	Avoid when
SCIM provisioning	Directory lifecycle should manage users/groups	Provisioning is mistaken for session authentication
Provider SDK	Maintained integration reduces protocol mistakes	Application policy becomes an unexplained default in a vendor console

[VERIFY: check SAML profile validation, SCIM RFC 7643/7644 behavior, PATCH/filter support, stable identifier mappings and Okta tenant-specific provisioning/deactivation semantics against configured products. No federation or provisioning environment was exercised.]

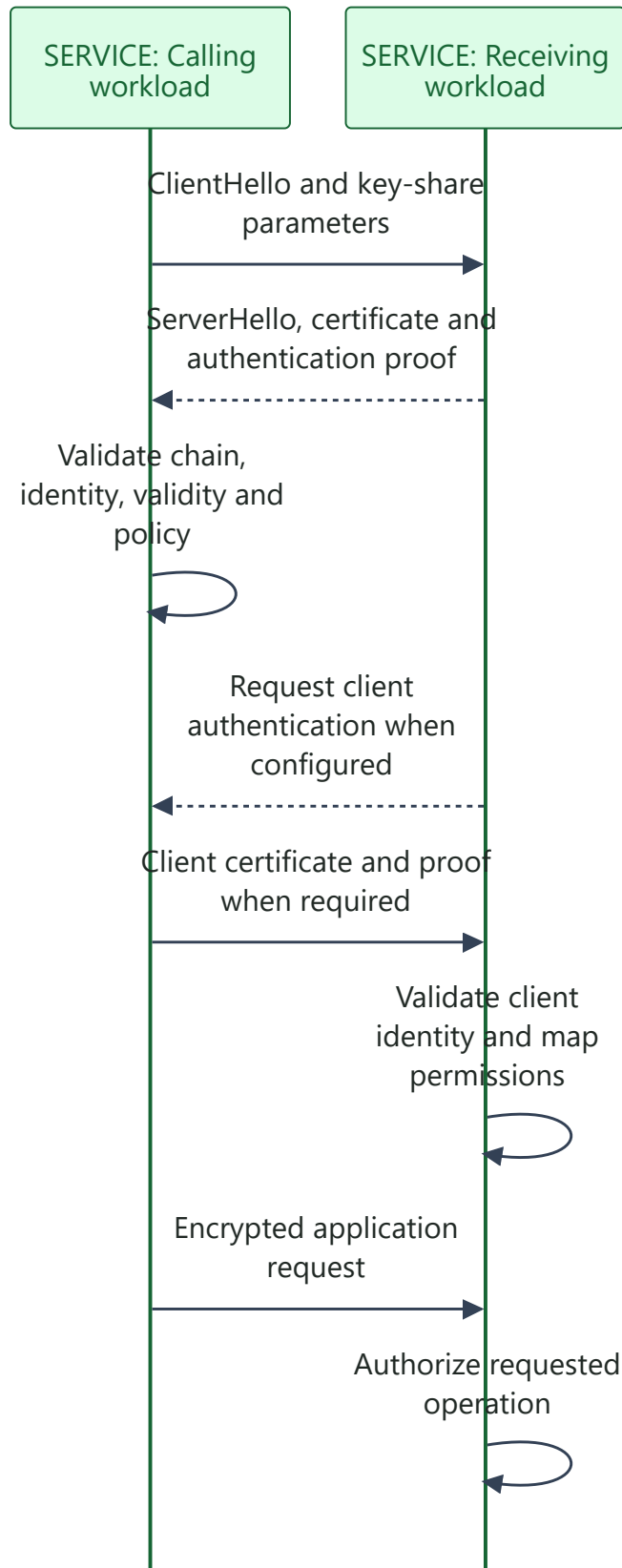
Interview checks: basic: SAML signs identity assertions; SCIM provisions resources. Internals: issuer plus stable subject defines identity. Debug: a deactivated account may retain existing sessions. Scenario: tenant federation onboarding needs a controlled trust-registration process.

5. PKI, X.509, Keystores, TLS and mTLS

PKI binds public keys to identities through certificates and trust policy. An X.509 certificate contains identity information, a public key, validity and extensions, and is signed by an issuer. A certificate chain is evaluated toward a configured trust anchor. Successful chain validation is not sufficient without checking the intended identity, such as the hostname in the subject alternative name, and permitted key usage.

TLS negotiates algorithms and establishes encrypted, integrity-protected transport with peer authentication under its configuration. The certificate's key participates in authentication; modern ephemeral key exchange derives session keys rather than simply encrypting all application data with the certificate's private key. Forward secrecy limits exposure of old traffic if a long-term authentication key is later compromised, subject to the negotiated protocol and secret retention.

Diagram 08.5



The sequence is a conceptual TLS 1.3 flow, not a packet transcript. Detailed ordering, resumption and certificate requests are covered by the negotiated protocol. 21 owns transport mechanics. A load balancer terminating TLS ends that protected channel; re-encrypt internal hops when the threat model requires it. An external HTTPS URL does not prove the database or service-mesh hop is encrypted.

In Java, a keystore can hold private-key entries and certificate chains; a truststore holds certificates used as trust anchors. The words describe roles, not necessarily distinct file formats. Protect private material, store passwords outside source control, and scope trust. Importing every certificate into a global truststore to fix a handshake failure silently enlarges the set of trusted peers. Disabling hostname verification removes an important identity check.

mTLS authenticates both transport peers. It does not decide whether workload A may export tenant B's data. Bind a validated workload identity to policy and still authorize application resources. Certificate rotation requires overlapping trust and usable private-key loading: replacing a file may not recreate an existing SSL context or connection pool. Measure remaining certificate lifetime and test reload/reconnect behavior before expiry.

Transport choice	Use when	Avoid when
Server-authenticated TLS	Clients need server identity and protected transport	Internal hops are assumed protected without checking termination
mTLS	Workloads require strong peer identity and managed certificate lifecycle	Client certificate possession grants unrestricted application access
Narrow truststore	A workload should trust a defined authority set	Troubleshooting imports unrelated roots without review

[VERIFY: confirm TLS version/cipher policy, certificate path and revocation behavior, hostname checks, keystore provider support and live rotation behavior for the deployed JVM, proxy and mesh. The sequence is conceptual; no handshake, certificate or reload test ran.]

Interview checks: basic: keystore identity differs from truststore trust. Internals: encryption and identity validation are distinct. Debug: inspect chain, SAN, time, trust and termination. Scenario: a certificate rotation is complete only when new connections actually use it.

6. CORS, CSRF and Browser Credentials

The same-origin policy limits how browser scripts read across origins. CORS lets a server selectively relax those browser read restrictions. It is not API authentication: nonbrowser clients do not enforce it, and some cross-origin requests can be sent without a preflight even when

their responses cannot be read. Never protect a state change solely by omitting CORS headers.

CSRF exploits automatically attached credentials, commonly cookies, to cause unwanted requests. Protect state-changing endpoints with the framework's CSRF mechanism and suitable origin checks, use safe HTTP methods correctly, and scope cookies with Secure, HttpOnly and an appropriate SameSite policy. SameSite is a valuable layer, not a universal replacement for explicit CSRF defenses. Cross-site and cross-origin are related but different concepts.

For credentialed CORS, allow explicit trusted origins rather than reflecting arbitrary input, and vary/cache responses correctly. A wildcard policy is not suitable for credentialed browser reads. Preflight approval is not a resource authorization decision; the subsequent actual request still needs authentication and policy checks.

XSS can execute requests in the user's browser context even when HttpOnly prevents direct cookie reading. Render untrusted content as text, use framework escaping, avoid unsanitized HTML insertion, deploy an appropriate Content Security Policy, and review third-party scripts. A bearer token in localStorage is readable by injected script; putting it in memory reduces persistence but does not remove active-script compromise.

Long-lived SSE and WebSocket connections need origin validation and credential-expiry/revocation policies. A handshake authenticated yesterday does not authorize a subscription forever. Check each subscription's resource scope; terminate or reauthorize connections under the session contract. The native EventSource limitation on arbitrary authorization headers makes the baseline same-origin cookie/BFF arrangement useful, as [05](#) explains.

Defense	Use when	Avoid when
Explicit CORS allowlist	Trusted browser origins need cross-origin response access	It is treated as protection against scripts or servers outside browsers
CSRF token and origin checks	Cookies authenticate state-changing requests	GET changes state or SameSite alone is assumed universally sufficient
Output encoding and CSP	Untrusted values reach UI rendering	HttpOnly is assumed to eliminate XSS impact

Interview checks: basic: CORS controls browser access, not caller identity. Internals: cookies can be sent without script reading them. Debug: a blocked response does not prove the request had no effect. Scenario: use separate XSS and CSRF controls for a BFF session.

7. OWASP API Risks and Secure Input Boundaries

Broken object-level authorization is especially relevant to tenant systems: valid authentication plus a guessed resource ID can become data exposure if the service does not authorize that object. Broken property-level authorization appears when input binding permits fields such as tenant, owner or approval status to be changed. Narrow command DTOs and allowlisted transitions are safer than deserializing a persistence entity and merging it.

Resource consumption is also security. Bound page sizes, upload size, decompression, JSON nesting, query complexity, concurrent jobs and expensive provider calls. A small request can trigger large downstream work. Rate limits by IP alone are insufficient for shared networks and authenticated tenants; quotas and concurrency controls belong near the expensive resource as well as the edge.

SSRF is relevant to connector and webhook URLs. Resolve and validate allowed destinations under a controlled egress policy, reject unwanted address ranges and schemes, re-evaluate redirects, and prevent DNS changes from bypassing the connection-time policy. Registration-time string checks are not enough. Do not fetch arbitrary URLs with privileged cloud-network access.

Use parameterized SQL and safe library APIs rather than concatenating input into commands or queries. Validate business meaning as well as type, canonicalize under a defined policy, and do not expose stack traces, secrets or raw vendor responses. Inventory old endpoints and schemas: an unmaintained version can bypass controls added to the current one.

[M] MECHANISM

A secure boundary narrows authority. The browser supplies a request, not a trusted identity; the service derives a principal, scopes the resource and sends the smallest permitted command downstream. Validation and authorization should reduce what the next component can do.

[VERIFY: compare controls with the current OWASP API Security Top 10 and ASVS requirements appropriate to the product. The categories here are an engineering review checklist, not a security assessment, penetration test or certification.]

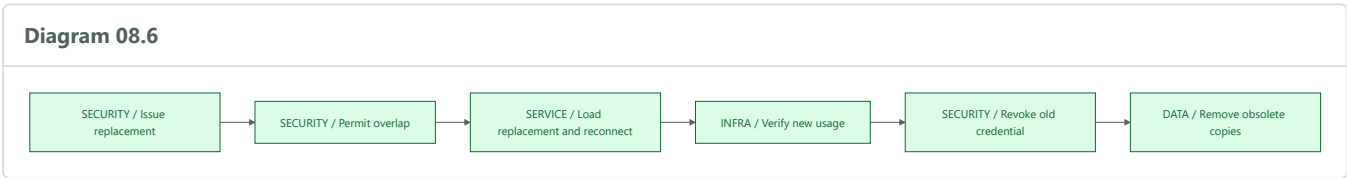
Interview checks: basic: an authenticated endpoint can still expose other users' objects. Internals: mass assignment bypasses field ownership. Debug: trace outbound requests through redirect and DNS policy. Scenario: threat-model amplification and egress, not only request syntax.

8. Secrets, Vault and Rotation

A secret has an owner, authorized consumers, an issuance mechanism, an expiry/rotation policy and a revocation path. Environment variables are a delivery mechanism, not a complete secret-management system. They can appear in process inspection, diagnostic dumps or accidental logs. Kubernetes Secret objects are not automatically a claim that every storage and access path is encrypted and least-privileged.

Vault separates authentication from policies and secret engines. Workloads authenticate through a supported identity mechanism and receive permissions; a secrets engine may supply static versioned values or dynamic credentials with leases. Dynamic database credentials can narrow lifetime and attribution, but renewal, lease expiry, database connection lifetime and revocation must be coordinated. A pool holding old authenticated connections may outlive the file that delivered its password.

Prefer platform/workload identity to a long-lived bootstrap token embedded in an image. Scope access by application and environment. A connector worker should request a permitted secret reference for its assigned tenant/job, not list every tenant's credentials. Audit secret access without recording the secret itself, and consider the availability policy when the secret store is temporarily unavailable.



Rotation is a rollout, not a file write. Support overlap where possible, publish the new version, make consumers reload or rebuild clients, verify use, then revoke old credentials and remove copies. For a suspected compromise, shorten or eliminate overlap according to risk and accept the availability consequences deliberately. A rollback must not restore a known-compromised credential.

Secret strategy	Use when	Avoid when
Dynamic leased credentials	The backend supports bounded, attributable access	Consumers cannot renew/reconnect before expiry
Versioned static secret	An external provider only supports static credentials	Rotation has no overlap or application reload plan
Workload identity	Platform identity can authorize secret/KMS access	A universal bootstrap token is copied into every image

[VERIFY: confirm Vault authentication, policy, lease, renewal and revocation semantics, Kubernetes delivery/reload behavior and provider-specific credential overlap in the actual deployment. No secret store, rotation or revoked-connection test ran.]

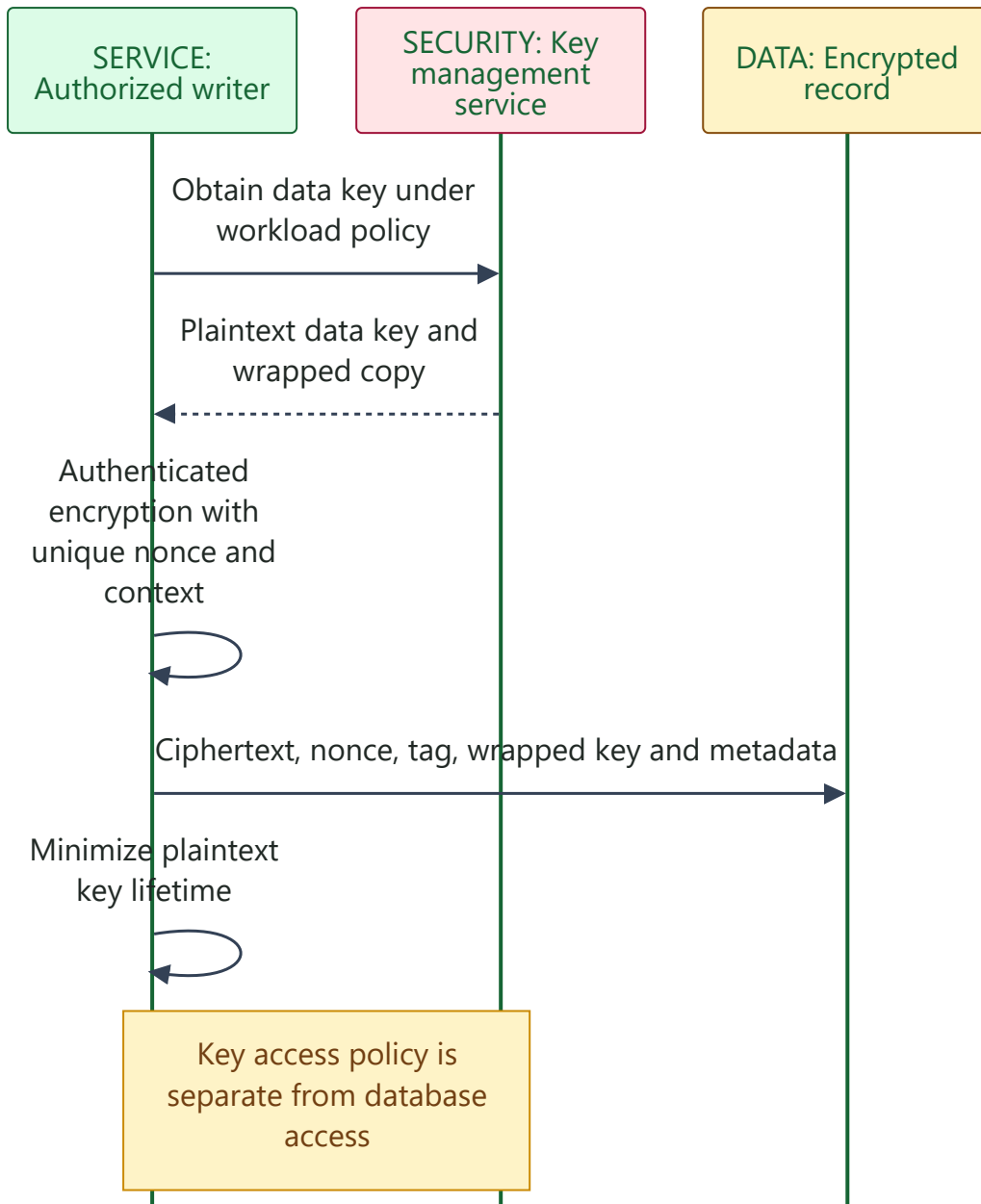
Interview checks: basic: delivery does not define lifecycle. Internals: leased credentials and authenticated connections can have different lifetimes. Debug: check whether clients reloaded, not only whether the secret changed. Scenario: test rotation with old and new workers running together.

9. KMS and Envelope Encryption

Encryption at rest protects particular storage exposures; transport encryption protects particular hops. Neither prevents an authorized but overly privileged application from reading data. Start by naming the threat: stolen disks, backup exposure, tenant isolation, cloud operator access, or application compromise. Different controls cover different threats.

Envelope encryption uses a data-encryption key to encrypt application data and a key-encryption key, often controlled by KMS, to wrap that data key. Store ciphertext, wrapped key and required authenticated metadata together. On read, an authorized workload asks KMS to unwrap the data key and then decrypts locally. KMS does not need to receive every large payload; it controls key use and policy.

Diagram 08.7



Use maintained cryptographic libraries and an authenticated-encryption scheme appropriate to the platform. Nonce uniqueness and authenticated context matter; do not invent an encryption format. Bind context such as tenant and record identity where appropriate so ciphertext cannot silently be moved into another logical context. Java's managed memory complicates guaranteed secret erasure; minimizing lifetime and exposure is more honest than claiming a garbage-collected string was securely wiped.

Key rotation can mean rotating the KMS key, rewrapping data keys, or decrypting and re-encrypting payloads. These are different operations with different costs and implications. Disabling an old key before backups and retained data have a recovery plan can make

legitimate restoration impossible. Back up key metadata and recovery procedures under a secure ownership model, not the plaintext keys in the same storage bucket as the data.

Encryption layer	Use when	Avoid when
Storage encryption	Disks/snapshots need protected storage	It is presented as object-level authorization
Application envelope encryption	Selected fields need separate key-access control	Query/search requirements and key lifecycle are ignored
TLS/mTLS	Data needs protected authenticated hops	One encrypted public hop is taken to cover the whole path

[VERIFY: verify provider KMS envelope-encryption APIs, authenticated context, key versioning, quotas, availability and recovery behavior; confirm algorithm/nonce requirements with the chosen library. No cryptographic code or key-rotation experiment is supplied or executed here.]

Interview checks: basic: data key encrypts data, wrapping key protects data key. Internals: authenticated context prevents unnoticed reassociation. Debug: inaccessible old ciphertext can be key-lifecycle failure. Scenario: recovery objectives must include key availability.

10. PII, Tokenization, Audit and Regulatory Engineering

Inventory personal data before choosing controls. Track where it enters, which purposes require it, where it is copied, who accesses it, retention and deletion paths. Logs, DLQs, browser analytics, search indexes and backups are data stores too. A redacted API response does not help if an exception logger captured the original payload.

Tokenization substitutes a reference for a sensitive value and keeps the mapping under controlled access. Encryption transforms data under a key and is generally reversible by an authorized decryptor. Hashing supports selected equality/verification use cases but does not automatically anonymize predictable values. Pseudonymized or tokenized data can remain personal data when reidentification is possible.

Audit records should capture actor, action, authorized tenant/resource, decision, timestamp, correlation and relevant version without raw credentials or unnecessary payloads. Restrict both writes and reads, protect integrity, synchronize clocks under a defined policy and make records searchable for investigations. Audit logs differ from diagnostic logs: silently sampling them may break accountability, while dumping every request body into them creates another exposure.

Deletion must propagate through source records, projections, caches and retained copies under a documented policy. A backup may need controlled expiry rather than arbitrary mutation, plus procedures preventing restoration from reintroducing deleted records into service. Immutable event history and privacy obligations require an explicit design review, not the assertion that append-only storage is exempt.

[VERIFY: GDPR applicability, lawful bases, rights handling, retention, breach notification and cross-border transfer obligations require current official guidance and qualified legal/privacy review. Tokenization or pseudonymization is not by itself anonymization. This chapter provides engineering considerations, not legal advice or a compliance determination.]

For payment data, prefer provider-hosted collection and tokens so the application need not receive primary account numbers or sensitive authentication data. Scope reduction is a design objective, not proof of being out of scope. Segmentation, access control, logging, vulnerability management and evidence collection depend on the actual payment flow and responsibility split.

[VERIFY: check the currently applicable PCI DSS version, card-data retention restrictions, sensitive-authentication-data rules and eligibility for any reduced-scope assessment with PCI SSC guidance, the payment provider and a qualified assessor. No PCI compliance or scope determination is claimed; Chapter 27 expands the payment architecture.]

Data control	Use when	Avoid when
Minimize and delete	Data is unnecessary or past its justified lifetime	Retain everything because storage is cheap
Tokenize	Most components need a reference, not the sensitive value	Tokens are assumed non-sensitive without evaluating mapping access
Restricted audit trail	Security decisions need accountable reconstruction	Raw secrets or entire customer payloads are recorded
Hosted payment collection	Provider can keep card entry away from the application	Integration is assumed automatically outside all obligations

[D] **DECISION**

Reduce the number of places that can see a secret before adding more encryption layers. A component that never receives the credential or sensitive field cannot leak that value through its ordinary logs, cache or error response. Encryption and access policy still protect the remaining necessary copies.

11. Security Review and Evidence

Review the request journey as a sequence of authority changes. For each hop, state the authenticated identity, permitted action, resource boundary, credential lifetime, replay protection, sensitive fields and audit evidence. Include degraded modes: identity-provider outage, unavailable key store, delayed deprovisioning, clock skew, stale JWK cache and old long-lived connections.

Executable security acceptance tests should include wrong issuer/audience, expired and not-yet-valid credentials, missing required claims, unsupported algorithms, cross-tenant IDs, unauthorized fields, CSRF failure, untrusted origins, stale sessions after deprovisioning, certificate identity mismatch and interrupted key rotation. Assert rejection without disclosure; do not only test successful login. Use maintained protocol libraries and a disposable controlled environment, never real credentials in a guide.

Execution evidence: this chapter contains no hand-written authentication or cryptographic implementation. All IdP, JWT, SAML, SCIM, TLS, KMS, Vault and browser security integration tests are NOT EXECUTED. Protocol and regulatory references are pointers for verification, not claims that this environment retrieved every specification or established compliance. Publication validation checks the document, not the controls.

[?] INTERVIEW

Answer security questions in layers: trusted identity, token/protocol validation, resource authorization, data protection, revocation and evidence. Naming OAuth or TLS without explaining the remaining boundaries is incomplete.

Explained-Here Index and Sources

Concepts	Explanation
Authentication, authorization, RBAC, tenant scope	Boundaries
OAuth 2.0, PKCE, OIDC	Login flow
JWT, refresh token and signing-key rotation	Token lifecycle
SAML, SCIM and Okta	Enterprise identity
PKI, X.509, keystore, TLS and mTLS	Transport identity
CORS, CSRF and XSS	Browser credentials
OWASP API controls	Input and authority boundaries

Concepts	Explanation
Secrets, Vault and rotation	Secret lifecycle
KMS and envelope encryption	Key hierarchy
PII, tokenization, audit logging, GDPR and PCI DSS	Data protection

Owning references: [OAuth security best current practice](#), [OIDC Core](#), [JWT best practices](#), [SCIM protocol](#), [OWASP API Security](#), [Spring Security](#), [Vault documentation](#), [GDPR official text](#) and [PCI SSC](#). These are verification destinations, not locally executed evidence.

Related Chapters

Return to [00](#). Connect [03 Spring](#), [05 APIs](#), [06 databases](#), [07 messaging](#), [09 browser](#), [17 cloud](#), [18 operations](#), [19 tests](#), [21 networking](#) and [27 payments](#). Generated links are reciprocal.

RELATED CHAPTERS / RECIPROCAL LINKS

00 The System Before the Details	18 Observability and Reliability
02 Concurrency and the Memory Model	19 Testing and Quality
21 Networking and Operating Systems	10 System Design Foundations
03 Spring and Spring Boot Internals	11 Object Design, Patterns and LLD
04 Hibernate and JPA	12 Twelve System Design Case Studies
06 Databases and Durable State	13 Data Integration and iPaaS
05 APIs and Real-Time Communication	14 Docker and Container Internals
07 Messaging, Batch and Streams	15 Kubernetes and Production Workloads
26 Data Platform Basics	16 Build, Ship and Roll Back
09 The Browser and the Java Backend	23 JVM Performance Engineering
17 Cloud and Infrastructure	24 IntegrationHub in Motion
27 Fintech and Payments Engineering	20 Appendix: Interview Toolkit and Index

One-Page Cheat Sheet

Authority: authenticate the caller, then authorize action and tenant-scoped resource. Gateway checks, valid signatures and private networks do not replace service policy. Provisioning and session revocation are separate lifecycles.

Login: OAuth delegates access; OIDC adds authentication. Code plus PKCE binds redemption to the initiating flow. Validate callback correlation, issuer and token purpose. Public clients cannot protect embedded secrets. ID tokens are not arbitrary API access tokens.

Tokens: validate algorithm, trusted key, issuer, audience, time and required claims with a maintained library. JWT signing does not encrypt claims. Coordinate refresh rotation and family reuse policy. Key rotation needs overlap and bounded refresh, while compromise response may deliberately favor containment.

Enterprise and transport: SAML federates login; SCIM manages lifecycle. X.509 chain and hostname checks establish peer identity under policy. Keystore holds identity; truststore supplies trust. mTLS still needs operation authorization. Inspect every TLS termination hop.

Browser: CORS is not authentication. Cookies need CSRF defenses. HttpOnly reduces credential reading but does not prevent XSS from making requests. Reauthorize long-lived subscriptions under the session policy.

Data: minimize copies, scope secret access, test rotation, and distinguish data-key wrapping from payload encryption. Tokenization is not automatically anonymization or exemption from obligations. Audit decisions without secrets. GDPR and PCI conclusions require current qualified review, not a checklist assertion.

Interview Corner

Basic: OAuth Versus OIDC?

OAuth delegates API access; OIDC adds authentication and an ID token for the client. A resource server validates access credentials intended for it, not any signed identity assertion.

Internals: What Does PKCE Prove?

Code redemption possesses the verifier matching the challenge bound to the authorization request. It does not turn a public client into a confidential one or make compromised browser code trustworthy.

Trace/Debug: Login Works but the API Rejects the Token. Why?

Check token purpose, issuer, audience, expiration, allowed algorithms and required scopes before changing signature settings. An ID token or token for another resource should be rejected even after a valid login.

Scenario: A Tenant Member Guesses Another Tenant's Job ID.

Resolve authorized tenant scope and query/authorize the resource within it. Return a safe denial without leaking existence where the policy requires that. Test bulk, export and background paths as well as individual reads.

Basic: SAML Versus SCIM?

SAML carries federation assertions for login; SCIM manages users and groups. Deprovisioning through SCIM must connect to application entitlement and session policies rather than assuming a directory update ends all access.

Internals: Why Is a Valid JWT Signature Insufficient?

It establishes integrity under a key, not the right issuer/audience/purpose, current validity or resource authorization. Apply all protocol and application checks using constrained trusted configuration.

Trace/Debug: Rotation Broke Half the Fleet.

Inspect which workers loaded the new credential, which connections still use the old one, overlap timing and JWK/trust cache refresh. Updating the secret store alone does not rebuild clients or connection pools.

Scenario: Refresh Token Reuse Is Detected.

Apply the provider's family revocation and investigation policy, terminate affected sessions as required and require reauthentication. Account for legitimate concurrent refresh races through deliberate client coordination; never log raw tokens.

Basic: CORS Versus CSRF?

CORS controls browser cross-origin response access; CSRF defense prevents unwanted actions using automatically attached credentials. A response blocked by CORS can still follow a request that changed state.

Internals: Does mTLS Replace OAuth or Authorization?

No. It authenticates transport peers. The application still needs permissions and tenant/resource checks; user delegation and workload identity can coexist with different scopes.

Trace/Debug: A Secret Changed but Old Connections Still Work.

The connection authenticated earlier and may remain usable under backend policy. Check pool lifecycle and revocation semantics, force appropriate reconnection and verify new use before concluding rotation completed.

Scenario: How Would You Reduce Sensitive Data Exposure?

Remove unnecessary collection and copies first, then tokenize/restrict remaining fields, encrypt appropriate layers, scope key access and audit decisions. Include logs, queues, projections and backups in retention and deletion design.

Basic: What Is Envelope Encryption?

A data key encrypts the payload, and a KMS-controlled key wraps the data key. Store the wrapped key with ciphertext and authenticated metadata; key policy and recovery remain separate responsibilities.

Internals: Is a Hash of an Email Anonymous?

Not necessarily. Predictable input can be guessed and linked, and context may allow reidentification. Privacy classification requires the actual threat and legal context, not the name of a transformation.

Trace/Debug: Deleted Data Reappeared After Restore.

The restore procedure reintroduced retained backup state without replaying deletion policy before serving traffic. Design restoration controls and retention jointly, and validate projections and caches too.

Scenario: Does Hosted Card Collection Guarantee PCI Compliance?

No. It can reduce exposure and scope, but actual obligations depend on the integration and responsibility model. Verify against current PCI SSC/provider guidance and qualified assessment; do not infer a compliance result from architecture alone.

09 / The Browser and the Java Backend

The UI is a replica of selected server facts, not their authority. It holds form edits, cached responses and optimistic predictions while requests complete out of order. A tenant switch or network reconnect changes which results may still be applied. IntegrationHub is fictional; its React console illustrates these boundaries rather than a real company's frontend.

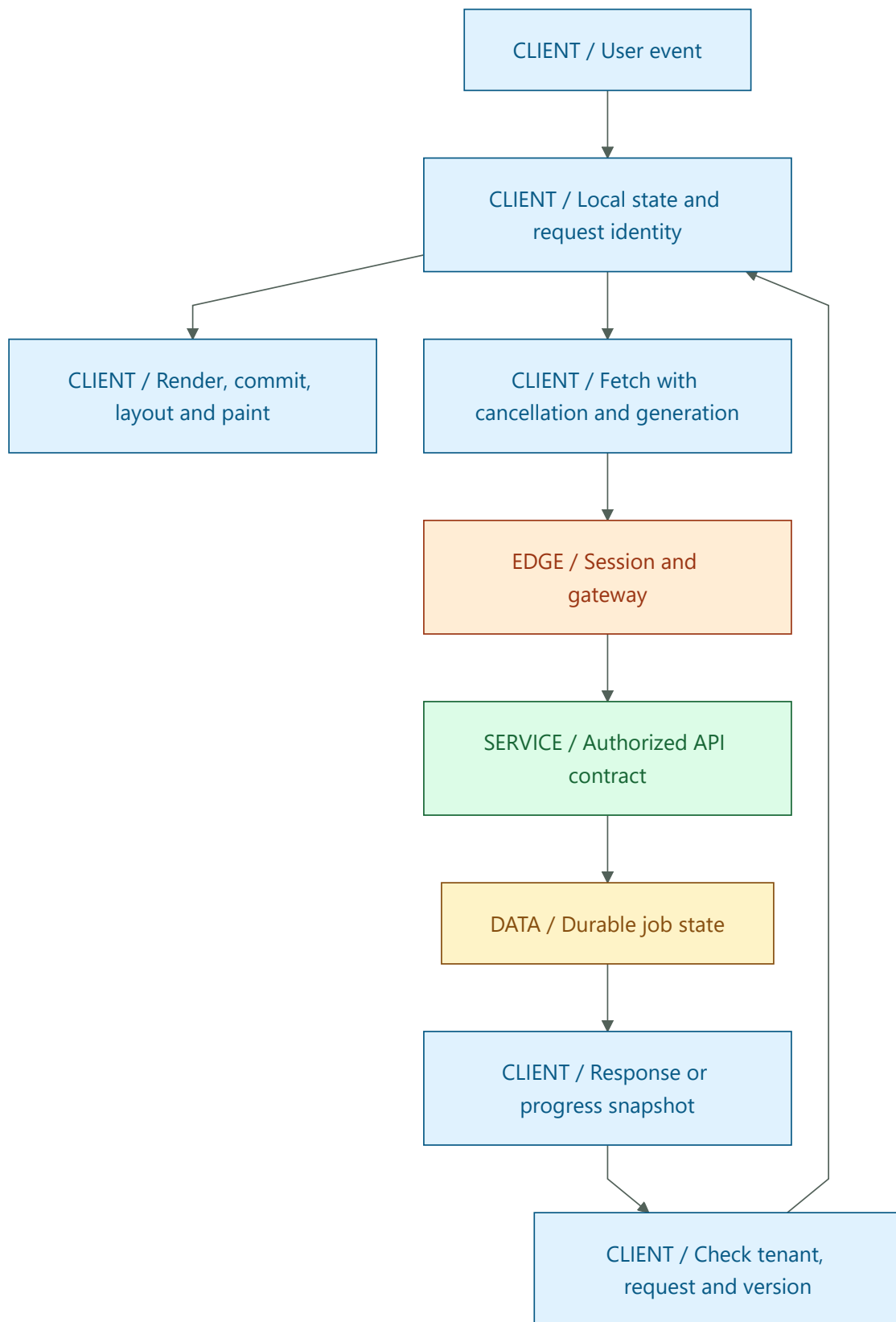
Version assumptions: modern evergreen browsers, ECMAScript modules, TypeScript 5.x, React 19.x and a contemporary Angular release for comparison. The backend remains Java 21, Spring Boot 4.0.x / Cloud 2025.1.x, PostgreSQL 17, Kafka 4.x and Kubernetes 1.34. Framework patch versions are not resolved locally. This is an explanatory chapter and a pure JavaScript lab, not a newly deployed frontend application.

Execution status: the dependency-free ViewStateLab.mjs ran on the existing Node-compatible runtime v24.20.0 and passed five deterministic contract checks. React rendering, Angular, TypeScript compilation, real HTTP and browser SSE behavior are NOT EXECUTED. No package downloads were attempted.

[VERIFY: confirm React/TypeScript/Angular versions, React Compiler adoption, framework scheduling/effect behavior and target-browser APIs before applying examples to a product. The local lab validates a pure state model, not installed framework integration.]

Big Picture

Diagram 09.1



What You Will Be Able to Explain

- Trace document loading, HTTP caching, JavaScript execution, rendering and user interaction.
- Explain values, references, closures, promises and runtime validation at a Java backend boundary.
- Distinguish TypeScript's compile-time checks from runtime schemas and Java's server-side validation.
- Explain React render/commit, state identity, hooks, effect cleanup and server-state fetching without assuming render equals DOM mutation.
- Compare React composition with Angular templates, DI, signals and RxJS at an architectural level.
- Design login, tenant switches, optimistic writes and SSE reconnect without stale data crossing identities.
- Investigate user-visible performance using network, main-thread and rendering evidence rather than bundle size alone.

[P] PRODUCTION

Where this lives in IntegrationHub. The React console submits jobs to 05's API, authenticates through 08's same-origin BFF, and displays 07's durable progress. 06 remains authoritative. 21 explains connection behavior; 18 correlates browser latency with backend traces and 19 owns end-to-end tests.

1. Browser and HTTP Fundamentals

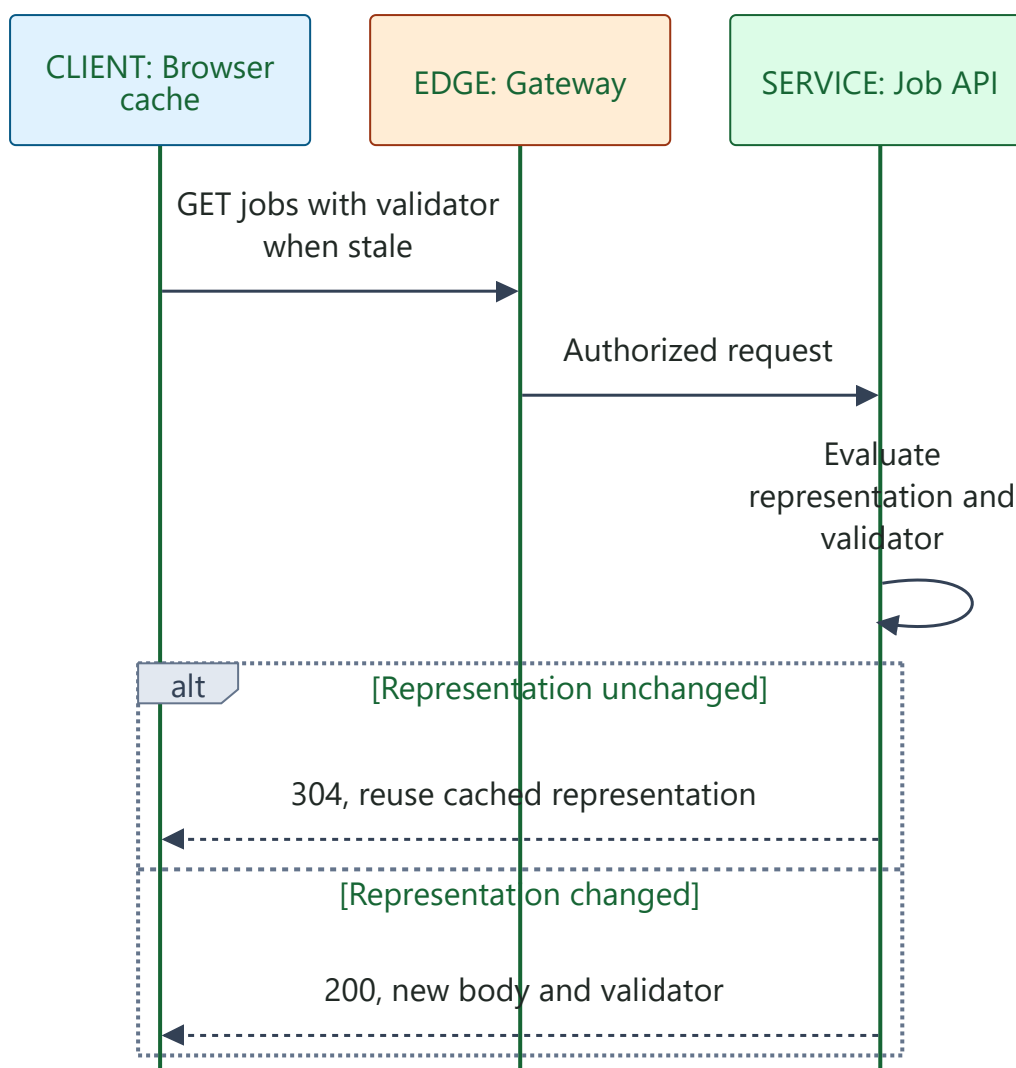
A navigation obtains a document through DNS, transport and TLS, then parses HTML into a DOM and styles into CSS rules. Scripts, styles, fonts and images create more work and network dependencies. Layout determines geometry; paint and compositing produce pixels. Not every change repeats every stage, but a large synchronous script can prevent the main thread from responding even when all backend requests are fast.

The browser's event loop runs tasks and performs microtask checkpoints. Promise callbacks are scheduled as microtasks; they do not run on a new application thread simply because the function is async. Long synchronous work before or between awaits still blocks the main thread. Workers can move suitable CPU work off-thread, but cannot freely manipulate the document DOM and have data-transfer costs.

HTTP caches may reuse fresh responses or revalidate stale ones with validators such as ETags. A 304 allows reuse of the cached representation; it is not a new JSON response body. Cache-Control private and shared-cache policy matter for personalized data. Include the correct

identity and vary dimensions in cache keys, and do not let a CDN reuse one tenant's authenticated response for another.

Diagram 09.2



Browser HTTP failure has several forms: transport rejection, abort, HTTP error status and invalid response body. Fetch normally resolves for an HTTP 404 or 500; the caller must inspect status. A proxy can return HTML for an error, so parsing every response as JSON can obscure the real failure. Network cancellation does not establish that the server rolled back a submitted job.

Mechanism	Use when	Avoid when
Conditional HTTP read	Server state changes less often than clients inspect it	Every refresh downloads the same large payload
Web Worker	Measured CPU processing blocks interaction and is transferable	Network I/O is blamed for main-thread CPU work

Mechanism	Use when	Avoid when
Shared cache	Public representations have explicit cache identity and policy	Authenticated tenant responses are reused across callers

Interview checks: basic: async is not automatically parallel CPU execution. Internals: promise continuation and rendering share scheduling constraints. Debug: inspect status before parsing. Scenario: cache identity includes the security context, not only the URL path.

2. JavaScript: Values, Closures and Asynchronous Work

JavaScript objects and arrays are mutable references. Copying an object with spread copies its immediate properties, not every nested object. A state update that mutates a nested object shared by previous state can make comparisons and historical snapshots misleading. Primitive values behave differently; strict equality avoids coercion surprises but does not make object identity equivalent to structural equality.

Closures retain lexical bindings. In React, each render creates values and callbacks associated with that render. A delayed callback can use an old prop or state value if the design does not account for it. This is not Java-style shared volatile state; it is a question of which render's values the callback captured and how current work should be identified.

Promises represent eventual completion. Await sequences dependency, while Promise.all starts no work itself: it observes the promises supplied to it. Creating all requests for a million items and passing them to Promise.all is unbounded admission, even if the network eventually limits connections. Bound concurrency and batch work according to the [02 concurrency principles](#).

Cancellation is cooperative. AbortController can signal fetch and other supporting APIs; a promise that ignores the signal can still resolve. Even a cancelled fetch may have caused a server-side effect before cancellation. Correct UI state uses request identity/generation checks in addition to aborting obsolete work. Do not rely on timing alone to decide which response is current.

Errors should preserve meaning across layers. A rejected promise is not always retryable; malformed data, forbidden access and validation conflicts need different states. Avoid an empty catch that turns failure into an empty successful list. Empty, loading, error and ready-with-data are different states that help a user reason about what is known.

JavaScript choice	Use when	Avoid when
Immutable state replacement	Render logic depends on explicit state changes	Mutated shared objects make old and new state indistinguishable

JavaScript choice	Use when	Avoid when
Promise.all	A bounded set of independent tasks must complete together	It creates unbounded requests or masks partial-result policy
Abort plus identity guard	Navigation or filters make older requests obsolete	Abort alone is treated as proof no late completion can arrive

Interview checks: basic: spread is shallow. Internals: callbacks capture a render's values. Debug: a late response can be valid data for the wrong current screen. Scenario: bound admission before composing asynchronous results.

3. TypeScript and the Java Contract Boundary

TypeScript adds compile-time structure to JavaScript. Types are erased from the emitted program, so declaring that a response is Job does not validate the JSON. The network boundary still needs parsing, runtime schema checks where appropriate, allowed enum handling and safe failure behavior. Generated clients help maintain shapes; they do not prove that every deployed server matches the document or that authorization succeeded.

Use discriminated unions to represent meaningful states such as idle, loading, ready and failed, with fields available only in the relevant case. Avoid an object full of independent booleans that permits impossible combinations like loading=false, success=true and error present. Prefer unknown at untrusted boundaries over any, then narrow through validation. A type assertion silences a compiler check; it does not create evidence.

Java and JavaScript numbers differ. JavaScript Number exactly represents integers only within its safe integer range, while Java long can exceed it. Serialize identifiers as strings when range or semantics demand it; do not let a large job ID round into a different identity. For money use an explicit minor-unit/decimal contract with currency and rounding rules, not casual floating-point arithmetic. Date-time contracts need timezone/offset and unit definitions, not a number called time that one side interprets as seconds and another as milliseconds.

Optional, absent and null fields should have documented meanings. A PATCH may distinguish leave unchanged from clear value. A client that blindly converts absent values to null can erase data. Unknown enum values need a safe fallback rather than a crash, especially when the backend deploys before cached frontend assets are refreshed.

Type technique	Use when	Avoid when
Discriminated state union	UI states have different valid fields	Many independent booleans encode impossible combinations

Type technique	Use when	Avoid when
Runtime schema validation	External data needs trustworthy shape before use	A TypeScript assertion is mistaken for input validation
Generated API types	A reviewed OpenAPI contract should reduce drift	Generation is assumed to test behavior or authorization

[VERIFY: check TypeScript strictness flags, JSON-schema/OpenAPI tooling, unknown-field handling and the Java serializer's numeric/date conventions in the selected versions. The chapter describes contracts; no TypeScript compilation or generated-client integration ran.]

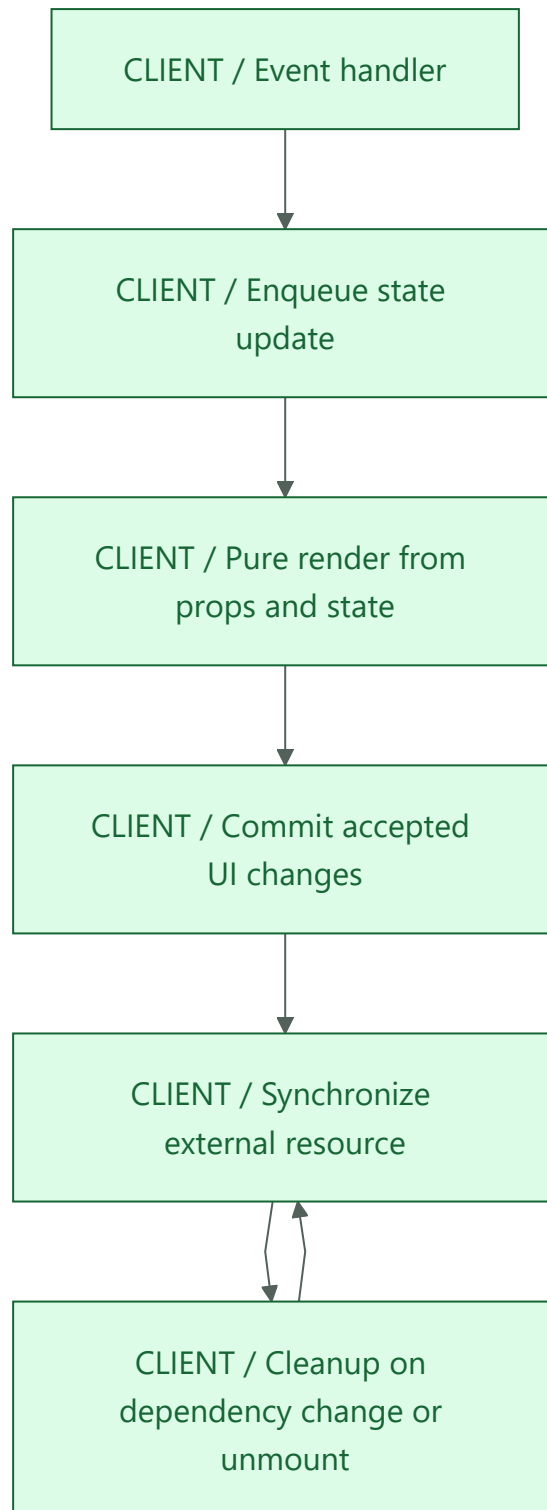
Interview checks: basic: types are erased. Internals: narrowing requires evidence from runtime values. Debug: investigate precision and timestamp units across Java/JavaScript. Scenario: contract evolution includes nullability and enum fallback, not just field names.

4. React Rendering, State and Identity

React components describe UI for given props and state. Rendering computes that description; committing applies the accepted changes to the host environment. A render can be repeated, interrupted or discarded under the framework's scheduling model, so rendering must stay pure: do not submit a job, mutate a global registry or open a socket merely because a component function ran.

State belongs to a component's position and identity in the tree. Keys tell React which sibling represents which logical item. A stable job ID makes sense for a job row; an array index can attach edit state to the wrong row after sorting or insertion. Changing a key deliberately resets state, which is useful for a new editing identity but destructive if used accidentally on every render.

Diagram 09.3



State updates are scheduled and may be batched. Reading a state variable immediately after requesting an update does not make it the next render's value. Functional updates derive the next state from the previous queued state and avoid lost increments when several updates depend on prior state. A reducer is useful when transitions must preserve invariants across related fields; it is not mandatory for every input field.

Separate local interaction state from server state. A modal's visibility, draft filter text and unsaved form belong locally. A job's accepted status and version come from the server and need freshness, invalidation and error policy. Duplicating the same server field into several independent local states creates reconciliation work and stale views. Derive inexpensive display values from existing state rather than storing redundant copies.

State arrangement	Use when	Avoid when
Local component state	One component owns an interaction	It duplicates shared authoritative server data
Lift state to common owner	Siblings coordinate one value	Everything becomes global regardless of lifetime
Reducer	Related transitions need explicit invariants	A trivial boolean needs a framework within the framework
Server-state cache	Requests need deduplication, freshness and invalidation	The cache key omits tenant or filters

Interview checks: basic: render is not commit. Internals: keys preserve logical state identity. Debug: a reordered list with wrong edit state suggests index keys. Scenario: a reducer makes valid transitions explicit while the server stays authoritative.

5. Hooks, Effects and Resource Lifetimes

Hooks must follow the framework's ordering rules. Do not call ordinary state/effect hooks conditionally inside branches whose order changes across renders. Custom hooks share logic, not automatically one shared state instance. A hook named `useJobs` invoked twice can establish two subscriptions unless its implementation or a shared cache deliberately deduplicates them.

An effect synchronizes with an external system such as a subscription, timer or imperative widget. Dependencies describe values used by that synchronization. Omitting a dependency to silence a linter can leave old tenant identity or callbacks active. The cleanup must undo the established resource: close the old `EventSource`, remove listeners, clear timers and abort obsolete requests before a new scope becomes authoritative.

Effects are not the default place for everything. User commands such as `submit` should normally originate from the corresponding event handler. Values computable during rendering should be derived there when cheap. An effect that copies props into state can create a delayed extra render and two sources of truth. Measure before introducing memoization; `useMemo` and `useCallback` are optimization tools, not correctness mechanisms or a requirement on every function.

Development Strict Mode can intentionally exercise setup/cleanup and rendering to expose unsafe assumptions. Code must tolerate cleanup and re-setup without leaking a subscription or repeating an irreversible action. This does not mean every production effect always runs twice. An effect that submits a payment is unsafe because resource synchronization is the wrong owner for that command, not because the development check is inconvenient.

Transitions can mark eligible updates as nonurgent, and deferred values can let expensive derived views lag urgent input. They do not cancel HTTP requests, move CPU work to a worker or make an unbounded list cheap. APIs such as `useEffectEvent` have specific semantics and linting constraints; follow the team's adopted React version and compiler guidance rather than adding them by name.

[!] **TRAP**

Cleanup is part of the contract, not housekeeping. A tenant switch that leaves the old subscription active can display data from the previous scope. Closing the resource and checking incoming identity/version together protect the UI state; neither substitutes for backend authorization.

Interview checks: basic: effects synchronize external resources. Internals: dependencies and cleanup define lifetime. Debug: duplicated subscriptions often indicate setup without matching teardown. Scenario: commands belong to intentional user/workflow events, not arbitrary renders.

6. Data Fetching, Errors and Optimistic Writes

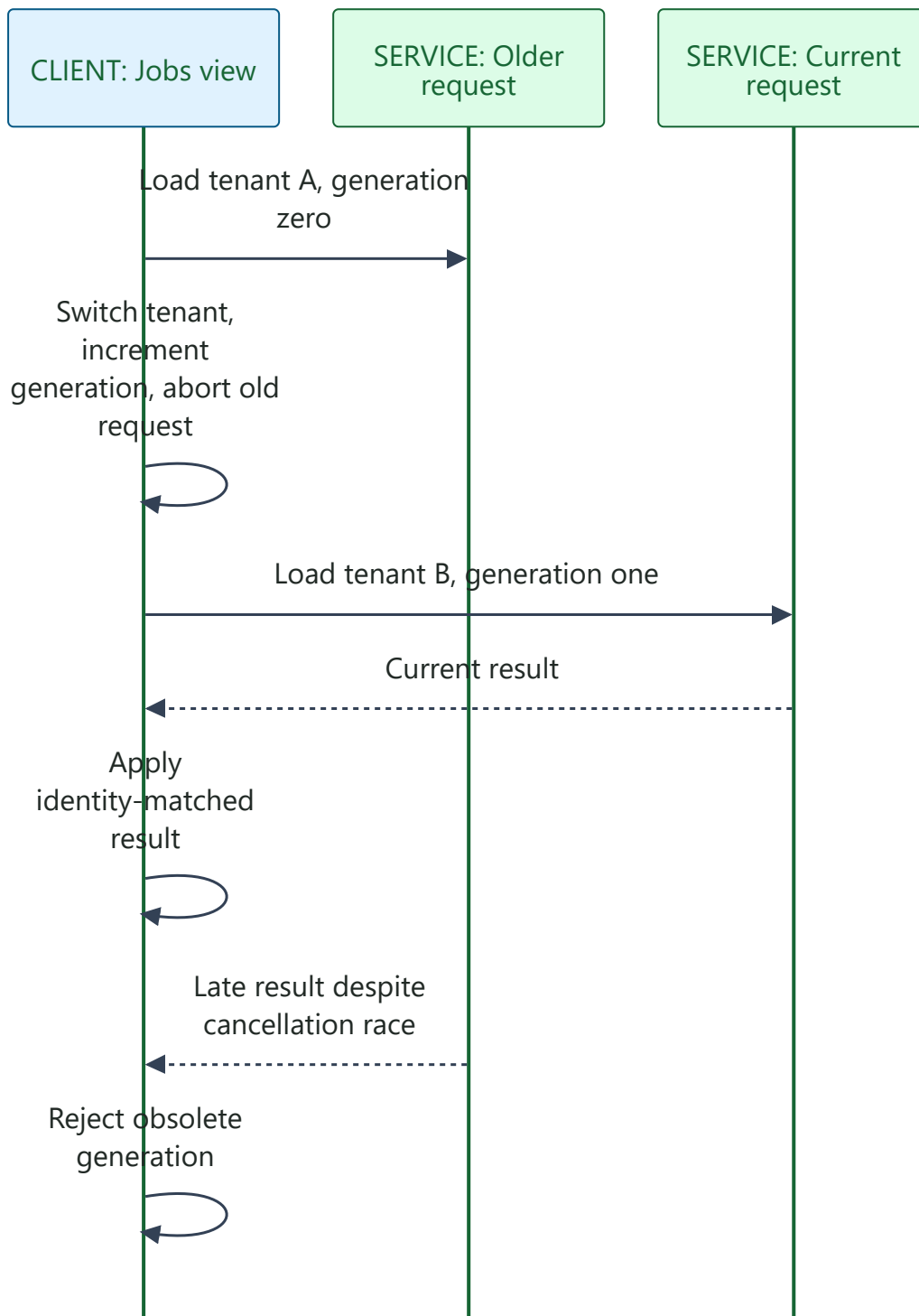
A query cache key should include every dimension that changes the result: tenant, resource, filters, ordering and pagination cursor. Authorization context may require cache reset or partitioning even if a URL is unchanged. Distinguish stale data from absent data; a background refresh can retain useful prior results while showing an update state, but must not retain another tenant's data during a switch.

The lab assigns a monotonically increasing request ID and a generation that changes whenever the tenant context changes. A response applies only when both still match. The generation matters when a user switches A to B and back to A: tenant name alone would accept an old A response from the earlier session. Request ID handles repeated queries inside one generation.

[M] MECHANISM

Response acceptance is a state transition with a guard. Compare the captured tenant, generation and request identity with the current state before applying a completion. Abort reduces obsolete work, while the guard makes an obsolete result unable to replace current data even if cancellation was ignored.

Diagram 09.4



Server-state libraries can handle much of caching, deduplication, cancellation and invalidation. Adopt one consistent query policy rather than hand-writing separate caches in every component. Still understand mutation races: invalidating a list after submit may refetch from an asynchronous replica that has not seen the write. Use the accepted job response and authoritative status endpoint under [05's contract](#), then reconcile the list.

Optimistic updates predict a server outcome before confirmation. They fit reversible interactions when rollback and conflict behavior are understandable. A submitted long-running job can appear as submitting, then accepted with its durable identity; it should not appear completed merely because POST resolved. Preserve the idempotency key across retries of the same logical submission. A user editing the command into a different request should establish a different logical identity or receive a content-binding conflict.

Fetch/mutation choice	Use when	Avoid when
Keep previous results during refresh	Same authorized scope and stale data is useful	Tenant/user changes leave old private rows visible
Optimistic update	Reversible actions have a clear reconciliation policy	Irreversible effects are shown as confirmed before the server accepts them
Query cache invalidation	A mutation changes known cached resources	Every query is globally reset without regard to cost or scope
Retry with stable key	The same logical command has an unknown outcome	Every network attempt gets a new idempotency key

Interview checks: basic: HTTP errors need explicit status handling. Internals: request and generation identities solve different races. Debug: compare response context with the current view context. Scenario: keep unknown outcome separate from definite rejection.

7. SPA Authentication and Session Changes

The baseline browser uses a same-origin secure HttpOnly session through a BFF. The BFF performs OIDC code/PKCE handling and stores tokens server-side. State-changing cookie requests need CSRF protection. A pure SPA can use code plus PKCE as a public client, but must deliberately handle token lifetime, storage, refresh and cross-origin policy. [08](#) owns the protocol details.

Do not hide unauthorized buttons and call that authorization. The backend must reject forbidden operations regardless of UI state. The UI can present permitted actions for usability, but entitlements may change between render and submit. Handle 401 as an authentication/session concern and 403 as a permission decision without endlessly refreshing credentials for a policy denial.

Logout and tenant changes must close subscriptions, invalidate in-flight generations, clear or partition caches and remove stale optimistic records. Server logout/session revocation and provider single sign-out are distinct operations; document what actually ends. Multiple tabs need coordination for session changes and refresh to avoid inconsistent displays and refresh races. Communication between tabs should carry state-change notifications, not raw credentials.

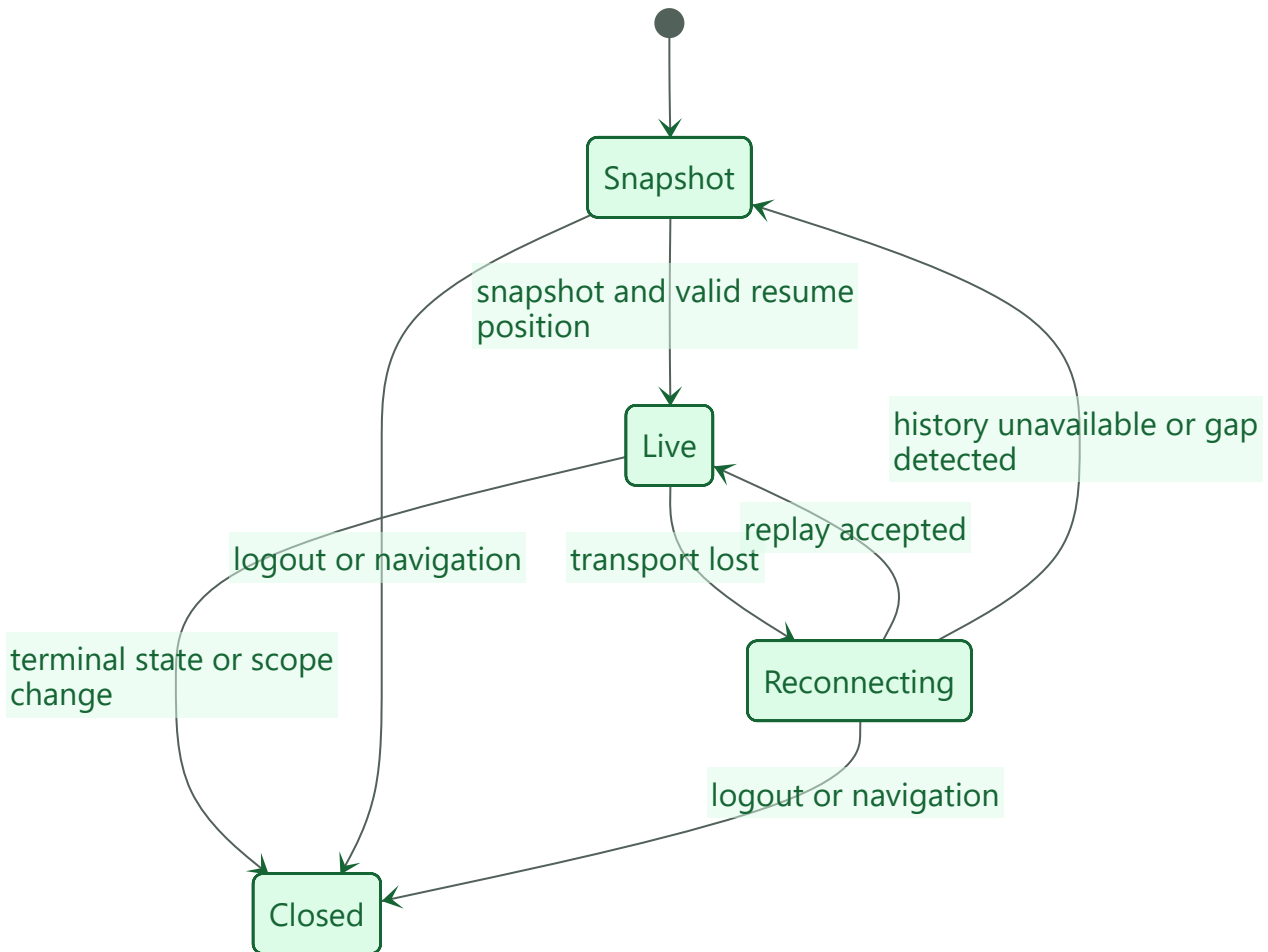
Session approach	Use when	Avoid when
Same-origin BFF session	Server-managed credentials and browser simplicity fit the deployment	CSRF and BFF session-store security are ignored
Public SPA with PKCE	Direct API access and identity-provider support justify browser token handling	A client secret is embedded in the JavaScript bundle
Entitlement-driven controls	UI should avoid offering invalid actions	Hidden controls replace server authorization

Interview checks: basic: frontend permission checks are usability, not enforcement. Internals: logout includes cache/subscription lifecycle. Debug: repeated refresh on 403 is a category error. Scenario: a tenant switch is an identity-generation change, not just a filter update.

8. SSE, Snapshot Recovery and Slow Rendering

Live progress is a view of committed server state. The client needs an initial snapshot with a position, ordered updates within the stream's scope, duplicate/stale-version handling and a recovery path when retained history cannot satisfy its cursor. The server must arrange a gap-free snapshot/subscription boundary or detect a gap and require resynchronization; reading a snapshot and then subscribing later can otherwise miss changes in between.

Diagram 09.5



Progress snapshots with increasing versions can replace older snapshots. Replayed version 8 after version 9 must not regress displayed progress. That policy is not valid for arbitrary deltas: dropping a missing intermediate delta could lose work. The lab only checks versioned cumulative snapshots and says so explicitly. Server-side validation must also prevent invalid terminal-state transitions; frontend version guards are not business correctness.

EventSource provides browser reconnection and Last-Event-ID handling within its contract, but the application owns retention fallback and authorization. Do not put long-lived bearer tokens in URLs. A fetch-based streaming client permits headers but requires correct incremental frame parsing, cancellation and reconnect handling. TCP chunks are not event frames; use a tested SSE parser rather than splitting arbitrary received chunks on assumptions about boundaries.

High-frequency events can overwhelm rendering even when the network and server are fine. Coalesce disposable progress snapshots to an appropriate display cadence, bound queued work and avoid rebuilding an entire table for every percentage update. Do not silently coalesce audit records that each represent a distinct business fact. Close a terminal job's stream under a documented policy rather than leaving it reconnecting forever.

[VERIFY: confirm EventSource/browser reconnect behavior, fetch-stream parsing support, credentials/origin policy and the backend snapshot/replay protocol against target browsers and gateways. ViewStateLab checks snapshot versions only; it does not exercise SSE transport or React rendering.]

Interview checks: basic: cursor requests history, not infinite replay. Internals: snapshot-to-stream handoff can have a gap. Debug: growing client queues can be rendering backpressure. Scenario: distinguish cumulative snapshots from deltas before discarding older events.

9. Angular Overview for a Java Engineer

Angular combines templates, components, dependency injection, routing, forms and HTTP facilities in a more integrated framework. Services injected into components should have deliberate scopes; a root-provided service can share state across screens, while a narrower provider can create isolated instances. This resembles familiar Java DI concepts but follows Angular's own injector hierarchy, not Spring bean semantics.

Signals model reactive values and derived computations; RxJS Observables model streams of values with completion, cancellation/unsubscription and operators. They are different abstractions that can interoperate. An HTTP observable can be cold, meaning subscription triggers work; multiple subscriptions can duplicate a request unless the design shares it intentionally. Unsubscribing from client work still does not prove a server mutation never happened.

Change detection determines which bindings must be checked and rendered. Immutable state, stable identities and suitable change-detection strategy can reduce unnecessary work, but exact behavior depends on the Angular version and whether zone-based or newer scheduling modes are adopted. Do not assume every event forces a full repaint, or that a performance flag fixes unstable input identities.

Framework direction	Use when	Avoid when
React with selected libraries	The team values composition and owns conventions explicitly	Every feature invents a different fetching/form/state stack
Angular integrated tooling	Standardized DI, routing/forms and framework conventions fit the team	Familiar Java syntax is the only justification
RxJS stream composition	Cancellation, combination and time-based events need operators	A simple value is wrapped in an elaborate subscription graph

[VERIFY: verify Angular signals, change-detection mode, HTTP observable cancellation and lifecycle cleanup APIs against the selected major release. Angular is an architectural comparison here; no Angular application or dependency set was installed.]

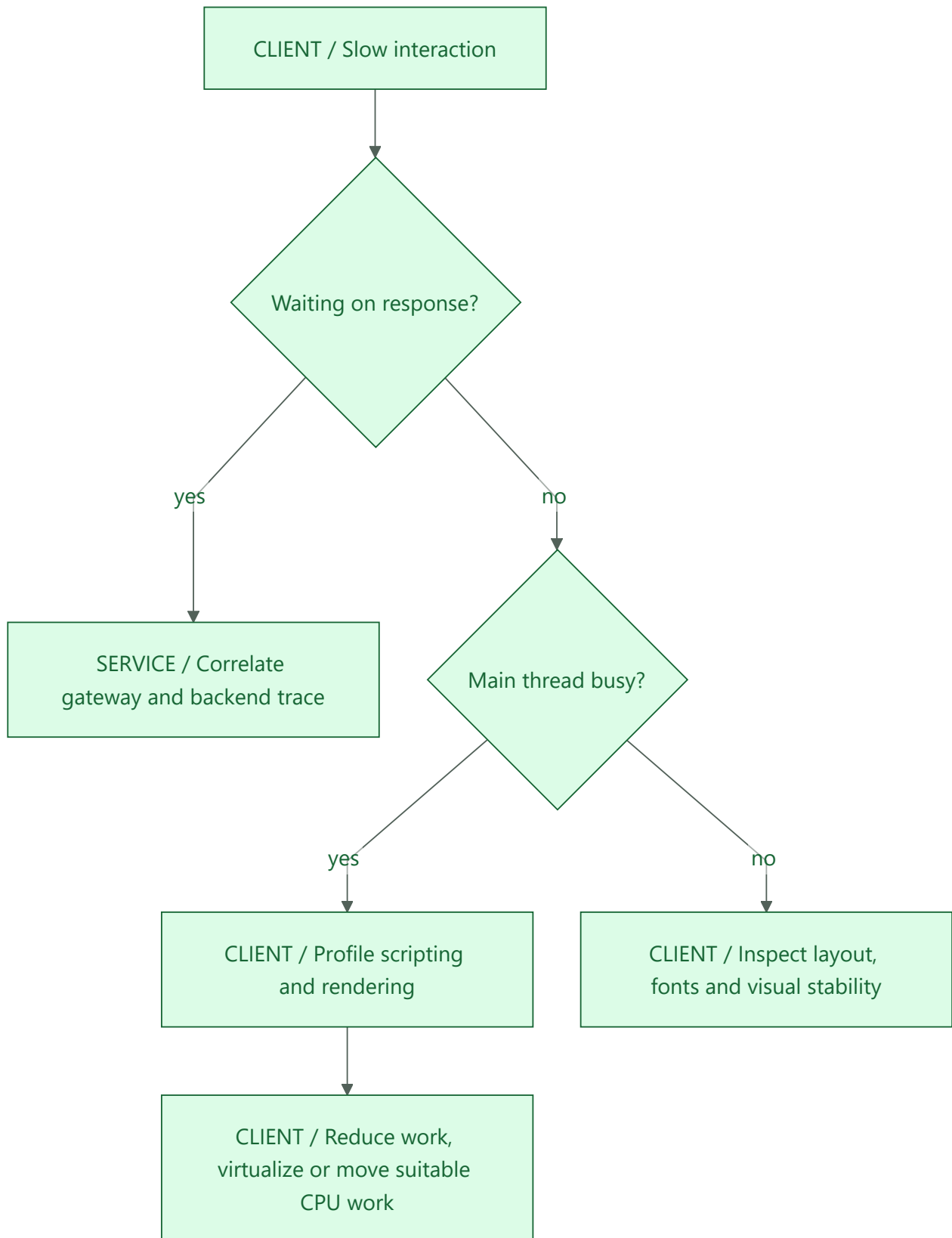
Interview checks: basic: Angular DI scopes affect shared state. Internals: subscribing can start new work. Debug: duplicate requests may be multiple subscriptions. Scenario: choose team conventions and workload fit, not a claim that one framework eliminates state problems.

10. Performance, Accessibility and Testing

Start with the user's operation: initial load, typing a filter, opening a job, scrolling a table or receiving progress. Break elapsed time into network, server, JavaScript, layout/paint and waiting on other resources. A small bundle can still render an expensive table; a fast API can still deliver too much JSON to parse and render on a slow device.

Measure field experience and controlled traces. LCP concerns when substantial content becomes visible; INP concerns interaction responsiveness; CLS concerns unexpected layout shift. Percentiles and device/network segmentation matter. A lab trace helps explain a problem but does not replace real-user observations, and telemetry must avoid leaking URLs or identifiers containing sensitive data.

Diagram 09.6



Code splitting can defer rarely used routes; uncontrolled splitting can add request waterfalls. Virtualization renders only visible rows but needs stable keys, focus and screen-reader considerations. Pagination bounds data transfer; virtualization alone does not stop downloading a million rows. Images and media need dimensions to avoid layout shifts, and render-blocking font/style decisions can dominate first display.

Accessibility is part of functional correctness: semantic controls, labels, keyboard navigation, visible focus, meaningful error association and non-color-only status. Loading/error announcements should be useful without reading every progress update aloud. Preserve focus through updates and test zoom/reflow, not only a screenshot at one desktop size.

Testing layers should match risks. Pure reducer tests validate transition logic and races with deterministic fake completions. Component tests validate interaction, accessible names and cleanup. Contract tests validate DTO/status behavior. Browser tests validate real routing, cookies, CORS, keyboard behavior, reconnect and server effects. Do not mock away the gateway when the failure is proxy buffering.

[VERIFY: confirm current Core Web Vitals definitions, measurement tools, browser support and accessibility requirements appropriate to the product. No field performance measurements, accessibility conformance audit or framework/browser end-to-end test is claimed by the pure-model lab.]

[D] DECISION

Optimize the measured stage. Network caching does not repair synchronous rendering work, and memoization does not repair an authorization or stale-response race. Change the layer that consumes the budget or violates the contract.

11. Executed View-State Lab

The complete dependency-free source is `code/09-frontend/ViewStateLab.mjs` ; its Windows PowerShell entry point is `& './docs/java-fs-guide/code/09-frontend/run.ps1'` from the workspace root. It uses an existing Node runtime or the same VS Code runtime fallback as the publisher and restores its environment afterward. No administrator rights, server or downloads are needed.

The reducer carries tenant, generation, request and progress version explicitly. Its loader aborts old work and still rejects obsolete completions. The fake transport exposes promises whose resolution order the test controls; it intentionally ignores abort so correctness cannot rely on convenient cancellation timing. There are no sleeps or fabricated latency figures.

Executed check	What passed	Scope limit
Older request response	Cannot overwrite the current request	No network stack involved
Tenant switch and switch-back	Earlier generation is rejected even for the same tenant name	Not backend authorization
Progress duplicates and invalid versions	Current snapshot does not regress	Not ordered application of deltas
Reverse completion order	Current tenant data survives old completion despite ignored abort	Not fetch/browser cancellation behavior
Disposal	Late completion is ignored	Not a React effect/unmount test

The generated execution.json records PASS for these five checks on v24.20.0. This result does not establish React behavior, DOM accessibility, TypeScript compilation or security. Those remain NOT EXECUTED. The model is deliberately small: production code needs schema validation, cache lifetime, pagination, user-visible errors and the complete session policy.

[?] INTERVIEW

Explain stale UI with identities, not timing guesses. Ask which user/tenant generation, resource, request and server version a result belongs to. Then show which comparison makes the obsolete result unable to mutate the current view.

Explained-Here Index and Sources

Concepts	Explanation
Browser and HTTP	Loading, caches and event loop
JavaScript	References, closures and promises
TypeScript	Compile-time versus runtime contracts
React, state and rendering	Identity and render/commit
Hooks	Synchronization and cleanup
Data fetching	Request identity and optimistic writes
SPA authentication	Sessions and tenant transitions
Real-time UI	Snapshot and reconnect
Angular	Framework comparison
Frontend performance	Measured user experience

Verification references: [React learn](#), [TypeScript handbook](#), [MDN Fetch](#), [Angular documentation](#), [Web Vitals](#) and [WCAG](#). These are owning references rather than claims of local framework execution.

Related Chapters

Return to 00. Read 02 concurrency, 05 APIs, 06 database authority, 07 progress delivery, 08 security, 10 caching and capacity, 18 telemetry, 19 testing and 21 network. Related navigation is reciprocal.

RELATED CHAPTERS / RECIPROCAL LINKS

00	The System Before the Details	18	Observability and Reliability
04	Hibernate and JPA	19	Testing and Quality
06	Databases and Durable State	02	Concurrency and the Memory Model
05	APIs and Real-Time Communication	10	System Design Foundations
08	Security, Identity and Data Protection	12	Twelve System Design Case Studies
07	Messaging, Batch and Streams	24	IntegrationHub in Motion
21	Networking and Operating Systems	20	Appendix: Interview Toolkit and Index

One-Page Cheat Sheet

Browser: async does not make CPU work parallel. Inspect transport, HTTP status and body parsing separately. Cache policy and keys include identity. Main-thread work can make a fast backend feel slow.

Types and values: object spread is shallow; callbacks can retain old render values. TypeScript types are erased. Validate network data, preserve large IDs as appropriate strings, and specify time units/null semantics.

React: render is pure computation, commit applies accepted changes. Stable keys preserve identity. Separate local interaction from server state. Effects synchronize external resources and must clean up; commands belong to deliberate events. Measure before memoizing.

Requests: abort obsolete work and guard results by tenant, generation and request ID. A switch-back is a new generation. Optimistic state is a prediction until the server confirms it. Retry the same command with the same idempotency key.

Sessions and streams: BFF cookies still need CSRF defenses. Clear caches and subscriptions on identity changes. Snapshot/replay needs a gap-free boundary, version guards and retention fallback. Coalesce snapshots only when semantics permit it.

Evidence: pure state tests passed; React, Angular, TypeScript and browser integration did not run. Profile the actual slow stage and test keyboard/focus/error states, not just the happy screenshot.

Interview Corner

Basic: Does TypeScript Validate JSON?

No. Types disappear at runtime. Validate untrusted responses with runtime checks or an appropriate schema library before relying on a declared shape; a type assertion is not validation.

Internals: Why Can a Callback See Old State?

It may close over values from the render that created it. Model the intended lifetime, use correct effect dependencies or appropriate update patterns, and avoid pretending every callback reads one mutable global state cell.

Trace/Debug: A New Filter Shows Old Results.

An older request may complete last and overwrite the new one. Inspect query keys and request/generation guards. Abort helps resource use, but identity checks protect correctness when cancellation races or is ignored.

Scenario: Switching Tenant A to B and Back to A.

Increment a context generation on every switch, abort/close old work and clear scoped data. Tenant equality alone cannot distinguish an old A response from the new A session. The executed lab tests exactly that distinction.

Basic: Render Versus Commit?

Render computes UI from props and state; commit applies accepted host changes. Render must be pure because it can repeat or be discarded. External resource management belongs in appropriate lifecycle synchronization.

Internals: Why Not Use Array Index Keys?

They identify positions rather than logical records. Inserting or sorting can attach existing component state to a different job. Use stable record identity unless position truly is the intended identity.

Trace/Debug: Why Did an Effect Open Two Streams?

Check dependencies, cleanup, multiple hook instances and development lifecycle checks. Each setup needs a matching teardown. Do not suppress lifecycle checks to hide a resource leak.

Scenario: Should Every Derived Value Use useMemo?

No. Compute cheap values normally; memoize when measurement and the team's compiler/version conventions justify it. Memoization is not a substitute for correct dependencies or immutable state.

Basic: Why Can fetch Resolve for a 500?

HTTP error statuses are still HTTP responses. Check response status and content type before parsing and classify the failure; transport rejection is a different path.

Internals: Does Abort Undo a Submitted Job?

No. It can stop supported client work, but the server may already have committed. Preserve request identity and use the API's idempotency/status contract to resolve unknown outcomes.

Trace/Debug: Progress Goes Backward After Reconnect.

Inspect snapshot versions and duplicate replay. Apply only newer snapshots within the correct tenant/job generation. Do not reuse that discard rule for deltas whose every intermediate effect matters.

Scenario: A Million-Row Table Is Slow.

Bound server results with pagination, then consider virtualization and rendering cost. Virtualization alone does not prevent downloading/parsing the whole dataset. Measure network, scripting and layout separately.

Basic: React Versus Angular?

React emphasizes component composition with chosen ecosystem conventions; Angular integrates more routing/forms/DI conventions. Both still need explicit state ownership, request identity and resource cleanup.

Internals: Why Might Two Observable Subscriptions Send Two Requests?

A cold observable can execute its producer separately per subscription. Share deliberately where appropriate and understand teardown; do not assume a service field automatically caches a result.

Trace/Debug: Fast API, Slow Interaction.

Profile the main thread, expensive rendering, parsing, layout and third-party scripts. Correlate browser timing with backend traces before assuming the server is responsible.

Scenario: What Must Logout Clear?

Relevant server sessions plus local caches, in-flight generations, subscriptions and optimistic data under the actual session policy. Coordinate tabs and avoid showing stale private data while reauthentication occurs.

10 / System Design Foundations

A system design is an argument that a set of constraints can be met under stated failures.

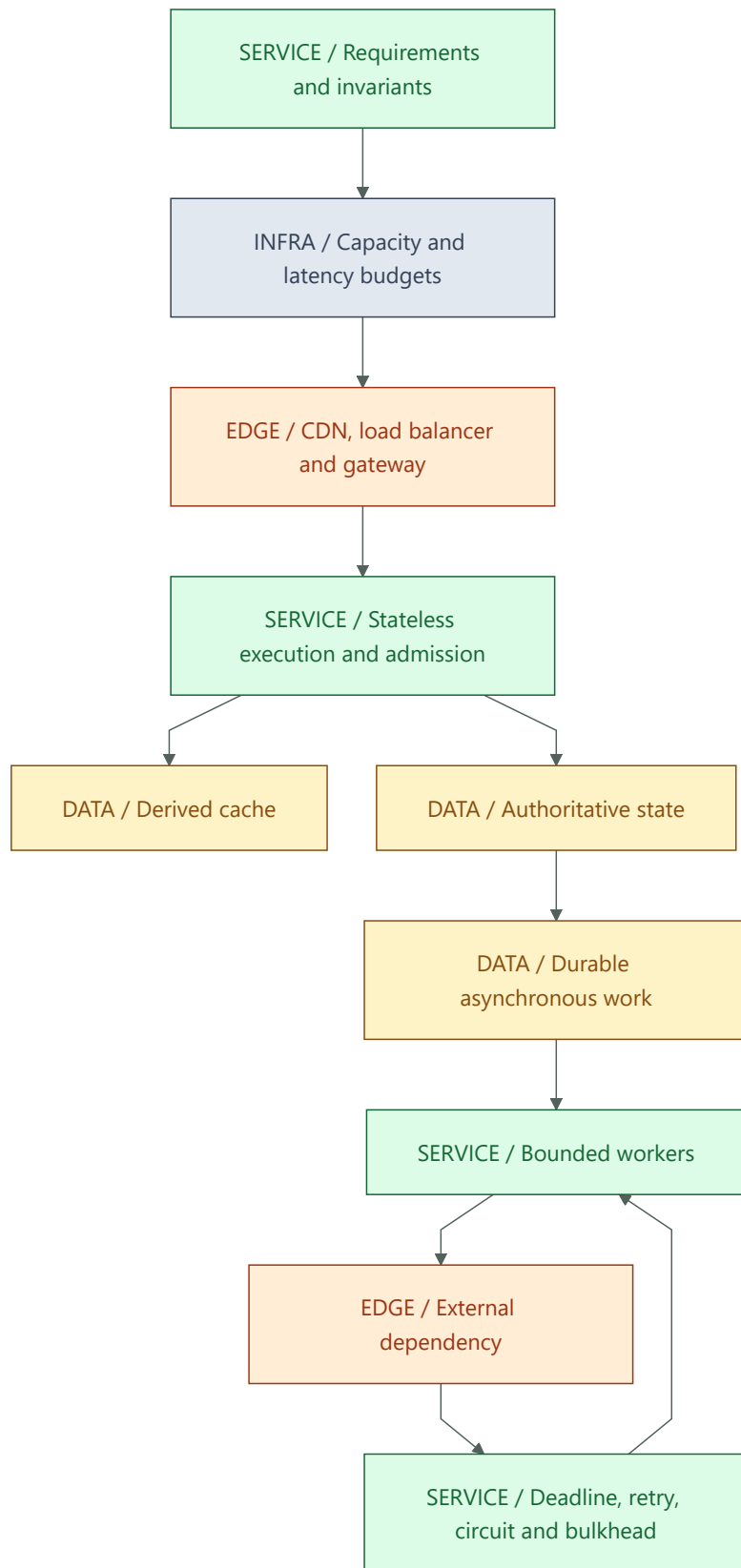
Boxes are the consequence of that argument, not its starting point. IntegrationHub is fictional; every capacity number below is an explicit exercise assumption, never measured production traffic.

Version assumptions: Java 21, Spring Boot 4.0.x / Cloud 2025.1.x, PostgreSQL 17, Kafka 4.x and Kubernetes 1.34 remain the guide baselines. Resilience4j is discussed by policy mechanism, without an installed artifact version. CDN, gateway and managed-database behavior require provider-specific confirmation.

Execution status: CapacityLab.mjs passed four arithmetic/validation checks using the existing Node-compatible runtime. These tests calculate hypothetical capacity, backlog drain time and retry budgets; they are not load tests. Resilience4j, distributed rate limits, replication and multi-region failover are NOT EXECUTED.

Big Picture

Diagram 10.1



What You Will Be Able to Explain

- Translate a product requirement into invariants, freshness, latency, throughput and recovery requirements.
- Estimate order-of-magnitude storage, concurrency and backlog without confusing averages, peaks and percentiles.
- Choose scaling, balancing, caching, queues and partitioning based on the actual constrained resource.
- Explain consistency, CAP and PACELC without the misleading choose-any-two slogan.
- Compose timeouts, retries, jitter, circuit breakers and bulkheads without amplifying an outage.
- Design tenant fairness and multi-region recovery, including which writes must stop rather than split authority.
- Present a repeatable interview answer with assumptions, failure traces and a simpler rejected alternative.

[P] PRODUCTION

Where this lives in IntegrationHub. 05 defines accepted work and caller retry identity; 06 owns local transactions and replicas; 07 owns asynchronous recovery. 08 supplies isolation, 09 supplies user-visible freshness and 12 applies these foundations to twelve designs. Deep distributed theory remains in 22, not hidden behind product names here.

1. Requirements Before Components

Start with actors, commands, reads and the business invariant. For job submission: an authorized tenant can accept one logical submission under a key, observe status and cancel when allowed. The job must not be silently lost after acceptance. A progress screen may lag briefly; tenant authorization must not. Those different requirements imply different consistency and recovery choices.

Clarify scope: one region or many, one tenant size or a skewed distribution, which integrations are external, what is the largest request, and how long must history remain? State availability and recovery objectives with their operation boundaries. An uptime target for the API is different from the fraction of jobs completed within a deadline. A service can return 202 reliably while its worker backlog is growing without limit.

Choose a simple baseline first: one write authority, indexed storage, stateless application replicas and a durable queue only where asynchronous completion is useful. Introduce a component only when a requirement or measured limit needs it. Each additional store adds

ownership, backup, schema, security, monitoring and recovery work. A design that names more products without explaining those obligations is less complete, not more scalable.

Question	Use when	Avoid when
What must never happen?	Identify duplicate money movement, tenant exposure or lost accepted work	Availability targets replace correctness invariants
What can lag?	Choose caches, replicas and projections deliberately	Every read is labelled eventually consistent without a user contract
What must recover, and by when?	Set replay, backup and spare-capacity budgets	A backup schedule is mistaken for a recovery objective
What is outside scope?	Bound the interview or implementation	Scope expands silently until no design can be evaluated

Interview checks: basic: functional requirements differ from quality attributes. Internals: SLO boundaries must match user operations. Debug: green API health can hide stalled work. Scenario: identify the simplest viable authority before introducing sharding.

2. Estimation, Units and Capacity

Use units in every calculation. Requests/second multiplied by seconds yields requests in flight; records/day multiplied by bytes/record yields bytes/day. Keep average, peak, burst and tenant skew separate. A daily average says little about a synchronized nightly import. Percentile latency is not interchangeable with mean residence time in Little's law.

For a stable system and consistent boundaries, Little's law relates mean concurrency to throughput times mean residence time. If we **assume** 2,000 requests/second and 0.125 seconds mean residence, mean in-flight work is 250. This is neither a recommended thread count nor a guarantee about the 99th percentile. Queue time, service time and external waiting must be included consistently with the chosen boundary.

Assume every request produces one 1,000-byte retained record, traffic continues at that rate all day, retention is seven days and there are three copies. That gives 172,800,000 records/day and 3,628,800,000,000 retained bytes, about 3.63 decimal TB. This deliberately excludes indexes, metadata, compression, WAL, temporary rebuild space and backups. The omitted costs need separate estimates; multiplying by replication is not a complete disk forecast.

Backlog drain needs surplus throughput. With **assumed** backlog 120,000 items, arrivals 1,800/s and completions 2,000/s, the net drain is 200/s and recovery takes 600 seconds under constant rates. If arrivals equal completions, the backlog does not drain. Adding replicas helps only if workers, partitions, provider quotas and the database permit extra completion capacity.

Estimate	Use when	Avoid when
Mean concurrency	A stable boundary needs occupancy reasoning	It is used as a tail-latency guarantee or universal pool size
Storage by retention and copies	Establish order of magnitude	Indexes, backups and rebuild headroom are silently excluded
Burst/backlog model	Capacity must recover after downtime	Steady-state throughput is assumed sufficient for catch-up
Per-tenant distribution	Heavy tenants affect fairness and hot keys	Average tenant size hides a dominant customer

[M] MECHANISM

Recovery consumes capacity too. A system provisioned to barely match live arrivals has no spare throughput to replay yesterday's backlog, rebuild an index or restore stream state. Recovery time is constrained by surplus work capacity, not only by how fast a pod starts.

The source `code/10-system-design/CapacityLab.mjs` asserts these numbers and rejects invalid inputs. Its report is observed arithmetic execution, not an observation of traffic. 23 expands sizing and profiling once real measurements exist.

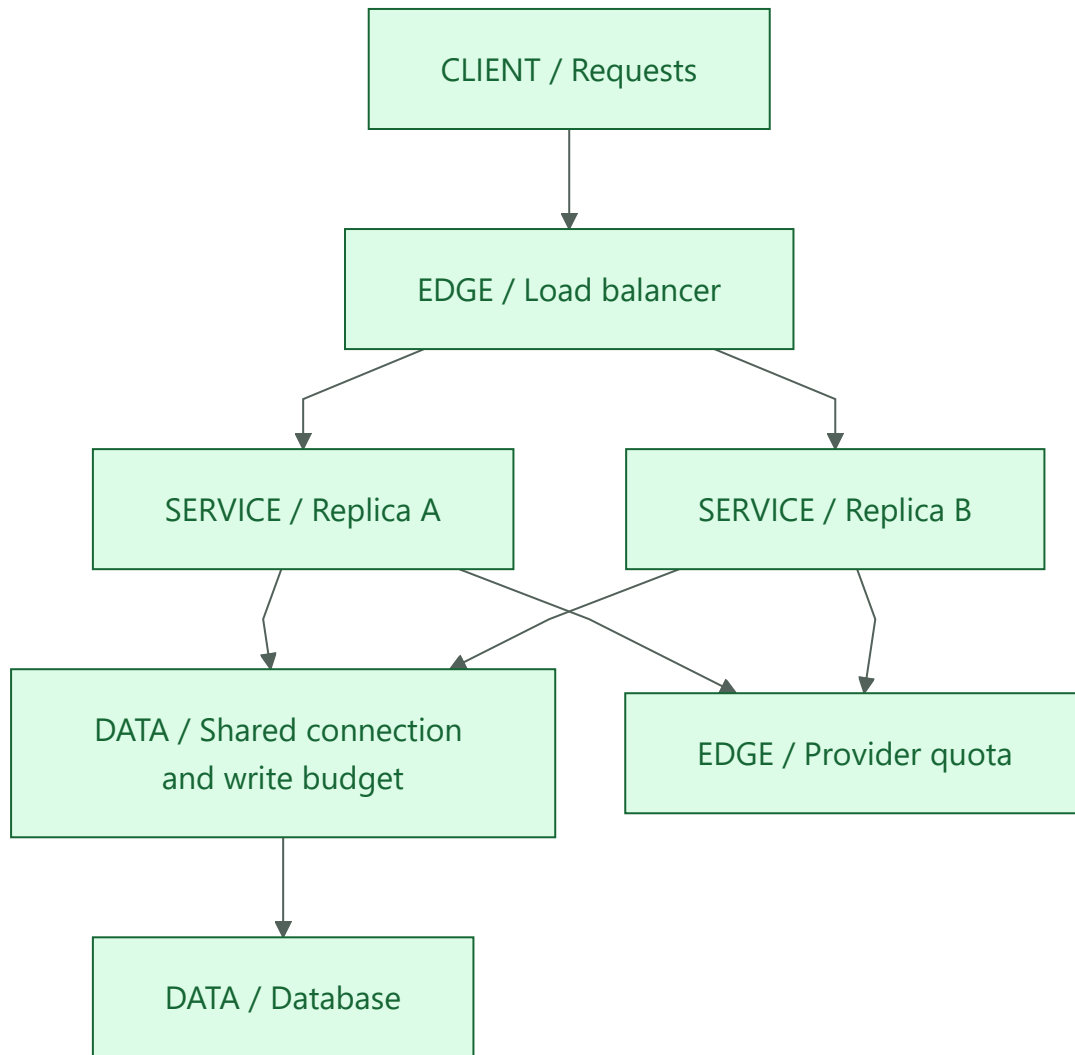
Interview checks: basic: attach units and assumptions. Internals: Little's law uses means under appropriate stability. Debug: lag that never decreases means no net drain. Scenario: reserve capacity for catch-up and failover, not just ordinary demand.

3. Scalability and Load Balancing

Vertical scaling adds resources to one node; horizontal scaling distributes work among nodes. Horizontal scaling is effective when work can be partitioned and shared bottlenecks do not dominate. Stateless HTTP replicas may scale easily while one hot database row or external provider quota fixes the system's maximum throughput. Measure the constrained resource before changing replica count.

L4 balancing routes transport connections without understanding application URLs. L7 balancing can inspect HTTP attributes, terminate TLS and route by host/path. Both require health and connection policies. Round-robin distributes assignments, not equal work cost; least-connections can help with varied connection durations but is not a direct measure of CPU or tenant fairness.

Diagram 10.2



Health checks should answer whether a node can accept the relevant work, not recursively require every optional dependency to be healthy. An overstrict check can remove every node during a downstream incident and turn graceful degradation into total unavailability. Readiness, liveness and business correctness are separate. Connection draining lets existing work finish during rollout, within a bounded shutdown policy; it does not make long-running requests immortal.

Sticky sessions can reduce the need to share ephemeral local state but create imbalance and failover questions. Prefer explicit durable session/state ownership when a node loss must not lose progress. A persistent connection naturally stays on its chosen node; that does not guarantee reconnect returns to the same node or restores missed history.

Scaling choice	Use when	Avoid when
Larger node	Simpler headroom meets requirements	One node exceeds practical capacity or failure tolerance

Scaling choice	Use when	Avoid when
More stateless replicas	Work is independently routable	Shared DB/provider limits are unchanged
L7 routing	HTTP-aware policies justify termination/inspection	Connection-level traffic needs unnecessary application complexity
Sticky routing	A deliberate local-state optimization has recovery	Stickiness is the only state durability plan

[VERIFY: validate load-balancer health checks, retries, connection draining, idle timeouts and protocol handling against the actual gateway/cloud deployment. No balancing, failover or autoscaling load test ran.]

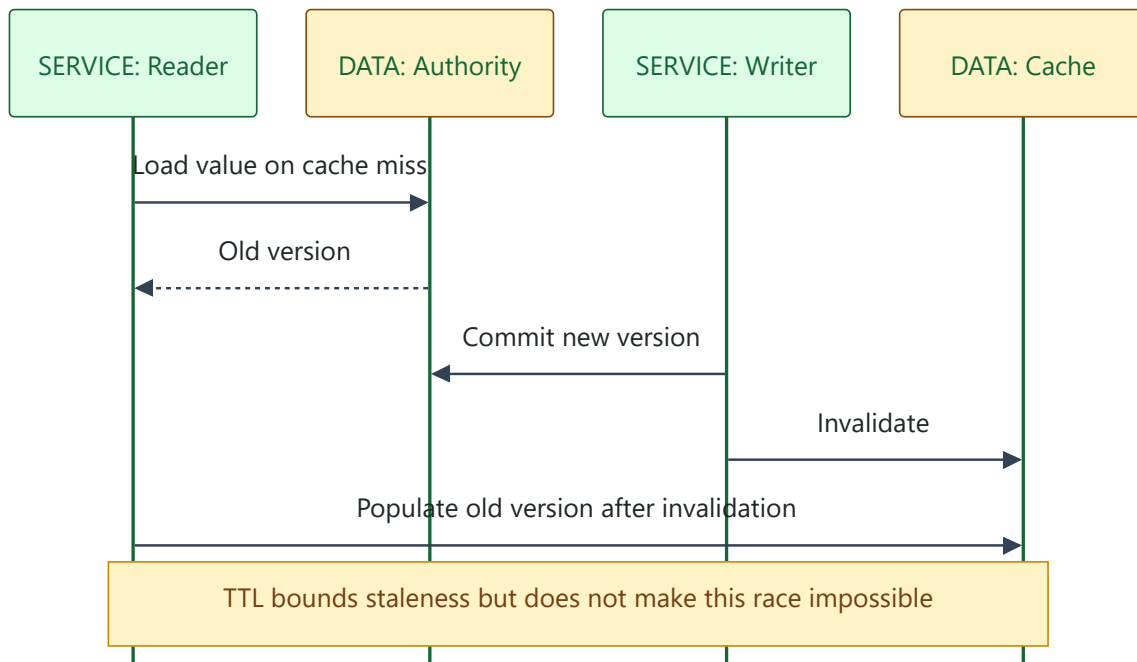
Interview checks: basic: replicas do not multiply every dependency's capacity. Internals: routing policy balances a chosen proxy for load. Debug: inspect shared bottlenecks. Scenario: define what happens to in-flight work when an instance is removed.

4. Caching Layers, CDN and Invalidation

A cache stores a derived value to avoid repeated expensive work. State its key, authority, population strategy, eviction policy, freshness bound and invalidation behavior. A cache without a staleness contract is an undocumented replica. Authorization-sensitive responses also need identity dimensions; leaking another tenant's cached response is not a performance bug.

Cache-aside reads the cache, loads from authority on miss and populates the entry. It is simple but has race windows. A reader can fetch old database state, a writer can commit and invalidate, and then the reader can repopulate the stale value. TTL limits duration but does not remove the race. Versioned entries, conditional writes, generation keys or a deliberately accepted freshness window are possible responses.

Diagram 10.3



Write-through synchronously updates through a cache layer, but atomicity with an external database still needs a specific contract. Write-behind acknowledges before authority persistence and risks loss if the buffer is not durable; do not use it casually for accepted jobs. Read-through encapsulates loading behind the cache API. These names describe responsibilities, not universal consistency guarantees.

A stampede happens when many callers miss an expensive key together. Single-flight loading can collapse concurrent misses within its coordination scope, TTL jitter can avoid synchronized expiry, and stale-while-revalidate can serve bounded stale data while one refresh proceeds. Cache penetration by many nonexistent IDs may justify short negative caching, but negative results need invalidation when records are created and must not leak authorization distinctions.

CDNs cache near clients and can terminate TLS, compress and protect origin capacity. Use immutable versioned asset names with long cache lifetimes for frontend bundles; serve the entry document under a policy that can point clients to new assets. Purge is an operational tool, not a magic atomic global update. Personalized API caching requires explicit keys and credential policy, not only an Edge product checkbox.

Cache policy	Use when	Avoid when
Cache-aside plus TTL	Bounded stale reads are acceptable	Cached values enforce a strict write invariant
Versioned entries	Older loads must not overwrite newer values	Version semantics differ across writers

Cache policy	Use when	Avoid when
Stale-while-revalidate	Availability benefits outweigh bounded freshness loss	Stale permission or balance data authorizes an unsafe action
CDN immutable assets	Build artifacts have content-versioned identity	Same URL is overwritten while clients retain it for months

Interview checks: basic: cache policy includes freshness. Internals: invalidation and repopulation race. Debug: high hit rate can still hide hot-key or stale-data problems. Scenario: keep balance/authorization decisions at their authority unless a stronger protocol proves safety.

5. Replication, Partitioning, CAP and PACELC

Replication copies state for availability, recovery or reads; partitioning divides ownership or placement. Replicas can lag, so a read-after-write may need the leader or a known-position wait. Partitioning can distribute load but makes cross-partition operations and rebalancing harder. The natural tenant key improves locality but can create a hot shard for one unusually large tenant.

Linearizability means each operation appears to take effect at one point between invocation and response while respecting real-time ordering. Eventual consistency means replicas converge under the stated conditions when updates stop and communication/repair succeeds; it says little by itself about what a user may read during convergence. Read-your-writes, monotonic reads and causal guarantees describe useful intermediate application needs. [22](#) gives the deeper formal treatment.

CAP concerns behavior during a network partition: a system cannot guarantee both linearizable consistency and the formal availability property for every request under that failure. It is not an ordinary choose-two product classification, and partition tolerance is not a switch that removes network failure from reality. Practical designs choose which operations reject, queue or serve stale data when communication is lost.

PACELC adds the ordinary-operation trade-off: even without a partition, stronger cross-node coordination can cost latency, while weaker coordination can change consistency. Neither acronym selects your database or tells you whether a stale job list is acceptable. Tie the trade-off to a user-visible operation and invariant.

Operation	Use when	Avoid when
Serve stale job history	Read-only browsing tolerates a labeled delay	It determines whether an irreversible action is allowed

Operation	Use when	Avoid when
Reject conflicting writes during partition	One authoritative decision must be preserved	Rejection is hidden as a successful acceptance
Queue explicit pending work	Durable pending semantics are part of the contract	Enqueue is claimed as completed execution
Per-tenant partitioning	Most operations stay within tenant ownership	A hot tenant or global transaction is ignored

[VERIFY: verify consistency guarantees, read-after-write mechanisms, replica promotion/data-loss policies and partitioning constraints against the chosen database/service configuration. CAP/PACELC are reasoning tools, not evidence of an untested product guarantee.]

Interview checks: basic: replication and partitioning solve different problems. Internals: CAP's availability differs from an uptime percentage. Debug: stale replica reads do not imply lost leader writes. Scenario: decide per operation what must stop and what may degrade.

6. Queues, Backpressure and Admission

A queue absorbs a mismatch in arrival and service rates over time; it does not eliminate that mismatch. If arrivals exceed sustainable completions, the backlog grows until storage, retention or the user's patience is exhausted. A bounded queue turns unlimited delay into an explicit admission decision. Accepted durable work still needs a recovery owner if the consumer is unavailable.

Backpressure communicates limited downstream capacity upstream. It can be broker flow control, bounded prefetch, semaphore admission, HTTP rejection or a demand protocol. Simply adding a queue shifts where work waits. An unbounded executor behind a bounded broker prefetch can recreate the same memory problem inside the application if acknowledgements occur too early.

Partition workers by meaningful isolation boundaries. Per-provider concurrency limits protect downstream APIs; per-tenant budgets prevent one customer from occupying every slot. Global capacity still matters: multiplying tenant limits by thousands of tenants can exceed the shared pool. Use a hierarchy of limits and decide fairness among accepted jobs, retries and new work.

Queue design	Use when	Avoid when
Durable asynchronous queue	Accepted work can finish later	Clients require an immediate completed result
Bounded local queue	Short waiting is useful and loss on restart is acceptable	It is the only durable record of accepted work

Queue design	Use when	Avoid when
Per-tenant/provider lanes	Slow dependencies need isolation	One queue lets a single outage block all customers
Load shedding	Capacity is exhausted and rejection is safer than collapse	A rejected request is reported as accepted work

Interview checks: basic: queues trade capacity mismatch for waiting. Internals: arrival/completion rates determine backlog growth. Debug: inspect oldest work age, not just queue depth. Scenario: retries and replay must share capacity with new traffic under a deliberate policy.

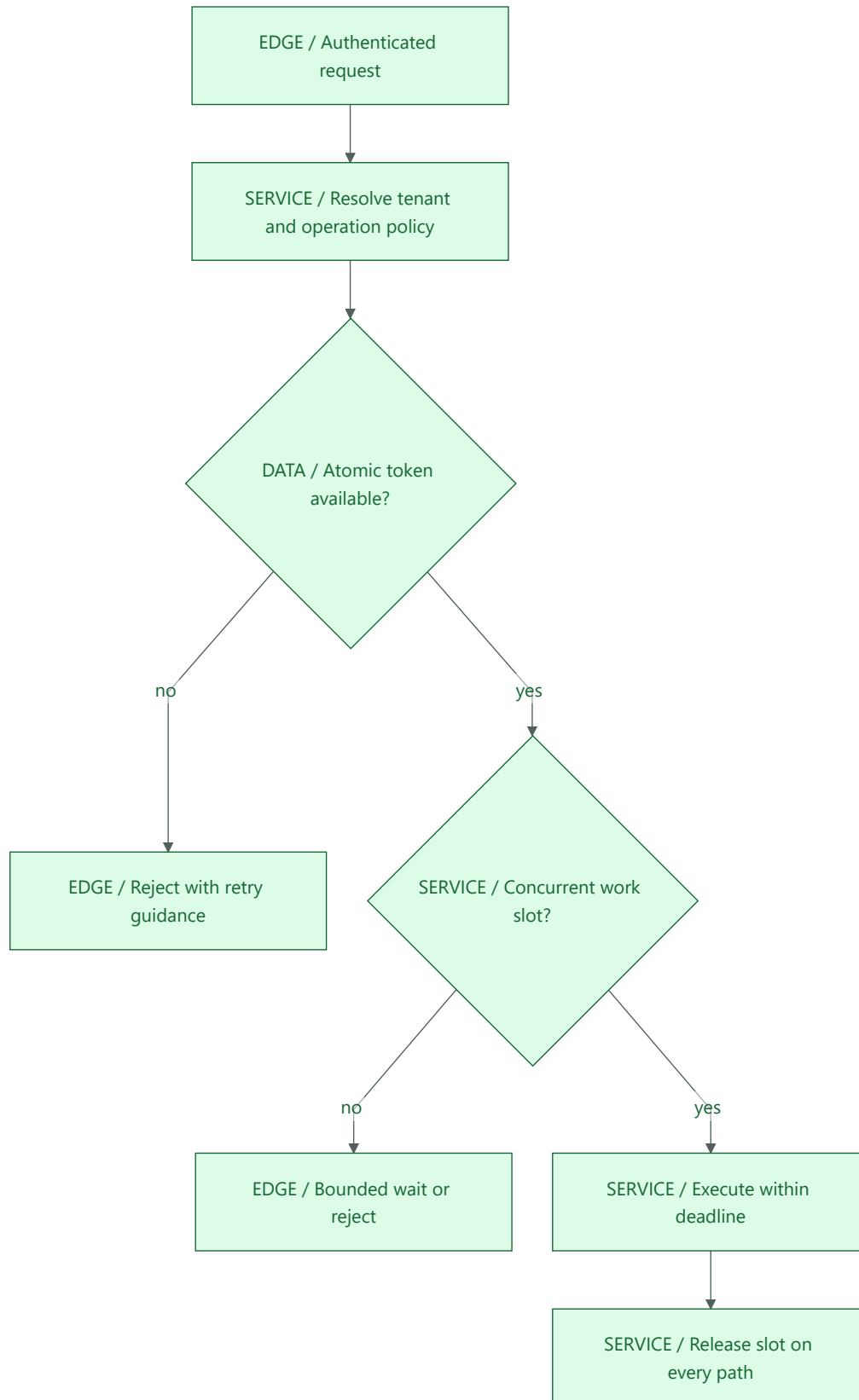
7. Rate Limits and API Gateway Responsibilities

A rate limit bounds activity over time; a concurrency limit bounds active work; a quota accounts for consumption over a longer interval. All three can coexist. A gateway can enforce coarse client/tenant limits before expensive application work, while the service enforces the actual resource budget. A gateway cannot know every domain permission or transaction invariant.

Token buckets refill capacity at a rate up to a maximum burst. They allow short bursts while bounding long-run admitted rate. Fixed windows are simple but can allow bursts across adjacent boundaries. Sliding logs track exact recent requests at memory cost; sliding counters approximate with lower storage. Choose precision, burst behavior and coordination cost from the contract.

Distributed limits require atomic update under a shared authority or a designed allocation scheme. One independent bucket per pod multiplies permitted global rate by pod count. Central counters introduce latency and availability dependence; regional allocations reduce coordination but can strand unused capacity. State explicitly whether limiter failure should fail open, fail closed or use a bounded local fallback, especially for cost-sensitive provider calls.

Diagram 10.4



The gateway's other useful responsibilities include routing, coarse authentication, request size limits, protocol adaptation and correlation. Keep it from becoming an unversioned second business service that performs multi-step writes with no domain ownership. Internal services must still validate the trusted identity and authorize resource access.

Interview checks: basic: token bucket permits a defined burst. Internals: limit atomicity matters across replicas. Debug: doubling pods and doubling allowed rate exposes local-only enforcement. Scenario: document fail-open/closed behavior instead of discovering it during an outage.

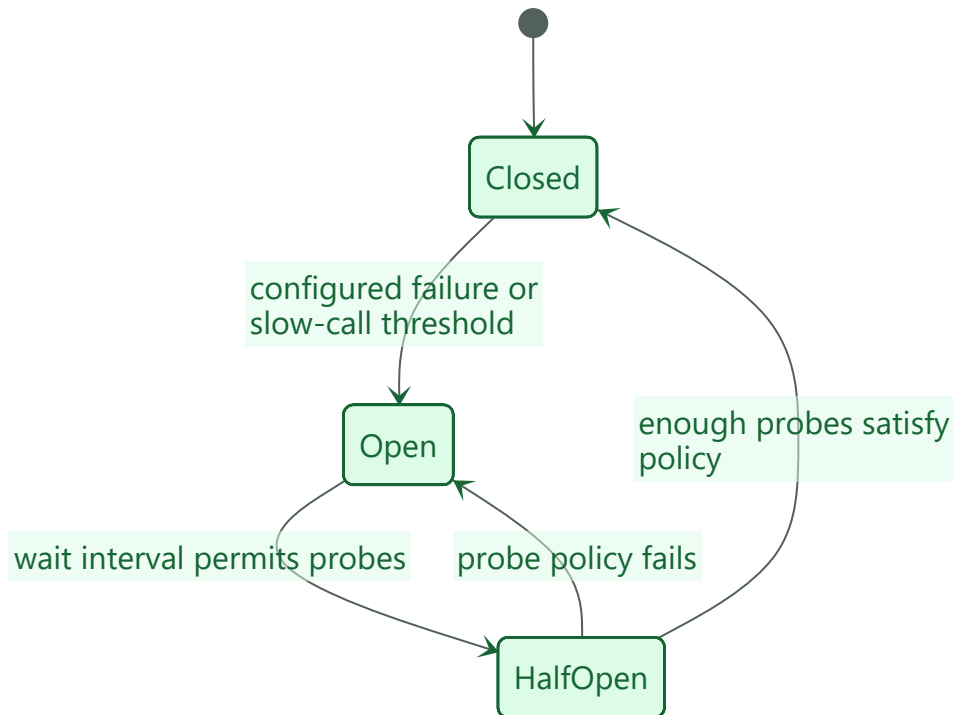
8. Timeouts, Backoff, Jitter and Resilience4j

A timeout bounds how long one layer waits, but may not cancel underlying work. Distinguish connection acquisition, connection establishment, response/read inactivity, total call deadline and the parent request deadline. Independent timeouts can add up beyond the caller's patience. Propagate remaining budget and reserve time for response serialization, cleanup and meaningful errors.

Assume a parent budget of 500 ms, an attempt budget of 150 ms including connection work, 25 ms between attempts and a 25 ms final reserve. Two attempts fit: $2 \times 150 + 25 + 25 = 350$ ms. Three need $3 \times 150 + 2 \times 25 + 25 = 525$ ms, so they do not fit. The lab verifies this arithmetic. These are assumed budgets, not measured latencies or recommended defaults.

Retry only when the failure is eligible, the operation is safe to repeat and budget remains. Exponential backoff increases delay between attempts; jitter distributes clients so they do not retry simultaneously. A retry budget limits extra load at the service or fleet level as well as per request. Three attempts at three nested layers can create up to 27 leaf attempts in a worst-case retry tree; centralize ownership of retries where the full context is known.

Diagram 10.5



A circuit breaker stops some calls to an unhealthy dependency to avoid repeated expensive failures. Closed records outcomes; open rejects according to policy; half-open allows limited probes. It is not a mutex, a rate limiter or a concurrency cap. A breaker can be closed while thousands of slow calls are already in flight. A bulkhead limits concurrent or queued work so that one dependency does not occupy all resources.

Resilience4j provides policy implementations such as retry, circuit breaker, rate limiter, bulkhead and time limiter. Decorator order changes behavior: a breaker outside a retry can observe one final outcome, while a breaker inside can observe each attempt. Timeout placement changes whether it covers one attempt or the whole retry sequence. Explain the desired budget and measurement semantics before choosing annotation order.

Fallback must be honest. Cached stale data may be acceptable for a dashboard; returning an empty successful list can conceal an outage, and inventing a successful payment result is unacceptable. A fallback is another operation with its own cost and correctness limits, not a default success path. Protect it from becoming the next overloaded dependency.

Policy	Use when	Avoid when
Deadline/timeout	Waiting must be bounded	It is assumed to roll back remote effects
Retry with jitter	Transient failures and safe repetition fit remaining budget	Permanent errors or unknown non-idempotent effects are repeated blindly

Policy	Use when	Avoid when
Circuit breaker	Failing dependencies should receive limited probe traffic	It is expected to cap all active calls
Bulkhead	One dependency must not exhaust shared execution capacity	An unbounded queue hides behind the limit
Stale fallback	Product explicitly tolerates bounded stale reads	A failed write is reported as successful

[VERIFY: check Resilience4j version compatibility, exception/slow-call classification, decorator/aspect order, scheduler behavior and cancellation semantics against the selected Spring stack. No Resilience4j application, timer or circuit transition test ran; the lab tests budget arithmetic only.]

[!] TRAP

More retries can reduce availability. During overload, each retry competes with useful work and can keep a dependency saturated after the original spike. Bound attempts, elapsed time and aggregate retry load; do not interpret eventual success of one request as evidence that the fleet policy is healthy.

Interview checks: basic: breaker, limiter and bulkhead are different. Internals: composition order changes observations. Debug: inspect retry amplification and underlying work after timeouts. Scenario: allocate one parent budget across all attempts and cleanup.

9. Multi-Tenancy and Fairness

Tenancy has at least three independent dimensions: authorization, data placement and resource fairness. A perfectly authorized tenant can still monopolize CPU, queues or provider connections. A tenant-specific schema can improve maintenance isolation but does not automatically isolate the shared database process. A separate database improves some boundaries while increasing cost and operational overhead.

The baseline uses tenant-scoped tables and permissions, explicit per-tenant/provider budgets, and one write region. Large tenants may later move to dedicated placement. Routing metadata must be versioned and consistent enough that a migration does not write to both old and new authorities. Background jobs and cache keys must carry the same tenant scope as HTTP requests.

Fair scheduling can rotate between active tenants, allocate weighted shares or maintain separate classes of work. Bound per-tenant queues and total queued work. Account for variable job cost: one request can synchronize ten records while another synchronizes millions. Rate limits measured only in requests can be fair numerically and unfair operationally.

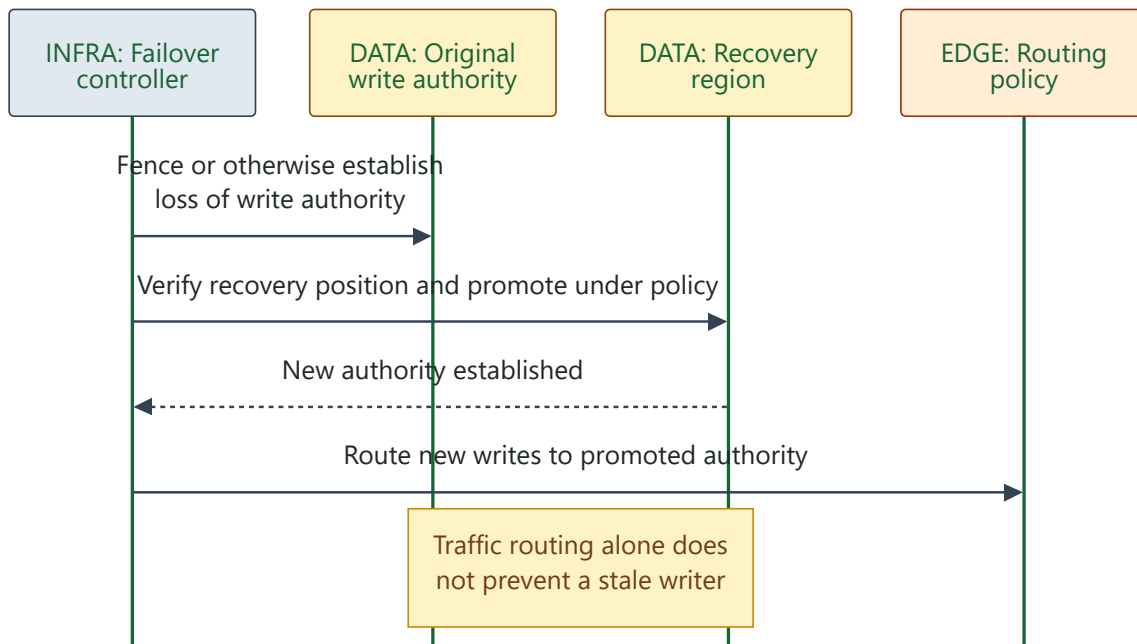
Isolation option	Use when	Avoid when
Shared tables with tenant keys	Operational simplicity and tested logical isolation fit	Every query path is not reliably scoped
Separate schema/database	Compliance, scale or maintenance justifies stronger placement isolation	Operational multiplication is ignored
Weighted work budgets	Tenants have different cost and service commitments	Request count is assumed equal to consumed capacity

Interview checks: basic: security isolation is not resource isolation. Internals: shared bottlenecks remain under separate schemas. Debug: inspect tenant skew and job cost. Scenario: dedicated placement requires an explicit migration/authority protocol.

10. Multi-Region and Recovery

Multi-region can improve latency, disaster recovery or geographic requirements, but each objective suggests a different topology. One write region plus remote replicas is simpler than active-active writes. Active-active requires conflict handling or coordination for overlapping ownership; calling a database global does not explain what happens to two concurrent updates during partition.

Diagram 10.6



Define RPO, RTO and the accepted consistency loss for each operation. Promotion of an asynchronous replica can lose acknowledged writes that never reached it. Synchronous cross-region coordination can reduce that risk but adds latency and availability coupling. A failover runbook must handle fencing the old authority, selecting a recovery point, routing, secrets/keys, idempotency records and replaying queued work.

DNS changes do not instantly move all clients: caching and existing connections persist. Nor do they fence an old writer that is still connected to storage or downstream providers. A routing change and an authority change are different. On failback, reconcile divergent state before reopening both paths; do not simply reverse DNS and hope replicas match.

Regional model	Use when	Avoid when
Single write region with tested DR	Simpler authority and bounded recovery meet requirements	Recovery has never been rehearsed
Regional read replicas	Local read latency matters and lag is acceptable	Reads authorize actions requiring current global truth
Partitioned regional ownership	Tenants can have one clear home authority	Migration and cross-region operations have no owner
Active-active overlapping writes	Conflict/coordination semantics are designed and tested	Product requirements do not justify the complexity

[VERIFY: verify managed-service RPO/RTO behavior, replication durability, fencing, DNS/load-balancer propagation and regional key/secret availability in the actual deployment. No failover rehearsal or regional data-loss experiment ran.]

Interview checks: basic: multi-region is not automatically active-active. Internals: routing and authority are separate. Debug: stale writers survive DNS changes. Scenario: specify what can be lost and how the old region is prevented from writing.

11. Repeatable Interview Framework and Lab

Use a consistent sequence: clarify actors and scope; name invariants and freshness; estimate load/storage with units; define the API and authoritative data model; draw the simplest request path; trace writes and failures; identify the likely bottleneck; propose the smallest justified scale change; finish with operations, security and trade-offs. Revisit assumptions when an estimate changes the architecture.

For IntegrationHub, start with the invariant that an accepted job has durable identity and publication intent. Draw the transaction before adding workers. Explain a timeout after commit, a duplicate event, a slow provider and a disconnected browser. Only then discuss partitions, caches and extra regions. This connects the design to [00's request journey](#) and gives [12's cases](#) a repeatable form.

The runnable source is `code/10-system-design/CapacityLab.mjs` ; Windows PowerShell entry point: `& './docs/java-fs-guide/code/10-system-design/run.ps1'` from the workspace root. It uses the existing Node-compatible runtime, does not start a server and downloads nothing. `execution.json` records four PASS checks: capacity arithmetic, surplus-capacity drain time, attempt budgeting and invalid-input rejection.

The stored calculations are derived from explicitly supplied hypothetical values. They do not establish actual system throughput, payload size, latency, fault tolerance or a recommended configuration. Verify real assumptions with representative load, production telemetry and failure tests before sizing infrastructure.

[D] DECISION

Keep a rejected simpler alternative. If a single database and worker pool fail a concrete requirement, state which one and why. That explanation is stronger than adding a cache, broker and second region merely because they are familiar interview boxes.

[?] INTERVIEW

An estimate should change a decision. If your calculation never affects storage, concurrency, partitioning, cost or recovery headroom, it may be arithmetic decoration. Show the assumption, units, result and the choice it informs.

Explained-Here Index and Sources

Concepts	Explanation
Requirements and design framework	Invariants first
Estimation and capacity	Units and recovery headroom
Scalability and load balancing	Execution and shared constraints
Caching and CDN	Freshness and invalidation
Replication and consistency	CAP/PACELC boundaries
Queues	Admission and backlog
Rate limiting and API gateway	Atomic limits
Resilience4j, circuit breaker, backoff, jitter, bulkhead, timeouts	Resilience composition
Multi-tenancy	Isolation and fairness
Multi-region	Authority and recovery

Owning references for verification: [Resilience4j documentation](#), [HTTP caching RFC 9111](#), [PostgreSQL replication](#) and [AWS reliability guidance](#). These are reference destinations, not local deployment evidence.

Related Chapters

Connect 00, 02 concurrency, 05 APIs, 06 storage, 07 messaging, 08 security, 09 browser, 11 LLD, 12 HLD, 18 operations, 22 distributed theory, 23 performance and 25 architecture. Generated navigation provides reciprocal links.

RELATED CHAPTERS / RECIPROCAL LINKS

- | | |
|---|---|
| 00 The System Before the Details | 06 Databases and Durable State |
| 01 Core Java and the JVM | 09 The Browser and the Java Backend |
| 02 Concurrency and the Memory Model | 05 APIs and Real-Time Communication |
| 21 Networking and Operating Systems | 07 Messaging, Batch and Streams |
| 03 Spring and Spring Boot Internals | 08 Security, Identity and Data Protection |

12	Twelve System Design Case Studies
22	Distributed Systems Theory
23	JVM Performance Engineering
11	Object Design, Patterns and LLD
18	Observability and Reliability
25	Architecture Styles and Domain Design

27	Fintech and Payments Engineering
15	Kubernetes and Production Workloads
16	Build, Ship and Roll Back
17	Cloud and Infrastructure
24	IntegrationHub in Motion
20	Appendix: Interview Toolkit and Index

One-Page Cheat Sheet

Requirements: name actors, invariants, freshness, load, latency and recovery before components. Accepted asynchronous work and completed work need separate SLOs.

Capacity: keep units and assumptions explicit. Mean concurrency is throughput times mean residence under a stable boundary. Backlog drains only with surplus throughput. Storage includes retention, copies and additional overhead; averages hide bursts and tenant skew.

Scale: replicate stateless work only when shared dependencies permit more throughput. Balance according to actual workload and drain old connections during deploys. Partitioning changes ownership; replication copies it.

Caches: define identity, authority, freshness, miss handling and invalidation. TTL bounds stale duration but does not eliminate races. Use single-flight/jitter where justified and keep strict write decisions at authority.

Resilience: one parent deadline covers attempts and cleanup. Retry eligible, repeatable work within per-request and aggregate budgets. Breakers reject unhealthy dependencies; bulkheads cap occupied resources; rate limits cap activity over time. Fallbacks must not invent success.

Regions and tenants: fairness is separate from access control. Multi-region needs an authority/fencing protocol and tested recovery; DNS is only routing. CAP/PACELC explain trade-offs, not product guarantees. The lab passed arithmetic checks, not a load or failover test.

Interview Corner

Basic: What Do You Ask Before Drawing a System?

Actors, commands/reads, invariants, scale/skew, freshness, latency, availability and recovery objectives. State what is out of scope and start with a simple authority model.

Internals: Why Does Little's Law Not Give a Thread-Pool Size Directly?

It relates mean occupancy, throughput and residence for a defined stable boundary. Threads, queues, virtual-thread waiting and downstream pool capacity may occupy different boundaries, and tail latency needs more evidence.

Trace/Debug: Consumers Match Incoming Rate but Lag Never Falls.

There is no surplus capacity to drain the existing backlog. Net drain is completion rate minus arrival rate. Increase sustainable completion capacity or reduce arrivals, then include restore/replay overhead.

Scenario: Which Component Do You Scale First?

The measured constraint: CPU, database work, provider quota, partition or network. More replicas behind the same saturated authority can make latency worse rather than improve throughput.

Basic: Does a TTL Make a Cache Consistent?

No. It bounds how long an entry may remain without refresh. Stale repopulation and concurrent writes still need a freshness contract or a stronger version/coordination scheme.

Internals: Why Can Invalidation Race With a Read?

A reader loads old state, a writer commits and invalidates, then the reader repopulates the old state. Versioned conditional population or an accepted bounded-stale policy is needed; ordering one delete is insufficient.

Trace/Debug: Pods Doubled and the Global Rate Limit Doubled.

The bucket was local to each pod. A global contract needs shared atomic authority or a deliberate allocation scheme. Autoscaling changes the aggregate if independent local limits are simply multiplied.

Scenario: Should the Limiter Fail Open?

Choose per risk: availability-sensitive public reads may permit a bounded fallback, while expensive provider mutations may need fail-closed or reserved local allowance. Specify and test the degraded policy.

Basic: Circuit Breaker Versus Bulkhead?

A breaker limits calls based on dependency health; a bulkhead limits occupied resources. A healthy/closed breaker can still admit enough slow calls to exhaust the application without a bulkhead.

Internals: Why Does Decorator Order Matter?

It changes whether policies observe each attempt or the whole logical call and whether deadlines cover attempts or retry sequences. Explain desired measurements and budgets before selecting annotations.

Trace/Debug: Retries Kept an Outage Going.

Inspect nested retry amplification, synchronization without jitter and work continuing after caller timeout. Bound aggregate retry load, classify failures and give recovery probes room to succeed.

Scenario: What Is a Safe Fallback for a Failed Write?

Usually an explicit failure or durable pending/unknown state, not fabricated success. A read may serve bounded stale data if the product permits it; a write needs its authority's outcome.

Basic: Is CAP Choose Any Two?

No. It concerns consistency versus formal availability during a partition. Networks can partition regardless of preference; decide which operations reject or degrade under that failure.

Internals: What Does PACELC Add?

It highlights that even without partitions, cross-node coordination can trade latency against consistency. Apply it to a specific operation rather than assigning simplistic labels to products.

Trace/Debug: DNS Failover Did Not Stop Old Writes.

DNS routing does not fence existing clients or a stale authority. Establish a storage/application authority protocol, then route traffic. Reconcile before fallback.

Scenario: One Tenant Dominates the Queue.

Introduce per-tenant and global admission, cost-aware fairness, provider isolation and possibly dedicated placement. Keep security scope, resource budget and data ownership explicit and separate.

25 / Architecture Styles and Domain Design

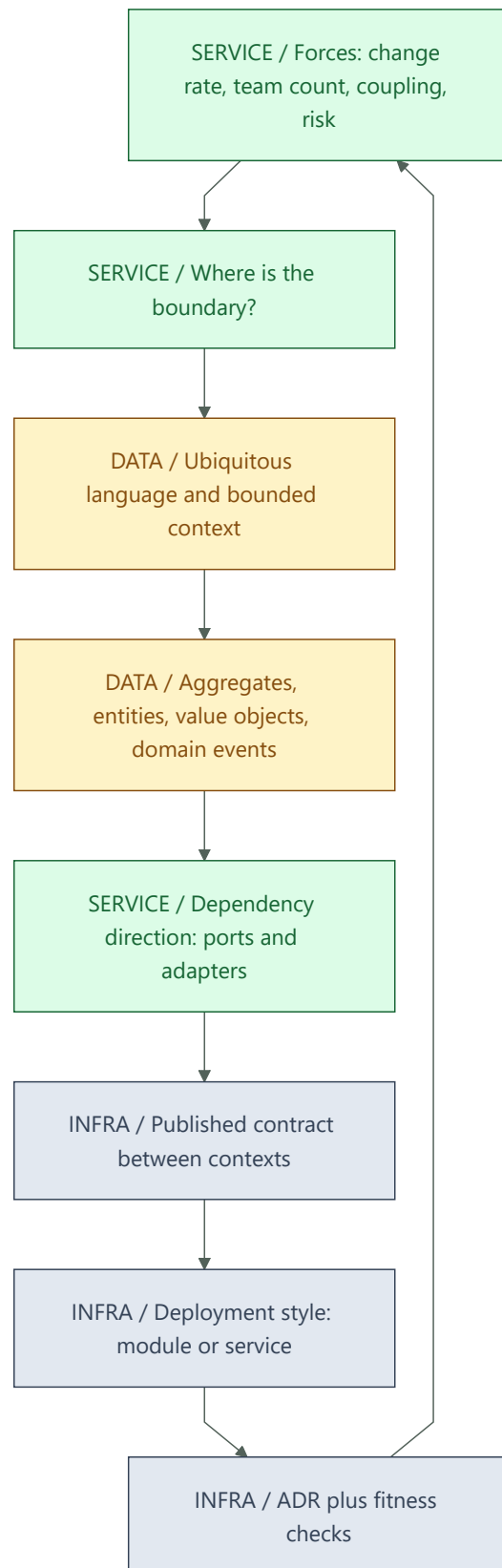
Architecture is the set of decisions that are expensive to reverse. Choosing a deployment style, drawing a boundary around a model, and deciding which direction dependencies point will outlive most of the code written inside them. IntegrationHub is fictional. This chapter explains how to argue those decisions from forces that can be checked, instead of from a preferred diagram shape.

Version assumptions: Java 21, Spring Boot 4.0.x / Cloud 2025.1.x, PostgreSQL 17 and Kafka 4.x, matching the rest of the guide. Package layouts, build-time enforcement tooling and specification versions change independently of this text, so treat every named tool version as a verification target rather than a fixed fact.

Execution status: the model in `code/25-architecture/ArchitectureLab.mjs` ran and PASSED five checks on the existing Node runtime. Java and Spring shapes in this chapter are structural illustrations and are NOT EXECUTED: no JDK, Maven or Gradle is available here, so no compilation, no build-time architecture rule and no running application was produced. No organizational claim in this chapter was measured.

Big Picture

Diagram 25.1



What You Will Be Able to Explain

- State the forces that favor a monolith, a modular monolith or separate services, and the costs each one adds.
- Migrate a running system incrementally with a strangler-fig approach instead of a rewrite.
- Find bounded contexts from language and change patterns, then map the relationships between them.
- Design aggregates that own an invariant, and separate entities from value objects deliberately.
- Publish domain events without turning them into a hidden synchronous call chain.
- Map hexagonal and clean architecture onto a concrete Spring package structure with an enforceable dependency rule.
- Run API-first and contract-first work so consumers are not blocked and breaking changes are visible.
- Write an ADR that records forces and consequences, and manage technical debt as a tracked decision rather than a complaint.

[P] PRODUCTION

Where this lives in IntegrationHub. This chapter sets the boundaries that the rest of the guide fills in. 10 sizes and shapes the system, 11 designs the classes inside a boundary, 12 applies both to case studies, 03 wires the chosen structure, 05 publishes the contract, 07 carries events across boundaries and 22 explains what distribution costs.

1. Monolith, Modular Monolith and Microservices

A **monolith** is one deployable unit containing the whole application. Internal calls are in-process method calls, one transaction can span multiple features, and one build and release covers everything. Nothing about a monolith requires bad structure; the common failure is that nothing stops any class from reaching any other class, so boundaries erode silently over years.

A **modular monolith** keeps the single deployable unit but makes modules explicit: each module owns its data and its public API, and cross-module access goes through that API rather than through another module's internals. The enforcement has to be mechanical, because convention alone does not survive deadlines. Java tooling for this includes the module system, build-module separation, package-private visibility and build-time dependency rules [VERIFY: current ArchUnit and Spring Modulith capabilities and versions at <https://www.archunit.org/> and <https://spring.io/projects/spring-modulith>].

Microservices split the system into independently deployable services with explicit data ownership. The potential gains are independent deployment, scaling and failure isolation, provided contracts and resource boundaries support them. Calls crossing a new service boundary become remote operations that can be slow, duplicated or lost; internal method calls remain local. A local transaction no longer spans independently owned stores. Those costs belong to 22.

[M] **MECHANISM**

A service boundary converts a compiler error into a runtime failure. Inside one deployable unit, an incompatible change to a method signature fails the build. Across a service boundary, the same change compiles on both sides and fails in production when an old consumer sends an old payload. That is the single most important mechanical difference between the styles, and it is why contract testing (19) and versioning (05) become mandatory rather than optional once you split.

Style	Use when	Avoid when
Monolith	One small team, unclear domain boundaries, early product, strong need for simple transactions	Many teams block each other on one release train, or one feature's load forces the whole app to scale
Modular monolith	Boundaries are becoming clear but deployment independence is not yet needed, and operational budget is limited	Modules genuinely need different runtimes, scaling profiles or release cadences, and the team will not enforce module rules
Microservices	Independent deployment, independent scaling or failure isolation is a real and current requirement, and platform/observability investment exists	Boundaries are still being discovered, the team is small, or distributed transactions would be introduced for a single workflow

2. Deciding, and the Costs People Forget

Decide from forces, not from fashion. The useful questions are: how many teams need to release without coordinating, which parts change at different rates, which parts have different availability or scaling requirements, where does a failure need to stay contained, and what is the blast radius of a bad deploy. If the honest answer to all of these is "one team, similar rates, same requirements", a well-structured single deployable unit is the cheaper correct answer.

The costs that get omitted from the optimistic version are operational. Every service needs its own pipeline, image, configuration, secrets, dashboards, alerts, on-call ownership and data store lifecycle. Cross-service debugging needs distributed tracing (18). Cross-service consistency needs sagas or outbox patterns (07) because a single database transaction no longer covers the workflow. None of that is impossible; it simply has to be budgeted as part of the decision instead of discovered afterward.

A common anti-pattern deserves a name: the **distributed monolith**, where services are deployed separately but must be released together because their contracts are coupled, they share a database, or a single user request fans out through a synchronous chain where any failure fails the whole request. It pays the full cost of distribution and collects none of the independence benefit. The diagnostic question is simple: can you deploy one of these services, alone, on a Friday, without coordinating with another team?

[!] **TRAP**

"We will split it later" and "we will fix the boundaries later" are different promises.

Splitting a well-modularized system is mechanical work. Splitting a system whose modules share tables, share mutable entities and call each other's internals is a rewrite, because you must first discover the boundary that was never written down. The boundary work is the hard part, and it is equally valuable inside a single deployable unit — which is why a modular monolith is a legitimate destination, not only a waypoint.

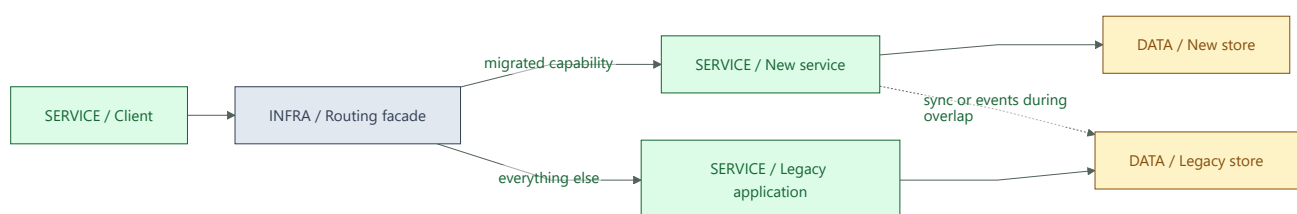
Interview checks: basic: name the real forces, not team preference. Internals: explain what the compiler stops enforcing after a split. Debug: identify a distributed monolith from its release coupling. Scenario: propose the cheapest structure that satisfies the stated requirement.

3. Strangler-Fig Migration

A rewrite freezes delivery, invites scope drift and produces one enormous cutover risk. The strangler-fig approach instead routes traffic through a facade, moves one capability at a time behind that facade, and lets the old system shrink until it can be removed [VERIFY: original formulation and current guidance at

<https://martinfowler.com/bliki/StranglerFigApplication.html>].

Diagram 25.2



The sequence that works: put the facade in first and prove it is transparent; pick a capability with a clear boundary and real value; move reads before writes where possible; run the new path in parallel and compare outputs before trusting it; shift traffic incrementally with a fast

rollback; then delete the old path and its data. The deletion step is the one most often skipped, and skipping it leaves you permanently paying for two systems.

Data is the hard part, not routing. During the overlap window, two stores can hold the same facts. Pick one writer per fact at any moment and synchronize in one direction, using change data capture or events (07, 26). Bidirectional synchronization creates conflicts that you then have to resolve with rules nobody can state precisely. Define the overlap end date when you start it, because an indefinite dual-write is a permanent correctness liability.

4. Domain-Driven Design and Ubiquitous Language

Domain-driven design is primarily about modelling a business domain in code using the same language the domain experts use [VERIFY: terminology and canonical definitions against Evans' and Vernon's published material and

<https://martinfowler.com/tags/domain%20driven%20design.html>]. The **ubiquitous language** is that shared vocabulary: if the business says "a sync run is quarantined", the code says

`SyncRun.quarantine()` and not `JobEntity.setStatus(7)`. Translation layers between business language and code language are where misunderstanding accumulates.

The practical test for whether the language is working: can a domain expert read a method name, an event name or a state name and confirm or reject it without a developer explaining it? If every concept needs translation, the model is encoding the database schema or the UI, not the domain.

DDD is usually divided into strategic design — bounded contexts, context mapping, how teams and systems divide the domain — and tactical design — aggregates, entities, value objects, repositories, domain services, domain events. Strategic design is what matters most in architecture interviews, because getting the boundaries wrong cannot be fixed by writing better classes inside them. Tactical patterns applied inside badly drawn boundaries just make the wrong model more elaborate.

Not every part of a system deserves this investment. A **core domain** — the part that differentiates the business — justifies careful modelling. Supporting and generic subdomains (notifications, file storage, audit export) are usually better served by simple code or a bought product. Spending the richest modelling effort on a generic subdomain is a common and expensive misallocation.

5. Bounded Contexts and Context Mapping

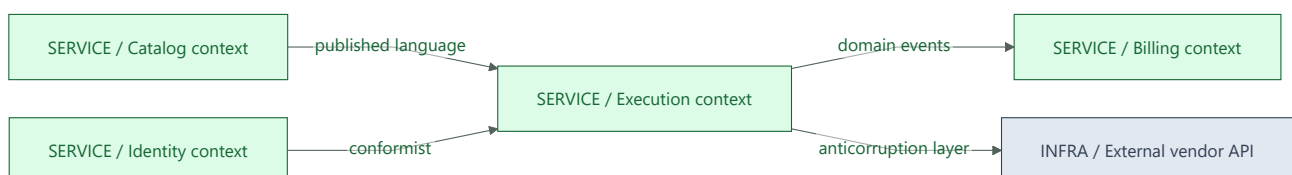
A **bounded context** is the scope within which a model and its language are consistent. The same word can legitimately mean different things in different contexts: in IntegrationHub, a "connector" in the catalog context is a purchasable capability with a tier and a price, while in the execution context it is a runtime adapter with credentials, a rate limit and a health state. Forcing one `Connector` class to serve both produces a type with two unrelated halves and conflicting invariants.

Find candidate boundaries by looking for language that changes meaning, data that changes at different rates, features that are always deployed together, and parts the organization already treats as separate responsibilities. Organizational communication structure tends to show up in system structure, so boundaries that cut across team ownership tend to be contested continuously [VERIFY: Conway's law statement and the "inverse Conway" practice as described at <https://martinfowler.com/bliki/ConwaysLaw.html>].

Context mapping names the relationship between two contexts, which is what tells you how much coupling you are accepting:

Relationship	What it means	Use when / avoid when
Shared kernel	Two contexts share a small common model and must agree on changes	Use when two teams genuinely share an invariant and will coordinate; avoid when it becomes a dumping ground for shared DTOs
Customer/supplier	Downstream's needs are negotiated into the upstream's plan	Use when teams can prioritize together; avoid when the upstream has no incentive to deliver
Conformist	Downstream accepts the upstream model as-is	Use when the upstream is a fixed external system and translation is not worth it; avoid when the foreign model pollutes your core domain
Anticorruption layer	Downstream translates the foreign model into its own at the boundary	Use when integrating a legacy or external model you cannot change; avoid when the upstream model is already a good fit and translation adds pure cost
Published language	Both sides use an explicit, versioned, documented contract	Use for contracts with several consumers; avoid when a single internal consumer makes the formality unnecessary
Separate ways	No integration at all	Use when duplication is cheaper than coupling; avoid when the two copies must agree on a fact

Diagram 25.3



The executed model detects a cycle in a declared context dependency map. A cycle signals coupling to investigate, not proof that independent testing or deployment is impossible. Compatible contracts and asynchronous interaction can preserve independence; coordinated releases and synchronous availability dependencies are the stronger warning signs.

6. Aggregates, Entities and Value Objects

An **entity** has identity that persists through change: a sync run with id `job-1` is the same run whether its status is `RUNNING` or `DONE`. A **value object** has no identity and is defined entirely by its attributes: `Money(500, "EUR")` equals any other `Money(500, "EUR")`, and changing it means producing a new one. The executed model asserts exactly this distinction — equality by value for one, equality by identifier for the other.

Value objects are undervalued. Making `TenantId`, `Money`, `EmailAddress` and `RateLimit` into types instead of `String`, `BigDecimal` and `int` moves a class of bugs from runtime to compile time, gives validation one home, and makes method signatures self-documenting. Java records fit value objects well: they are final, value-like and give equality by components.

An **aggregate** is a cluster of objects treated as one unit for the purpose of enforcing an invariant, with a single **aggregate root** as the only entry point. The root enforces the rule; nothing outside holds a reference to an internal part and mutates it directly. The invariant is the reason the aggregate exists — if you cannot name the rule that must always hold, you probably have a data grouping rather than an aggregate.

java

```
// Structural illustration. NOT EXECUTED: no JDK is available in this environment.
public final class ConnectorCatalog {           // aggregate root
    private final TenantId tenant;              // value object
    private final int connectorLimit;           // invariant parameter
    private final Set<ConnectorName> connectors = new LinkedHashSet<>();
    private final List<DomainEvent> events = new ArrayList<>();

    public boolean add(ConnectorName name) {
        if (connectors.contains(name)) {
            return false;                       // idempotent, no event
        }
        if (connectors.size() >= connectorLimit) {
            throw new ConnectorLimitReached(tenant, connectorLimit);
        }
        connectors.add(name);
        events.add(new ConnectorAdded(tenant, name));
        return true;
    }
}
```

The executed model implements that same logic and asserts the two properties that matter: a command violating the invariant is rejected **and records no event**, and a duplicate command is idempotent **without producing a second event**. Both are frequent production bugs — the first publishes an event for a change that never happened, the second causes downstream double-processing.

Keep aggregates small. One aggregate per transaction is a useful design heuristic, not a database law. Reference other aggregates by identifier and use asynchronous consistency where the business allows it. A genuine cross-aggregate invariant may justify a coordinated local transaction or a revised boundary. Large aggregates can increase contention (02), loading (04) and conflicts on unrelated fields; choose from the actual invariant rather than table relationships alone.

[!] **TRAP**

An aggregate is not a table group and not a JPA entity graph. Mapping every foreign key into a bidirectional association and calling the top object an aggregate root produces an object that loads half the database and locks rows nobody touched. Start from the invariant, decide the smallest consistency boundary that enforces it, and only then decide the persistence mapping.

7. Domain Events

A **domain event** states that something meaningful happened in the domain, in past tense, with the facts needed to react to it: `SyncRunCompleted`, `ConnectorAdded`, `PaymentCaptured`. It is not a command, and the publisher does not know or care what happens next. That decoupling is the point: the execution context does not need to know that billing exists.

External consumers must not observe an accepted fact before its state is durable. In-process handlers can participate in the current transaction if their effects roll back with it; irreversible external actions cannot. For cross-process delivery, commit publication intent in an outbox and relay it after commit (07). The local model records events only for accepted in-memory changes; it does not test transaction commit or publication.

Event payload design is a boundary decision. A thin event carries identifiers and forces consumers to call back for details, which couples them to your API and multiplies load. A fat event carries a snapshot of the facts, which decouples consumers but publishes model details you must then keep stable. Inside one context, thin is usually fine; across a published boundary, carry the facts the consumer needs and treat the event schema as a versioned contract.

[D] DECISION

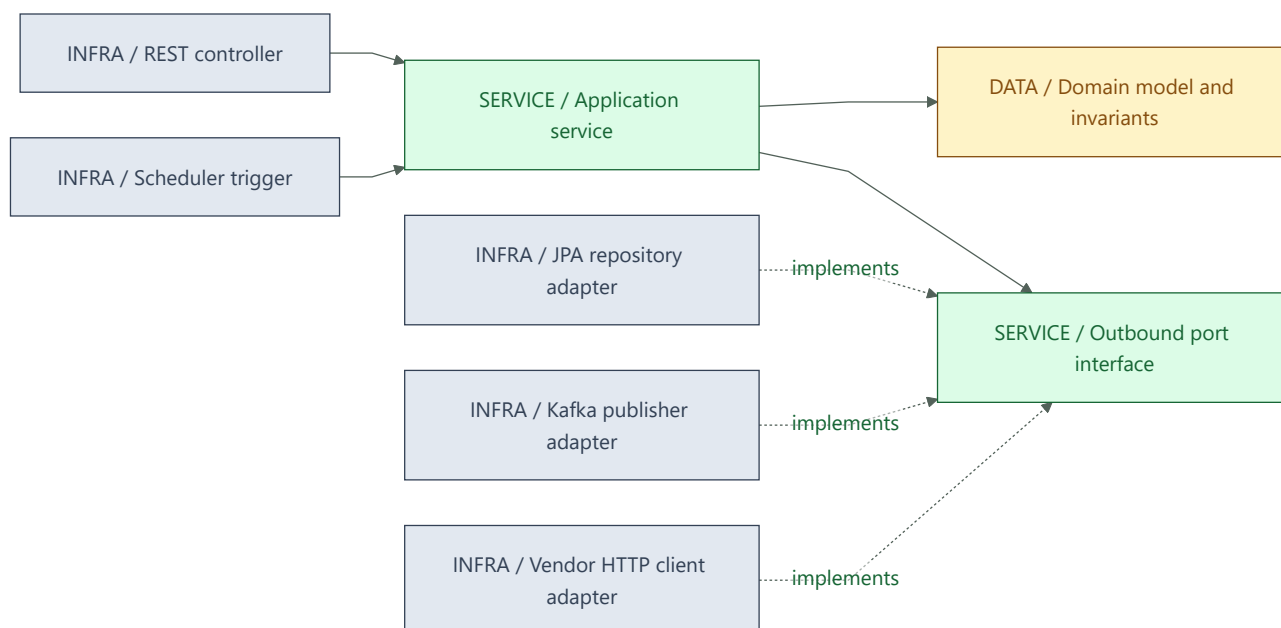
Use events to decouple reactions, not to hide a required sequence. If step B must happen for step A to be correct and the user is waiting for the result, an event plus eventual consistency has made a synchronous requirement invisible rather than removing it. Either keep it in one transaction inside one aggregate, or make the asynchronous workflow explicit with a saga that has compensation and a visible state (07, 22).

8. Hexagonal and Clean Architecture

Hexagonal architecture — ports and adapters — puts the application at the center, defines **ports** as interfaces describing what the application needs or offers, and makes **adapters** the implementations that connect those ports to HTTP, databases, message brokers or external APIs [VERIFY: original description at <https://alistair.cockburn.us/hexagonal-architecture/>]. Clean architecture, onion architecture and similar formulations differ in vocabulary and layer count but share the governing rule [VERIFY: layer naming and the dependency rule as published at <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>].

That governing rule is **dependency direction**: source-code dependencies point inward, toward the domain. The domain knows nothing about HTTP, JPA, Kafka or Spring. An inbound adapter calls into the application; the application calls an outbound port interface that it owns; infrastructure implements that interface. This is dependency inversion applied at architectural scale (11).

Diagram 25.4



The benefit is testability and replaceability: the domain and application can be tested with plain unit tests and in-memory port implementations, with no container start and no database (19). The cost is indirection — more interfaces, and mapping between domain types and persistence or transport types. That mapping is the honest price of the boundary, not an accident to be optimized away by annotating domain classes with `@Entity` and `@JsonProperty`.

The executed model checks the rule mechanically on a declared module graph: it accepts application depending on domain and rejects a domain module depending on an adapter module. Real projects enforce the same rule at build time with an architecture-test library, which turns an architectural agreement into a failing build instead of a code-review comment.

9. Mapping It onto a Spring Package Structure

The abstraction only pays off when it becomes a package layout people can follow. Organize by feature first, then by layer inside the feature — the reverse (top-level `controller`, `service`, `repository`, `model`) groups unrelated features together and makes any boundary invisible.

text

```
com.integrationhub
  syncrun
    domain
      SyncRun.java          <- aggregate root, invariants
      SyncRunId.java         <- value object
      SyncRunCompleted.java  <- domain event
    application
      StartSyncRunService.java <- use case, @Transactional boundary
    port
      SyncRunRepository.java  <- outbound port, owned by application
      ConnectorGateway.java   <- outbound port for the vendor call
  adapter
    in
      web
        SyncRunController.java <- @RestController, DTOs, validation
      messaging
        SyncCommandListener.java <- @KafkaListener
    out
      persistence
        SyncRunJpaEntity.java  <- @Entity, persistence shape only
        SyncRunJpaRepository.java <- implements SyncRunRepository
      vendor
        HttpConnectorGateway.java <- implements ConnectorGateway
  SyncRunModule.java         <- module-level configuration
  catalog                    <- another bounded context
  billing                    <- another bounded context
  shared                     <- deliberately small: ids, Money, errors
```

Practical rules that make this hold up. Keep the `domain` package free of framework imports; it should compile against plain Java. Put the transaction boundary in the application service, one use case per class, so the unit of work matches the aggregate (04). Keep the persistence entity separate from the domain aggregate when their shapes diverge; collapsing them is a legitimate pragmatic choice for simple modules, but it is a choice with consequences, not a default. Make

cross-module access go through a small published interface and keep everything else package-private. Keep `shared` tiny — a shared package that grows without limit is the shared kernel turning into a distributed global object.

Constructor injection keeps collaborators explicit. This illustrated layout permits Spring's `@Transactional` in the application layer, so only the domain is strictly framework-free. For a fully framework-free application layer, put transaction interception in an outer decorator or configuration. Extraction can preserve much domain code, but remote failure, authorization and data-migration contracts still need redesign; an adapter swap alone does not prove a safe service migration.

10. API-First and Contract-First Design

API-first means the interface between a provider and its consumers is designed, reviewed and agreed before the implementation is written. Contract-first means that agreement is captured in a machine-readable artifact — an OpenAPI document for HTTP, a schema for events — which is then used to generate or validate both sides [VERIFY: current OpenAPI specification version and tooling at <https://spec.openapis.org/> and schema-registry behavior for your broker].

The mechanical benefits are concrete. Consumers can build against a mock derived from the contract instead of waiting for the provider. Both sides generate types from the same source, which removes an entire category of field-name and nullability mismatch. Incompatible changes become visible in a diff of the contract rather than in a production error. And the contract becomes the artifact that contract tests verify, so "we agreed on this" is enforced by CI (19).

The discipline that matters is compatibility. Adding an optional field is usually safe; removing a field, renaming it, narrowing a type, making an optional field required or changing an enum's meaning is not. Decide the versioning strategy before the first external consumer exists, because retrofitting it is far more expensive (05). For events, the schema is the contract and the compatibility mode of the registry is the enforcement point (07).

Contract-first does not mean generated code must leak into the domain. Generated request and response types belong in the inbound adapter, and the application service takes domain types. Otherwise a contract change propagates straight into your core model, which is precisely the coupling the boundary was supposed to prevent.

11. Architecture Decision Records

An ADR is a short document capturing one decision, written when it is made [VERIFY: template variants and current practice at <https://adr.github.io/>]. The reason to write one is that six months later nobody remembers which constraints were in force, so a decision made under a real constraint gets reversed casually, or an obsolete decision gets preserved out of fear.

A usable ADR has: a title naming the decision, a status (proposed, accepted, superseded by a later ADR), the context and forces in play, the decision itself stated plainly, the alternatives considered with why they were rejected, and the consequences including the negative ones. The consequences section is the one that earns its keep, because it is where you record what you knowingly gave up.

Keep them immutable and append-only. You do not edit an accepted ADR when you change your mind; you write a new one that supersedes it. The chain of superseded decisions is the architecture's history, and losing it means relitigating the same arguments. Store them in the repository next to the code so they are versioned with the system they describe.

Write one when a decision is costly to reverse, affects more than one team, or will look arbitrary to someone who was not in the room — choosing a messaging technology, defining a module boundary, accepting eventual consistency for a workflow, or picking a migration strategy. Do not write one for choices that a single commit can undo.

12. Evolutionary Architecture and Fitness Functions

Architecture is never finished, because the forces that justified a decision change. Evolutionary architecture treats change as expected and asks what you can automate to keep the system from drifting away from its stated properties [VERIFY: fitness function terminology and current guidance at <https://www.thoughtworks.com/insights/topic/evolutionary-architecture>].

A **fitness function** is an automated check for an architectural characteristic, run in CI like a test. Examples that are practical to implement: a dependency-direction rule that fails the build if the domain package imports an adapter package; a check that no module imports another module's internal package; a limit on p95 latency in a performance test (19); a license and vulnerability scan (16); a check that every public endpoint appears in the OpenAPI document. Each one converts an intention into a mechanism.

The executed model in this chapter is a miniature fitness function: it fails if a declared module graph violates the inward dependency rule or if the context map contains a cycle. The real version runs against compiled bytecode or the actual package graph; the principle is identical,

which is that an architectural rule nobody can break accidentally is worth more than a diagram nobody reads.

[?] INTERVIEW

The strongest architecture answer names the enforcement mechanism. Anyone can say "we used hexagonal architecture". The follow-up that separates candidates is: what stopped a developer from calling the repository directly from the controller? Acceptable answers are a build-time dependency rule, module visibility, a separate build module, or a review checklist — and admitting it was only a convention, with the drift that caused, is a better answer than implying an enforcement that did not exist.

13. Managing Technical Debt

The debt metaphor is about interest, not about bad code: a deliberate shortcut taken to ship sooner, which then costs extra on every subsequent change until it is repaid. Distinguish deliberate debt taken with a known reason from accidental debt caused by not knowing better, and from code that is simply old but working. Code you dislike is not debt. Debt is a shortcut that measurably slows the next change or raises the risk of the next failure.

Make it visible and comparable. Record each item with where it is, what it costs now (time per change, incident frequency, blocked work), what repaying it costs, and what triggers repayment. That turns "we need to refactor" into a comparison a product owner can actually weigh against a feature. Items that cannot be stated in those terms usually turn out to be preferences.

Repay it where you are already working. Opportunistic improvement inside a module you are changing anyway is cheap and low-risk. A large separate refactoring project with no feature attached tends to lose funding halfway, which leaves the system in a worse state than either endpoint. Reserve big-bang repayment for cases where the debt blocks a committed objective, and then treat it with the same migration discipline as a strangler-fig move.

Prevent new unexamined debt by making the cost visible at decision time, which is exactly what an ADR's consequences section does. A shortcut recorded as "we accepted this to hit the launch date, revisit when the second tenant onboards" is manageable. The same shortcut with no record becomes mysterious code that nobody dares to touch.

14. Executed Model and Evidence

`code/25-architecture/ArchitectureLab.mjs` and its Windows PowerShell runner use the existing Node-compatible runtime. Command from the workspace root: `& ' ./docs/java-fs-guide/code/25-architecture/run.ps1'` . The five PASS checks are: the inward dependency rule detects a domain-to-adapter leak; a context-map cycle is detected; the aggregate invariant rejects a command **and records no event**; a duplicate command is idempotent **without a second event**; and value equality differs from entity identity.

The scope limits are deliberate. The module graph is declared in the test rather than scanned from Java bytecode, so this is not ArchUnit and does not prove anything about a real codebase. The aggregate is in-memory and single-threaded, with no persistence, no transaction, no concurrency and no event publication to a broker. Nothing here measures team structure, delivery speed or migration outcomes. It demonstrates that these rules are mechanically checkable, which is the only claim being made.

Explained-Here Index and Sources

Concepts	Explanation
Monolith, modular monolith, microservices, distributed monolith	Deployment styles and choosing
Strangler fig, overlap window, dual-write risk	Incremental migration
DDD, ubiquitous language, core and generic subdomains	Domain modelling
Bounded context, context mapping, anticorruption layer	Boundaries
Aggregate, aggregate root, entity, value object	Consistency boundaries
Domain event, thin versus fat payload	Events
Hexagonal architecture, clean architecture, ports and adapters	Dependency direction
Spring package structure, module visibility	Concrete layout
API-first, contract-first, compatibility	Contracts
ADR, superseding, consequences	Decision records
Evolutionary architecture, fitness function	Keeping it true
Technical debt, deliberate versus accidental	Debt management

Consulted on 2026-10-06 as verification destinations rather than quoted authorities: [strangler fig](#), [Conway's law](#), [hexagonal architecture](#), [clean architecture](#), [ADR templates](#), [OpenAPI specification](#), [Spring Modulith](#) and [ArchUnit](#). No tool was installed, run or benchmarked.

Related Chapters

Use 00, 03 wiring, 04 persistence mapping, 05 published contracts, 07 events and sagas, 10 system design, 11 class-level design, 12 case studies, 16 delivery gates, 19 contract tests, 22 distribution costs, 24 the system end to end and 26 analytics boundaries. Generated links are reciprocal.

RELATED CHAPTERS / RECIPROCAL LINKS

00	The System Before the Details	22	Distributed Systems Theory
26	Data Platform Basics	19	Testing and Quality
10	System Design Foundations	18	Observability and Reliability
11	Object Design, Patterns and LLD	02	Concurrency and the Memory Model
12	Twelve System Design Case Studies	04	Hibernate and JPA
03	Spring and Spring Boot Internals	16	Build, Ship and Roll Back
05	APIs and Real-Time Communication	24	IntegrationHub in Motion
07	Messaging, Batch and Streams	27	Fintech and Payments Engineering
		20	Appendix: Interview Toolkit and Index

One-Page Cheat Sheet

Styles: decide from release independence, change rate, scaling profile and failure isolation. A split converts compile-time errors into runtime failures, so contracts and tracing become mandatory. Separate deployment with coupled releases is a distributed monolith and is the worst of both.

Migration: facade first, one capability at a time, reads before writes, parallel comparison, incremental traffic shift, then delete. One writer per fact during the overlap, and an agreed end date for the overlap.

Strategic DDD: ubiquitous language means the code uses the business's words. A bounded context is where one model stays consistent. Context mapping names the coupling you accepted; a cycle means neither side stands alone.

Tactical DDD: value objects are equal by value, entities by identity. An aggregate owns named invariants; keep it small and reference other aggregates by id. One aggregate per transaction is a heuristic, not an absolute rule. Rejected commands produce no accepted event; duplicates need no second effect.

Dependency rule: source dependencies point inward. The domain compiles without the framework. Ports are owned by the application, adapters implement them. Package by feature, then by layer; keep `shared` small; generated contract types stay in the adapter.

Governance: ADRs record forces, alternatives and consequences, and are superseded rather than edited. Fitness functions turn architectural rules into failing builds. Debt is a measurable interest payment, repaid where you are already working. Five local model checks passed; no Java build, no architecture scan and no organizational claim was executed.

Interview Corner

Basic: Monolith or Microservices?

Neither by default. Name the forces: number of teams needing independent release, differing change rates, differing scaling or availability needs, and required failure isolation. If those are weak, a modular monolith delivers the boundary benefit without the operational cost.

Internals: What Exactly Does a Service Boundary Change?

It replaces an in-process call with a network call that can be slow, duplicated, reordered or lost, removes the shared transaction, and turns a compile-time contract check into a runtime failure. That is why idempotency, timeouts, retries, tracing and contract tests stop being optional.

Trace/Debug: Why Can We Not Deploy This Service Alone?

Look for a shared database, synchronous call chains in the request path, and contracts that change on both sides together. Each one is a coupling that has to be removed before independence exists; until then you are paying distribution costs without the benefit.

Scenario: Break a Ten-Year-Old Monolith Apart.

Put a routing facade in front, verify transparency, choose one bounded capability with real value, move reads first, run both paths in parallel and compare, shift traffic gradually with rollback, synchronize data in one direction only, then delete the old path and its data by an agreed date.

Basic: Entity or Value Object?

If identity survives attribute change, it is an entity. If the attributes are the thing, it is a value object and should be immutable. `Money` and `TenantId` are value objects; a sync run is an entity.

Internals: How Do You Choose an Aggregate Boundary?

Name the invariant that must always hold, then take the smallest cluster that can enforce it in one transaction. Reference other aggregates by identifier and reach consistency between them with events. A large aggregate causes lock contention and loads data no rule needs.

Trace/Debug: A Downstream Consumer Processed a Change That Was Rolled Back.

The event was published before the state change was durable, or outside the transaction. Record events on the aggregate when the change is accepted and publish them after commit, using an outbox for cross-process delivery.

Scenario: Two Teams Disagree About What "Connector" Means.

That is evidence of two bounded contexts, not of a naming problem. Let each context keep its own model and language, choose the relationship explicitly — published language, anticorruption layer or conformist — and translate at the boundary instead of merging the two meanings into one class.

Internals: What Is the Dependency Rule and Why Does It Matter?

Source-code dependencies point inward toward the domain, so the domain never imports HTTP, persistence or broker code. It makes the core testable without infrastructure and lets adapters be replaced or extracted into a separate service without touching the model.

Trace/Debug: Our Hexagonal Architecture Drifted.

Ask what enforced it. If the answer is code review, drift was inevitable. Add a build-time dependency rule or module visibility so a violating import fails the build, then fix violations incrementally rather than in one large change.

Scenario: A Consumer Team Is Blocked Waiting for Your API.

Agree the contract first as a machine-readable document, publish it, and let them build against a generated mock while you implement. Keep generated types in the adapter layer, and verify both sides against the contract in CI so "we agreed" is enforced rather than remembered.

Basic: What Belongs in an ADR?

The decision, its status, the context and forces, the alternatives rejected with reasons, and the consequences including what you gave up. Supersede it with a new record rather than editing it, and keep it in the repository with the code.

Internals: What Is a Fitness Function?

An automated check for an architectural property, run in CI like a test: a dependency-direction rule, a module-visibility rule, a latency ceiling, a dependency scan, or a check that every endpoint is in the published contract. It converts an architectural intention into a mechanism.

Scenario: The Team Wants a Six-Month Refactoring Project.

Ask what it unblocks and what the current cost is per change or per incident. Prefer opportunistic repayment inside modules already being changed, record the shortcut and its trigger in an ADR, and reserve a large migration for debt that genuinely blocks a committed objective — then run it with facade, parallel run and incremental cutover.

22 / Distributed Systems Theory

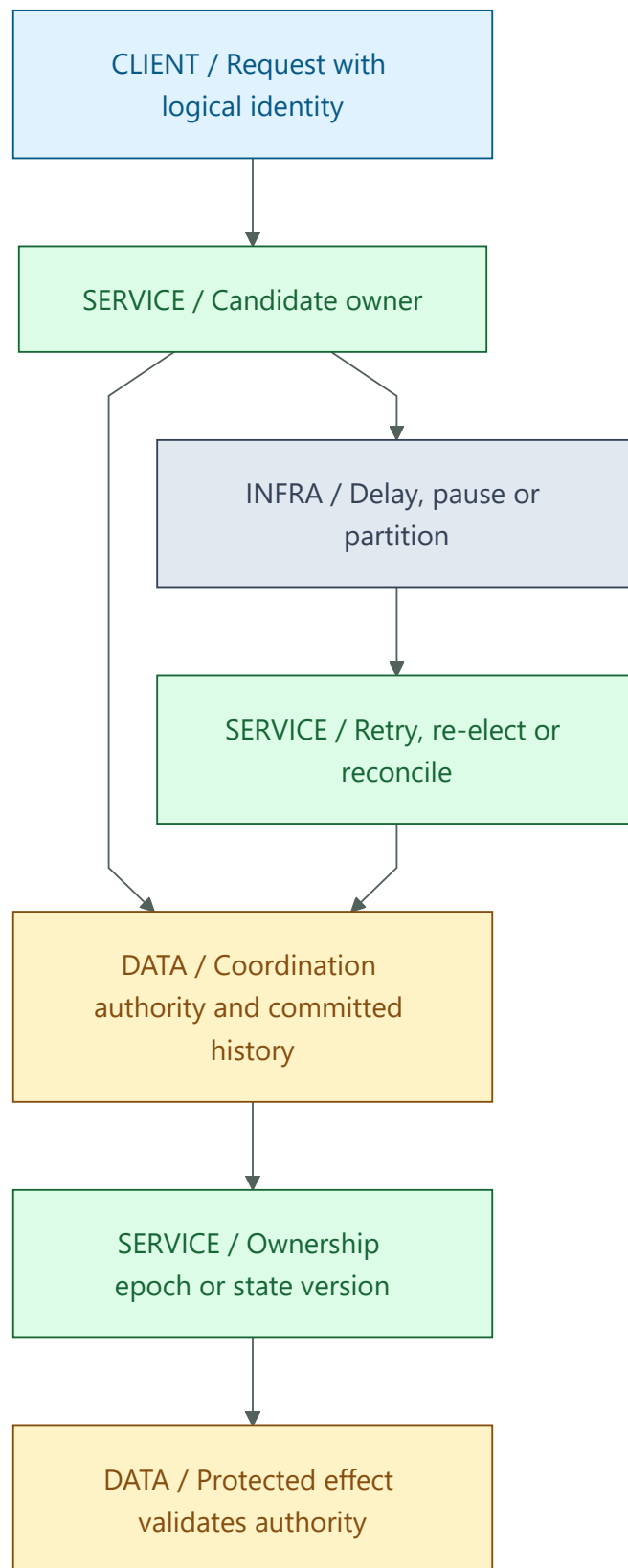
A timeout is an observation of silence, not proof of failure. A process can pause, a response can be delayed, and two healthy regions can disagree about who is reachable. IntegrationHub is fictional. This chapter makes the assumptions behind its leadership, storage and retry contracts explicit rather than borrowing guarantees from product names.

Version assumptions: Java 21, Spring Boot 4.0.x / Cloud 2025.1.x, PostgreSQL 17, Kafka 4.x and Kubernetes 1.34 remain illustrative baselines. Raft/Paxos concepts are algorithmic, not claims about a configured cluster. etcd, ZooKeeper, lease clients and Redis implementations need pinned-version review. The Redis locking documentation, Martin Kleppmann's critique and Salvatore Sanfilippo's response were consulted for the contested Redlock section.

Execution evidence: OrderingLab.mjs passed four single-process logical-clock/fencing checks. No distributed consensus, lease, quorum storage, clock fault, Redlock or regional-failover experiment was executed. A local model illustrates a boundary; it does not certify a distributed implementation.

Big Picture

Diagram 22.1



What You Will Be Able to Explain

- Separate crash, pause, omission, partition and Byzantine failure models, and safety from liveness.
- Explain physical clock error, Lamport/vector/HLC ordering and what timestamps cannot prove.
- Trace basic Paxos and Raft agreement, leader election, quorum and log-safety reasoning.
- Design one scheduled job across many Pods using durable identity, election and effect-side fencing/idempotency.
- Present Redlock's assumptions and critique fairly instead of calling every majority algorithm consensus.
- Distinguish linearizable, sequential, causal and eventual consistency; explain CAP/PACELC precisely.
- Reason about replication, quorum reads/writes, partitioning, failure detectors and the limits of exactly-once/2PC/saga claims.

[P] PRODUCTION

Where this lives in IntegrationHub. 12's scheduler needs durable fire identity and stale-owner protection; 07's consumers need effect/ack recovery; 06 owns local constraints. 10 chooses consistency by operation, 15 replaces processes, and 18 observes recovery without treating a heartbeat as proof of correctness.

1. Failure Models, Safety and Liveness

Crash-stop means a process stops and does not return in the modeled execution; crash-recovery permits return with whatever durable state survived. Omission means messages or actions may not occur; delay/reordering and partitions can make otherwise healthy peers appear unavailable. Byzantine behavior permits arbitrary faulty or malicious responses. An algorithm designed for crash faults is not automatically safe when members lie or its persistent state is corrupted outside the model.

Safety says something bad never happens, such as two conflicting committed values for one log position. Liveness says useful progress eventually occurs under stated conditions. A system can preserve safety by refusing writes when it lacks enough authority. Improving availability by accepting uncoordinated writes changes the consistency contract rather than proving the old contract still holds.

In an asynchronous model, no fixed upper delay bound distinguishes a slow process from a failed one. FLP's result concerns guaranteed termination of deterministic consensus under its fully asynchronous failure assumptions; it does not mean practical consensus is useless. Practical systems use conditions such as eventual synchrony, timeouts and randomized election behavior for progress while preserving algorithmic safety under their fault model.

Assumption	Use when	Avoid when
Crash-fault model	Members follow protocol but can stop/recover	Malicious members or corrupted persistence are silently included
Eventual synchrony	Progress is expected once delays become well behaved	A timeout is treated as proof of permanent death
Refuse unsafe operations	A consistency invariant must survive partition	Rejection is mislabeled as successful acceptance

Interview checks: basic: safety and progress differ. Internals: timeouts are suspicions. Debug: ask whether a peer stopped or only communication did. Scenario: define the fault model before claiming a quorum makes a design safe.

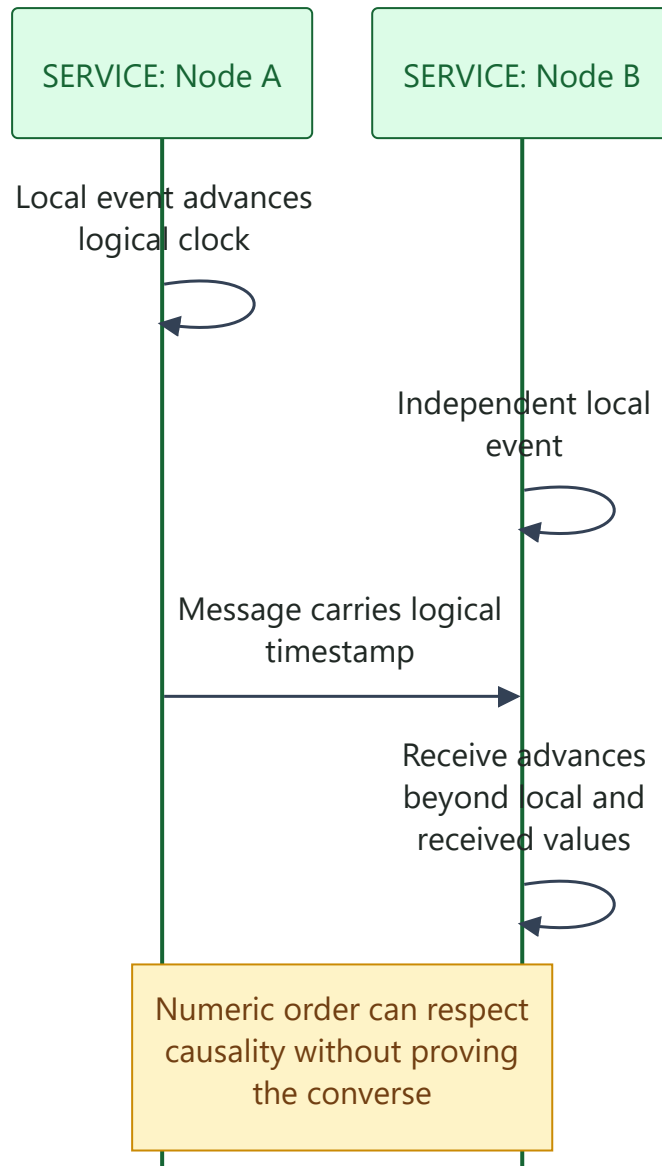
2. Physical Time, Lamport Clocks, Vectors and HLC

Wall clocks can have offset and rate error and can be adjusted. Monotonic clocks help measure elapsed time within one process/host contract, but do not create a globally comparable instant across machines. Lease safety that depends on bounded clock drift must state and enforce that assumption. A log timestamp alone is not proof of causal order.

Lamport clocks increment locally and on receive advance beyond both local and received values. If event A causally precedes B under the protocol, A's Lamport value is smaller. The converse is false: a smaller value does not prove causality. Adding a node-ID tie breaker gives a deterministic total order, not a proof that it matches physical time or business causality.

Vector clocks track components for participating actors. A vector precedes another when every component is no greater and at least one is smaller. Incomparable vectors can represent concurrent histories. This retains more causal information but increases metadata and requires careful actor lifecycle/pruning policy. Deleting components casually can discard information needed to identify conflicts.

Diagram 22.2



Hybrid logical clocks combine a physical-time component with a logical counter so timestamps can remain causally ordered under the algorithm while staying near wall time under assumptions. They do not make clocks perfectly synchronized or capture all concurrency information that a vector can express. Timestamp-based last-write-wins can discard concurrent updates; a deterministic winner is not necessarily the business-correct one.

[VERIFY: verify physical/monotonic clock, drift/offset bounds and the selected HLC/vector metadata policy in the actual system. The local lab checks Lamport receive and vector comparison only; no clock synchronization or lease-timing experiment ran.]

Interview checks: basic: physical and logical time answer different questions. Internals: Lamport's implication is one-way. Debug: equal-looking timestamps can hide concurrent events. Scenario: select version/conflict semantics from the business invariant.

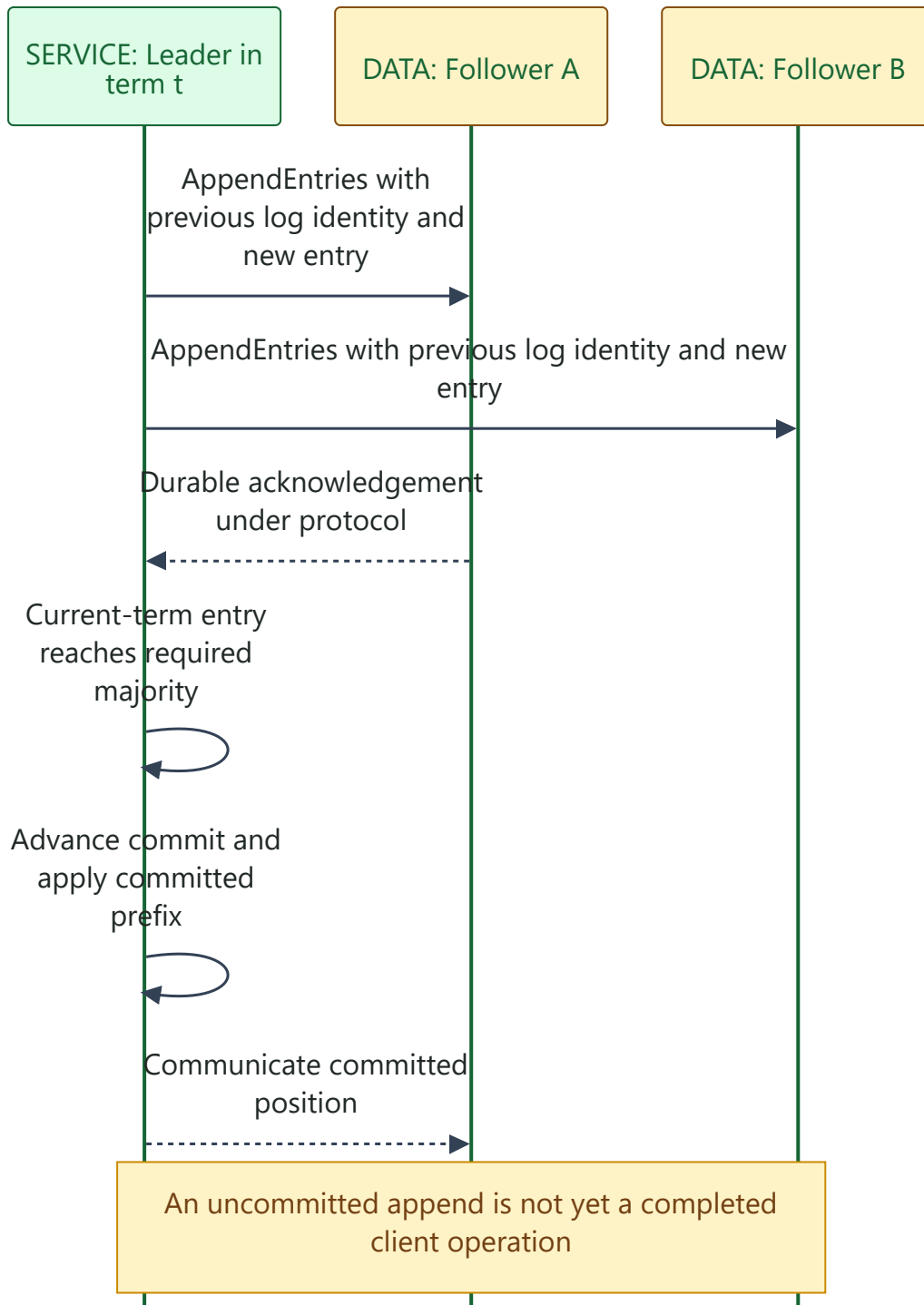
3. Consensus, Paxos and Raft

Consensus lets participants agree on a value/history under a fault model. Replicated state machines use an agreed order of commands so deterministic application of that history produces coherent state. This does not automatically include an external database transaction, HTTP call or nondeterministic side effect. Those need their own coordination/idempotency boundaries.

In basic Paxos, a proposer uses a numbered proposal. A prepare phase obtains promises from a quorum and learns relevant previously accepted proposals; the proposer must preserve a value required by that history rather than freely choose a conflicting one. An accept phase asks acceptors to accept the selected proposal, and quorum acceptance can establish the chosen value. Persistent promises/accepted state and overlapping quorums are essential. Multi-Paxos amortizes parts of agreement across a sequence under suitable leader behavior; it is not simply broadcast-and-count.

Raft organizes the problem around terms, leader election and log replication. A candidate increments term, votes for itself and requests votes. A voter grants at most its permitted vote per term and checks log freshness under the protocol. The winner needs a majority of the configured voting set. Randomized election timeout reduces repeated split votes; it does not make an isolated candidate authoritative.

Diagram 22.3



Raft checks previous log index/term to repair divergent uncommitted suffixes. Term/vote/log persistence must satisfy the implementation's crash-recovery assumptions. A leader does not declare arbitrary old-term entries committed solely by counting replicas; the current-term commitment rule is part of preserving safety, with preceding entries committed through the prefix once the rule permits it.

A leader can be stale after partition. Linearizable reads need the protocol's authority confirmation or a correctly implemented lease/read-index mechanism; merely reading a former leader's local state is insufficient. Membership changes must preserve the required quorum relationships through a supported reconfiguration protocol. Independently changing node lists can destroy the intersection reasoning.

Coordination choice	Use when	Avoid when
Proven consensus service	Critical shared metadata needs one agreed authority	A hand-written majority loop substitutes for a complete protocol
Replicated state machine	Ordered deterministic commands fit the domain	External side effects are assumed rolled back by log agreement
Quorum refusal	Safety must survive lost majority	Split clusters both accept conflicting authoritative writes

[VERIFY: verify Raft/Paxos implementation persistence, read semantics, membership change and crash-recovery assumptions against the selected proven service and primary algorithm references. No consensus algorithm or cluster was implemented or tested here.]

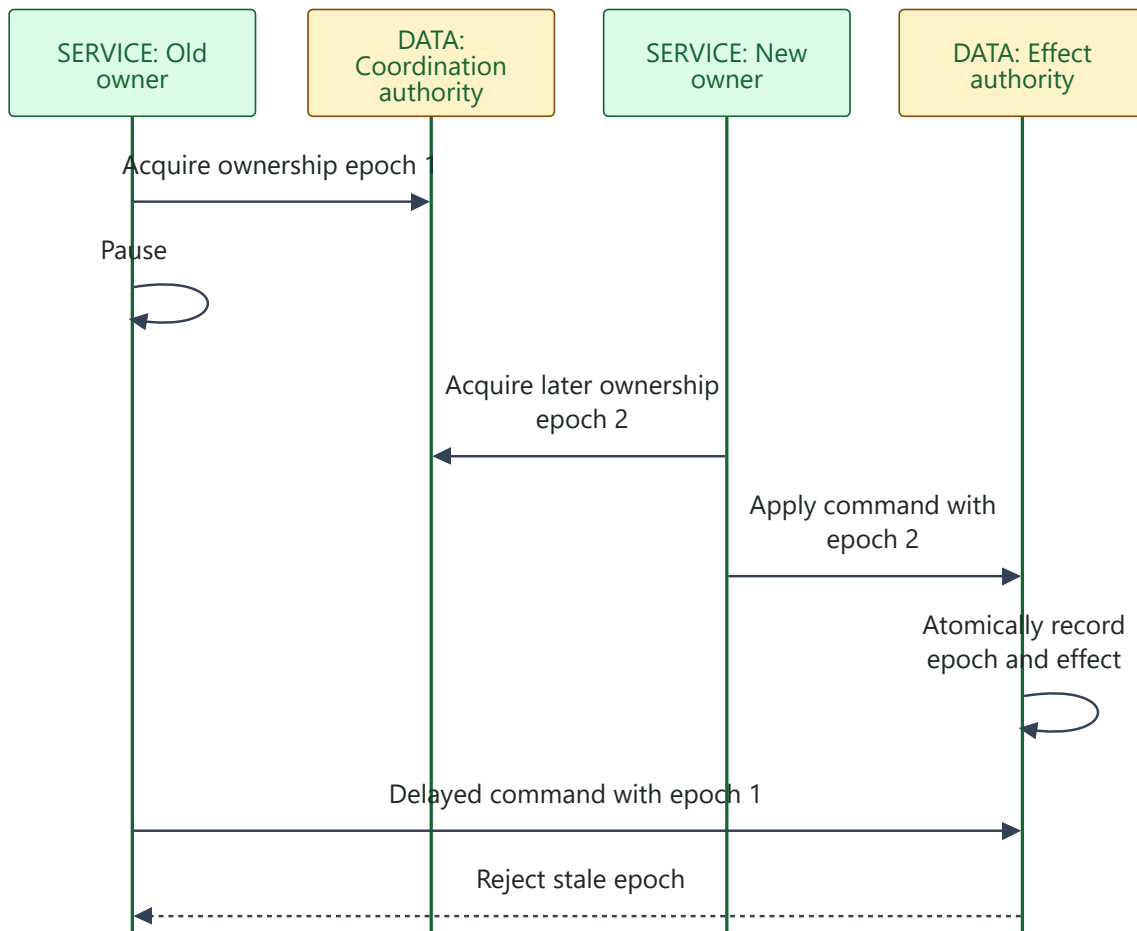
Interview checks: basic: agreement is more than replication. Internals: quorum overlap works with protocol rules and durable history. Debug: distinguish append from commit. Scenario: use a proven coordination service and explain how its guarantee reaches the application effect.

4. Leader Election and One Scheduled Job Per Fleet

etcd provides a coordination store built around its consistency and lease/watch APIs; ZooKeeper provides coordinated state, sessions and recipes such as ephemeral sequential-node election. Kubernetes Lease objects can coordinate leader-election clients under their own update/timing protocol. A database can arbitrate ownership through transactional rows, conditional writes or advisory locking. Each mechanism has a different failure/session/resource scope.

The useful application question is not simply which Pod is leader, but what the old leader can still do. It may pause after checking ownership, lose its lease, then resume while a new leader runs. A watch/disconnect notification is not instantaneous and cannot reliably recall an already sent provider request.

Diagram 22.4



For one scheduled IntegrationHub sync, use a stable fire-instance identity such as schedule/version/logical fire time, durable claiming and an idempotent effect. Election can reduce duplicate planning work, but unique fire records and protected execution remain valuable after leader loss. A cron callback on each Pod plus a local boolean is not distributed scheduling.

Database advisory locks can be session- or transaction-scoped under engine rules; losing a connection releases ownership according to that contract but does not undo an already sent external effect. Row leases need atomic ownership/expiry updates and a clock authority policy. Avoid holding a database transaction open for arbitrary long provider work merely to simulate a distributed mutex.

[VERIFY: check etcd/ZooKeeper lease/session/watch behavior, Kubernetes leader-election client semantics, database lock scope and usable fencing revisions for the selected versions. No election, disconnect, expired lease or singleton scheduling experiment ran.]

Interview checks: basic: election selects an owner under a protocol. Internals: lost ownership does not stop a process. Debug: inspect durable fire identity and external effects. Scenario: protect the effect, not only the election record.

5. Distributed Locks and Fencing Tokens

A lease bounds how long ownership is intended to remain valid. A random unique lock value can help release only the caller's own lock using atomic compare-and-delete. It is not automatically a monotonically ordered fencing token. Deleting an expired/replaced lock without checking its owner can remove someone else's lock.

Fencing requires a monotonically ordered authority token and an effect-side check that rejects stale authority atomically with the protected mutation. Every relevant writer must participate. Sending the token to a service that ignores it establishes nothing. The resource's remembered highest epoch must survive the failure modes for which safety is claimed.

There is an important boundary: issuing epoch 2 in a coordination service does not instantly inform every protected resource. A resource that has only seen epoch 1 can still accept it under a highest-seen-token policy until the newer fence reaches it. If the requirement is immediate revocation at grant time, the grant/resource protocol must establish that stronger property. Within one epoch, message ordering and duplicate effects may still need sequence/version or idempotency checks.

OrderingLab makes that distinction explicit: it accepts an old-owner write before the higher epoch reaches the local resource, then rejects it after epoch 2 is applied. This is not a lease implementation, durable storage or a distributed safety proof. It illustrates where enforcement occurs.

[M] MECHANISM

Fencing works at the effect boundary. The resource atomically compares authority with its stored state before applying a mutation. A client-side expiry check can be followed by an arbitrary pause and is therefore a different, weaker mechanism.

Lock use	Use when	Avoid when
Best-effort duplicate-work suppression	Occasional duplicate computation is acceptable	Correctness depends on perfect exclusion but this is undocumented
Lease plus protected effects	Stale owners must not corrupt a supported resource	The target ignores fencing/idempotency metadata

Lock use	Use when	Avoid when
Local transaction/constraint	The invariant fits one database authority	A distributed lock is added instead of using the existing atomic boundary

Interview checks: basic: unique owner token and fencing order differ. Internals: enforcement must be atomic with effect. Debug: check every write path and durable epoch state. Scenario: a provider without fencing support needs idempotency or reconciliation under its own contract.

6. Redlock: Algorithm, Critique and Decision

The Redis documentation describes acquiring a key with the same unique owner value and TTL on independent masters, accepting the acquisition only after obtaining the required majority quickly enough and subtracting elapsed time plus a drift allowance from validity. Failed partial acquisitions are released, and release must compare the owner value atomically. This is not a Raft log and does not inherit Raft's guarantees merely because it uses a majority.

The proposed argument relies on an interval in which a majority of lock keys coexist under timing/persistence assumptions. During that interval, a competing acquisition cannot also set a majority of the same keys. Client work must stay inside the valid interval, and restart/persistence policy must prevent old lock information from disappearing unsafely. Lease extension also needs a valid protocol, not an assumption that a live process owns the lock forever.

Kleppmann's critique distinguishes efficiency locks from correctness locks. For correctness, a paused client or delayed request can act after lease expiry, and timing/clock assumptions can fail. He argues for effect-side fencing and a stronger coordination approach where required, noting that a random token alone is not a monotonic fencing token. He also questions the value of a multi-node protocol for merely approximate duplicate-work suppression.

Sanfilippo's response defends the algorithm under bounded relative-time error and emphasizes that it measures elapsed acquisition time: a delayed acquisition response should not simply be accepted as a fresh full lease. He separates delays during acquisition from work continuing after expiration, discusses resource-specific check-and-set/token approaches and persistence/restart assumptions, and argues that the critique's requirements do not fit every use case. That is not equivalent to claiming arbitrary pauses after validation are harmless.

The current Redis page itself highlights fencing and warns about TTL/clock concerns. The practical decision is to specify the target resource and failure assumptions, not declare every Redis lock safe or every Redis use invalid. For IntegrationHub's correctness-sensitive job/ledger

state, prefer its database transaction/unique identity or a proven coordination service plus effect protection. Use approximate locks only when duplicate work is an accepted efficiency cost.

[VERIFY: Redlock is contested. Consult the Redis distributed-lock specification, Kleppmann's analysis and Sanfilippo's response, and validate the exact client/server timing, persistence, renewal and effect-side fencing assumptions. All three sources were consulted; no Redis/Redlock or fault-injection experiment was executed.]

Interview checks: basic: majority does not automatically mean consensus. Internals: elapsed acquisition checks and post-expiry effects are separate issues. Debug: state whether exclusion protects efficiency or correctness. Scenario: choose a resource-enforced safety contract rather than relying on a library name.

7. Consistency Models

Linearizability makes each operation appear atomic at a point between invocation and response while respecting real-time ordering. Sequential consistency provides one order consistent with each process's program order but need not preserve the same real-time precedence across processes. Causal consistency preserves dependencies between causally related operations while allowing unrelated operations to be ordered differently. Eventual consistency promises convergence under its stated conditions, not any particular intermediate read result.

Read-your-writes and monotonic reads are useful session guarantees. A client switching replicas can otherwise see its own write disappear or observe older state after newer state. A browser's local optimistic state is not a storage consistency guarantee; the API must explain when authority or projections become visible.

Linearizability of individual operations is not the same as serializability of multi-operation transactions. Two individually atomic reads/writes do not automatically make a read-modify-write sequence atomic. Strict serializability combines transaction serializability with real-time constraints under its definition. Pick the scope needed by the invariant, not the strongest-sounding adjective.

Model	Use when	Avoid when
Linearizable operation	One latest authority decision is required	Multiple operations are assumed to form a transaction automatically
Causal/session guarantees	Dependency order and user continuity matter	Concurrent updates need a total business order that is not supplied

Model	Use when	Avoid when
Eventual projection	Search/analytics can tolerate bounded operational lag policy	Stale data authorizes spending or cross-tenant access

Interview checks: basic: eventual does not mean instantly correct. Internals: real-time and program/causal order differ. Debug: identify which replica and session read the value. Scenario: use the minimum guarantee that actually preserves the invariant.

8. CAP and PACELC in Practice

CAP concerns the inability to guarantee both linearizable consistency and its formal availability property during a network partition under the theorem's assumptions. It is not a permanent choose-any-two menu, and partition tolerance is not an option to turn real network failures off. Practical systems choose per operation what may reject, wait or return weaker results.

For IntegrationHub, a stale search result may be acceptable, but a conflicting authoritative ownership grant should not be reported as valid on both sides of a partition. A pending durable command can be a valid API result if that is the contract, but relabeling uncommitted work successful does not satisfy the original consistency promise. Timeouts and retries need explicit user-visible outcomes.

PACELC points to another trade-off: without a partition, coordination for stronger consistency can still cost latency. Cross-region acknowledgement choices affect normal operation as well as disaster behavior. Neither acronym tells you whether a specific database configuration has a guarantee; inspect the service's actual protocol, read mode and failure policy.

[!] TRAP

Availability percentages and CAP availability are not the same definition. A system can have an excellent operational uptime SLO while deliberately rejecting unsafe writes during a partition. State which meaning you are using.

Interview checks: basic: CAP is about a failure condition and precise properties. Internals: normal-operation coordination still has latency cost. Debug: identify which operation degraded and how. Scenario: decide stale read versus rejected write per business requirement.

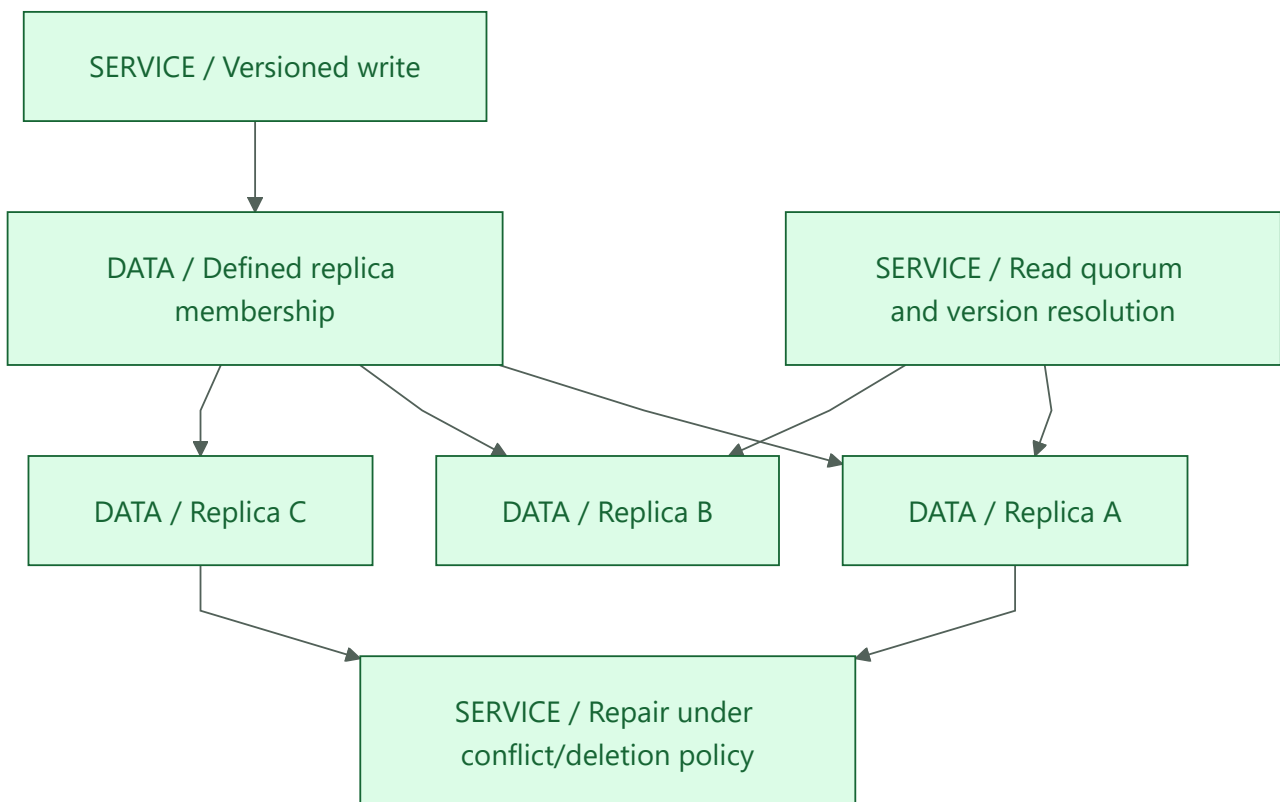
9. Replication and Quorum Reads/Writes

Leader-follower replication centralizes write ordering and replicates to followers with chosen acknowledgement and read policies. Multi-leader replication permits writes in several places and must handle conflicting histories. Leaderless systems coordinate reads/writes across replica sets under versioning, repair and quorum policy. Each shifts conflict and availability responsibilities; none can be summarized safely by node count alone.

For a fixed replica set of size N , $R+W>N$ gives an intersection between a read set of size R and a completed write set of size W . This is a useful set fact, not a complete linearizability proof. Concurrent/partial writes, version selection, durable acknowledgements, membership changes and repair behavior matter. Sloppy quorums using fallback nodes can change the intersection assumptions entirely.

Read repair and anti-entropy help converge divergent replicas; hinted handoff can temporarily store work for an unavailable replica under its contract. Repair must distinguish obsolete versions and deletions. A deleted item can reappear if tombstone retention is shorter than the supported offline/repair horizon and no stronger reconciliation protects it.

Diagram 22.5



Last-write-wins using wall time can discard a valid concurrent update if clocks differ or operations conflict semantically. Vector versions can expose concurrency but do not choose the business resolution for you. Some operations can use carefully chosen convergent data types; not every invariant, such as bounded balance spending, can be solved by an arbitrary merge.

[VERIFY: validate read/write quorum membership, durability, conflict resolution, repair, tombstones and failover against the selected storage implementation. $R+W > N$ alone is not an asserted product consistency guarantee; no replicated-store experiment ran.]

Interview checks: basic: quorum overlap is necessary reasoning, not a whole protocol.

Internals: concurrent versions and membership affect results. Debug: inspect stale repair/tombstone behavior. Scenario: make conflict resolution a domain decision.

10. Partitioning, Consistent Hashing and Split Brain

Partitioning assigns keys/work to ownership ranges. Range partitioning supports ordered access and can hotspot recent ranges; hash partitioning spreads many keys but can make range queries expensive. Consistent hashing reduces expected remapping when membership changes under its model. Virtual nodes can improve distribution and rebalancing flexibility, but a single hot key still belongs somewhere.

Partition metadata itself needs an authority and transition protocol. During movement, define who accepts writes, how data is copied, how changes are caught up, how stale routes are handled and when the old owner is fenced. Copying rows then flipping an unversioned map can lose concurrent writes. Tenant locality helps common operations but a giant tenant may require a further partitioning strategy.

Split brain means participants operate under conflicting authority views, often after partition/failover. Preventing it may require rejecting writes without quorum, fencing old owners or using a conflict-resolution model that genuinely tolerates concurrent writers. Rejoining two regions is not automatically safe merely because connectivity returns; divergent effects need reconciliation.

Placement choice	Use when	Avoid when
Range partition	Ordered locality and retention operations matter	A moving hot range overloads one owner
Hash/consistent hashing	Broad keys and membership changes justify distribution	Hot keys and metadata authority are ignored
Tenant affinity	Most invariants stay within one tenant	One tenant exceeds a shard and no migration strategy exists

Interview checks: basic: hashing distributes keys, not arbitrary workload evenly. Internals: movement needs a write-authority protocol. Debug: inspect routing generation and overlapping ownership. Scenario: reconcile effects before reopening divergent writers.

11. Heartbeats, Phi Accrual and Gossip

A heartbeat tells an observer that some sender/path was responsive at a moment. Failure detection turns elapsed silence and other signals into suspicion under a policy. Phi-accrual detectors express a suspicion measure based on observed arrival behavior rather than only one fixed interval; threshold choice still trades false suspicion against detection delay and depends on the statistical model/workload.

Gossip disseminates membership/state through repeated peer exchanges. It can scale dissemination and tolerate partial failure, but membership convergence is not consensus on one authoritative write. A node absent from one observer's view may still be processing work elsewhere. Do not use gossip membership alone to grant mutually exclusive correctness-sensitive ownership.

Detection and action must be separated. Suspecting a leader can trigger an election attempt, but the new owner still needs the coordination protocol's authority and effect-side protections. Aggressive timeouts can create repeated elections and load precisely when the system is slow. Conservative timeouts may preserve stability but delay recovery; measure and set policy accordingly.

Interview checks: basic: detection is suspicion, not certainty. Internals: adaptive thresholds still rely on assumptions. Debug: distinguish membership dissemination from authoritative agreement. Scenario: a heartbeat timeout must not directly authorize conflicting external effects.

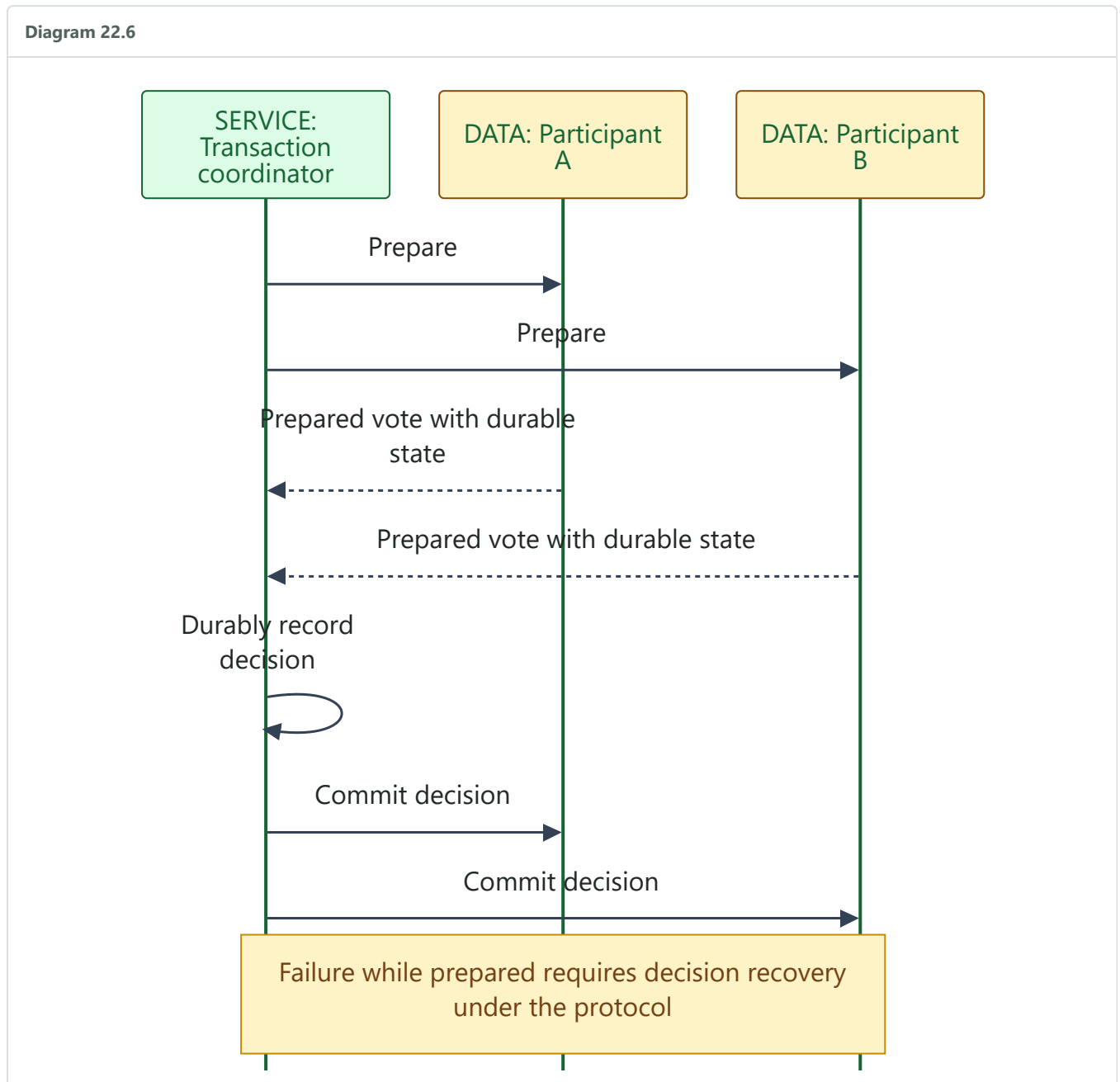
12. Exactly-Once Effects, 2PC and Saga

Exactly-once needs a named boundary: records emitted by a broker transaction, one local database mutation, or an external provider effect. Re-executing code can still produce one committed effect if identity and atomicity suppress duplicates. Conversely, processing two separately identified business duplicates once each still creates two effects.

Two-phase commit coordinates atomic outcome among participating resource managers. In prepare, participants durably promise that they can commit or vote no; after the required votes the coordinator records and communicates a decision. A prepared participant may need to

retain locks/resources while awaiting recovery of the decision. Coordinator replication can improve decision availability, but 2PC is not simply replaced by saying consensus, and arbitrary HTTP endpoints do not become transaction participants automatically.

Diagram 22.6



A saga commits local steps and uses business compensations, with visible intermediate states and possible failed/unknown compensation. It is not an isolated distributed rollback. Choose 2PC when actual supported participants and operational constraints justify atomic coordination; choose saga when business semantics can tolerate and repair staged effects. Keep one local transaction when the invariant already belongs to one owner.

Interview checks: basic: execution count and effect count differ. Internals: prepared state has recovery obligations. Debug: inspect which resources actually participate. Scenario: external provider uncertainty often needs idempotency/reconciliation rather than pretending it joined a transaction.

13. Executed Model and Evidence

`code/22-distributed/OrderingLab.mjs` and its Windows PowerShell runner use the existing Node-compatible runtime. Command from workspace root: `& ' ./docs/java-fs-guide/code/22-distributed/run.ps1'`. The actual four PASS checks cover Lamport receive/overflow, vector order and concurrency, the need for a higher epoch to reach the resource, and stale-write rejection afterward.

The register is single-process and not persistent; epochs are supplied by the test rather than generated by a distributed authority. It has no lease expiry, membership, crash recovery, quorum storage or consensus. Multiple writes within the same epoch are allowed, so separate operation idempotency/order remains a different contract. These limits are intentional and prevent a local test being advertised as proof of Redlock or Raft.

[D] DECISION

Put the invariant at an authority that can enforce it. Prefer a database constraint/transaction when sufficient. Add a proven coordination service when ownership really spans processes, then make sure its authority is enforced at every relevant effect boundary.

[?] INTERVIEW

State assumptions before guarantees: who may fail, which state survives, what time can be trusted, which quorum/membership applies, and where a stale actor is rejected. That makes a distributed answer falsifiable.

Explained-Here Index and Sources

Concepts	Explanation
Failure models	Safety and liveness
Clock skew, Lamport, vector and hybrid logical clocks	Time and order
Consensus, Paxos, Raft, quorum	Agreed history

Concepts	Explanation
Leader election, etcd, ZooKeeper, Kubernetes lease	Ownership
Distributed lock and fencing token	Protected effect
Redlock	Contested assumptions
Linearizable, sequential, causal, eventual consistency	Read/write guarantees
CAP and PACELC	Availability and latency trade-offs
Leader-follower, multi-leader, leaderless	Replication
Consistent hashing and split brain	Partition ownership
Heartbeats, phi accrual and gossip	Failure detection
2PC and saga	Atomicity and compensation

Consulted on 2026-10-06: [Redis distributed locks](#), [Kleppmann's analysis](#) and [Sanfilippo's response](#). Additional verification destinations: [Raft paper/resources](#), [etcd documentation](#) and [ZooKeeper](#). Source discussion is not a fault test or an assertion that historical implementation details remain current.

Related Chapters

Use [00](#), [02](#) memory ordering, [05](#) identity, [06](#) database, [07](#) messaging/sagas, [10](#) consistency choices, [12](#) scheduler, [15](#) leases, [18](#) recovery, [21](#) transport, [23](#) pauses and [27](#) monetary effects. Generated links are reciprocal.

RELATED CHAPTERS / RECIPROCAL LINKS

00	The System Before the Details	25	Architecture Styles and Domain Design
02	Concurrency and the Memory Model	12	Twelve System Design Case Studies
21	Networking and Operating Systems	15	Kubernetes and Production Workloads
03	Spring and Spring Boot Internals	18	Observability and Reliability
06	Databases and Durable State	23	JVM Performance Engineering
05	APIs and Real-Time Communication	27	Fintech and Payments Engineering
07	Messaging, Batch and Streams	17	Cloud and Infrastructure
10	System Design Foundations	24	IntegrationHub in Motion
		20	Appendix: Interview Toolkit and Index

One-Page Cheat Sheet

Failures/time: silence is suspicion. Safety and progress have different assumptions. Wall time is not causal proof; Lamport order is one-way evidence, vectors can represent concurrency and HLC does not create perfect clocks.

Agreement: proven consensus preserves agreed history through its quorum, persistence and membership rules. Append is not commit; stale leaders need valid read/write authority. A majority loop is not automatically Raft/Paxos.

Ownership: leases expire without stopping clients. Unique lock values prevent another owner's unlock; monotonic fences require atomic effect-side enforcement. A new token must reach the protected resource to change its highest-seen policy.

Redlock: compare stated timing/persistence assumptions, elapsed acquisition checks and the separate post-expiry effect problem. Both critique and response matter. Use approximate locks only where duplicate work is acceptable; correctness needs the actual resource contract.

Storage: consistency scope matters; $R+W > N$ is an intersection fact, not a full proof. Membership, concurrent writes, repair and tombstones matter. Hashing does not remove hot keys. Gossip spreads views, not exclusive authority.

Transactions: exactly-once names an effect boundary. 2PC requires real prepared participants and decision recovery; sagas compensate visible local commits. Four local model checks passed; distributed algorithms and failures were not executed.

Interview Corner

Basic: Safety Versus Liveness?

Preventing invalid outcomes versus eventually making progress under stated conditions. A majority-loss write refusal can preserve safety while reducing availability.

Internals: Does a Smaller Lamport Timestamp Prove Causality?

No. Causality implies the ordering under the protocol, not the converse. Independent events can still receive numerically ordered timestamps.

Trace/Debug: Two Pods Both Think They Are Leader.

Inspect terms/lease state, delays and the effect-side authority check. Local belief or an old successful acquire response is not enough to permit a current protected write.

Scenario: Run One Scheduled Job Across Many Pods.

Use a durable fire identity and atomic claim, optionally coordinated election to reduce duplicate planning, plus idempotency/fencing at the effect. Recovery must handle a paused old owner returning.

Basic: Is Redlock Consensus?

No. Its independent-master timed acquisition protocol has different assumptions from a replicated consensus log. A shared use of majorities does not transfer guarantees.

Internals: What Must a Fencing Target Do?

Atomically compare authority and mutate protected state, persist relevant authority history, and reject stale writers on every path. A token ignored by the target does nothing.

Trace/Debug: The New Lease Exists but the Old Write Was Accepted.

Under a highest-seen fence, the higher token may not yet have reached that resource. Stronger immediate-revocation requirements need a protocol coordinating grant and resource authority.

Scenario: How Do You Present the Redlock Debate?

Explain elapsed acquisition validity and relative-time assumptions, then the critique's delayed-client/effect concerns and fencing requirement. State target resource and failure model instead of issuing an unqualified product verdict.

Basic: Linearizable Versus Serializable?

Linearizability concerns atomic operations respecting real time; serializability concerns transactions equivalent to a serial order. Scope and real-time requirements must be stated separately.

Internals: Why Is $R+W>N$ Insufficient Alone?

It assumes a fixed replica set and says nothing by itself about concurrent versions, durable acknowledgements, partial writes, sloppy membership or conflict resolution.

Trace/Debug: Deleted Data Returned After Repair.

Inspect tombstone retention, offline replica duration, stale versions and repair policy. Convergence must preserve deletion information under the supported recovery horizon.

Scenario: 2PC or Saga?

Use actual resource participation, isolation needs and recovery costs to decide. 2PC coordinates an atomic decision with prepared-state obligations; saga exposes local commits and business compensation. Prefer one local transaction when it already owns the invariant.

11 / Object Design, Patterns and Twelve LLDs

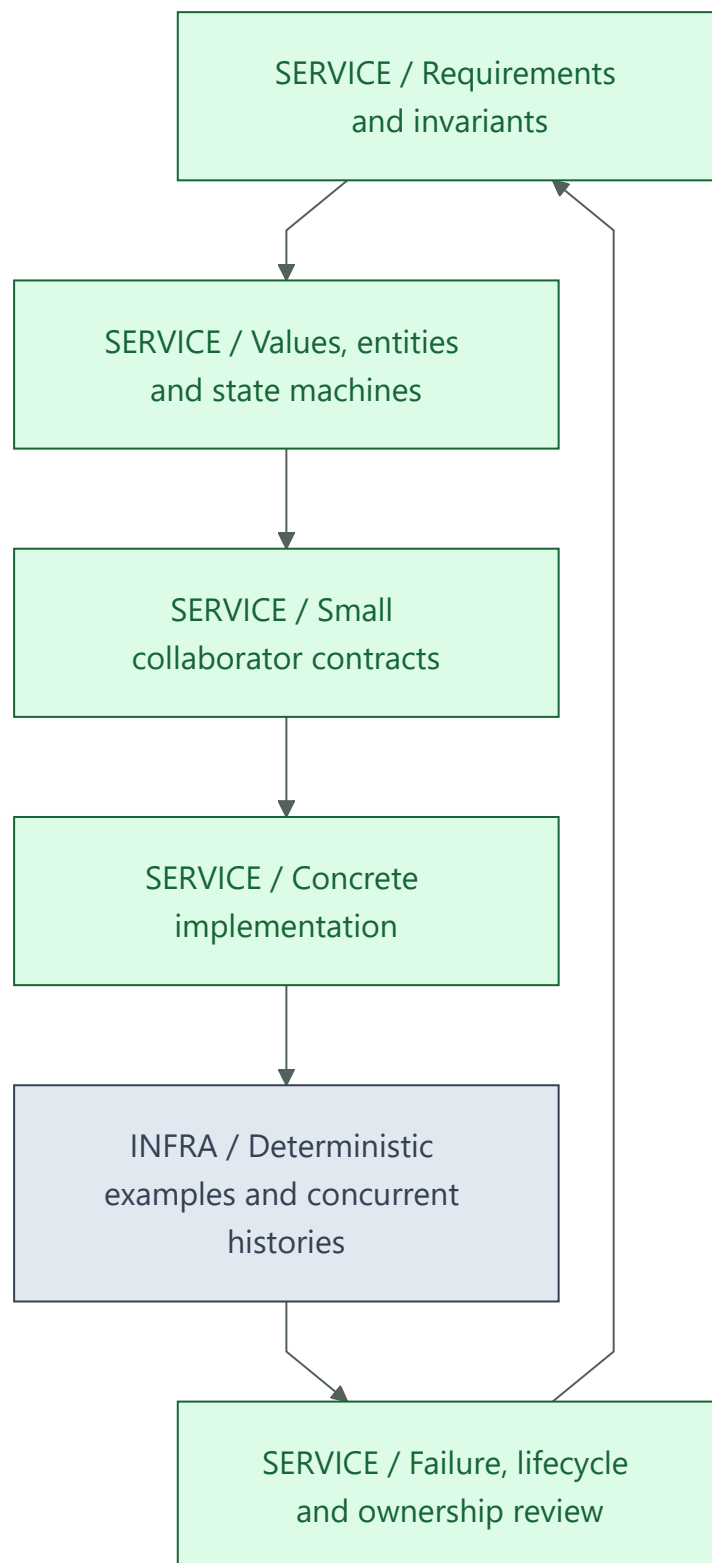
A useful object boundary protects an invariant and makes change local. A diagram with many interfaces is not automatically flexible, and a class named Manager is not automatically an owner. This chapter turns the fictional IntegrationHub's local contracts into twelve small, complete in-process models. They are teaching implementations, not production services or hardware controllers.

Version assumptions: Java 21 without preview features; Spring Boot 4.0.x / Cloud 2025.1.x, PostgreSQL 17, Kafka 4.x and Kubernetes 1.34 remain the surrounding guide baselines. These plain-Java examples require none of those frameworks. Their clocks, senders and tasks are injected so tests need no network, sleeps or external accounts.

Execution status: NOT EXECUTED. The compiler preflight found no existing JDK and downloads remain unauthorized. `code/11-11d/L1dLab.java` contains all twelve implementations and twelve test methods, including an arranged concurrent idempotency check. The offline runner is supplied; compilation and assertion outcomes are unverified, not passes.

Big Picture

Diagram 11.1



What You Will Be Able to Explain

- Apply encapsulation, substitution, composition and SOLID to real contracts rather than slogans.
- Choose Strategy, Factory, Builder, Singleton, Observer, Decorator, Adapter, Template Method, Proxy, Chain of Responsibility, Command and State deliberately.
- Design twelve requested problems with explicit APIs, bounded assumptions, ownership and tests.
- Identify linearization points, callback hazards, expiry races, duplicate ownership and unknown external outcomes.
- Distinguish a complete local teaching model from the additional persistence, recovery, scale and security needed in production.

[P] PRODUCTION

Where this lives in IntegrationHub. Connector selection uses strategies and factories; job admission uses rate/concurrency policies; local caches and progress subscriptions have lifecycle contracts. 02 owns the memory model, 05 owns HTTP identity, 07 owns durable delivery, 08 owns trust and 10 owns distributed capacity. 25 extends local boundaries into domain architecture.

1. OOP, SOLID and Contracts

Encapsulation hides representation behind operations that preserve invariants. Making every field private but exposing unrestricted setters does not achieve that. An immutable value represents a fact by content; an entity represents continuity of identity through state changes. Equality, ownership and lifecycle should follow that distinction. Inheritance can express substitutability, but composition often keeps independent reasons for change separate.

Single responsibility means a coherent reason to change, not one method per class. A parking allocator should not also format invoices and send emails. Open/closed design makes a useful extension axis replaceable without changing unrelated code; it does not require an interface for every private helper. Liskov substitution means clients relying on the base contract remain correct for each implementation, including error, mutability and timing expectations.

Interface segregation keeps clients dependent on capabilities they actually use. A read-only cache consumer should not need an administrative eviction API. Dependency inversion places stable domain needs above concrete adapters: a notifier depends on a sender contract, not a vendor SDK everywhere. DI is one way to supply collaborators, not proof that the chosen abstractions are good.

Write the contract before the classes: valid input, results, errors, state transition, concurrency scope, lifetime, resource bounds and recovery behavior. For a cache, decide whether get changes recency and whether null values exist. For a scheduler, decide whether cancel can stop an already claimed task. Those decisions determine locks and APIs more directly than a pattern name.

Principle	Use when	Avoid when
Encapsulated transition	Several fields must change coherently	Setters allow impossible intermediate state
Composition	Behavior varies independently across collaborators	Inheritance multiplies combinations of unrelated choices
Small interface	Multiple useful implementations or a clear boundary exist	Every class gets a matching interface without a reason
Immutable value	Shared observations should be stable	A mutable external object remains reachable through the value

Interview checks: basic: private fields are not enough. Internals: substitution includes failure semantics. Debug: look for multiple owners of one state transition. Scenario: choose the actual extension axis before adding an interface.

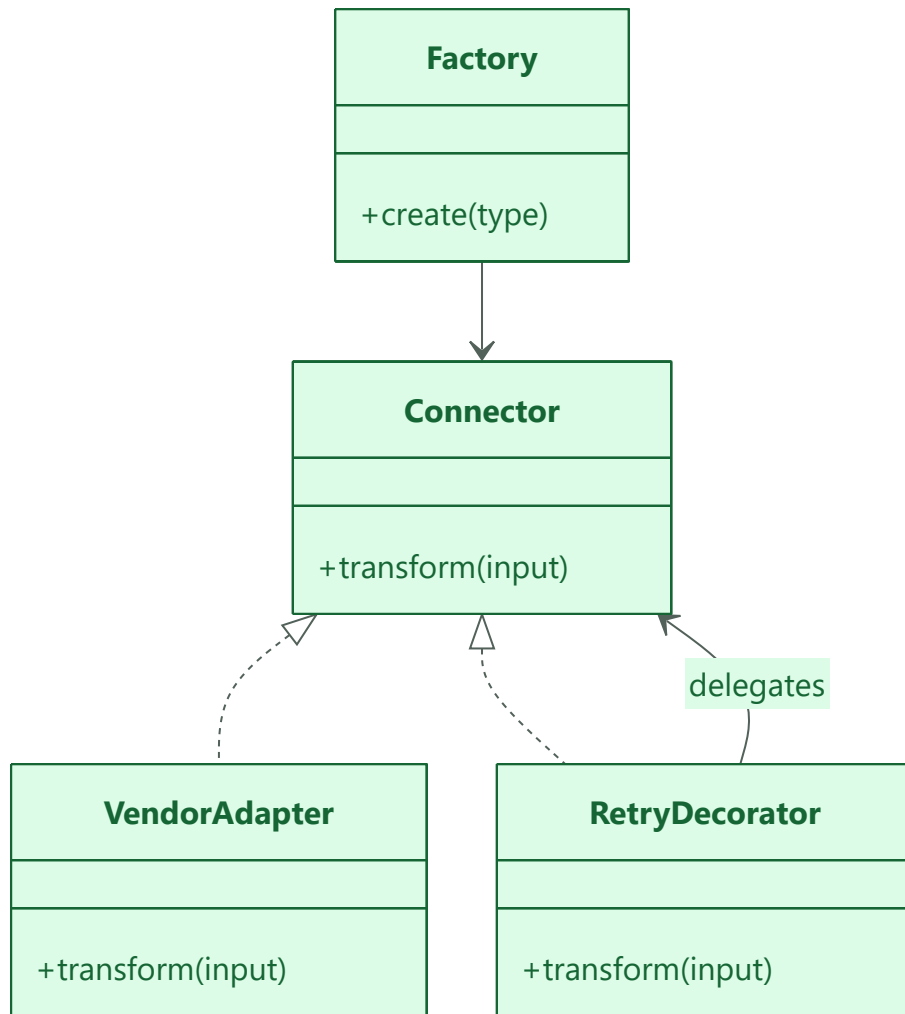
2. Pattern Vocabulary With Consequences

Patterns name recurring design choices. They are useful when the name communicates intent and trade-offs, not when it substitutes for a contract. Several can coexist: a factory selects a strategy, an adapter translates a vendor client, and a decorator applies a bounded retry policy around it.

Pattern and mechanism	Use when	Avoid when
Strategy selects an interchangeable algorithm	Connector mapping or allocation policy varies	Tiny fixed branching becomes an unnecessary hierarchy
Factory centralizes construction/selection	Type names/configuration choose valid implementations	Unknown types silently fall back to the wrong connector
Builder accumulates configuration then validates once	Many optional parameters have cross-field constraints	A two-field value gets a verbose mutable construction API
Singleton constrains one instance per scope	A process-scoped stateless registry has explicit ownership	It is claimed to be one instance across a cluster or used as global mutable state
Observer notifies registered listeners	Local change fan-out is useful	Durable messaging is implied by an in-memory callback list

Pattern and mechanism	Use when	Avoid when
Decorator wraps the same interface	Retry, metrics or validation should compose	Ordering and repeated effects are undocumented
Adapter translates between contracts	A vendor API differs from the domain port	Vendor errors/leaky types spread into the domain
Template Method fixes an algorithm skeleton with hooks	Subclasses truly share lifecycle and invariants	Hooks expose fragile superclass state or substitution fails
Proxy controls access to another object	Lazy access, remoting or authorization needs mediation	Calls bypass the proxy while advice is assumed to run
Chain of Responsibility passes through ordered handlers	Validation/routing stages may handle or delegate	Order-dependent behavior is hidden and no handler owns failure
Command packages an operation	Queuing, scheduling or auditing an action is useful	A lambda is mistaken for a serializable durable job
State delegates behavior by lifecycle state	Allowed transitions differ materially by state	Classes obscure a small, clearer enum transition table

Diagram 11.2



The lab implements Strategy/Factory/Decorator in the connector example, Observer in pub/sub, Command in scheduling and explicit State enums in dispatch. Builder, Singleton, Adapter, Template Method, Proxy and Chain are explained alternatives, not artificially inserted into every exercise. 03 explains real Spring proxy boundaries, including self-invocation.

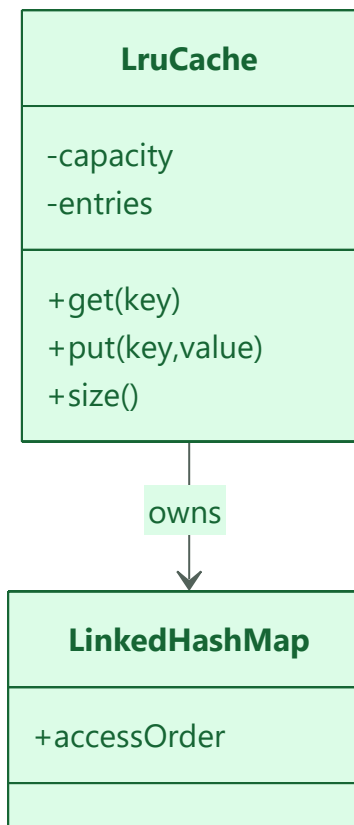
[D] **DECISION**

Use the smallest structure that explains the behavior. An enum and transition function can be a better State implementation than five classes. A constructor can be a better Builder. Show the growing requirement that would justify the more elaborate form.

3. LLD 1: LRU Cache

Requirements/API: fixed positive entry capacity; put inserts or replaces; get returns an optional value and makes a hit most recently used; null keys/values are rejected. Capacity bounds entry count, not bytes. The cache is process-local and does not load missing values or expire entries.

Diagram 11.3



Mechanism: an access-ordered `LinkedHashMap` maintains the least-recent entry at the iteration head. A hit moves the entry, insertion appends, replacement updates recency, and overflow removes the first key. This reuses the standard library rather than inventing a linked list. The `LruCache` implementation in `LldLab.java` synchronizes `get`, `put` and `size` on one monitor because a successful `get` mutates recency.

Concurrency/failure: insertion and eviction happen within the same critical section, so a completed operation leaves `size` within capacity. Returned values are not deep copies; callers must use immutable values or provide their own ownership rule. A slow loader should not be added inside this monitor. A production weighted cache, expiry and single-flight loader are separate requirements and may justify an established cache library.

Tests: testLru covers hit recency, eviction of the untouched key, replacement without growth and invalid capacity. Add concurrent history testing when a JDK exists; the current tests do not prove contention performance. IntegrationHub uses this shape for bounded local metadata, not authoritative job state.

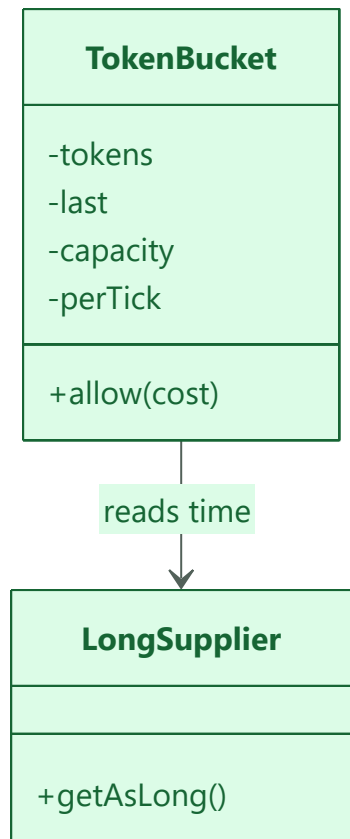
Choice	Use when	Avoid when
Access-ordered map plus one lock	Small local cache needs a clear invariant	High contention, weight-based eviction or advanced expiry require a mature cache

Interview checks: basic: get changes order. Internals: eviction must be atomic with insertion. Debug: unsynchronized reads can corrupt recency assumptions. Scenario: add a loader outside the lock with separate duplicate-load coordination.

4. LLD 2: Token-Bucket Rate Limiter

Requirements/API: positive capacity and finite refill rate; allow(cost) admits atomically if enough tokens exist; refill never exceeds capacity. The injected clock returns monotonically increasing abstract ticks. This is one local bucket, not a globally distributed per-tenant limit or a concurrency semaphore.

Diagram 11.4



Mechanism: read time, reject backwards movement, compute elapsed refill, clamp tokens to capacity, then deduct cost only if sufficient. All steps share one synchronized operation. The double token representation is adequate for this model's simple test values; precision and time-unit choices need explicit consideration for long-running fractional production budgets.

Concurrency/failure: separate check and decrement would let two callers spend the same token. A monotonic time source prevents wall-clock adjustment from minting arbitrary capacity. Per-key buckets additionally need bounded registry size and idle eviction. IntegrationHub must combine provider and tenant limits with the global capacity policy in 10; a bucket per pod cannot enforce a fleet-wide limit by itself.

Tests: testRateLimiter exhausts the initial burst, verifies exact test-clock refill, verifies the capacity clamp after a long advance and rejects costs larger than capacity. No sleeps or throughput measurements occur.

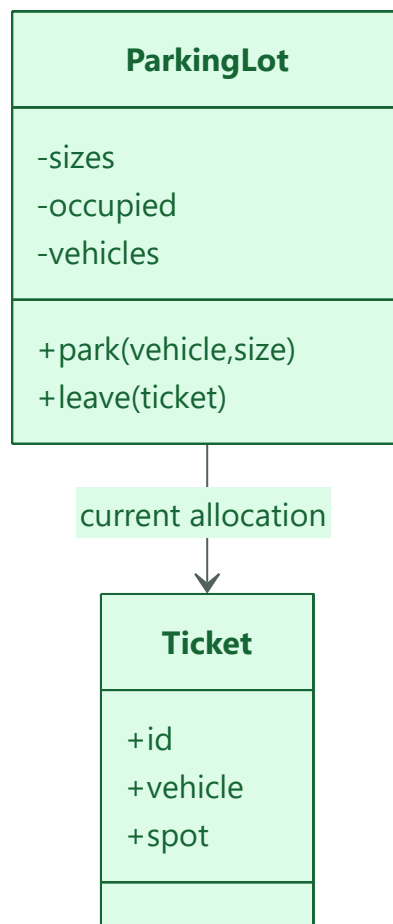
Choice	Use when	Avoid when
Local synchronized bucket	One process owns the quota or receives an allocated share	Autoscaling independently multiplies a claimed global limit

Interview checks: basic: burst and sustained rate differ. Internals: refill/check/spend form one operation. Debug: inspect clock units and replica scope. Scenario: use shared atomic authority or explicit regional allocations for a global contract.

5. LLD 3: Parking Lot

Requirements/API: fixed spots have SMALL or LARGE capacity; park(vehicle,size) returns a ticket or no space; leave(ticket) releases only the matching current ticket. Pricing, reservations and multiple entry gates over a database are excluded from this local allocator.

Diagram 11.5



Mechanism: scan for the first free compatible spot, allocate a monotonically numbered ticket and update both spot and vehicle indexes atomically. Large vehicles cannot use small spots. Leave compares the full current ticket before removing both indexes, preventing an old ticket from freeing a spot now owned by someone else. The first-fit policy is deliberately simple and can waste large spaces if small vehicles reach them first; an allocation Strategy could improve that policy.

Concurrency/failure: one monitor covers duplicate-vehicle detection, selection and assignment. In production, concurrent gates require a shared transactional uniqueness/reservation rule and ticket identity durable across restarts. The local sequence is not globally unique and physical occupancy may disagree with software after sensor failures.

Tests: testParking checks size compatibility, capacity exhaustion, duplicate vehicle rejection, idempotent failure of repeated release and reuse of released space. IntegrationHub's analogous invariant is allocating scarce worker slots without two jobs owning the same lease.

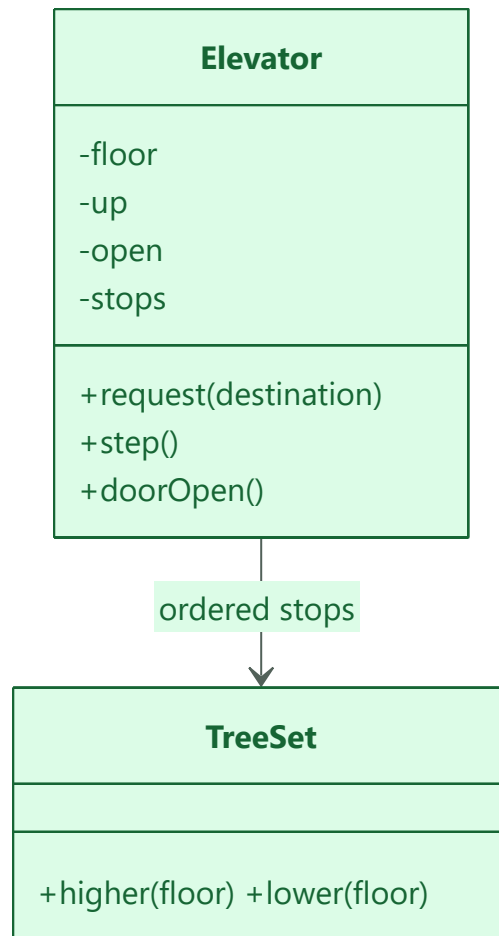
Choice	Use when	Avoid when
First compatible spot	Small bounded allocator favors clarity	Fragmentation or preferences justify a smarter allocation policy

Interview checks: basic: ticket is allocation identity. Internals: two indexes change together. Debug: stale release must compare current ownership. Scenario: moving from one gate to many requires shared authority, not more local locks.

6. LLD 4: Elevator Simulation

Requirements/API: one simulated car, integer floors from ground to a configured top, internal destination requests and one deterministic step at a time. Doors must close before movement. Pending floors are unique. This is not a physical controller, safety system, real-time design or multi-car dispatch algorithm.

Diagram 11.6



Mechanism: a step first closes an open door without moving. If the current floor is requested, it opens there. Otherwise choose the next stop in the current direction, reverse when none remains in that direction and move one floor. Arrival at a requested floor opens the door. The ordered set supports nearest-stop selection without scanning all possible floors.

Concurrency/failure: request and step are serialized under one monitor. The model does not guarantee fairness under an infinite stream of new requests in one direction; an age/deadline policy would be an extension. A multi-car version separates assignment strategy from each car's state machine. Hardware interlocks, emergency behavior and sensors are out of scope and must never be inferred from this simulation.

Tests: testElevator checks stops in ascending order, no movement while closing a door, reversal toward a lower pending stop and invalid floor rejection. IntegrationHub uses the same separation between job scheduling policy and worker lifecycle, not the physical-domain details.

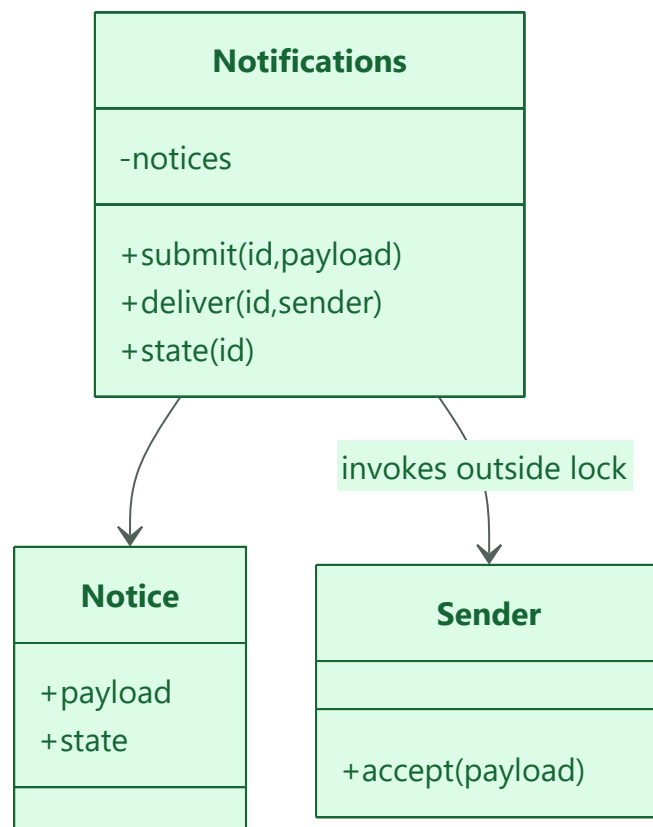
Choice	Use when	Avoid when
Ordered-stop simulation	Explain policy versus state transitions	Real elevators or hard-real-time safety require certified domain engineering

Interview checks: basic: door state constrains movement. Internals: ordering policy and transition validity differ. Debug: a movement tick must not skip closing. Scenario: add a scheduler above several cars rather than entangling their state.

7. LLD 5: Notification Service

Requirements/API: `submit(id,payload)` coalesces identical submissions and rejects conflicting content; `deliver(id,sender)` claims one queued notice, sends it and records local success. States are QUEUED, SENDING and SENT. This local example has no durable outbox or actual email/SMS integration.

Diagram 11.7



Mechanism: submission uses put-if-absent under a monitor. Delivery marks SENDING while holding the lock, releases the lock for the injected callback, then records SENT or QUEUED in finally. A second delivery cannot claim a SENDING or SENT notice. The payload is immutable

text, so callbacks do not mutate internal state.

Concurrency/failure: callback isolation prevents a slow provider from holding the global monitor, but a process crash loses this in-memory state. A thrown sender can have an unknown remote outcome: resetting to QUEUED is safe only under the lab's side-effect-free/failure-before-effect assumption or a provider idempotency contract. The webhook model below makes UNKNOWN explicit. Production notification delivery needs [07's outbox and receipt design](#), retention and provider-specific reconciliation.

Tests: testNotifications checks identical submission, content conflict, failure returning to queued, successful retry and no resend after local completion. IntegrationHub uses this for the conceptual completion-notification workflow, not a claim of exactly-once email.

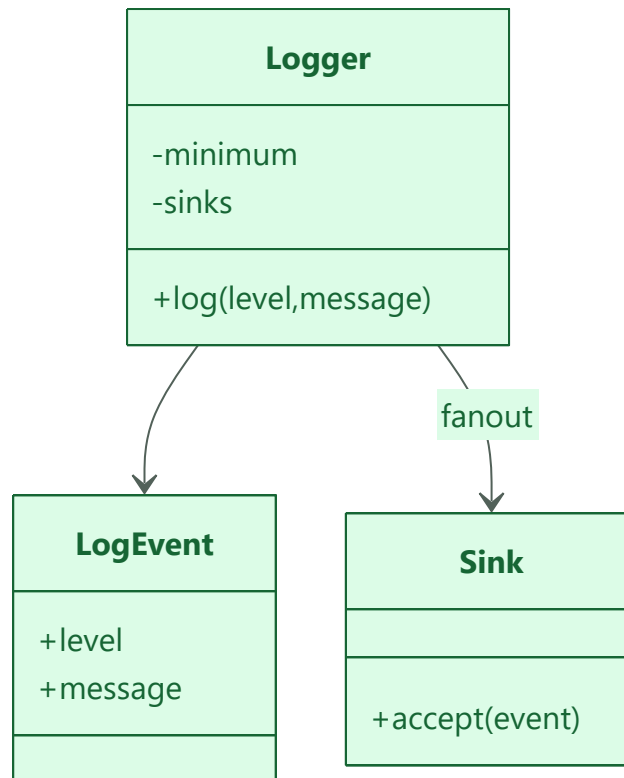
Choice	Use when	Avoid when
In-process sender port	Test ownership and failure transitions	A callback exception is assumed to prove no remote effect

Interview checks: basic: submission identity differs from attempt identity. Internals: claim before releasing the lock. Debug: callback failure can be uncertain. Scenario: store durable intent and use stable provider keys before real delivery.

8. LLD 6: Logger Framework

Requirements/API: immutable minimum level and sink list; log(level,message) filters lower levels, sends a LogEvent to each sink and reports the number of runtime sink failures. A failing sink must not prevent later sinks from receiving the event. The example is synchronous and intentionally contains no background queue or file rotation.

Diagram 11.8



Mechanism: compare severity, create one immutable event, iterate an immutable snapshot of sinks and count RuntimeExceptions. Do not log a sink failure through the same failing logger recursively. Production systems normally use mature logging facades/backends; this exercise exists to expose filtering, fan-out and failure ownership.

Concurrency/failure: the logger's configuration is immutable, but sink thread safety is a separate contract. Concurrent callers may invoke the same sink at once; the test's ArrayList sink is used only sequentially. An asynchronous logger would need a bounded queue, overflow policy, shutdown/drain rules and a decision about audit events that must not be dropped. Avoid credentials and PII in message construction before formatting or sink selection.

Tests: testLogger checks level suppression and isolation of one failed sink from a succeeding sink. IntegrationHub's diagnostic logging can fail without blocking every job, while its restricted audit trail has a stronger persistence contract from 08.

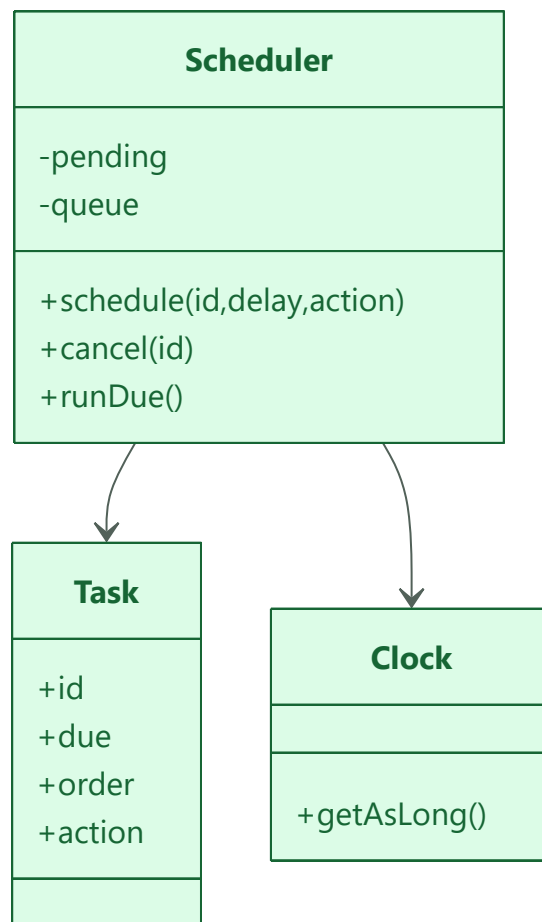
Choice	Use when	Avoid when
Synchronous immutable fan-out	Low-volume deterministic behavior is sufficient	Slow sinks must be decoupled without a defined bounded queue policy

Interview checks: basic: level filtering occurs before sink work. Internals: immutable configuration does not make sinks thread-safe. Debug: recursive error logging can amplify failure. Scenario: separate best-effort diagnostics from mandatory audit evidence.

9. LLD 7: Task Scheduler

Requirements/API: one-shot tasks with unique currently pending IDs, nonnegative delay and deterministic equal-deadline order. schedule registers, cancel removes pending tasks, runDue claims all currently due tasks and executes each once locally. No background thread, cron, durable recovery or preemption is implied.

Diagram 11.9



Mechanism: a priority queue orders by deadline then insertion sequence; a map tracks IDs for cancellation. Claiming removes due tasks from both indexes under one lock. Execution happens outside it, so task callbacks can schedule other tasks without blocking unrelated registration. Newly scheduled due tasks run on the next `runDue` call, not recursively inside the current claimed batch.

Concurrency/failure: cancel succeeds only before claim; after claim it returns false and does not interrupt work. Concurrent runDue callers claim disjoint tasks, but callbacks from separate batches may overlap. This model forgets completed IDs, so reusing an ID later is a new local task. A process crash after claim loses work; a durable scheduler needs leases, fencing and idempotent execution, as 12 explains.

Tests: testScheduler covers no early execution, deadline order, pending cancellation, one-shot behavior and reported callback failure. IntegrationHub uses the concept for retry timing; production retries need durable scheduling rather than an in-memory Runnable.

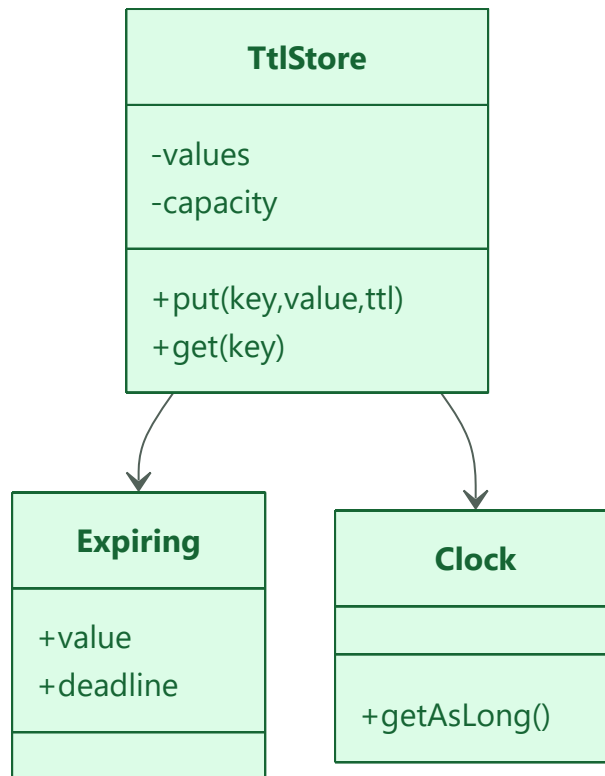
Choice	Use when	Avoid when
Local priority queue	Process-lifetime scheduling is sufficient	Accepted jobs must survive process failure

Interview checks: basic: cancellation has a boundary. Internals: claim and execute are distinct. Debug: inspect both indexes after cancel. Scenario: Runnable is not a durable command description across versions and restarts.

10. LLD 8: TTL Key-Value Store

Requirements/API: fixed positive capacity, positive TTL, injected monotonic clock and optional get. A value is absent at or after its deadline. put replaces an existing key with a new deadline and removes expired entries before checking capacity. No persistence, background expiry thread or LRU eviction is included.

Diagram 11.10



Mechanism: store absolute deadline computed from the injected clock using checked addition. `get` evaluates the current entry and lazily removes it if expired. `put` sweeps expired entries under the same monitor, then either replaces or adds within capacity. This simple sweep is linear in entry count; the bounded teaching store chooses clarity over an expiry heap.

Concurrency/failure: check and removal occur under one lock, so an old expiry observation cannot remove a replacement inserted in between. A scalable timer-based implementation needs entry generations or compare-and-remove: an expiry task created for version 1 must not delete version 2. Clock arithmetic overflow fails explicitly. Values may still be mutable references; the caller owns that contract.

Tests: `testTtl` checks visibility before expiry, equality-boundary expiry, capacity rejection, replacement lifetime and final expiration. `IntegrationHub` might cache short-lived metadata this way, but token revocation and security-sensitive expiry require the correct authoritative policy.

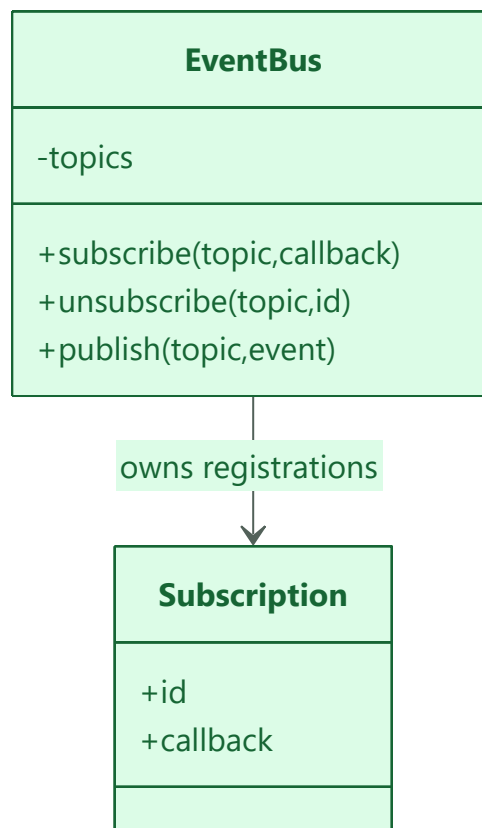
Choice	Use when	Avoid when
Lazy expiry plus bounded sweep	Small store benefits from simple deterministic semantics	Large cardinality requires indexed expiry and predictable write latency

Interview checks: basic: expiry boundary is explicit. Internals: old timers must not delete replacements. Debug: distinguish wall-clock time from elapsed lifetime. Scenario: select a mature cache when expiry/weight/concurrency requirements expand.

11. LLD 9: In-Memory Pub/Sub

Requirements/API: subscribe(topic,callback) returns a subscription ID; unsubscribe removes it; publish takes a snapshot and delivers to each callback in registration order for that publication. Runtime failure of one callback is counted without preventing later callbacks. No persistence, replay or global ordering across concurrent publishers is promised.

Diagram 11.11



Mechanism: a topic owns an insertion-ordered map of callbacks. Publication copies that topic's callbacks under a lock, then invokes them outside the lock. Subscription changes do not mutate the in-progress snapshot. Empty topic maps are removed on unsubscribe to release references.

Concurrency/failure: unsubscribe prevents inclusion in future snapshots but cannot recall a callback already captured by a publication. Concurrent publishers can invoke the same subscriber concurrently; callbacks must be thread-safe or delivery must be serialized by a

separate policy. Slow subscribers block the current synchronous publisher, so an asynchronous extension needs per-subscriber bounds, overflow and cleanup rather than an unbounded executor.

Tests: testPubSub covers topic isolation, fan-out with a failed callback and unsubscribe effects. IntegrationHub can use local notifications for UI-node internals, but durable job delivery belongs to 07. An Observer list is not a broker.

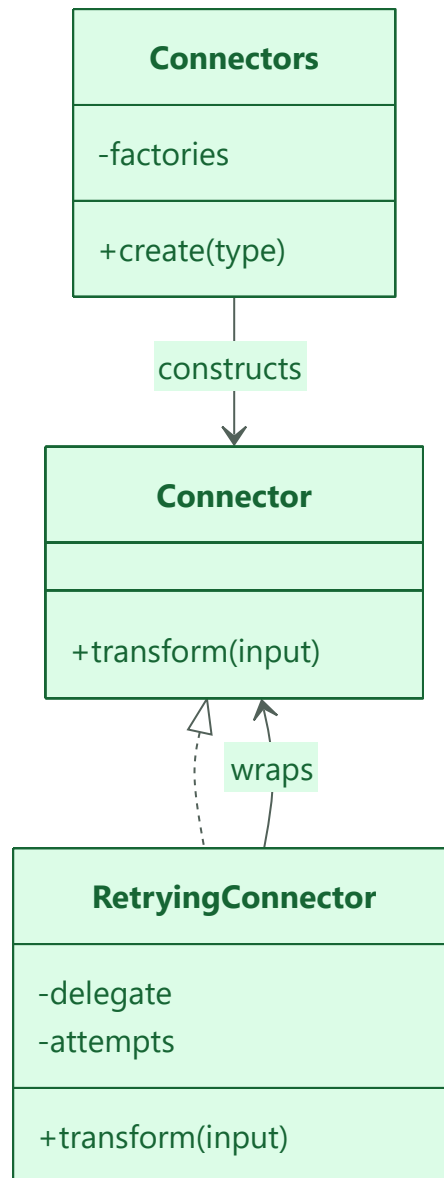
Choice	Use when	Avoid when
Snapshot-based local bus	Transient process-local notification fits	Delivery must survive restarts or disconnected subscribers

Interview checks: basic: publication snapshots define unsubscribe semantics. Internals: callbacks run outside the registry lock. Debug: retention of callbacks can retain whole object graphs. Scenario: add bounded asynchronous delivery only with a clear overflow contract.

12. LLD 10: Pluggable Connector Framework

Requirements/API: Connector exposes transform(input); Connectors selects a factory by registered type and rejects unknown types; RetryingConnector retries only a classified TransientFailure up to a fixed attempt count. The operation is a local idempotent transformation, not a network write. Real backoff, deadlines and circuit policies belong to a maintained resilience library.

Diagram 11.12



Mechanism: an immutable type-to-supplier registry owns construction. Each call can create a fresh strategy. The decorator loops attempts, delegates, returns immediately on success and retries only the explicit transient category. Permanent input errors pass through immediately. An Adapter would translate a real provider's API/errors into this domain contract without leaking vendor types into callers.

Concurrency/failure: registry configuration is immutable; constructed connectors decide their own thread safety. A decorator does not make a mutable delegate safe. Bounded attempts are necessary but not sufficient for network resilience: add elapsed budget, jitter, bulkheads and safe external idempotency through 10. Do not retry arbitrary provider mutations just because an exception says timeout.

Tests: testConnectors selects the upper-case strategy, arranges one transient failure then success, rejects unknown types and asserts a permanent error is attempted once. IntegrationHub's connector layer gains a replaceable algorithm and explicit policy composition, not an excuse for untrusted dynamic class loading.

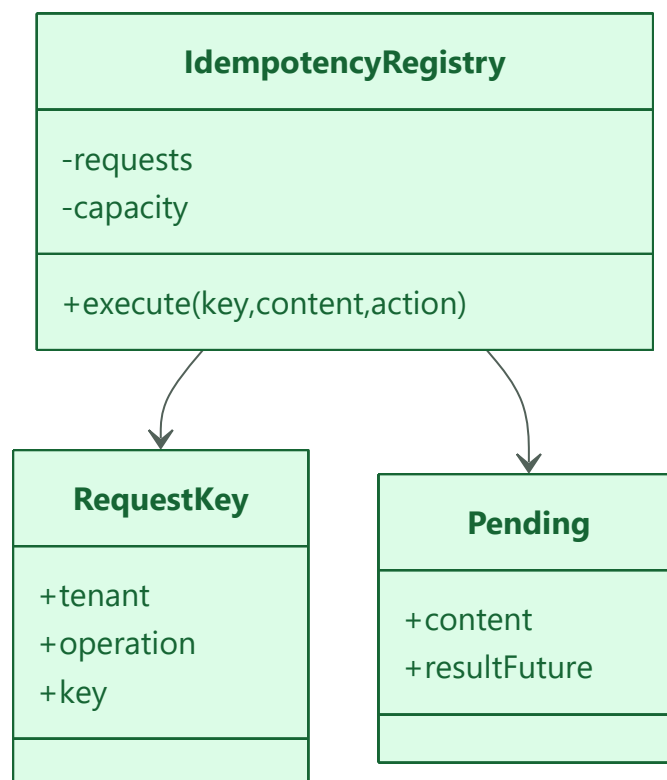
Choice	Use when	Avoid when
Factory plus strategy and decorator	Construction, algorithm and policy vary separately	All concerns collapse into a large type switch in every caller

Interview checks: basic: factory creates, strategy executes. Internals: decorator ordering changes policy. Debug: permanent exceptions should not consume a retry storm. Scenario: adapt provider details at the edge and keep domain commands narrow.

13. LLD 11: Idempotency-Key Registry

Requirements/API: execute(tenant/operation/key,content,action) shares one result among identical concurrent calls and rejects content conflicts. A positive capacity bounds stored requests. Successful results are retained for the process lifetime; there is no TTL or eviction. Failed local computations release the entry so an eligible future attempt can retry.

Diagram 11.13



Mechanism: under a monitor, find or reserve a Pending result. The winner owns execution; duplicates validate content and wait on the same `CompletableFuture`. The owner computes outside the monitor and completes the future. On runtime failure, it completes exceptionally and removes only its own entry. This avoids running a slow callback while holding the registry lock and prevents two simultaneous owners for a successful retained key.

Concurrency/failure: a future can wait forever if the owner never returns, so production needs bounded execution and waiter deadlines without prematurely creating a second owner. Recursive execution for the same key from its own action can self-wait and is outside this contract. Removing a failed entry is safe only for local computations or externally idempotent actions; a remote unknown outcome cannot simply be forgotten. A durable registry must atomically couple accepted business state and identity under 05.

Tests: `testIdempotency` releases two caller tasks through one latch, verifies identical results and one supplier effect, rejects conflicting content, distinguishes tenant scope and rejects new work beyond capacity. Bounded Future waits and executor termination are guards, not performance measurements. No concurrent assertion has run without the JDK.

IntegrationHub mapping: the reservation/result contract explains duplicate job submissions; the real acceptance path moves this identity and the job/outbox effect into PostgreSQL together rather than relying on this process-local registry.

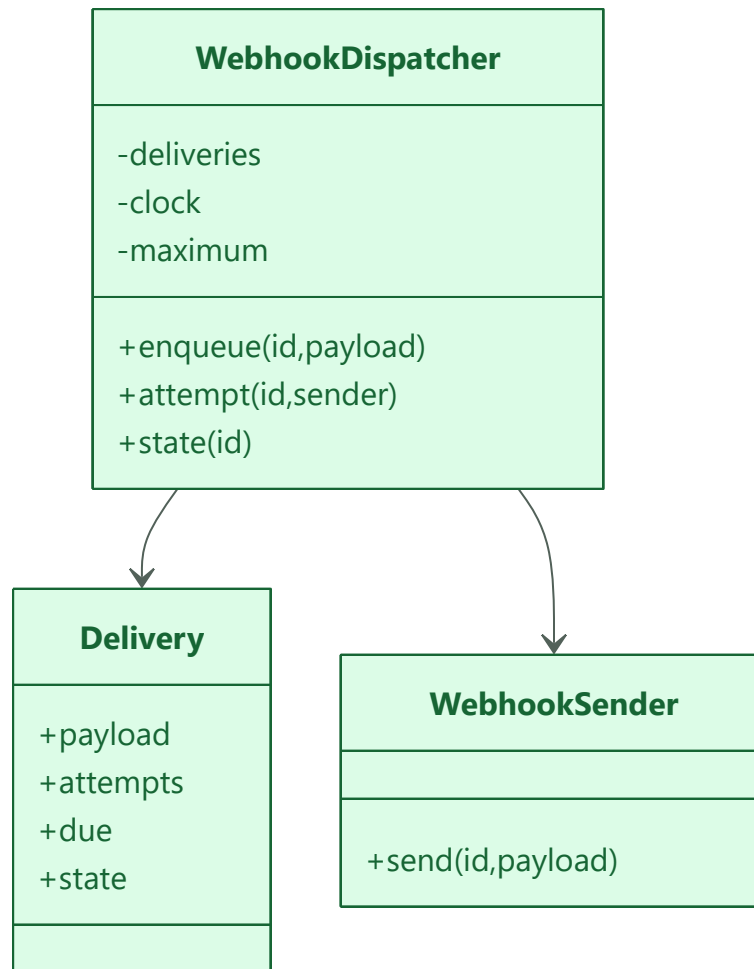
Choice	Use when	Avoid when
In-memory shared future	Process-local duplicate computation needs one owner	Restart-safe HTTP acceptance or unknown external effects require durable authority

Interview checks: basic: key scope is composite. Internals: reservation is the local ownership point. Debug: expired/removed pending entries can create concurrent effects. Scenario: define retention and unknown-outcome recovery before converting it into an API registry.

14. LLD 12: Webhook Dispatcher

Requirements/API: `enqueue(id,payload)` preserves logical identity; attempt claims due PENDING work, calls an injected `WebhookSender` outside the lock and classifies SUCCESS, RETRYABLE, PERMANENT or UNKNOWN. States are PENDING, IN_FLIGHT, SENT, DEAD and REVIEW. Retry attempts are bounded and scheduled using the injected clock.

Diagram 11.14



Mechanism: claim eligible work and increment the attempt, invoke the sender with stable identity, then transition under the lock. Success becomes SENT; a permanent error becomes DEAD; unknown outcome becomes REVIEW; a retryable result returns to PENDING only while budget remains. Due time advances by the attempt number in abstract ticks to keep the model deterministic; production requires its actual bounded backoff/jitter policy, not this toy timing rule.

Concurrency/failure: IN_FLIGHT prevents another local caller claiming the same delivery. A process crash loses all state; production needs durable claims/leases, attempt records and recovery. Signature creation, secret rotation, URL/redirect/DNS validation and HTTP classification belong to a maintained transport adapter and 08's [policy](#). The model intentionally sends no network requests and contains no credentials.

Tests: testWebhooks covers retry not running before due time, later success, no local resend after success, UNKNOWN to REVIEW and attempt exhaustion to DEAD. A real webhook retry often remains safe only because receivers deduplicate the stable event ID; a delivery attempt ID alone cannot provide that contract.

IntegrationHub mapping: job completion callbacks use this classification with durable delivery records, tenant-scoped destination/secret policy and the receiver's duplicate-handling contract. The local model demonstrates transitions, not an operational callback service.

Choice	Use when	Avoid when
Explicit result/state classification	Unknown and permanent failures need different recovery	Every exception is retried indefinitely with a fresh event ID

Interview checks: basic: event identity survives attempts. Internals: claim and callback are separate boundaries. Debug: unknown outcome is not ordinary rejection. Scenario: transport security and durable recovery are required before this becomes a real dispatcher.

15. Concurrency Review and Running the Suite

Across these designs, choose a linearization point for each local atomic operation. Cache mutation/eviction, bucket refill/spend and parking allocation happen within one critical section. Notification, scheduler, pub/sub and webhook callbacks run outside registry locks after claim or snapshot. Immutable configuration does not automatically make injected collaborators thread-safe.

The common failure pattern is check-then-act across an unprotected gap. Another is holding a global lock during unbounded user/provider work. The alternative is not always a lock-free structure: an explicit claim, snapshot or shared future often gives a simpler correct protocol. Revisit interruption, timeouts, overload and shutdown when moving beyond process-local models.

[VERIFY: compile LldLab.java with an existing JDK 21 or later using --release 21 and run all twelve test methods. Source review and publication checks do not establish Java compilation, concurrent runtime behavior or performance; the runner preflight reported NOT EXECUTED.]

The Windows PowerShell command from workspace root is `& ' ./docs/java-fs-guide/code/11-11d/run.ps1 ' ; optional -JdkHome selects an existing installation. It downloads nothing. The runner compiles LldLab.java into out/, runs the main method and records compiler and process outcomes in execution.json when a JDK exists. The current report records NOT EXECUTED.`

Model	Saved implementation	Test method
LRU	LruCache	testLru
Rate limiter	TokenBucket	testRateLimiter

Model	Saved implementation	Test method
Parking lot	ParkingLot	testParking
Elevator	Elevator	testElevator
Notification service	Notifications	testNotifications
Logger framework	Logger	testLogger
Task scheduler	Scheduler	testScheduler
TTL store	TtlStore	testTtl
Pub/sub	EventBus	testPubSub
Connector framework	Connectors, Connector, RetryingConnector	testConnectors
Idempotency registry	IdempotencyRegistry	testIdempotency
Webhook dispatcher	WebhookDispatcher	testWebhooks

[VERIFY: before adapting any model, validate actual capacity bounds, clock units/precision, collaborator thread safety, persistence, shutdown and external side-effect contracts. These are intentionally bounded interview models, not tested production cache, scheduler, identity or delivery libraries.]

[M] MECHANISM

A monitor protects only code participating in its protocol. Returning a mutable object or calling an unsynchronized collaborator can move the race outside the protected section. State the ownership of everything that escapes.

[!] TRAP

A local exactly-once supplier invocation is not a durable exactly-once external effect. Process failure, provider uncertainty, retention expiry and distributed writers change the proof. Reuse the identity/transaction principles from earlier chapters instead of promoting a local test into a distributed guarantee.

[?] INTERVIEW

Make the interviewer choose a boundary with you. Ask whether the scheduler is durable, whether the cache has TTL, whether parking gates share a database and whether webhook timeouts are retryable. A precise small implementation beats a broad interface sketch with undefined behavior.

Explained-Here Index and Sources

Concepts	Explanation
OOP and SOLID	Contracts and ownership
Strategy, Factory, Builder, Singleton, Observer, Decorator, Adapter, Template Method, Proxy, Chain of Responsibility, Command, State	Pattern decisions
LRU cache and rate limiter	LRU and bucket
Parking lot and elevator	Allocation and simulation
Notification service and logger	Notification and logging
Task scheduler and TTL store	Scheduling and expiry
Pub/sub and connector framework	Subscriptions and strategies
Idempotency registry and webhook dispatcher	Ownership and delivery

Library references for verification: [LinkedHashMap Java 21](#), [PriorityQueue Java 21](#) and [CompletableFuture Java 21](#). Saved code is original teaching code and contains no copied library implementation.

Related Chapters

Return to 00. Link [01 Java](#), [02 concurrency](#), [03 proxies](#), [05 API contracts](#), [07 durable messaging](#), [08 security](#), [10 system design](#), [12 distributed cases](#), [19 tests](#) and [25 architecture](#). Generated links are reciprocal.

RELATED CHAPTERS / RECIPROCAL LINKS

[00](#) The System Before the Details

[01](#) Core Java and the JVM

[10](#) System Design Foundations

[25](#) Architecture Styles and Domain Design

[02](#) Concurrency and the Memory Model

[05](#) APIs and Real-Time Communication

[07](#) Messaging, Batch and Streams

[08](#) Security, Identity and Data Protection

03	Spring and Spring Boot Internals	13	Data Integration and iPaaS
12	Twelve System Design Case Studies	24	IntegrationHub in Motion
19	Testing and Quality	20	Appendix: Interview Toolkit and Index
27	Fintech and Payments Engineering		

One-Page Cheat Sheet

Contracts: name valid inputs, state, results, errors, lifetime, bounds and concurrency.

Encapsulation protects invariants; substitution includes failures and mutability. Use composition for independently varying behavior.

Patterns: Strategy executes, Factory creates, Builder validates complex construction, Adapter translates, Decorator wraps, Proxy mediates, Observer fans out, Command packages action, State constrains lifecycle. Singleton is scoped, not cluster-wide. Template Method and Chain need explicit ordering/substitution rules.

Atomic local work: LRU get mutates recency; bucket refill and spend belong together; allocation updates all indexes atomically; TTL removal must not delete a replacement. A bounded map counts entries, not memory bytes.

Callbacks: claim or snapshot under lock, execute outside. Cancel/unsubscribe has a boundary after which work may still run. Immutable registries do not make callbacks thread-safe. Bound queues before adding asynchronous delivery.

Identity: local idempotency reserves one owner and shares a result; webhook IDs survive attempts. Unknown external outcomes need explicit review or provider idempotency. Local state disappears on process failure.

Evidence: all twelve implementations and test methods are saved, but Java compilation/execution remains NOT EXECUTED. They are complete for the stated local contracts, not production systems or a physical elevator controller.

Interview Corner

Basic: When Is an Interface Useful?

When it expresses a stable capability across useful implementations or an ownership boundary. It is not automatically useful merely because a class exists; explain the extension or testing need.

Internals: What Is a Linearization Point?

A point at which a concurrent operation can be treated as taking effect atomically in its specified history. In the local models, lock-protected mutation or reservation supplies that point; it does not establish disk durability.

Trace/Debug: Why Does a Synchronized Class Still Race?

Some mutable state may escape, a collaborator may be unsynchronized, or a check and later action may use separate critical sections. Inspect the whole protocol, not the keyword on one method.

Scenario: How Would You Make the Scheduler Durable?

Persist a command description and due time, claim with a recoverable lease/fencing policy, and make effects idempotent. A serialized Runnable or an in-memory queue is not a sufficient recovery contract.

Basic: Strategy Versus State?

Strategy selects an algorithm under a stable interface; State varies behavior with lifecycle and allowed transitions. Either can use classes or simpler representations depending on complexity.

Internals: Why Invoke Callbacks Outside a Registry Lock?

Callbacks can block, reenter or fail. Claiming/snapshotting first preserves ownership while keeping registry operations available. The resulting cancellation and concurrent-callback semantics must be documented.

Trace/Debug: An Expiry Task Deleted a Fresh Value.

The task probably identified only the key, not the version it was created for. Compare current entry identity/generation before removal, or evaluate expiry under the same lock as replacement.

Scenario: Where Does Local Idempotency Stop Helping?

At process loss, multiple independent registries, retention expiry or uncertain external effects. Move identity and business state to the appropriate durable authority and define provider reconciliation rather than claiming a shared future solves distribution.

12 / Twelve System Design Case Studies

The same technology can solve different problems under different guarantees. A cache used to speed a job list is not an authority for spending money. A queue that distributes jobs is not a broadcast to every browser node. These twelve case studies apply 10's [framework](#) and 11's [local contracts](#) to distributed boundaries. IntegrationHub is fictional throughout.

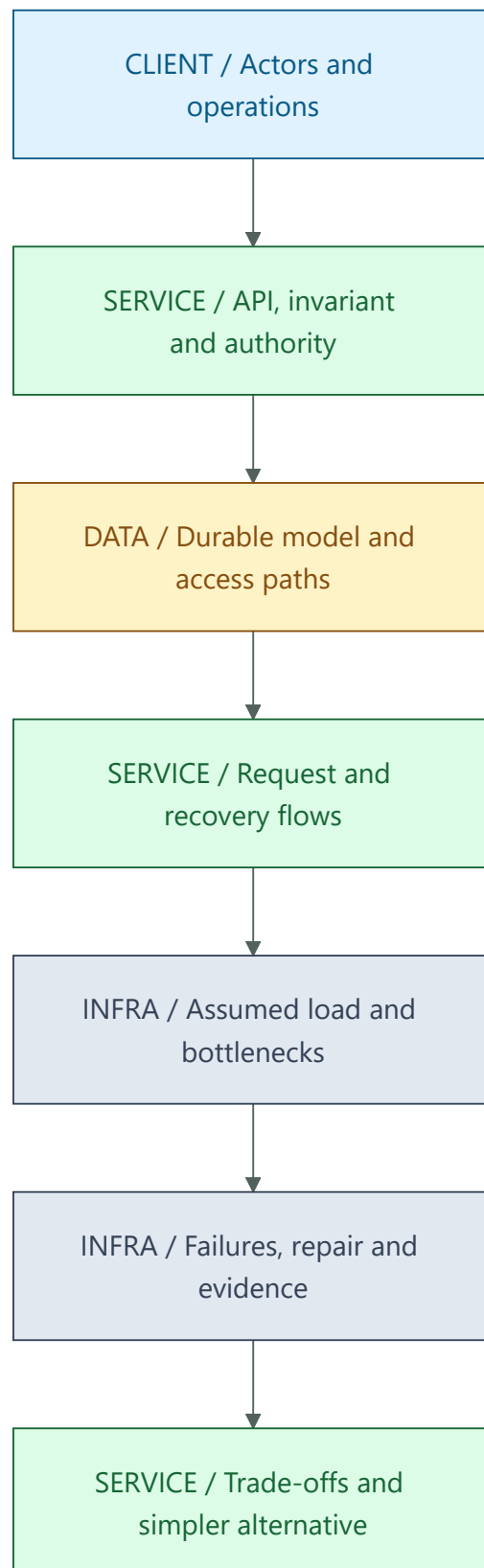
Version assumptions: Java 21, Spring Boot 4.0.x / Cloud 2025.1.x, PostgreSQL 17, Kafka 4.x and Kubernetes 1.34 remain the illustrative baseline. Redis, Elasticsearch, identity providers, payment providers and managed infrastructure are options whose exact versions/configuration must be selected separately. These are designs, not deployed implementations.

Execution evidence: no HLD service, broker, database, provider or load test was executed. Every number below is an explicit hypothetical assumption or arithmetic consequence, not a benchmark. Diagrams and links are publication-tested; that does not validate capacity or availability. Each case has requirements, an estimate, APIs, a data model, architecture, flow/deep dive, trade-offs, failures and an IntegrationHub mapping.

[VERIFY: validate all service/version capabilities, topology limits, data-retention policies, security controls and operational capacity against the selected implementations before building these designs. No HLD deployment, load test, failover rehearsal or provider integration ran.]

Big Picture

Diagram 12.1



What You Will Be Able to Explain

- Design URL shortening, global rate limiting, notifications and token/session services with explicit authority.
- Design bulk sync and webhook pipelines that survive duplicates, cursor failures and unknown provider outcomes.
- Design wallet/ledger operations without confusing a cached balance with authorized spendable state.
- Design distributed caching, gateways, durable scheduling, search and live status with correct recovery boundaries.
- Use estimates to choose bottlenecks and trade-offs while keeping unmeasured capacity claims conditional.
- Trace each design back to one coherent IntegrationHub rather than twelve unrelated technology lists.

[P] PRODUCTION

Where this lives in IntegrationHub. The cases are views of the same system: links point to shared reports, rate limits protect providers, notifications and webhooks report work, identity gates every operation, bulk sync performs it, optional billing motivates wallets, and cache/gateway/scheduler/search/live-status services support the workflow. Start with the simplest subset needed; the teaching catalog is not a mandate to deploy twelve microservices.

1. URL Shortener

Requirements and Estimate

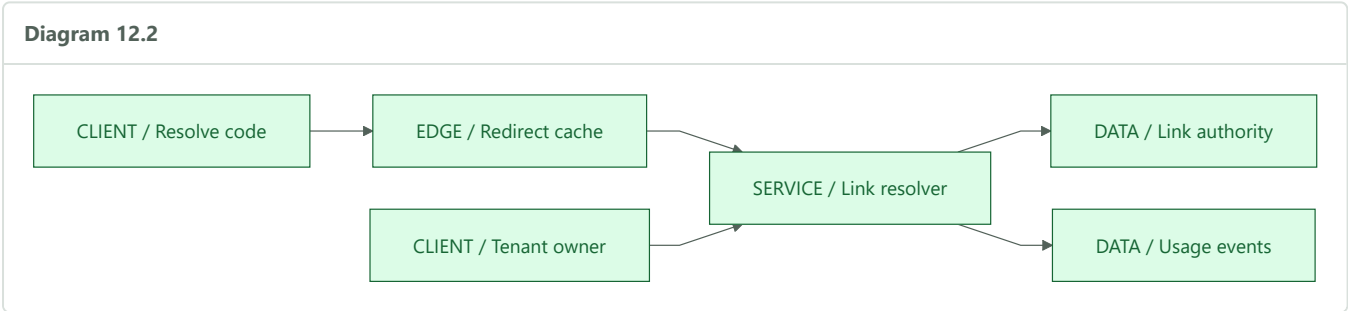
Create tenant-owned short links, resolve a code to a permitted destination, expire or disable links, and optionally record aggregate usage. Public link access and authenticated private report links are different products; choose which is required. A random-looking code is not authorization for private data. The invariant is that one active code resolves to one authoritative link record and disabled/expired links stop being served within the stated cache bound.

Assume 20 creates/s and 20,000 redirects/s, continuously, with 300 bytes of raw link metadata. Creates are 1,728,000/day and raw new metadata is 518,400,000 bytes/day before indexes, replication or analytics. The 1,000:1 read/write ratio suggests caching popular redirects. It says nothing yet about hot-key distribution or abuse volume.

API and Data Model

POST /v1/links accepts destination and optional expiry under a tenant-scoped idempotency key; GET /r/{code} resolves; DELETE /v1/links/{id} disables an authorized link. Return an explicit creation resource. Redirect status/cache policy must match whether destinations can change; do not permanently cache a mutable redirect accidentally.

Link stores code, tenant_id, destination, visibility, expires_at, disabled_at and version, with a unique code constraint. Creation identity is separate from code uniqueness. Usage events are asynchronous records keyed by link and event identity, with privacy-aware retention; do not update one hot counter synchronously on every redirect.



Flow, Deep Dive and Failures

Creation validates allowed URL schemes and destination policy, generates a code, inserts under uniqueness and retries a collision without creating duplicate business requests. Random code size determines collision probability and guessability but must be chosen from expected population/security requirements, not folklore. Resolution checks cache or authority, enforces visibility/expiry and returns the destination. Private links still pass through authentication/authorization.

The main deep dive is revocation versus cache lifetime. A moderation action must invalidate or version cached entries and account for CDN/browser behavior. Short negative caching can protect nonexistent-code traffic, but creation must not remain hidden behind a long negative TTL. Rate-limit creation and abusive resolution, keep analytics off the critical path, and decide what happens when the authority is unavailable: bounded stale public redirects may be acceptable, private/disabled-sensitive decisions may need denial.

Design choice	Use when	Avoid when
Cached public redirect	Read ratio and bounded staleness justify it	Strict immediate revocation is promised without a coherent invalidation path
Authenticated resolution	Link exposes private reports	Code entropy is treated as the only access control

IntegrationHub mapping: links can reference exported report resources; report access stays in 08, not in possession of a short code. **Acceptance check:** disable a hot cached link, exercise expiry and collision retries, and verify analytics failure does not block authorized redirects.

2. Rate-Limiter Service

Requirements and Estimate

Enforce per-tenant, per-operation and provider budgets across gateway/worker replicas, with explicit burst semantics and degraded behavior. Separate rate from active concurrency and long-term quota. The invariant is that concurrent decisions cannot spend the same shared allowance beyond the chosen approximation/allocation contract.

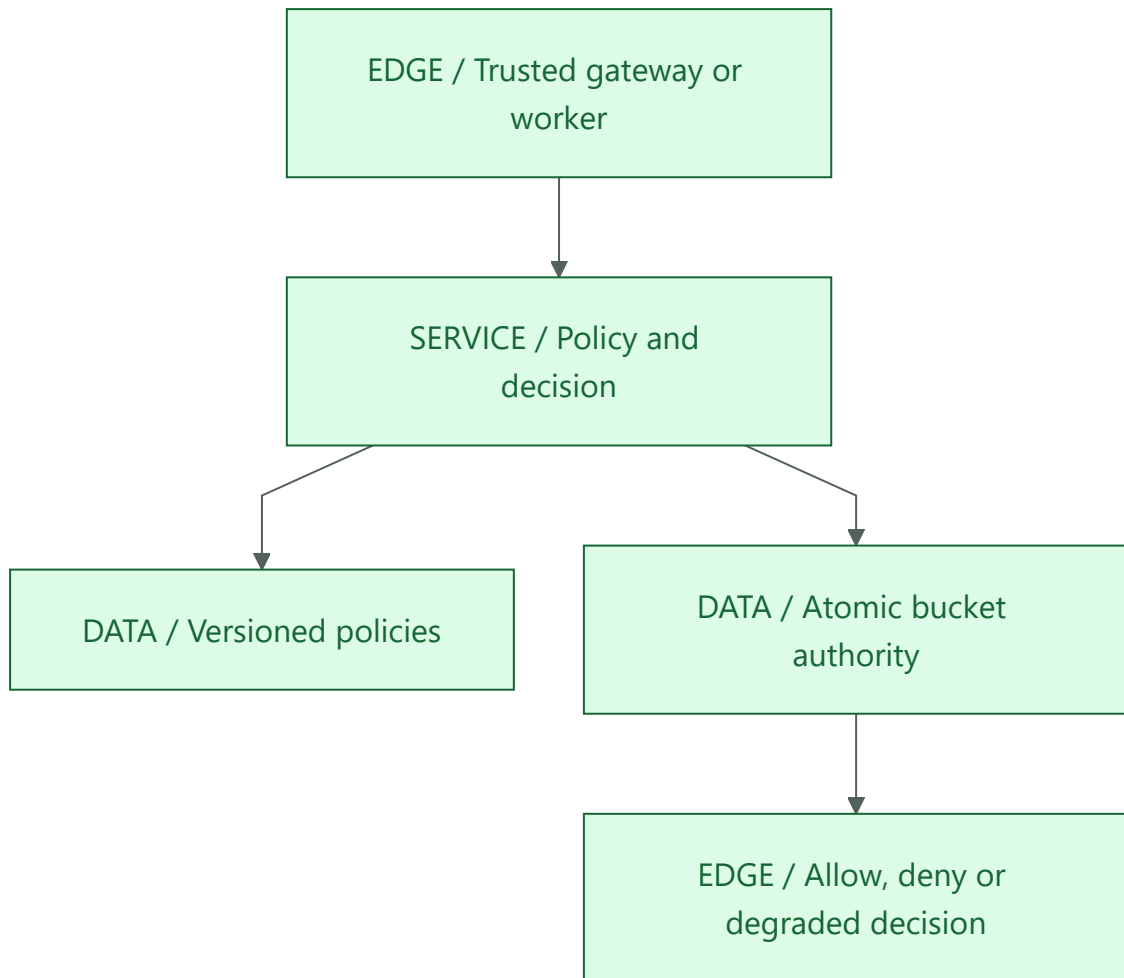
Assume 50,000 checks/s and 100,000 active bucket keys. At an assumed 128 bytes of raw state per bucket, payload state is 12.8 MB before allocator, key, replication and protocol overhead. Network round trips at 50,000/s can matter more than raw bucket memory. A hot global provider bucket may serialize a large share of traffic even when tenant keys shard well.

API and Data Model

An authenticated internal POST /decisions takes a trusted subject, dimension, cost and policy version and returns allow/deny plus retry guidance. The public caller does not get to choose a cheaper policy or forge cost. If decision retries are deduplicated, the gateway-owned decision identity must be bound to one request so clients cannot reuse an allowed identity for unlimited new work.

Bucket state includes scope key, tokens, last_refill, capacity/rate policy version and expiry. Policy configuration has owner, revision and effective time. A separate concurrency lease can hold active capacity with bounded expiry/release; a token bucket alone cannot count in-flight work accurately.

Diagram 12.3



Flow, Deep Dive and Failures

Resolve policy, atomically refill/check/spend at the authoritative bucket and return the decision. Avoid separate GET and SET across a network race. For very high scale, allocate bounded shares to nodes/regions, accepting unused capacity and bounded overshoot according to the allocation design. Policy changes must define how existing tokens are clamped or migrated; a lower limit should not leave an old larger burst indefinitely available.

Limiter outage policy depends on the protected resource. Fail-open might preserve low-risk read availability but can expose expensive provider calls; fail-closed protects budget but couples API availability to the limiter. A bounded local fallback can soften either extreme only if its aggregate allowance is understood. Monitor denied requests, decision latency, authority errors and actual downstream consumption so a broken limiter is distinguishable from legitimate demand.

Design choice	Use when	Avoid when
Shared atomic bucket	Strict global accounting fits coordination cost	Latency or one hot key dominates without a plan
Allocated local shares	Low latency and bounded approximation are acceptable	Independent local buckets claim to enforce one exact global limit

IntegrationHub mapping: tenant API admission and external connector quotas use separate dimensions from 10. **Acceptance check:** concurrent requests at a boundary, policy reduction, node scaling and authority outage must obey the documented allowance/degraded policy.

3. Notification System

Requirements and Estimate

Deliver completion/failure messages through selected channels while respecting tenant preferences, opt-outs and template versions. Accept durable intent before reporting accepted. Do not promise exactly-once email/SMS receipt: provider timeouts and recipient systems are outside the local transaction. Preserve one logical notification identity across attempts.

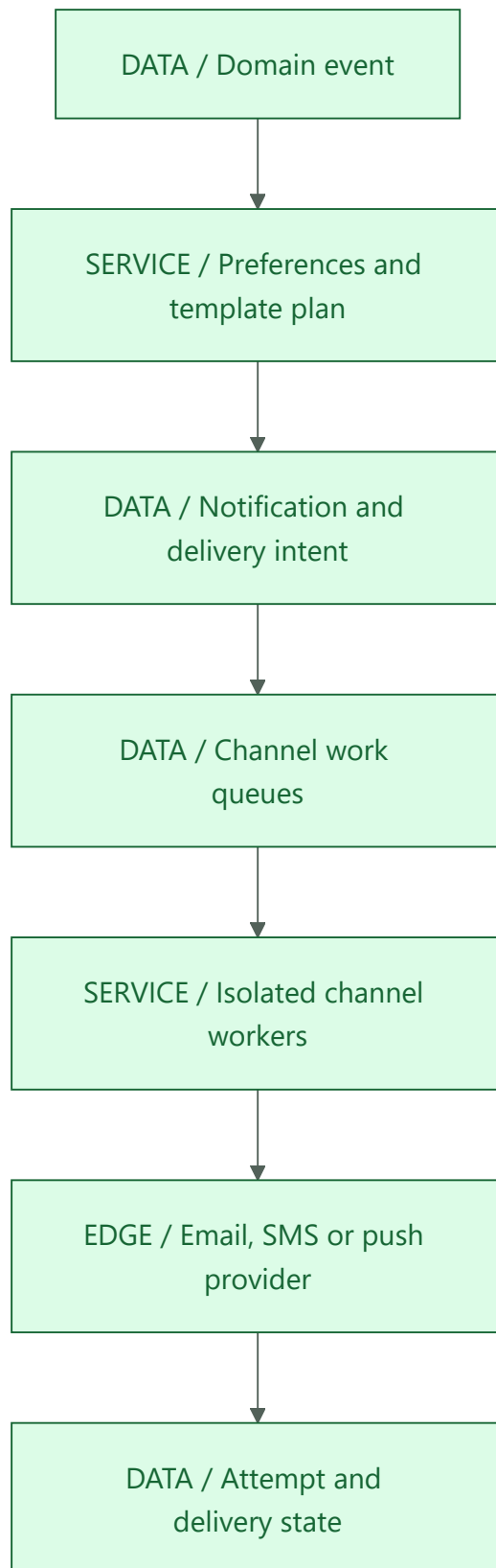
Assume 100,000 triggering events/day and three channel deliveries per event: 300,000 delivery intents/day, about 3.47/s average. A hypothetical 100-times burst is roughly 347/s, so average rate alone is an inadequate worker target. Provider quotas and retry distribution must be measured; this burst multiplier is an assumption, not an observed pattern.

API and Data Model

POST /notifications accepts event/template/recipient references and a logical key; GET /notifications/{id} reports per-channel state. Preference updates are authenticated and versioned. Internal consumption of a JobCompleted event uses a receipt to avoid creating duplicate notification intents.

Notification stores tenant, event ID, template version, recipient reference and content inputs; Delivery stores channel, stable delivery ID, status, attempts and next_due; Attempt records sanitized provider outcome and correlation. Preferences have version and applicability rules. Decide whether preference evaluation happens at creation, send time or both, especially for opt-out changes.

Diagram 12.4



Flow, Deep Dive and Failures

Atomically record the consumed event receipt and notification/channel intents, then dispatch through durable publication. Workers claim due work, resolve permitted recipient/secret data, render the selected template and call the provider with bounded deadlines. Persist outcomes and retry only under policy. A provider receipt may establish accepted delivery, not that a human read it.

The deep dive is duplicate and preference semantics. Stable delivery IDs enable provider deduplication where supported; otherwise reconcile or tolerate bounded duplicate notifications rather than claim impossible certainty. Templates must be versioned so retries do not send different content unexpectedly. Channel bulkheads keep SMS outages from blocking email, and secret/personal-data controls apply to templates, logs and DLQs. Track oldest delivery age and permanent failures, not only successful HTTP calls.

Design choice	Use when	Avoid when
Channel-specific workers	Providers have independent latency/quotas	One shared pool lets a slow channel starve all messages
Versioned templates and inputs	Retries should preserve content meaning	A retry silently changes after an unrelated template deployment

IntegrationHub mapping: this extends 11's notification model with 07's durable intent.
Acceptance check: duplicate domain events, provider success with lost response, opt-out change and template rollout produce the documented per-channel outcomes.

4. OAuth/Token and Session Service

Requirements and Estimate

Provide standard user login/delegation and workload access using a maintained identity platform, not a custom OAuth protocol. Resource servers validate access credentials and independently authorize tenant resources. Session/refresh revocation latency, key rotation and identity-provider outage behavior are explicit requirements.

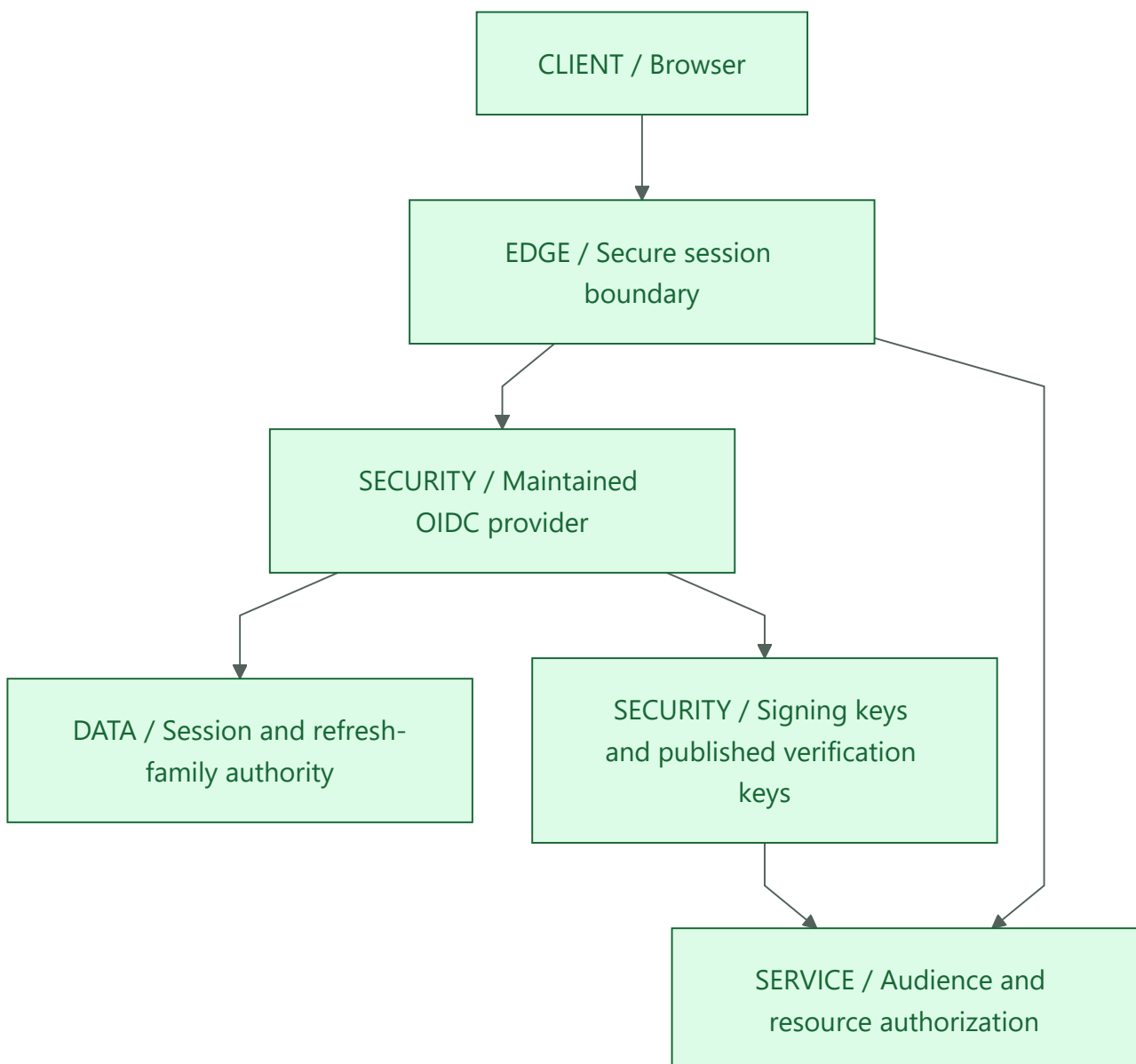
Assume 100,000 continuously active sessions with a 900-second access-token lifetime and uniformly distributed renewal: about 111 renewals/s. This excludes initial logins, retries and synchronized expiry. JWT validation can occur locally at APIs, while refresh remains an authoritative state-changing flow. It is incorrect to size issuance as one token per API request by default.

API and Data Model

Use provider-standard authorization, token, revocation/introspection and discovery/JWKS endpoints as applicable. The BFF exposes login callback, session status and logout under its browser security policy. Do not invent a generic GET token endpoint or accept arbitrary issuer metadata supplied by a caller.

Client registrations bind redirect URIs, client type and allowed flows. Session records bind issuer/subject, tenant context, authentication time and revocation state. Refresh families track hashed/otherwise securely protected handles, replacement/reuse state, expiry and device/session association. Signing keys have active and verification-overlap lifecycle metadata; private keys remain under controlled key management.

Diagram 12.5



Flow, Deep Dive and Failures

Follow 08's [code/PKCE sequence](#): correlate browser flow, redeem code, validate identity response and establish a bounded session. APIs accept the intended access token or trusted session-derived context, not an ID token meant for the client. Refresh rotation consumes an old handle and establishes the replacement atomically under family policy.

Key rotation is the deep dive: publish new verification material before relying on it, issue under the new key, retain old keys only as needed for accepted tokens and handle compromised-key revocation differently from routine rotation. Bound JWKS refresh to avoid a bad key ID causing a request storm. During provider outage, already valid locally verifiable access tokens may continue under policy while new login/refresh fails; do not silently extend expired credentials. A regional failover must preserve refresh ownership or intentionally revoke/re-authenticate, not issue from divergent families.

Design choice	Use when	Avoid when
Managed/maintained IdP	Standard flows and operational controls are required	A hand-rolled token endpoint replaces mature protocol validation
Local JWT validation	Bounded token lifetime and revocation trade-off fit	Immediate revocation is required but no live policy mechanism exists

[VERIFY: confirm identity-provider protocol conformance, refresh rotation/reuse behavior, client registration, key storage and regional recovery semantics for the actual provider. No authorization-server, OIDC or session failover test was executed.]

IntegrationHub mapping: Auth/Token and BFF responsibilities stay separate from connector authorization. **Acceptance check:** wrong audience, simultaneous refresh, rotated/unknown keys, deprovisioning and provider outage must fail or degrade exactly as documented.

5. Bulk Data Sync Pipeline

Requirements and Estimate

Accept a tenant's finite sync request, preserve source identity, map/validate records, write safe target effects, report progress and restart without silently skipping input. Partial success, quarantine, cancellation and source mutation policies must be explicit. An API 202 means durable acceptance, not completed sync.

Assume ten million input records averaging 2,000 bytes: 20 GB raw input. At an assumed sustainable 5,000 records/s, the processing-only lower bound is 2,000 seconds, about 33.3 minutes. Network, provider limits, retries, validation and checkpoints can increase it. This

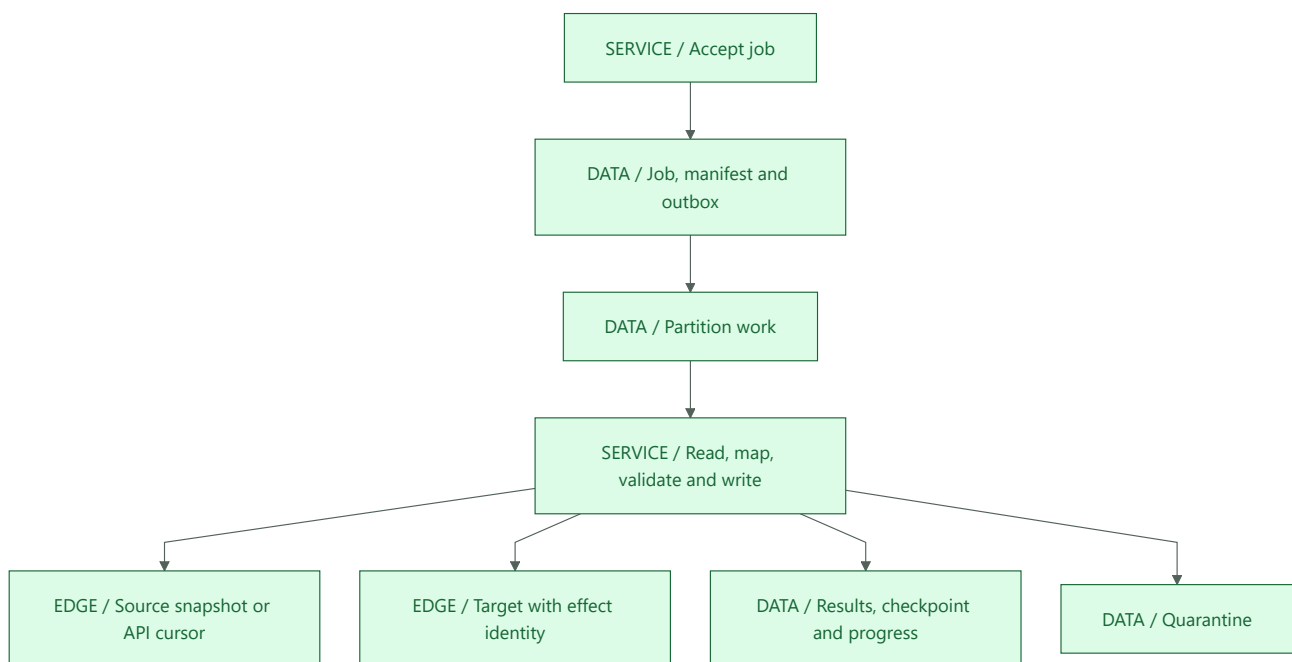
arithmetic does not establish that any connector can sustain that rate.

API and Data Model

POST /sync-jobs uses tenant/operation/key plus content binding; GET /sync-jobs/{id} reports accepted/running/partial/completed/failed; cancellation is an authorized state transition with a documented point after which effects may already exist. The progress stream is a separate optional read path.

Job contains tenant, source snapshot/reference, mapping version, status and version. InputManifest/Partition contains immutable selection and cursor range. ItemResult or a suitable aggregate/checkpoint representation records idempotent effect identity. Checkpoint, processed-event receipts, progress and next outbox intent commit together where they share an authority. Quarantine stores source identity and sanitized failure detail under retention policy.

Diagram 12.6



Flow, Deep Dive and Failures

Resolve immutable input selection, partition it into disjoint work ranges and dispatch through an outbox. Workers obtain permitted secret references, obey tenant/provider budgets, read bounded pages/chunks, apply mapping and write under target idempotency. Commit progress only after the defined output boundary. A resumed reader must interpret the saved cursor against the same input; offset pagination over a changing source is not a reliable restart protocol.

The deep dive is output/checkpoint atomicity. A local transactional sink can commit effect and checkpoint together. An external target needs stable effect keys, result reconciliation or an explicitly accepted duplicate policy. A timeout may leave the target changed even when the checkpoint remains old. Cancellation stops future work cooperatively; it cannot erase already committed target changes. Expose partial outcomes instead of pretending rollback crossed every provider.

Design choice	Use when	Avoid when
Chunked finite batch	Input identity and restart positions are stable	A changing source is traversed with untracked shifting offsets
Per-provider bulkheads	One target's quota/latency should not block others	Worker count grows without respecting target capacity

IntegrationHub mapping: this is the golden-thread connector workflow across 05, 07 and 13.
Acceptance check: restart before/after target effect, expire a source cursor, duplicate a work message and cancel during an in-flight provider call.

6. Webhook Delivery Platform

Requirements and Estimate

Deliver committed events to tenant-registered destinations with authentication, bounded retries, per-destination isolation and replay. Define whether ordering is required per resource, whether receivers deduplicate and how retention limits manual replay. A successful HTTP response means receiver acceptance under its contract, not proof of every downstream effect.

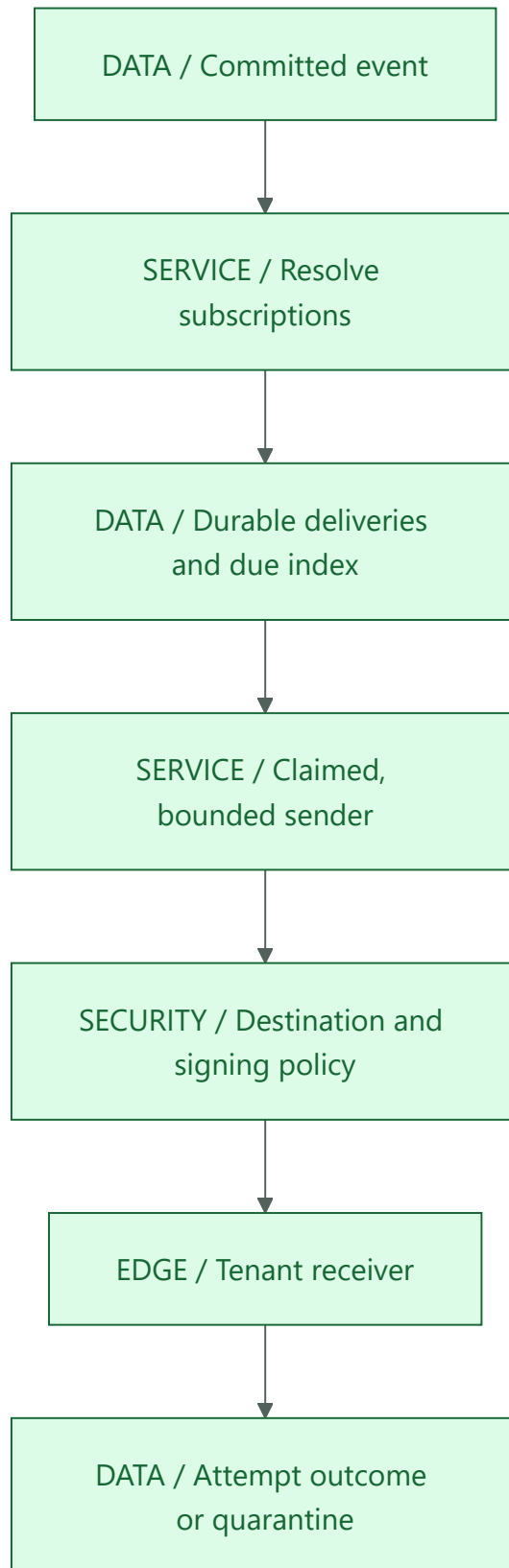
Assume 500,000 source events/day and two destinations per event: one million initial deliveries/day, about 11.57/s average. If every delivery needed three attempts, the upper scenario is three million attempts/day, about 34.72/s before bursts. This is a scenario bound, not an observed retry distribution. Payload and signature overhead determine egress separately.

API and Data Model

Tenant APIs register/rotate/disable endpoints, inspect delivery state and request authorized replay. Internal delivery workers consume event/destination identities. Endpoint changes are versioned; decide whether existing queued work follows the original endpoint version or the new one.

Endpoint stores authorized tenant, destination policy, secret reference and version. Event stores immutable logical ID, aggregate ID/version and safe payload. Delivery is unique by event and subscription identity; Attempt tracks due time, lease owner, status, deadline and provider result. Preserve logical event identity across attempts while assigning attempt IDs for diagnostics.

Diagram 12.7



Flow, Deep Dive and Failures

Plan durable deliveries, claim due items with bounded leases, validate destination at connection time, sign the agreed raw payload/timestamp/identity under the endpoint's key version, send with a deadline and record outcome. Validate redirects and DNS resolution according to egress policy. Registration-time URL checks alone do not prevent a later destination from reaching internal infrastructure.

Unknown outcome is the central trade-off. The receiver may commit then lose the response. Retrying with the same logical event ID allows receiver deduplication; without that contract duplicates remain possible. Ordering can be preserved by serializing a subject's deliveries, but a poison event then blocks later ones. Moving it aside changes the guarantee and needs explicit receiver behavior. Retries use jitter and budgets; hot/slow endpoints get separate limits so they cannot occupy every sender.

Design choice	Use when	Avoid when
Per-destination isolation	Third-party receivers vary in health	One slow tenant stalls all delivery
Signed at-least-once delivery	Receiver can verify origin and deduplicate	A signature is mistaken for exactly-once processing

IntegrationHub mapping: this production design extends 11's [dispatcher](#), 08's [SSRF/secret controls](#) and 07's [replay policy](#). **Acceptance check:** receiver commits then times out, DNS changes, keys rotate, replay is duplicated, and one endpoint remains slow for hours.

7. Payment/Wallet With Idempotency

Requirements and Estimate

Represent money movement as explicit state and balanced ledger entries, reject unauthorized/invalid currency operations and prevent duplicate accepted transfers. Available balance, booked balance and held funds are different values. Never infer spendable funds solely from a cache or asynchronously updated search view. External gateway status can remain unknown and needs reconciliation.

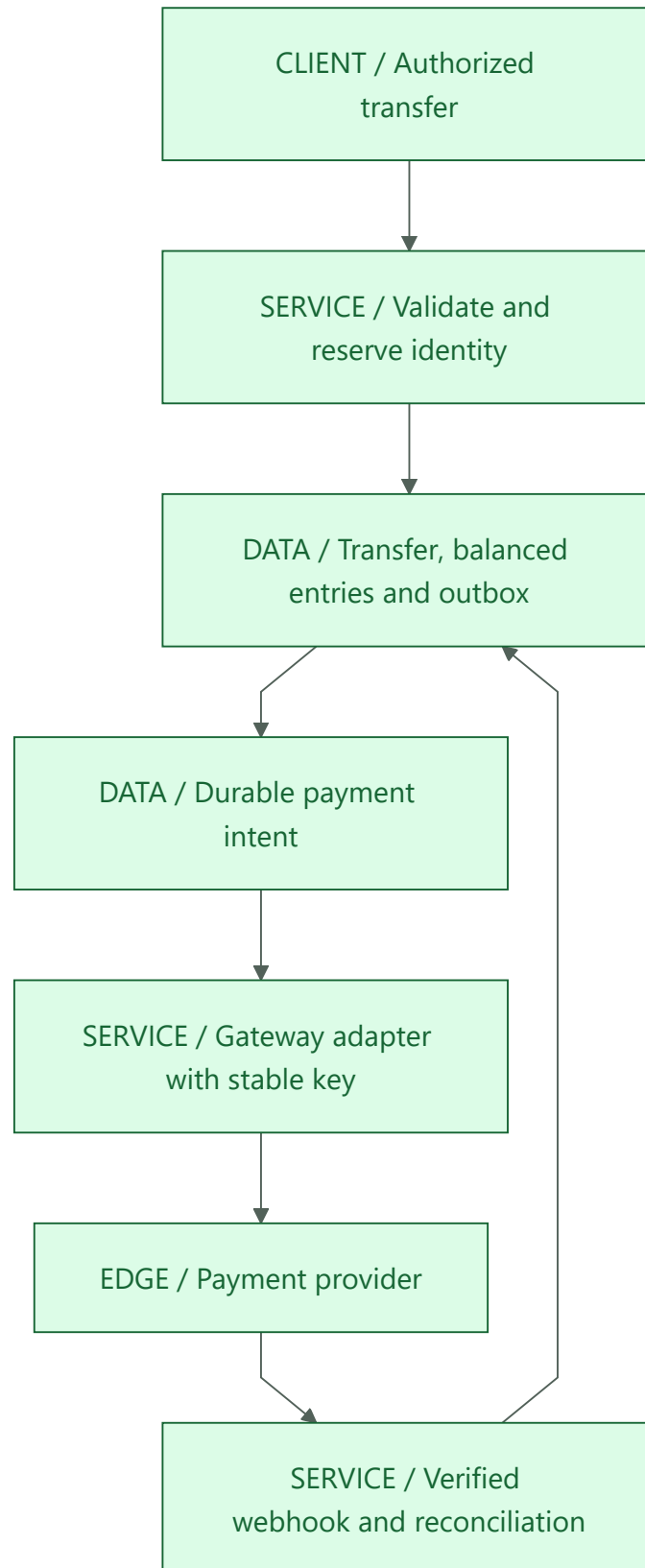
Assume 500 posted transfers/s and exactly two ledger entries per simple internal transfer: 1,000 entries/s or 86,400,000/day. At an assumed 500 raw bytes/entry this is 43.2 GB/day before indexes, replication and audit data. Fees, foreign exchange and multi-leg transactions increase entry count; the two-entry assumption is deliberately limited to one-currency transfers between two accounts.

API and Data Model

POST /transfers accepts source, destination, currency, amount and an idempotency key bound to the authorized request; GET /transfers/{id} exposes lifecycle; holds/releases/refunds are separate authorized operations. Validate positive exact amounts, permitted currency scale and account ownership. A refund is a new recorded movement, not deletion of history.

Account identifies tenant/legal ownership and currency. Transfer carries request digest, state and external reference. JournalTransaction groups immutable Entry rows whose signed amounts balance per currency; posting rules and account constraints enforce the invariant. A materialized balance can be updated in the same transaction or derived with a controlled freshness contract. Unique provider references and processed-event receipts prevent repeated webhook effects.

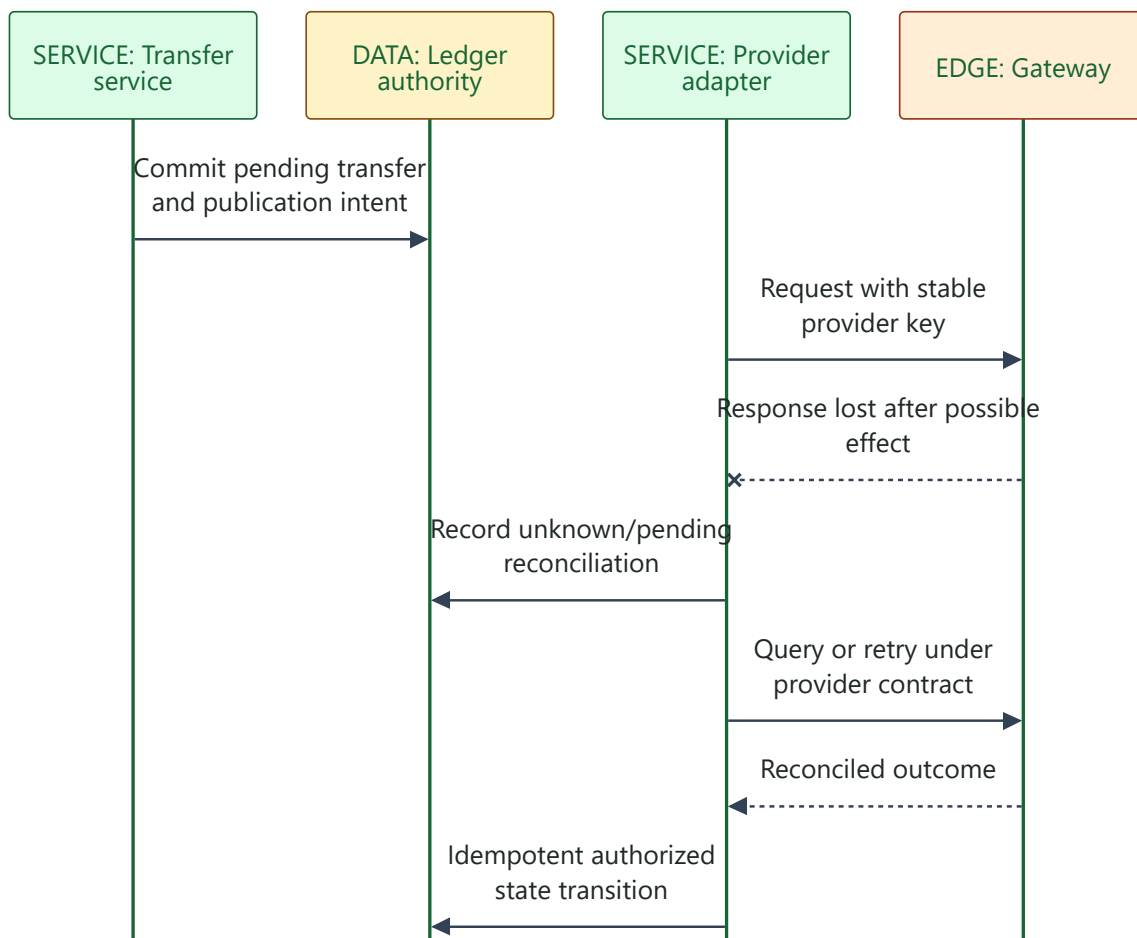
Diagram 12.8



Flow, Deep Dive and Failures

For an internal transfer, acquire the relevant concurrency protection in a stable account order, check authoritative available funds, create balanced entries and commit transfer/idempotency result/outbox together. A stale detached balance check is insufficient. Database uniqueness protects duplicate logical submission; version checks, locks or a serializable transaction protect competing spend under the chosen model.

Diagram 12.9



For external movement, local commit cannot atomically include the provider. Persist intent, call with a stable provider identity and reconcile unknown outcomes through query/webhooks/settlement evidence. Do not release a hold or initiate a second charge merely because the first call timed out. Separate authorization, capture, settlement, refund and dispute states; the detailed lifecycle belongs to [27](#).

Design choice	Use when	Avoid when
Transactional ledger authority	Money invariants require auditable balanced postings	A mutable balance field alone is the only record

Design choice	Use when	Avoid when
Provider idempotency plus reconciliation	External effects cross an unreliable boundary	Local retries are claimed to guarantee provider exactly-once behavior

[VERIFY: monetary posting rules, payment-provider idempotency/retention, authorization/capture/refund behavior, PCI DSS scope and regulatory obligations require current provider documentation and qualified financial/compliance review. This is a design exercise, not a tested or compliant payment system.]

IntegrationHub mapping: an optional billing bounded context uses the same identity/outbox mechanisms but stricter monetary invariants. **Acceptance check:** concurrent spends, duplicate keys with changed amounts, response loss, duplicate/out-of-order provider events and reconciliation mismatch must preserve the ledger and expose unresolved states.

8. Distributed Cache

Requirements and Estimate

Reduce latency/load for derivable data with explicit namespace, tenant, version, TTL and eviction policies. Decide whether cache loss is acceptable and how authority handles a cold-start stampede. The cache is not the source of truth in this case; anything stored only there would change the durability requirement.

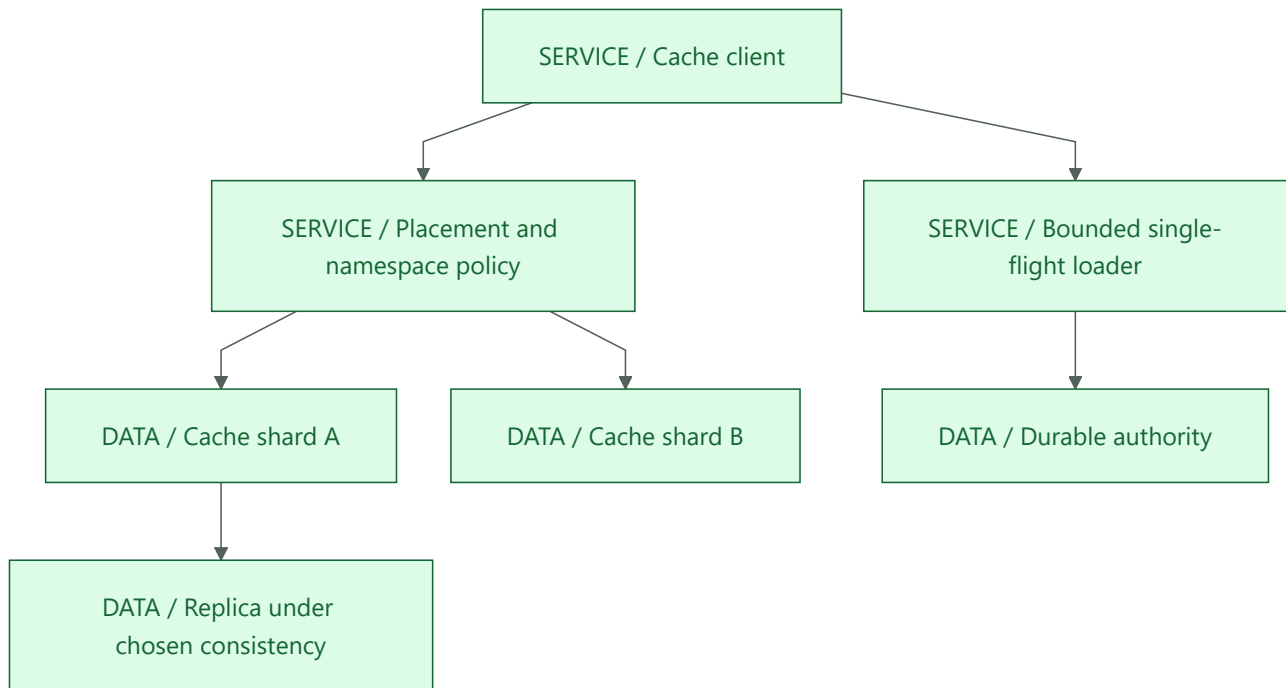
Assume two million active keys with 1,000-byte values: 2 GB raw values, or 4 GB with two copies, before key/object/allocator/replication overhead. This is not a Redis memory estimate. Value distribution, hot-key concentration and miss cost can dominate capacity. A 99% hit ratio at high traffic can still produce an origin overload if all misses target one expensive query.

API and Data Model

Internal GET(namespace,tenant,key), PUT(value,version,ttl), conditional replace and invalidate operations need authenticated namespace access and size limits. Do not expose arbitrary cross-tenant cache administration to application callers. Administrative flushes require separate authorization and an origin-load plan.

Entry includes namespace, tenant, key, value, version, expiry and optional loading lease. Placement metadata maps key hash ranges to nodes; replication policy states whether acknowledgement waits for copies. Eviction weight should approximate actual cost when entry sizes vary. Tombstones/generations may be needed to prevent late loads resurrecting invalidated content.

Diagram 12.10



Flow, Deep Dive and Failures

Read the cache; on miss acquire appropriate single-flight/loading coordination, fetch from authority and conditionally populate a versioned entry. A writer commits at authority and invalidates/updates according to its contract. The stale-load-after-invalidation race from 10 remains unless version/generation policy prevents it or bounded staleness is accepted.

Rebalancing can remap keys and create misses. Consistent hashing reduces expected movement under membership change, but does not solve hot keys, uneven weights or split membership views. Replication can improve cache availability while still serving stale data. On cache outage, shed or bound origin traffic; sending every client directly to the database can turn an optional cache failure into an authoritative database outage. Rebuild incrementally and monitor eviction, misses, load collapse and object sizes.

Design choice	Use when	Avoid when
Disposable replicated cache	Bounded staleness and rebuild fit the workload	It authorizes spend or access without required freshness
Single-flight and load budgets	Miss storms threaten the authority	The loading lock can block forever or one global lock serializes unrelated keys

IntegrationHub mapping: connector metadata and read models can be cached; job acceptance and ledger state stay in durable authorities. **Acceptance check:** node loss, mass expiry, hot key, stale loader and cache flush must not overload or corrupt the source of truth.

9. API Gateway

Requirements and Estimate

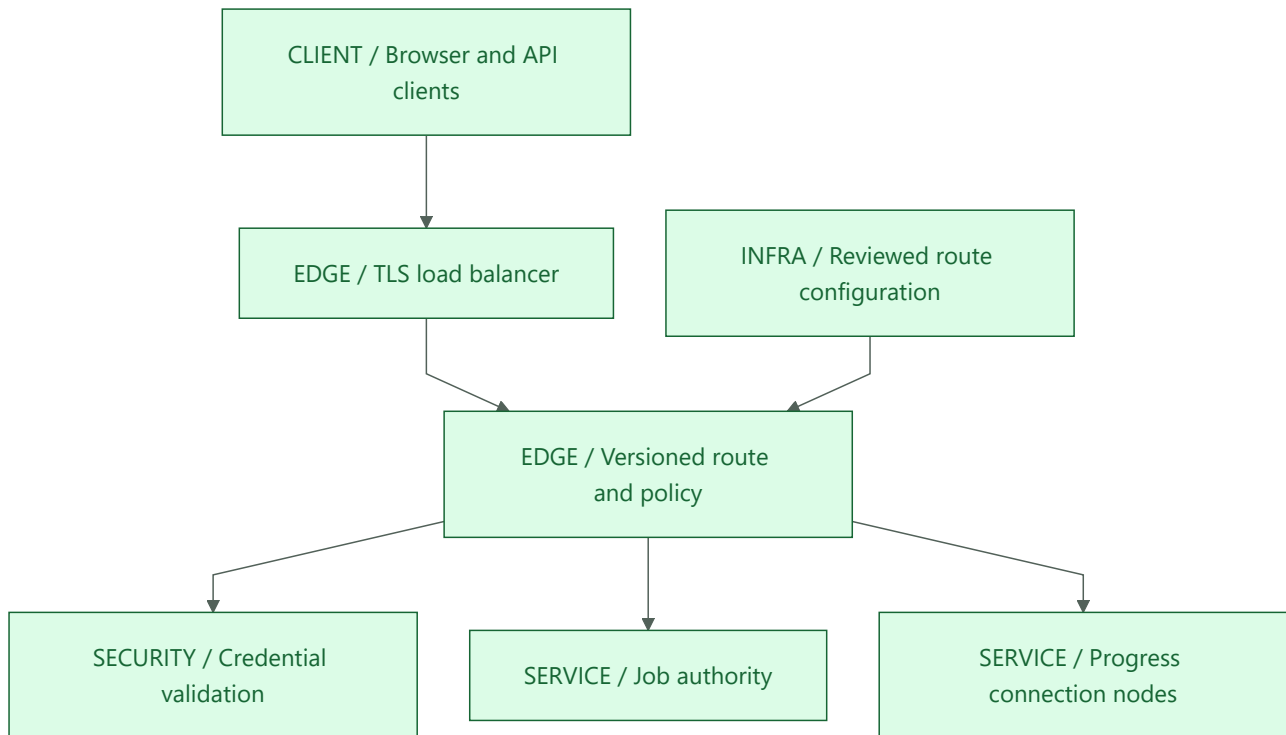
Route external traffic to owned services, enforce coarse authentication/limits, protect request size and propagate correlation. Support normal HTTP and long-lived progress paths with explicit buffering and timeout policy. The gateway must not become the sole owner of business authorization or a hidden distributed transaction coordinator.

Assume 50,000 requests/s and 0.020 seconds mean total residence at the proxy boundary: 1,000 mean in-flight requests under a stable workload. If upstream residence rises, occupancy rises proportionally. Long-lived SSE connections need a separate connection/memory estimate; they cannot be sized using the short-request average.

API and Data Model

Public routes forward versioned service APIs. Administrative configuration APIs are isolated from public traffic and protected by stronger identity. Route metadata includes host/path predicate, upstream pool, timeout/buffering policy, auth audience, size limits and revision. Policy metadata has rollout/version state so nodes do not silently run incompatible routing rules forever.

Diagram 12.11



Flow, Deep Dive and Failures

Validate request limits and trusted forwarding metadata, authenticate under configured issuer/audience, apply coarse quota, choose route and forward within the remaining deadline. Services receive authenticated context through a protected mechanism and still authorize resource operations. Strip or replace spoofable client headers rather than accepting caller-supplied internal identity headers.

Retries are the deep dive. A proxy retry of a mutation can duplicate work after the upstream committed but the response was lost. Retry only under method/operation-specific safety and the service's idempotency contract, within one parent budget. Error normalization should preserve status and safe correlation without exposing upstream internals. SSE needs buffering disabled on the relevant path and compatible idle/keep-alive settings. Health checks should not remove every gateway merely because an optional upstream is failing.

Design choice	Use when	Avoid when
Central coarse policy	Shared edge protections and routing reduce duplication	Domain resource policy is implemented only at the gateway
Immutable reviewed route revisions	Consistent rollout and rollback matter	Ad hoc production edits create untraceable policy drift

IntegrationHub mapping: Spring Cloud Gateway or an infrastructure gateway implements selected edge responsibilities, while 03 and 08 retain service/security ownership. **Acceptance check:** a route rollout, upstream timeout after commit, forged forwarding headers, blocked stream buffering and dependency outage follow documented policy.

10. Durable Job Scheduler

Requirements and Estimate

Schedule one-shot or recurring work durably, prevent conflicting ownership of a fire instance, recover abandoned work and expose missed/late execution policy. Exactly-once physical execution is not promised. A stale worker may run after its lease expires, so effects need fencing or idempotency at their authority.

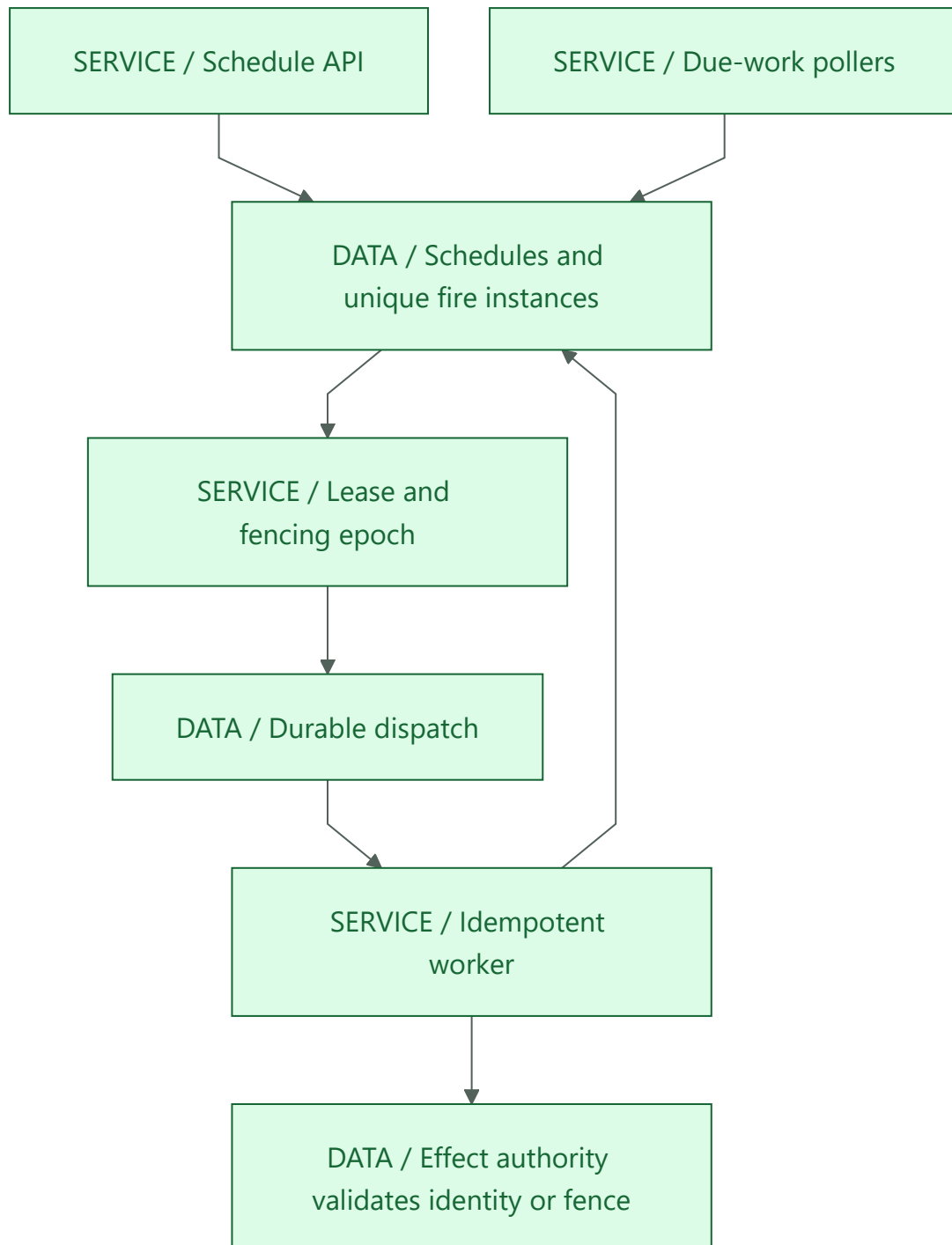
Assume one million scheduled fires/day: about 11.57/s average. A hypothetical 100-times burst is about 1,157/s. If a batch of overdue work accumulates during downtime, live capacity must exceed new arrivals to catch up. Due-time index selectivity and worker/provider budgets matter more than the daily average alone.

API and Data Model

POST /schedules stores command reference, timing rule, tenant and idempotency key; PATCH changes future behavior through a version; DELETE disables future fires; GET /executions/{id} reports one fire attempt/result. Cancellation is cooperative for already running work. Define timezone, daylight-saving, missed-fire and overlapping-run behavior for recurrence.

Schedule stores owner, rule, timezone and version. FireInstance is uniquely identified by schedule/version/logical fire time and stores due time, status, lease owner/deadline and fencing epoch. Attempt stores execution history. Outbox/receipt records coordinate durable dispatch and effect identity. Index status/due/partition so claiming does not scan all history.

Diagram 12.12



Flow, Deep Dive and Failures

Materialize or claim due fire instances transactionally, publish durable dispatch and let workers act with the fire identity. Renew ownership only within a bounded policy and record completion. Recovery reclaims expired leases, but a lease's expiry does not physically stop a

paused old worker. The effect authority must reject a stale fencing epoch where applicable, or deduplicate the same fire identity. External providers that support neither require a different reconciliation/duplicate policy.

Leader election can reduce duplicate scheduling scans but is not itself exactly-once execution. Multiple pollers using correct transactional claims may be simpler. A recurring schedule edit must not reinterpret an already accepted fire as a new identity unexpectedly. During clock skew or long downtime, choose whether to skip, coalesce or execute every missed fire and expose that result to the tenant.

Design choice	Use when	Avoid when
Transactional due claims	One database authority can coordinate scheduling	Workers assume expired leases stop old processes
Leader plus idempotent fires	Coordination reduces duplicate planning	Leadership alone is claimed to make external effects exactly once

IntegrationHub mapping: periodic connector syncs extend 11's local scheduler with 22's fencing. **Acceptance check:** pause an owner beyond its lease, reclaim work, resume the old owner and verify the effect boundary rejects or deduplicates it.

11. Search Service

Requirements and Estimate

Search tenant-owned jobs/connectors with bounded queries, filters, paging and an explicit indexing-lag contract. The search index is derived and must be rebuildable. It must not leak another tenant's documents or become authority for write permissions and job state transitions.

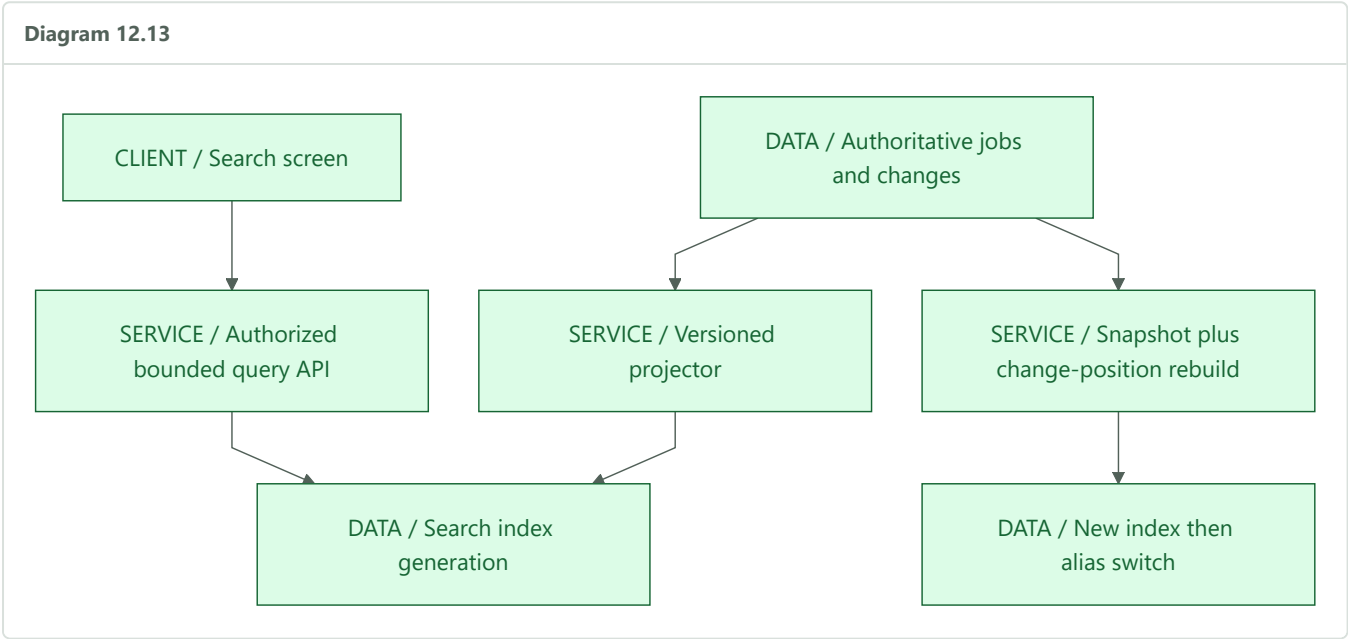
Assume 100 source updates/s: 8,640,000 updates/day. At 2,000 bytes/update, raw update traffic is 17.28 GB/day before transport overhead. Separately assume ten million currently indexed documents at 2,000 raw bytes: 20 GB raw document content. Update traffic and retained index size are different quantities; inverted-index structures, replicas and rebuild overlap require additional measured capacity.

API and Data Model

GET /search accepts bounded text, allowed filters, ordering and cursor. The server injects mandatory authorized tenant/resource filters rather than trusting a client filter expression. GET /jobs/{id} remains the authoritative detail route. Administrative reindex requests use a separate

restricted workflow.

SearchDocument includes tenant, resource ID, source version, selected searchable fields and projection schema version. ProjectionCheckpoint records input position per partition. IndexGeneration records build state, source snapshot position and active alias. Deletions require tombstones/version-aware handling so a late update does not resurrect a removed document.



Flow, Deep Dive and Failures

Consume committed events/CDC, transform only permitted fields and apply version-aware updates/deletes. Commit projection progress according to the sink's retry contract. A repeated event should not duplicate documents; an older version should not overwrite a newer document. Query service validates complexity and always applies tenant access scope, returning freshness where useful.

Reindexing is the deep dive. Build a new generation from a consistent snapshot plus a compatible change-stream position, replay/catch up, validate counts and sampled content, then switch a controlled alias. Retain the prior generation briefly for rollback. Insufficient source retention can make catch-up impossible, so rebuild rate and history availability belong in the plan. A green search cluster with a stuck projector can serve very stale results; monitor lag and deletion propagation, not only query latency.

Design choice	Use when	Avoid when
Derived search index	Relevance and query shapes justify another store	Index is the only copy of business data
Versioned rebuild plus alias	Availability requires online index evolution	Clearing the live index is treated as a safe migration

IntegrationHub mapping: Elasticsearch holds a selected searchable projection while PostgreSQL authorizes writes; [26](#) expands downstream lineage. **Acceptance check:** replay old updates after delete, pause projection, rebuild during writes and attempt cross-tenant queries.

12. Live-Status Service

Requirements and Estimate

Show committed job progress with low perceived latency, support reconnect and tenant authorization, and bound resources for slow clients. Polling remains an authoritative fallback. SSE fits one-way progress; WebSocket is justified only when real bidirectional interaction requires it. State a finite replay window and snapshot fallback when history is missing.

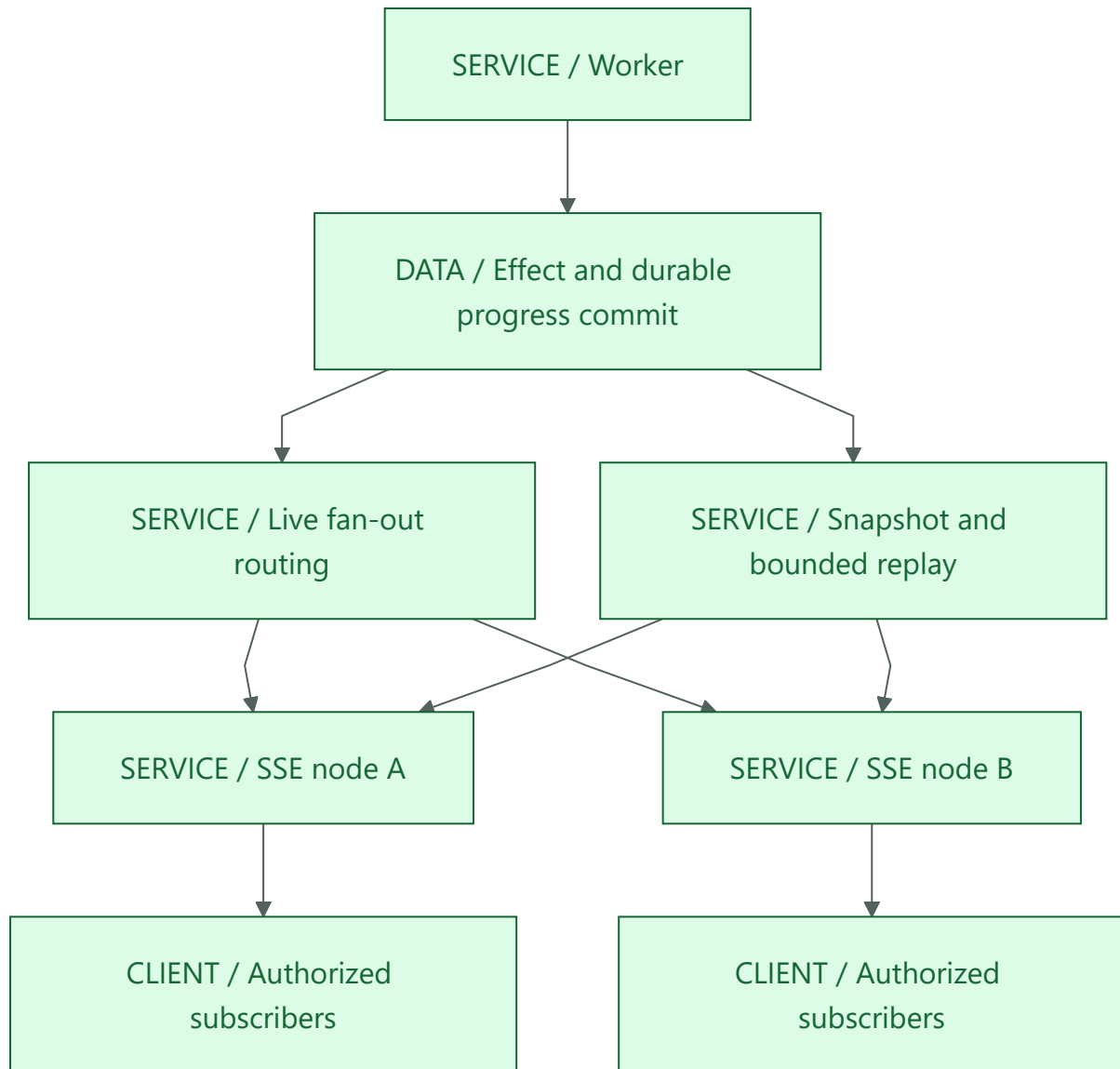
Assume 100,000 concurrent connections, each receiving one 300-byte update every five seconds. That is 20,000 delivered updates/s and 6 MB/s payload egress before framing/TLS/heartbeats. At an assumed 8 KiB of application state per connection, total application connection state is about 819.2 MB before runtime/socket buffers. Actual per-connection memory must be measured; this assumption is not a framework claim.

API and Data Model

GET /jobs/{id} returns a snapshot with version/cursor; GET /jobs/{id}/events opens an authorized stream and resumes from a supported cursor. Native EventSource uses the baseline same-origin session; a fetch-stream alternative can carry headers with its own reconnect/parser implementation. State-changing actions remain normal authorized HTTP commands.

ProgressEvent stores tenant, job, monotonic per-job sequence/version, cumulative state and retention timestamp. Job is current durable authority. ConnectionRegistry is ephemeral per node and maps authorized subscriptions to bounded buffers. Subscription generation prevents old-session updates entering a new browser context. Live fan-out routing metadata is not a substitute for retained progress.

Diagram 12.14



Flow, Deep Dive and Failures

Workers commit effect and progress, then publish live notification through the established durable intent path. Nodes route events to authorized local subscribers. One Kafka group divides partitions among nodes and is not automatically broadcast to every node with interested browsers. Use deliberate routing, separate fan-out semantics or a backplane; Redis Pub/Sub can carry transient notifications only because durable replay recovers losses.

The deep dive is snapshot-to-live continuity. Establish a consistent snapshot/cursor and replay after it, while buffering or otherwise coordinating concurrent live events, deduplicating by version. If a gap or expired cursor cannot be repaired, require a fresh snapshot rather than silently pretending continuity. On a slow subscriber, coalesce disposable cumulative progress

or disconnect and resume; do not let one socket block the producer for everyone. Disable intermediary buffering and align keep-alives with idle timeouts. Reauthorize/close on session changes and terminal completion under policy.

Design choice	Use when	Avoid when
SSE plus polling fallback	Server-to-client progress and recoverable state are enough	Bidirectional interaction is genuinely required
Durable replay plus ephemeral fan-out	Live loss is recoverable from retained authority	Pub/Sub alone is presented as a replayable history

IntegrationHub mapping: this joins 05's transport, 07's progress commits and 09's generation/version guards. **Acceptance check:** node loss, lost notification, stale cursor, snapshot handoff race, slow reader and tenant switch cannot cross authorization or silently regress state.

Cross-Case Review

The repeated structure is intentional: a trusted command reaches an owner; that owner commits an invariant and durable intent; asynchronous effects cross another failure boundary; derived views lag and rebuild; the user gets explicit accepted, completed, rejected or unresolved state. Reuse these mechanisms instead of inventing a new reliability story for every box.

[M] MECHANISM

The crash after commit but before acknowledgement appears everywhere. Link creation, notification planning, transfer posting and scheduled work can all have a real effect with a lost response. Stable identity and the owning authority determine whether retry can replay the result safely.

[!] TRAP

Do not move an invariant into a derived view. A cached balance, search document, browser state or live notification can be stale. If a decision must prevent overspend, tenant exposure or duplicate acceptance, evaluate it at the authority or use a protocol that explicitly proves the required freshness.

[D] DECISION

Deploy the smallest subset that meets the case. A modular service with PostgreSQL and a worker can implement several of these capabilities. Split ownership or stores when isolation, scale, team boundaries or specialized queries justify their recovery and operational cost.

Each case's acceptance checks are proposed tests, not executed results. To build one, select artifacts, write concrete migrations and API contracts, implement the failure paths, execute controlled load/fault tests and update the estimates with measured distributions. A diagram cannot establish a provider guarantee or recovery objective by itself.

[?] INTERVIEW

Deep dive one invariant, not every product. Show the exact transaction, uniqueness/ordering rule, post-commit failure and recovery owner. Then use the estimate to explain which bottleneck changes the design first.

Explained-Here Index

Case	Owning explanation
URL shortener	Identity, redirect cache and revocation
Rate-limiter service	Atomic allowance and degraded policy
Notification system	Per-channel durable intent
Token service	Session, refresh and key ownership
Bulk sync pipeline	Stable input and effect/checkpoint boundary
Webhook delivery	Destination isolation and replay
Payment/wallet	Ledger and unknown provider effects
Distributed cache	Placement, staleness and cold-start protection
Gateway design	Routing and policy boundaries
Job scheduler	Fire identity, lease and fencing
Search service	Projection and online rebuild
Live-status service	Fan-out, replay and slow readers

Related Chapters

Return to 00; use 03 Spring, 05 APIs, 06 database, 07 messaging, 08 security, 09 frontend, 10 foundations, 11 local design, 13 integration, 18 operations, 19 tests, 22 distributed theory, 25 domain boundaries, 26 analytics and 27 payments. Generated navigation provides backlinks.

RELATED CHAPTERS / RECIPROCAL LINKS

00	The System Before the Details	13	Data Integration and iPaaS
06	Databases and Durable State	27	Fintech and Payments Engineering
10	System Design Foundations	03	Spring and Spring Boot Internals
25	Architecture Styles and Domain Design	26	Data Platform Basics
22	Distributed Systems Theory	09	The Browser and the Java Backend
11	Object Design, Patterns and LLD	18	Observability and Reliability
08	Security, Identity and Data Protection	19	Testing and Quality
07	Messaging, Batch and Streams	17	Cloud and Infrastructure
05	APIs and Real-Time Communication	24	IntegrationHub in Motion
		20	Appendix: Interview Toolkit and Index

One-Page Cheat Sheet

Links and limits: unique link identity plus cache revocation; token buckets with atomic global or explicitly allocated scope. Public cacheability is not private authorization.

Notifications and tokens: notification event identity survives per-channel attempts; preference/template versions need a policy. Standard identity protocols use the correct token purpose, refresh authority and key lifecycle.

Sync and webhooks: stable input selection, idempotent target effects and checkpoints make restart possible. Validate destinations at send time, isolate slow receivers, preserve event IDs and reconcile unknown outcomes.

Wallet and cache: balanced immutable ledger postings and authoritative spend checks own money. Cache is derived; protect the origin during cold starts and prevent stale loaders overwriting newer state.

Gateway and scheduler: edge policy is coarse; services authorize resources. Durable fire identities, claims and fencing protect schedules; leadership or DNS routing alone does not prevent stale effects.

Search and live status: versioned projections rebuild from sufficient retained history. SSE nodes need deliberate fan-out and bounded durable replay. A consumer group is not automatic broadcast, and a cursor is not unlimited history.

Evidence: all numbers are assumptions, all deployment/fault checks are unexecuted. Use estimates to choose a bottleneck and failure traces to prove a boundary before selecting products.

Interview Corner

Basic: What Is Common to All Twelve Designs?

An explicit authority, stable identity, bounded resources and a recovery contract. Components vary, but those questions determine whether the system can preserve its invariant after failures.

Internals: Why Are Cache and Search Not Good Write Authorities?

They are usually derived under a lag/rebuild contract. A stale representation cannot safely authorize a stricter current-state invariant unless a separate protocol supplies the necessary consistency.

Trace/Debug: A Request Timed Out but Work Happened.

That is an unknown outcome, not proof of failure. Find the durable identity and authoritative result, then replay/query/reconcile under its contract. Creating a new logical key can duplicate the effect.

Scenario: Which Case Would You Build First for IntegrationHub?

The job acceptance and bulk-sync path with tenant authorization, durable state, bounded workers and status reads. Add specialized search, cache or fan-out only when requirements justify their operational cost.

Basic: Why Include Estimates If They Are Assumptions?

They expose units, dominant costs and decisions to validate. They are useful precisely when labeled: a measured distribution can replace the assumption later without hiding how the architecture depended on it.

Internals: Why Does a Lease Need Fencing or Idempotency?

Expiry transfers permission but cannot stop a paused old process. The effect authority must reject stale ownership or recognize repeated intent; otherwise both old and new workers can act.

Trace/Debug: Broker Lag Is Low but Users See Old Progress.

Work may have moved into an application queue, a projection may be stalled, notifications may be lost, or browsers may be slow. Trace committed progress and age through every stage rather than equating fetch progress with completion.

Scenario: How Do You Make a Rebuild Safe?

Use a new destination, a consistent snapshot/change position, bounded catch-up, validation and a controlled routing/alias switch. Retain a rollback path and verify source history lasts long enough to finish.

Basic: Can a Gateway Enforce All Authorization?

It can validate coarse identity and route policy, but each service owns resource and domain permissions. Internal access paths and changing state still need service-side authorization.

Internals: Why Can One Kafka Group Miss Browser Subscribers?

It distributes partition processing among consumers. Subscribers can be attached to other nodes, so notifications need routing or appropriate fan-out semantics plus durable recovery.

Trace/Debug: Provider Success Was Followed by Duplicate Delivery.

The response or local completion record may have been lost. Use stable logical identity and provider/receiver deduplication where available; otherwise expose and reconcile the uncertainty rather than promising exactly once.

Scenario: How Do You Close an HLD Answer?

State the invariant, bottleneck, most important failure and recovery owner, rejected simpler alternative and what must be measured or verified. A clear unresolved assumption is better than an invented guarantee.

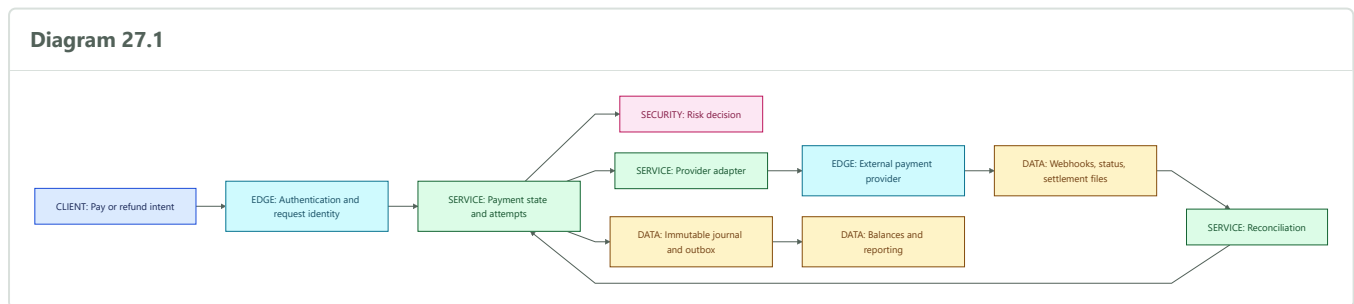
27 / Fintech and Payments Engineering

A timeout is not a financial outcome. Payment engineering combines ordinary distributed-system failures with obligations that cannot be repaired by silently changing a counter. Start with identities, money invariants, evidence and reconciliation; then select providers and deployment topology. IntegrationHub is fictional. The payment examples extend its connector and tenant model for teaching; they do not claim that it operates a real wallet or regulated payment business.

Version assumptions: Java 21, PostgreSQL 17 and Kafka 4.x are reference versions, not executed services. Spring transaction behavior follows 03. Provider APIs, currencies, network rules and regulatory obligations must be checked for the selected product, jurisdiction and date. This is engineering education, not legal, accounting or compliance advice.

Execution status: `code/27-payments/LedgerLab.mjs` passed eight local check groups on Node v24.20.0. It is a single-process, single-currency model. No bank, gateway, payment network, SQL transaction, compliance control or money movement was executed. The case studies and fault exercises below are NOT EXECUTED designs.

Big Picture



What You Will Be Able to Explain

- Distinguish authorization, capture, settlement, refunds and disputes without treating them as one boolean.
- Design a balanced, append-only journal and a concurrency-safe spend decision.
- Bind idempotency to immutable intent and recover unknown provider outcomes.
- Reconcile internal obligations against independent external evidence.
- Place fraud, identity, audit and sensitive-data controls at their actual boundaries.
- Present wallet, gateway integration and ledger designs with estimates, APIs, failure paths and testable invariants.

Where this lives in IntegrationHub. A provider connector uses the same retry and timeout machinery as bulk sync, but a payment timeout must remain unresolved until supported by evidence. Tenant authorization comes from 08, durable state from 06, delivery from 07, and financial analytics from 26. The ledger is authoritative for recorded obligations; a provider is authoritative for its own accepted operations. Reconciliation connects these views rather than pretending they share one commit.

1. Payment Lifecycle: More Than Paid or Failed

In a typical card-oriented flow, **authorization** asks whether a proposed payment may proceed and can reserve spending capacity; **capture** requests collection against an authorization; **clearing** exchanges the information needed to determine obligations; **settlement** discharges those obligations between participants. A merchant's payout timing can be a separate process. A **void** cancels an eligible uncaptured authorization. A **refund** is a new return-of-funds operation after an earlier payment. A **chargeback** is a network dispute mechanism, not merely a merchant-triggered refund. Exact stages, partial operations and deadlines depend on the provider and network. **[VERIFY: confirm authorization, capture, clearing, settlement, void, refund, payout and chargeback semantics, supported partial operations and deadlines against the selected acquirer/provider and current card-network rules. No network flow was executed.]**

Use these as distinct records or state dimensions. A payment can be captured, partly refunded, awaiting settlement and under dispute without fitting a clean linear enum. Maintain an intent, provider attempts, captures, refunds, disputes and settlement allocations as related records. A status summary is a projection of those records, not the only history.

Before sending an operation, persist its business identity and desired action. After receiving evidence, validate its relationship to that operation, transition under a version or lock, record the evidence and publish a durable event. Never let an uncorrelated webhook update a payment merely because the amount looks right.

Operation	Use when	Avoid when
Authorization then capture	The business needs to confirm eligibility before fulfillment and the provider supports the flow	You assume authorization means collected or settled funds
Immediate collection flow	The selected product combines the needed steps under its documented contract	The business needs a separate fulfillment decision but cannot reverse its effects safely
Refund	A confirmed earlier payment needs a tracked return operation	The original operation is still unknown and a refund would guess at its outcome

Operation	Use when	Avoid when
Reconciliation case	Evidence is missing, contradictory or unmatched	A normal timeout is being relabeled failure to simplify the UI

The table is a design choice, not a guarantee that every payment rail supports those operations. Preserve provider-specific meanings in the adapter while exposing a carefully defined internal vocabulary.

2. Money Representation and Posting Invariants

Money is amount plus currency and a rounding policy. Do not use binary floating-point for authoritative arithmetic. Integer minor units are convenient when the selected currency and operation have a defined fixed scale; exact decimal types are appropriate when the domain requires another scale. Java `BigDecimal` supports decimal arithmetic but still needs explicit construction, scale, rounding and comparison policies. An exchange rate, a tax calculation and a posted amount do not necessarily use the same precision.

[VERIFY: confirm current ISO 4217 currency metadata and provider-specific amount, rounding, zero-decimal and special-unit conventions before integrating. The lab uses synthetic EUR minor-unit integers and does not validate a currency registry or provider amount format.]

An amount entering an API must have valid sign, scale and range. A capture or refund request is normally modeled as a positive amount plus an operation type; signed journal lines express debit and credit direction internally. Reject zero, negative, fractional-for-this-contract and overflow values before mutation. Also reject a currency mismatch and changed content under an existing logical request identity.

A **double-entry ledger** records a journal transaction with at least two lines whose debits equal credits. Pick a consistent sign convention. In this chapter's examples, debit is positive and credit is negative. Assets and expenses normally increase with debits; liabilities, equity and income normally increase with credits. Account classifications and recognition events require an agreed accounting model; balanced mathematics alone does not prove the classification is correct.

For a purely illustrative 5,000-minor-unit payment receivable, debit a processor receivable by 5,000 and credit merchant payable by 5,000. On settlement, debit cash and credit processor receivable. A separately identified fee has its own expense, revenue or contractual allocation, not an unexplained difference hidden in the payment amount. These examples do not prescribe when a real business should recognize an asset or obligation.

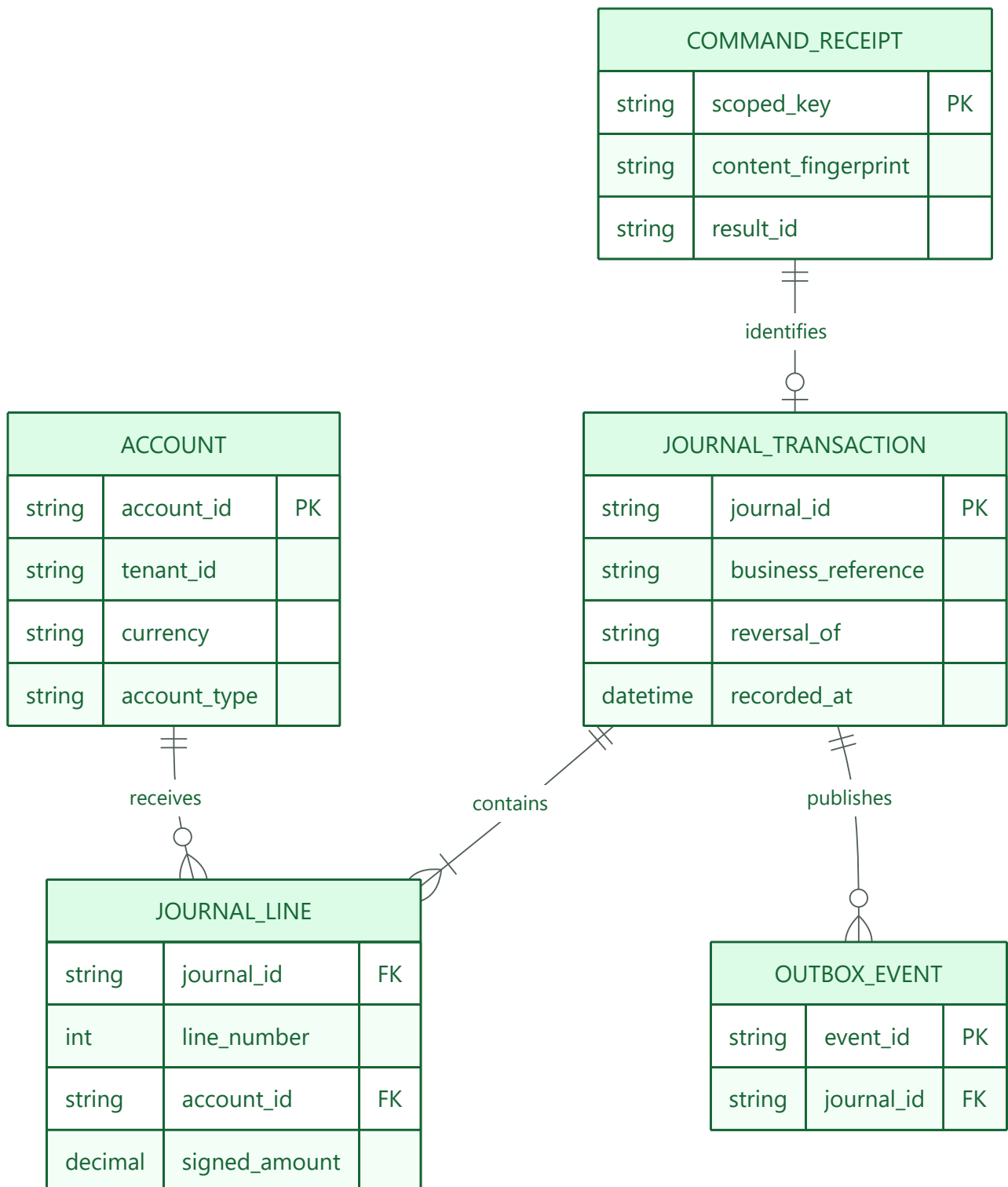
Every journal transaction must balance within each currency. Never sum EUR and INR into zero and call it balanced. An FX conversion involves separately balanced currency legs and explicit exchange, clearing or position accounts under an approved model. Preserve rate identity, rounding and residual treatment. Correct an erroneous posted journal with linked reversing and replacement entries; changing history destroys the evidence needed to explain the balance.

[M] **MECHANISM**

Commit the whole posting boundary. Validate command identity, account eligibility, currency and amount; serialize the relevant balance or version checks; insert journal header and all lines; persist the command result and outbox intent; then commit. A retry sees the stored result. A crash before commit leaves no accepted partial posting. A database constraint on one line cannot by itself enforce the sum of all lines.

3. Ledger Model, Balances and Concurrency

Diagram 27.2



Use stable journal identities and unique business command scope. A receipt may represent an accepted nonposting command, hence the optional journal relationship. In an implemented relational schema, line uniqueness, account tenant and currency consistency, referential

integrity and posting status all need enforcement. Keep draft preparation separate from irrevocably posted entries.

The journal is the audit source for recorded accounting effects. A balance table may be a transactionally maintained projection, or a rebuildable asynchronous projection used only for display. Decide which one authorizes spending. If two transfers both read a stale balance of 100 and spend 80, a perfectly balanced journal can still produce an unauthorized overdraft. Double entry preserves conservation; it does not impose a no-negative-available-balance policy.

For a single database wallet, lock or conditionally update the relevant authoritative balance rows in a deterministic order. In the same transaction, verify spendable funds, reserve or debit, write both sides of the transfer and record idempotency. Retry serialization failures under the same intent identity. If a hold already reduced availability, capturing that hold must consume the reservation without subtracting the same amount twice.

Define displayed balances: posted balance, outstanding holds, pending credits and spendable amount are not synonyms. Whether a pending credit becomes spendable is a product and risk rule. Do not turn delayed analytics or a cache into spend authority. See isolation and locks in 06.

[VERIFY: validate account constraints, posting atomicity, balance serialization, unique receipt scope, outbox commit and failover durability with the chosen database and accounting design. The JavaScript model has no SQL transaction, crash durability or concurrent spend test.]

4. Exactly-Once Effects Need a Named Boundary

An idempotency key identifies one logical operation, not one HTTP attempt. Scope it by tenant or merchant, operation and stable business identity. Bind it to canonical amount, currency, target and intent. Same key and same content replays the original result; same key with different content is a conflict. Preserve the result even if current payment state has since advanced: replaying a first partial refund can correctly return its original accepted response while the payment is now fully refunded.

Exactly-once effects means at most one committed business effect for that identity, plus a retry or recovery policy that eventually resolves valid intent under stated availability assumptions. It does not mean packets arrive once. Inside one database, a unique receipt committed with the effect can enforce the local duplicate boundary. Across a gateway, the provider must recognize a stable operation identity or supply an authoritative lookup and reconciliation process. Kafka's transactional scope does not automatically extend to a bank API.

Record provider idempotency key, merchant reference, request fingerprint and attempt identity before sending. Retrying the same provider operation uses the same identity. Switching providers after an unknown outcome is not a harmless retry: both providers may succeed. Route selection can change before submission, or after definitive evidence the previous path had no effect under its contract; otherwise resolve ambiguity first.

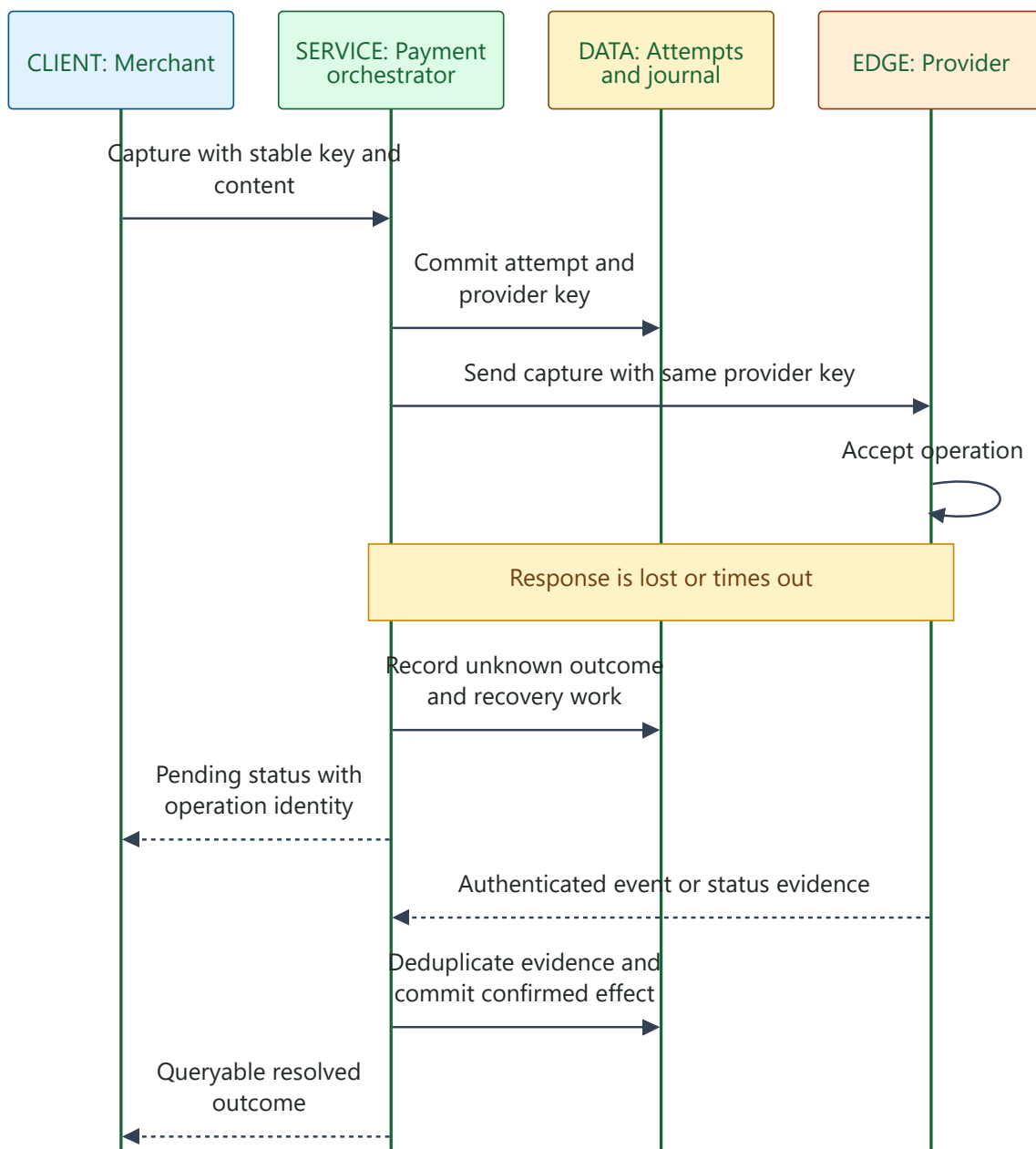
[VERIFY: confirm provider idempotency scope, key retention, content-conflict behavior, concurrent-request handling, status lookup and retry guidance for every integrated operation. No external idempotency or gateway failover was tested.]

[!] TRAP

A new key does not repair an old timeout. It creates a new operation. Retrying with a new key can charge twice, and automatically refunding one side before learning its outcome can compound the inconsistency. Keep the original operation pending and recover evidence.

5. Gateway Timeouts, Webhooks and Recovery

Diagram 27.3



Use bounded connect and read deadlines, an overall operation budget, backoff with jitter and per-provider bulkheads. A circuit breaker can stop new submissions during a failure burst; it does not cancel remote work already accepted. Read-only status recovery may need its own resource budget so submission failures do not starve reconciliation.

Authenticate webhooks using the provider's documented mechanism, preserve the raw bytes needed for verification, enforce replay defenses and validate merchant ownership. Store a unique event receipt before acknowledging durable acceptance. Process asynchronously if

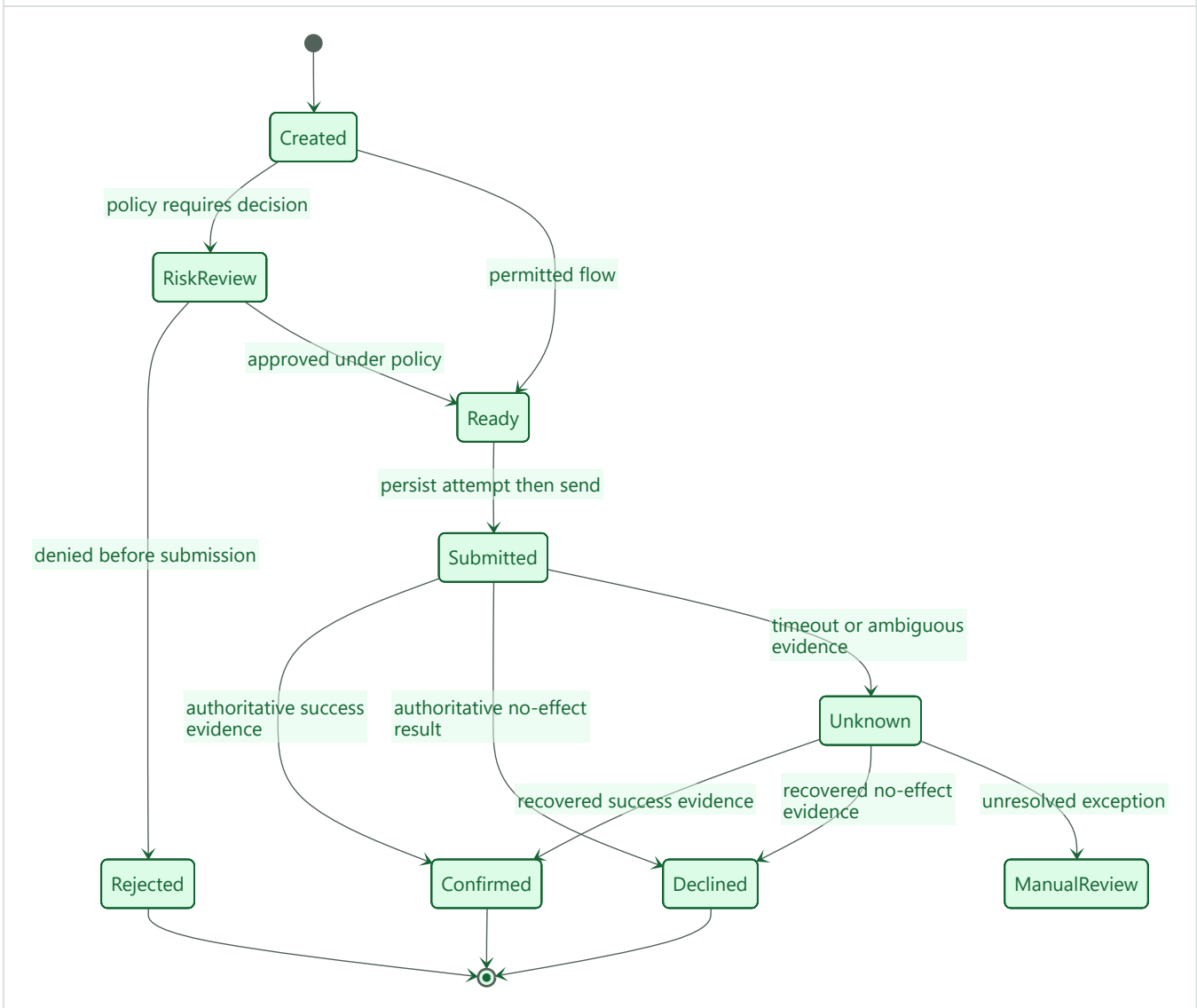
needed; acknowledge only what your contract has actually retained. Events can be duplicated or arrive out of order. Compare provider operation identities and versions where supplied; a late authorization notice must not revert a confirmed capture.

[VERIFY: verify webhook signature/certificate validation, signed payload format, timestamp tolerance, event identifiers, delivery ordering, redelivery and authoritative status semantics in the selected provider documentation. No webhook endpoint or provider signature was executed.]

A “failed to persist webhook” incident is different from “payment failed.” Keep operational delivery status separate from financial state. If webhook evidence conflicts with a query or settlement report, open a reconciliation exception, retain provenance and follow the provider's evidence hierarchy. Do not choose whichever event arrived last by local clock.

6. Money State Machines and Fraud Hooks

Diagram 27.4



This is an **operation-attempt** lifecycle, not a universal card-network state diagram. Separate capture, refund and dispute state machines reference the payment. Every transition needs allowed predecessor states, required evidence, amount constraints, a version check and an audit entry. "Unknown" is local knowledge about an attempt, not a claim that the provider has an unknown state.

Fraud hooks can run before authorization, capture, refund or payout as appropriate to the product. Store decision identity, model or rules version, reasons permitted for retention, expiry and any human approval. A risk approval for one amount and beneficiary must not authorize a later changed request. Automated retries must not bypass a required review. Timeouts in the risk service need an explicit fail-open, fail-closed or hold-for-review policy based on the operation's risk; there is no universally correct default.

Risk response	Use when	Avoid when
Reject or hold on unavailable checks	Unchecked movement exceeds the accepted risk	It is adopted accidentally without considering customer impact or recovery
Permit within bounded policy	A documented low-risk class and limits support it	It bypasses mandatory controls or has no exposure ceiling
Manual review	Evidence is contradictory or a policy requires judgment	An unbounded queue silently becomes permanent pending money

[?] INTERVIEW

State the invariant before the enum. “Refunded amount never exceeds confirmed captured amount” is testable under two concurrent refund requests. “We use a state machine” says nothing about whether the amount check and transition share one commit.

7. Reconciliation: Independent Evidence, Not a Retry Loop

Reconciliation compares internal intent and journal records with independent provider reports, settlement data or bank evidence at a defined cutoff. It answers whether accepted operations, fees, reversals and funds movements match. A status-polling worker can help resolve one timeout, but a full reconciliation also discovers operations you did not know were missing.

Ingest the report under an immutable file or API range identity and retain checksum, provider account, currency, timezone and version. Validate counts and control totals before parsing rows into matching candidates. Match first on stable references; then classify amount, currency, lifecycle or cutoff differences. Never auto-match solely by amount and nearby timestamp when several payments can share both.

Separate matched records, timing differences, missing-internal, missing-external, duplicate evidence and amount/fee discrepancies. Timing differences remain open until the expected window closes. A correction is an authorized command that appends the necessary journal entries with provenance, not a script overwriting a balance until the totals look equal. Maintain exception owner, age, next action and resolution evidence.

[VERIFY: confirm settlement and payout report schemas, fee/netting rules, currencies, business-day cutoffs, report revisions and authoritative matching references with the provider and finance team. No settlement file, bank statement or close process was processed.]

Observe unknown-outcome count and age, unmatched amount by currency, duplicate-event rate, failed posting count and reconciliation backlog. Never add unmatched amounts across currencies without an explicit valuation policy. Alert on unresolved financial exposure as well as API errors. A green HTTP dashboard can coexist with a serious settlement mismatch.

8. PCI Scope, Audit, KYC and AML

Scope reduction starts with data flow. Prefer a provider-hosted collection path and token references where appropriate so raw card data does not traverse ordinary application services, logs and support exports. Tokenization and outsourcing can reduce exposure but do not automatically remove all PCI responsibilities; the collection integration, scripts, systems and contracts still determine scope. Sensitive authentication data and cardholder-data handling have specific requirements. **[VERIFY: establish PCI DSS applicability, current version, assessment scope, hosted-payment integration responsibilities and storage prohibitions with the current PCI SSC documents, acquirer and qualified assessor. No compliance certification or scope determination is claimed.]**

An engineering audit trail records actor or workload identity, tenant, operation, before/after state references, authorization decision, request and provider identifiers, time and correlation. Keep secrets and unnecessary personal data out. Separate append permissions from correction approvals; record attempted denied operations where appropriate. Retention, tamper evidence, access review and recoverability matter more than merely naming a table `audit_log`.

KYC concerns establishing and verifying customer identity under applicable rules. **AML** concerns controls against money laundering, including risk-based monitoring and escalation under the relevant regime. Engineers build evidence collection, restricted access, workflow states, screening integrations and auditability; they do not invent legal thresholds or infer that a successful API response establishes compliance. **[VERIFY: confirm KYC, AML, sanctions-screening, reporting, retention and approval obligations with qualified compliance/legal owners for the actual jurisdiction, entity and product. No regulatory control or screening provider was validated.]**

Keep access boundaries narrow. Support staff may view a case without being permitted to issue a refund. A manual correction can require separate preparation and approval identities under the organization's policy. A stolen service credential must not authorize arbitrary journal entries; permitted command types and account scopes should constrain it. See [08](#) for authentication, data protection and secret rotation.

9. Card Networks and UPI: Conceptual Maps

A conceptual card payment connects the cardholder, merchant, gateway or processor, acquirer, network and issuer. Roles can be combined by a provider, but they represent different responsibilities. Authorization messaging and later clearing, settlement or merchant payout must not be collapsed into one synchronous API call. Authentication mechanisms such as 3-D Secure address a different question from the merchant's own API authorization. **[VERIFY: confirm current network participant roles, issuer/acquirer flows, 3-D Secure responsibilities, dispute processes and regional variations against selected network, EMVCo and provider documentation. This is a conceptual overview, not a certified integration.]**

UPI is an Indian account-to-account payment system operated by NPCI, with participating banks and application/provider roles. Its addressing and authorization flows differ from a card authorization/capture model. Do not assume a UPI API has a separate capture stage because your card adapter does. Internal intent, attempt, pending/confirmed evidence and reconciliation concepts still apply, while operation names and capabilities belong to the rail-specific adapter. **[VERIFY: confirm current UPI participant roles, permitted flows, status/reversal/dispute handling, limits, authentication and integration obligations using NPCI, RBI and the contracted PSP/bank documentation. No UPI integration, limits or regulatory status was tested.]**

An adapter translates a provider's documented states into internal evidence without discarding distinctions. Preserve the raw provider status and reference. A portable abstraction should expose capabilities rather than pretend every rail supports holds, partial capture, refunds and disputes in the same way. Avoid encoding network limits in this guide as timeless constants.

10. HLD Case Study: Wallet

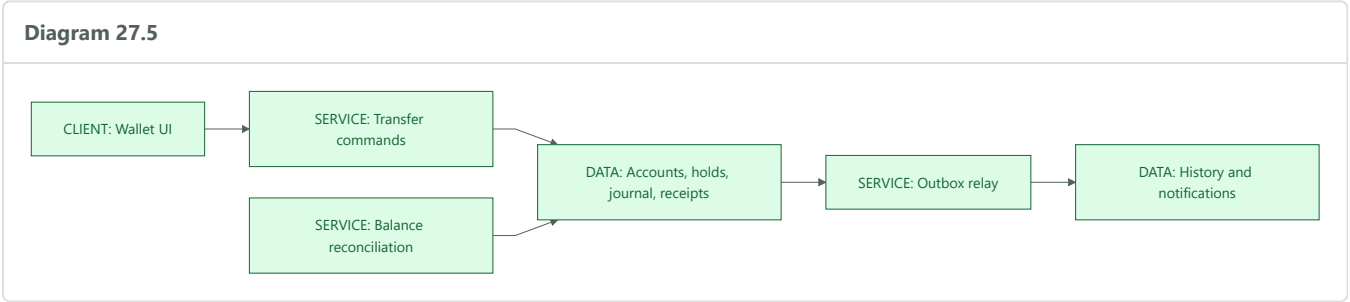
Requirements and Estimate

Design a fictional closed-scope wallet exercise: authenticated tenants can transfer an existing recorded balance between eligible accounts, query history and reserve funds. External funding, legal product classification, FX and real-world custody are outside the implemented exercise. Invariants: no unauthorized cross-tenant transfer, no double posting, no overspend under the configured policy, and every accepted transfer has balanced entries.

Hypothetical load: 100 transfers per second, one journal with two lines per transfer, gives 200 journal lines per second and 17,280,000 lines per 24-hour day at a constant rate. This excludes receipts, indexes, audit, outbox and replication. It is an estimate, not observed throughput. A single hot account can serialize traffic even if overall capacity is adequate.

API and Data Model

Design interfaces: `POST /transfers` with source, destination, currency, amount and idempotency key; `GET /transfers/{id}` ; account balance and history queries with scoped pagination. These are API sketches, NOT EXECUTED endpoints. Store accounts, holds, transfers, command receipts, journal transactions and lines, plus an outbox. Tenant and account authority come from validated identity, not a freely supplied tenant header.



Flow, Deep Dive and Failures

Authorize both account relationships, normalize intent, claim the unique request, acquire account serialization in deterministic order, validate available funds, debit and credit in one transaction, record result and outbox, then commit. On response loss, replay the stored transfer result. For a hold, create reservation identity and expiry policy under the same spend authority; a capture consumes the hold exactly once. Expiry and capture racing must have one winning transition.

Use one transactional boundary initially. Partitioning by account can split a transfer across shards and create a distributed money workflow. Do not introduce it before measuring a real limit and specifying reservations, recovery and reconciliation. A cache can serve a labeled display balance but cannot authorize spending from stale data.

Choice	Use when	Avoid when
One database posting boundary	Transfer volume fits and strong local atomicity is valuable	A measured scale or regulatory partition requirement forbids it
Cross-partition workflow	Scale or ownership requires separate authorities	The design cannot specify pending funds, compensation and recovery

IntegrationHub mapping: the tenant and connector ownership rules remain; wallet state belongs to a separate financial context, not connector-owned counters.

Acceptance check: race two spends whose sum exceeds available funds; exactly the allowed spend commits. Retry each accepted command after response loss; journal count does not grow. Crash before commit; neither side appears. These database/concurrency tests are NOT

EXECUTED.

11. HLD Case Study: Payment Gateway Integration

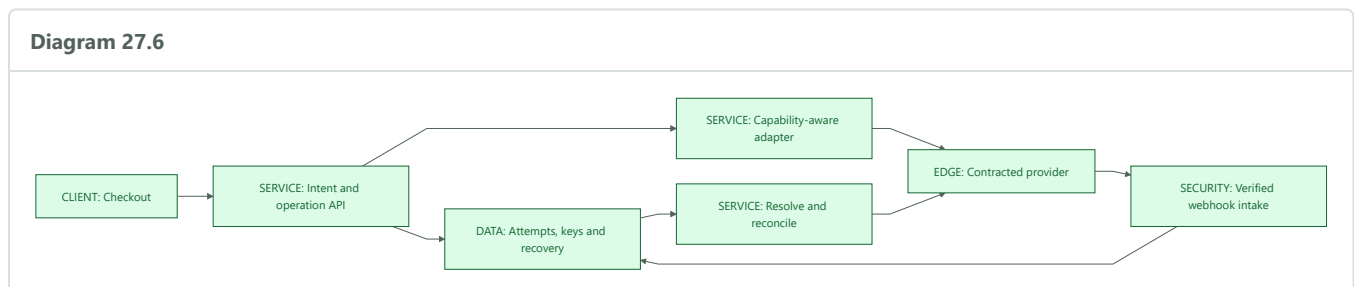
Requirements and Estimate

Support create, capture or equivalent provider operation, refund, status query and authenticated webhooks. Preserve provider capability differences and unresolved outcomes. Hypothetical traffic: 50 submissions per second with mean in-flight provider time of two seconds suggests 100 average in-flight submissions by Little's law. Tail latency, status polling and retries require a separate concurrency budget; the calculation does not justify an unbounded queue.

API and Data Model

Design `POST /payment-intents`, `POST /payments/{id}/captures`, `POST /payments/{id}/refunds` and `GET /operations/{id}`. A durable pending response includes a stable operation resource. Model intent, operation, attempt, provider reference, webhook receipt, risk decision and recovery work item. A refund identity is separate from the original payment identity and binds its own amount.

Diagram 27.6



Flow, Deep Dive and Failures

Persist the operation and selected provider before submission. Apply rate limit, timeout and risk policy. On authoritative success, transition and record the required accounting command. On definitive decline, store the no-effect outcome. On ambiguity, leave recovery pending and preserve the provider key. The browser may reconnect or repeat the same request without initiating another payment.

Webhook and polling workers can race. A unique event receipt suppresses repeated transport input; a conditional operation transition and journal command identity suppress repeated business effects. Those are separate deduplication boundaries. Retry only error classes allowed by the provider contract and the end-to-end budget; do not retry malformed requests indefinitely.

Choice	Use when	Avoid when
One provider initially	Simpler reconciliation and operational learning matter	Business requirements genuinely require independent provider paths
Capability-aware multi-provider routing	Routing and reconciliation can preserve per-provider identity and semantics	An unknown response would trigger blind failover and possible duplicate charge

IntegrationHub mapping: use Strategy/Adapter from 11, bulkheads from 10, and outbox delivery from 07.

Acceptance check: drop a successful provider response, replay the webhook, return an old event after a new state, and repeat a key with changed amount. One effect is recorded, state does not regress, and conflicting content is rejected. Provider tests remain NOT EXECUTED.

12. HLD Case Study: Ledger Service

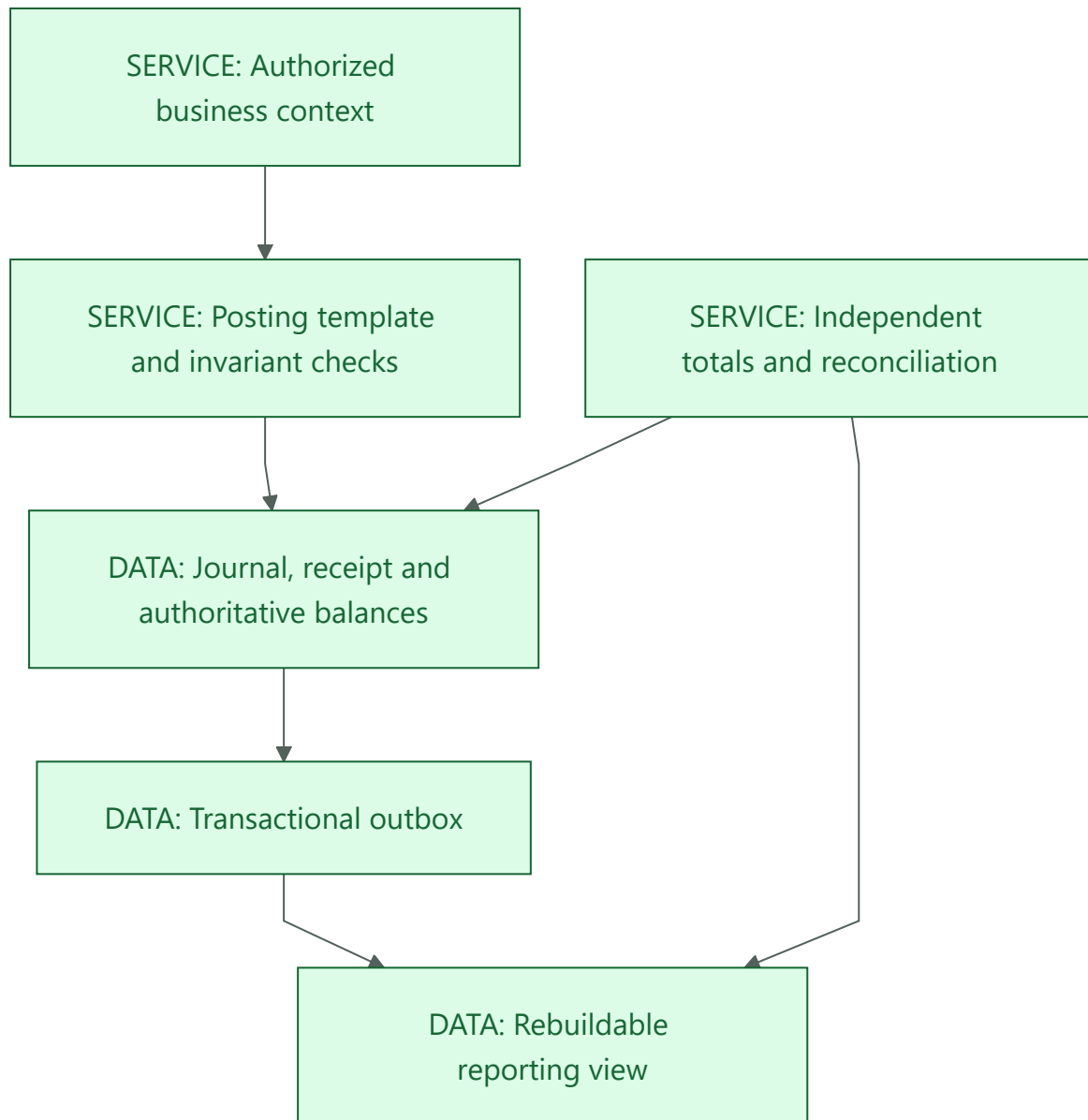
Requirements and Estimate

Accept authorized posting commands, expose immutable journals and balances, support reversals, and prove balanced entries per currency. The service is not a generic “write any debit” endpoint for every application. Define approved posting templates and caller account scope. Hypothetical load: 500 journals per second with four lines each means 2,000 lines per second; storage and index capacity need measured row sizes, retention and workload distribution.

API and Data Model

Design `POST /journal-commands` with command type, business reference, content and idempotency identity; `GET /journals/{id}`; balance queries with consistency and cutoff. Reuse the account/header/line/receipt model above. Store recorded time separately from effective business date. Backdated entries do not rewrite the order in which the system learned facts, and period-close policy must define whether they are allowed.

Diagram 27.7



Flow, Deep Dive and Failures

Resolve the permitted template, validate accounts and currency, bind content to command identity, serialize affected spend constraints, insert all lines, update any authoritative balance projection and commit the receipt/outbox. A read projection reports its cutoff; a rebuild compares per-account, per-currency sums against the journal at that same cutoff. If a projection is wrong, repair or rebuild it from authority rather than editing journal history to match the cache.

An immutable table is not magically tamper-proof: permissions, backup retention, audit controls, change approvals and restore procedures are separate. Prevent ordinary callers from UPDATE/DELETE on posted entries and constrain direct database access. A recovery must

preserve journal identity and idempotency receipts together; restoring only the balances can permit old requests to post again.

Choice	Use when	Avoid when
Transactionally updated balance	It participates in an immediate spend decision	Updates are outside the posting transaction
Asynchronous balance view	History and analytics tolerate declared lag	It is used as current spend authority
Reversal plus replacement	An accepted posting needs correction with preserved provenance	It is mistaken for deleting the original real-world event

IntegrationHub mapping: ledger ownership is a bounded context from 25. Analytics consumes it through 26, not by mutating its tables.

Acceptance check: attempt an unbalanced transaction, cross-currency sum, duplicate reference, unauthorized account and crash between inserts. None may produce a partial accepted journal. Rebuild balances at a fixed journal boundary and compare totals. Only the smaller local model checks were executed.

13. Local Model and Failure Review

`LedgerLab.mjs` tests rejection before posting, capture limits, stored capture replay, refund limits, unknown-outcome recording, conflicting request content, repeated refunds and invalid operation amounts. The Windows runner is `code/27-payments/run.ps1` ; its report records eight PASS check groups. The refund replay test intentionally returns the first response even after later refunds changed current state.

The fixture is not a production ledger. It has public mutable storage, no persistence or transactional rollback, no tenant boundary, no authenticated callers, no concurrent requests and only a simplified single-capture lifecycle. Its signed account labels are arithmetic examples, not the debit-positive account model described above. Individual amounts use safe integers, but accumulated balance reads are not a general arbitrary-precision accounting engine. Do not reuse it for real funds.

[D] **DECISION**

Choose the narrowest proven boundary. Local assertions establish arithmetic and a few transition rules. Database fault tests establish local atomicity under a tested failure model. Provider sandbox tests establish selected integration behavior. None establishes regulatory compliance or production readiness by itself.

Review failures by evidence: duplicate HTTP requests need request receipts; duplicate events need inbox receipts and effect identity; unknown submissions need provider recovery; unmatched settlement needs reconciliation; concurrent overspend needs authoritative serialization; cross-tenant access needs authorization; bad accounting classification needs finance-approved templates. One “exactly once” configuration cannot replace these different controls.

Concepts Explained Here

- [Authorization, capture, settlement, refund and chargeback.](#)
- [Double-entry ledger, ledger model and ledger HLD.](#)
- [Exactly-once effects, gateway timeout and reconciliation.](#)
- [Money state machine and fraud hooks.](#)
- [PCI scope, KYC and AML; UPI and card network.](#)
- [Wallet and payment gateway designs.](#)

Related Chapters

Use [00 master map](#), [03 transactions](#), [05 API identity](#), [06 durable invariants](#), [07 delivery](#), [08 security](#), [10 resilience](#), [11 adapters](#), [12 design method](#), [18 recovery](#), [19 testing](#), [22 distributed guarantees](#), [25 domain boundaries](#), [26 analytical evidence](#) and [24 synthesis](#). Generated relationships are reciprocal.

RELATED CHAPTERS / RECIPROCAL LINKS

00	The System Before the Details	06	Databases and Durable State
02	Concurrency and the Memory Model	11	Object Design, Patterns and LLD
04	Hibernate and JPA	10	System Design Foundations
07	Messaging, Batch and Streams	25	Architecture Styles and Domain Design
26	Data Platform Basics	05	APIs and Real-Time Communication
08	Security, Identity and Data Protection	18	Observability and Reliability
22	Distributed Systems Theory	19	Testing and Quality
12	Twelve System Design Case Studies	24	IntegrationHub in Motion
03	Spring and Spring Boot Internals	20	Appendix: Interview Toolkit and Index

One-Page Cheat Sheet

Lifecycle: payment intent, provider attempts, captures, refunds, disputes and settlement are distinct records. Preserve unknown outcomes and raw provider evidence.

Money: exact representation, explicit currency and rounding; positive command amounts; balanced journal lines per currency. Conservation does not prevent overspend.

Commit: command receipt, approved journal, authoritative balance change and outbox share the local transaction. A provider does not share it.

Identity: tenant and operation scope, content binding, stable provider key, retained original result. Never blindly fail over an unknown payment.

Recovery: verified duplicate-tolerant webhooks, status lookup, independent reconciliation, owned exceptions and auditable correcting entries.

Controls: minimize sensitive data, constrain posting authority, retain evidence and verify PCI, KYC, AML and rail-specific obligations. Eight local check groups passed; no real money or compliance workflow ran.

Interview Corner

Basic: Is Authorized the Same as Paid?

No. Explain the selected provider's lifecycle, distinguish capture and settlement, and preserve their evidence separately. Do not promise uniform rail semantics.

Internals: Why Is Double Entry Insufficient for a Wallet?

It balances journal effects but does not prevent two concurrent commands spending the same available funds. Spend authorization and posting must share the relevant serialized boundary.

Trace/Debug: The Provider Timed Out after a Capture.

Keep the operation unknown, preserve its provider key, and recover through authenticated evidence, status lookup and reconciliation. A timeout is not proof of no effect.

Scenario: A Retry Uses the Same Key and a Different Amount.

Reject a content conflict. The key names the original intent, not permission to replace it. Return the stored result only for the same content.

Basic: Refund or Reversal?

A refund is its own business operation under the provider contract; an accounting reversal corrects or offsets a journal entry. Their evidence and meaning must not be conflated.

Internals: How Does an Outbox Help?

It makes publication intent atomic with local financial state. Delivery can still repeat, so downstream receipt and business-effect identities remain necessary.

Trace/Debug: Two Webhooks Posted Two Credits.

Inspect both event receipts and journal command identity. Different event IDs can describe the same financial operation, so transport deduplication alone is insufficient.

Scenario: Capture Is Unknown and a Second Provider Is Healthy.

Do not resubmit blindly. Resolve the first operation or use an explicitly supported recovery contract before creating an independent effect at another provider.

Basic: What Is Reconciliation?

Comparison of internal records with independent external evidence at a defined cutoff, followed by owned exception resolution and auditable correction.

Internals: How Do You Handle Two Concurrent Refunds?

Serialize the remaining refundable amount and transition, bind each refund identity, and commit each accepted effect once. A status enum without amount concurrency control is insufficient.

Trace/Debug: All Journals Balance but Cash Does Not Match.

Check classification, missing operations, fees, settlement timing, currency, report revisions and cutoff alignment. Balanced entries can record the wrong business facts.

Scenario: Can Hosted Checkout Make Us PCI Compliant?

It can reduce exposure, but applicability and responsibilities require assessment of the actual data flow and current rules. Do not equate a provider token with a compliance conclusion.

Basic: Can a Card State Machine Be Reused for UPI?

Reuse intent identity, evidence and recovery concepts, not unsupported card-specific operations. The adapter must preserve each rail's verified capabilities and meanings.

Internals: What Must a Restore Preserve?

Journal identity, accepted receipts, state and reconciliation evidence at a coherent boundary. Restoring only balances can replay previously accepted financial effects.

Trace/Debug: The Original Partial Refund Now Replays as Fully Refunded.

The handler recomputed current state instead of replaying the original operation result. Keep command outcome and current payment projection separate.

Scenario: What Would You Test before Launch?

Concurrent overspend, commit/response gaps, duplicate and reordered evidence, unknown provider outcomes, reconciliation mismatches, tenant authorization, restore and correction approvals. The local lab covers only a small subset.

13 / Data Integration and iPaaS

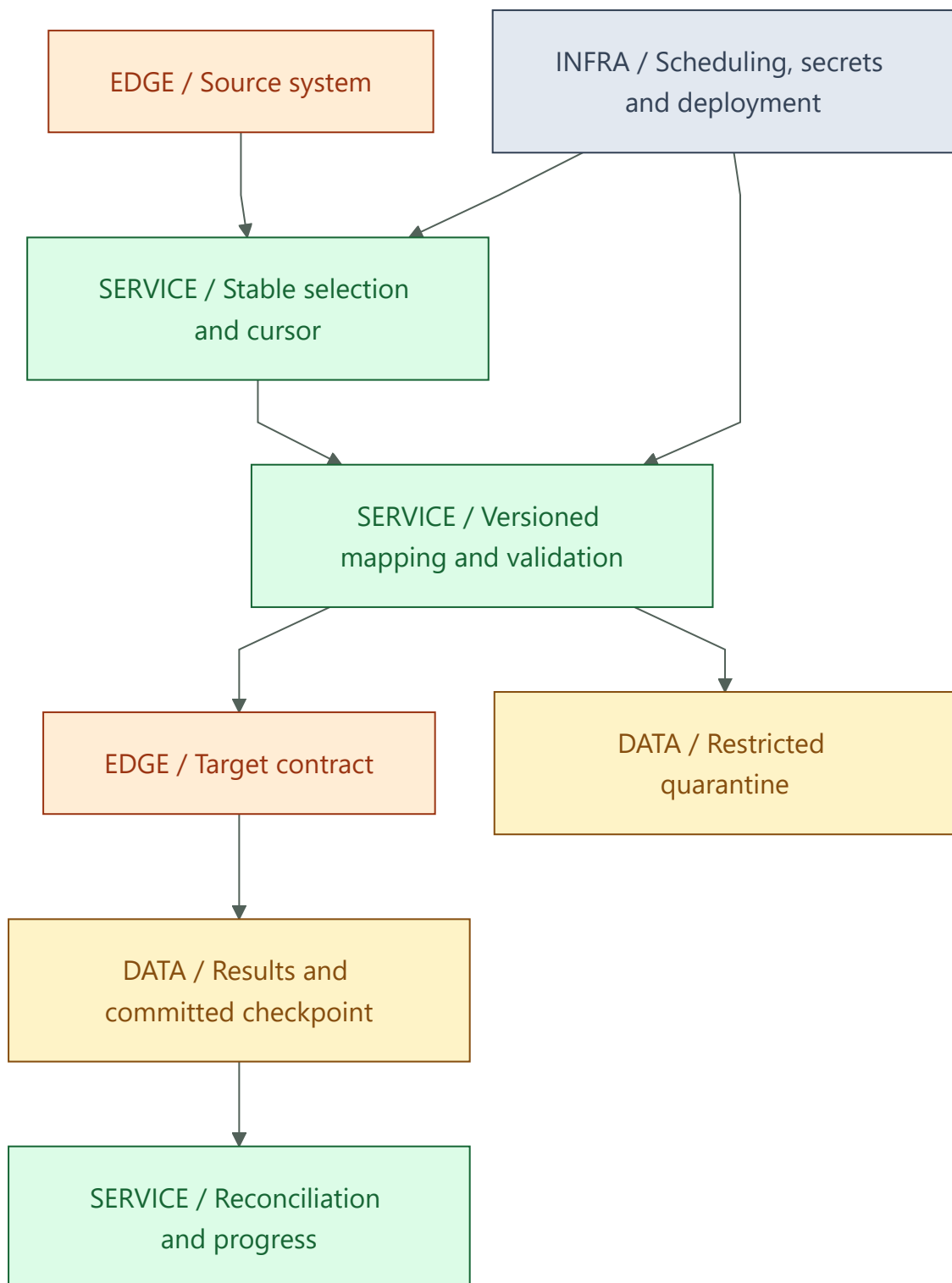
Moving bytes is easy; preserving their meaning through retries and change is the integration problem. IntegrationHub is fictional. Its connector service must know which source records were selected, which mapping interpreted them, which target effects committed, and which failures remain unresolved. A green scheduler status does not prove that all business records arrived correctly.

Version assumptions: Java 21, Spring Boot 4.0.x / Cloud 2025.1.x, PostgreSQL 17, Kafka 4.x and Kubernetes 1.34 remain the guide baseline. The optional Python model uses standard-library facilities available in Python 3.11+. IICS/IDMC, MuleSoft, Anaplan Connect and Power BI are discussed at architectural level; connector capabilities, authentication, licensing and quotas are product/version-specific.

Execution status: NOT EXECUTED. The Python runner found no non-store interpreter on PATH. Its four tests have not run. No Java toolchain, vendor runtime, credentials, source endpoint or target endpoint was installed or contacted. Downloads remain authorized for diagram dependencies only.

Big Picture

Diagram 13.1



What You Will Be Able to Explain

- Choose ETL, ELT, finite batch, incremental polling and CDC from the authority and freshness requirements.
- Define stable input, pagination, mapping versions, target identities and restart checkpoints.
- Distinguish filtering, quarantine, retry, partial completion and reconciliation.
- Explain iPaaS control-plane/runtime separation without assuming a connector name supplies delivery guarantees.
- Compare IICS, MuleSoft, Anaplan Connect and Power BI by the problem they solve.
- Design a connector port and a small Python transformation with explicit numeric and input contracts.

[P] PRODUCTION

Where this lives in IntegrationHub. 05 accepts a tenant-scoped sync job; 07 dispatches and checkpoints work; 06 owns durable local state. 08 authorizes secrets and destinations. 11 supplies connector Strategy/Factory concepts, 12 supplies the bulk-sync HLD, and 26 extends the analytical destination.

1. ETL, ELT and Execution Placement

ETL extracts from a source, transforms before loading the intended destination representation, and then loads. ELT loads selected raw or lightly normalized data into an analytical platform and transforms there. Both still require validation, security and lineage; ELT does not mean indiscriminately retaining every sensitive source field forever.

Placement changes the resource and governance boundary. Transforming in a Java worker consumes its CPU/memory and exposes data to that runtime. Pushing work into a database can exploit set-oriented execution but consume shared database capacity. Warehouse transformations can reuse retained input for new analyses, but raw data access and retention become a significant responsibility. Transformation location should follow data volume, trust, available compute and operational ownership.

Batch versus stream is a separate axis. A nightly file can be ETL or ELT; an event stream can be transformed before or after landing. Incremental polling asks for changes since a position; CDC follows database changes under its log/snapshot contract. A daily export is not automatically a complete snapshot if the source changes during extraction.

Approach	Use when	Avoid when
ETL	Only approved transformed data should reach the destination	Worker-side row loops replace a much cheaper set operation without evidence
ELT	Governed retained input and destination compute support multiple analyses	Raw sensitive data has no access or retention owner
Batch	A finite named input set and completion result matter	The freshness requirement genuinely demands continuing updates
CDC	Source changes should be captured from committed history	Log retention, snapshots and connector operations are unowned

Interview checks: basic: transformation location distinguishes ETL/ELT. Internals: placement moves compute and trust. Debug: verify whether an export is actually consistent. Scenario: choose freshness and input identity before choosing a product.

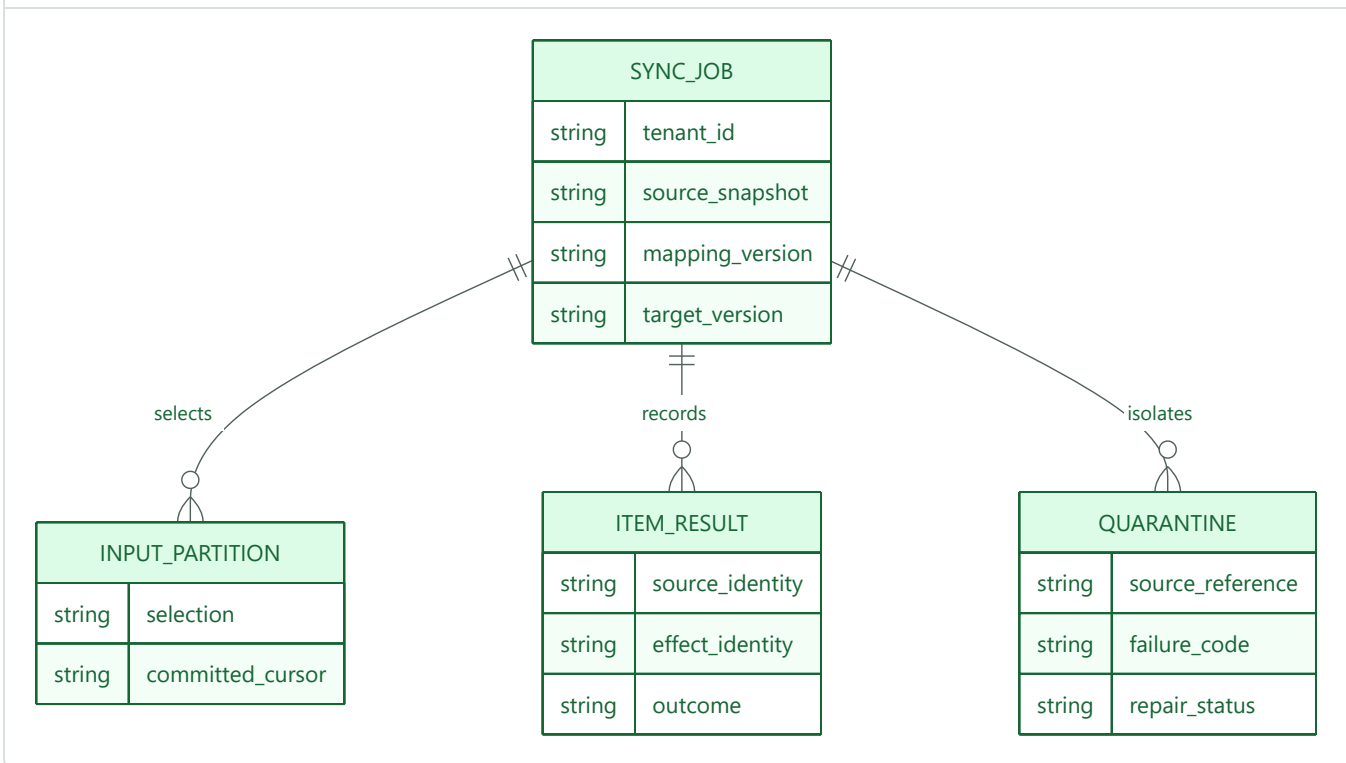
2. Source, Mapping and Target Contracts

A connector contract needs more than connect/read/write. Describe authentication and authorization scope, source identity, schema, selection rules, ordering, pagination, deletion semantics, target conflict policy and rate limits. A source can expose repeated IDs, changing rows, eventual consistency or an expiring cursor. A target can accept a batch partially even when the overall HTTP request fails.

The job manifest should bind tenant, source reference, immutable snapshot or selection boundary, schema/mapping version and destination configuration version. A restart must either reuse this contract or be explicitly treated as a new interpretation. Replaying yesterday's input through today's changed mapping without recording the change makes reconciliation and audit ambiguous.

Business keys and delivery identities differ. A customer ID identifies an entity; a source change ID or source version identifies an update; a job item identity identifies an attempt to apply a specific selected fact. Repeating a delivery should not repeat a non-idempotent target effect, while two real updates to the same customer must not be collapsed as duplicates.

Diagram 13.2



The diagram is conceptual, not a migration. Tenant scope belongs in actual keys, queries and authorization. Store only what is needed to diagnose/replay under a retention policy. A quarantine payload can contain more sensitive data than the normal mapped result, so treating it as a harmless log is a security mistake.

Contract decision	Use when	Avoid when
Immutable input manifest	Repeatable restart/reconciliation matters	A mutable file path is treated as immutable identity
Versioned mapping and target configuration	Replayed data must have explainable meaning	Deploying new code silently changes an existing job
Explicit per-item result	Target can partially accept a batch	One transport status is treated as every record's outcome

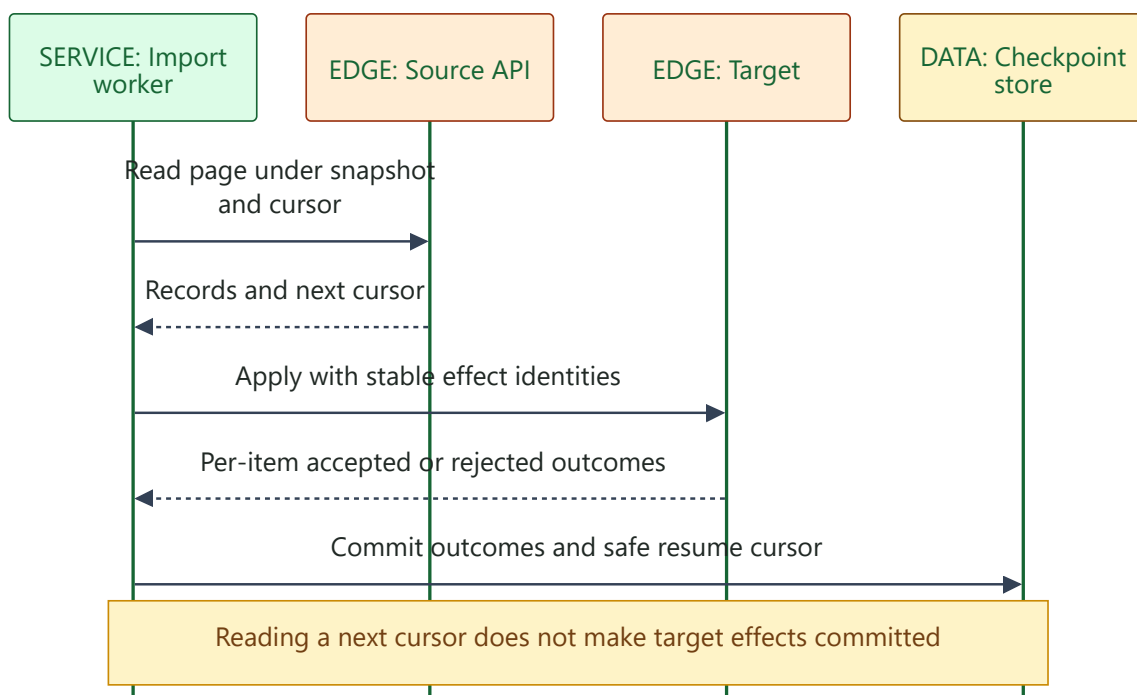
Interview checks: basic: entity identity is not update identity. Internals: restart binds input and interpretation. Debug: compare manifest versions before blaming retries. Scenario: model partial acceptance before implementing batch writes.

3. Pagination, Incremental Watermarks and Deletions

Offset pagination over mutable input can skip or duplicate rows as insertions and deletions shift positions. Keyset pagination uses a total order, such as timestamp plus stable ID, and a matching source capability. It improves traversal but does not freeze row contents. If consistent extraction matters, use a source snapshot, a stable change log or a documented overlap/reconciliation strategy.

A timestamp watermark needs tie handling, clock semantics and an upper boundary. Selecting `updated_at` greater than the last timestamp can miss another row with the same timestamp. A compound cursor or deliberate overlap plus deduplication is safer. Source timestamps may be assigned before commit, may have limited precision or may change late; a watermark is not automatically commit order.

Diagram 13.3



Deletion capture is often forgotten. An updated-row endpoint may never show removed rows. Options include tombstones/change feeds, soft-delete markers, periodic authoritative snapshots with set comparison, or source-supported deletion exports. Do not infer deletion merely from absence in an incomplete page or failed extraction.

When a cursor expires, do not invent a new one from the number of rows processed. Re-establish a supported snapshot/change position and reconcile overlap. For large extracts, bound how long source resources such as snapshots remain open; a long database snapshot can affect cleanup and storage, as [06](#) explains.

[VERIFY: confirm each source's cursor lifetime, snapshot consistency, timestamp/commit ordering, deletion feed and rate-limit contract against the selected connector/API version. No real pagination, CDC or cursor-expiry experiment ran.]

Interview checks: basic: stable order is not a frozen snapshot. Internals: equal timestamps require tie handling. Debug: investigate missing deletion events. Scenario: expired cursors require a source-supported recovery plan.

4. Batching, Idempotency and Checkpoints

Batch size balances overhead against memory, lock duration, target limits and replay cost. Bound by bytes as well as row count: a batch of one hundred large documents can be far larger than one thousand small rows. Chunk boundaries need not match source page boundaries; a page can be read once and processed in smaller target units if the checkpoint captures the exact safe position.

A local transactional target can commit output, duplicate receipt, progress and checkpoint together. An external target is a separate authority. If its effect succeeds and the response is lost, the worker must query, retry under a stable target key or reconcile. Advancing the checkpoint before target acceptance can lose data; recording it later without duplicate-safe effects can repeat changes.

Retry the smallest unit whose outcome and repeatability are known. Some batch APIs return accepted item IDs while others report an all-or-nothing contract. Retrying the entire batch after partial success is safe only when target identities or operations make it so. A job's HTTP idempotency key does not automatically deduplicate every target record.

[M] MECHANISM

A checkpoint is a claim about completed effects, not reading progress. Store it only when every earlier selected item has a committed accepted result or an explicitly permitted durable quarantine result. Otherwise restart skips unresolved input.

Batch policy	Use when	Avoid when
Small bounded chunks	Failures should replay little work	Per-request overhead dominates without measuring alternatives
Larger batch	Target supports efficient bulk semantics	Partial results, byte limits or long transactions are ignored
Idempotent target upsert	Repeated selected state should converge	A new source update is incorrectly suppressed as an old duplicate

Batch policy	Use when	Avoid when
Reconciliation	External outcome cannot be decided locally	A timeout is assumed to mean no effect

Interview checks: basic: page and commit chunk differ. Internals: target and checkpoint authority must agree. Debug: partial batch success needs per-item evidence. Scenario: preserve target effect identity across retries.

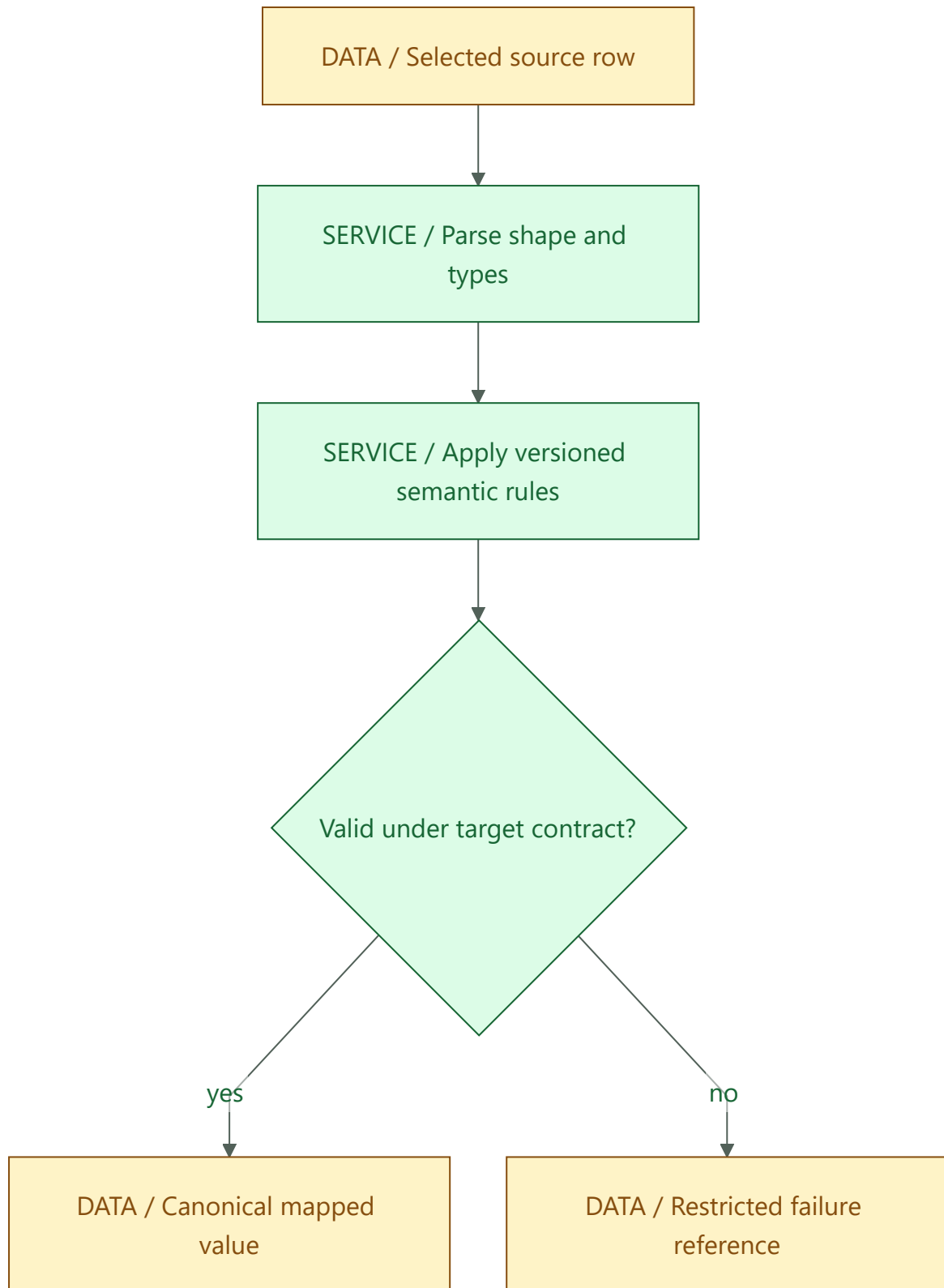
5. Schema Mapping and Data Quality

Mapping is a versioned interpretation of data: field names, types, units, nullability, enumerations, timezones and key relationships. A syntactically valid JSON object can still have the wrong currency scale or date semantics. Distinguish malformed input, unsupported schema and valid records rejected by business rules.

Use exact decimal handling for decimal quantities; distinguish a monetary minor-unit contract from arbitrary rounded floats. Define whether excess fractional precision is rejected or rounded and by which rule. Identifiers are often strings even when they contain digits. Preserve leading zeros where meaningful. A timestamp must specify instant versus local date/time, timezone and units; guessing from locale makes identical input map differently on different workers.

Schema evolution needs compatibility tests on historical fixtures. Adding a required field breaks old data; renaming or changing units changes meaning even if types still parse. Unknown enumeration values need deliberate handling. Mapping code should not silently substitute a default that turns a missing value into a legitimate business fact.

Diagram 13.4



Record counts are necessary but insufficient for reconciliation. Compare source selection counts, accepted/rejected counts, business-key uniqueness and relevant control totals or hashes under a defined canonicalization. Equal counts can still hide wrong values or duplicated

identities. A control total can also hide compensating errors, so combine checks according to risk.

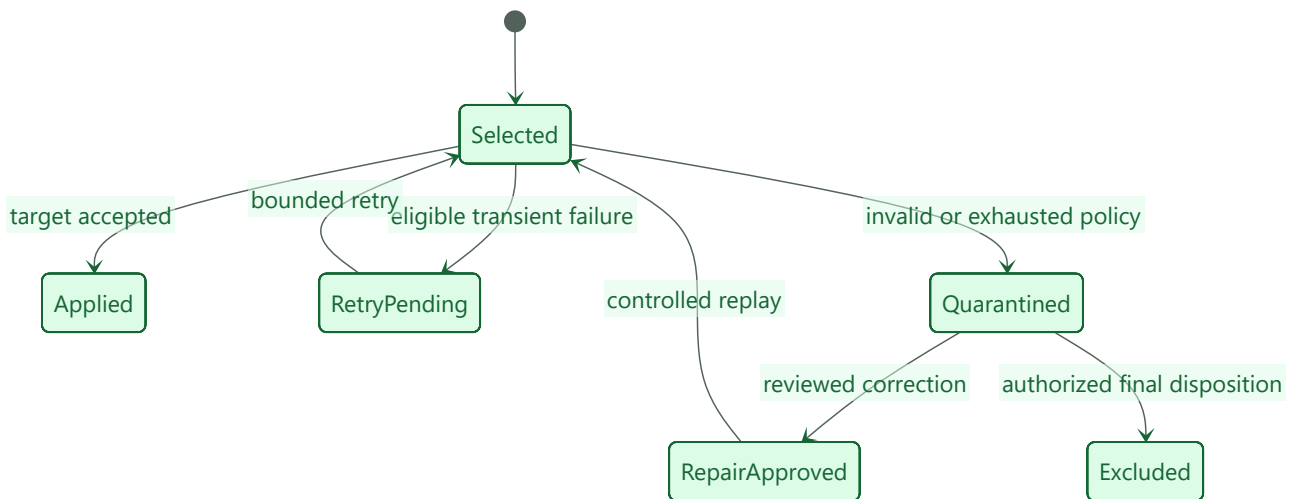
Interview checks: basic: mapping includes meaning, not only names. Internals: exact units and canonicalization affect reconciliation. Debug: compare timezone/decimal rules across versions. Scenario: test old fixtures before changing a mapping.

6. Quarantine, Repair and Partial Completion

Filtering is intentional exclusion under a rule. Quarantine records a selected item that could not be safely applied. A transient retry keeps work pending. These are different outcomes, and combining them into a skipped count prevents the operator from understanding data loss.

Quarantine needs source identity/reference, mapping version, sanitized failure class, correlation, attempt history, owner and repair state. Retain raw payload only when necessary, with restricted access and expiry. A repair can correct source data, update mapping or acknowledge an intended exclusion. Reprocessing must preserve the relationship to the original failure and target effect identity; it must not create an unaudited duplicate import.

Diagram 13.5



Partial completion is a product contract. If one invalid record can be quarantined while valid rows continue, the job should expose accepted, quarantined and unresolved counts and a completed-with-errors state. If any missing record invalidates the dataset, stop and reconcile instead. A success email must not conceal a job that dropped important records.

[!] TRAP

Quarantine is not successful delivery. It is a durable unresolved or explicitly excluded result. A job may advance past it only under a documented partial-completion policy with an owner and a reconciliation path.

Failure policy	Use when	Avoid when
Retry with budget	Failure is transient and repeated effect is safe	Permanent mapping errors repeat forever
Quarantine and continue	Partial completion is allowed and repair is owned	Missing critical records are hidden behind success
Fail the partition/job	Further progress would violate dataset integrity	One isolated optional field unnecessarily stops unrelated tenants

Interview checks: basic: filter, retry and quarantine differ. Internals: checkpoint policy must include quarantine durability. Debug: a green run with fewer records is not enough evidence. Scenario: make repair replay authorized and traceable.

7. iPaaS and Informatica IICS

An integration platform typically separates a design/control plane from execution runtimes. The control plane manages connections, mappings, schedules, deployment metadata and monitoring; runtimes execute data movement near accessible sources and targets. Hosting choices affect network reachability, secrets, upgrade responsibilities and where sensitive records travel.

Informatica IICS is a familiar name for cloud integration capabilities; current product naming and packaging may appear under IDMC. At architectural level, distinguish connection metadata, reusable mappings, executable tasks/taskflows and the runtime used for supported data integration. A Secure Agent-style runtime can provide connectivity to protected networks, but installation alone does not authorize every source or guarantee every task is executed in the same location.

Map the platform back to the earlier contracts: what identifies a run, where are parameters/versioned mappings stored, how does the runtime authenticate, which transformations are pushed down, what is the restart behavior, and how are partial target results surfaced? A connector's existence is not evidence that all API features or transactional guarantees are available. Inspect the specific connector operation and license/runtime mode.

Platform responsibility	Use when	Avoid when
Managed design/orchestration	Standardized connectors and operations reduce custom work	Generated mappings are accepted without understanding target semantics
Private-network runtime	Sources cannot be reached by a public control-plane execution path	Agent reachability is confused with least-privilege authorization
Pushdown transformation	Supported database execution is more efficient	Shared source/database capacity is overloaded by an opaque generated query

[VERIFY: confirm current Informatica IICS/IDMC naming, Secure Agent/runtime choices, connector operation support, pushdown, taskflow/restart semantics, licensing and network requirements in the vendor documentation for the selected service. No Informatica environment was accessed.]

Interview checks: basic: control plane and execution plane differ. Internals: inspect actual runtime/data placement. Debug: connection success is not task permission or correct mapping. Scenario: compare platform guarantees with the required source/target contract.

8. MuleSoft and API-Led Integration

MuleSoft commonly organizes integrations as flows in a Mule runtime, using connectors and transformations such as DataWeave. A flow receives an event, routes/transforms it and invokes other systems under error-handling policies. Understand the event/message context, streaming versus materialized payloads and where retry/error scopes begin and end; a graphical flow does not remove those decisions.

API-led connectivity often distinguishes system APIs exposing controlled access to systems of record, process APIs composing business behavior and experience APIs shaping responses for consumers. These are responsibility categories, not a rule that every operation must cross three separately deployed network services. Extra synchronous hops add latency, failures and duplicated transformation work.

A retry scope around a multi-step flow can repeat an earlier successful external call when a later step fails. Define stable identities and compensation/reconciliation instead of assuming flow-level retry is transactional. Streaming support can bound memory, but transformations that sort, join or repeatedly traverse a payload may require materialization. Inspect payload size and actual connector behavior.

Approach	Use when	Avoid when
Reusable system API	Multiple consumers need a governed source boundary	It exposes raw internal schema without a stable contract

Approach	Use when	Avoid when
Process orchestration	A visible workflow owns multiple interactions	Hidden retries repeat non-idempotent steps
Experience-specific API	Consumer needs justify a tailored representation	Three layers are deployed solely to match a slogan

[VERIFY: check Mule runtime/DataWeave versions, streaming/repeatability, transaction boundaries, connector retry/error semantics and deployment options against the chosen MuleSoft release. No Mule flow, transformation or connector was executed.]

Interview checks: basic: layers name responsibilities. Internals: retry scope determines repeated side effects. Debug: look for accidental payload materialization. Scenario: preserve business identity across multi-step integrations.

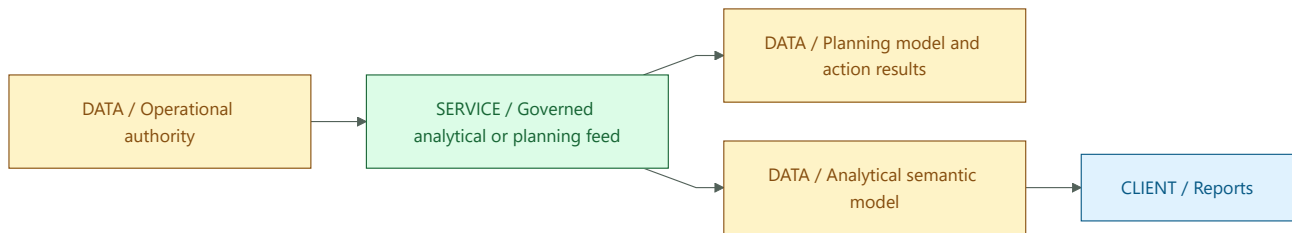
9. Anaplan Connect and Power BI

Anaplan integration connects operational data with planning models. Think in terms of workspace/model identity, supported import/export actions, mapping to model dimensions, job submission and result polling. Anaplan Connect provides a command-line integration route for supported operations; it is not a general-purpose database replication engine. Authentication, file formats, action configuration and result interpretation must match the selected environment.

An accepted import action can still produce rejected records or model-level failures. Preserve the action/result identity and inspect detailed outcomes, not only the command process exit status. A rerun against a mutable file or changed model mapping may produce different planning state; version input and configuration where the workflow requires repeatability.

Power BI is an analytical consumption/modeling platform, not the operational authority for job acceptance. Import-mode models retain data for refresh; DirectQuery-style access delegates queries to supported sources; newer storage modes have their own service/capacity constraints. On-premises data access can require a gateway. Dataset/semantic-model credentials, refresh ownership and row-level security are distinct from backend API authorization.

Diagram 13.6



Destination	Use when	Avoid when
Planning-model import	Business planning needs governed dimensions and actions	CLI completion is assumed to mean every record loaded
Imported analytical model	Refresh cadence and analytical performance fit	Users are promised real-time operational truth
Source-query mode	Supported source access and capacity meet the query workload	Report traffic overloads the transactional database

[VERIFY: verify Anaplan Connect authentication/actions/file limits/result semantics and Power BI semantic-model storage modes, gateway/refresh/RLS behavior, quotas and licensing in current product documentation. No tenant, workspace, report or import operation was accessed.]

Interview checks: basic: planning/BI is not the transactional source of truth. Internals: job acceptance and item outcomes differ. Debug: inspect model/action/refresh identities. Scenario: choose freshness and source-load policy explicitly.

10. Connector Design and the Python Lab

Separate source acquisition, mapping, target application and checkpoint coordination. A source adapter exposes a supported selection/cursor contract; a mapper produces a canonical value or classified rejection; a target adapter returns explicit item outcomes; the coordinator decides which results permit progress. Keep provider exceptions and credential details at the adapter boundary rather than spreading them through domain logic.

Python is useful for small transformation and reconciliation tasks when its operational footprint is understood. Iterators and `csv.DictReader` can avoid materializing a whole file; `Decimal` expresses decimal policy more clearly than binary floating point. Context managers close files. Explicit encodings, deterministic sorting/canonicalization and parameterized external access matter as much as language syntax. A short script still needs input identity, errors, tests and deployment ownership.

The complete source is `code/13-integration/IntegrationLab.py` , with Windows PowerShell runner `& './docs/java-fs-guide/code/13-integration/run.ps1'` from workspace root. It maps source identity/decimal amounts into minor units, rejects nonfinite/negative/overprecise values, and atomically replaces an in-memory state containing accepted rows, quarantine and checkpoint. It binds restart to an input fingerprint and mapping version.

Test method	Intended check	Evidence limit
test_decimal_contract	Exact conversion and rejection of invalid numeric values	Not executed; no target schema involved
test_chunk_rollback_and_resume	Failure before state replacement leaves no checkpoint/effects	Not database durability or process-crash recovery
test_quarantine_and_checkpoint_agree	Accepted plus quarantined rows account for committed progress	Not a real partial-success API
test_restart_contract_and_tenant_scope	Changed input/mapping rejected; tenant receipt scopes differ	Not backend authorization

Receipt identity uses selected source position within a fingerprinted immutable input, not a claim that duplicate business entities have been deduplicated across unrelated jobs. State is in memory and process loss discards it. Quarantine records sanitized reasons instead of echoing raw invalid values. A passing future test would validate this model, not IICS, MuleSoft, Anaplan, Power BI or an actual connector.

[D] **DECISION**

Use a platform when its supported contracts and operations reduce total ownership cost. Use custom adapters when the required semantics or domain behavior cannot be expressed safely. Count runtime upgrades, secret rotation, incident diagnosis and reconciliation, not just initial connector setup.

[VERIFY: run the four Python standard-library tests with an existing compatible interpreter and validate real source/target adapters separately. The current preflight is NOT EXECUTED; in-memory state replacement and input hashing are not persistence, authenticity or vendor-runtime evidence.]

[?] **INTERVIEW**

Explain a failed restart with four identities: selected input, mapping version, target effect and committed checkpoint. If any is unstable, a retry can interpret different data, repeat an effect or skip unresolved records.

Explained-Here Index and Sources

Concept	Explanation
ETL and ELT	Placement
Connector contracts and identity	Source/target boundaries
Pagination and watermarks	Stable traversal
Batching and idempotency	Commit boundaries
Schema mapping	Semantic validation
Quarantine	Repair and partial completion
IICS	Platform/runtime
MuleSoft	Flow and API layers
Anaplan Connect and Power BI	Planning and analytics
Connector design and Python	Ports and model lab

Verification destinations: [Informatica documentation](#), [MuleSoft documentation](#), [Anaplan support](#), [Power BI documentation](#) and [Python Decimal](#). References are not claims of fresh vendor-document retrieval or executed integrations.

Related Chapters

Use [00](#), [05 APIs](#), [06 databases](#), [07 batch/messaging](#), [08 security](#), [11 connectors](#), [12 bulk design](#), [16 delivery](#), [18 operations](#), [19 tests](#) and [26 analytics](#). Generated navigation is reciprocal.

RELATED CHAPTERS / RECIPROCAL LINKS

00 The System Before the Details	08 Security, Identity and Data Protection
01 Core Java and the JVM	11 Object Design, Patterns and LLD
03 Spring and Spring Boot Internals	16 Build, Ship and Roll Back
05 APIs and Real-Time Communication	18 Observability and Reliability
07 Messaging, Batch and Streams	19 Testing and Quality
26 Data Platform Basics	17 Cloud and Infrastructure
12 Twelve System Design Case Studies	24 IntegrationHub in Motion
06 Databases and Durable State	20 Appendix: Interview Toolkit and Index

One-Page Cheat Sheet

Contract: bind tenant, stable input, schema/mapping version and target configuration. Entity ID, change ID, delivery ID and attempt ID solve different problems.

Placement: ETL transforms before loading the intended target; ELT transforms after governed landing. Batch/stream is a separate axis. Platform control plane and execution runtime have different network/security responsibilities.

Traversal: stable order is not snapshot consistency. Handle timestamp ties, deletions and cursor expiry. Do not restart a changed file by line count alone.

Effects: source pages are not commit chunks. Bound bytes and rows. Commit target outcomes with checkpoints where possible; external uncertain outcomes need stable keys and reconciliation.

Quality: define units, decimal precision, nulls, timezones and unknown enums. Filtering is intentional; quarantine is unresolved or explicitly excluded work. Partial success must be visible and repairable.

Products: IICS/MuleSoft orchestrate supported integrations; Anaplan consumes planning data/actions; Power BI consumes analytical models. Names do not establish transactional guarantees. Python lab and vendor integrations remain NOT EXECUTED.

Interview Corner

Basic: ETL Versus ELT?

Transformation placement differs. Select based on compute, trust, retention and analytical reuse, not an assumption that one removes validation or governance.

Internals: Why Does Timestamp-Only Incremental Extraction Miss Rows?

Ties, precision, late changes or allocation before commit can invalidate a simple greater-than watermark. Use supported compound positions, overlap/deduplication or a source change log with a defined recovery contract.

Trace/Debug: The Run Is Green but Records Are Missing.

Compare selected, accepted, quarantined and unresolved counts and inspect target per-item results. A transport or orchestrator success may not mean all business records were applied.

Scenario: A Target Times Out After a Batch Write.

Treat outcome as unknown. Inspect item receipts, query supported status or retry with stable target identities. Do not advance the checkpoint or create fresh effect IDs merely to clear the error.

Basic: What Does an iPaaS Runtime Add?

Managed execution/connectivity and operational capabilities under supported connector contracts. It does not automatically make arbitrary source/target writes atomic.

Internals: Why Version the Mapping?

The same input can produce different values under changed units, defaults or transformations. Restart/replay must preserve or explicitly record that interpretation change.

Trace/Debug: A Retry Reprocessed the Wrong File.

The job stored a mutable path without immutable version/fingerprint. Validate input identity before resuming and treat a changed source as a new or reconciled job.

Scenario: One Bad Row in a Large Import?

Choose fail or partial completion from business rules. If quarantine is allowed, persist source reference, failure and repair owner with progress, and expose completed-with-errors rather than hiding loss.

Basic: SCIM, iPaaS and BI Are Interchangeable?

No. Provisioning, integration orchestration and analytical consumption have different authority and lifecycle contracts. A shared connector ecosystem does not erase those distinctions.

Internals: Why Can a Flow Retry Duplicate Work?

The retry scope may include an already successful external step. Track each effect identity and use compensation/reconciliation rather than assuming the entire flow rolled back.

Trace/Debug: Memory Rises Despite a Streaming Connector.

Inspect transformations that sort, join or materialize payloads, unbounded batches and target buffering. Streaming acquisition does not imply an entirely streaming pipeline.

Scenario: Platform or Custom Connector?

Compare supported semantics, security, runtime operations and long-term cost. Use a maintained platform where it fits; keep custom adapters narrow and test the exact contracts it does not supply.

14 / Docker and Container Internals

A container is a constrained process environment, not a miniature independent kernel.

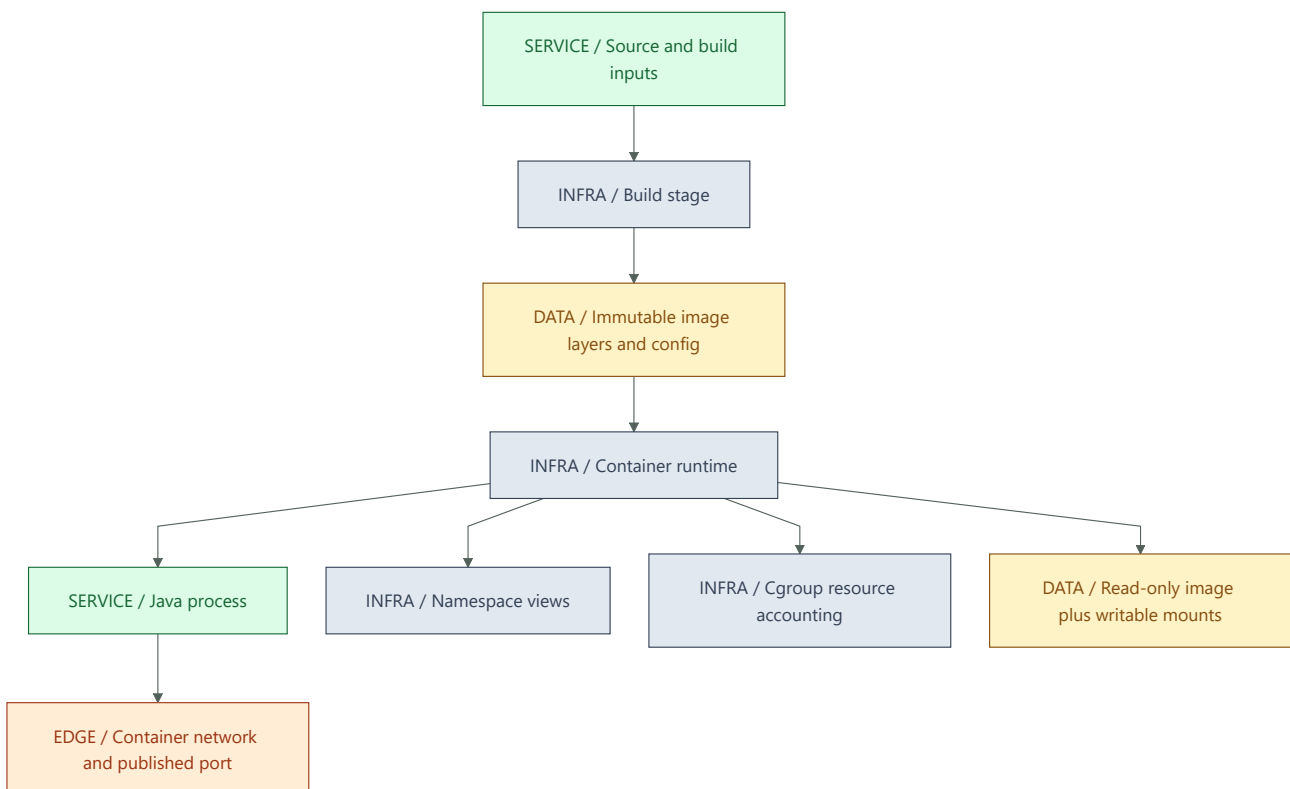
IntegrationHub is fictional. Packaging its Java service as an image makes the filesystem and startup contract portable, but resource budgets, secrets, networking and graceful shutdown still need deliberate design.

Version assumptions: Linux containers, an OCI-compatible image/runtime path and modern Docker Engine/BuildKit/Compose v2 concepts; Java 21, Spring Boot 4.0.x / Cloud 2025.1.x, PostgreSQL 17, Kafka 4.x and Kubernetes 1.34 remain surrounding baselines. On Windows, a Linux-container engine normally runs through a Linux VM/backend; Linux namespaces and cgroups are properties of that Linux kernel, not the Windows host kernel.

Execution status: NOT EXECUTED. Docker and javac are not installed on PATH. The saved Compose JSON parsed and local contract checks passed, but image build, Java compilation, daemon behavior, networking and container execution did not run. No base images were downloaded.

Big Picture

Diagram 14.1



What You Will Be Able to Explain

- Distinguish image, container, runtime, host kernel, namespaces and cgroups.
- Trace image construction, layer caching and the difference between tags and digests.
- Build a multi-stage Java image without accidentally retaining build secrets or tools.
- Explain PID 1, signal delivery, shutdown deadlines and the JVM's full process memory budget.
- Diagnose container DNS, ports, localhost, volumes and Compose lifecycle assumptions.
- Apply non-root, read-only, capability and supply-chain controls without claiming containers are perfect isolation.

[P] PRODUCTION

Where this lives in IntegrationHub. 16 builds and promotes an immutable service image; 15 schedules it and manages readiness/rollout. 01 and 23 explain JVM memory, 21 owns network/OS mechanics, and 08 owns secrets and trust. A container restart does not undo 07's committed job effects.

1. Image, Container and Runtime

An image is a content-addressed filesystem/configuration artifact. A container is a runtime instance with a process tree, namespace/cgroup membership, mounts and network configuration. Multiple containers can start from the same image while having different runtime secrets, environment, volumes and limits. Those differences are part of the deployment contract and must be reproducible too.

Docker's user-facing API coordinates image and container operations through an engine and lower-level runtime components. OCI specifies interoperable image/runtime concepts; it does not mean every engine setting or Compose feature is identical. Kubernetes typically talks to a CRI-compatible runtime rather than requiring the Docker daemon on every node.

Containers share the kernel of their execution host, unlike a full virtual machine with its own guest kernel. Kernel vulnerabilities, privileged mode, host mounts and daemon access can weaken isolation. A container boundary can be useful and still be insufficient for running mutually hostile workloads under a particular threat model.

Execution unit	Use when	Avoid when
Container	Reproducible process packaging and density fit the isolation requirement	A separate kernel is required but assumed rather than supplied

Execution unit	Use when	Avoid when
Virtual machine	Kernel separation or OS-level isolation matters	Every tiny process gets unnecessary guest-OS overhead
Local Compose environment	A bounded multi-service development topology is useful	It is presented as a production cluster scheduler

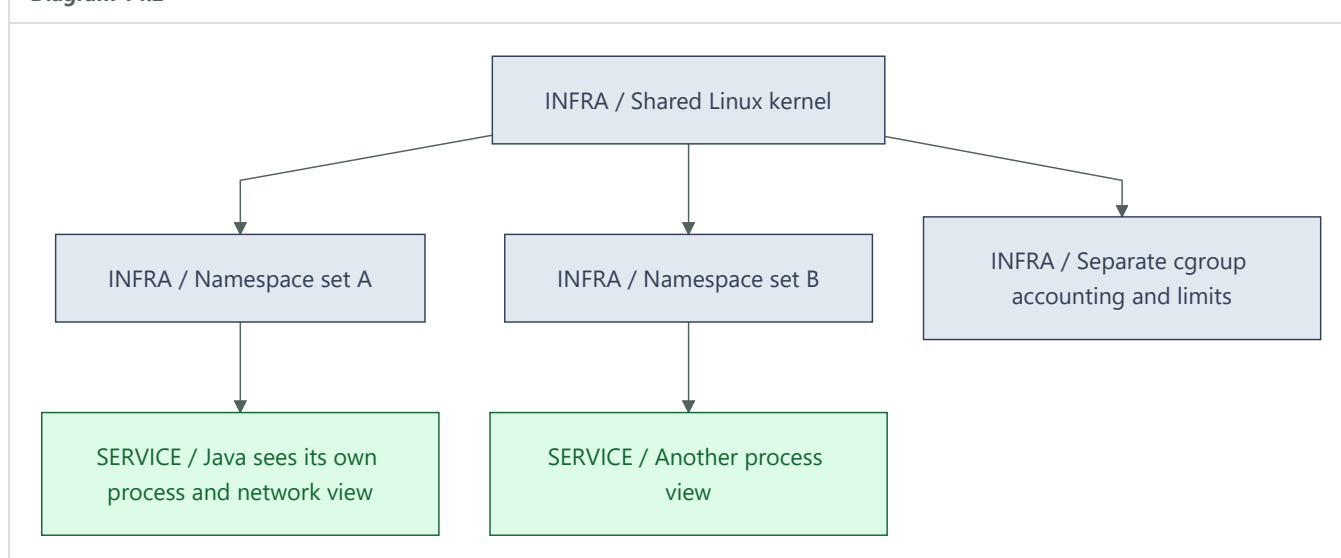
Interview checks: basic: an image is not a running process. Internals: namespace/cgroup setup occurs at runtime. Debug: compare runtime configuration as well as image identity. Scenario: choose isolation from threat and operations requirements.

2. Namespaces: Views of Shared Kernel Resources

Namespaces give processes different views of resources such as process IDs, mount trees, networking, hostnames, IPC and user IDs. A PID namespace can make a process see itself as PID 1 while the host sees a different PID. A mount namespace controls visible mount points; a network namespace has its own interfaces, routes and port bindings.

Visibility isolation is not resource capacity. A process can see a private PID space and still consume the host's CPU unless resource controls constrain it. User namespaces can map container user IDs to different host IDs, but support and interaction with storage/networking depend on the runtime. Running as root inside a default container is not automatically harmless just because it is not an ordinary host shell.

Diagram 14.2



Sharing the host network or PID namespace intentionally removes part of this separation. Mounting the host filesystem or Docker socket grants powerful access outside the image's apparent boundaries. The Docker socket is effectively an administrative capability for the

engine and often the host; do not pass it to untrusted application containers as a convenient build API.

[VERIFY: confirm namespace/user-namespace/rootless behavior, runtime privileges and Docker Desktop backend details for the actual OS/engine version. No namespace, host-mount or daemon inspection ran.]

Interview checks: basic: namespaces change visibility. Internals: host and container PID identities differ. Debug: inspect which namespaces are shared. Scenario: a privileged container or daemon socket changes the trust boundary substantially.

3. Cgroups and JVM Resource Budgets

Cgroups account for and control resource usage of a process group. CPU quota can throttle execution even when the application has runnable threads; memory accounting can trigger termination when the group exceeds its configured limit. Relative CPU weights and hard quotas solve different scheduling problems. Neither a container's advertised CPU count nor a Java thread count is a throughput guarantee.

The JVM process consumes more than heap: metaspace, code cache, direct buffers, thread stacks, native allocations and other runtime overhead also matter. Setting heap maximum equal to the container memory limit leaves no room for those costs. Memory-mapped files and page-cache accounting can complicate diagnosis. Java container awareness helps interpret limits, but the exact runtime/collector behavior must be verified for the chosen JDK and cgroup version.

A Java OutOfMemoryError is not the same as a cgroup OOM kill. The JVM may throw an error while still alive, whereas the kernel can kill the process without giving Java a chance to execute cleanup. Exit 137 commonly indicates SIGKILL, but it is not sufficient evidence of a memory kill by itself. Check runtime termination reason and host/cgroup evidence instead of guessing from a number.

Resource policy	Use when	Avoid when
CPU quota	Bound a workload's share of execution time	Throttling is mistaken for a Java deadlock
Memory limit with process headroom	Prevent one process from exhausting shared memory	Heap alone is treated as total process memory
Measured heap/native budget	Container limits need predictable behavior	A percentage is copied without workload evidence

[!] **TRAP**

A healthy heap graph does not rule out container memory exhaustion. Direct/native allocations, stacks and other charged memory can exhaust the cgroup while Java heap appears comfortable. Correlate JVM and container metrics.

[VERIFY: verify cgroup v1/v2 CPU, memory and swap accounting, JDK container awareness and collector/native headroom against the selected runtime and limits. No resource-limit or OOM/throttling experiment was executed.]

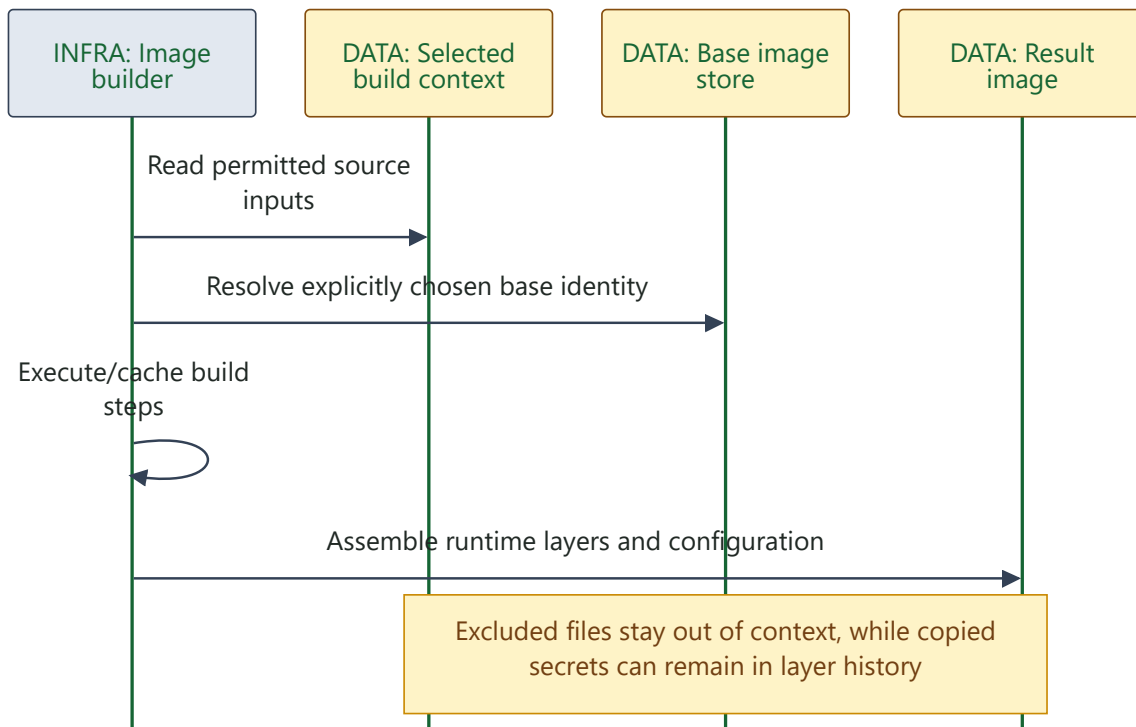
Interview checks: basic: quotas limit resources, namespaces isolate views. Internals: process memory exceeds heap. Debug: distinguish JVM error from kernel kill. Scenario: size from measured memory categories and CPU throttling, not default flags alone.

4. Image Layers, Build Context and Identity

Image layers represent filesystem changes; image configuration records startup and metadata. Build steps reuse cached results when inputs and builder rules allow it. Place stable dependency/build configuration before frequently changed source where it meaningfully improves caching, but do not confuse cache hits with reproducible dependency resolution.

A tag is a mutable name. A digest identifies content, and a multi-platform reference may identify an index whose platform-specific image is selected for a node. Record the appropriate digest/platform identity when promoting an artifact. A digest proves which bytes are referenced, not that those bytes are trusted, vulnerability-free or built from reviewed source.

Diagram 14.3



The build context is a security and performance boundary. An overly broad COPY can include source credentials, caches, logs or huge unrelated directories. Use a narrow context and `.dockerignore`. Copying a secret and deleting it in a later layer does not guarantee removal from historical layers or build caches. Build arguments and environment variables are not appropriate general secret channels; use supported secret mounts and verify how the build tool handles them.

The writable container layer is ephemeral with the container lifecycle. Persistent data belongs in deliberately managed volumes or external services with backup/recovery policy. A bind mount can hide files that were present in the image and can give the container access to host data. Neither volume persistence nor a snapshot is automatically a verified backup.

Interview checks: basic: tag is a name, digest is identity. Internals: layers retain filesystem history. Debug: inspect context and mounts when files differ. Scenario: preserve artifact provenance as well as byte identity.

5. Dockerfile and Multi-Stage Java Builds

A multi-stage build separates the toolchain from the runtime artifact. The build stage contains a compiler and any required build tooling; the runtime stage receives only the outputs it needs. This reduces runtime surface and avoids shipping caches or source unintentionally, but the build stage remains part of the supply chain and needs trusted inputs.

The complete saved Dockerfile under `code/14-docker/` is reproduced below. BUILD_IMAGE must be an explicitly chosen existing Java 21-or-later JDK image and RUNTIME_IMAGE a compatible runtime containing the jdk.httpserver module. There are intentionally no implicit default base tags. Select supported, patched identities and record their digests before an authorized build.

dockerfile

```
ARG BUILD_IMAGE
ARG RUNTIME_IMAGE
FROM ${BUILD_IMAGE} AS build
WORKDIR /work
COPY ProbeServer.java .
RUN javac --release 21 --add-modules jdk.httpserver -d out ProbeServer.java
RUN java --add-modules jdk.httpserver -cp out ProbeServer --self-test

FROM ${RUNTIME_IMAGE}
WORKDIR /app
COPY --from=build /work/out /app/classes
USER 10001:10001
EXPOSE 8080
ENTRYPOINT ["java", "--add-modules", "jdk.httpserver", "-cp", "/app/classes", "ProbeServer"]
```

NOT EXECUTED: neither base is supplied or downloaded, and no Docker/JDK exists locally. This is an executable build recipe once prerequisites are deliberately provided, not a claim of a built image. The build's self-test checks only the probe's status-selection function, not real HTTP shutdown or production readiness.

Use exec-form ENTRYPOINT so the intended process receives signals without an unexpected shell wrapper. USER supplies a numeric non-root identity, but file permissions and mounted volumes must allow the required access. EXPOSE documents the application port; it does not publish that port to the host. A runtime image without a shell can reduce surface but changes debugging options; do not assume curl or a package manager is available for health checks.

For a real Spring Boot service, copy the intended application artifact/layers, verify the selected launcher contract and avoid guessing archive paths. Maven/Gradle dependencies should be resolved from controlled repositories with reproducible versions. Do not embed database credentials into an image built once and promoted across environments.

Build choice	Use when	Avoid when
Multi-stage build	Compiler/tooling should not ship in runtime	Build-stage secrets/caches are treated as harmless
Explicit base digest	Promotion requires stable byte identity	Digest pinning is mistaken for patch management
Non-root runtime	Application needs no elevated privileges	Volume ownership requires root because permissions were never designed

[VERIFY: build the recipe with explicitly approved compatible local base images, confirm module availability, target architecture, permissions and image scanning, and test the actual Java process. Dockerfile text checks are not Docker parser/build/runtime evidence.]

Interview checks: basic: build stage and runtime stage have different responsibilities. Internals: EXPOSE does not bind a host port. Debug: distinguish missing runtime modules from compilation success. Scenario: patch bases while preserving traceable immutable promotion.

6. PID 1, Signals and Shutdown

Linux PID 1 has special responsibilities and signal behavior, including child reaping. A shell-form command can leave the application behind a shell that does not forward signals as expected. Exec-form entry points and, when appropriate, a minimal init process make the process tree and signal path clearer. An init does not implement the application's graceful business shutdown for it.

On termination, stop admitting new work, mark readiness false where applicable, finish or safely abandon in-flight work within a budget, release resources and exit. After the configured grace period, the runtime may force termination. Shutdown hooks cannot be relied on after SIGKILL, node loss or all fatal failures. Durable work must remain recoverable without a clean shutdown.

The probe sets its readiness flag false and stops its HTTP server in a shutdown hook. It has no business transactions, background queue or target integration. A real worker must coordinate poll/ack/commit boundaries from [07](#); simply closing the JVM does not settle an external provider's uncertain outcome.

[M] MECHANISM

Graceful shutdown is a bounded protocol. The platform sends a signal and eventually enforces a deadline. The application must decide which work can finish, which identity/checkpoint is durable and what a replacement can safely retry.

Interview checks: basic: signal delivery is not business rollback. Internals: the process tree affects signal forwarding. Debug: inspect PID 1 and stop timing. Scenario: test termination between effect commit and acknowledgement.

7. Container Networking and Storage

Inside a container, localhost refers to that network namespace. Another service in a separate container is normally addressed by its service name on a shared network, not localhost. Port publishing maps a host-facing endpoint to a container port under engine/network rules. The server must listen on a reachable container interface; binding only 127.0.0.1 inside it can make published traffic fail.

Compose can provide service-name discovery on its networks. Name resolution, connection establishment, TLS identity, application readiness and authorization are separate checks. A successful DNS lookup does not prove a database is listening or accepting this credential. Networks marked internal and port bindings can reduce exposure, but they are not substitutes for application authorization or host/firewall policy.

The lab publishes 127.0.0.1:18080 to container port 8080, deliberately avoiding a public host binding. Its filesystem is read-only with a temporary /tmp mount. The Java process listens on 0.0.0.0:8080 inside the container. These choices describe reachability and write permissions, not an authentication scheme; the probe is not a public service.

Connectivity choice	Use when	Avoid when
Service-name networking	Containers on one intended network communicate	Host ports and hard-coded container IPs are used as service discovery
Loopback-only host publish	A development fixture should be reachable only locally	An unauthenticated diagnostic service is exposed to all interfaces
Named volume	Stateful local data must outlive a container	Persistent storage is assumed to be a tested backup
Read-only root plus explicit mounts	Required write paths are known	Broad host filesystem mounts compensate for unclear permissions

[VERIFY: confirm engine/Compose network isolation, DNS, port binding, rootless networking and volume permissions for the selected OS/backend. No container socket, network or storage experiment ran.]

Interview checks: basic: localhost is relative to a namespace. Internals: EXPOSE, listen address and published port are distinct. Debug: trace DNS, port, TLS and readiness separately. Scenario: keep diagnostic bindings local by default.

8. Compose and the Saved Fixture

Compose describes a local multi-container application's services, networks, volumes and runtime configuration. It is useful for repeatable development topology, but not a replacement for cluster scheduling, node failure recovery or a production rollout controller. A service dependency declaration does not automatically prove business readiness; readiness conditions and retry behavior must be explicit.

The saved `code/14-docker/compose.json` uses JSON, which is compatible with the YAML data model used by Compose, so built-in parsers can check it without new dependencies. It runs the locally built `java-guide-probe:local` image with `pull_policy=never`, `init` enabled, a read-only root, dropped capabilities, `no-new-privileges`, explicit memory/CPU examples and a ten-second stop grace period. Those resource values are teaching settings, not measured sizing recommendations.

Preflight command, Windows PowerShell from workspace root: `& ' ./docs/java-fs-guide/code/14-docker/run.ps1 ' .` It parsed the JSON and checked no implicit pull, loopback binding, read-only runtime and expected multi-stage/non-root instructions. `execution.json` records `staticStatus PASS` and overall `NOT EXECUTED`.

When an approved engine and images exist, first inspect the selected base images and supply `BUILD_IMAGE/RUNTIME_IMAGE` explicitly. A build flag that disables pulling newer images does not guarantee a missing base will never be downloaded; enforce the no-download policy separately. Compose configuration validation and a local launch require separate execution authorization/prerequisites. No `build/up/down` command was run here.

Interview checks: basic: Compose describes local service topology. Internals: startup order is not readiness. Debug: inspect the resolved configuration rather than only its source template. Scenario: missing images are a prerequisite issue, not permission to download them silently.

9. Image and Runtime Security

Minimize runtime contents, run with an appropriate non-root identity, remove unnecessary capabilities, use `seccomp`/other supported confinement, and avoid privileged mode and uncontrolled host mounts. Read-only filesystems reduce accidental write surfaces but do not prevent data exfiltration or abuse of authorized network access. Supply secrets at runtime through a controlled mechanism and keep them out of image layers, environment dumps and logs.

Scan both application dependencies and OS packages, record an SBOM where the pipeline supports it, and verify provenance/signatures under a defined trust policy. A signed image proves something about signer and artifact integrity, not that the code is correct. A scan is a time-dependent input to risk decisions; newly discovered vulnerabilities can affect an unchanged digest.

Rebuild and promote patched images instead of manually upgrading packages inside running containers and losing reproducibility. Keep rollback artifacts and schema compatibility, but do not treat rollback to a vulnerable or compromised image as automatically acceptable. 16 owns the promotion and evidence chain.

[D] **DECISION**

Separate build trust from runtime privilege. The build needs a controlled compiler/dependency environment; the running service should receive only its required files, identity, mounts and network access. Neither stage should inherit broad host authority for convenience.

[?] **INTERVIEW**

Trace one concrete failure: image starts, port is published, but the process is unready or killed. Explain image identity, namespace reachability, process signals and cgroup evidence before changing random Docker flags.

Explained-Here Index and Sources

Concept	Explanation
Docker image/container/runtime	Execution model
Namespaces	Resource views
Cgroups	Resource accounting
Image layers	Build context and identity
Dockerfile and multi-stage build	Saved recipe
Container networking	Reachability and mounts
Compose	Local fixture
Image security	Trust and runtime privilege

Verification destinations: [Docker build documentation](#), [Docker security](#), [Compose specification](#), [OCI specifications](#) and [Linux cgroup v2](#). They are reference pointers, not newly measured runtime evidence.

Related Chapters

Return to [00](#). Connect [01 JVM](#), [07 durable work](#), [08 security](#), [15 Kubernetes](#), [16 delivery](#), [17 cloud](#), [18 operations](#), [21 OS/network](#) and [23 profiling](#). Related links are reciprocal.

RELATED CHAPTERS / RECIPROCAL LINKS

- | | | | |
|--------------------|---|--------------------|--|
| 00 | The System Before the Details | 08 | Security, Identity and Data Protection |
| 21 | Networking and Operating Systems | 07 | Messaging, Batch and Streams |
| 16 | Build, Ship and Roll Back | 17 | Cloud and Infrastructure |
| 15 | Kubernetes and Production Workloads | 18 | Observability and Reliability |
| 01 | Core Java and the JVM | 24 | IntegrationHub in Motion |
| 23 | JVM Performance Engineering | 20 | Appendix: Interview Toolkit and Index |
-

One-Page Cheat Sheet

Runtime: image is filesystem/configuration identity; container is a process environment. Linux containers share their execution host kernel. Namespaces change views, cgroups account and limit resources.

Memory/CPU: process memory exceeds heap. JVM OOME and kernel OOM kill differ. Quotas can throttle runnable work. Size from evidence, not a copied heap percentage.

Build: use narrow context, .dockerignore, controlled bases and multi-stage outputs. Tags move; digests identify bytes, not trust. Deleted secrets can remain in layers/caches. Build arguments are not secret storage.

Process/network: exec-form entry points and appropriate init clarify signals. Shutdown is bounded and cannot be guaranteed. Localhost belongs to a namespace; EXPOSE does not publish. Keep local diagnostics bound to host loopback.

Operations: persistent volumes are not verified backups. Compose is local orchestration, not cluster HA. Non-root/read-only/capability controls reduce privilege but do not replace authorization. Static fixture checks passed; Java/image/container execution remains NOT EXECUTED.

Interview Corner

Basic: Container Versus VM?

A container shares an execution-host kernel with isolated views and resource controls; a VM has a guest kernel. Choose based on required isolation and operational trade-offs, not merely startup time.

Internals: Namespaces Versus Cgroups?

Namespaces change what a process can see; cgroups account for and control resources. Private PID/network views do not establish CPU or memory limits.

Trace/Debug: Heap Looks Fine but the Container Dies.

Check total cgroup/process memory, native/direct buffers, stacks and termination reason. A cgroup kill can happen outside JVM heap management; exit code alone is insufficient diagnosis.

Scenario: How Would You Package a Java Service?

Use controlled build/runtime bases, explicit artifact outputs, non-root identity, required modules/files only and runtime-injected configuration. Test signals, write paths and resource limits as well as startup.

Basic: Does EXPOSE Open a Port?

No. It documents a port. The application listen address and engine/network publication determine reachability.

Internals: Why Is COPY Secret Then Delete Unsafe?

Earlier layers and caches can retain the copied value. Keep secrets out of context and use supported temporary secret delivery with controlled build tooling.

Trace/Debug: Published Port Is Unreachable.

Inspect whether the process started, listens on the correct container interface/port, is ready, and has an actual host mapping. Then check host/backend networking, not only EXPOSE.

Scenario: What Must a Shutdown Test Cover?

Signal forwarding, readiness/admission, in-flight effects, checkpoint/ack gaps and enforced grace timeout. Durable recovery must remain correct even when hooks do not run.

Basic: Is a Digest a Security Guarantee?

It identifies content. Trust additionally needs source/build provenance, signatures under policy, vulnerability assessment and appropriate runtime permissions.

Internals: Why Is the Docker Socket Sensitive?

It grants control over the engine and can enable powerful host/container operations. Treat it as administrative authority rather than a harmless local API.

Trace/Debug: Compose Dependency Started but the App Fails.

Container start is not dependency readiness or credential validity. Inspect resolved configuration, health/readiness conditions and the application's bounded connection retry behavior.

Scenario: Can You Build Without Network Access?

Only when every required base and build input already exists and the builder is configured accordingly. Missing images or dependencies are blockers, not implicit download authorization.

15 / Kubernetes and Production Workloads

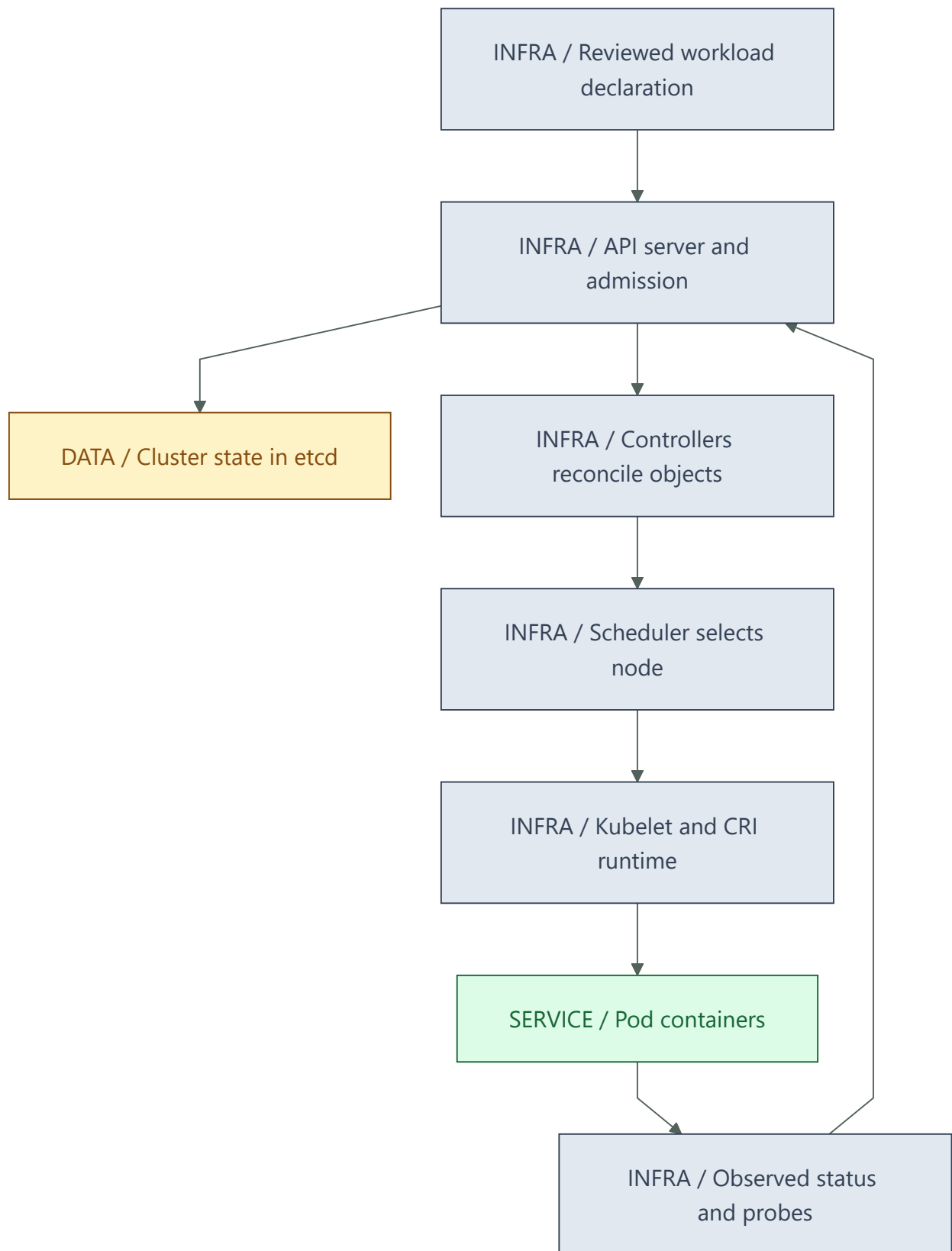
Kubernetes reconciles desired objects with observed infrastructure; it does not prove that your business operation succeeded. IntegrationHub is fictional. A healthy Deployment can coexist with stuck Kafka work, rejected provider requests or an unsafe migration. This chapter connects platform state to the application contracts already established.

Version assumptions: Kubernetes 1.34, Linux nodes and a compatible CRI runtime; Java 21, Spring Boot 4.0.x / Cloud 2025.1.x, PostgreSQL 17 and Kafka 4.x remain the guide baseline. CNI, CSI, ingress/Gateway controllers, metrics components, VPA and service meshes are separately installed implementations with their own versions.

Execution status: NOT EXECUTED. No kubectl or cluster is available. Five local checks of the saved JSON manifest passed, but Kubernetes API schema validation, admission, image execution, probes, autoscaling and rollout were not exercised. No cluster resources or cloud load balancers were created.

Big Picture

Diagram 15.1



What You Will Be Able to Explain

- Trace a workload declaration through admission, controllers, scheduling and node execution.
- Distinguish Pods, Deployments, Services, ingress, configuration, secrets and persistent storage.
- Design startup/readiness/liveness probes that do not cause cascading restarts.
- Explain requests, limits, HPA/VPA, resource pressure and JVM process memory.
- Trace a rolling deployment and explain why stalled progress does not automatically roll back.
- Diagnose Pending, ImagePullBackOff, CrashLoopBackOff, readiness failures and OOMKilled using evidence.
- Explain RBAC, Helm and Istio/Linkerd mesh responsibilities without outsourcing application authorization to them.

[P] PRODUCTION

Where this lives in IntegrationHub. 14 supplies the image and process contract; 16 changes desired image/configuration through GitOps. 07 protects durable work during restarts, 08 owns identity and secrets, and 18 correlates platform status with business SLOs. 17 supplies the managed/cloud boundaries beneath the cluster.

1. Control Plane and Reconciliation

The API server validates and admits resource changes, applies authentication/authorization and persists accepted state through the cluster's storage path. etcd stores cluster state; it is not the application's ordinary database. Controllers observe desired and current resources and take actions toward convergence. The scheduler binds eligible unscheduled Pods to nodes based on constraints and available requested resources. The kubelet coordinates execution and reports status on each node.

These components are asynchronous. An accepted API write is not a completed rollout. A controller can be delayed, an image can fail to pull, a volume can fail to attach, or application startup can fail. Read status conditions and events to identify which stage owns the current failure instead of treating apply success as readiness.

Desired and observed generations help detect whether a controller has processed a specification revision. Controllers can retry actions, so application operations still need idempotency. Kubernetes recreating a worker does not know whether an external connector effect committed before the old worker disappeared.

Platform boundary	Use when	Avoid when
Declarative controller	A stable desired state should be continually reconciled	An accepted declaration is called completed execution
Job/CronJob-style execution	Finite work needs platform scheduling and status	Platform completion is assumed to mean exactly-once business effects
External managed state	Stateful reliability is better delegated to a database service	A Deployment is expected to supply database replication/backups

Interview checks: basic: reconciliation is asynchronous. Internals: scheduler selects, kubelet executes. Debug: find the first unsatisfied stage. Scenario: durable job identity must survive replacement Pods.

2. Pods, Deployments and Workload Identity

A Pod is a scheduling unit containing one or more containers that share a network namespace and selected volumes. Containers in one Pod can reach each other through localhost; containers in different Pods generally cannot. A Pod has its own lifetime and UID. When a node is lost or a Pod is replaced, the controller creates another Pod rather than moving the same running process to a new node.

A Deployment manages ReplicaSets to maintain and roll out interchangeable replicas. Labels and selectors connect ownership and traffic targeting. Accidental selector changes or overly broad selectors can orphan intended traffic or include the wrong Pods. A Service should select only the intended workload, not every application in a namespace with a vague label.

StatefulSet adds stable ordinal/network/storage associations under its contract, useful for some stateful systems. It does not implement application replication or elect a database primary correctly by itself. DaemonSets place node-oriented work on eligible nodes. Jobs and CronJobs have different completion and concurrency semantics from long-running Deployments; all need application-level duplicate-safe effects.

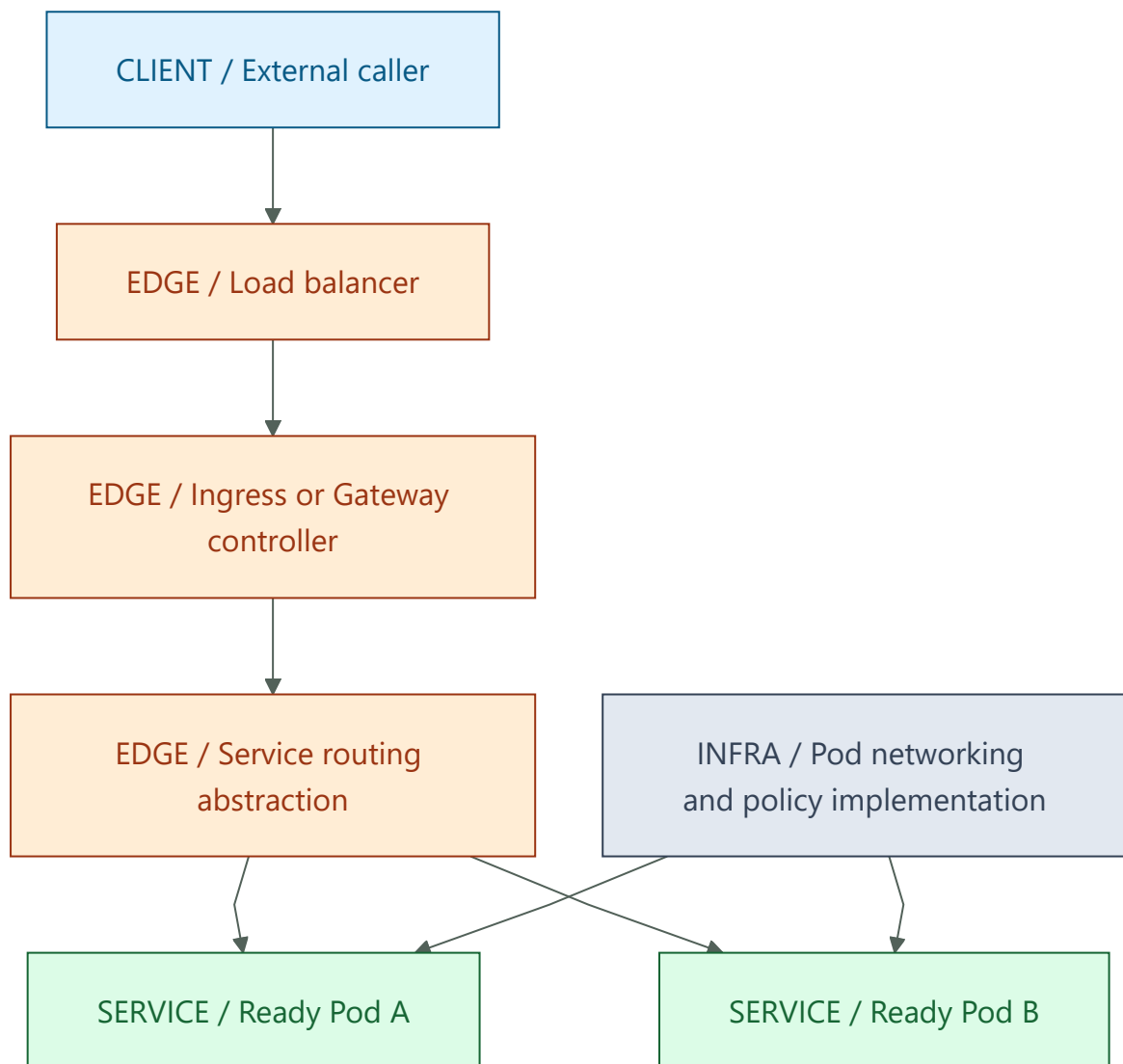
Workload	Use when	Avoid when
Deployment	Replicas are replaceable service instances	Stable per-instance data identity is required but ignored
StatefulSet	Stable identities/volume associations support a designed stateful system	It is treated as an automatic database HA solution
Job/CronJob	Finite or scheduled execution fits the lifecycle	Repeated scheduling is assumed impossible
DaemonSet	Node-local agents should follow eligible nodes	Ordinary request-serving replicas are tied unnecessarily to every node

Interview checks: basic: a Pod is not a durable server identity. Internals: controller and Service selectors serve different roles. Debug: compare label sets before changing networking. Scenario: separate application identity from Pod UID/IP.

3. Services, Ingress and Network Policy

A Service provides a stable discovery/routing abstraction over endpoints. ClusterIP is internal service access; NodePort and LoadBalancer expose other access patterns according to implementation. EndpointSlices describe eligible backends. Data traffic normally goes to the selected application endpoint through the cluster's networking implementation, not through the Kubernetes API server for every request.

Diagram 15.2



Ingress declares HTTP routing intent and needs an implementing controller. Gateway API offers another set of traffic-management resources; support and behavior depend on installed controllers. Do not assume applying an Ingress object creates a functioning public route without a controller, address, DNS, TLS and policy. Long-lived SSE paths require appropriate buffering and idle-timeout configuration at each intermediary.

CNI supplies Pod connectivity according to its implementation. Service routing can use kube-proxy rules or other mechanisms such as eBPF-based implementations; packet paths are not identical across clusters. NetworkPolicy declares allowed traffic under a supporting enforcement engine. Without such support, a policy object may not enforce the isolation you expect. Policies are not HTTP resource authorization.

[VERIFY: confirm Kubernetes API support, CNI/Service routing, NetworkPolicy enforcement, ingress/Gateway controller behavior and load-balancer/TLS integration for the chosen cluster. No network policy, ingress or packet path was tested.]

Interview checks: basic: Service discovery and HTTP ingress differ. Internals: API server is not the normal application data path. Debug: inspect endpoints/selectors before blaming DNS. Scenario: verify controller and policy enforcement, not merely object existence.

4. ConfigMap, Secret and Configuration Rollout

ConfigMaps hold nonsecret configuration; Secrets hold sensitive configuration under Kubernetes access controls. Base64 encoding is representation, not encryption. Encryption at rest, API authorization, audit policy and external secret integration require explicit configuration and operational controls. Do not put credentials into a ConfigMap because it is easier to inspect.

Environment variables are read when the container process starts; updating the source object does not rewrite a running process's environment. Mounted configuration can update under supported volume behavior, but the application still needs to notice/reload it. Subpath mounts and immutable objects have important update distinctions. Rebuilding a datasource, SSL context or connection pool is a separate step from replacing a file.

Version configuration or include content hashes in workload templates when a rollout should establish the new configuration consistently. Keep old versions available long enough for safe rollback, subject to secret compromise policy. A mixed fleet can run old/new config during a rollout, so both states must be compatible where they share dependencies.

Delivery method	Use when	Avoid when
Environment variables	Startup configuration is simple and non-dynamic	Live refresh is assumed after editing a ConfigMap
Mounted files	Application supports controlled reload	File updates are assumed to rebuild stateful clients automatically
Versioned references	Reviewed rollout and rollback need explicit identity	Old/new pods use incompatible schemas or secrets without overlap policy

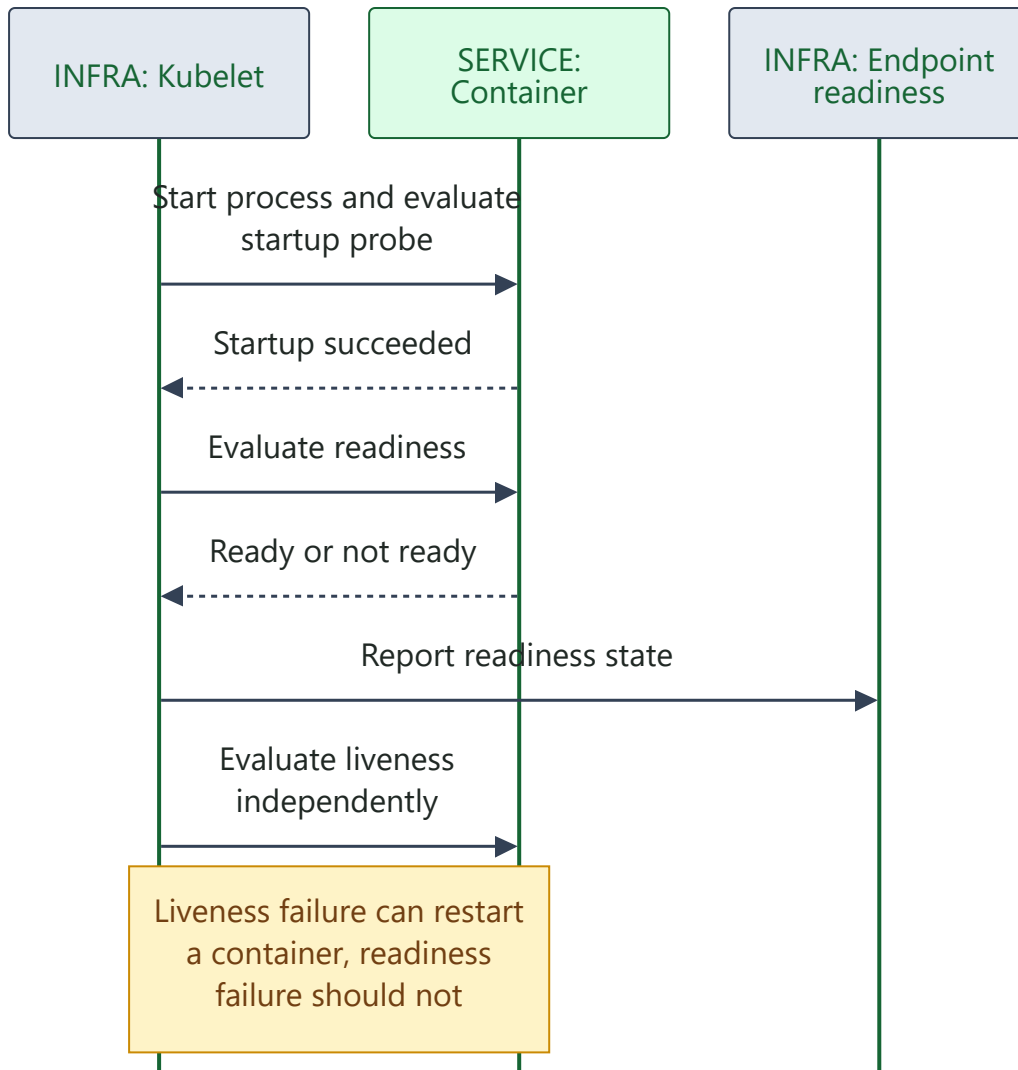
[VERIFY: verify Secret at-rest/access configuration, ConfigMap/Secret volume propagation, subpath behavior, application reload and external-secret rotation for the selected cluster/runtime. No configuration or secret rollout ran.]

Interview checks: basic: encoded is not encrypted. Internals: delivery and reload are different transitions. Debug: inspect what the process actually loaded. Scenario: config compatibility matters during rolling coexistence.

5. Startup, Readiness and Liveness

A startup probe gives a slow-starting container a startup policy before ordinary readiness/liveness probing proceeds under the kubelet contract. Readiness answers whether this endpoint should receive new traffic. Liveness answers whether the container should be restarted after a configured failure threshold. They are not interchangeable URLs with different names.

Diagram 15.3



Do not include a remote database or optional downstream service in liveness without a compelling recovery argument. If the dependency fails, restarting every application replica does not repair it and may increase load. Readiness may reflect dependencies necessary for serving, but an overbroad readiness check can remove every backend and eliminate useful degraded behavior.

Readiness changes do not instantly terminate all existing connections or guarantee propagation through every external load balancer. During shutdown, stop admission, withdraw readiness, drain under a budget and preserve durable work. A probe responding 200 means the implemented check passed, not that a user's sync completed or a payment settled.

The fixture uses `/live` for startup/liveness and `/ready` for readiness. Its Java probe only models its own startup/shutdown flag. A real Spring Boot readiness group must be chosen according to actual service behavior rather than copied from this diagnostic server.

[VERIFY: confirm probe timing, startup gating, endpoint propagation, termination behavior and Spring Boot health-group configuration against the selected versions. The fixture's routes are statically matched; no kubelet probe or shutdown test ran.]

Interview checks: basic: unready is not dead. Internals: probe policy affects routing versus restart. Debug: identify whether a remote outage caused a restart loop. Scenario: define the smallest check that proves the intended property.

6. Requests, Limits, HPA and VPA

Resource requests influence scheduling and other resource-policy behavior; limits constrain runtime consumption under the container runtime/kernel. A CPU limit can cause throttling; memory pressure can cause kills or eviction depending on the boundary. A memory request is not a promise that every allocation will succeed. Account for JVM heap plus native/process memory as [14](#) explains.

HPA adjusts replica count using configured metrics. CPU utilization targets are commonly relative to requested CPU, so requests are part of the control loop. Missing metrics, inappropriate requests, slow startup and a bottleneck outside the Pods can make scaling ineffective. Scaling a worker fleet beyond database connections, partition parallelism or provider quotas simply moves the failure.

VPA can recommend or adjust resource requests under its installed mode and version. It is a separate component, not a universal default feature. Combining VPA changes to CPU requests with HPA utilization targets can alter the denominator and create undesirable interactions. Choose complementary signals and verify behavior, including whether changes require disruption or support in-place updates.

Control	Use when	Avoid when
Explicit requests	Scheduling needs honest resource demand	Zero/arbitrary requests distort packing and HPA interpretation
HPA	More replicas can increase useful throughput	Shared dependencies or hot partitions are the real bottleneck
VPA recommendations	Resource evidence should improve requests	Automatic changes conflict with another scaling loop
Load/backlog metric	CPU does not represent the constrained work	Queue depth alone ignores age, service time and downstream quota

[M] MECHANISM

Autoscaling is a feedback loop with delay. Metrics arrive after work occurs, replicas take time to schedule/start, and dependencies may not gain capacity. Preserve headroom and admission control instead of expecting scaling to absorb every spike instantly.

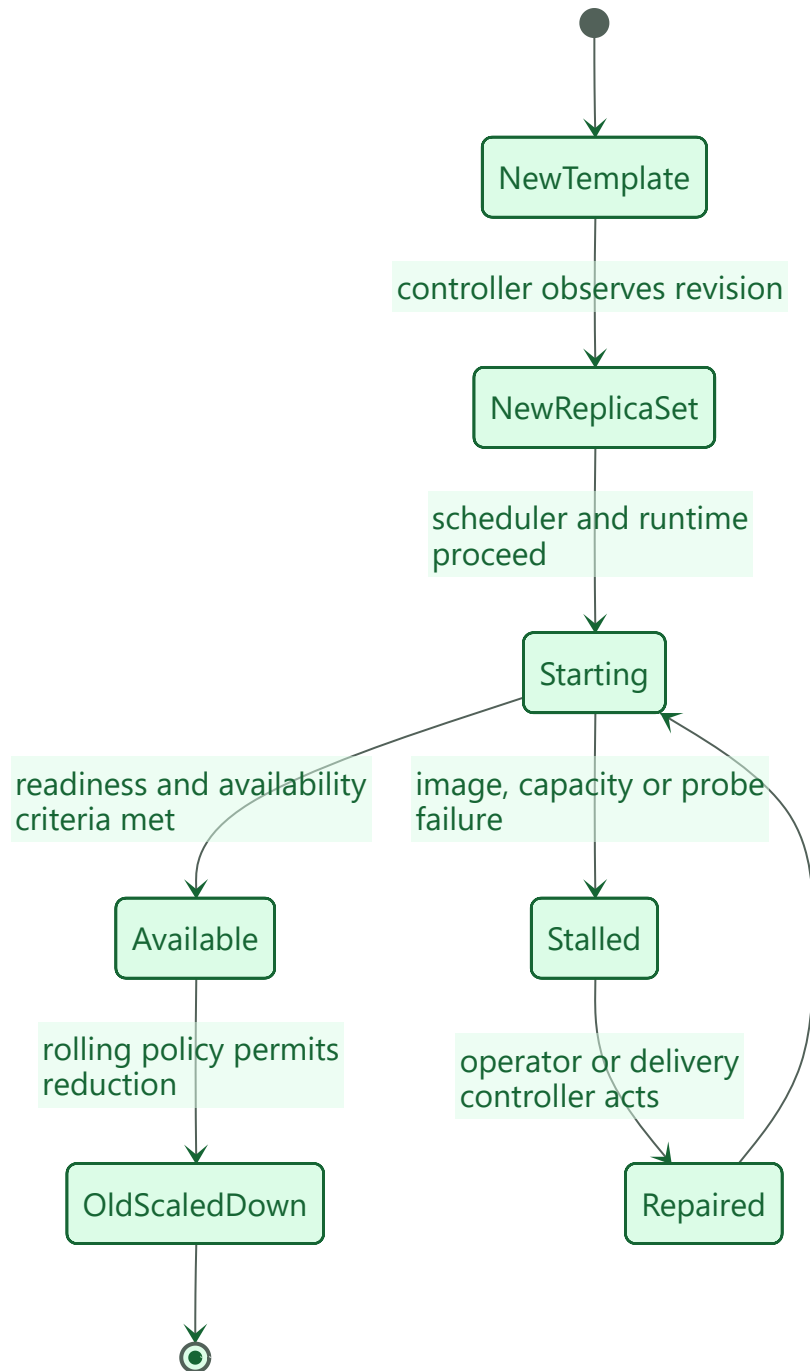
[VERIFY: verify requests/limits enforcement, metrics availability, HPA stabilization, VPA modes and in-place resizing support for the actual Kubernetes/components/JDK combination. No autoscaling, resource-pressure or OOM experiment ran.]

Interview checks: basic: requests and limits differ. Internals: HPA's metric denominator matters. Debug: check missing metrics and shared constraints. Scenario: coordinate scaling with admission and recovery headroom.

7. Rollouts, Disruption Budgets and Helm

A Deployment rolling update creates new replicas and removes old ones according to surge/unavailable policy, readiness and progress. maxSurge needs spare schedulable capacity; maxUnavailable constrains rollout availability but does not prove enough business throughput. A new Pod that becomes ready too early can pass a rollout while failing real requests.

Diagram 15.4



A progress deadline reports stalled progress; the basic Deployment controller does not promise automatic business-aware rollback. An external delivery controller or operator can choose rollback under evidence. A PodDisruptionBudget constrains eligible voluntary disruptions, not all node failures and not every Deployment rollout action. Plan application redundancy and rollout strategy separately.

Helm packages and templates Kubernetes resources with values and release metadata. Rendering a chart can produce syntactically valid but semantically unsafe resources. Review rendered output, scope values, avoid plaintext secrets in values/history, and validate chart changes like code. Helm rollback changes selected release resources; it cannot restore a database column that a migration deleted.

Rollout tool	Use when	Avoid when
Rolling Deployment	Old/new versions can coexist and spare capacity exists	Shared schema changes break one version during rollout
PDB	Controlled voluntary eviction needs an availability budget	It is called protection against every outage
Helm	Parameterized reusable resource packages help	Template abstraction hides unsafe rendered policies

Interview checks: basic: rollout progress is not business correctness. Internals: surge consumes capacity. Debug: inspect the new ReplicaSet and its Pods. Scenario: rollback includes config/schema compatibility, not only an image revision.

8. Stateful Workloads and Persistent Storage

PersistentVolumeClaims request storage under a StorageClass/CSI contract; volume provisioning, attachment, topology and access modes affect scheduling and failover. A Pod can remain Pending because a volume cannot bind or attach even when CPU and memory are available. An application that expects multiple writers cannot safely assume a single-writer volume will provide that behavior.

StatefulSet identity helps associate a replica with its volume and network identity. Replication, leader election, corruption recovery and backups still belong to the stateful application/operator. Recreating a Pod with the same claim does not mean the database has recovered or caught up. Test restore and failover, including key/credential availability, rather than equating persistent storage with durability against all failures.

Volume deletion/retention depends on resource policy and workflow. Do not casually delete claims to resolve an attachment problem. emptyDir is Pod-lifetime scratch, not persistent business storage. Node disk pressure and ephemeral-storage usage can evict workloads even when Java heap is fine.

[VERIFY: confirm CSI/StorageClass topology, access modes, reclaim/retention policy, snapshot consistency and StatefulSet update/recovery behavior against the selected storage and operator. No volume, database operator or restore was exercised.]

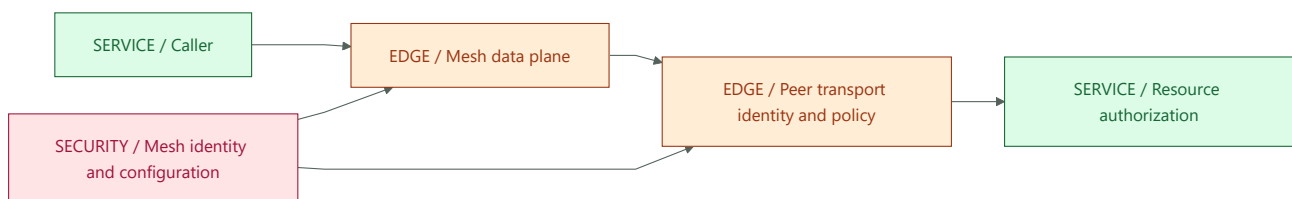
Interview checks: basic: persistence is not backup. Internals: storage topology constrains placement. Debug: inspect PVC/PV/events before deleting anything. Scenario: choose a managed database when it reduces the operational burden appropriately.

9. RBAC and Service Mesh

RBAC grants verbs on resource types within a scope. Bind the smallest roles required by a workload or operator; namespace scope is useful but not a complete tenant-security guarantee. Avoid mounting service-account credentials into a Pod that needs no Kubernetes API access. Admission/security policies, network policy and application authorization are complementary controls.

Istio and Linkerd are service-mesh options that provide configured transport identity, traffic policy and observability through their control/data-plane designs. Sidecars and other mesh architectures differ by product/version. A mesh can establish mTLS between workloads but cannot infer whether a caller may access a particular tenant's job.

Diagram 15.5



Traffic splitting can support canaries, but evaluation and promotion still need workload metrics and a rollback decision. Mesh retries can compound application/gateway retries and repeat unsafe mutations. Mesh injection adds resource, startup and debugging responsibilities; validate probe interception, shutdown order and protocol behavior rather than assuming transparency.

[D] DECISION

Adopt a mesh for a specific operational need. Consistent workload identity or traffic policy can justify it, but a small system may be simpler with explicit TLS, good clients and platform routing. Count data-plane cost and failure diagnosis along with features.

[VERIFY: confirm RBAC/admission enforcement and the chosen Istio/Linkerd data-plane architecture, mTLS identity, traffic split, retry, probe and shutdown behavior for installed versions. No mesh or RBAC authorization test ran.]

Interview checks: basic: RBAC governs Kubernetes API actions. Internals: mesh transport identity is not tenant authorization. Debug: count retries at all layers. Scenario: add a mesh only with an owner for its configuration and failure modes.

10. Troubleshooting Playbook

Start with the intended revision, namespace, Pod UID and timeline. Inspect status conditions and recent events, then correlate container logs, previous termination state, resource metrics and application traces. These labels are diagnostic categories, not captured output from this environment.

Symptom	First discriminating evidence	Direction of repair
Pending	Scheduling events, requested resources, taints/affinity, PVC binding	Fix placement/capacity/storage constraints rather than restarting the app
ImagePullBackOff	Image reference, registry credentials, node reachability and platform	Correct image availability/authentication; do not change application probes
CrashLoopBackOff	Previous container logs, exit reason, command/config and liveness events	Fix startup failure or inappropriate restart policy; backoff is a symptom
Readiness failure	Exact probe response/port/path and required dependencies	Correct readiness contract or serving failure; avoid automatic restart as the only response
OOMKilled	Termination reason plus cgroup/process/JVM memory	Bound allocations and budget process memory; distinguish heap/native pressure
Running but not serving	Service selectors/endpoints, listen port, network policy, TLS and ingress	Follow the data path hop by hop

For CrashLoopBackOff, current logs may miss the prior process's failure; inspect previous container evidence under authorized access. For image failures, a mutable tag that existed on one node may not be available elsewhere. For Pending during rollout, maxSurge may request capacity the cluster lacks. For readiness oscillation, aggressive thresholds can cause unstable routing even when restarting is unnecessary.

Read-only diagnostics such as get/describe/logs still require authorization and can expose sensitive metadata or logged values. Select the approved cluster/context and namespace before use. Do not paste secrets or raw customer payloads into incident notes. This chapter executes no kubectl command and performs no forced deletion or resource mutation.

[!] TRAP

Do not erase the evidence first. Deleting Pods or raising limits immediately may hide the failure while preserving its cause. Capture termination reason, previous logs, events and resource history, then make the smallest change that tests the diagnosis.

Interview checks: basic: status reason narrows the owning layer. Internals: a restart loop is not a root cause. Debug: compare image, scheduling, process, probe and network evidence. Scenario: identify one falsifiable hypothesis before changing several settings.

11. Saved Workload and Validation Limits

`code/15-kubernetes/workload.json` is a native JSON Kubernetes List containing Namespace, Deployment, ClusterIP Service and autoscaling/v2 HPA. It references the local image from Chapter 14 with imagePullPolicy Never, two initial replicas, bounded surge/unavailability, non-root/read-only security settings, disabled service-account token mounting and explicit scratch storage.

The CPU/memory/probe/HPA values are illustrative, not benchmark-derived. The HPA needs a functioning metrics pipeline; the image must exist on every eligible node under the local-image approach. Production should promote approved immutable registry identities with the right pull policy and credentials. This diagnostic workload is not IntegrationHub-lite and contains no business backend or database.

Windows PowerShell from workspace root: `& ' ./docs/java-fs-guide/code/15-kubernetes/run.ps1 '`. The actual preflight parsed JSON and checked selector agreement, named ports, HPA target, local-image/token policy and probe paths. It recorded staticStatus PASS and overall NOT EXECUTED. It does not validate against Kubernetes OpenAPI schemas, admission policies or a running API server.

[?] INTERVIEW

Describe which controller owns the next action. API acceptance, Pod scheduling, image startup, readiness and business processing have different owners. This keeps an operational answer precise instead of treating every symptom as Kubernetes being down.

Explained-Here Index and Sources

Concepts	Explanation
Kubernetes and control plane	Reconciliation
Pod, Deployment and StatefulSet	Workloads
Service and Ingress	Network path
ConfigMap and Secret	Configuration
Probes and readiness	Health contracts
HPA, VPA and resource limits	Resource control
Rollout and Helm	Change management
Stateful storage	Persistence
RBAC, Istio, Linkerd and service mesh	Policy
Pending, image pull, crash loop, OOM and readiness failures	Troubleshooting

Verification destinations: [Kubernetes concepts](#), [probe configuration](#), [resource management](#), [Helm](#), [Istio](#) and [Linkerd](#). No installed behavior is inferred solely from these pointers.

Related Chapters

Use [00](#), [06 state](#), [07 workers](#), [08 security](#), [10 capacity](#), [14 containers](#), [16 delivery](#), [17 cloud](#), [18 operations](#), [21 network](#) and [23 JVM performance](#). Generated navigation is reciprocal.

RELATED CHAPTERS / RECIPROCAL LINKS

- | | |
|---|---|
| 00 The System Before the Details | 16 Build, Ship and Roll Back |
| 01 Core Java and the JVM | 07 Messaging, Batch and Streams |
| 02 Concurrency and the Memory Model | 08 Security, Identity and Data Protection |
| 21 Networking and Operating Systems | 18 Observability and Reliability |
| 03 Spring and Spring Boot Internals | 17 Cloud and Infrastructure |
| 06 Databases and Durable State | 10 System Design Foundations |
| 22 Distributed Systems Theory | 23 JVM Performance Engineering |
| 14 Docker and Container Internals | 24 IntegrationHub in Motion |
| | 20 Appendix: Interview Toolkit and Index |
-

One-Page Cheat Sheet

Control: API acceptance is not convergence. Controllers reconcile, scheduler binds, kubelet/runtime executes. Replaced Pods have new lifetimes; business identity must outlive them.

Traffic/config: Service selectors and endpoints connect traffic, ingress requires a controller, CNI/policy implementation matters. Base64 is not encryption. Environment and file configuration have different refresh behavior.

Health/resources: startup, readiness and liveness prove different things. Do not restart every Pod because a remote dependency failed. Requests drive scheduling/metric interpretation; limits affect runtime. HPA/VPA are delayed loops with dependencies and constraints.

Change/state: surge needs capacity, stalled Deployment progress does not imply automatic rollback, and PDB covers particular voluntary disruptions. StatefulSet/PVC identity does not supply replication or backups. Helm output still requires review.

Diagnosis: Pending is placement/storage, ImagePullBackOff is image access, CrashLoopBackOff is repeated process/restart failure, OOMKilled needs memory evidence, and unready needs the exact probe contract. Mesh mTLS is not tenant authorization. File checks passed; all cluster runtime behavior remains NOT EXECUTED.

Interview Corner

Basic: What Happens After Applying a Deployment?

The API accepts desired state, controllers reconcile ReplicaSets/Pods, the scheduler selects nodes and kubelets start containers. Each stage can fail independently; apply success is not readiness.

Internals: Is a Pod Moved to Another Node?

Normally a replacement Pod is created with a new identity/lifetime. The application must recover durable work independently of the original process or IP.

Trace/Debug: A Service Has No Reachable Backends.

Check selectors, EndpointSlices, readiness and ports before DNS or ingress changes. Then inspect CNI/policy and the actual listen address.

Scenario: Database Outage Causes Every Pod to Restart.

The liveness check may include a remote dependency that restarting cannot repair. Separate local liveness from readiness/degraded service policy and bound downstream retries.

Basic: HPA Versus VPA?

HPA changes replica count from metrics; VPA recommends/adjusts resource requests under its mode. Their feedback loops can interact, especially when CPU utilization uses requests as a denominator.

Internals: Why Can Rollout Stay Pending With Healthy Old Pods?

Surge requires spare resources and compatible placement/storage. Old replicas can remain healthy while new ones cannot schedule, pull an image or become ready.

Trace/Debug: Exit 137 Means Heap Exhaustion?

Not necessarily. Inspect termination reason and cgroup/JVM evidence; SIGKILL can have other causes, and process memory includes more than heap.

Scenario: A Secret Was Rotated but the App Uses the Old Value.

Check whether it was injected as environment, mounted with update limitations, or read into a long-lived client. Secret delivery and application reload/reconnect are separate.

Basic: Does StatefulSet Make PostgreSQL Highly Available?

No. Stable identity/storage supports an application/operator design; replication, promotion, backups and restore testing remain separate responsibilities.

Internals: Does PDB Prevent Every Outage?

No. It constrains eligible voluntary disruptions, not all failures or every rollout action. Redundancy, capacity and application recovery still matter.

Trace/Debug: Mesh Enabled and Requests Are Duplicated.

Inspect retry policies at client, gateway, mesh and application layers. Multiple retry owners can multiply attempts, especially after uncertain mutation outcomes.

Scenario: What Does a Manifest-Only Test Prove?

Only the checked file properties. API schemas, admission, scheduling, probes, networking and application correctness need their own environments and tests.

16 / Build, Ship and Roll Back

A release is a traceable change to running behavior, not merely a successful build.

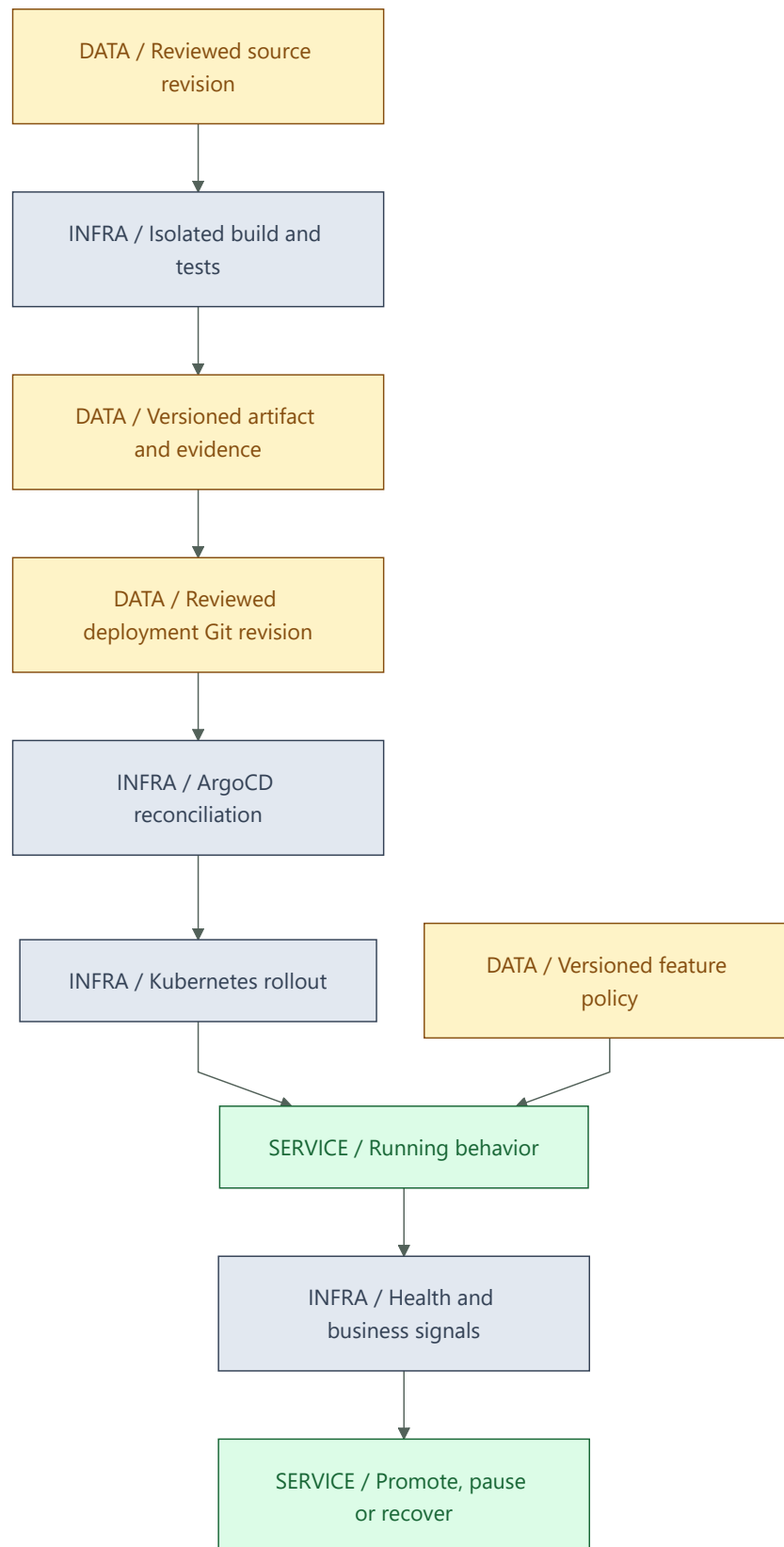
IntegrationHub is fictional. Source, dependencies, tests, image, deployment configuration, schema and feature flags all contribute to what a user experiences. Each needs identity, ownership and a recovery policy.

Version assumptions: Java 21, Spring Boot 4.0.x / Cloud 2025.1.x, PostgreSQL 17, Kafka 4.x and Kubernetes 1.34 remain the baseline. Maven/Gradle, Jenkins/plugins, ArgoCD, Harness and LaunchDarkly versions must be chosen and verified separately. The supplied Jenkinsfile expects an approved Linux agent with JDK 21+, not the current Windows shell.

Execution evidence: PromotionLab.mjs passed four synthetic policy checks on the existing Node-compatible runtime. Jenkins pipeline execution, Java builds, Maven/Gradle resolution, image publishing, GitOps reconciliation and deployment are NOT EXECUTED. No credentials, registry, cluster or cloud account were used.

Big Picture

Diagram 16.1



What You Will Be Able to Explain

- Trace a reviewed source revision to an immutable promoted artifact and running configuration.
- Distinguish Maven lifecycle/scopes from Gradle task graphs and understand reproducible/offline build limits.
- Design Jenkins stages, shared-library use, agents and credentials around trust boundaries.
- Explain ArgoCD desired-state reconciliation and Harness-style orchestration without giving both conflicting ownership.
- Compare rolling, blue-green and canary releases and separate deployment from feature exposure.
- Plan application, schema, configuration and flag rollback together.
- Connect Agile/Kanban work-in-progress discipline with small, testable changes and operational feedback.

[P] PRODUCTION

Where this lives in IntegrationHub. Jenkins builds/test-checks 14's container inputs, deployment Git references immutable image identity, and ArgoCD drives 15's workload. 06 owns schema compatibility, 08 owns pipeline trust, 18 supplies promotion evidence, and 19 explains quality gates. The reference design does not require every delivery product to be deployed together.

1. Git Workflow and Release Identity

Short-lived branches and frequent integration reduce the amount of unreviewed divergence, while protected review/check policies make the mainline a useful integration point. Long-lived release branches can be justified for supported versions, but every additional line creates merge, testing and patch-propagation work. Branch naming does not establish quality; enforced checks and ownership do.

A release should identify source commit, dependency/build inputs, artifact digest, test/security evidence, configuration revision and applicable schema/flag state. A version label alone is too weak if the same tag can point to rebuilt bytes. Promote the same tested artifact rather than rebuilding independently for each environment and hoping outputs are identical.

Reverting a published change creates an auditable new history entry; rewriting shared history can break collaborators and evidence. A revert may still be unsafe if later data or schema changes depend on the removed code. Treat source reversal as one input to recovery, not an automatic undo of every side effect.

Workflow	Use when	Avoid when
Frequent integration with protected checks	Small changes and rapid feedback fit the team	Mainline is deployable only by convention with skipped gates
Maintained release branches	Supported versions need independent patches	Branches drift without clear support and merge policy
Build once, promote by identity	Test evidence must follow exactly the deployed bytes	Each environment silently rebuilds from a moving dependency set

Interview checks: basic: source revision and artifact identity differ. Internals: promotion preserves evidence-to-bytes linkage. Debug: compare digests, not only version names. Scenario: a revert still requires schema/data compatibility review.

2. Maven and Gradle Builds

Maven's lifecycle phases organize work: validate, compile, test, package, verify, install and deploy are not synonymous commands. Plugins bind goals to phases. verify can include integration/quality gates configured in the project; install places artifacts in the local repository; deploy publishes to an artifact repository, not automatically to Kubernetes. Skipping tests to get an artifact quickly changes what evidence that artifact has.

Dependency scopes separate production compilation/runtime from testing or platform-provided APIs. dependencyManagement/BOM entries control versions for dependencies actually used; they do not necessarily add all listed libraries to the application. Plugin versions and repositories matter too. A transitive upgrade can change behavior even when direct source did not change.

Gradle models a task graph supplied by plugins and project configuration. With common JVM plugins, build coordinates checks and assembly, but exact tasks and integration-test wiring are project-specific. api versus implementation under the Java Library plugin controls intended dependency exposure. A version catalog names coordinates; dependency locking and verification address different reproducibility/trust needs.

Java toolchains select a compiler/runtime, while --release 21 constrains Java language/API/bytecode compatibility for the compilation target. Running the build with a newer JDK does not automatically make every emitted artifact compatible with Java 21 unless configured appropriately. Record compiler and build-tool identity along with the dependency graph.

Offline flags mean required artifacts must already be available; they are not an instruction to install missing dependencies. A wrapper may need its own distribution before the build tool can even honor an offline flag. Maven's offline verify or Gradle's offline build still fail when plugins/dependencies are absent. Do not retry online silently under the guide's restriction.

Build technique	Use when	Avoid when
Pinned toolchain and dependency policy	Repeated builds should have explainable inputs	Moving snapshots/plugins silently change outputs
Verified dependency cache	CI needs efficient repeatable resolution	Untrusted branches can poison a trusted cache
Separate integration-test gate	Real boundaries require more than unit tests	package success is assumed to include unconfigured integration tests
Offline mode	Inputs are intentionally preprovisioned	It is presented as a way to obtain missing artifacts

[VERIFY: confirm Maven/Gradle plugin lifecycle, toolchain, dependency locking/verification, wrapper distribution and test-task behavior against the actual project and versions. Neither build tool nor its dependency cache is available here; no Maven/Gradle build ran.]

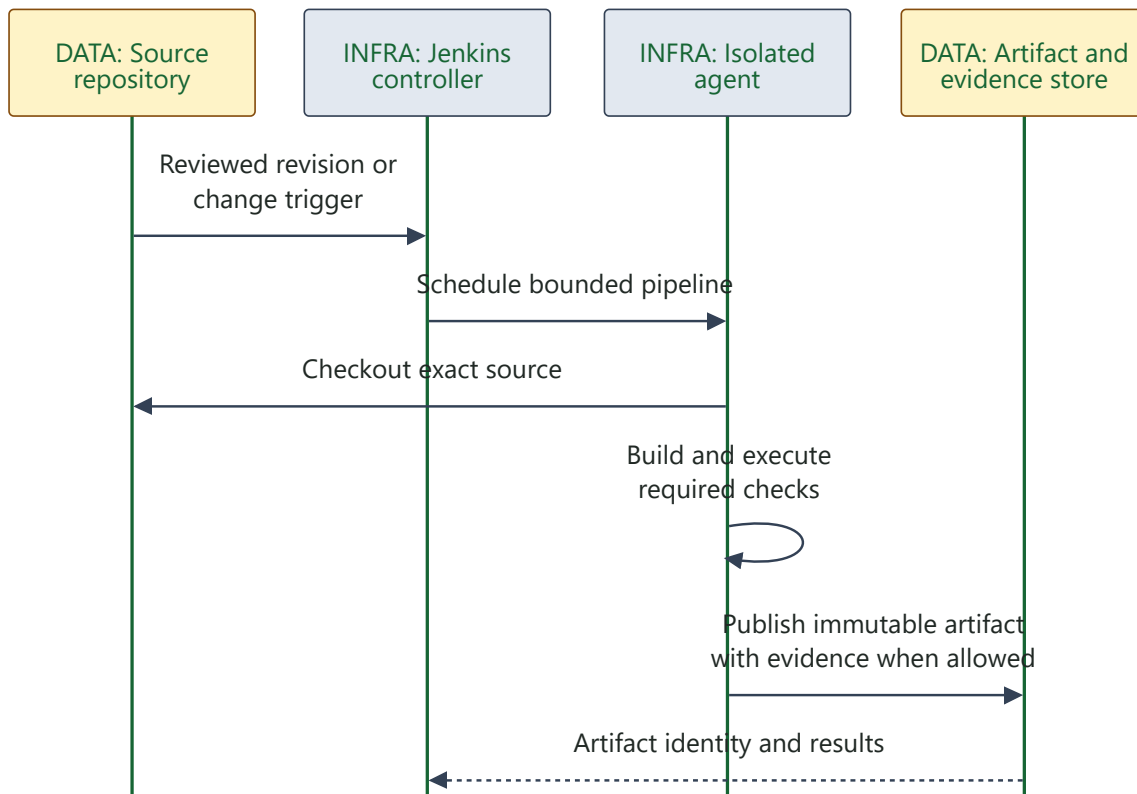
Interview checks: basic: Maven deploy is repository publication. Internals: phases/goals and task graphs differ. Debug: inspect effective configuration and resolved graph. Scenario: reproducibility includes plugins, toolchains and trusted caches.

3. Jenkins Pipelines, Agents and Shared Libraries

The controller coordinates jobs, configuration and scheduling; agents execute build steps in their assigned environments. Keep untrusted builds away from controller administration and protected credentials. An agent label expresses an environment requirement, not proof that the environment is correct or isolated. Prefer disposable or well-controlled workers with bounded workspaces and explicit tool versions.

A declarative pipeline organizes stages and policy such as timeouts, concurrency and post actions. Checkout identifies source, build produces outputs, tests create evidence, and artifact publication/promotion occurs only after required gates. A stage green because errors were swallowed is not evidence of success. Archive test results even on failure where safe, but do not publish a release artifact as approved after mandatory checks failed.

Diagram 16.2



Shared libraries reduce repeated pipeline code but become powerful supply-chain inputs. Pin reviewed library versions, define stable step contracts and test changes before broad rollout. A trusted library may execute privileged logic; letting an untrusted pull request select arbitrary library code can cross a security boundary. Avoid a library that hides all stages and leaves teams unable to explain what ran.

Credential binding narrows how secrets are exposed to a step, but log masking is not containment. Malicious build code can exfiltrate values even if common string patterns are masked. Do not give untrusted change builds production deploy/signing credentials. Use short-lived scoped identity when supported, isolate environments, and prevent secrets entering command-line history, artifacts or caches.

The saved `code/16-delivery/Jenkinsfile` checks out this repository on a linux-jdk21 agent, compiles Chapter 14's `ProbeServer.java`, runs its self-test and archives class files. It deliberately has no registry push, credential binding or deployment step. Those require a real trusted environment, not placeholders that appear production-ready. The pipeline itself is NOT EXECUTED.

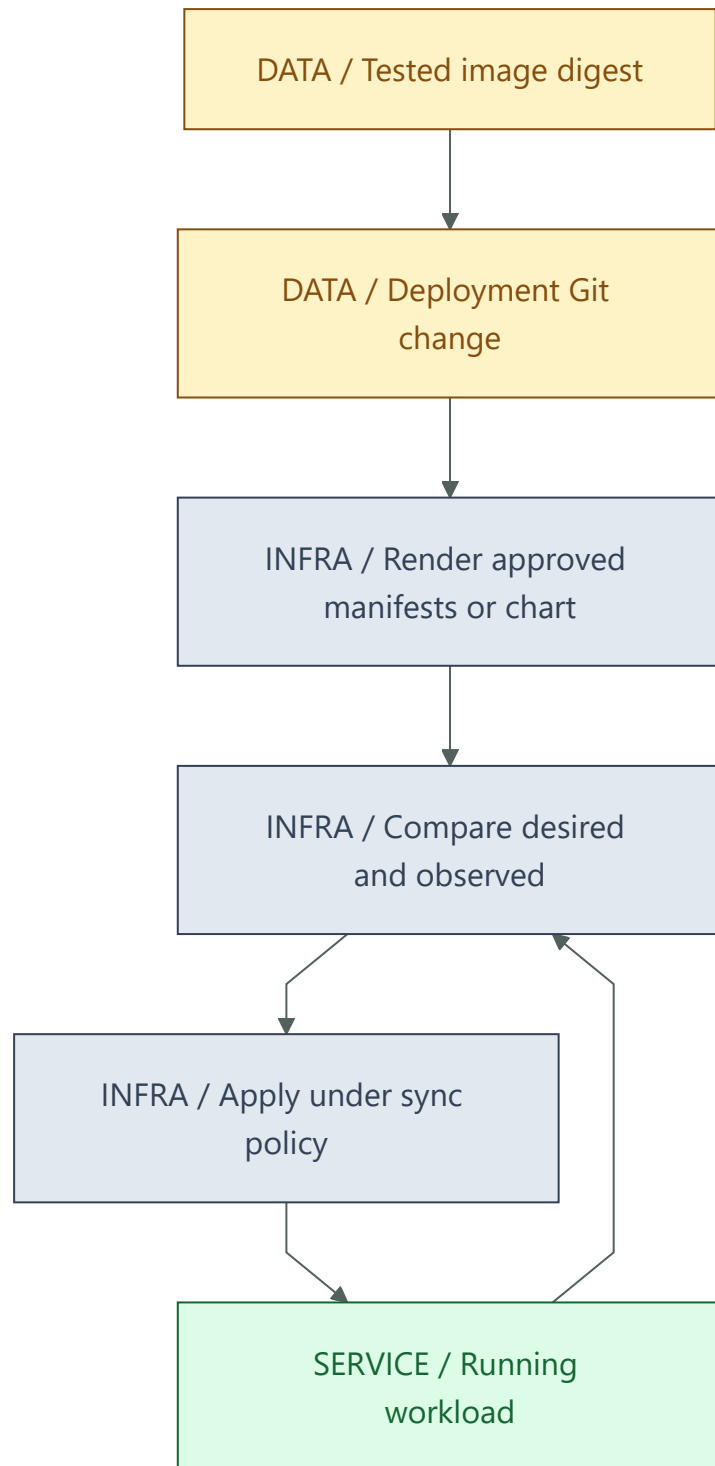
[VERIFY: validate Jenkins declarative syntax, plugin/shared-library versions, agent isolation, credential binding and SCM/approval permissions in the selected controller. The saved Jenkinsfile is source-reviewed only; no Jenkins parser or agent executed it.]

Interview checks: basic: agents execute, controller orchestrates. Internals: shared libraries and credentials are trust inputs. Debug: inspect actual stage exit status and archived results. Scenario: separate untrusted tests from privileged publication.

4. ArgoCD and Desired-State Ownership

GitOps makes reviewed configuration the desired-state source. ArgoCD compares rendered desired resources with the cluster, applies permitted changes under sync policy and reports sync/health status. It is not the same job as compiling source or proving business outcomes. A Synced application can be unhealthy; a healthy Kubernetes resource can still violate a business SLO.

Diagram 16.3



Self-healing can restore declared state after manual drift; pruning can remove resources no longer desired. Both need boundaries and safeguards. A broad prune affecting shared or stateful resources can be destructive. A manual `kubectl rollback` may be reverted by reconciliation if Git still demands the new image, so durable rollback normally changes the desired revision as well.

Controllers may legitimately own fields such as replica count under HPA. Define ownership and precise diff/ignore behavior rather than letting GitOps and autoscaling repeatedly overwrite each other. Avoid blanket ignore rules that hide unsafe drift. Secrets should be referenced or rendered through an approved secret-management path, not committed as plaintext under the excuse that the repository is private.

GitOps policy	Use when	Avoid when
Automatic reconciliation	Desired state is reviewed and changes are bounded	Unreviewed changes can immediately reach production
Controlled pruning	Resource ownership and deletion consequences are known	Shared data resources disappear from a chart accidentally
Explicit field ownership	HPA/operators legitimately mutate fields	Broad diff suppression hides all configuration drift

Interview checks: basic: sync, resource health and business health differ. Internals: reconciliation can undo manual recovery changes. Debug: inspect rendered desired state and field ownership. Scenario: recover through the declared source, with emergency exceptions controlled and reconciled.

5. Harness and Delivery Orchestration Choices

Harness represents another delivery-platform approach with pipeline orchestration, environment/service abstractions, connectors and execution components such as delegates depending on product configuration. Compare it by which decisions it owns: build, deployment, verification, approvals, rollback and evidence. Do not assume a product integration makes the whole pipeline safe or that its default verification matches the service's SLO.

An execution delegate/agent needs scoped network reachability and credentials. A platform that can deploy to many environments can become a powerful central trust boundary; partition permissions and approval scopes deliberately. If Jenkins builds and another platform deploys, preserve artifact identity and evidence across the handoff. If ArgoCD reconciles the cluster, avoid an independent pipeline continuously writing conflicting live state.

Delivery ownership	Use when	Avoid when
Jenkins plus ArgoCD	Build and Git-driven reconciliation have clear separate ownership	Jenkins also continually patches resources against GitOps intent
Managed delivery orchestration	Central approval/verification workflows justify the platform	Broad connector credentials span unrelated environments

Delivery ownership	Use when	Avoid when
Simpler single pipeline	Small deployment scope does not need extra control planes	Product count grows without reducing operational risk

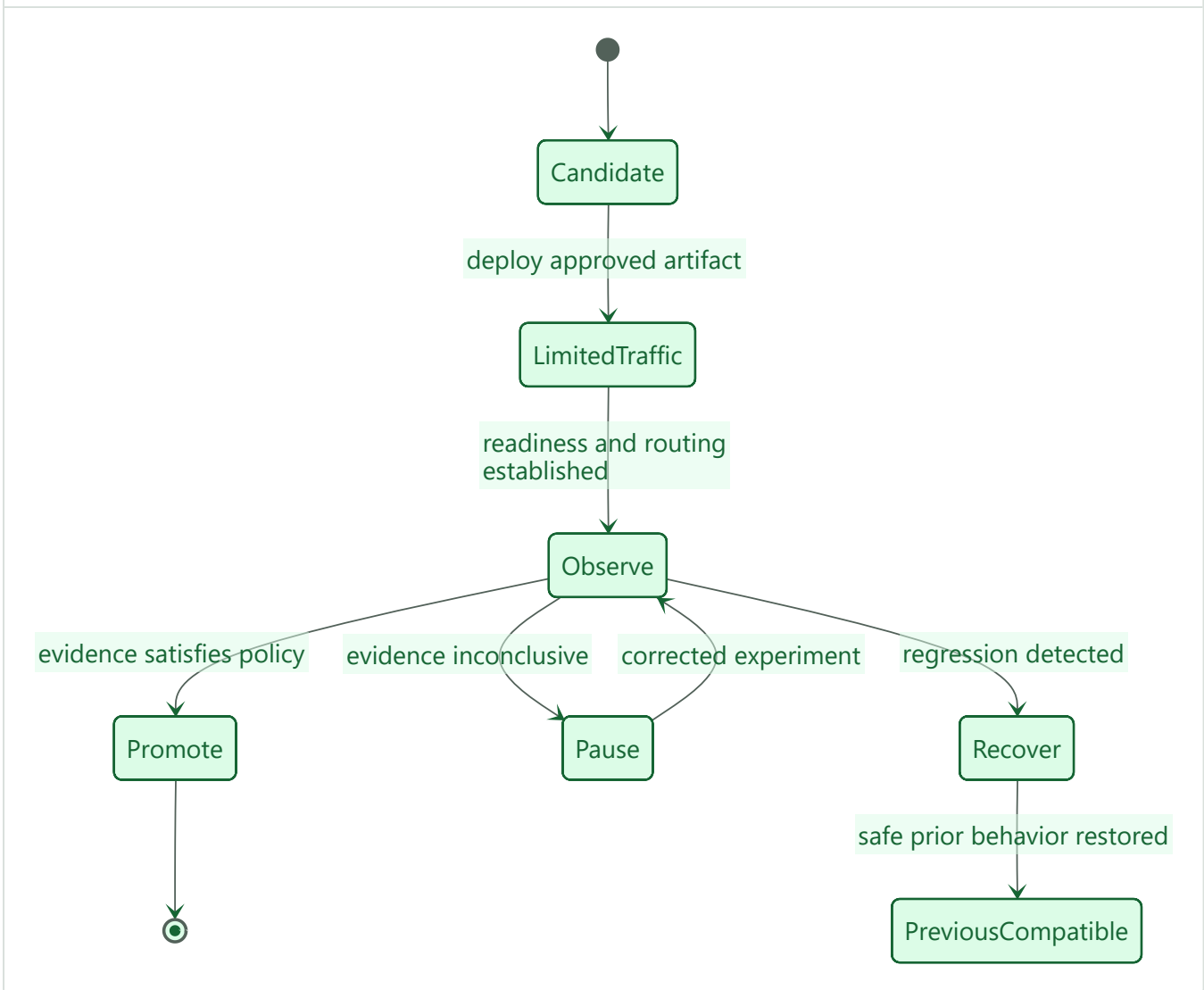
[VERIFY: confirm ArgoCD sync/prune/health/rollback behavior and Harness product/delegate/connector/verification capabilities against deployed versions and policies. No GitOps controller or Harness pipeline was contacted or executed.]

Interview checks: basic: compare responsibilities, not feature lists. Internals: handoffs must preserve artifact identity. Debug: look for two writers of desired state. Scenario: choose one clear deployment authority per resource.

6. Rolling, Blue-Green and Canary

Rolling updates gradually replace replicas and usually need old/new compatibility against shared dependencies. Blue-green maintains two environments/sets and switches traffic under a routing policy, improving rollback options but requiring capacity and careful shared-state handling. Canary exposes a new version to a limited cohort or traffic share, compares meaningful signals and decides whether to continue.

Diagram 16.4



A native Deployment replica ratio is not a precise HTTP traffic-weight guarantee. Real canary routing may need a supported ingress/mesh/progressive-delivery controller and stable cohort rules. Low-volume canaries can produce inconclusive metrics; a few successes do not establish tail-latency safety. Compare error budgets, latency, correctness and tenant/provider-specific outcomes while accounting for baseline changes.

Blue-green switches do not instantly move existing connections. Long-lived SSE and worker leases require a drain/reconnect/recovery policy. Both environments may write one database, so schema compatibility remains necessary. A traffic reversal cannot erase new-version writes already committed.

Strategy	Use when	Avoid when
Rolling	Compatible versions and modest spare capacity fit	Every old/new overlap breaks a shared schema

Strategy	Use when	Avoid when
Blue-green	Fast routing reversal and duplicate capacity are valuable	Shared data/configuration makes the old environment incompatible
Canary	Representative traffic and measurable decision criteria exist	Traffic split is assumed exact or evidence is statistically meaningless

[M] MECHANISM

A rollout changes exposure over time. Scheduling, readiness, routing and existing connections have different delays. Record which revision handled a request so an observed regression can be attributed to the correct cohort.

Interview checks: basic: deploy and traffic switch are separate. Internals: replica count does not necessarily equal traffic weight. Debug: compare cohorts and existing connections. Scenario: design the recovery decision before sending canary traffic.

7. Feature Flags and LaunchDarkly

A flag changes behavior independently of artifact deployment. This supports progressive exposure and rapid containment when old/new paths coexist safely. The flag has an owner, default, targeting policy, lifecycle and removal plan. Long-lived unused flags multiply test combinations and make behavior difficult to reconstruct.

LaunchDarkly is an example of managed flag evaluation and targeting infrastructure. Client/server SDKs and data delivery modes have different security and freshness properties. A browser flag must not enforce authorization; the backend still rejects forbidden operations. Never expose server-side SDK credentials to the browser or send unnecessary sensitive attributes to a targeting service.

Define behavior when the flag service is unreachable or initialization is incomplete. A cached last-known value and a code default are different fallback states. Record relevant flag/configuration version with diagnostics where privacy permits. Avoid doing a network round trip on every request if the chosen SDK is designed for local evaluation, but verify its actual caching/update model.

Flag use	Use when	Avoid when
Gradual feature exposure	Both paths are compatible and observable	Disabled UI is treated as authorization

Flag use	Use when	Avoid when
Kill switch	A bounded reversible behavior can be stopped safely	Turning off a flag leaves data halfway through an incompatible transition
Temporary migration flag	Controlled old/new coexistence is needed	The flag has no owner, expiry or cleanup test

[VERIFY: validate LaunchDarkly/selected SDK evaluation, offline/default behavior, targeting data, credential scope and event/privacy settings for the actual environment. No SDK, flag service or cohort experiment ran.]

Interview checks: basic: feature release can differ from deployment. Internals: local/default/stale evaluation states differ. Debug: correlate flag version and artifact revision. Scenario: remove flags after the transition rather than retaining permanent accidental branches.

8. Rollback, Schema and Configuration

Rollback is restoring an acceptable behavior under current state, not rewinding time. A prior binary may no longer read the database after a destructive migration. A prior secret may be revoked. A prior flag value may invoke code that assumes fields no longer exist. Keep an explicit compatibility window across application, schema, messages and configuration.

Expand/contract migrations allow old/new versions to coexist: add compatible structure, dual-write/backfill where appropriate, switch readers and remove old structure only after evidence that it is no longer needed. Reversing traffic during the window is easier than reconstructing dropped data. 06 owns the database mechanics; the delivery pipeline must respect their ordering.

Approvals should bind to exact artifact/configuration intent and relevant current state. If another release changes the target after approval, the old approval may no longer be valid. Use a reviewed generation/revision or equivalent compare-and-set precondition so two promotions do not silently overwrite each other. Approval is not merely a reusable green button.

The PromotionLab models this with a current generation/schema, a synthetic artifact digest and evidence flags, and an approval matching that digest/generation. It rejects missing evidence, mutable tag-shaped identity, stale approval and an older artifact unable to read the current schema. Its booleans are placeholders for independently verified evidence; checking them does not validate a real signature, image or migration.

[!] **TRAP**

A previous image is not automatically a safe rollback. Compatibility depends on the current data, schema, secrets and behavior. A rollback that restarts successfully can still corrupt data or break consumers.

Interview checks: basic: rollback does not undo committed effects. Internals: approvals have a target revision. Debug: check schema/message compatibility before image reversal. Scenario: prefer a small forward repair when the old behavior is no longer compatible.

9. Supply Chain and Quality Gates

Protect source review, dependency resolution, build execution, artifact storage and deployment authority as one chain. Pin and verify inputs where appropriate, isolate untrusted builds, generate useful provenance/SBOM evidence and restrict signing/publishing rights. A digest proves identity; signatures and attestations are meaningful only under a defined trusted signer and verification policy.

Quality gates should correspond to risks: compilation, unit tests, integration/contract tests, static analysis, dependency/image policy, migration compatibility and deployment health. A scan's absence of findings is not proof of security. Thresholds and exceptions need owners and expiry, not blanket bypasses hidden in a shared library.

Keep release evidence retrievable: commit, build tool identity, resolved inputs, test outcomes, image digest, config revision and approval decision. Avoid collecting secrets or raw customer data in artifacts. Reproducible builds and provenance complement each other; neither alone proves business correctness.

[D] **DECISION**

A gate should either prevent a named failure or supply a decision-relevant signal. Add the check at the owning boundary and preserve its failure. Do not accumulate green badges whose scope nobody can explain.

Interview checks: basic: identity, provenance and correctness differ. Internals: untrusted caches/build code can cross trust boundaries. Debug: trace an artifact back to exact evidence. Scenario: exceptions require a reviewed risk decision and expiry.

10. Agile, Kanban and Operational Feedback

Small changes are easier to review, test, observe and reverse. Agile practices should shorten the feedback loop from a user need to evidence, not merely increase ceremony. A definition of done for a backend change includes tests, compatibility, operational visibility and recovery implications appropriate to its risk.

Kanban makes work and bottlenecks visible and limits work in progress. If many changes are waiting for one environment or reviewer, starting more work can increase lead time without increasing delivery throughput. Distinguish active implementation from blocked, review, validation and production observation. Operational work and incident follow-ups should be visible, not permanently squeezed between feature tickets.

Use deployment/recovery metrics to identify system constraints, not rank individuals without context. Faster deployment frequency is not useful if each release consumes error budget or hides untested changes. Retrospectives and incident learning should change a test, design, runbook or decision rule rather than only produce a document.

Interview checks: basic: WIP limits reduce uncontrolled queues. Internals: throughput and lead time depend on the bottleneck. Debug: inspect blocked work, not only coding speed. Scenario: make operability part of done for a change that affects production.

11. Saved Artifacts and Evidence

`code/16-delivery/Jenkinsfile` is a complete small declarative teaching pipeline for the existing Java probe. It requires configured Jenkins SCM/plugins and a Linux JDK agent; those are unavailable. It does not build IntegrationHub-lite, publish images, mutate Git or deploy anything.

`code/16-delivery/PromotionLab.mjs` and its Windows PowerShell runner execute only local synthetic policy checks. From workspace root: `& ' ./docs/java-fs-guide/code/16-delivery/run.ps1'`. The actual result is PASS for four checks: valid reviewed intent promotes without mutating prior state; stale approval is rejected; missing evidence or a mutable tag is rejected; schema-incompatible rollback is rejected. The report explicitly records Jenkins NOT EXECUTED.

[VERIFY: execute the saved Jenkinsfile on an approved isolated controller/agent and replace synthetic promotion evidence with validated artifact, test, approval and schema records. The local JavaScript PASS is not pipeline syntax, build, signature, deployment or rollback evidence.]

[?] INTERVIEW

Trace one immutable artifact through the pipeline. Say which checks ran, who may publish it, how desired state references it, what observes its rollout and which compatibility facts permit recovery. That is a delivery design rather than a tool list.

Explained-Here Index and Sources

Concepts	Explanation
Git	Source and release identity
Maven and Gradle	Build models
Jenkins, shared libraries, agents, credentials	Trusted execution
ArgoCD and GitOps	Desired-state ownership
Harness	Orchestration choices
Rolling deployment, blue-green, canary	Exposure strategy
LaunchDarkly and feature flags	Behavior release
Rollback	Compatibility
Supply chain	Evidence and gates
Agile and Kanban	Feedback and WIP

Verification destinations: [Maven lifecycle](#), [Gradle user manual](#), [Jenkins Pipeline](#), [ArgoCD](#), [Harness](#) and [LaunchDarkly](#). These are reference destinations, not claims of fresh installed-system validation.

Related Chapters

Use [00](#), [03](#) Spring, [06](#) migrations, [08](#) security, [10](#) resilience, [13](#) integration, [14](#) images, [15](#) orchestration, [17](#) cloud, [18](#) operations, [19](#) quality and [25](#) architecture. Generated links are reciprocal.

RELATED CHAPTERS / RECIPROCAL LINKS

[00](#) The System Before the Details

[01](#) Core Java and the JVM

[03](#) Spring and Spring Boot Internals

[06](#) Databases and Durable State

[25](#) Architecture Styles and Domain Design

[13](#) Data Integration and iPaaS

[14](#) Docker and Container Internals

[15](#) Kubernetes and Production Workloads

[08](#) Security, Identity and Data Protection

[18](#) Observability and Reliability

19	Testing and Quality
10	System Design Foundations
17	Cloud and Infrastructure

24	IntegrationHub in Motion
20	Appendix: Interview Toolkit and Index

One-Page Cheat Sheet

Identity: reviewed commit, controlled build inputs, immutable artifact, deployment revision, schema and flag state all matter. Build once and promote the tested bytes.

Build: Maven phases bind plugin goals; Gradle runs a configured task graph. install/deploy in Maven do not mean application rollout. Pin toolchains and verify dependencies/caches. Offline means inputs already exist.

Trust: agents execute code; controllers and shared libraries hold powerful authority. Masking is not secret containment. Keep untrusted builds away from release credentials and trusted caches.

Delivery: GitOps compares desired/observed resources, not business truth. Define sync/prune/field ownership. Rolling, blue-green and canary trade capacity and exposure risk; route changes do not undo data writes.

Recovery: flags alter behavior separately from deployment and need defaults/owners/cleanup. Old artifacts must remain compatible with current schema/secrets. Bind approval to exact intent and current revision.

Evidence: four synthetic promotion checks passed. Jenkins, Java, Maven/Gradle, registry, GitOps and deployment remain NOT EXECUTED. Small changes and WIP limits improve feedback only when quality and operability are part of done.

Interview Corner

Basic: CI Versus CD?

CI integrates and validates changes; delivery/deployment moves approved artifacts toward or into environments under policy. A build passing does not establish rollout or business success.

Internals: Why Build Once and Promote?

It binds test evidence to the same bytes deployed later. Rebuilding per environment can introduce dependency/toolchain differences even from one source commit.

Trace/Debug: Maven Package Passed but Integration Is Broken.

Inspect which goals/tests were actually bound and executed. Package success is not evidence for integration gates that were skipped or never configured.

Scenario: How Do You Protect Jenkins Credentials?

Separate trusted publication from untrusted change builds, use scoped short-lived identity where possible, isolate agents and keep secrets out of arguments/artifacts/logs. Masking alone cannot contain malicious code.

Basic: ArgoCD Synced Means Healthy?

No. Sync means desired and observed resources match under comparison rules. Resource health and user-visible behavior are separate signals.

Internals: Why Did Manual Rollback Get Undone?

Desired Git state still required the newer revision and reconciliation restored it. Update the authoritative desired state or follow a controlled emergency override/reconciliation process.

Trace/Debug: Canary Looks Fine but Full Rollout Fails.

The canary may have unrepresentative traffic, insufficient duration, different tenants or inadequate capacity signals. Compare cohorts and the bottleneck, not just a few successful responses.

Scenario: Can You Roll Back After Dropping a Column?

Only if the chosen prior artifact is compatible with the current schema or a safe data recovery plan exists. Prefer expand/contract and retain a tested compatibility window.

Basic: Flag Off Equals Rollback?

It changes one behavior path, not necessarily the artifact, schema or side effects. Define safe defaults and whether the old path still works with current data.

Internals: Why Bind Approval to a Generation?

The environment may change after review. A stale approval must not silently overwrite a newer intent or promote against different compatibility assumptions.

Trace/Debug: Dependency Cache Made a Build Untrustworthy.

Inspect cache writers, artifact verification and resolved coordinates. Reused bytes from untrusted builds can cross a supply-chain boundary even when source review was sound.

Scenario: What Should a Delivery Retrospective Change?

A concrete gate, test, design, runbook or policy tied to the observed failure. A document alone does not improve the next release's safety.

17 / Cloud and Infrastructure

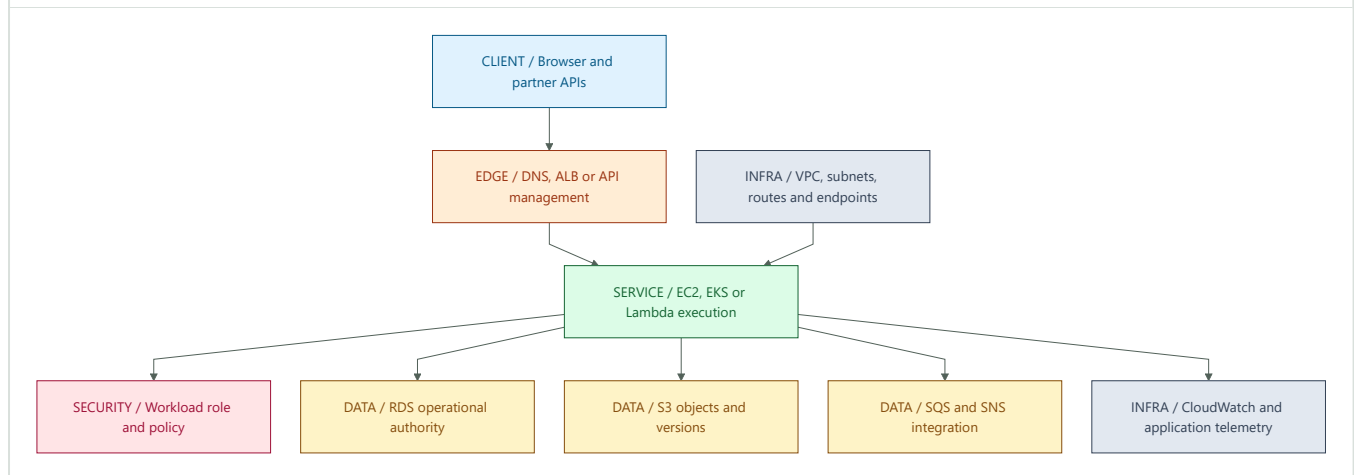
Managed infrastructure changes who operates a component; it does not remove your application contract. IntegrationHub is fictional. AWS can operate control planes and storage services while the team still owns authorization, data classification, quotas, retry semantics, recovery objectives and cost. A cloud service name is not a substitute for those decisions.

Version assumptions: Java 21, Spring Boot 4.0.x / Cloud 2025.1.x, PostgreSQL 17, Kafka 4.x and Kubernetes 1.34 remain illustrative baselines. AWS features, regional availability and quotas change independently of those versions. The saved Terraform JSON uses a declared Terraform 1.5+ range and AWS provider 5.x constraint as an unexecuted example, not a current-patch recommendation.

Execution status: NOT EXECUTED. Terraform and AWS CLI are unavailable. Five local JSON/declaration checks passed, but no Terraform provider was installed, no plan or apply ran, and no AWS credentials/account/API were accessed. The examples create no billable resources in this session.

Big Picture

Diagram 17.1



What You Will Be Able to Explain

- Separate region/AZ, account, network, workload and data ownership boundaries.
- Trace VPC routing, security groups, NACLs, NAT and private service access.
- Choose EC2, EKS or Lambda from runtime, operations and workload constraints.
- Explain S3 object/version behavior and RDS availability/read-replica/backup distinctions.

- Use IAM roles and temporary credentials without confusing network access with authorization.
- Compare SQS and SNS, ALB and API management, and reason about duplicates and deadlines.
- Explain Terraform plan/state/provider behavior and the difference between local file checks and actual infrastructure validation.

[P] PRODUCTION

Where this lives in IntegrationHub. 15's [Kubernetes workloads](#) may run on EKS; RDS can host 06's [PostgreSQL authority](#); S3 can hold governed import/export objects; queue/notification services complement 07's [delivery design](#). 08 owns tenant and key policy, 16 owns infrastructure change control, and 18 owns observed reliability and cost signals.

1. Accounts, Regions, Availability Zones and Responsibility

An account is an important administrative/billing/permission boundary. Regions provide geographically distinct service deployments; Availability Zones provide separated infrastructure within a region under each service's design. Multi-AZ improves resilience to particular infrastructure failures, but does not automatically provide cross-region recovery or protect against a mistaken authorized delete.

Shared responsibility varies by service. On EC2, the customer commonly manages guest OS patching and application runtime in addition to data and identity. A managed database shifts more engine/infrastructure operations to the provider, but schema, access, capacity, backups/retention choices and application behavior remain customer responsibilities. Serverless shifts execution management again, not business correctness.

Choose account/environment separation and least-privilege roles before adding connectivity. Development automation should not inherit production permissions merely because both environments use the same repository. Region selection must account for data residency, latency, service availability, quotas and recovery strategy, not only a nearby map location.

Boundary choice	Use when	Avoid when
Separate accounts/environments	Administrative blast radius and audit separation matter	One universal deploy identity controls every environment
Multi-AZ design	Zone-level failures should be tolerated	It is called a complete multi-region DR plan

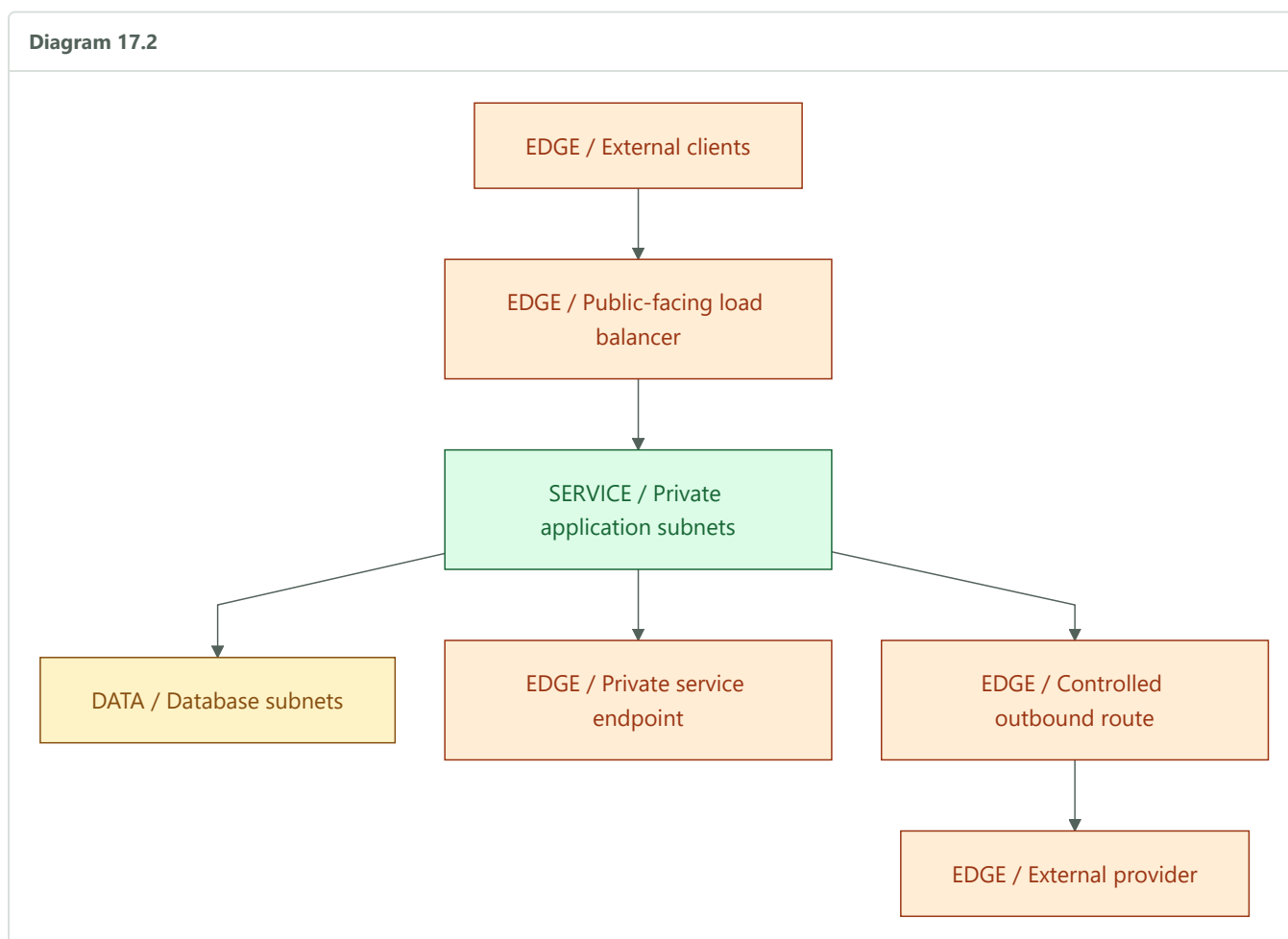
Boundary choice	Use when	Avoid when
Managed service	Provider operations reduce total burden	Application/data responsibility is assumed to disappear

Interview checks: basic: AZ and region differ. Internals: responsibility changes per service. Debug: identify which account/region actually owns a resource. Scenario: recovery requirements determine topology before product selection.

2. VPC Networking and Reachability

A VPC defines a logical network boundary with subnets, routes and controls. A subnet is associated with an AZ and routing configuration. A route to an internet gateway is one ingredient of public access, not proof that every instance in the subnet is reachable: addresses, security policy, listener and return path still matter. Private subnets may reach external services through NAT or reach supported AWS services through endpoints/private connectivity.

Diagram 17.2



Security groups provide stateful filtering for supported resources; network ACLs are subnet-level stateless controls whose return paths also need consideration. Routing sends packets toward a destination; policy decides whether they are allowed. DNS resolution is a separate step. A private endpoint may change both routing and DNS behavior, and its endpoint policy can add another authorization layer.

NAT supports outbound connectivity patterns but is not application authentication or a complete exfiltration defense. Centralizing NAT can create cost and availability dependencies; distribute or use private endpoints according to workload and service support. Do not assume all traffic to an AWS service is automatically private or free merely because both endpoints are in AWS.

For a connection failure, trace name resolution, route, security controls, port listener, TLS identity and IAM/application policy in order. A timeout and an access-denied response point at different layers. Avoid opening all ingress to debug a permissions problem.

[VERIFY: confirm AWS regional/AZ availability, VPC routing, security-group/NACL/endpoint behavior, quotas and cost/availability implications for the selected services. No AWS network or account configuration was inspected or changed.]

Interview checks: basic: a route is not permission. Internals: stateful and stateless return-path behavior differs. Debug: separate timeout from authorization denial. Scenario: private reachability still needs least-privilege identity and egress policy.

3. EC2 and EKS

EC2 supplies virtual compute with a selected instance type, image, storage/network configuration and identity. The team chooses capacity and owns the guest/application lifecycle appropriate to the service. Auto Scaling can replace or add instances according to policy, but it does not make stateful work safe to repeat or automatically fix a saturated database.

EBS-style block volumes are not interchangeable with S3 objects; their attachment, AZ, performance and recovery properties matter. Instance-local storage can disappear with instance lifecycle events. Externalize durable authority and test snapshots/restores under application consistency requirements. A volume persisting longer than an instance is not by itself protection against logical corruption.

EKS operates a managed Kubernetes control-plane service, while worker execution choices, add-ons, identity integration, upgrades, workload policy and application reliability still need ownership. Managed node groups and serverless worker options shift some responsibilities differently. The Kubernetes mechanisms from [15](#) still apply; EKS does not remove probes, resource budgets or CNI/CSI understanding.

Compute option	Use when	Avoid when
EC2	OS/runtime control and predictable long-lived processes are needed	Guest patching and capacity operations have no owner
EKS	Kubernetes ecosystem/operational consistency justifies it	A small workload gains unnecessary cluster complexity
Managed function execution	Bounded event/request work fits supported limits	Long-running or specialized runtime behavior conflicts with the service model

Interview checks: basic: managed control plane is not managed application. Internals: worker/storage choices change responsibilities. Debug: inspect shared dependencies before adding nodes. Scenario: choose the simplest execution model that meets lifecycle and control requirements.

4. S3 and RDS

S3 stores objects under keys in buckets; prefixes are not ordinary filesystem directories. An object version can identify immutable input for an import job, while overwriting a key without recording its version can change what a restart reads. Versioning helps recover some overwrite/delete scenarios but is not an unlimited backup guarantee: lifecycle, permissions, key availability and independent recovery still matter.

Do not confuse documented object-operation consistency with asynchronously delivered notifications or cross-region replication. A newly stored object can be available while its event consumer has not processed a notification. Design event handling for duplicate/reordered delivery where applicable, and reconcile from authoritative object/version identity. Presigned URLs delegate bounded access under their policy; they can leak through logs or forwarding and need appropriate lifetime and scope.

RDS manages database infrastructure/operations under engine/deployment choices. Multi-AZ and read replicas are different concepts, and specific Multi-AZ DB-instance versus DB-cluster offerings have different standby/read behavior. Do not assert that every Multi-AZ deployment has readable standbys or that every replica is synchronous. Ask which engine and topology are actually configured.

Read-after-write may require the writer or a supported consistency mechanism rather than an asynchronous read replica. Failover changes connections/endpoints under service behavior, so clients need bounded reconnect and transaction retry semantics without duplicating external effects. Total connection budgets must include all application replicas, workers and maintenance jobs.

Storage choice	Use when	Avoid when
S3 object/version	Files, exports and immutable input identities fit object semantics	Random filesystem operations or unversioned restart identity are assumed
RDS writer	Relational invariants need an operational authority	Managed hosting is treated as a replacement for schema/index design
Read replica	Reads tolerate the actual lag/consistency	Immediate read-after-write is promised without a supporting mechanism
Backup/PITR	Recovery from logical damage is required	Replication alone is called backup

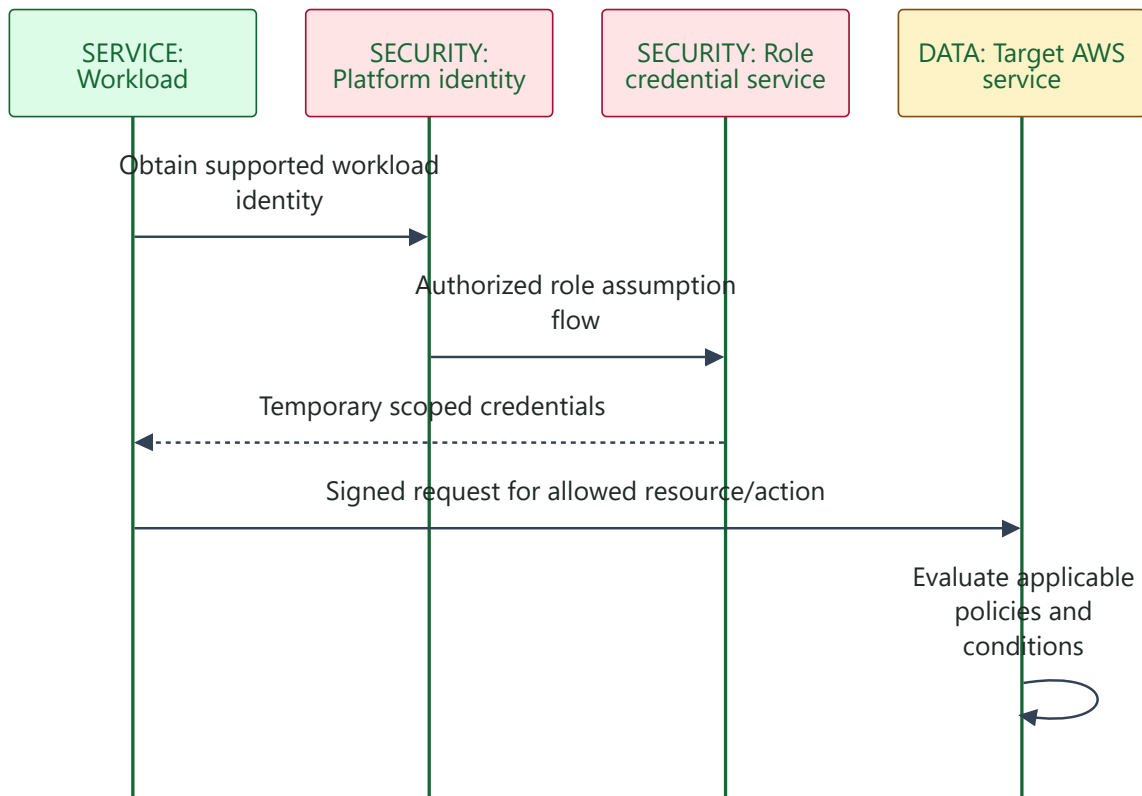
[VERIFY: confirm S3 consistency, versioning, notification and replication behavior and RDS engine-specific Multi-AZ, replica, failover, backup/PITR and connection semantics for the selected deployment. No storage, failover or restore operation ran.]

Interview checks: basic: object, block and relational storage differ. Internals: availability copies and read replicas are not one feature. Debug: inspect object versions and writer/reader endpoints. Scenario: restore testing belongs in the design, not after an incident.

5. IAM and Workload Identity

IAM controls access through principals, actions, resources and conditions under AWS policy evaluation. Roles let workloads obtain temporary credentials through supported trust relationships instead of embedding long-lived access keys. Trust policy determines who may assume a role; permission policy determines what the assumed role may do. Those are distinct decisions.

Diagram 17.3



Policy evaluation can involve identity/resource policies, session restrictions, permission boundaries and organization controls. Explicit deny is powerful, while boundaries and service-control policies constrain rather than independently grant permission. Avoid reducing all cases to one simplistic union/intersection rule; principal type, account relationship and service policy matter.

EC2 instance profiles, EKS workload-identity integrations and Lambda execution roles supply different acquisition paths. Scope each role to required resources/actions, including environment and tenant-relevant data boundaries where enforceable. A node role with broad permission is not automatically an appropriate role for every Pod. Application tenant authorization remains necessary even if a role can access a shared bucket or database.

Protect metadata/credential access and validate outbound destinations as [08](#) describes. Stronger metadata protocols and network policy reduce risk but do not make SSRF or overbroad IAM safe. Avoid logging credential values and account for temporary-credential renewal in long-lived clients.

[!] **TRAP**

Private subnet does not mean authorized workload. Network reachability, role trust, AWS permission and application tenant policy are separate layers. A private process with an overbroad role can still read or modify too much.

[VERIFY: verify IAM role trust, policy evaluation, permission boundaries/organization controls, EC2 metadata protections and EKS/Lambda workload-identity configuration against the actual account and service. No credentials or IAM APIs were accessed.]

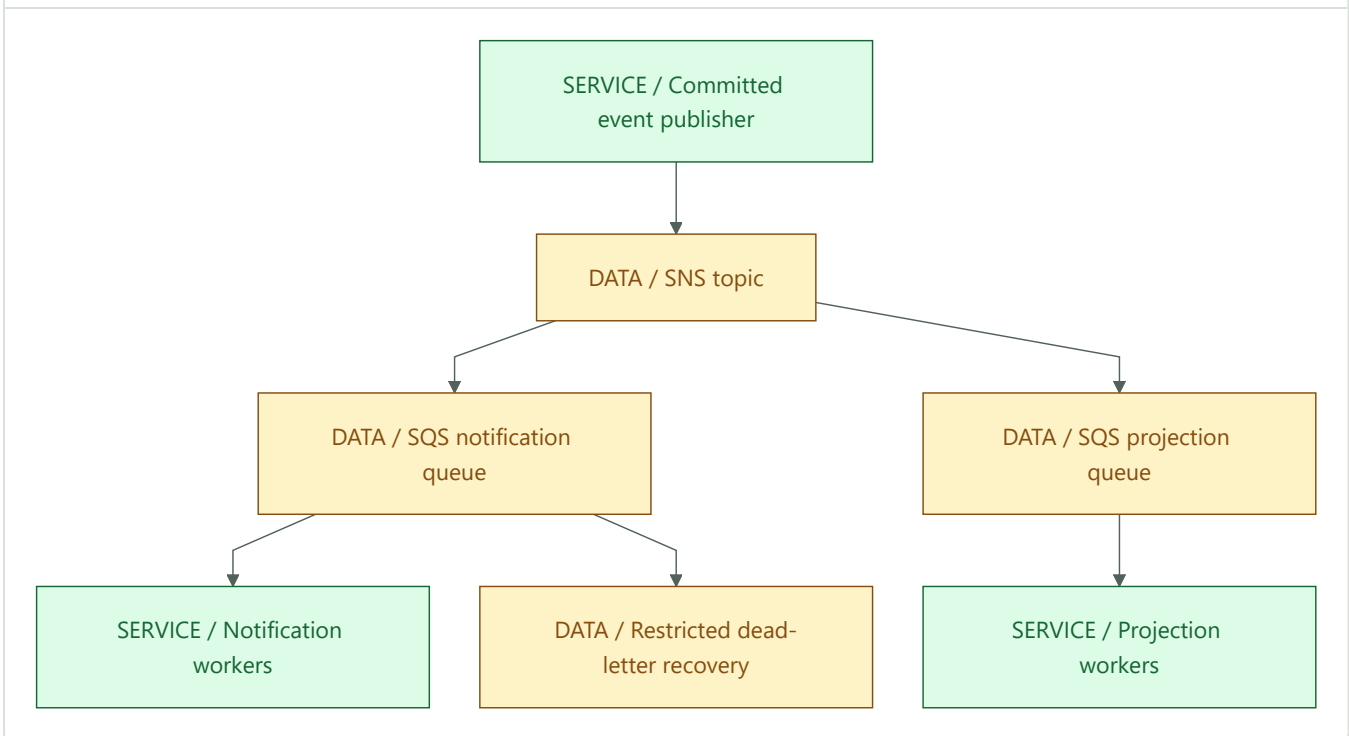
Interview checks: basic: trust and permission policies differ. Internals: temporary identity still needs least privilege. Debug: identify the effective principal and policy denial layer. Scenario: give workloads their own scoped roles rather than inheriting node-wide authority.

6. SQS and SNS

SQS provides queued work consumption; SNS distributes notifications to configured subscriptions. Together, SNS-to-SQS can fan an event into durable independent consumer queues. That is different from placing many consumers on one queue, where they compete for deliveries. Keep 07's [distinction](#) between work distribution and independent fan-out.

With a standard queue, design for duplicate delivery and the documented ordering behavior. Receiving a message makes it temporarily invisible under visibility timeout; successful processing normally deletes it. If processing exceeds visibility, another consumer can receive it. Extend visibility under a bounded ownership policy when appropriate, but do not assume invisibility physically prevents an old worker from acting.

Diagram 17.4



FIFO features provide ordering/deduplication under their documented group/window constraints, not unlimited exactly-once external effects. Stable business identities and transactional receipts are still needed. A DLQ needs an owner, age/volume alerts and controlled replay; queue transfer does not make failed business processing successful.

Lambda event-source mappings add batch and failure semantics. If one item fails, the configured response/retry policy determines what is retried; repeated already-applied items must remain safe. SNS endpoint delivery policies vary by subscription type. Do not copy Kafka offset semantics directly onto an SQS delete/visibility workflow.

[VERIFY: confirm SQS standard/FIFO ordering, deduplication windows, visibility limits, DLQ/redrive and SNS subscription delivery behavior, including Lambda partial-batch handling, for the selected configurations. No queue, notification or event-source mapping was created or tested.]

Interview checks: basic: queue consumption and topic fan-out differ. Internals: visibility timeout is not effect completion. Debug: inspect duplicate deliveries and partial-batch retries. Scenario: commit effect/receipt before deleting the message where that boundary is supported.

7. ALB and API Management

An Application Load Balancer routes supported application traffic to target groups under listener/rule and health policies. TLS termination, target registration, connection draining and idle timeouts affect behavior. A healthy target check does not authorize a user's resource request or prove a long-running operation completed.

API management adds capabilities such as API publication, stages, authentication integration, quotas/throttling, request policies and observability under the chosen service/product. AWS API Gateway is a particular product family; the architectural idea of an API gateway is broader. Choose features from client and governance requirements rather than stacking gateways without a clear responsibility split.

Long-lived SSE/WebSocket paths have service-specific protocol/duration/buffering support. Verify the exact endpoint type rather than assuming ordinary request routing applies unchanged. A usage-plan API key can identify a consumer or apply a plan, but is not automatically adequate user/resource authorization. Backend services retain domain policy.

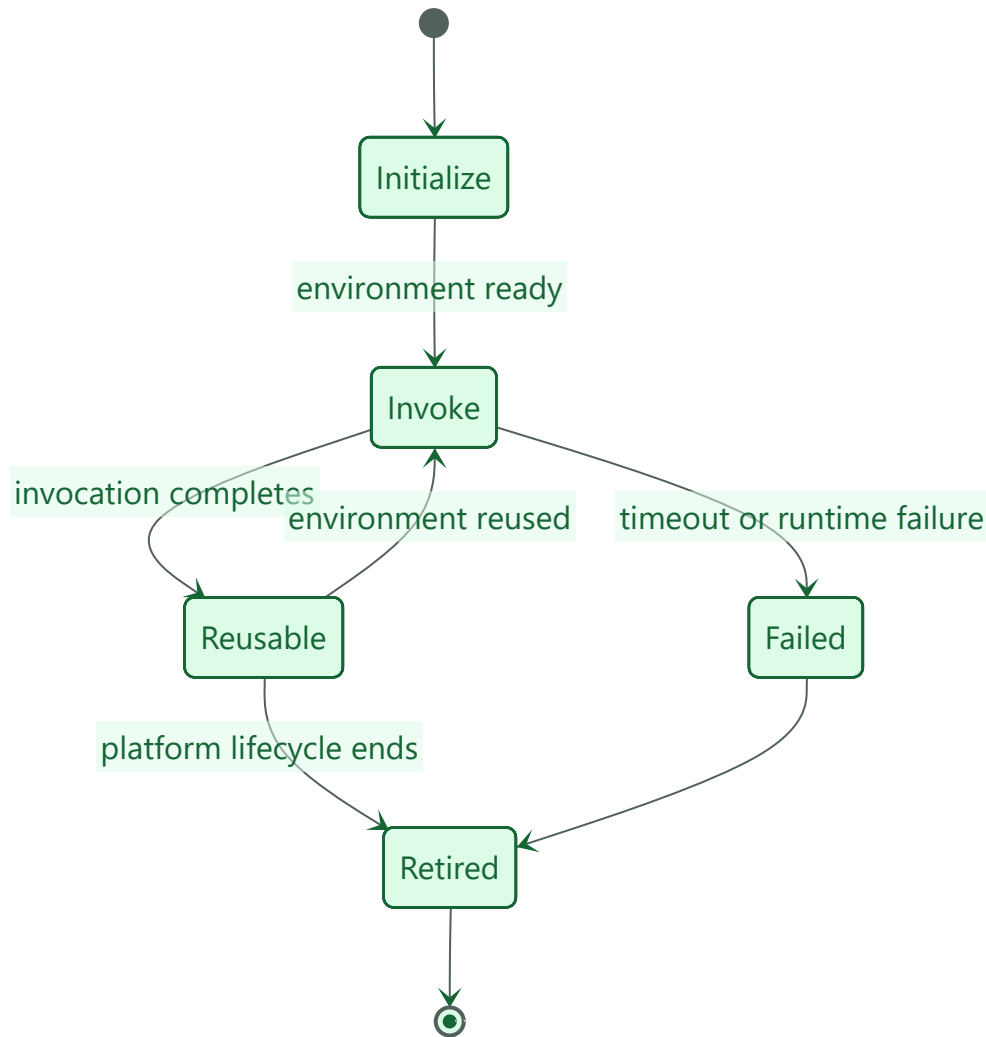
Edge option	Use when	Avoid when
ALB-style routing	HTTP-aware balancing and target health meet the need	Every domain workflow is implemented in listener rules
API management	Consumer/API policy and supported integrations justify it	Throttling or API keys are mistaken for complete authorization
Explicit streaming route	Protocol and connection limits are verified	A buffered/request-limited path is assumed to carry an indefinite stream

Interview checks: basic: load balancing and API governance differ. Internals: health and authorization are separate. Debug: inspect edge timeouts and buffering. Scenario: give each intermediary one clear policy role and deadline budget.

8. Lambda and Serverless Trade-Offs

Lambda executes functions in managed execution environments. A cold invocation can include initialization and runtime/application setup; a warm invocation may reuse an environment, but reuse is not guaranteed. Java initialization, dependency size and JIT behavior can affect latency. Do not promise a fixed cold-start penalty independent of runtime, configuration and workload.

Diagram 17.5



Environment reuse permits caching clients/configuration within their lifecycle, but mutable tenant data must not leak between invocations. Temporary filesystem contents are not a durable business store. Open connections and cached credentials need renewal/health policy; stale clients can fail after idle periods or failover.

Reserved concurrency and provisioned concurrency address different concerns: limiting/allocating concurrent execution versus preparing execution capacity under service semantics. Neither creates more database or provider capacity. Many concurrent functions can overwhelm an RDS connection budget, so use appropriate pooling/proxy patterns and admission controls with measured limits.

Invocation modes have different retry behavior. Synchronous caller retries, asynchronous delivery and event-source mapping retries must be designed separately. A function timeout does not prove the remote side effect did not happen. Idempotency and reconciliation remain necessary; returning an error may repeat input under the configured source policy.

Function execution	Use when	Avoid when
Bursty bounded event work	Managed scaling/lifecycle and supported limits fit	Work requires indefinite execution or unsupported process/runtime behavior
Prepared execution capacity	Measured startup latency justifies its cost	It is assumed to eliminate every latency source
Long-lived service on EC2/EKS	Persistent connections/control or steady workload fit	Functions are forced into a model requiring continual local state

[VERIFY: confirm Lambda runtime support, initialization/cold-start options, timeout/concurrency limits, VPC networking, temporary storage and retry/event-source semantics for the selected region/configuration. No function or latency benchmark was deployed or measured.]

Interview checks: basic: serverless still has execution limits. Internals: warm reuse is an optimization, not durable identity. Debug: correlate initialization, duration and downstream saturation. Scenario: use idempotent effects regardless of invocation retry mode.

9. CloudWatch, Cost and Recovery

CloudWatch provides monitoring/logging/alarm capabilities whose coverage depends on service configuration and instrumentation. Provider metrics such as instance CPU or queue age complement application SLIs; they do not automatically measure successful tenant jobs. OpenTelemetry and application correlation connect the cloud resources to the request journey in 18.

CloudTrail-style account API auditing and application request logs serve different purposes. An infrastructure change audit can show who altered a policy, while a business audit records who authorized a tenant export. Keep sensitive data out of both and scope access/retention. High-cardinality metrics, verbose logs and unbounded retention can create cost and privacy problems.

Cost is another resource budget. Include compute, idle headroom, storage versions/backups, data transfer, NAT/endpoints, requests and observability. Pricing varies by region/service and time; no price is assumed here. Budget alerts are not universal hard spending caps. Tag ownership and environment, set reviewed retention/lifecycle policy and make teardown responsibility explicit for sandbox resources.

Recovery requires both data and the control/identity path needed to use it. A backup in another region is insufficient if the needed key, role, configuration or connectivity is unavailable. Test restore/promotion with RPO/RTO evidence. Multi-region doubles some

operational surfaces and introduces authority/fencing questions from 10, not just another copy of a diagram.

[M] **MECHANISM**

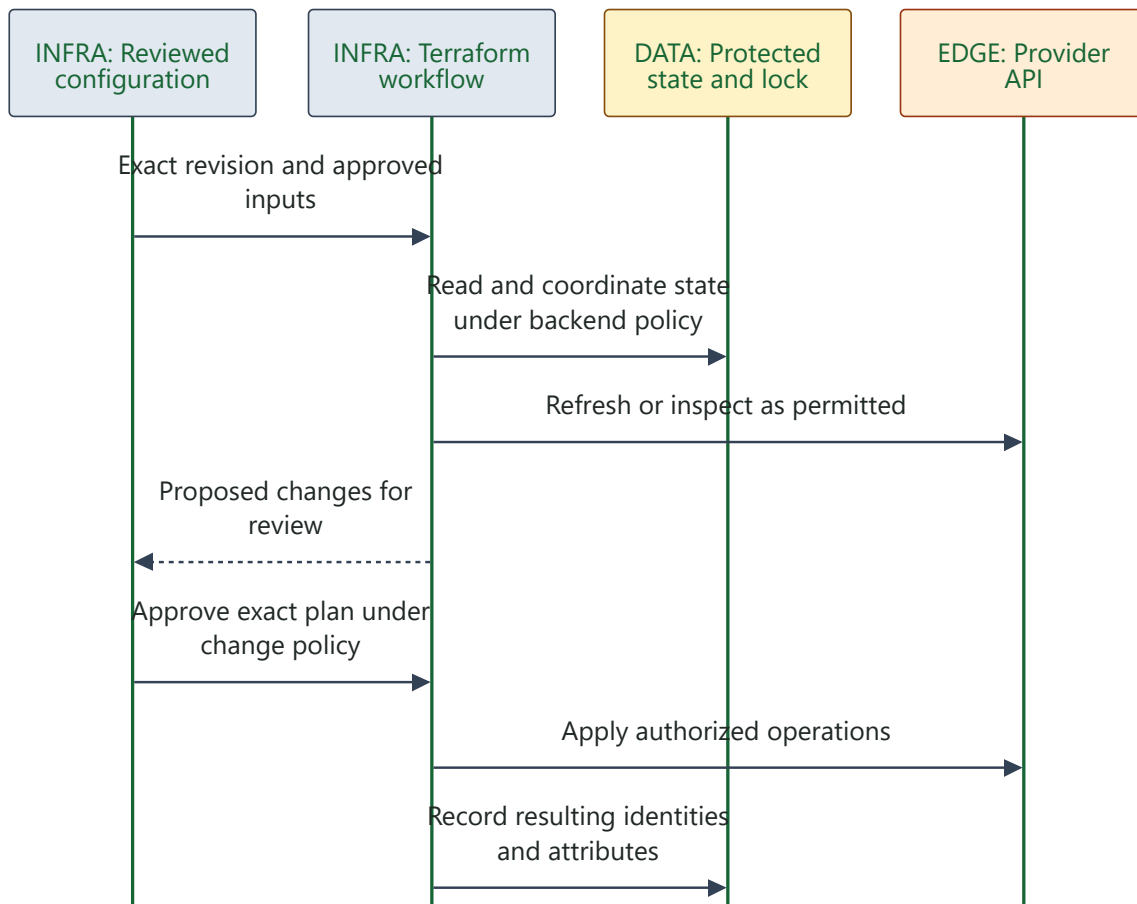
Managed service health and business health are different observations. A queue can be available while every consumer fails authorization; a database can be healthy while a bad query exhausts connection pools. Alert on the user's outcome and use provider signals to locate its cause.

Interview checks: basic: provider metrics do not define the business SLO. Internals: cost follows several resource dimensions. Debug: correlate identity/configuration changes with failures. Scenario: recovery rehearsals must include keys, roles and network paths.

10. Terraform, State and Reviewed Infrastructure Change

Terraform configuration declares resources and relationships, providers translate them to service APIs, and state maps declarations to observed resource identities and attributes. A plan compares intended changes with state/provider observations under the selected workflow. Applying changes is a separate authorized action. A plan is not a guarantee that conditions will remain unchanged until apply.

Diagram 17.6



Remote state needs encryption, access control, backups and supported concurrency/locking behavior. State can contain sensitive values even when output is marked sensitive; that flag is not encryption. Do not let unrelated pipelines concurrently own the same resources/state without coordination. Imported resources and manual drift need review, not blind state surgery.

Provider initialization can download plugins and contact configured backends. validate requires suitable initialization/provider schemas for many configurations; parsing JSON alone is much narrower. Pin provider constraints and record a reviewed dependency lock where generated appropriately. Do not invent a lockfile checksum without actually resolving the provider.

The saved `code/17-cloud/main.tf.json` declares a bucket with public-access blocks, ownership controls, versioning and S3-managed AES256 storage encryption. It requires an explicitly selected bucket name and does not embed credentials. `force_destroy=false` and `prevent_destroy=true` are Terraform guardrails under their configuration semantics, not protection against every console/API deletion or removal of the declaration itself. Real retention and access policy are separate controls.

laC practice	Use when	Avoid when
Protected remote state and locking	Multiple operators need coordinated ownership	State files or secrets are committed casually to source control
Reviewed saved plan and revision	Approval should bind to exact changes	Approval is reused after configuration or target state changed
Deletion guardrails	Accidental destructive changes should be caught	They are mistaken for complete backup or account-policy protection

Windows PowerShell from workspace root: `& './docs/java-fs-guide/code/17-cloud/run.ps1'`. Five local checks actually passed: JSON parse, deletion guards, public-access blocks, versioning and encryption declaration. The report remains NOT EXECUTED for Terraform syntax/provider validation, initialization, plan/apply and AWS operations. This is not a deployed private bucket or a compliance result.

[VERIFY: validate native Terraform JSON, provider 5.x resource schemas, state/backend locking, plan/apply identity and deletion/encryption/access semantics using approved installed tools and a sandbox account. No provider download, Terraform validation or AWS API operation ran.]

[D] DECISION

Prefer the smallest managed-service set with a clear owner for each remaining responsibility. EC2, EKS and Lambda are execution choices; S3, RDS and queues own different data contracts. Adding all of them does not by itself improve reliability.

[?] INTERVIEW

Trace one request and one failure across account, network, identity and data boundaries. State which service owns the fact, which permission allows the action, what happens after a timeout and how recovery is verified.

Explained-Here Index and Sources

Concepts	Explanation
AWS and shared responsibility	Boundaries
VPC	Routes and policy
EC2 and EKS	Compute

Concepts	Explanation
S3 and RDS	Storage
IAM	Workload identity
SQS and SNS	Message contracts
ALB and API management	Edge responsibilities
Lambda and cold start	Execution lifecycle
CloudWatch	Observability and cost
Terraform	State and change

Verification destinations: [AWS documentation](#), [AWS shared responsibility](#), [IAM policy evaluation](#), [Lambda documentation](#), [Terraform JSON syntax](#) and [AWS provider](#). These are pointers for validation, not claims of fresh retrieval or account inspection.

Related Chapters

Return to [00](#). Connect [06](#) databases, [07](#) messaging, [08](#) identity, [10](#) system design, [12](#) HLD, [13](#) integration, [14](#) containers, [15](#) Kubernetes, [16](#) delivery, [18](#) reliability, [21](#) networking and [22](#) distribution. Generated links are reciprocal.

RELATED CHAPTERS / RECIPROCAL LINKS

00 The System Before the Details	18 Observability and Reliability
06 Databases and Durable State	10 System Design Foundations
05 APIs and Real-Time Communication	12 Twelve System Design Case Studies
08 Security, Identity and Data Protection	13 Data Integration and iPaaS
14 Docker and Container Internals	21 Networking and Operating Systems
15 Kubernetes and Production Workloads	22 Distributed Systems Theory
16 Build, Ship and Roll Back	24 IntegrationHub in Motion
07 Messaging, Batch and Streams	20 Appendix: Interview Toolkit and Index

One-Page Cheat Sheet

Boundaries: account, region and AZ serve different purposes. Managed infrastructure changes responsibilities, not application invariants. Multi-AZ is not every form of disaster recovery.

Network/identity: route, security group/NACL, DNS, TLS, IAM and tenant policy are distinct. Private connectivity does not grant permission. Prefer scoped temporary workload roles over embedded access keys.

Data: S3 object versions can stabilize input identity; notifications/replication have separate timing. RDS topology determines standby/read/failover behavior. Backups and replicas solve different failures.

Execution/messages: EC2/EKS provide control with operational obligations; Lambda fits bounded work with explicit limits and retry modes. SQS queues work, SNS fans out; visibility/FIFO features do not remove external duplicate effects.

Operations/IaC: CloudWatch metrics complement business SLIs. Budget alerts are not universal spending caps. Terraform state is sensitive ownership metadata, plan is not apply, and local JSON checks are not provider validation. No AWS resource was created or accessed here.

Interview Corner

Basic: Region Versus Availability Zone?

Regions provide geographically distinct service deployments; AZs separate infrastructure within a region. Select topology from failure and recovery requirements rather than treating the terms as interchangeable.

Internals: What Makes a Subnet Public?

Routing toward an internet gateway is an important property, but actual reachability also needs addresses, security controls and a listening service. A route alone does not expose every resource.

Trace/Debug: An AWS Call Times Out Versus Access Denied.

A timeout suggests a network/endpoint/service path to investigate; access denied points toward effective identity/policy. Trace DNS/routes/security/TLS and principal permissions separately before broadening access.

Scenario: EC2, EKS or Lambda?

Choose the required runtime control, duration/connection model, scaling shape and operational ownership. Do not adopt a cluster for a small function or force an indefinite worker into a bounded invocation contract.

Basic: RDS Multi-AZ Equals Read Replica?

No. Behavior depends on engine and deployment type, including DB-instance versus DB-cluster offerings. Verify which instances accept reads and what replication/failover guarantees apply.

Internals: Why Do SQS Consumers Still Need Idempotency?

Messages can be redelivered, visibility can expire and a consumer can commit before delete succeeds. Protect the effect at its authority instead of assuming queue receipt equals completion.

Trace/Debug: A Function Times Out but the Provider Charged.

The remote effect may have committed before local timeout. Use stable provider identity and reconciliation; a failed invocation is not proof of no external effect.

Scenario: How Would You Give a Pod AWS Access?

Use the cluster's supported workload-role integration with scoped trust and resource permissions. Do not inherit a broad node role or embed long-lived keys; retain tenant authorization in the application.

Basic: Terraform State Versus Configuration?

Configuration declares desired resources; state tracks resource identities/attributes used by the workflow. Both need controlled ownership, and state can contain sensitive values.

Internals: Does Sensitive Output Encrypt State?

No. It controls presentation in supported contexts. Backend encryption/access policy and secret minimization are separate responsibilities.

Trace/Debug: An IaC Plan Passed but Apply Failed.

The target, permissions, quotas or provider conditions may have changed, or an operation failed during execution. Inspect partial results and state under the approved workflow; do not blindly repeat destructive actions.

Scenario: What Must a Cloud DR Rehearsal Include?

Data restore/promotion plus keys, roles, configuration, network paths, application compatibility and stale-writer prevention. Measure RPO/RTO rather than assuming a remote copy is enough.

18 / Observability, Reliability and Operations

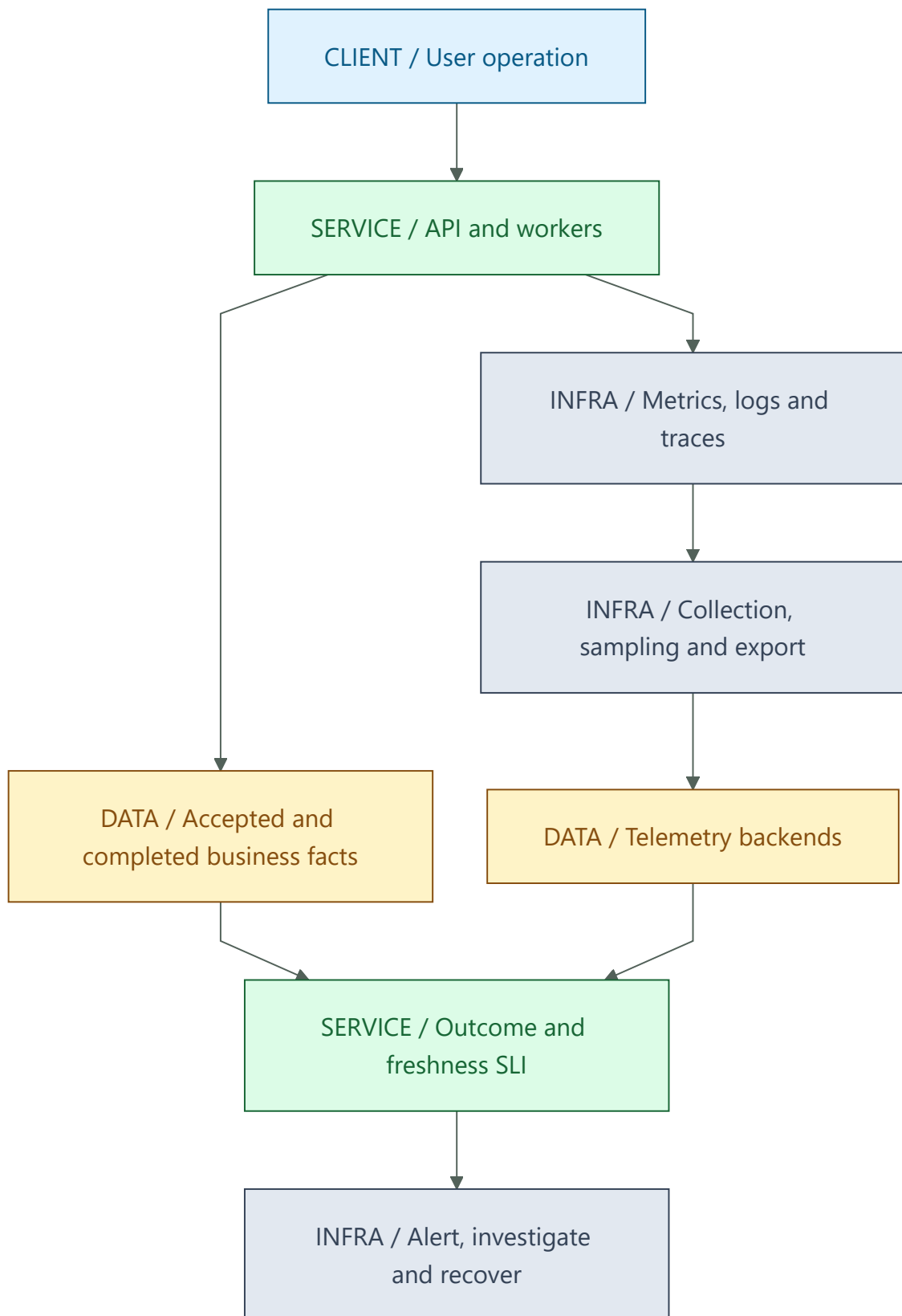
An operational signal is useful when it changes a decision. IntegrationHub is fictional. Its API can be healthy while accepted jobs never finish, and its consumer offset can advance while work waits in an unbounded application queue. Observe the user's outcome, then use infrastructure evidence to explain it.

Version assumptions: Java 21, Spring Boot 4.0.x / Cloud 2025.1.x, PostgreSQL 17, Kafka 4.x and Kubernetes 1.34 remain the guide baseline. Prometheus, Grafana, Loki and OpenTelemetry versions/export protocols need explicit selection. The existing Boot/Cloud family check is retained; no telemetry deployment is installed here.

Execution evidence: SloLab.mjs passed five synthetic arithmetic checks on the existing Node-compatible runtime. No production metrics, logs, traces, load test, chaos experiment, restore or JVM profile was collected. Calculations below are illustrative inputs, not measured IntegrationHub behavior.

Big Picture

Diagram 18.1



What You Will Be Able to Explain

- Design logs, metrics and traces around questions rather than collecting everything.
- Trace OpenTelemetry context and collector behavior across HTTP and asynchronous work.
- Explain Prometheus counters/histograms, label cardinality, Grafana views and Loki label design.
- Define request and asynchronous-work SLIs, SLOs, error budgets and actionable alerts.
- Run an evidence-driven incident response and write a usable runbook/postmortem.
- Diagnose high CPU, memory growth, blocked work, slow queries and consumer lag at their owning layer.
- Plan capacity, load/chaos experiments and disaster recovery without inventing measurements.

[P] PRODUCTION

Where this lives in IntegrationHub. 05's accepted job and 07's committed worker result define separate outcomes. 15 supplies process/platform evidence; 16 supplies release identity; 17 supplies managed-service signals. 23 owns deep CPU, allocation and heap analysis. Correlation crosses these boundaries without placing credentials or personal payloads in telemetry.

1. Logs, Metrics and Traces

Metrics aggregate numeric observations over time and dimensions; they are good for rates, distributions and alerting. Logs record discrete events and contextual details; they help explain individual failures. Traces connect timed operations across execution boundaries; they help locate where an observed request spent time. None alone proves every causal relationship, and collecting all three does not automatically make a system observable.

Start from an operational question. Is the service rejecting requests? Are accepted jobs waiting too long? Is one dependency slow? Which release introduced a change? Choose a signal that can answer it with bounded storage, cardinality and privacy cost. A structured failure code plus job correlation can be more useful than ten stack traces containing raw input.

Use consistent service/environment/version identifiers. Trace IDs can correlate restricted logs, but should not become a Prometheus label on every measurement. Request IDs and tenant IDs are often high-cardinality or sensitive. Logging a token or putting customer data in trace baggage creates additional copies beyond the application's normal data controls.

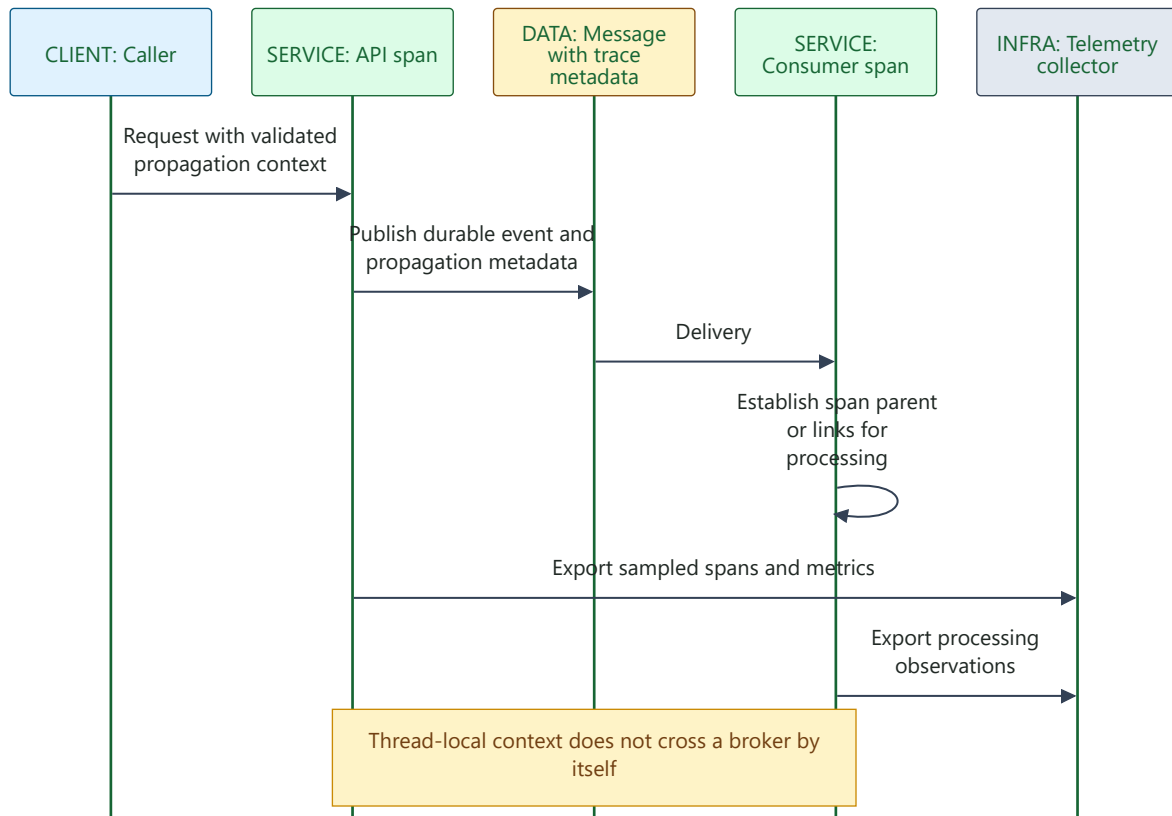
Signal	Use when	Avoid when
Metric	Aggregate rates/distributions support diagnosis and alerting	Every request ID creates a separate series
Structured log	A specific event needs safe context	Full payloads or credentials are dumped for convenience
Trace	Cross-service timing and parent/link relationships matter	Sampled traces are treated as a complete event ledger
Durable audit record	Accountable business/security decisions must be reconstructable	Best-effort diagnostic retention is assumed sufficient

Interview checks: basic: the signals answer different questions. Internals: sampling and aggregation discard information. Debug: identify the missing question before adding logs. Scenario: instrument accepted and completed work separately.

2. OpenTelemetry and Context Propagation

OpenTelemetry provides instrumentation, APIs/SDKs and a collection/export ecosystem. It is not itself the durable storage or visualization backend. An SDK creates observations under configured resource attributes, processors, sampling and exporters. A Collector can receive, batch, transform and export telemetry while enforcing resource/privacy policy. Its queue, retry and memory behavior are another operational system to monitor.

Diagram 18.2



HTTP propagation carries context through agreed headers. Async callbacks, executors, virtual threads and message consumers need correct context capture/restoration. A pooled worker must not retain the previous task's context. For batches or fan-in, span links may describe multiple causal inputs more accurately than arbitrarily selecting one parent. Trace context is correlation metadata, not authorization evidence.

Head sampling decides early whether to retain a trace; tail sampling can select after more of the trace is available, such as errors or high latency, at the cost of buffering, routing and completeness assumptions. A trace absent from storage is not proof that the request did not occur. Baggage travels widely and should contain only allowed bounded information, not tenant secrets or arbitrary client input.

Exporter failure must not silently become a business outage. Bound queues and memory, choose drop/retry policy, and monitor exporter/collector loss. Critical audit facts belong in an appropriate durable path rather than relying on best-effort span export. Instrumentation overhead also needs measurement; an allocation-heavy tracing configuration can alter the workload it observes.

[VERIFY: check OpenTelemetry SDK/agent, semantic conventions, propagation, sampling, Collector queue/retry behavior and Spring instrumentation compatibility for selected versions. No agent, collector or cross-process trace was executed.]

Interview checks: basic: OTel is not a storage backend. Internals: async propagation needs explicit supported context handling. Debug: distinguish missing instrumentation from sampled-out traces. Scenario: isolate telemetry export failure from normal business execution.

3. Prometheus Metrics and Cardinality

Counters accumulate events and can reset when a process restarts. Gauges represent current quantities such as queue depth. Histograms record distributions in a representation that supports selected aggregate calculations. Choose units and dimensions consistently; seconds and milliseconds in similarly named metrics are an expensive source of false conclusions.

Compute counter rates per original series before aggregation so resets remain detectable under the query function's semantics. Summing process counters and then taking a derivative can obscure restarts. Scrape gaps, stale series and missing targets need a policy; no observed errors is not the same as a healthy observable service.

Classic histograms expose cumulative bucket series plus count and sum. A quantile query estimates within bucket boundaries; it is not the exact raw sample percentile. Aggregate compatible bucket distributions before estimating a fleet percentile. Averaging per-instance p99 values does not produce the fleet p99. Native histogram support and query behavior differ by selected versions and configuration.

Assume ten routes, six outcome categories and five instances, all combinations present, with twelve classic bucket series including the infinity bucket. Buckets plus count and sum give 10 times 6 times 5 times 14 = 4,200 series for that metric family. This is an illustrative upper-combination count, not observed cardinality. Adding an unbounded user/job label can dominate it.

Metric design	Use when	Avoid when
Bounded route/outcome labels	Aggregate service behavior needs segmentation	Raw URLs with IDs create unbounded route labels
Histogram	Latency distributions need fleet aggregation	Bucket boundaries cannot resolve the relevant SLO threshold
Gauge	Current backlog or occupancy matters	A gauge is used as a monotonic event counter without reset semantics
Exemplars/correlation	Selected traces explain aggregate observations	Trace identity becomes a dimension of every series

[VERIFY: verify Prometheus rate/reset/staleness semantics, classic/native histogram support, query aggregation and scrape/export configuration against the selected stack. SloLab checks arithmetic only; no PromQL or Prometheus rule evaluation ran.]

Interview checks: basic: counters reset and gauges can fall. Internals: quantiles are not generally composable averages. Debug: inspect label combinations and missing scrape targets. Scenario: choose histogram resolution from the decision threshold.

4. Grafana, Loki and Useful Dashboards

Grafana organizes queries and visualizations over configured data sources; it is not automatically the source of metric truth. A dashboard should lead from user outcome to likely constrained resource: request success/latency, accepted-work age, completion rate, dependency failures, queue/pool occupancy and relevant platform state. Annotate releases and policy changes so timelines have operational meaning.

Loki indexes log labels under its storage design rather than making every log field an index dimension. Use low-cardinality labels such as service/environment where appropriate and parse or query selected structured content deliberately. Per-job labels can create excessive streams and operational cost. Redaction must happen before sensitive data reaches the shared backend where possible, not only in a dashboard view.

Use a common time range and understand clock skew, ingestion delay and query resolution. Two charts sampled at different windows can appear correlated or contradictory for purely technical reasons. A dashboard that shows averages alone may hide a small but important tail or tenant cohort; segmentation must remain bounded and authorized.

[VERIFY: confirm Grafana data-source/query behavior and Loki label, ingestion, retention and access policies for the deployed versions. No dashboard, log backend or telemetry retention policy was exercised.]

Interview checks: basic: visualization and storage are different roles. Internals: labels determine indexing cost. Debug: compare time windows and ingestion delays. Scenario: build a dashboard around decisions rather than every available metric.

5. SLI, SLO, SLA and Error Budgets

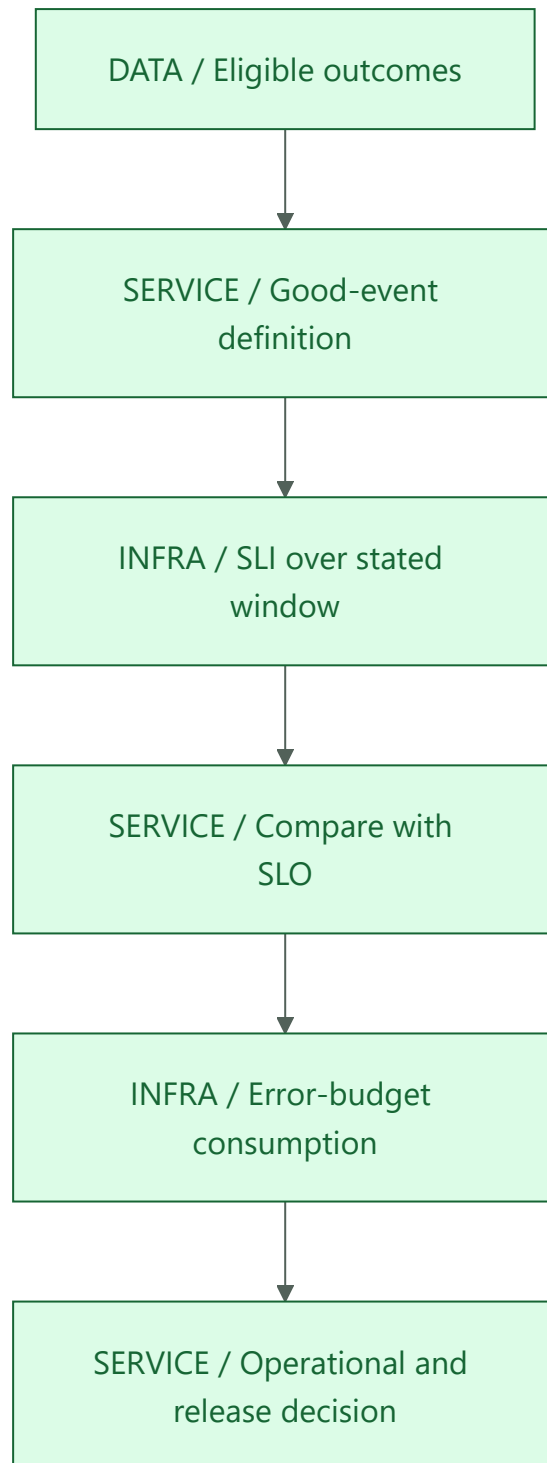
An SLI is a defined measurement of service behavior; an SLO is a target for it over a window; an SLA is an external agreement with its own definitions and consequences. They should be related but are not interchangeable. Specify eligible events, success criteria, window and data source before quoting a percentage.

For API availability, decide which requests count and where observation occurs. Server-side metrics can miss traffic rejected before the application. A client error caused by a documented invalid request differs from a valid request rejected because infrastructure failed; status class alone may not capture that distinction. Exclusions must be reviewed, not tuned to make the graph green.

For asynchronous jobs, measure accepted work that completes correctly within a defined duration, plus backlog age and unresolved outcomes. A fast 202 response does not satisfy the completion SLO. Define how cancelled, quarantined or partially completed jobs count. Reconcile the measurement against durable job records rather than treating a sampled trace as the ledger.

Assume a request SLO of 99.9% over a window containing one million eligible requests. The allowance is 1,000 bad events. If 2,000 are bad, the observed error fraction consumes allowance at two times the target rate for that window, and the window budget is exceeded by 1,000 events. These are synthetic arithmetic inputs. A request-based budget is not automatically convertible into a fixed number of downtime minutes unless its assumptions support that conversion.

Diagram 18.3



No traffic or missing data should not silently evaluate as perfect health. The lab returns unknown availability for a zero denominator. Alerting needs a separate telemetry/traffic health policy so that absence can be distinguished from a legitimately idle service.

[M] MECHANISM

Burn rate compares observed bad-event fraction with the allowed fraction. It describes how quickly budget is consumed under the chosen definition. It does not explain the root cause or prove that future traffic will behave the same way.

Interview checks: basic: measurement, target and agreement differ. Internals: denominator/exclusions decide meaning. Debug: inspect missing traffic and asynchronous completion. Scenario: define both acceptance and completion objectives for IntegrationHub.

6. Alerting and Incident Response

Page when timely human action can reduce material user impact. Route lower-urgency capacity or hygiene issues to a ticket/work queue. A CPU threshold without an action or user-risk link can generate noise, while a rapidly consuming error budget or stuck accepted-work queue can justify urgent intervention.

Multi-window burn alerts can require both a longer window showing sustained consumption and a shorter window showing the problem is still active. Choose windows/thresholds from the service's objective and operational response; copying a numeric threshold without its assumptions is not SLO engineering. Suppress duplicate symptoms under a known incident carefully, without hiding independent failures.

An alert should include service/operation, impact, observed condition, time window, environment/revision, a focused dashboard and a runbook owner. Do not include raw credentials or customer payloads. Test routing, silences, escalation and recovery notification under controlled conditions; an untested contact chain is part of the failure surface.

During an incident, establish an incident lead, communication channel, scribe/timeline and bounded hypotheses. Stabilize before deep optimization: stop a bad rollout, reduce harmful retries, shed noncritical load or isolate a failing tenant/provider when those actions are safe. Record commands, owners, timestamps and observed effects. Keep speculation separate from confirmed facts.

Response	Use when	Avoid when
Page	Immediate action can protect a meaningful objective	A low-value metric is noisy but nonactionable
Ticket	A trend needs planned remediation	A fast outage is deferred into an ordinary backlog
Mitigation	A reversible bounded action reduces impact	Several unrelated changes erase diagnostic evidence
Escalation	Authority or expertise is missing	One responder silently owns everything indefinitely

Interview checks: basic: pages require actionability. Internals: window choice affects detection/recovery delay. Debug: distinguish root signal from symptom fan-out. Scenario: assign roles and preserve evidence while restoring service.

7. Production Playbooks: Find the Owning Layer

Begin with scope and recent changes: affected operation, tenant/provider cohort, release, start time and whether demand changed. Compare healthy and unhealthy instances only when workloads/configuration are comparable. A single screenshot is rarely enough to establish a trend.

Symptom	First discriminating evidence	Next bounded action
High CPU	Process/container CPU, throttling, request volume and CPU profile	Separate demand, hot code, GC and quota pressure before tuning
Memory growth	Heap after collection, allocation rate, native/RSS and retained paths	Bound collection or capture approved evidence under disk/privacy limits
Blocked/slow requests	Pool wait, thread states, dependency latency and deadlines	Find the occupied resource and its owner before enlarging pools
Slow queries	Actual plan/rows, lock waits, transaction age and query volume	Test index/plan/transaction hypothesis on representative data
Consumer lag	Per-partition lag, oldest event age, processing time, retries and rebalance/restore state	Distinguish hot partition, slow sink and lost capacity
OOMKilled or restart loop	Platform termination reason, previous logs, probe and memory evidence	Correct the specific runtime or health policy, not only replica count

Thread dumps show sampled thread states and stacks, not an exact CPU accounting history. Several spaced observations can distinguish stable waiting from transient work. Heap dumps show retained object graphs and can contain credentials or personal data; capturing one can require substantial pause/disk and should be authorized. [23](#) owns deeper interpretation and tool commands.

Consumer lag can fall because offsets advanced into a local queue rather than because business work completed. Track the oldest accepted but incomplete job and committed progress as well. If completion rate equals arrival rate, an existing backlog cannot drain; more consumers help only if partition and downstream capacity permit it. A paused stateful stream restoring local state can be alive but not yet productive.

[!] **TRAP**

Do not destroy evidence as the first diagnostic step. Restarting every Pod or flushing every cache can hide the cause and trigger a recovery storm. Capture the relevant state, make one justified mitigation and watch its expected effect.

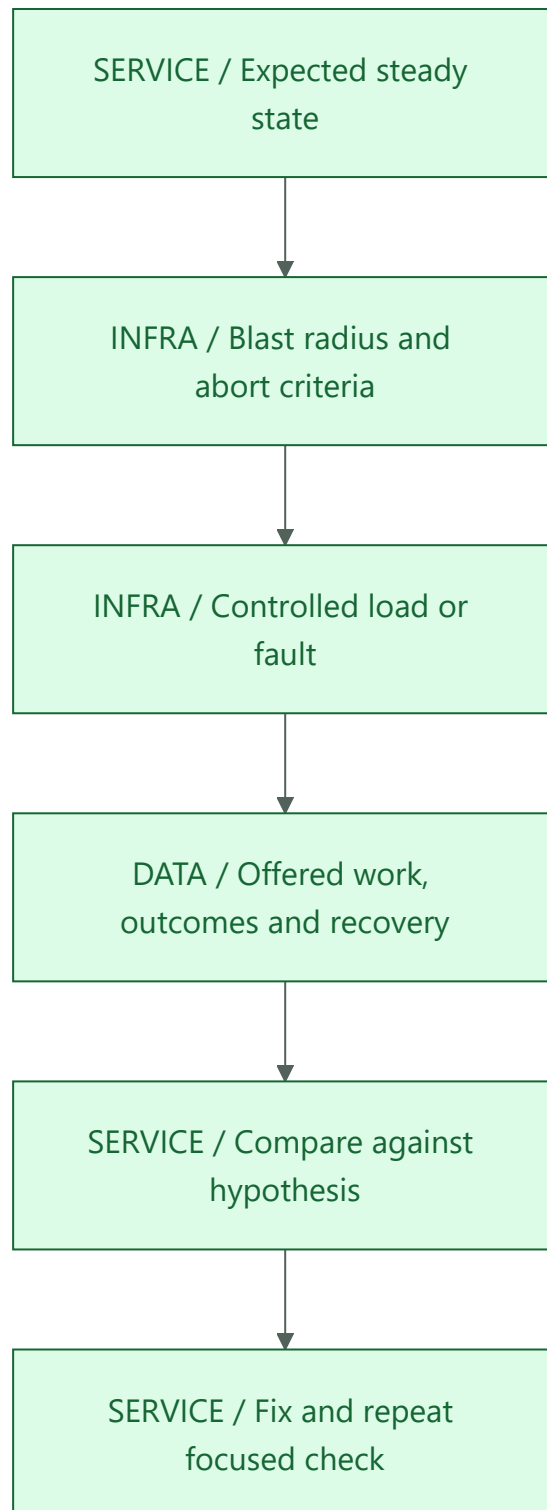
Interview checks: basic: high CPU is a symptom, not a cause. Internals: lag and completed work differ. Debug: choose a measurement that separates local hypotheses. Scenario: make dump collection a controlled operational action.

8. Capacity, Load Tests and Chaos

Capacity planning uses arrival distribution, service demand, concurrency, headroom and recovery work. Include tenant skew, burst timing, expensive requests, cache cold starts and dependency quotas. Little's law can relate stable mean quantities under a consistent boundary; it does not predict tail latency or choose a safe thread pool without more evidence. 10 supplies the arithmetic foundation.

Load tests should state the workload model, data distribution, environment, warmup, duration, offered/achieved throughput, error classification and latency percentiles. Closed-loop clients issue new work after prior completion and can reduce offered load when the server slows; open-loop arrival models expose different queueing behavior. Coordinated omission can underrepresent latency during stalls when the generator stops issuing expected work. Record generator saturation and dropped/timed-out work too.

Diagram 18.4



Chaos testing tests a specific resilience hypothesis under bounded fault injection. Define authorization, target, stop conditions, safe rollback, monitoring and excluded systems before injecting anything. Start with a controlled environment and small blast radius. A random production outage is not automatically a useful chaos experiment. Failure injection can have cost/data consequences even when it targets infrastructure rather than business code.

Examples for future tests: pause one consumer after effect commit but before ack; interrupt one provider path; make one replica unready; restore a cold cache under admitted traffic. Assert durable outcomes, bounded recovery and isolation, not simply that all Pods returned to Running. None of these faults is injected here.

[VERIFY: select load-generator arrival semantics, latency recording, telemetry sampling and fault-injection controls for the actual environment. No load, chaos, saturation or production fault experiment was run; all suggested experiments require explicit scoped authorization.]

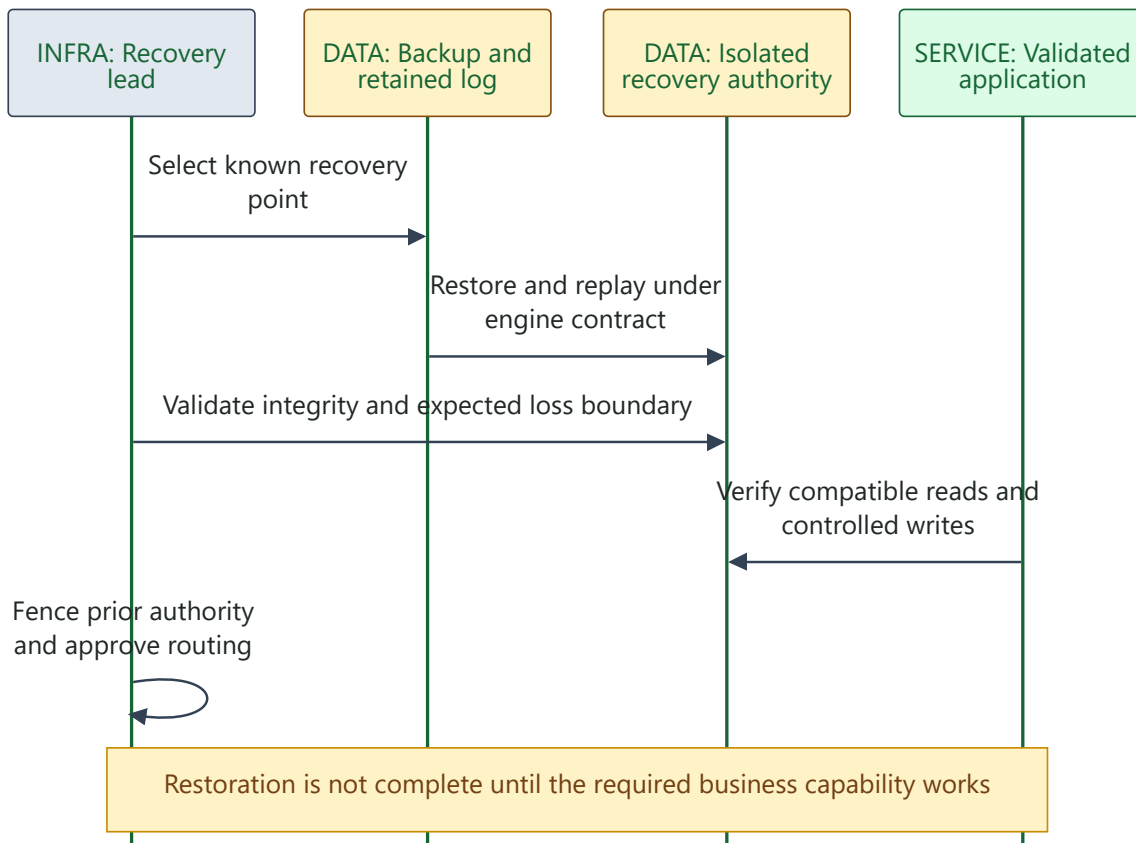
Interview checks: basic: a benchmark needs a workload and environment. Internals: offered and completed rates differ. Debug: inspect the generator before trusting a flat latency curve. Scenario: every chaos experiment needs an abort path and a falsifiable hypothesis.

9. Disaster Recovery and Restore Evidence

RPO defines acceptable loss of data under a stated failure; RTO defines acceptable restoration time for a stated capability. Define whether recovery includes only database availability or also keys, roles, networking, configuration, clients, replay and reconciliation. A backup timestamp is not a measured RPO, and a replica existing in another region is not a tested RTO.

Replication handles selected infrastructure failures but can reproduce mistaken deletes or bad writes. Base backups plus retained logs can support point-in-time recovery under the engine's contract. Keep copies and required keys accessible to the recovery environment without placing all authority in one failure domain. Verify retention, integrity and restoration regularly.

Diagram 18.5



Promotion/failover must prevent stale writers, not merely change DNS. Reconcile queued events, idempotency records and external uncertain effects after recovery. A rollback to an earlier database point can make previously completed external operations appear pending again; blindly replaying them can duplicate effects. Restore procedures must account for this mismatch explicitly.

[VERIFY: verify backup/PITR integrity, key availability, regional promotion/fencing, replay/reconciliation and measured RPO/RTO with the selected engines/providers. No backup, restore or failover rehearsal ran.]

Interview checks: basic: replica is not backup. Internals: recovery includes keys and effect identities. Debug: compare restored state with external committed effects. Scenario: rehearse a business operation, not just a database start.

10. Runbooks, RCA and Postmortems

A runbook should make the next safe action clear under pressure. Include purpose, affected service/owner, required access, trigger symptoms, first evidence to collect, decision branches, bounded mitigation, abort/rollback conditions and how to verify recovery. Commands need

approved context/namespace/account and explicit side-effect warnings. Do not include secrets in the document or assume every responder has administrative access.

Use the following compact runbook structure as content in the service's operational system, not as fabricated incident evidence: identify revision and impact; check user SLI plus dependency/resource signals; separate likely causes with one discriminating check; apply a scoped mitigation; verify outcome and backlog drain; escalate if the expected change does not occur. Record the actual action and result.

RCA should distinguish trigger, contributing conditions and missing controls. Human action can be part of the timeline without becoming the entire explanation. Ask why the action was possible, why feedback did not prevent it, why impact spread and why recovery took as long as it did. A single deepest root is often less useful than a causal chain with fixable boundaries.

Postmortem format: summary and impact; timeline with evidence links; triggering change; contributing technical and organizational conditions; detection/response assessment; what worked; corrective actions with owner, priority and verification; unresolved questions. Separate confirmed facts from hypotheses. Follow-ups should change a test, guardrail, design or runbook and be checked later, not disappear as a vague promise to be more careful.

Operational artifact	Use when	Avoid when
Runbook	A repeated failure needs safe consistent response	It is a long unexplained command dump
Postmortem	Learning should produce verifiable system improvements	Blame or hindsight replaces evidence
Capacity review	Demand/recovery needs changed	Last year's averages are treated as permanent guarantees

[D] **DECISION**

Prefer one tested recovery procedure over many unverified settings. The design should state who can act, which boundary is changed, what can go wrong and what observation proves success.

11. Executed Arithmetic Lab

`code/18-operations/SloLab.mjs` and its Windows PowerShell runner use only the existing Node-compatible runtime. From workspace root, run `& './docs/java-fs-guide/code/18-operations/run.ps1'`. `execution.json` records five PASS checks: error-budget arithmetic, no-

traffic unknown state, two-window condition, classic histogram cardinality and invalid-input rejection.

The model uses floating-point calculations with explicit comparison tolerances in tests; it is not an accounting system. Its `shouldPage` function demonstrates the conjunction of two burn windows, not a complete production alert: minimum traffic, no-data monitoring, scheduling, notification routing and SLO policy remain outside it. No PromQL expression or live alert was evaluated.

[?] INTERVIEW

Start an incident answer with impact, evidence and the next discriminating check.

Then explain mitigation and verification. Listing tools before stating the failing user contract usually hides the important reasoning.

Explained-Here Index and Sources

Concepts	Explanation
Logs, metrics, traces	Signal choice
OpenTelemetry	Propagation and export
Prometheus	Counters, distributions and labels
Grafana and Loki	Views and log storage
SLI, SLO and SLA	Outcome contract
Alerting and incident response	Actionable signals
Consumer lag and production playbooks	Diagnosis
Capacity, load and chaos testing	Experiments
Disaster recovery, RPO, RTO	Recovery evidence
RCA, runbook and postmortem	Operational learning

Verification destinations: [OpenTelemetry](#), [Prometheus practices](#), [Grafana documentation](#), [Loki](#) and [Google SRE workbook](#). References are pointers, not claims of new runtime evidence.

Related Chapters

Connect 00, 05 APIs, 06 databases, 07 messaging, 08 security, 10 capacity, 15 Kubernetes, 16 releases, 17 cloud, 19 tests, 21 network, 22 distributed recovery and 23 JVM profiling.

Generated links are reciprocal.

RELATED CHAPTERS / RECIPROCAL LINKS

00	The System Before the Details	22	Distributed Systems Theory
02	Concurrency and the Memory Model	12	Twelve System Design Case Studies
21	Networking and Operating Systems	27	Fintech and Payments Engineering
03	Spring and Spring Boot Internals	13	Data Integration and iPaaS
06	Databases and Durable State	14	Docker and Container Internals
05	APIs and Real-Time Communication	15	Kubernetes and Production Workloads
07	Messaging, Batch and Streams	16	Build, Ship and Roll Back
26	Data Platform Basics	17	Cloud and Infrastructure
08	Security, Identity and Data Protection	23	JVM Performance Engineering
09	The Browser and the Java Backend	19	Testing and Quality
10	System Design Foundations	24	IntegrationHub in Motion
25	Architecture Styles and Domain Design	20	Appendix: Interview Toolkit and Index

One-Page Cheat Sheet

Signals: metrics aggregate, logs explain events, traces connect sampled execution. OTel instruments/exports; backends store and query. Context is not authorization and baggage is not a secret channel.

Metrics: counters reset; histogram quantiles are estimates; do not average p99s. Bound labels and distinguish missing observations from success. Loki labels and Prometheus labels both need cardinality discipline.

Reliability: define eligible good outcomes, window and authority. Acceptance and completion need separate SLIs. SLO is a target, SLA an agreement. Burn rate consumes a stated error allowance; no traffic is not automatically healthy.

Response: page actionable impact, preserve evidence, mitigate one bounded cause and verify recovery. Diagnose CPU, heap/native memory, locks, queries and consumer lag at the owning layer. Deep JVM work belongs to 23.

Recovery: load models need offered rates and generator evidence; chaos needs authorization and abort conditions. Restore data, keys, identity and business capability, and fence old writers. Five arithmetic checks passed; production telemetry and recovery tests did not run.

Interview Corner

Basic: SLI, SLO and SLA?

A measurement, a target for that measurement, and an external agreement. Define eligible outcomes and window before interpreting any percentage.

Internals: Why Not Average Instance p99 Values?

Quantiles are not generally composable that way. Aggregate compatible observations/distributions and estimate the desired fleet quantile, with resolution and sampling limits understood.

Trace/Debug: API Is Green but Jobs Never Finish.

Acceptance health is not completion health. Inspect oldest accepted work, committed progress, consumer processing, provider outcomes and local queue occupancy rather than only HTTP status.

Scenario: What Should Page an Engineer?

A condition where timely action can protect a meaningful objective, with context, ownership and a runbook. Use lower-urgency workflows for trends that do not require immediate intervention.

Basic: Is OpenTelemetry the Monitoring Database?

No. It supplies instrumentation and collection/export components. Storage/query/visualization are backend responsibilities, and collector failure needs its own bounds.

Internals: What Does Tail Sampling Cost?

It needs later trace information and therefore buffering, coordination/routing and incomplete-trace policies. It can select important traces but does not make the stored set complete.

Trace/Debug: Consumer Lag Falls but Completion Is Flat.

Offsets may have advanced before work left a local queue, or downstream effects are stalled. Compare broker position with durable business completion and oldest pending age.

Scenario: How Do You Investigate Memory Growth?

Separate allocation churn from retained heap and native/process memory. Capture approved evidence with disk/privacy/overhead bounds, then inspect retained paths rather than assuming every growing chart is a leak.

Basic: RPO Versus RTO?

Acceptable data loss versus acceptable restoration time under a defined failure and capability. Both require observed restore/recovery evidence.

Internals: Why Can Restore Repeat an External Effect?

The database may return to a point before a provider action that already committed. Reconciliation and stable external identity are needed before replaying restored pending work.

Trace/Debug: A Load Test Shows Excellent Latency During Stalls.

Check whether the generator stopped issuing expected work, dropped samples or saturated. Closed-loop behavior and coordinated omission can hide waiting that real arrivals would experience.

Scenario: What Makes a Postmortem Useful?

Evidence-backed impact and timeline, contributing conditions, and corrective actions with owners and verification. Blame or an untracked promise does not reduce recurrence.

23 / JVM Performance Engineering

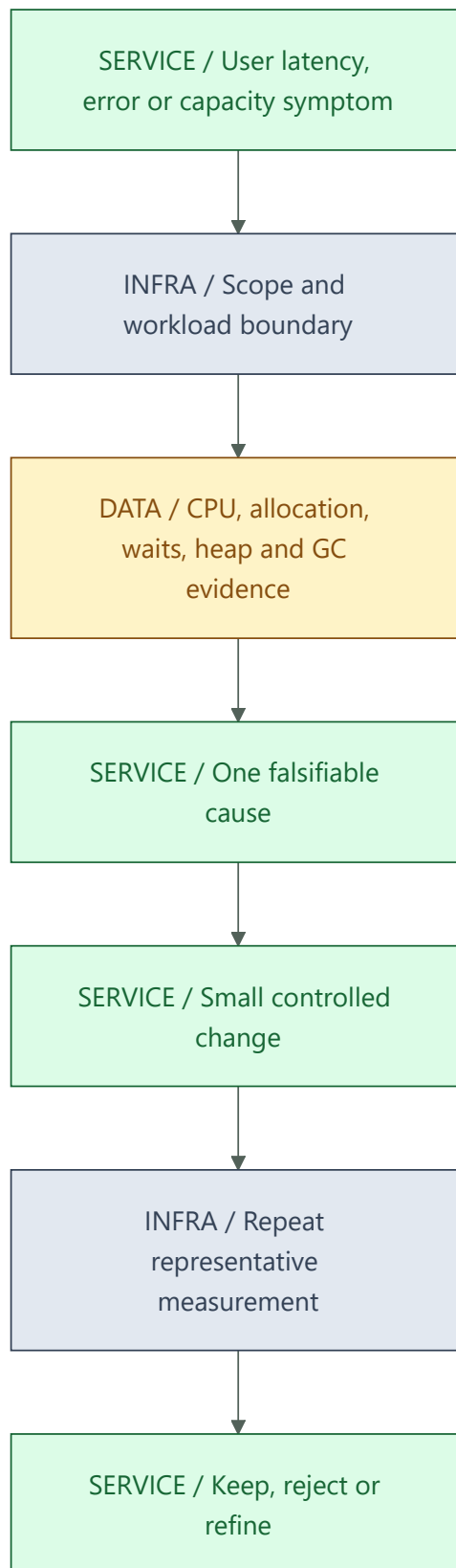
Measure the constrained resource before changing the runtime. A slow request can be waiting for a connection, blocked on a lock, throttled by its cgroup, allocating rapidly or executing expensive code. IntegrationHub is fictional. This chapter explains how to distinguish those cases without converting guesses into benchmark claims.

Version assumptions: Java 21 HotSpot-oriented examples, Spring Boot 4.0.x / Cloud 2025.1.x, PostgreSQL 17, Kafka 4.x and Kubernetes 1.34. Collector modes, container support, diagnostic events and flags can change in later JDKs. The optional benchmark pins JMH 1.37 and build plugins as unexecuted teaching inputs, not current-patch recommendations.

Execution status: NOT EXECUTED. The runner parsed the POM and found JFR/JMH sources, but no JDK compiler is available. No recording, profile, flame graph, heap dump, thread dump, GC log or benchmark result was generated. No existing process was attached to. Every workload amount in the fixture is an input choice, not a measured capacity result.

Big Picture

Diagram 23.1



What You Will Be Able to Explain

- Select JFR/JMC, async-profiler, thread dumps and heap analysis for the question at hand.
- Read CPU versus allocation profiles and flame graphs without treating them as request timelines.
- Diagnose retained-memory growth, direct/native pressure, lock waits and collector behavior separately.
- Explain G1/ZGC choices, unified GC logging and container headroom under a pinned JDK.
- Explain warmup, tiered JIT, inlining, speculative optimization and deoptimization.
- Use JMH with parameters, state, warmup and forks while recognizing its scope limits.
- Size pools using consistent boundaries, occupancy and downstream limits rather than a universal formula.

[P] PRODUCTION

Where this lives in IntegrationHub. 18 identifies impact and operational evidence; this chapter deepens JVM interpretation. 02 owns task/lock contracts, 06 owns query and connection cost, 14/15 bound process resources, and 21 explains OS/socket waits. Optimization must preserve tenant, idempotency and transaction guarantees.

1. A Repeatable Investigation

Define the symptom precisely: p99 response latency under a named load, completion throughput, allocation rate, post-collection heap growth or CPU per operation. Capture workload shape, data sizes/skew, runtime flags, container limits, release/configuration identity and warm/cold state. Comparing two runs with different input or resource allocation cannot isolate a code change reliably.

Start with cheap discriminating observations. Low CPU with high latency suggests waiting or throttled/blocked progress rather than a pure compute hot loop, but inspect the relevant CPU denominator and cgroup throttling. High CPU can be application computation, garbage collection, serialization, retries or native code. A large heap alone is not proof of a leak.

Choose one hypothesis and one measurement capable of rejecting it. If query time dominates, a method microbenchmark may not improve the request. If the heap remains stable after collection but allocation is high, investigate churn rather than retained roots. Change one meaningful factor, rerun under comparable conditions, and inspect correctness plus resource trade-offs.

Question	Use when	Avoid when
CPU profile	Runnable compute cost is suspected	Most elapsed time is waiting on a remote service
Allocation profile	Object creation/GC work is suspected	Retained-memory leaks are inferred from allocations alone
Thread/wait evidence	Locks, pools or blocking dominate	One sampled stack is called a complete timeline
Heap retained graph	Objects remain live unexpectedly	A dump is captured without privacy/disk/pause planning

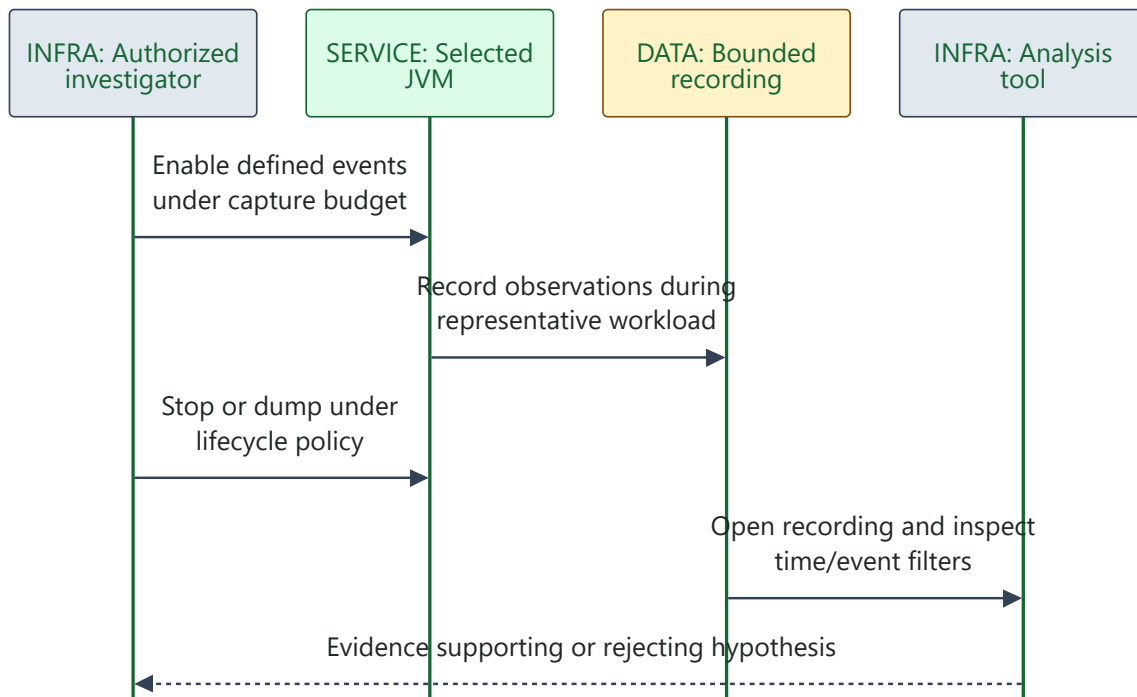
Interview checks: basic: latency is not CPU time. Internals: workload boundaries determine comparable evidence. Debug: distinguish waiting, computation and memory pressure. Scenario: optimize the measured contribution, not the most familiar method.

2. JFR and JMC

Java Flight Recorder records selected JVM/application events under a configuration. Events can include execution samples, allocations, locks, GC, threads and I/O depending on the runtime/settings. Java Mission Control is a tool for inspecting recordings and related diagnostics, not the source of all events itself. A recording's event configuration and time window determine which conclusions it can support.

JFR can run with bounded size/age policies, and custom events can describe meaningful application operations when carefully designed. Record only safe bounded attributes, not credentials or entire payloads. More events, stack traces and shorter intervals can increase overhead/storage, so low overhead is a property to verify for the chosen configuration rather than a universal promise.

Diagram 23.2



For an existing process, diagnostic attachment needs the correct PID/namespace and permissions. Recording can contain sensitive paths, arguments, SQL-related context or application fields. Agree on output location, retention, access and capture duration. Do not run privileged attachment variations blindly when access is denied; use the approved operations path.

Correlate event time with the incident window, workload and release. Absence of a sampled stack does not prove absence of execution. A short recording can legitimately contain no GC or CPU samples for the question, especially when the workload ends quickly. Distinguish unavailable/disabled events from a measured zero.

[VERIFY: verify JFR event names/defaults, sampling/stack settings, recording bounds, JMC compatibility and attach permissions for the selected JDK/OS. No JFR recording or JMC analysis ran locally.]

Interview checks: basic: event configuration controls evidence. Internals: sampling and thresholds omit information. Debug: inspect the recording's actual event/time coverage. Scenario: capture under a privacy, disk and overhead budget.

3. async-profiler and Flame Graphs

async-profiler can collect selected CPU, allocation and other profiles under supported runtime/OS mechanisms. Native and Java stack visibility, permissions, kernel perf restrictions and profiling mode affect results. It is useful for questions that ordinary Java-only stack sampling may not fully explain, but it is not a universal zero-overhead truth source.

A flame graph aggregates stack weights. In a CPU sample graph, width commonly represents sampled on-CPU weight; in an allocation graph it may represent sampled allocated bytes or objects according to the mode. The horizontal axis is usually an arrangement of stacks, not chronological time. Depth represents call nesting, not directly request latency. Read the legend/units before making a claim.

Inclusive cost includes callees; self cost attributes work directly to the frame under the tool's model. A wide serialization frame may represent many ordinary calls rather than one slow call. A narrow but blocking operation can dominate user elapsed time while consuming little CPU. Compare CPU and wall-clock/wait evidence where supported and appropriate.

Profile	Use when	Avoid when
CPU samples	Identify where execution consumes processor time	Off-CPU waits are expected to appear as wide CPU bars
Allocation samples	Locate high object/byte creation paths	Allocation volume is treated as retained heap size
Wall/wait-oriented evidence	Elapsed waiting and blocked work matter	Different sample units are compared as if identical

[VERIFY: confirm async-profiler release, supported event modes, native stack visibility, kernel permissions and interpretation of flame-graph units on the actual host. No profiler was installed, attached or executed.]

Interview checks: basic: flame width depends on metric. Internals: inclusive and self cost differ. Debug: do not read stack layout as a timeline. Scenario: select CPU versus allocation/wait mode from the hypothesis.

4. Allocation Rate Versus Retained Memory

Allocation creates objects; retention keeps them reachable. A workload can allocate heavily and retain little, producing collection work without an ever-growing live set. Another workload can allocate modestly but retain references indefinitely, causing a slow leak. Comparing only heap used at an arbitrary instant conflates both.

Measure allocation sites and bytes/operation under representative traffic. Temporary DTOs, parsing buffers, boxing, string construction and repeated serialization can create churn. Some source-level allocations may be eliminated by JIT optimization, so counting new expressions is not a runtime allocation measurement. A cache that improves CPU can increase retained heap; evaluate the trade-off instead of assuming every reduced allocation is beneficial overall.

For suspected leaks, compare live-set trends after comparable collections or equivalent workload phases, not two random heap samples. A larger workload legitimately retains more state. Identify expected lifetime and ownership: who should release the object graph when a request, tenant session, subscription or class loader ends?

[M] **MECHANISM**

GC reclaims unreachable memory, not obsolete business data. A completed job still referenced by a static map remains live from the collector's perspective. Fix the reference/lifetime contract rather than asking the collector to recognize business completion.

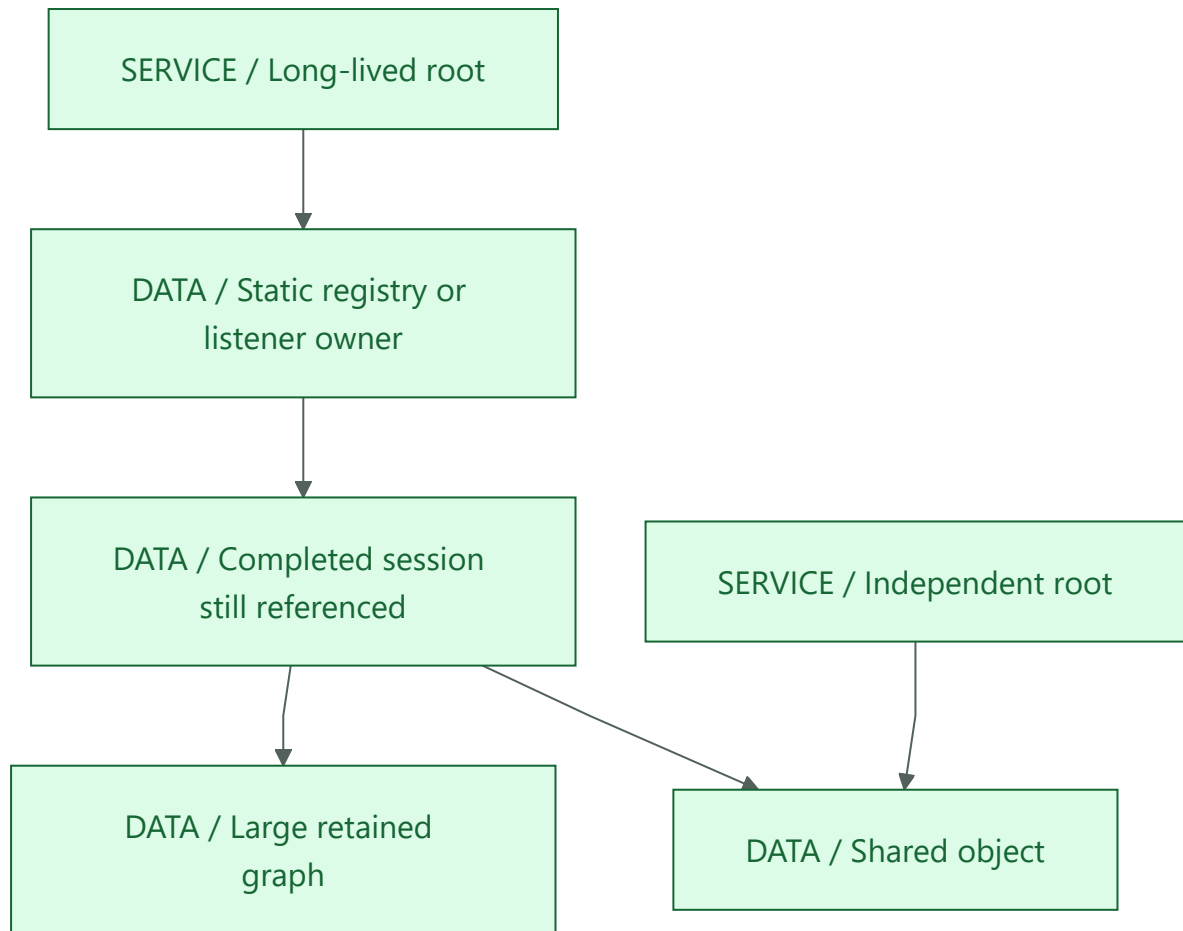
Interview checks: basic: allocation and retention are different. Internals: reachability defines collection eligibility. Debug: compare live set under comparable workload. Scenario: inspect ownership of long-lived registries and callbacks.

5. Heap Dumps and MAT

A heap dump represents an object graph at a capture point under the tool's semantics. It can be large and sensitive. Collection/pause behavior and inclusion of unreachable objects depend on the capture option/runtime. Plan disk space, process impact, secure transfer and analysis access before capturing one from production.

MAT and similar tools provide histograms, dominator analysis and paths to GC roots. Shallow size counts an object's immediate storage; retained size concerns memory that remains dependent on its reachability under the graph analysis. A dominator lies on every root path to dominated objects. A large retained subtree is a clue to ownership, not automatically proof that the subtree is a leak.

Diagram 23.3



Inspect root paths and expected lifecycle. Common patterns include unbounded caches, listeners never removed, ThreadLocal values on pooled threads, class-loader retention, accumulated futures/callbacks and unclosed resources retaining buffers. Weak or soft references are not universal fixes; they change reachability/reclamation semantics and may make cache behavior unpredictable without solving ownership.

Compare several captures or a controlled reproduction when possible. The biggest object can be legitimate, and the actual leak may be a small root retaining many graphs. Native/direct memory may not appear as ordinary payload bytes in the heap dump; a small Java wrapper can reference substantial external allocation.

[VERIFY: confirm heap-dump capture semantics, pause/disk impact, MAT retained-size interpretation and sensitive-data handling for the actual JVM and environment. No heap dump or leak analysis was performed.]

Interview checks: basic: shallow and retained size differ. Internals: root paths explain lifetime. Debug: a large graph is not necessarily a leak. Scenario: fix removal/closure and verify the retained graph disappears under comparable load.

6. Thread Dumps, Locks and Pool Waits

A thread dump samples execution states and stacks. RUNNABLE can include work in native calls under JVM reporting and does not automatically mean a busy CPU loop. WAITING or TIMED_WAITING can be healthy idle behavior. BLOCKED has a particular Java monitor meaning and is not a generic label for every resource wait. Lock ownership and thread names help trace contention, but compare with CPU and timing evidence.

Repeated dumps can show the same owner holding a lock while many callers wait, or a pool whose workers wait for child tasks queued to the same exhausted executor. Database acquisition waits point to query duration, leaks or admission pressure rather than necessarily a network connect failure. [02](#) owns those concurrency contracts.

Virtual-thread diagnostics differ across JDK/tool versions and can contain far more logical tasks than OS threads. Carrier pinning behavior also changed after Java 21 for some synchronization cases. Do not copy an old monitor-pinning conclusion into every later runtime. Resource limits still matter even when thousands of blocked tasks are cheap to represent.

Interview checks: basic: state is a sampled classification. Internals: monitor wait, pool wait and native I/O differ. Debug: correlate repeated stacks with CPU and ownership. Scenario: a larger executor can worsen downstream contention instead of resolving it.

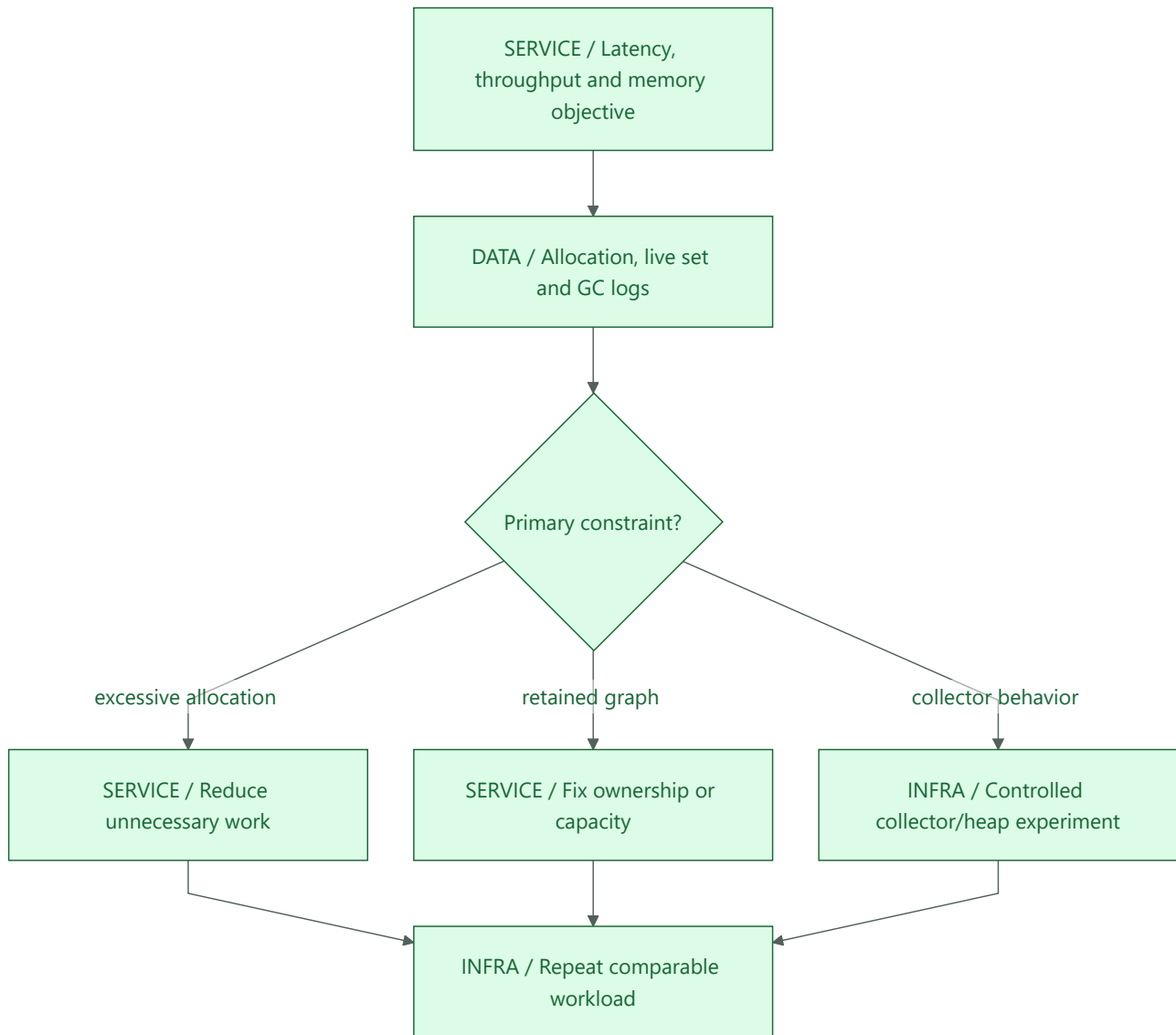
7. G1, ZGC and Unified GC Logging

Collector selection trades pause behavior, CPU, throughput, memory footprint and available headroom. Start with application allocation/live-set evidence and current defaults rather than a bag of tuning flags. A pause target is generally a goal under the collector's control policy, not a hard real-time guarantee.

G1 organizes heap into regions, performs young evacuation and uses concurrent marking/mixed collection work to manage old regions under its design. Remembered-set/barrier work tracks references across relevant boundaries. Large objects and fragmentation can affect region use. Insufficient headroom or changing allocation patterns can trigger expensive recovery behavior; inspect logs rather than assuming every pause has the same cause.

ZGC moves much marking/relocation work concurrently using barriers and its object/reference representation. That can reduce pause exposure while requiring CPU and memory headroom for concurrent work. It does not eliminate all pauses, stop-world events or the effect of allocation outrunning collection. Java 21 offers generation-related choices that changed in later releases; pin the exact version before selecting flags.

Diagram 23.4



Unified logging selects tags, levels, decorators and output rotation. GC and safepoint logs help separate heap-collection pauses from other stop events; choose bounded file size/count and safe locations. Useful flag concepts include explicit heap sizing, collector selection and unified-log configuration. Do not mix flags from incompatible JDK generations or infer that a process accepted a flag merely because it appears in a deployment template.

Compare allocation rate, pause distribution, concurrent CPU, full-cycle behavior, throughput and memory after the change. Reducing one pause percentile while exhausting CPU or increasing failure rate is not an unqualified improvement. Keep changes reversible and record actual effective flags with the result.

[VERIFY: verify G1/ZGC mode availability and defaults, GC/safepoint log syntax, pause-target semantics and effective flags for the selected JDK. Java 21 and later ZGC generations differ; no collector comparison or GC-log experiment ran.]

Interview checks: basic: collector choice is a trade-off. Internals: concurrent work still consumes resources. Debug: classify pauses and allocation/live-set behavior. Scenario: change one supported setting and remeasure correctness and the whole resource budget.

8. Container Awareness and MaxRAMPercentage

The JVM's view of processors/memory can be influenced by container limits and its implementation. MaxRAMPercentage participates in heap sizing under the relevant configuration; it is not a cap on total process memory and can interact with explicit heap flags. Native memory, direct buffers, stacks, metaspace, code cache and other charged resources need room beyond the heap.

CPU quota/throttling can delay request progress and concurrent GC even while host-wide CPU is not fully occupied. A pool sized from physical host cores may be inappropriate inside a constrained container if runtime detection/configuration differs. Memory-backed volumes and mapped/file cache accounting can also affect cgroup pressure.

A kernel OOM kill can prevent Java shutdown hooks and heap-dump-on-error paths from running. A JVM OutOfMemoryError may arise from heap, direct buffer, thread creation or other resource limits depending on the failure. Treat termination reason, effective limits and native/heap evidence as separate inputs. [15](#) covers platform diagnosis.

[VERIFY: verify container processor/memory detection, MaxRAMPercentage and explicit heap interaction, native-memory visibility, virtual-thread diagnostics and pinning behavior against the actual JDK/cgroup version. No container or thread-memory experiment ran.]

Interview checks: basic: heap percentage is not total RSS. Internals: throttling can change GC and request latency. Debug: inspect effective limits/flags, not only declared YAML. Scenario: reserve process headroom and verify under representative concurrency.

9. JIT, Warmup, Inlining and Deoptimization

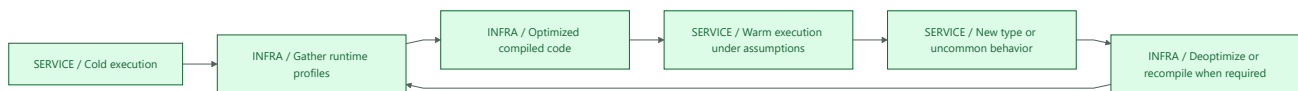
HotSpot can start with interpreted execution, gather profiling information and compile hot code through tiers according to its configuration. Optimized code depends on observed behavior and compiler assumptions. Cold startup and warmed steady state are therefore different workloads; neither is inherently the only one that matters.

Inlining removes call boundaries and exposes further optimization opportunities. Escape analysis and related optimizations may eliminate some allocations/locking when semantics permit. Constant inputs, unused results and predictable state can let a microbenchmark

measure optimized-away work. Conversely, forcing every operation through an artificial blackhole or interface can change the code path compared with production.

Speculative assumptions can become invalid as new types or paths appear, causing deoptimization and later recompilation. Code-cache pressure, class loading and uncommon paths can affect tail latency. Do not attribute every slow warmup interval to GC without compilation/class-loading evidence.

Diagram 23.5



Interview checks: basic: Java performance changes with runtime history. Internals: inlining enables other optimizations. Debug: check compile/deopt activity as well as GC. Scenario: benchmark the startup or steady-state requirement you actually need.

10. JMH and Benchmark Pitfalls

JMH supplies a harness for controlled JVM microbenchmarks. It manages warmup, measurement iterations, forks and result consumption under its configuration. It cannot choose representative inputs, remove environmental noise or turn a local loop result into a distributed service throughput claim.

The saved SumBenchmark compares indexed iteration and a long stream over runtime-initialized arrays. @Param varies array size, @State(Scope.Thread) gives per-thread state, setup checks equivalent results, @Warmup/@Measurement define trial phases and @Fork separates JVM runs. Each benchmark returns its result so the harness consumes it. The reported unit would be nanoseconds per whole method invocation, not automatically per array element.

Warmup counts and trial duration in the fixture are teaching choices, not proof of stabilized production behavior. Separate forks help reduce cross-benchmark JVM-history effects, but hardware, OS load, CPU quotas, frequency/power state and data cache warmth still matter. Inspect run variance and confidence, not just a winning mean with misleading decimals.

Avoid timing a tiny operation with one manual nanoTime loop and reporting it as reliable. Measurement overhead, dead-code elimination, constant folding, batching, loop optimization and allocation elimination can dominate. Do not add a loop inside the benchmark solely to make numbers larger without understanding that it changes optimization and units.

Benchmark decision	Use when	Avoid when
JMH local comparison	A CPU/allocation hypothesis has a representative isolated operation	The real bottleneck is network, locks or database I/O elsewhere
Parameterized runtime state	Different input sizes/shapes matter	One constant example is optimized into an unrepresentative path
Separate forks and enough warmup	JVM history must be controlled	One warmed method contaminates another comparison invisibly
End-to-end load test	System latency/throughput is the requirement	A microbenchmark result is extrapolated without the rest of the workload

[VERIFY: resolve JMH 1.37 and its annotation processor/shaded harness, verify benchmark discovery and run representative trials on an approved JDK/host. The fixture was not compiled or run; no benchmark ranking or timing is claimed.]

Interview checks: basic: JMH is a harness, not a guarantee of representativeness. Internals: setup, forks and consumption affect optimization. Debug: verify discovery, units and variance. Scenario: use a microbenchmark only for the local contribution it actually measures.

11. Pool Sizing and Little's Law

Under stable conditions and a consistent boundary, mean occupancy equals throughput times mean residence time. This can inform expected in-flight work, but it does not directly determine a safe maximum thread or connection count. Tail distributions, bursts, queueing, CPU service demand and downstream saturation need separate evidence.

A connection pool is an admission/concurrency boundary for a finite database. Increasing it can increase contention rather than capacity. An executor with cheap virtual threads still needs bounded access to expensive resources. Separate how much work may wait from how much may actively use the dependency, and budget across all replicas rather than one process.

If response time rises with offered load while throughput plateaus, queueing near a bottleneck is a strong hypothesis. Lowering concurrency can sometimes improve throughput/latency by reducing contention, but measure the workload rather than treating that as a universal rule. Include recovery headroom so backlog can drain after failure.

[!] TRAP

A formula cannot repair a mismatched boundary. Using end-to-end latency with a database-only throughput count, or using p99 in a mean-value relation, produces an impressive-looking but unjustified pool size.

Interview checks: basic: mean occupancy is not a tail guarantee. Internals: active use and admitted waiters are separate. Debug: look for throughput plateaus and growing queues. Scenario: size all replicas against shared database/provider capacity.

12. Saved Fixtures and Safe Execution

`code/23-jvm-performance/ProfilingLab.java` records its own bounded synthetic workload through the JFR API. It retains only a small fixed number of byte arrays while allocating repeated replacements, then stops/dumps the recording and checks that a nonempty file exists. A nonempty JFR file can contain metadata without enough samples for a useful performance conclusion; the fixture is a recording-path smoke check, not a profile of a real service.

`code/23-jvm-performance/src/main/java/guide/performance/SumBenchmark.java` and `pom.xml` form a separate JMH project. The annotation processor generates harness metadata and the package step creates an executable benchmark jar. Dependencies and plugin versions are explicit, and Maven runs offline only.

Windows PowerShell from workspace root: `& ' ./docs/java-fs-guide/code/23-jvm-performance/run.ps1 '` attempts only the self-generated JFR fixture with an existing JDK. Add -Benchmark only when an existing Maven and approved cached dependencies are available. No command attaches to an existing user process; no tool or dependency download is attempted. Actual preflight: POM/source checks PASS, JFR and JMH NOT EXECUTED.

[VERIFY: execute the self-generated JFR smoke fixture and optional offline JMH project with the approved toolchain, inspect recording event coverage and actual benchmark output, and record the environment before drawing performance conclusions. No recording, dump, profile or benchmark output exists from this session.]

[D] DECISION

Choose the least intrusive evidence that can reject the hypothesis. Start with existing metrics and bounded recordings, then escalate to dumps or specialized profiles only with an impact/privacy plan. More data is not automatically a better diagnosis.

[?] INTERVIEW

Close the loop: symptom, workload, evidence, hypothesis, change, repeated measurement and trade-off. Naming JFR or ZGC without that loop is not performance engineering.

Explained-Here Index and Sources

Concepts	Explanation
JFR and JMC	Recording and analysis
async-profiler, flame graph, CPU profiling	Stack weights
Allocation profiling	Churn versus retention
Heap dump, MAT, memory leak	Retained graphs
Thread dump	Waits and ownership
G1, ZGC and GC logging	Collector workflow
MaxRAMPercentage	Container budget
JIT, warm-up, inlining, deoptimization	Runtime optimization
JMH	Microbenchmark design
Little's law and pool sizing	Boundary-consistent sizing

Verification destinations: [JFR API](#), [JDK Mission Control](#), [async-profiler](#), [Eclipse MAT](#), [JMH](#) and [Java 21 GC tuning guide](#). These are owning references, not fresh runtime evidence.

Related Chapters

Use [00](#), [01 JVM](#), [02 concurrency](#), [04 ORM](#), [06 database](#), [08 evidence privacy](#), [10 capacity](#), [14 containers](#), [15 resource limits](#), [18 operations](#), [19 testing](#), [21 OS](#) and [22 pauses/leases](#). Generated links are reciprocal.

RELATED CHAPTERS / RECIPROCAL LINKS

00 The System Before the Details	22 Distributed Systems Theory
01 Core Java and the JVM	14 Docker and Container Internals
02 Concurrency and the Memory Model	15 Kubernetes and Production Workloads
21 Networking and Operating Systems	18 Observability and Reliability
03 Spring and Spring Boot Internals	06 Databases and Durable State
04 Hibernate and JPA	08 Security, Identity and Data Protection
05 APIs and Real-Time Communication	19 Testing and Quality
10 System Design Foundations	24 IntegrationHub in Motion
	20 Appendix: Interview Toolkit and Index

One-Page Cheat Sheet

Workflow: define user symptom, workload and runtime/resource identity. Measure the suspected boundary, make one falsifiable hypothesis, change one factor and repeat comparable validation.

Evidence: JFR event configuration controls coverage; JMC analyzes it. CPU profiles show CPU weights, allocation profiles creation, dumps retained graphs and thread samples waits/ownership. Flame width is metric weight, not chronological duration.

Memory/GC: allocation churn is not a leak. Find roots and expected lifetime. G1/ZGC trade pause/CPU/memory/throughput; targets are not hard guarantees. Process memory exceeds heap and kernel kills can bypass Java cleanup.

JIT/JMH: warmup and speculative optimization matter. Use representative runtime state, forks and correct consumption/units. A local benchmark does not prove service throughput. Do not invent results for unexecuted code.

Sizing: Little's law uses consistent mean quantities; pool limits still face bursts and shared bottlenecks. Cheap threads do not expand database capacity. JFR/JMH sources are supplied, but compilation, recordings and benchmarks remain NOT EXECUTED.

Interview Corner

Basic: CPU Profile Versus Allocation Profile?

One attributes sampled execution cost, the other object/byte creation under its event model. Neither alone proves retained-memory growth or end-to-end latency.

Internals: What Does a Wide Flame-Graph Frame Mean?

A large aggregate weight in the selected metric, often inclusive of callees. It is not necessarily one slow invocation or a chronological interval.

Trace/Debug: Heap Grows Between Collections but Then Falls.

That can be allocation churn rather than a leak. Compare retained live set under comparable workload and collection phases before diagnosing lost ownership.

Scenario: How Would You Capture Production Evidence Safely?

Select an approved process and bounded event/time/size configuration, assess overhead and privacy, choose secure storage and record the workload/revision. Escalate only when the current evidence cannot distinguish causes.

Basic: Shallow Versus Retained Size?

Immediate object storage versus the graph whose liveness depends on it under retained-size analysis. Root paths and expected lifetime determine whether the retained graph is wrong.

Internals: Why Can JIT Eliminate Benchmark Work?

Unused results, constant inputs and provably nonescaping allocations can be optimized away. Use a proper harness and representative state while understanding that the harness cannot choose a realistic workload for you.

Trace/Debug: Low Heap but OOMKilled.

Inspect total cgroup memory, native/direct allocation, stacks and other charged resources, plus the termination reason. JVM heap is only part of process/container memory.

Scenario: Would You Switch Collectors First?

Only if evidence identifies collector behavior as the relevant constraint. Allocation, retention, blocked work or throttling may be the root cause; measure trade-offs with one controlled change.

Basic: Warmup Versus Measurement?

Warmup exercises runtime compilation/profile behavior before recorded trials; measurement observes the selected phase. Both need durations appropriate to the actual benchmark, not blindly copied counts.

Internals: Why Use Separate JMH Forks?

They help separate JVM histories and reveal run variation. They do not remove hardware noise, bad input design or incorrect interpretation of units.

Trace/Debug: More Connections Made the Database Slower.

The server may already be saturated; extra concurrency adds contention and queueing. Inspect query cost, locks, active work and aggregate pools across replicas.

Scenario: What Can the Saved Fixture Prove Today?

Only that its source and POM passed local presence/XML checks. Without a JDK, no compilation or recording ran. A future smoke PASS still would not establish a production performance conclusion.

19 / Testing and Quality

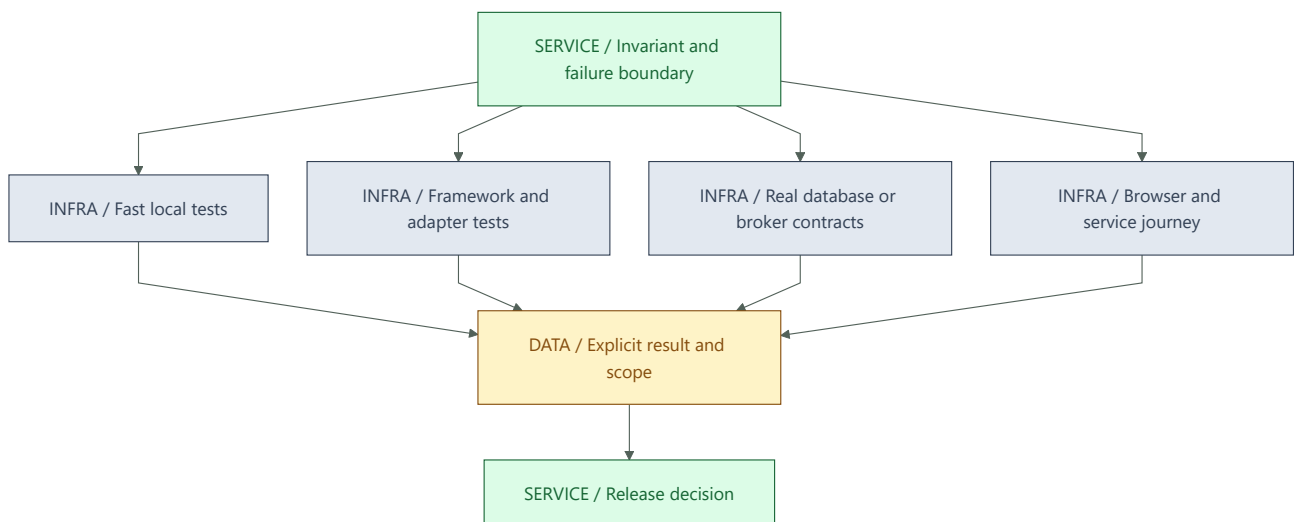
A test is evidence for the boundary it actually exercises. A mocked repository cannot establish PostgreSQL isolation, and a source-listing comparison cannot establish Java compilation. IntegrationHub is fictional. This chapter uses that distinction to build a layered test strategy without inflating the meaning of a green result.

Version assumptions: Java 21, Spring Boot 4.0.x / Cloud 2025.1.x, PostgreSQL 17, Kafka 4.x and Kubernetes 1.34 remain the surrounding baseline. The standalone lab deliberately pins JUnit Jupiter 5.11.4, Mockito 5.14.2, compiler plugin 3.13.0 and Surefire 3.5.2; it does not import the Boot BOM or claim these are its managed/current versions. Framework imports and Testcontainers/Playwright versions require their own compatibility check.

Execution status: NOT EXECUTED. The runner parsed the POM and found both source files, but no existing JDK/Maven is available. No dependency resolution, Java test, Spring context, container or business-browser integration ran. The guide's publication checks and prior pure JavaScript models are separate evidence.

Big Picture

Diagram 19.1



What You Will Be Able to Explain

- Choose tests by risk and ownership rather than a rigid numerical pyramid.

- Use JUnit lifecycle/parameterization and Mockito deliberately, without testing only mock setup.
- Distinguish Spring test slices, full application contexts and deployed network behavior.
- Design Testcontainers tests around the real engine, transaction and crash boundaries.
- Separate schema compatibility, consumer contracts and end-to-end behavior.
- Use Playwright assertions, fixtures and traces without arbitrary sleeps or shared-state leakage.
- Make performance, flakiness and quality gates explicit rather than hiding failed or skipped checks.

[P] PRODUCTION

Where this lives in IntegrationHub. Test 05's idempotent acceptance, 06's constraints, 07's effect/ack gap, 08's tenant authorization and 09's request generations at their actual boundaries. 16 consumes the results, and 18 determines which load/failure signals matter after deployment.

1. Test Pyramid, Scope and Risk

The pyramid encourages many fast focused tests and fewer expensive broad tests because speed, isolation and diagnosis affect feedback. It is not a requirement that every system have the same percentage of unit versus integration tests. A thin SQL-heavy service may gain more value from engine-backed tests than from mocking every query. A complex pure algorithm deserves extensive fast cases.

For each invariant, identify the smallest environment that could falsify it. Decimal conversion can be tested locally; tenant scoping may need both policy tests and real query paths; transaction atomicity needs the actual transaction arrangement; a buffering proxy requires the proxy in the path. An end-to-end test can show a journey failed but may be poor at explaining which layer caused it.

Distinguish tests from checks. A static schema check, source presence, compilation, unit assertions, deployed health and measured SLO each establish different facts. Mark unavailable tests as NOT EXECUTED rather than silently skipping them under a PASS umbrella. A command exiting successfully with no discovered tests should fail the intended test gate.

Layer	Use when	Avoid when
Pure unit	Local rules can be verified without infrastructure	A mock stands in for the very behavior under question
Component/slice	Framework wiring or one adapter is the risk	Slice success is called proof of the whole deployment

Layer	Use when	Avoid when
Real dependency integration	SQL, broker or network semantics matter	A substitute engine is assumed identical
End-to-end	A critical user journey crosses several boundaries	Every edge case becomes a slow broad browser test

Interview checks: basic: test shape follows risk. Internals: the exercised boundary defines evidence. Debug: inspect discovered tests and skipped reasons. Scenario: choose a test that would fail for the actual former bug.

2. JUnit 5 and Deterministic Fixtures

JUnit Platform discovers/runs test engines; Jupiter supplies the JUnit 5 programming and extension model. Test annotations alone are insufficient if the engine or build plugin is missing/misconfigured. Keep test discovery, lifecycle and assertion policy explicit in the build.

Use focused arrange/act/assert structure and meaningful assertion messages where they improve diagnosis. Parameterized tests apply one behavioral contract to multiple inputs, such as invalid lifecycle states or boundary values. Avoid a loop that catches every exception and lets the test pass. Assert the expected failure category and important state/effects after it.

Fixtures should isolate mutable state. A static shared collection or reused database row can make tests depend on order. Setup/cleanup must remain correct when a test fails partway through. Time-dependent logic should accept a clock or controllable scheduler; asynchronous tests should use gates, completion signals and bounded deadlines rather than sleeps that happen to work locally.

Timeouts are guards, not performance assertions unless the test is explicitly a controlled performance experiment. Preemptively interrupting a test can change thread-local transaction/context behavior. A bounded future wait that fails gives evidence of no completion within that guard, not a calibrated throughput result.

[VERIFY: resolve and execute the pinned standalone JUnit/Mockito/plugin versions on Java 21, and separately verify the actual Boot-managed test stack, discovery and lifecycle behavior. The POM was parsed only; no JUnit engine or compiler ran.]

Interview checks: basic: engine discovery matters. Internals: fixtures and lifecycle define isolation. Debug: look for shared static state and cleanup after failure. Scenario: inject time and use completion signals for reproducible async cases.

3. Mockito, Fakes and Behavioral Assertions

A mock supplies configured behavior and records interactions; a stub is often described by the behavior it supplies; a fake has a working simplified implementation. These labels help communicate intent, but none establishes the real dependency's semantics. An in-memory repository can be useful and still fail to model unique constraints, collation or transaction isolation.

Mock only collaborators outside the local rule under test. If a service's entire implementation is configured away, the test may only verify Mockito. Prefer assertions on returned state and required effects. Interaction checks are valuable when the interaction is itself the contract, such as denying a tenant before accessing a repository or ensuring an invalid transition never writes.

Avoid asserting every internal call in order when the public behavior permits alternatives. Such tests break during harmless refactoring and pressure code toward an incidental implementation. Conversely, verifying no unauthorized repository interaction is a meaningful security-boundary assertion. Argument capture helps inspect a command actually sent to a dependency, but captured values need semantic assertions.

The saved lab's Repository interface exposes `find(tenant,id)` and conditional `replace(expected,next)`. Mockito can arrange `replace=false` to test the service's conflict response. It cannot prove the production adapter performs an atomic versioned SQL update; that needs an engine-backed adapter test.

Test double	Use when	Avoid when
Mock	Local orchestration needs controlled dependency outcomes	Mock success is treated as SQL/broker correctness
Fake	A small deterministic working model improves tests	Missing real constraints create false confidence
Real adapter	Behavior depends on actual serialization/storage	Every local validation case pays unnecessary infrastructure cost

Interview checks: basic: test doubles replace evidence at a boundary. Internals: verify interactions that are contractual. Debug: a green mocked repository can hide a broken query. Scenario: pair service conflict tests with real optimistic-write tests.

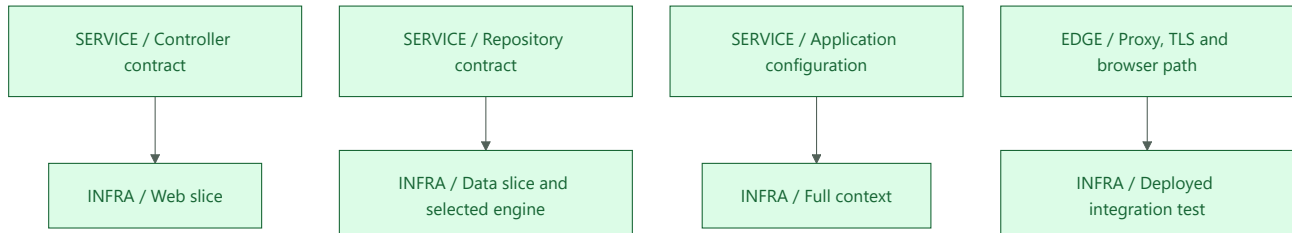
4. Spring Test Slices and Full Contexts

A web slice can exercise controller mapping, validation, serialization, exception handling and selected web/security configuration without loading every service. A data slice can exercise entity/repository integration with its configured database and transaction defaults. A full application-context test checks more wiring, but still may not include the deployed gateway, identity provider, broker or real network path.

Do not disable security filters to make a controller test pass and then claim authorization coverage. Import the required configuration deliberately and test unauthenticated, forbidden and authorized cases. MockMvc exercises a servlet application path without necessarily opening a real TCP socket; other web test arrangements exercise different boundaries. Name which one ran.

Transactional test rollback can hide commit-time failures, detach/serialization behavior and outbox publication timing. Tests of commit durability need explicit committed transactions and independent reads where appropriate. A JPA test using an embedded database can be useful for mapping but is not evidence of PostgreSQL locking, JSON operators or query plans.

Diagram 19.2



Context caching speeds tests but can conceal shared singleton state or leaked resources. Rebuilding every context is not the first solution; keep configuration stable and fixtures isolated. External property binding and startup profiles should be explicit, especially when tests might accidentally select a production endpoint.

[VERIFY: check Spring Boot test-slice packages/imports, security inclusion, database replacement, transaction defaults and context caching for the selected Boot generation. No Spring slice or full-context test ran; older tutorial imports may not match Boot 4.]

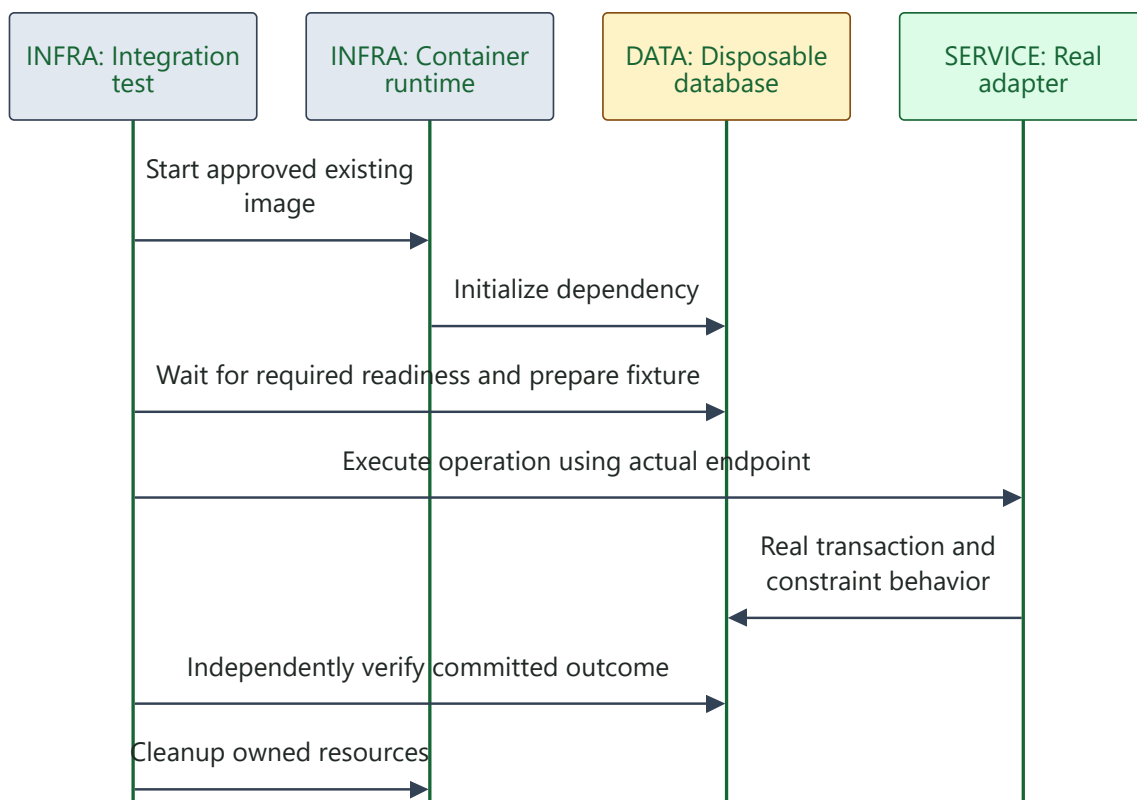
Interview checks: basic: a slice is intentionally incomplete. Internals: transaction rollback changes what is observed. Debug: inspect effective test database and security configuration. Scenario: run a real network test for proxy/TLS behavior.

5. Testcontainers and Real Infrastructure Contracts

Testcontainers can provision disposable dependencies under an available container runtime and expose their actual connection details to tests. It reduces environment drift but does not automatically make a test isolated, fast or offline. Images may need downloading; the guide's no-download restriction is still in force. No container is launched in this chapter.

Use the production engine family/version where semantics matter, initialize a minimal schema/fixture and wait for the intended service readiness rather than only a process start. Obtain dynamic ports from the container API, not a hard-coded assumption. Test ownership of lifecycle/cleanup, especially under failures and parallel execution. Container reuse changes isolation and needs an explicit policy.

Diagram 19.3



For idempotent acceptance, send concurrent requests with one logical key against a real unique constraint, then verify one durable job/result/outbox set. For consumer processing, fail before and after the local commit and verify effect/receipt consistency under redelivery. For locking, use independent connections with arranged interleavings; a single test transaction cannot demonstrate concurrent isolation.

Container health does not prove representative performance. Small disposable data can hide an index/plan issue, and host resource noise changes timing. Separate correctness tests from deliberate load experiments with documented volume, warmup and measurement. Use timeouts to avoid hanging tests, not to claim a stable production latency limit from an uncalibrated laptop run.

[VERIFY: validate Testcontainers version, runtime/image availability, readiness strategy, lifecycle cleanup and production-engine compatibility before execution. No images were pulled, containers started or database/broker tests run.]

Interview checks: basic: disposable real engines improve semantic evidence. Internals: readiness and dynamic endpoints matter. Debug: isolate data and connections per case. Scenario: control commit/ack interleavings rather than sleeping and hoping to hit the race.

6. Contract Testing and Evolution

A schema/OpenAPI document describes allowed shapes and protocol details. A consumer-driven contract captures assumptions a particular consumer needs and can be verified against the provider. Neither automatically proves every business invariant, performance property or cross-tenant permission. Contract scope should include meaningful error/status/nullability behavior, not only one happy response.

Test compatibility across deployment skew: old client with new provider, new client with still-old provider where that rollout is possible, and historical events with new consumers. Additive fields are compatible only under the agreed reader behavior. Unknown enum values, changed defaults and stricter validation can be breaking even if JSON still parses.

A provider can satisfy a stubbed contract while its production adapter fails; keep the provider verification boundary clear. Publish contracts and verification evidence with immutable versions and a deployment decision that understands which consumers exist. Do not let a stale contract repository become a substitute for actual traffic and deprecation telemetry.

Contract check	Use when	Avoid when
Schema validation	Representation shape should be enforced	Parsing success is called business correctness
Consumer contract	Independently deployed clients need explicit expectations	A generated mock is never verified against the provider
Historical event compatibility	Retained messages outlive code versions	Every fixture is generated only by today's serializer

Interview checks: basic: schema and consumer expectation are related but distinct. Internals: deployment skew defines compatibility cases. Debug: inspect nulls/enums/errors, not just added fields. Scenario: gate deployment on relevant verified contracts rather than one static document.

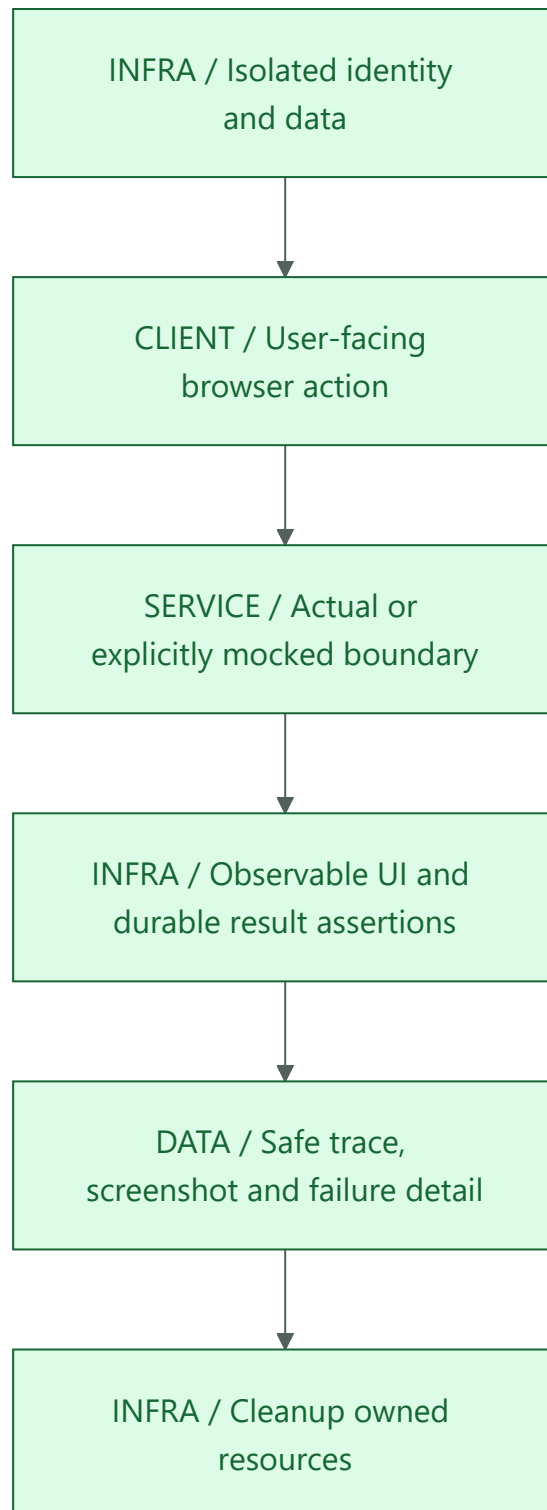
7. Playwright and Browser Journeys

Playwright exercises a browser through controlled pages/contexts, locators and assertions. Prefer user-facing roles/labels and explicit contracts over brittle CSS positions. Locator auto-waiting helps synchronize supported actions, but it does not infer arbitrary business completion. Wait for an observable result or a specific response/state, not an arbitrary sleep.

Isolate browser contexts and test identities. Persisted authentication state can contain sensitive cookies/tokens; keep it out of ordinary source control and logs. A reused tenant fixture can leak between tests or create order dependence. Seed approved disposable data through a controlled test interface or setup path, and clean only resources the test owns.

Test failure and recovery journeys: wrong tenant access, stale request after navigation, logout while work is in flight, failed submission with safe retry, expired replay cursor and reconnect through the actual gateway. Browser mocks can deterministically arrange races but remove network/server evidence. Maintain a smaller set of deployed-path tests for cookies, CORS, proxies and actual effects.

Diagram 19.4



Screenshots help visual diagnosis, but pixel equality alone does not establish accessibility or business correctness. Check keyboard navigation, focus, accessible names, error association and responsive overflow. Capture traces/screenshots on failure under a privacy policy; they can contain sensitive page data. A test-runner retry may distinguish flaky symptoms but must not erase the initial failure from quality metrics.

[VERIFY: check Playwright/browser versions, locator/assertion behavior, authentication-state handling and test-environment routing before running business journeys. Reader publication browser checks are separate; no IntegrationHub end-to-end application was deployed or tested.]

Interview checks: basic: auto-waiting is not business completion. Internals: contexts isolate browser state. Debug: inspect traces and test data ownership. Scenario: retain actual gateway coverage where the bug can only occur through it.

8. Performance, Fault and Concurrency Tests

A unit test deadline guards liveness; a performance test measures a workload under controlled conditions. Keep them distinct. Define target operations, arrival model, payload/data distribution, warmup, duration, environment, errors and acceptance criteria. Record offered and achieved throughput, percentiles and resource use. [18](#) explains coordinated omission and generator limits; [23](#) explains JMH for local JVM questions.

Concurrency tests should arrange the contested boundary using latches/barriers or deterministic schedulers where possible. Assert an invariant over results, not an imagined thread order. Repeating a test many times can reveal a race but a pass is not a proof of all interleavings. Use specialized tools or model checking when risk and complexity justify them, without pretending the local environment ran them.

Fault tests need a named hypothesis and authorized blast radius. Injecting failure before transaction commit differs from failure after commit before ack. Test the durable state from an independent observer and verify retry/cleanup. A mocked exception at the wrong layer may not reproduce the production failure at all.

[M] MECHANISM

Make the failure happen at the boundary the invariant depends on. If the bug is commit-time, a method stub throwing before any transaction starts tests another scenario. Arrange the actual commit or use a model whose narrower meaning is explicit.

Interview checks: basic: time guard is not benchmark. Internals: interleaving placement determines the test. Debug: inspect what work actually executed before failure. Scenario: verify durable effects independently after injected faults.

9. Quality Gates and Flaky Tests

Gates should reject a specific unacceptable risk or supply a decision-relevant signal. Compilation, unit tests, adapter integration, contract compatibility, static analysis, security/dependency policy and deployment health have different scopes. Preserve each result and skipped reason. A mandatory check unavailable because tooling is missing is NOT EXECUTED, not silently successful.

Coverage reports execution of instrumented code, not assertion quality. A test can execute a branch and fail to check its result. Mutation testing can reveal some weak assertions by changing behavior and seeing whether tests fail, but surviving/killed mutants still require interpretation. Avoid chasing one percentage while critical failure cases remain absent.

Flakiness often comes from shared state, uncontrolled time, order assumptions, resource contention or external services. Quarantine a flaky test only with an owner and expiry/repair plan; do not permanently remove coverage to restore a green dashboard. Reruns should remain visible so reliability of the test suite can be measured.

[!] TRAP

A green command with zero discovered tests is not a test pass. Configure discovery checks and inspect test counts. Likewise, a source parser, mocked adapter and real system test cannot all be reported as the same kind of validation.

Gate	Use when	Avoid when
Fast mandatory checks	Every change needs quick reliable feedback	Slow environmental noise blocks unrelated local work unnecessarily
Boundary integration gate	A change can break storage/protocol behavior	Mock-only tests replace required real-engine evidence
Flaky-test quarantine	Temporary isolation protects delivery while repair is owned	The missing test is forgotten indefinitely

Interview checks: basic: coverage is not correctness. Internals: a gate's scope should be auditable. Debug: track initial failures and discovery counts. Scenario: choose a repair that reduces nondeterminism without weakening assertions.

10. Saved JUnit/Mockito Fixture

The complete sources are `code/19-testing/src/main/java/guide/testing/JobService.java` and `code/19-testing/src/test/java/guide/testing/JobServiceTest.java`. The POM is standalone and pins its test tools deliberately. The service cancels only an ACCEPTED job in an authorized tenant, creates the next version and requires conditional repository replacement to succeed.

Four test methods describe five planned cases: tenant denial before repository interaction; accepted cancellation with expected version; optimistic conflict; and two non-ACCEPTED states through parameterization. These are meaningful local assertions, but the repository is a mock. The fixture cannot establish database locking, transactions, Spring security, external cancellation or provider effects.

Windows PowerShell from workspace root: `& ' ./docs/java-fs-guide/code/19-testing/run.ps1 '`. Optional `-JdkHome` uses an existing JDK. The runner parses XML, checks source presence and uses Maven `-o -B test` if tools exist. Surefire is configured to fail if no tests are found. It never downloads a JDK or switches to online Maven.

The actual preflight recorded `staticStatus` PASS for the POM and source files and overall NOT EXECUTED with zero tests run. Compilation, dependency compatibility, Mockito instrumentation and Jupiter assertions remain unverified. A future PASS must be read alongside the actual Surefire results and scope.

[D] DECISION

Spend infrastructure where it can falsify a real assumption. Use a pure unit test for transition rules, a database for conditional-write semantics and a deployed browser path for cookie/proxy behavior. Do not make every test large or every dependency imaginary.

[?] INTERVIEW

For each test, name what stays real and what is replaced. Then state the invariant asserted and the important behavior it cannot prove. That is stronger than listing test frameworks or a coverage percentage.

Explained-Here Index and Sources

Concept	Explanation
Test pyramid	Risk and scope
JUnit 5	Discovery and fixtures
Mockito	Doubles and assertions
Test slices	Spring boundaries
Testcontainers	Real adapters
Contract testing	Evolution
Playwright	Browser journeys
Performance testing	Measurement and faults
Quality gates	Evidence and flakiness

Verification destinations: [JUnit 5 guide](#), [Mockito](#), [Spring Boot testing](#), [Testcontainers Java](#) and [Playwright](#). These are reference pointers, not resolved local dependencies.

Related Chapters

Use [00](#), [02](#) concurrency, [03](#) Spring, [04](#) ORM, [05](#) APIs, [06](#) database, [07](#) messaging, [08](#) security, [09](#) browser, [16](#) gates, [18](#) reliability and [23](#) benchmarking. Generated links are reciprocal.

RELATED CHAPTERS / RECIPROCAL LINKS

00 The System Before the Details	09 The Browser and the Java Backend
01 Core Java and the JVM	25 Architecture Styles and Domain Design
02 Concurrency and the Memory Model	11 Object Design, Patterns and LLD
03 Spring and Spring Boot Internals	12 Twelve System Design Case Studies
04 Hibernate and JPA	27 Fintech and Payments Engineering
06 Databases and Durable State	13 Data Integration and iPaaS
05 APIs and Real-Time Communication	16 Build, Ship and Roll Back
07 Messaging, Batch and Streams	18 Observability and Reliability
26 Data Platform Basics	23 JVM Performance Engineering
08 Security, Identity and Data Protection	24 IntegrationHub in Motion
	20 Appendix: Interview Toolkit and Index

One-Page Cheat Sheet

Scope: unit rules, framework wiring, real dependency semantics and deployed journeys need different evidence. A test double cannot validate the behavior it replaces.

JUnit/Mockito: ensure engine discovery, isolated fixtures and meaningful assertions. Inject clocks and arrange concurrency. Verify contractual interactions, not every incidental call. Fail gates when zero required tests are discovered.

Spring/infrastructure: slices are intentionally partial. Test rollback can hide commit failures. Use the actual engine family for locks/dialects and controlled container lifecycle with approved images.

Contracts/browser: test deployment skew, historical events, errors and nullability. Browser contexts isolate state; locators and observable completion beat arbitrary sleeps. Keep a real gateway path for cookie/CORS/streaming behavior.

Quality: coverage is not assertion strength, retries do not erase flakiness, and deadlines are not benchmarks. The saved five-case JUnit fixture remains NOT EXECUTED; POM/source checks passed only.

Interview Corner

Basic: What Should Be Mocked?

Dependencies outside the local contract when controlling them isolates the rule. Do not mock the database if the question is whether the SQL constraint works.

Internals: Why Can Transactional Tests Hide Failures?

Automatic rollback and one persistence context can bypass real commit timing, independent visibility and detach/serialization paths. Exercise explicit commits and independent observers for those contracts.

Trace/Debug: CI Passed With No Tests.

Inspect engine/plugin discovery, naming and task configuration. Require nonzero discovery for mandatory suites and retain reports rather than trusting the exit code alone.

Scenario: How Would You Test Concurrent Idempotent Submission?

Arrange callers around the same logical key against a real unique constraint, then inspect durable job/result/outbox rows. A map-based model can teach the idea but not prove database wiring.

Basic: Slice Test Versus Full Context?

A slice loads a targeted framework surface; a full context loads broader application wiring. Neither automatically includes actual proxies, networks or remote infrastructure outside that process.

Internals: Why Is a Fake Not a Real Database?

It can omit locking, transaction timing, collation, constraints and SQL behavior. Pair it with adapter tests for the assumptions it cannot express.

Trace/Debug: Browser Test Passes Only With Sleeps.

Identify the real completion condition and test data/lifecycle race. Replace timing guesses with observable assertions or controlled signals, and inspect failure traces.

Scenario: When Is Testcontainers Worth Its Cost?

When engine/protocol semantics are the risk and disposable real dependencies make the test reproducible. It still requires approved images, runtime resources and clear cleanup.

Basic: Does High Coverage Mean Good Tests?

No. It records execution, not whether meaningful outcomes were asserted. Inspect failure cases, invariants and mutation evidence where useful.

Internals: What Does a Consumer Contract Prove?

The verified provider satisfies specified consumer expectations under the verification setup. It is not universal business correctness or evidence for every deployed adapter.

Trace/Debug: A Retry Made the Test Green.

Preserve the first failure and investigate shared state, time, external dependencies or contention. Rerunning can diagnose flakiness but must not silently waive a regression.

Scenario: How Do You Report Missing Tools?

State NOT EXECUTED, identify which checks did run and list the blocked scope. Do not relabel parsing or source review as compilation or integration success.

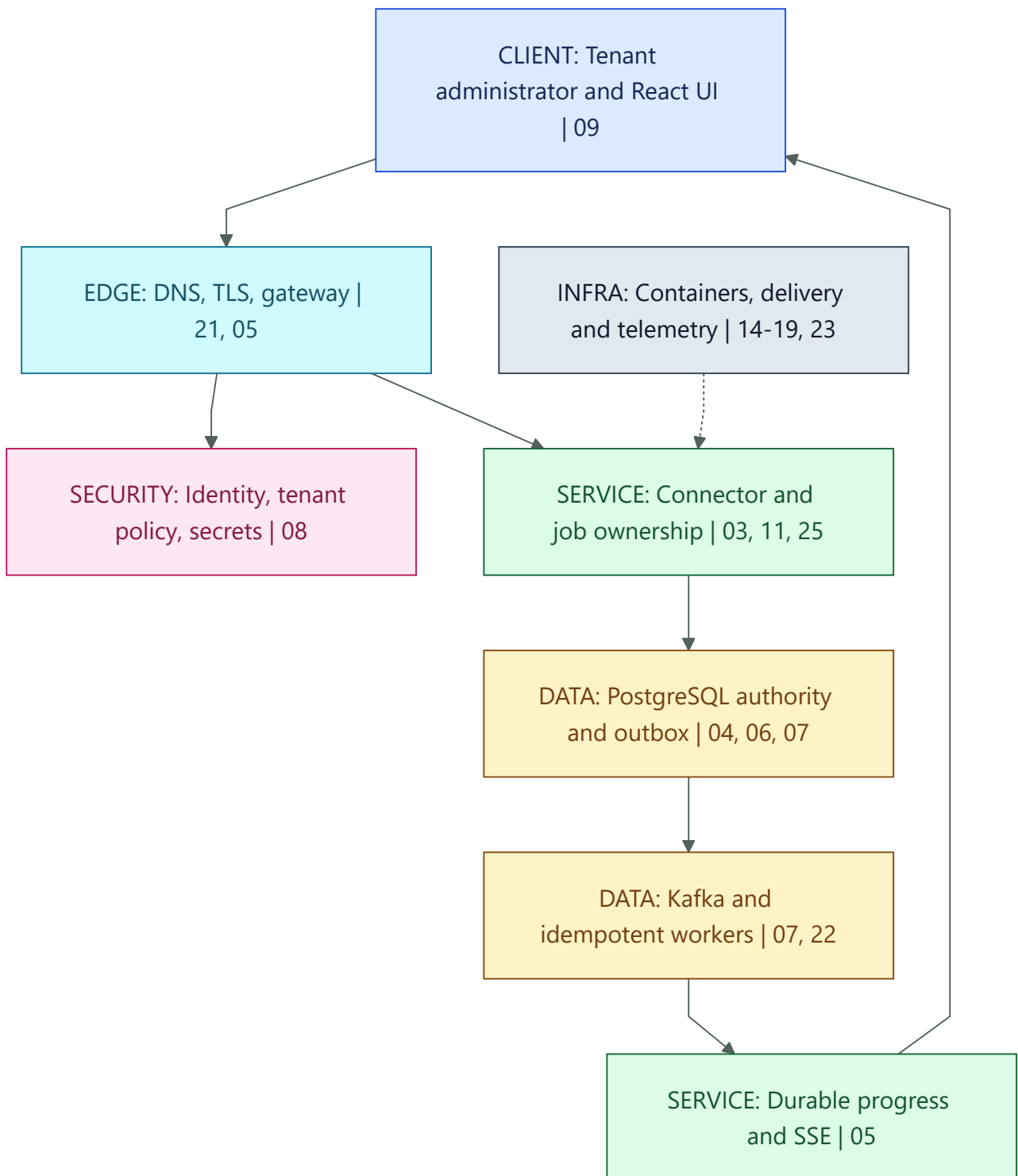
24 / IntegrationHub in Motion

This is Part A: one connected system, not another theory chapter. IntegrationHub is the fictional multi-tenant platform used throughout the guide. Follow one accepted job from identity to durable state, delivery, live progress and operations. Each stop points back to the chapter that explains its mechanism.

Version assumptions: reuse the guide's Java 21, Spring, PostgreSQL 17, Kafka 4.x and Kubernetes assumptions with their original verification markers. This chapter introduces no new compatibility claim. **Execution status:** the story is a design walkthrough, NOT EXECUTED as an integrated application. Chapter-level models are not an end-to-end deployment. Part B, `integrationhub-lite`, is not built without an explicit `lite` request.

Big Picture

Diagram 24.1



What You Will Be Able to Explain

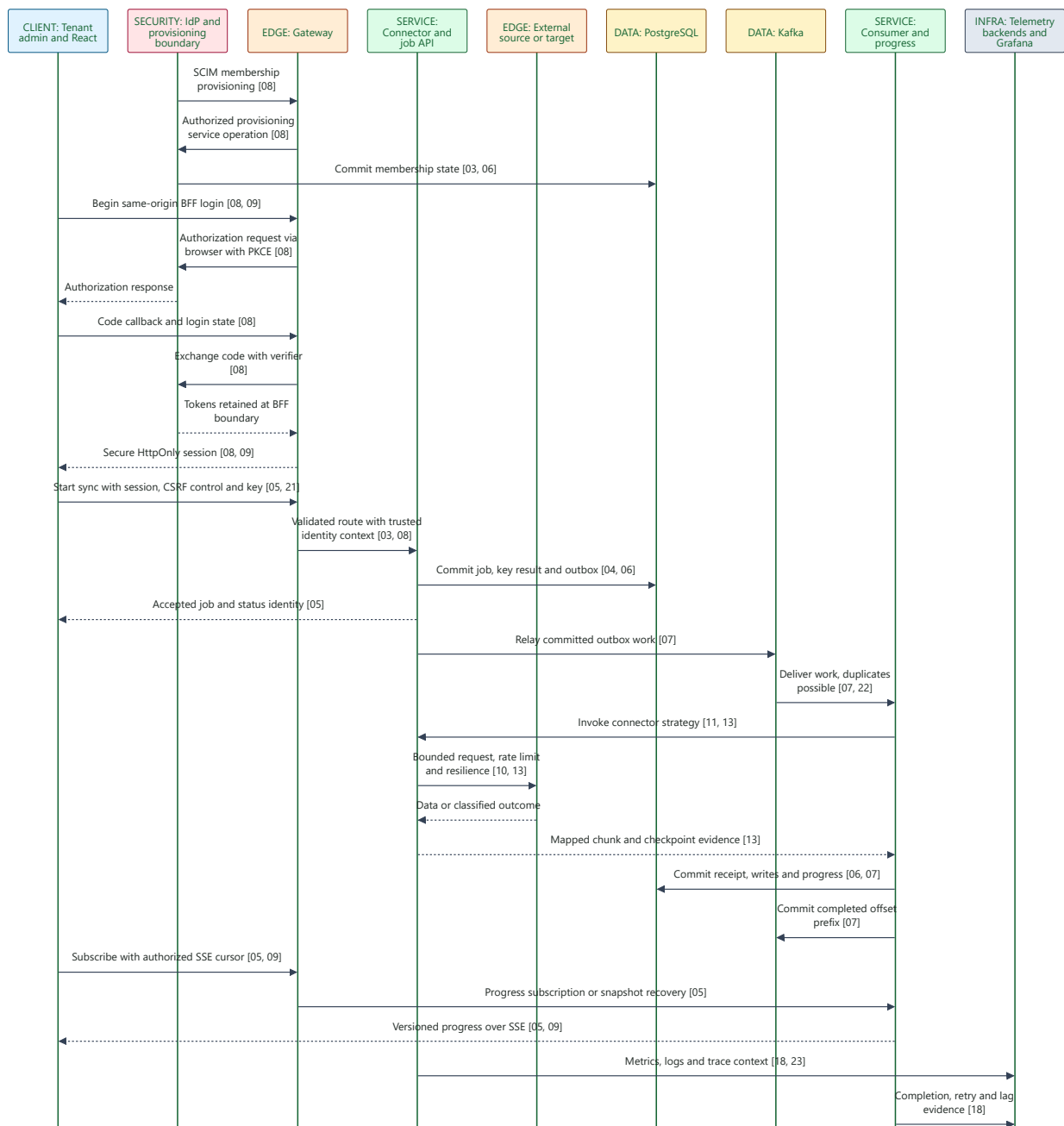
- Narrate one accepted sync without confusing login, authorization, commit, delivery and completion.
- Connect a deployment to probes, business evidence and rollback compatibility.
- Walk from a production symptom to evidence, the responsible boundary and a repair.
- Explain what could saturate at 10x traffic under explicit assumptions.
- Describe refresh-token reuse containment without promising immediate JWT revocation.
- Draw the whole system and explain it in approximately five minutes.

[P] PRODUCTION

Where this lives in IntegrationHub. This is the whole fictional system from 00. The diagrams show responsibilities, not a claim that every box must be a separately deployed microservice. Architecture ownership comes from 25; 20 supplies revision routes and the final index.

1. A Day in the Life

Diagram 24.2



Onboard: an authorized enterprise provisioning client creates or changes membership through SCIM. SCIM does not log the user in. An invitation, tenant membership and permission to configure a particular connector remain separate checks. See [08 identity and provisioning](#).

Log in: React starts the selected Authorization Code + PKCE flow through the same-origin gateway/BFF. The BFF validates the login transaction, exchanges the code and retains tokens; the browser receives a secure HttpOnly session. State-changing cookie requests need CSRF protection, and SSE does not put bearer tokens in URLs. Downstream APIs still validate

authority and resource ownership. PKCE protects the code exchange, not every later use of a stolen token. UI state belongs to the active identity generation so an old request cannot populate a new tenant's screen. See [09 browser state](#).

Accept: DNS, TCP/TLS and the edge get the request to the gateway (21). The gateway routes and applies its controls; the service still authorizes the requested tenant resource. A local transaction commits the job, bound idempotency result and outbox. The response means accepted, not completed. The exact transaction and entity mechanics live in [03](#), [04](#) and [06](#).

Execute: committed work reaches Kafka through the outbox relay. A worker uses the connector strategy with tenant-aware rate limits, time budgets, backoff and circuit-breaker policy. A timeout can leave an external effect unknown; the connector contract determines how to recover it. Chunk writes, duplicate receipt and checkpoint progression follow [07](#) and [13](#). Payments need the stricter reconciliation treatment in [27](#).

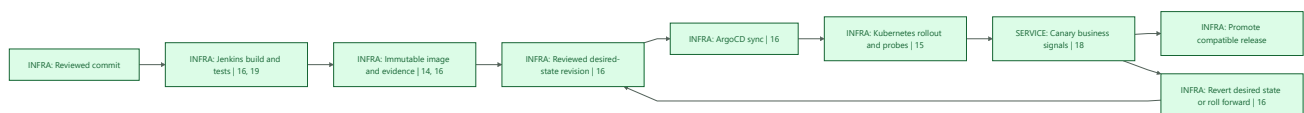
Observe: durable progress backs live SSE. A reconnect either replays retained events or fetches a versioned snapshot before resuming. React ignores obsolete generations and versions. A slow browser does not block shared workers. Metrics, traces and logs reach appropriate backends; Grafana queries them rather than being assumed to store every signal itself. See [05](#), [09](#) and [18](#).

[M] MECHANISM

There are several acknowledgements. API acceptance follows the job commit. Broker acknowledgement concerns delivery storage. Consumer completion follows the relevant effect commit. SSE delivery only tells a client what it observed. None is a substitute for the others.

2. The Deploy Story

Diagram 24.3



Jenkins checks the reviewed source, runs the appropriate tests, and produces a versioned artifact and image. Evidence binds to the image digest; it does not apply to whichever mutable tag is current tomorrow. A reviewed configuration change selects that digest. ArgoCD

reconciles declared state; Kubernetes creates and replaces Pods. These boundaries are explained in [14](#), [16](#) and [19](#).

Startup, readiness and liveness checks answer different questions. A ready process can still double-publish, reject tenants or make slow queries. A rolling release therefore needs business signals and compatible schema. For a weighted canary, explicitly supply the rollout or routing mechanism; a plain Deployment is not assumed to provide percentage traffic control. See [15](#) and [18](#).

Rollback changes desired state back to a known compatible artifact, not just an imperative Pod mutation that GitOps will overwrite. If new code wrote a schema the old code cannot read, rolling forward or restoring compatible behavior may be necessary. Expand/contract migrations keep both versions viable through the overlap. **[VERIFY: validate the actual Jenkins, ArgoCD, rollout-controller/routing, Kubernetes probe and schema-compatibility configuration before performing this deployment story. No integrated build, rollout, canary or rollback was executed.]**

3. Five Failure Stories

Downstream Timeout and Circuit Breaker

Symptom: sync jobs stay active while retries increase. **Detect:** per-provider timeout counts, queue age and traces show where time is spent. **Root-cause path:** follow one attempt through DNS/connect/read timing and provider evidence; distinguish overload from expired credentials or bad mapping. **Fix:** bound retries and concurrency, respect rate limits, stop new work through the breaker where appropriate, and reconcile ambiguous external writes before retrying. **Chapters involved:** [10 resilience](#), [13 connector recovery](#), [18 signals](#), [21 network timing](#).

Duplicate Request and Idempotency

Symptom: a user double-clicks or retries after losing a response. **Detect:** correlate logical key, tenant, content fingerprint and job ID; transport request IDs may differ. **Root-cause path:** verify the unique key and job commit share a transaction, and that key expiry did not silently permit new work. **Fix:** replay the stored accepted result for identical content, reject changed content, and repair existing duplicates through an explicit domain workflow. **Chapters involved:** [05 request identity](#), [06 transaction constraints](#), [07 duplicate effects](#).

Pod OOMKilled

Symptom: a worker disappears and Kubernetes reports OOMKilled. **Detect:** termination evidence, container memory, restart counts, native/direct usage and workload size. **Root-cause path:** compare process memory with container budget; distinguish heap retention, direct buffers, thread stacks and a large in-flight batch. **Fix:** cap admission and batch size, correct retention or resource configuration, replay from the committed checkpoint, and profile an authorized representative workload. Increasing heap alone can reduce native headroom. **Chapters involved:** 14 cgroups, 15 resource troubleshooting, 23 JVM evidence, 07 restart.

Kafka Consumer Lag

Symptom: API acceptance is fast but completion falls behind. **Detect:** lag by partition, oldest work age, worker error rate and sink latency. **Root-cause path:** inspect hot keys, slow database calls, poison input, rebalance churn and retry loops before adding consumers. **Fix:** remove the bottleneck, quarantine under policy, adjust partition/worker capacity only where ordering permits, and scale within sink limits. Preserve the completed offset prefix. **Chapters involved:** 07 ordering, 06 pooling, 18 capacity, 22 partitioning.

Schema Migration Mismatch across Regions

Symptom: one region rejects writes or cannot decode new events. **Detect:** schema versions, migration history, deploy revision and region-labeled errors. **Root-cause path:** compare actual database and application compatibility, event schemas and traffic routing; do not assume a successful migration job ran everywhere. **Fix:** stop the incompatible rollout, restore a compatible route or code path, apply the approved additive change, reconcile partial work, and delay destructive contraction until every consumer is ready. **Chapters involved:** 06 expand/contract, 16 rollback, 17 regional boundaries, 22 failure models.

[!] TRAP

Restarting is not the root-cause explanation. For every story, identify the violated budget or contract and the durable recovery point. A restart can restore service while leaving duplicate work, lost evidence or the same saturation trigger intact.

4. The 10x Traffic Story

This ordering is conditional, not a measured forecast. Assume the database and provider budgets stay fixed, average job size stays similar, and external calls already dominate service time. Ten times the request rate does not create ten times the downstream capacity.

First: admission and provider concurrency. Requests accumulate faster than the provider permits. Bound per-tenant accepted work and active calls, return an explicit overload policy, and prioritize existing work. Queue depth without an age budget only delays the failure. Use [10](#) and [02](#).

Second: database and consumer throughput. More workers reach the same connection pool, hot rows and I/O budget. Measure transaction duration, pool waits, query plans and partition skew; tune access and batching before raising connection counts. Add workers only when the sink has capacity. Use [06](#), [07](#) and [23](#).

Third: progress fan-out and telemetry. More jobs and browsers amplify SSE updates and high-cardinality labels. Coalesce allowed progress states, bound per-client buffers, preserve snapshot recovery and reduce unbounded label dimensions. Analytical reporting can lag under its own contract. Use [05](#), [18](#) and [26](#).

[D] **DECISION**

Scale the limiting resource, not the most visible box. If the bottleneck is provider quota, more Pods increase retries. If it is one hot key, more Kafka consumers do not split its ordered stream. If it is one query, more connections can worsen contention. Revise the order above when measured evidence contradicts its assumptions.

5. Stolen or Reused Refresh Token

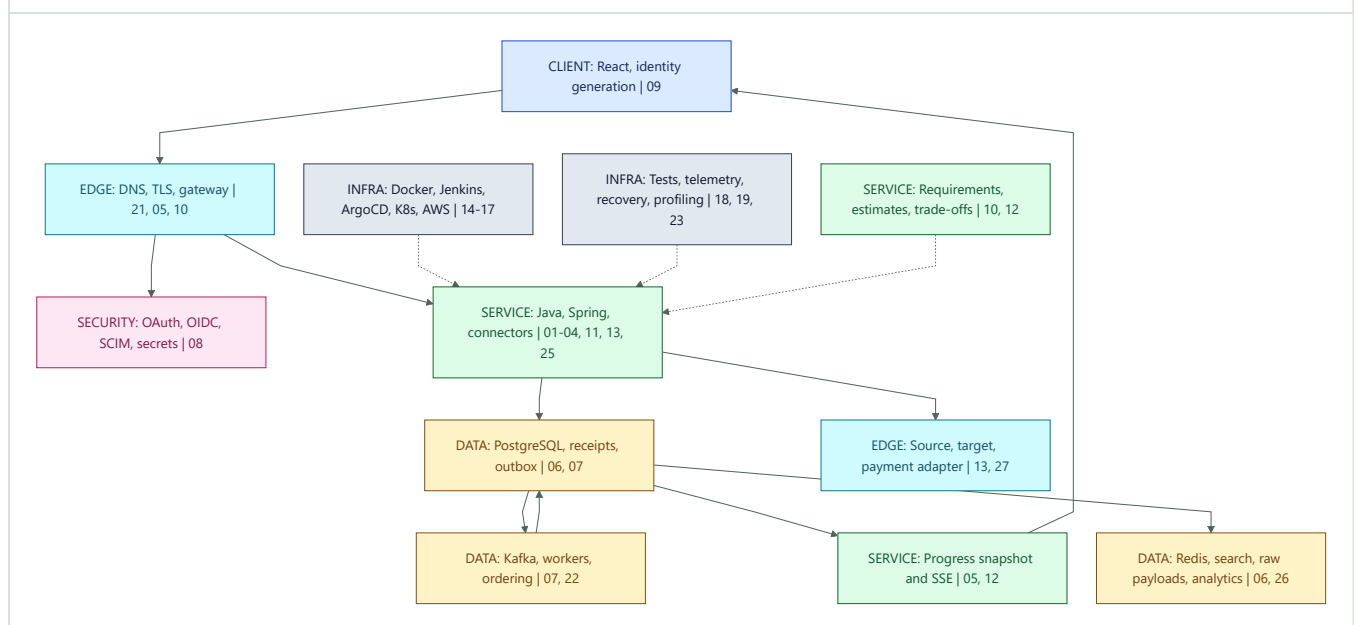
With a rotation-and-reuse-detection policy, a successful refresh replaces the old credential in a tracked token family. Later use of an invalidated old credential can signal theft or a client race. Record the event without logging tokens, preserve relevant session/client context, and apply the issuer's family revocation and reauthentication policy. Contain exposed sessions and investigate access already granted; do not assume revoking a refresh token instantly invalidates every issued JWT.

Already-issued access tokens remain subject to resource-server validation and revocation strategy. Depending on the design, containment can include short remaining lifetime, session/version checks, introspection or deny controls, alongside credential cleanup and user notification. UI logout clears active state and subscriptions but does not undo external effects. See [08](#) and [09](#).

[VERIFY: verify the selected issuer's refresh rotation, reuse detection, concurrent refresh handling, family revocation and access-token containment behavior against current OAuth security guidance and provider configuration. This security story was not exercised against an identity provider.]

6. Whole System on a Whiteboard

Diagram 24.4



This compact poster groups auxiliary stores for legibility. Redis is a cache or coordination tool under explicit contracts, Elasticsearch is a derived search view, and MongoDB can retain selected raw payloads under access and retention policy. None replaces the job authority merely by appearing in the same box.

7. Five-Minute Spoken Script

0:00-1:00 / Goal and boundaries. "IntegrationHub is a fictional multi-tenant integration platform. A tenant configures a connector and starts a bulk sync from a React UI. My first concern is ownership: the authenticated user must belong to the tenant and be authorized for that connector. SCIM manages membership; OAuth and OIDC support login and API access. Those are different responsibilities. DNS, TLS and the gateway deliver the request, but service-level authorization remains mandatory. I would begin with clear modules and extract services only when independent deployment, scaling or failure isolation justifies the cost."

1:00-2:00 / Acceptance and durable work. "Starting a sync carries an idempotency key bound to the intent. In one PostgreSQL transaction the service stores the job, its accepted result and an outbox record. Only then do I return acceptance. This is not a promise that the sync has completed. A relay sends committed work to Kafka. If it crashes after publishing but before recording progress, it can publish again; the consumer therefore needs duplicate-safe effects. The important unit is the business identity and transaction boundary, not an assumption that the transport delivers once."

2:00-3:00 / Execution and progress. “A worker invokes the selected connector strategy with bounded concurrency, rate limits and a time budget. Retries use classified errors, backoff and jitter; a circuit breaker prevents continued submissions into a failing dependency. An ambiguous external write is recovered rather than assumed to have failed. Mapped chunks, receipts and durable progress are committed before advancing the relevant checkpoint. The UI subscribes over SSE and can recover from a retained cursor or a versioned snapshot. Slow clients have bounded buffers and cannot hold up shared processing.”

3:00-4:00 / Delivery and diagnosis. “Jenkins builds and tests a reviewed revision, produces an immutable image and binds evidence to it. A reviewed desired-state change lets ArgoCD reconcile Kubernetes. Probes manage process startup, routing and restarts, but I also watch completion latency, errors and lag before promotion. Rollback requires compatibility with current data, so migrations expand first and contract later. In an incident I follow one request and job identity through traces, logs and metrics, compare demand with resource budgets, and preserve evidence before changing capacity.”

4:00-5:00 / Trade-offs and recovery. “At higher traffic, the first bottleneck depends on measured service time and downstream limits. More Pods cannot overcome a fixed provider quota or a serialized account. I separate authoritative state from caches, search and analytics, and state how stale each view may be. Payment integrations need balanced journals, stable provider identity and independent reconciliation because a timeout is not a financial result. My final checks are tenant isolation, duplicate and crash recovery, restore evidence and operational ownership. The diagrams are a proposed design; I distinguish local model tests from an actually deployed end-to-end system.”

The timing is an approximate rehearsal allocation, not a measured speaking result. Adjust pacing instead of skipping the commit and unknown-outcome boundaries.

8. Technology Poster

Technology or concept	Problem it solves	Chapter
Java, collections, JVM and concurrency	Implement and schedule bounded application work	01, 02
Spring Boot, MVC, proxies and transactions	Wire the service and define request and transaction interception	03
Hibernate and JPA	Map object state to persistence operations deliberately	04
REST, OpenAPI and SSE	Publish contracts and recoverable live progress	05
PostgreSQL and Liquibase	Durable state, constraints and compatible schema evolution	06
Redis, Elasticsearch and MongoDB	Explicit cache, search and raw-data roles	06
Kafka, outbox and Spring Batch	Decouple work while preserving recovery identities	07

Technology or concept	Problem it solves	Chapter
OAuth, OIDC, PKCE, SCIM, TLS and secret management	Establish identity, provision membership and protect data	08
React and browser request generations	Keep the visible state attached to the current user and tenant	09
Resilience4j and capacity budgets	Bound retries, calls and failure propagation	10
Strategy, Adapter, state and design method	Turn requirements into testable component and system boundaries	11, 12
IICS, MuleSoft, Anaplan and connector contracts	Integrate external systems with mapping and restart rules	13
Docker and Kubernetes	Package workloads and reconcile resource-constrained execution	14, 15
Jenkins, ArgoCD, Harness and feature flags	Build evidence and control promotion and exposure	16
AWS and Terraform	Define infrastructure, identity and recovery responsibilities	17
OpenTelemetry, Prometheus, Grafana and Loki	Relate user outcomes to metrics, traces and logs	18
JUnit, Mockito, Testcontainers and Playwright	Test progressively wider contracts and user journeys	19
DNS, TCP, Linux and event loops	Explain transport, resource limits and waiting	21
Consensus, fencing and consistency models	State distributed guarantees and failure assumptions	22
JFR, profilers and JMH	Investigate JVM cost with scoped evidence	23
Bounded contexts, ports and ADRs	Preserve ownership and reasons for decisions	25
CDC, facts, dimensions and lineage	Publish trustworthy analytical views	26
Journal, payment identity and reconciliation	Preserve financial intent and resolve external evidence	27

[?] INTERVIEW

Draw less, explain the boundaries. Choose one user action, name its authoritative commit, show where duplication can happen, and explain how an operator proves completion. Add a technology only when it solves a stated problem.

Concepts Explained Here

The [day in the life](#), [deploy story](#), [failure stories](#), [10x traffic](#), [security story](#), [system whiteboard](#), [spoken script](#) and [technology poster](#) connect earlier explanations rather than introduce new theory.

Related Chapters

Start at [00](#); the journey and poster above link every technical chapter. Finish with [20 toolkit and index](#). The build generates reciprocal links so any chapter can return to this synthesis.

RELATED CHAPTERS / RECIPROCAL LINKS

00	The System Before the Details	05	APIs and Real-Time Communication
26	Data Platform Basics	18	Observability and Reliability
25	Architecture Styles and Domain Design	14	Docker and Container Internals
27	Fintech and Payments Engineering	16	Build, Ship and Roll Back
20	Appendix: Interview Toolkit and Index	19	Testing and Quality
08	Security, Identity and Data Protection	15	Kubernetes and Production Workloads
09	The Browser and the Java Backend	10	System Design Foundations
21	Networking and Operating Systems	23	JVM Performance Engineering
03	Spring and Spring Boot Internals	22	Distributed Systems Theory
04	Hibernate and JPA	17	Cloud and Infrastructure
06	Databases and Durable State	02	Concurrency and the Memory Model
07	Messaging, Batch and Streams	01	Core Java and the JVM
13	Data Integration and iPaaS	11	Object Design, Patterns and LLD
		12	Twelve System Design Case Studies

One-Page Cheat Sheet

Journey: provision, authenticate, authorize, commit job and outbox, deliver, deduplicate, execute, commit progress, replay to the UI.

Deploy: reviewed source, tested artifact, immutable image, desired-state change, reconciliation, probes, business signals, compatible rollback.

Failure: symptom, evidence, violated boundary, bounded repair, durable recovery point. A timeout is not failure proof; a restart is not an explanation.

Scale: name assumptions and measured bottlenecks. Admission, external quotas, database limits and progress fan-out have different controls.

Security: detect credential reuse, contain the token family under issuer policy, address already-issued access, clear client state and preserve audit evidence.

Whiteboard: identity, edge, service, durable state, transport, external effects, progress, delivery and operations. Part A is a design synthesis; Part B is not built.

Interview Corner

Basic: Explain the System in One Minute.

An authorized tenant starts a sync; a durable job and outbox record establish acceptance; duplicate-tolerant workers perform bounded connector work; committed progress reaches a recoverable UI stream; delivery and observability protect that flow.

Internals: Where Is the Most Important Boundary?

The answer depends on the question. For acceptance it is the database commit; for external effects it is the provider's accepted operation; for tenant safety it is resource authorization. State which guarantee you mean.

Trace/Debug: The UI Says Running Forever.

Check durable job state, worker and provider progress, consumer lag, SSE cursor retention and UI generation. Do not assume a stale stream means unfinished durable work.

Scenario: Roll Back a Bad Release.

Stop promotion, preserve evidence, verify old-code compatibility with current data, change desired state or roll forward safely, and validate user outcomes after controller convergence.

Scenario: What Changes at 10x?

Recalculate budgets from demand and service time, identify the first saturated resource, and test the proposed change. Ten times the Pods is not a capacity model.

Scenario: What Has Actually Been Proven Here?

Chapter-specific local models and publication checks have their recorded evidence. This connected application, its deployment and its failure drills were not executed. The distinction is part of a credible design explanation.

20 / Appendix: Interview Toolkit and Index

Use this appendix to retrieve, compare and explain. It was written after the technical chapters and the Part A capstone. It does not replace their mechanisms or turn local model checks into production evidence. IntegrationHub remains fictional.

Version assumptions: inherit each linked chapter's version and verification notes. A revision table is not a new claim of cross-version compatibility. **Execution status:** this appendix contains no application implementation; its catalog-derived index and internal links are publication-checked. Java, provider and deployment exercises retain their original NOT EXECUTED status.

Big Picture

Diagram 20.1



What You Will Be Able to Explain

- Compare technologies from constraints rather than preference.
- Build a 30-day revision loop covering every chapter, including payments and the capstone.
- Retrieve precise definitions and distinguish commonly confused terms.
- Find every registered concept at its owning explanation.
- Describe what was executed, what was only reviewed and what remains uncertain.

[P] PRODUCTION

Where this lives in IntegrationHub. Every revision exercise returns to one fictional job, one tenant boundary or one failure. Use [24](#) to join the pieces, and [00](#) to recover the recommended reading order. Do not present these examples as personal employment experience or a real company's design.

1. X versus Y Decision Tables

The choices below are starting rules, not universal winners. "Avoid" means the stated requirement conflicts with that option, not that the tool is inherently bad. Follow the source link for the mechanism, failure modes and version qualifications.

Runtime, Application and Data

Choice	Use when	Avoid when	Explain through
Value object versus entity	Value object for immutable value semantics; entity for identity across change	Mutable equality keys or identity inferred from all current fields	01 , 25
synchronized versus atomics	Monitor for a compound critical section; atomics for suitable single-state transitions	A multi-field invariant is split across unrelated atomics	02
Platform versus virtual threads	Platform workers for bounded CPU work; virtual threads for supported blocking workloads	Cheap waiting is mistaken for more database or provider capacity	02 , 23
Explicit Spring configuration versus auto-configuration	Explicit beans for owned policy; defaults for verified conditional wiring	Hidden conditions or overrides make the actual runtime graph unclear	03
Optimistic versus pessimistic locking	Version checks for tolerable conflict; locks for required serialization under a bounded transaction	Retries hide external effects or long locks hold scarce resources	04 , 06
Offset versus keyset pagination	Offset for shallow arbitrary page access; keyset for stable indexed traversal	Concurrent shifts or deep skipping violate the API's promised behavior	05
Polling versus SSE versus WebSocket	Polling for simple bounded freshness; SSE for server updates; WebSocket for justified bidirectional messaging	Persistent connections have no replay, authorization or backpressure contract	05
Blocking MVC versus reactive processing	Blocking style for understandable bounded request work; reactive for demand-aware pipelines and supported nonblocking dependencies	Blocking calls occupy event loops or virtual threads are claimed to provide backpressure	05
PostgreSQL versus document storage	Relational invariants and joins versus an appropriately bounded document model	Product labels substitute for access-pattern and consistency analysis	06
B-tree versus LSM design	Evaluate point/range access, write patterns and maintenance costs under the selected engine	A generic write/read slogan replaces actual compaction or index evidence	06
Cache versus authoritative store	Cache for reconstructible acceleration with a staleness policy; authority for accepted facts	A stale cache authorizes spending or proves durable completion	06 , 27
Kafka versus RabbitMQ	Retained partitioned log versus routed broker work under the chosen product semantics	Ordering, retention, acknowledgements and replay remain unspecified	07
Outbox versus direct dual write	Outbox for local state plus publication intent; direct calls only with explicit gap recovery	Two independent writes are described as atomic	07

Choice	Use when	Avoid when	Explain through
Event sourcing versus state storage	Authoritative event history when reconstruction and domain history justify it; current state otherwise	Replay, event evolution and operational costs are not owned	07
Batch versus streaming	Batch for bounded inputs and relaxed freshness; streaming for justified continuous decisions	Low latency is pursued without late-data or replay policy	07, 26
ETL versus ELT	Transform before load for minimization; transform after controlled landing for governed reprocessing	Raw sensitive data lands without permission or source replay is assumed	13, 26
Warehouse versus lake versus lakehouse	Governed analytical tables, retained diverse objects, or table-managed object data	Terminology is treated as a quality or transaction guarantee	26

Security, Architecture and Delivery

Choice	Use when	Avoid when	Explain through
OAuth versus OIDC versus SCIM	Delegated API authority, login identity and provisioning respectively	One protocol is assumed to perform all three roles	08
JWT local validation versus introspection	Local validation for bounded independent checks; introspection for an online authority contract	Immediate revocation is promised without a mechanism or online dependency budget	08
React local versus shared state	Local state for component-owned interaction; shared state for real cross-view ownership	A global store hides request identity and tenant-generation races	09
Retry versus circuit breaker versus bulkhead	Retry recoverable attempts, stop unhealthy submissions, isolate resource budgets	Any is described as undoing an accepted remote effect	10
Strategy versus Adapter	Strategy for interchangeable behavior; Adapter for translating an external interface	Patterns are added without a changing behavior or incompatible boundary	11
Modular monolith versus microservices	One deployable with enforced modules versus justified independent operations	Distribution is added before ownership and data contracts exist	25
Leader lease versus fencing	Lease for leadership policy; effect-side fence to reject obsolete epochs	Permission expiry is assumed to stop already-running code	22
2PC versus saga	Coordinated atomic commit in a supported scope versus explicit local commits and business recovery	Compensation is described as erasing history or 2PC as removing failure blocking	22, 27
L4 versus L7 balancing	Transport routing versus application-aware policy	TLS termination and protocol ownership are unspecified	21
Image versus running container	Immutable packaged filesystem versus a process with runtime limits	Building an image is treated as a deployment or security proof	14
Readiness versus liveness	Routing eligibility versus restart policy	A temporary downstream outage restarts the entire fleet	15

Choice	Use when	Avoid when	Explain through
Rolling versus blue-green versus canary	Gradual replacement, environment switch or measured exposure	Old and new schemas are incompatible or rollback evidence is absent	16
VM/container versus Lambda	Managed long-lived workload versus suitable bounded invocation	Runtime, state, networking, cost and concurrency constraints are ignored	17
Replication versus backup	Availability copy versus retained recovery history	Replicated corruption is mistaken for recoverability	17, 18
Metrics versus logs versus traces	Aggregate behavior, contextual events and request-path evidence	High-cardinality labels or missing sampling context obscure the question	18
Unit versus integration versus end-to-end tests	Narrow invariants, real boundaries and selected user journeys	Mock success is reported as database or browser correctness	19
JFR versus microbenchmark	Diagnose a representative running workload versus isolate a specific cost	Synthetic rankings are generalized to whole-system latency	23
Payment retry versus reconciliation	Repeat the same supported operation identity versus resolve independent evidence	A new payment is created to hide an unknown previous outcome	27

[D] DECISION

A defensible answer has a reversal condition. State the chosen option, the requirement it satisfies, the cost accepted and the evidence that would make you change it. For HLD, use the full case-study method in 12, not just this table.

2. A 30-Day Revision Plan

Treat each day as a bounded study session, not a claim about how quickly any person will master the guide. A workable allocation is reading, closed-book explanation and one evidence-based exercise; adjust duration to availability. Repeat weak topics instead of adding more sources. Where tools are unavailable, perform a trace and state NOT EXECUTED rather than invent results.

Day	Focus	Deliverable and self-check
1	00 master map	Draw the request journey and identify authority at each boundary
2	01 JVM and values	Explain class identity, reachability and one collection mutation trap
3	02 memory model	Trace a happens-before edge and a compound operation that remains unsafe
4	02 executors, 21 I/O	Explain queue growth, blocking and event-loop ownership without equating threads with capacity

Day	Focus	Deliverable and self-check
5	21 networking	Follow DNS through TLS and distinguish connect, read and overall deadlines
6	03 Spring	Trace request security, proxy interception and a self-invocation transaction trap
7	04 JPA	Explain flush versus commit and diagnose an N+1 fetch plan
8	06 transactions	Trace two concurrent writers and name the invariant their isolation must protect
9	06 storage and recovery	Explain index, WAL, vacuum, pool wait and restore boundaries; note the open SQL workbook gap
10	05 APIs	Design a versioned resource, bound idempotency result and pagination contract
11	05 live channels, 09 UI	Trace cursor expiry, snapshot recovery and tenant-switch races
12	07 messaging	Crash an imagined outbox/consumer flow at each commit gap and predict replay
13	07 batch and streams	Distinguish event, replacement state, window and restart identity
14	13 integration, 26 analytics	Declare mapping version, quarantine policy, fact grain and freshness cutoff
15	08 identity	Separate authentication, authorization, provisioning and token purposes
16	08 protection, 09 frontend	Review tenant enforcement, token handling, data minimization and effect cleanup
17	10 foundations	Estimate demand under named assumptions and identify the first capacity budget
18	25 architecture	Draw contexts and a Spring dependency rule; write one ADR with consequences
19	22 distributed systems	Explain safety, quorum assumptions, lease versus fence and unknown outcomes
20	11 LLD	Choose two cases; explain APIs, invariant, concurrency and tests before patterns
21	12 HLD	Design bulk sync and live status, including estimates, data model and failure recovery
22	27 payments	Explain wallet overspend, gateway ambiguity and ledger reconciliation as separate problems
23	14 Docker, 15 Kubernetes	Trace image to process, resource limit, readiness and graceful shutdown
24	16 delivery	Bind test evidence to an image and explain schema-compatible GitOps rollback
25	17 infrastructure	Separate routing, IAM and managed-service responsibility; outline a tested restore
26	18 operations	Define an outcome SLI, burn alert, incident timeline and recovery evidence
27	23 performance	Choose CPU, allocation, heap or thread evidence from one symptom; avoid invented measurements
28	19 quality	Map each claimed guarantee to its narrowest useful test and list untested boundaries
29	24 capstone	Rehearse the five-minute system script and the five failure stories closed-book
30	20 toolkit and weakest chapters	Run a mock interview, record missing mechanisms and recheck their source explanations

[M] MECHANISM

Use retrieval, then correction. Answer without the guide, compare against the owning chapter, identify the exact missing step, and repeat the explanation with that step included. A correct keyword list is not the same as tracing a commit or a race.

3. Glossary of High-Risk Distinctions

The global index below covers the full registered vocabulary. This shorter glossary prioritizes distinctions that commonly change the correctness of an answer; it links out instead of repeating entire chapters.

Term or pair	Precise working meaning	Owner
Authority versus projection	Accepted facts versus a derived representation with a declared lag or recovery rule	00
Reachable versus useful	GC reachability can retain objects the business no longer needs	01
Equality versus identity	Same value under an equality contract versus the same domain/runtime entity	01, 25
Happens-before	A synchronization-derived visibility/order relation, not timestamp order	02
Atomicity versus visibility	Indivisible relevant change versus guaranteed observation of a write	02
Admission versus concurrency	Bound accepted/waiting work versus active use of a resource	02
Proxy boundary	Calls through the eligible proxy are intercepted; ordinary self-calls may not be	03
Flush versus commit	Synchronize pending SQL versus accept the transaction under its durability contract	04
Idempotency	Repeating one identified logical operation does not create another accepted effect	05
Replay cursor	Position for requesting retained history, not a guarantee the history still exists	05
Backpressure	Explicit demand or bounded overload handling, not merely faster asynchronous calls	05
MVCC	Visibility of row versions under transaction rules; not freedom from all locks or conflicts	06
Replica versus backup	A live copy versus retained recovery history and a restore procedure	06
Outbox versus inbox receipt	Durable publication intent versus duplicate input/effect tracking	07
Event sourcing versus CQRS	Authoritative event history versus separate command and query models	07
Compensation versus rollback	A new business action after commit versus rejection of an uncommitted local transaction	07
Authentication versus authorization	Establish identity versus permit a specific resource operation	08
Provisioning	Manage account and membership lifecycle; not login	08

Term or pair	Precise working meaning	Owner
Access, ID and refresh tokens	API authority, client login assertion and renewal credential respectively	08
CORS versus CSRF	Browser cross-origin response policy versus unwanted credential-bearing actions	08
UI generation	Identity-lifetime marker that invalidates obsolete requests and subscriptions	09
Bulkhead	An isolated resource budget that limits failure propagation	10
Estimate versus measurement	Assumption-driven arithmetic versus observed behavior in a recorded environment	10
Invariant	A property that must remain true under the stated concurrency and failure model	11
Quarantine	Durable classified exception requiring a declared repair or exclusion policy	13
Namespace versus cgroup	Resource visibility boundary versus accounting and control	14
Startup, readiness and liveness	Initial allowance, routing eligibility and restart policy	15
Desired versus observed state	Declarative target versus current controller/runtime result	15
Artifact identity versus trust	Exact bytes versus provenance, approval and security evidence	16
Terraform configuration versus state	Desired declarations versus tracked remote identities and attributes	17
SLI, SLO and SLA	Outcome measure, target and external agreement	18
RPO versus RTO	Recovery data-loss objective versus recovery-time objective	18
Test double versus integration evidence	Simulated collaborator behavior versus an exercised real boundary	19
Readiness versus message completion	Bytes may be available without a complete application message	21
Safety versus liveness	Invalid outcomes do not occur versus progress eventually occurs under assumptions	22
Lease versus fence	Permission lifetime versus effect-side rejection of obsolete ownership epochs	22
Linearizability versus eventual consistency	Operations fit a real-time-respecting order versus replicas converge under stated conditions	22
Allocation versus retained memory	Object creation over time versus reachable objects kept alive	23
Heap versus process memory	Managed object region versus total runtime resource usage	23
Bounded context versus aggregate	Scope of a consistent domain model versus a boundary enforcing particular invariants	25
Port versus adapter	Application-facing contract versus an implementation connecting a technology	25
Grain versus dimension	Meaning of one fact row versus descriptive attributes for interpreting facts	26
Event time versus processing time	When the source event occurred versus when a system handled it	26
Lineage	Evidence connecting derived data to sources, transformations and versions	26
Unknown payment outcome	The caller lacks authoritative effect evidence; it is not a decline	27
Balanced journal versus available funds	Conservation of posted accounting effects versus permission to spend	27

Term or pair	Precise working meaning	Owner
Reconciliation	Match independent evidence at a cutoff and resolve owned exceptions	27

[!] TRAP

Do not promote a definition into a guarantee. Naming an outbox, an aggregate, a lease or a journal does not show that the correct data shares a commit, concurrency boundary or recovery policy. Ask what enforces it.

4. Evidence and Answer Discipline

A concise answer follows this order: requirement, invariant, mechanism, failure, recovery, trade-off and evidence. For debugging, start from an observed symptom and choose the next discriminating check; do not list every tool you know. For design, ask which dimensions can vary: tenant, scale, latency, cost, failure isolation and regulatory scope.

When discussing the guide's labs, say which runtime and boundary were actually exercised. Local JavaScript checks do not establish Java compilation, database isolation, payment-provider behavior or deployment safety. A parsed POM is not a passing JUnit suite. A rendered Mermaid diagram is not a deployed architecture. These are useful but different evidence.

The guide has all chapter sources after this batch, but earlier editorial work remains: Chapter 06 still needs its SQL query-writing workbook and expanded Redis patterns, and the final Chapter 05/06 accuracy/coverage pass remains open. Syntax highlighting is unavailable in the current offline edition. Those limitations are not resolved by the presence of an index.

[VERIFY: recheck version-sensitive APIs, provider/network contracts, regulatory claims and every remaining chapter verification item against current primary documentation and an approved execution environment. This appendix does not close the guide's outstanding verification or editorial gaps.]

[?] INTERVIEW

A useful uncertainty statement is specific. "I would verify the provider's idempotency retention and status-lookup contract before retrying this unknown operation" is stronger than claiming certainty about every gateway. Pair uncertainty with the exact evidence needed.

5. Global Concept Index

This alphabetical index is generated from `catalog.json` during the build and links every registered term to its owning explanation. A term can have more than one owner; virtual threads deliberately links both core Java and concurrency. All 28 chapters, including this appendix and the capstone, are covered. The catalog records coverage vocabulary, not proof that every editorial or runtime requirement is complete.

Concept and explanation	Owning chapter
10x traffic	24 IntegrationHub in Motion
2PC	22 Distributed Systems Theory
30-day revision plan	20 Appendix: Interview Toolkit and Index
Actuator	03 Spring and Spring Boot Internals
Adapter	11 Object Design, Patterns and LLD
ADR	25 Architecture Styles and Domain Design
agents	16 Build, Ship and Roll Back
aggregate	25 Architecture Styles and Domain Design
Agile	16 Build, Ship and Roll Back
ALB	17 Cloud and Infrastructure
alerting	18 Observability and Reliability
allocation profiling	23 JVM Performance Engineering
AML	27 Fintech and Payments Engineering
analytics CDC	26 Data Platform Basics
Anaplan Connect	13 Data Integration and iPaaS
Angular	09 The Browser and the Java Backend
AOP	03 Spring and Spring Boot Internals
API gateway	10 System Design Foundations
API management	17 Cloud and Infrastructure
API-first	25 Architecture Styles and Domain Design
ArgoCD	16 Build, Ship and Roll Back
async-profiler	23 JVM Performance Engineering
atomics	02 Concurrency and the Memory Model

Concept and explanation	Owning chapter
audit logging	08 Security, Identity and Data Protection
authorization	27 Fintech and Payments Engineering
auto-configuration	03 Spring and Spring Boot Internals
AWS	17 Cloud and Infrastructure
B-tree	06 Databases and Durable State
backoff	10 System Design Foundations
backpressure	05 APIs and Real-Time Communication
backups	06 Databases and Durable State
batching	13 Data Integration and iPaaS
bean lifecycle	03 Spring and Spring Boot Internals
bloat	06 Databases and Durable State
blocking I/O	21 Networking and Operating Systems
blue-green	16 Build, Ship and Roll Back
bounded context	25 Architecture Styles and Domain Design
browser	09 The Browser and the Java Backend
buffer pool	06 Databases and Durable State
Builder	11 Object Design, Patterns and LLD
bulk sync pipeline	12 Twelve System Design Case Studies
bulkhead	10 System Design Foundations
caching	10 System Design Foundations
canary	16 Build, Ship and Roll Back
CAP	22 Distributed Systems Theory
capacity planning	18 Observability and Reliability
capture	27 Fintech and Payments Engineering
card network	27 Fintech and Payments Engineering
causal consistency	22 Distributed Systems Theory
CDC	07 Messaging, Batch and Streams
CDN	10 System Design Foundations
cgroups	14 Docker and Container Internals

Concept and explanation	Owning chapter
Chain of Responsibility	11 Object Design, Patterns and LLD
chaos testing	18 Observability and Reliability
chargeback	27 Fintech and Payments Engineering
chunk	07 Messaging, Batch and Streams
circuit breaker	10 System Design Foundations
class loading	01 Core Java and the JVM
clean architecture	25 Architecture Styles and Domain Design
clock skew	22 Distributed Systems Theory
CloudWatch	17 Cloud and Infrastructure
cold start	17 Cloud and Infrastructure
collections	01 Core Java and the JVM
Command	11 Object Design, Patterns and LLD
CompletableFuture	02 Concurrency and the Memory Model
Compose	14 Docker and Container Internals
concurrent collections	02 Concurrency and the Memory Model
Config Server	03 Spring and Spring Boot Internals
ConfigMap	15 Kubernetes and Production Workloads
configuration	03 Spring and Spring Boot Internals
connect timeout	21 Networking and Operating Systems
connection pooling	06 Databases and Durable State
connector design	13 Data Integration and iPaaS
connector framework	11 Object Design, Patterns and LLD
consensus	22 Distributed Systems Theory
consistent hashing	22 Distributed Systems Theory
consumer lag	18 Observability and Reliability
container networking	14 Docker and Container Internals
context mapping	25 Architecture Styles and Domain Design
context switch	21 Networking and Operating Systems
contract testing	19 Testing and Quality

Concept and explanation	Owning chapter
contract-first	25 Architecture Styles and Domain Design
control plane	15 Kubernetes and Production Workloads
CORS	08 Security, Identity and Data Protection
CPU profiling	23 JVM Performance Engineering
CQRS	07 Messaging, Batch and Streams
CrashLoopBackOff	15 Kubernetes and Production Workloads
credentials	16 Build, Ship and Roll Back
CSRF	08 Security, Identity and Data Protection
curl	21 Networking and Operating Systems
data fetching	09 The Browser and the Java Backend
data lake	26 Data Platform Basics
data quality	26 Data Platform Basics
database locks	06 Databases and Durable State
day in the life	24 IntegrationHub in Motion
DDD	25 Architecture Styles and Domain Design
deadlocks	02 Concurrency and the Memory Model
Debezium	07 Messaging, Batch and Streams
decision tables	20 Appendix: Interview Toolkit and Index
Decorator	11 Object Design, Patterns and LLD
delivery semantics	07 Messaging, Batch and Streams
deoptimization	23 JVM Performance Engineering
deploy story	24 IntegrationHub in Motion
Deployment	15 Kubernetes and Production Workloads
dig	21 Networking and Operating Systems
disaster recovery	18 Observability and Reliability
distributed cache	12 Twelve System Design Case Studies
distributed lock	22 Distributed Systems Theory
DLQ	07 Messaging, Batch and Streams
DNS	21 Networking and Operating Systems

Concept and explanation	Owning chapter
Docker	14 Docker and Container Internals
Dockerfile	14 Docker and Container Internals
domain event	25 Architecture Styles and Domain Design
double-entry ledger	27 Fintech and Payments Engineering
EC2	17 Cloud and Infrastructure
EKS	17 Cloud and Infrastructure
Elasticsearch	06 Databases and Durable State
elevator	11 Object Design, Patterns and LLD
ELT	13 Data Integration and iPaaS
entity	25 Architecture Styles and Domain Design
entity states	04 Hibernate and JPA
envelope encryption	08 Security, Identity and Data Protection
error model	05 APIs and Real-Time Communication
estimation	10 System Design Foundations
etcd	22 Distributed Systems Theory
ETL	13 Data Integration and iPaaS
Eureka	03 Spring and Spring Boot Internals
event loop	21 Networking and Operating Systems
event sourcing	07 Messaging, Batch and Streams
eventual consistency	22 Distributed Systems Theory
evolutionary architecture	25 Architecture Styles and Domain Design
exactly-once	07 Messaging, Batch and Streams
exactly-once effects	27 Fintech and Payments Engineering
exceptions	01 Core Java and the JVM
expand/contract	06 Databases and Durable State
EXPLAIN	06 Databases and Durable State
Factory	11 Object Design, Patterns and LLD
failure models	22 Distributed Systems Theory
failure stories	24 IntegrationHub in Motion

Concept and explanation	Owning chapter
feature flags	16 Build, Ship and Roll Back
fencing token	22 Distributed Systems Theory
file descriptors	21 Networking and Operating Systems
flame graph	23 JVM Performance Engineering
flush	04 Hibernate and JPA
Flyway	06 Databases and Durable State
fraud hooks	27 Fintech and Payments Engineering
frontend performance	09 The Browser and the Java Backend
G1	23 JVM Performance Engineering
gateway design	12 Twelve System Design Case Studies
gateway timeout	27 Fintech and Payments Engineering
GC	01 Core Java and the JVM
GC logging	23 JVM Performance Engineering
GDPR	08 Security, Identity and Data Protection
generics	01 Core Java and the JVM
Git	16 Build, Ship and Roll Back
GitOps	16 Build, Ship and Roll Back
global concept index	20 Appendix: Interview Toolkit and Index
glossary	20 Appendix: Interview Toolkit and Index
gossip	22 Distributed Systems Theory
Gradle	16 Build, Ship and Roll Back
Grafana	18 Observability and Reliability
GraphQL	05 APIs and Real-Time Communication
gRPC	05 APIs and Real-Time Communication
happens-before	02 Concurrency and the Memory Model
Harness	16 Build, Ship and Roll Back
heap dump	23 JVM Performance Engineering
heartbeats	22 Distributed Systems Theory
Helm	15 Kubernetes and Production Workloads

Concept and explanation	Owning chapter
hexagonal architecture	25 Architecture Styles and Domain Design
hooks	09 The Browser and the Java Backend
HPA	15 Kubernetes and Production Workloads
HTTP	09 The Browser and the Java Backend
HTTP/1.1	21 Networking and Operating Systems
HTTP/2	21 Networking and Operating Systems
HTTP/3	21 Networking and Operating Systems
hybrid logical clock	22 Distributed Systems Theory
IAM	17 Cloud and Infrastructure
idempotency	05 APIs and Real-Time Communication
idempotency registry	11 Object Design, Patterns and LLD
idempotent consumer	07 Messaging, Batch and Streams
IICS	13 Data Integration and iPaaS
image layers	14 Docker and Container Internals
image security	14 Docker and Container Internals
ImagePullBackOff	15 Kubernetes and Production Workloads
incident response	18 Observability and Reliability
indexes	06 Databases and Durable State
Ingress	15 Kubernetes and Production Workloads
inlining	23 JVM Performance Engineering
IntegrationHub	00 The System Before the Details
interview answer structure	00 The System Before the Details
IoC	03 Spring and Spring Boot Internals
iostat	21 Networking and Operating Systems
isolation	06 Databases and Durable State
Istio	15 Kubernetes and Production Workloads
JavaScript	09 The Browser and the Java Backend
Jenkins	16 Build, Ship and Roll Back
JFR	23 JVM Performance Engineering

Concept and explanation	Owning chapter
JIT	23 JVM Performance Engineering
jitter	10 System Design Foundations
JMC	23 JVM Performance Engineering
JMH	23 JVM Performance Engineering
JMM	02 Concurrency and the Memory Model
job repository	07 Messaging, Batch and Streams
job scheduler	12 Twelve System Design Case Studies
JPA cache	04 Hibernate and JPA
JUnit 5	19 Testing and Quality
JVM memory	01 Core Java and the JVM
JWT	08 Security, Identity and Data Protection
Kafka	07 Messaging, Batch and Streams
Kafka Streams	07 Messaging, Batch and Streams
Kanban	16 Build, Ship and Roll Back
keep-alive	21 Networking and Operating Systems
keystore	08 Security, Identity and Data Protection
KMS	08 Security, Identity and Data Protection
KStream	07 Messaging, Batch and Streams
KTable	07 Messaging, Batch and Streams
Kubernetes	15 Kubernetes and Production Workloads
Kubernetes lease	22 Distributed Systems Theory
KYC	27 Fintech and Payments Engineering
L4 vs L7	21 Networking and Operating Systems
lakehouse	26 Data Platform Basics
Lambda	17 Cloud and Infrastructure
Lamport clock	22 Distributed Systems Theory
Last-Event-ID	05 APIs and Real-Time Communication
LaunchDarkly	16 Build, Ship and Roll Back
lazy loading	04 Hibernate and JPA

Concept and explanation	Owning chapter
leader election	22 Distributed Systems Theory
leader-follower	22 Distributed Systems Theory
leaderless	22 Distributed Systems Theory
ledger	27 Fintech and Payments Engineering
lineage	26 Data Platform Basics
linearizability	22 Distributed Systems Theory
Linkerd	15 Kubernetes and Production Workloads
Linux	21 Networking and Operating Systems
Liquibase	06 Databases and Durable State
Little's law	23 JVM Performance Engineering
live-status service	12 Twelve System Design Case Studies
load balancing	10 System Design Foundations
load testing	18 Observability and Reliability
locks	02 Concurrency and the Memory Model
logger	11 Object Design, Patterns and LLD
logical replication	06 Databases and Durable State
logs	18 Observability and Reliability
Loki	18 Observability and Reliability
long polling	05 APIs and Real-Time Communication
LRU cache	11 Object Design, Patterns and LLD
LSM tree	06 Databases and Durable State
mapping	04 Hibernate and JPA
MAT	23 JVM Performance Engineering
Maven	16 Build, Ship and Roll Back
MaxRAMPercentage	23 JVM Performance Engineering
memory leak	23 JVM Performance Engineering
metrics	18 Observability and Reliability
microservices	25 Architecture Styles and Domain Design
Mockito	19 Testing and Quality

Concept and explanation	Owning chapter
modular monolith	25 Architecture Styles and Domain Design
money state machine	27 Fintech and Payments Engineering
MongoDB	06 Databases and Durable State
monolith	25 Architecture Styles and Domain Design
mTLS	08 Security, Identity and Data Protection
MuleSoft	13 Data Integration and iPaaS
multi-leader	22 Distributed Systems Theory
multi-region	10 System Design Foundations
multi-stage build	14 Docker and Container Internals
multi-tenancy	10 System Design Foundations
MVCC	06 Databases and Durable State
MySQL	06 Databases and Durable State
N+1	04 Hibernate and JPA
namespaces	14 Docker and Container Internals
NAT	21 Networking and Operating Systems
Netty	21 Networking and Operating Systems
NIO	21 Networking and Operating Systems
notification service	11 Object Design, Patterns and LLD
notification system	12 Twelve System Design Case Studies
OAuth 2.0	08 Security, Identity and Data Protection
Observer	11 Object Design, Patterns and LLD
OIDC	08 Security, Identity and Data Protection
Okta	08 Security, Identity and Data Protection
OLAP	26 Data Platform Basics
OLTP	26 Data Platform Basics
OOMKilled	15 Kubernetes and Production Workloads
OOP	11 Object Design, Patterns and LLD
OpenAPI	05 APIs and Real-Time Communication
OpenFeign	03 Spring and Spring Boot Internals

Concept and explanation	Owning chapter
OpenTelemetry	18 Observability and Reliability
optimistic locking	04 Hibernate and JPA
ordering	07 Messaging, Batch and Streams
outbox	07 Messaging, Batch and Streams
OWASP	08 Security, Identity and Data Protection
ownership boundary	00 The System Before the Details
PACELC	22 Distributed Systems Theory
page cache	21 Networking and Operating Systems
pagination	05 APIs and Real-Time Communication
parking lot	11 Object Design, Patterns and LLD
partitioning	06 Databases and Durable State
pattern matching	01 Core Java and the JVM
Paxos	22 Distributed Systems Theory
payment gateway	27 Fintech and Payments Engineering
payment/wallet	12 Twelve System Design Case Studies
PCI DSS	08 Security, Identity and Data Protection
PCI scope	27 Fintech and Payments Engineering
Pending	15 Kubernetes and Production Workloads
performance testing	19 Testing and Quality
persistence context	04 Hibernate and JPA
pessimistic locking	04 Hibernate and JPA
phi accrual	22 Distributed Systems Theory
PII	08 Security, Identity and Data Protection
PITR	06 Databases and Durable State
PKCE	08 Security, Identity and Data Protection
PKI	08 Security, Identity and Data Protection
Playwright	19 Testing and Quality
Pod	15 Kubernetes and Production Workloads
polling	05 APIs and Real-Time Communication

Concept and explanation	Owning chapter
pool sizing	23 JVM Performance Engineering
ports	21 Networking and Operating Systems
PostgreSQL	06 Databases and Durable State
postmortem	18 Observability and Reliability
Power BI	13 Data Integration and iPaaS
probes	15 Kubernetes and Production Workloads
processes	21 Networking and Operating Systems
Prometheus	18 Observability and Reliability
proxies	03 Spring and Spring Boot Internals
Proxy	11 Object Design, Patterns and LLD
proxy buffering	05 APIs and Real-Time Communication
pub/sub	11 Object Design, Patterns and LLD
Python	13 Data Integration and iPaaS
quality gates	19 Testing and Quality
quarantine	13 Data Integration and iPaaS
queues	10 System Design Foundations
QUIC	21 Networking and Operating Systems
quorum	22 Distributed Systems Theory
RabbitMQ	07 Messaging, Batch and Streams
Raft	22 Distributed Systems Theory
rate limiter	11 Object Design, Patterns and LLD
rate limiting	10 System Design Foundations
rate-limiter service	12 Twelve System Design Case Studies
RBAC	15 Kubernetes and Production Workloads
RCA	18 Observability and Reliability
RDS	17 Cloud and Infrastructure
React	09 The Browser and the Java Backend
Reactor	05 APIs and Real-Time Communication
read timeout	21 Networking and Operating Systems

Concept and explanation	Owning chapter
readiness	15 Kubernetes and Production Workloads
reconciliation	27 Fintech and Payments Engineering
reconnect	05 APIs and Real-Time Communication
records	01 Core Java and the JVM
Redis	06 Databases and Durable State
Redlock	22 Distributed Systems Theory
refresh token	08 Security, Identity and Data Protection
refund	27 Fintech and Payments Engineering
rendering	09 The Browser and the Java Backend
repartitioning	07 Messaging, Batch and Streams
replication	10 System Design Foundations
request journey	00 The System Before the Details
Resilience4j	10 System Design Foundations
resource limits	15 Kubernetes and Production Workloads
REST	05 APIs and Real-Time Communication
restartability	07 Messaging, Batch and Streams
reverse proxy	21 Networking and Operating Systems
rollback	16 Build, Ship and Roll Back
rolling deployment	16 Build, Ship and Roll Back
rollout	15 Kubernetes and Production Workloads
rotation	08 Security, Identity and Data Protection
RPO	18 Observability and Reliability
RTO	18 Observability and Reliability
runbook	18 Observability and Reliability
S3	17 Cloud and Infrastructure
saga	07 Messaging, Batch and Streams
SAML	08 Security, Identity and Data Protection
scalability	10 System Design Foundations
schema mapping	13 Data Integration and iPaaS

Concept and explanation	Owning chapter
SCIM	08 Security, Identity and Data Protection
sealed types	01 Core Java and the JVM
search service	12 Twelve System Design Case Studies
Secret	15 Kubernetes and Production Workloads
secrets	08 Security, Identity and Data Protection
security story	24 IntegrationHub in Motion
select/poll/epoll	21 Networking and Operating Systems
sequential consistency	22 Distributed Systems Theory
Service	15 Kubernetes and Production Workloads
service discovery	03 Spring and Spring Boot Internals
service mesh	15 Kubernetes and Production Workloads
settlement	27 Fintech and Payments Engineering
sharding	06 Databases and Durable State
shared libraries	16 Build, Ship and Roll Back
Singleton	11 Object Design, Patterns and LLD
skip/retry	07 Messaging, Batch and Streams
SLA	18 Observability and Reliability
SLI	18 Observability and Reliability
SLO	18 Observability and Reliability
SNI	21 Networking and Operating Systems
SNS	17 Cloud and Infrastructure
SOLID	11 Object Design, Patterns and LLD
source of truth	00 The System Before the Details
SPA authentication	09 The Browser and the Java Backend
split brain	22 Distributed Systems Theory
spoken script	24 IntegrationHub in Motion
Spring Batch	07 Messaging, Batch and Streams
Spring Cloud Gateway	03 Spring and Spring Boot Internals
Spring Data	04 Hibernate and JPA

Concept and explanation	Owning chapter
Spring MVC	03 Spring and Spring Boot Internals
Spring Security	03 Spring and Spring Boot Internals
SQL	06 Databases and Durable State
SQS	17 Cloud and Infrastructure
ss	21 Networking and Operating Systems
SSE	05 APIs and Real-Time Communication
star schema	26 Data Platform Basics
state / State	09 The Browser and the Java Backend / 11 Object Design, Patterns and LLD
state store	07 Messaging, Batch and Streams
StatefulSet	15 Kubernetes and Production Workloads
STOMP	05 APIs and Real-Time Communication
strace	21 Networking and Operating Systems
strangler fig	25 Architecture Styles and Domain Design
Strategy	11 Object Design, Patterns and LLD
streaming replication	06 Databases and Durable State
Streams	01 Core Java and the JVM
supply chain	16 Build, Ship and Roll Back
Swagger	05 APIs and Real-Time Communication
synchronized	02 Concurrency and the Memory Model
system whiteboard	24 IntegrationHub in Motion
task scheduler	11 Object Design, Patterns and LLD
TCP	21 Networking and Operating Systems
tcpdump	21 Networking and Operating Systems
tech debt	25 Architecture Styles and Domain Design
technology poster	24 IntegrationHub in Motion
Template Method	11 Object Design, Patterns and LLD
Terraform	17 Cloud and Infrastructure
test pyramid	19 Testing and Quality
test slices	19 Testing and Quality

Concept and explanation	Owning chapter
Testcontainers	19 Testing and Quality
thread dump	23 JVM Performance Engineering
ThreadPoolExecutor	02 Concurrency and the Memory Model
threads	21 Networking and Operating Systems
TIME_WAIT	21 Networking and Operating Systems
timeouts	10 System Design Foundations
TLS	08 Security, Identity and Data Protection
TLS handshake	21 Networking and Operating Systems
token service	12 Twelve System Design Case Studies
tokenization	08 Security, Identity and Data Protection
top	21 Networking and Operating Systems
traces	18 Observability and Reliability
Transactional	03 Spring and Spring Boot Internals
transactions	06 Databases and Durable State
TTL store	11 Object Design, Patterns and LLD
TypeScript	09 The Browser and the Java Backend
ubiquitous language	25 Architecture Styles and Domain Design
UPI	27 Fintech and Payments Engineering
URL shortener	12 Twelve System Design Case Studies
vacuum	06 Databases and Durable State
validation	03 Spring and Spring Boot Internals
value object	25 Architecture Styles and Domain Design
Vault	08 Security, Identity and Data Protection
vector clock	22 Distributed Systems Theory
versioning	05 APIs and Real-Time Communication
virtual memory	21 Networking and Operating Systems
virtual threads / virtual threads	01 Core Java and the JVM / 02 Concurrency and the Memory Model
vmstat	21 Networking and Operating Systems
volatile	02 Concurrency and the Memory Model

Concept and explanation	Owning chapter
VPA	15 Kubernetes and Production Workloads
VPC	17 Cloud and Infrastructure
WAL	06 Databases and Durable State
wallet	27 Fintech and Payments Engineering
warehouse	26 Data Platform Basics
warm-up	23 JVM Performance Engineering
WebFlux	05 APIs and Real-Time Communication
webhook delivery	12 Twelve System Design Case Studies
webhook dispatcher	11 Object Design, Patterns and LLD
webhooks	05 APIs and Real-Time Communication
WebSocket	05 APIs and Real-Time Communication
windowing	07 Messaging, Batch and Streams
X.509	08 Security, Identity and Data Protection
zero-copy	21 Networking and Operating Systems
ZGC	23 JVM Performance Engineering
ZooKeeper	22 Distributed Systems Theory

Concepts Explained Here

Use [decision tables](#), the [30-day revision plan](#), the [glossary](#) and the [global concept index](#). Read [evidence discipline](#) before describing lab results as production guarantees.

Related Chapters

The index supplies direct explanations and chapter destinations for every chapter. Return to [00 orientation](#) or [24 the connected system](#) for the narrative route. Generated links are reciprocal.

RELATED CHAPTERS / RECIPROCAL LINKS

- | | |
|--|---|
| 00 The System Before the Details | 23 JVM Performance Engineering |
| 24 IntegrationHub in Motion | 03 Spring and Spring Boot Internals |
| 01 Core Java and the JVM | 04 Hibernate and JPA |
| 25 Architecture Styles and Domain Design | 06 Databases and Durable State |
| 02 Concurrency and the Memory Model | 05 APIs and Real-Time Communication |

27	Fintech and Payments Engineering	21	Networking and Operating Systems
07	Messaging, Batch and Streams	14	Docker and Container Internals
26	Data Platform Basics	15	Kubernetes and Production Workloads
13	Data Integration and iPaaS	16	Build, Ship and Roll Back
08	Security, Identity and Data Protection	17	Cloud and Infrastructure
09	The Browser and the Java Backend	18	Observability and Reliability
10	System Design Foundations	19	Testing and Quality
11	Object Design, Patterns and LLD	12	Twelve System Design Case Studies
22	Distributed Systems Theory		

One-Page Cheat Sheet

Answer: requirement, invariant, mechanism, failure, recovery, trade-off, evidence. A product name is not an explanation.

Compare: use when, avoid when, accepted cost and reversal condition. Change the recommendation when constraints change.

Revise: retrieve closed-book, compare with the owning chapter, fix the missing step and repeat. Cover all 28 chapters rather than only familiar Java topics.

Trace: identity, authorization, commit, transport, effect, recovery and observability. Keep accepted, completed and observed separate.

Be precise: authority versus projection; atomicity versus visibility; retry versus reconciliation; lease versus fence; journal balance versus spendable funds.

Report honestly: local test, static review and unavailable execution are different. Use the complete index to find an explanation, not to infer that open verification items were closed.

Interview Corner

Basic: How Do You Choose between Two Technologies?

State the required behavior, eliminate options that violate it, compare remaining operational costs and identify the evidence that could reverse the choice.

Internals: What Makes an Answer More than Vocabulary?

Trace actual state changes and the enforcing boundary: lock, transaction, proxy, protocol message or durable receipt. Explain what happens when that step fails.

Trace/Debug: You Do Not Know the Root Cause Yet.

Name one plausible hypothesis and the cheapest observation that would disprove it. Gather that evidence before changing unrelated settings.

Scenario: A Design Works Except When a Response Is Lost.

Separate caller knowledge from remote effect. Identify the operation, stored result and recovery evidence. Add idempotency or reconciliation at the actual effect boundary.

Basic: What Does Completing the Guide Mean?

All planned chapter sources and index destinations exist. It does not mean every external claim was verified, every lab ran or earlier editorial gaps disappeared.

Scenario: Explain IntegrationHub without Claiming Experience You Do Not Have.

Call it a fictional reference design. Explain its requirements, mechanisms and trade-offs, then distinguish the executed local checks from proposed integration and production validation.

Coverage Index

Chapters 00, 01, 02, 21, 03, 04, 06, 05, 07, 26, 08, 09, 10, 25, 22, 11, 12, 27, 13, 14, 15, 16, 17, 18, 23, 19, 24, 20 are available in this edition. The complete section-level global index is also available in Chapter 20. Coverage does not close outstanding verification or editorial gaps.

10x traffic

24 IntegrationHub in Motion Available

2PC

22 Distributed Systems Theory Available

30-day revision plan

20 Appendix: Interview Toolkit and Index Available

Actuator

03 Spring and Spring Boot Internals Available

Adapter

11 Object Design, Patterns and LLD Available

ADR

25 Architecture Styles and Domain Design Available

agents

16 Build, Ship and Roll Back Available

aggregate

25 Architecture Styles and Domain Design Available

Agile

16 Build, Ship and Roll Back Available

ALB

17 Cloud and Infrastructure Available

alerting

18 Observability and Reliability Available

allocation profiling

23 JVM Performance Engineering Available

AML

27 Fintech and Payments Engineering Available

analytics CDC

26 Data Platform Basics Available

Anaplan Connect

13 Data Integration and iPaaS Available

Angular

09 The Browser and the Java Backend Available

AOP

03 Spring and Spring Boot Internals Available

API gateway

10 System Design Foundations Available

API management

17 Cloud and Infrastructure Available

API-first

25 Architecture Styles and Domain Design Available

ArgoCD

16 Build, Ship and Roll Back Available

async-profiler

23 JVM Performance Engineering Available

atomics

02 Concurrency and the Memory Model Available

audit logging

08 Security, Identity and Data Protection Available

authorization

27 Fintech and Payments Engineering Available

auto-configuration

03 Spring and Spring Boot Internals Available

AWS

17 Cloud and Infrastructure Available

B-tree

06 Databases and Durable State Available

backoff

10 System Design Foundations Available

backpressure

05 APIs and Real-Time Communication Available

backups

06 Databases and Durable State Available

batching

13 Data Integration and iPaaS Available

bean lifecycle

03 Spring and Spring Boot Internals Available

bloat

06 Databases and Durable State Available

blocking I/O

21 Networking and Operating Systems Available

blue-green

16 Build, Ship and Roll Back Available

bounded context

25 Architecture Styles and Domain Design Available

browser

09 The Browser and the Java Backend Available

buffer pool

06 Databases and Durable State Available

Builder

11 Object Design, Patterns and LLD Available

bulk sync pipeline

12 Twelve System Design Case Studies Available

bulkhead

10 System Design Foundations Available

caching

10 System Design Foundations Available

canary

16 Build, Ship and Roll Back Available

CAP

22 Distributed Systems Theory Available

capacity planning

18 Observability and Reliability Available

capture

27 Fintech and Payments Engineering Available

card network

27 Fintech and Payments Engineering Available

causal consistency

22 Distributed Systems Theory Available

CDC

07 Messaging, Batch and Streams Available

CDN

10 System Design Foundations Available

cgroups

14 Docker and Container Internals Available

Chain of Responsibility

11 Object Design, Patterns and LLD Available

chaos testing

18 Observability and Reliability Available

chargeback

27 Fintech and Payments Engineering Available

chunk

07 Messaging, Batch and Streams Available

circuit breaker

10 System Design Foundations Available

class loading

01 Core Java and the JVM Available

clean architecture

25 Architecture Styles and Domain Design Available

clock skew

22 Distributed Systems Theory Available

CloudWatch

17 Cloud and Infrastructure Available

cold start

17 Cloud and Infrastructure Available

collections

01 Core Java and the JVM Available

Command

11 Object Design, Patterns and LLD Available

CompletableFuture

02 Concurrency and the Memory Model Available

Compose

14 Docker and Container Internals Available

concurrent collections

02 Concurrency and the Memory Model Available

Config Server

03 Spring and Spring Boot Internals Available

ConfigMap

15 Kubernetes and Production Workloads Available

configuration

03 Spring and Spring Boot Internals Available

connect timeout

21 Networking and Operating Systems Available

connection pooling

06 Databases and Durable State Available

connector design

13 Data Integration and iPaaS Available

connector framework

11 Object Design, Patterns and LLD Available

consensus

22 Distributed Systems Theory Available

consistent hashing

22 Distributed Systems Theory Available

consumer lag

18 Observability and Reliability Available

container networking

14 Docker and Container Internals Available

context mapping

25 Architecture Styles and Domain Design Available

context switch

21 Networking and Operating Systems Available

contract testing

19 Testing and Quality Available

contract-first

25 Architecture Styles and Domain Design Available

control plane

15 Kubernetes and Production Workloads Available

CORS

08 Security, Identity and Data Protection Available

CPU profiling

23 JVM Performance Engineering Available

CQRS

07 Messaging, Batch and Streams Available

CrashLoopBackOff

15 Kubernetes and Production Workloads Available

credentials

16 Build, Ship and Roll Back Available

CSRF

08 Security, Identity and Data Protection Available

curl

21 Networking and Operating Systems Available

data fetching

09 The Browser and the Java Backend Available

data lake

26 Data Platform Basics Available

data quality

26 Data Platform Basics Available

database locks

06 Databases and Durable State Available

day in the life

24 IntegrationHub in Motion Available

DDD

25 Architecture Styles and Domain Design Available

deadlocks

02 Concurrency and the Memory Model Available

Debezium

07 Messaging, Batch and Streams Available

decision tables

20 Appendix: Interview Toolkit and Index Available

Decorator

11 Object Design, Patterns and LLD Available

delivery semantics

07 Messaging, Batch and Streams Available

deoptimization

23 JVM Performance Engineering Available

deploy story

24 IntegrationHub in Motion Available

Deployment

15 Kubernetes and Production Workloads Available

dig

21 Networking and Operating Systems Available

disaster recovery

18 Observability and Reliability Available

distributed cache

12 Twelve System Design Case Studies Available

distributed lock

22 Distributed Systems Theory Available

DLQ

07 Messaging, Batch and Streams Available

DNS

21 Networking and Operating Systems Available

Docker

14 Docker and Container Internals Available

Dockerfile

14 Docker and Container Internals Available

domain event

25 Architecture Styles and Domain Design Available

double-entry ledger

27 Fintech and Payments Engineering Available

EC2

17 Cloud and Infrastructure Available

EKS

17 Cloud and Infrastructure Available

Elasticsearch

06 Databases and Durable State Available

elevator

11 Object Design, Patterns and LLD Available

ELT

13 Data Integration and iPaaS Available

entity

25 Architecture Styles and Domain Design Available

entity states

04 Hibernate and JPA Available

envelope encryption

08 Security, Identity and Data Protection Available

error model

05 APIs and Real-Time Communication Available

estimation

10 System Design Foundations Available

etcd

22 Distributed Systems Theory Available

ETL

13 Data Integration and iPaaS Available

Eureka

03 Spring and Spring Boot Internals Available

event loop

21 Networking and Operating Systems Available

event sourcing

07 Messaging, Batch and Streams Available

eventual consistency

22 Distributed Systems Theory Available

evolutionary architecture

25 Architecture Styles and Domain Design Available

exactly-once

07 Messaging, Batch and Streams Available

exactly-once effects

27 Fintech and Payments Engineering Available

exceptions

01 Core Java and the JVM Available

expand/contract

06 Databases and Durable State Available

EXPLAIN

06 Databases and Durable State Available

Factory

11 Object Design, Patterns and LLD Available

failure models

22 Distributed Systems Theory Available

failure stories

24 IntegrationHub in Motion Available

feature flags

16 Build, Ship and Roll Back Available

fencing token

22 Distributed Systems Theory Available

file descriptors

21 Networking and Operating Systems Available

flame graph

23 JVM Performance Engineering Available

flush

04 Hibernate and JPA Available

Flyway

06 Databases and Durable State Available

fraud hooks

27 Fintech and Payments Engineering Available

frontend performance

09 The Browser and the Java Backend Available

G1

23 JVM Performance Engineering Available

gateway design

12 Twelve System Design Case Studies Available

gateway timeout

27 Fintech and Payments Engineering Available

GC

01 Core Java and the JVM Available

GC logging

23 JVM Performance Engineering Available

GDPR

08 Security, Identity and Data Protection Available

generics

01 Core Java and the JVM Available

Git

16 Build, Ship and Roll Back Available

GitOps

16 Build, Ship and Roll Back Available

global concept index

20 Appendix: Interview Toolkit and Index Available

glossary

20 Appendix: Interview Toolkit and Index Available

gossip

22 Distributed Systems Theory Available

Gradle

16 Build, Ship and Roll Back Available

Grafana

18 Observability and Reliability Available

GraphQL

05 APIs and Real-Time Communication Available

gRPC

05 APIs and Real-Time Communication Available

happens-before

02 Concurrency and the Memory Model Available

Harness

16 Build, Ship and Roll Back Available

heap dump

23 JVM Performance Engineering Available

heartbeats

22 Distributed Systems Theory Available

Helm

15 Kubernetes and Production Workloads Available

hexagonal architecture

25 Architecture Styles and Domain Design Available

hooks

09 The Browser and the Java Backend Available

HPA

15 Kubernetes and Production Workloads Available

HTTP

09 The Browser and the Java Backend Available

HTTP/1.1

21 Networking and Operating Systems Available

HTTP/2

21 Networking and Operating Systems Available

HTTP/3

21 Networking and Operating Systems Available

hybrid logical clock

22 Distributed Systems Theory Available

IAM

17 Cloud and Infrastructure Available

idempotency

05 APIs and Real-Time Communication Available

idempotency registry

11 Object Design, Patterns and LLD Available

idempotent consumer

07 Messaging, Batch and Streams Available

IICS

13 Data Integration and iPaaS Available

image layers

14 Docker and Container Internals Available

image security

14 Docker and Container Internals Available

ImagePullBackOff

15 Kubernetes and Production Workloads Available

incident response

18 Observability and Reliability Available

indexes

06 Databases and Durable State Available

Ingress

15 Kubernetes and Production Workloads Available

inlining

23 JVM Performance Engineering Available

IntegrationHub

00 The System Before the Details Available

interview answer structure

00 The System Before the Details Available

IoC

03 Spring and Spring Boot Internals Available

iostat

21 Networking and Operating Systems Available

isolation

06 Databases and Durable State Available

Istio

15 Kubernetes and Production Workloads Available

JavaScript

09 The Browser and the Java Backend Available

Jenkins

16 Build, Ship and Roll Back Available

JFR

23 JVM Performance Engineering Available

JIT

23 JVM Performance Engineering Available

jitter

10 System Design Foundations Available

JMC

23 JVM Performance Engineering Available

JMH

23 JVM Performance Engineering Available

JMM

02 Concurrency and the Memory Model Available

job repository

07 Messaging, Batch and Streams Available

job scheduler

12 Twelve System Design Case Studies Available

JPA cache

04 Hibernate and JPA Available

JUnit 5

19 Testing and Quality Available

JVM memory

01 Core Java and the JVM Available

JWT

08 Security, Identity and Data Protection Available

Kafka

07 Messaging, Batch and Streams Available

Kafka Streams

07 Messaging, Batch and Streams Available

Kanban

16 Build, Ship and Roll Back Available

keep-alive

21 Networking and Operating Systems Available

keystore

08 Security, Identity and Data Protection Available

KMS

08 Security, Identity and Data Protection Available

KStream

07 Messaging, Batch and Streams Available

KTable

07 Messaging, Batch and Streams Available

Kubernetes

15 Kubernetes and Production Workloads Available

Kubernetes lease

22 Distributed Systems Theory Available

KYC

27 Fintech and Payments Engineering Available

L4 vs L7

21 Networking and Operating Systems Available

lakehouse

26 Data Platform Basics Available

Lambda

17 Cloud and Infrastructure Available

Lamport clock

22 Distributed Systems Theory Available

Last-Event-ID

05 APIs and Real-Time Communication Available

LaunchDarkly

16 Build, Ship and Roll Back Available

lazy loading

04 Hibernate and JPA Available

leader election

22 Distributed Systems Theory Available

leader-follower

22 Distributed Systems Theory Available

leaderless

22 Distributed Systems Theory Available

ledger

27 Fintech and Payments Engineering Available

lineage

26 Data Platform Basics Available

linearizability

22 Distributed Systems Theory Available

Linkerd

15 Kubernetes and Production Workloads Available

Linux

21 Networking and Operating Systems Available

Liquibase

06 Databases and Durable State Available

Little's law

23 JVM Performance Engineering Available

live-status service

12 Twelve System Design Case Studies Available

load balancing

10 System Design Foundations Available

load testing

18 Observability and Reliability Available

locks

02 Concurrency and the Memory Model Available

logger

11 Object Design, Patterns and LLD Available

logical replication

06 Databases and Durable State Available

logs

18 Observability and Reliability Available

Loki

18 Observability and Reliability Available

long polling

05 APIs and Real-Time Communication Available

LRU cache

11 Object Design, Patterns and LLD Available

LSM tree

06 Databases and Durable State Available

mapping

04 Hibernate and JPA Available

MAT

23 JVM Performance Engineering Available

Maven

16 Build, Ship and Roll Back Available

MaxRAMPercentage

23 JVM Performance Engineering Available

memory leak

23 JVM Performance Engineering Available

metrics

18 Observability and Reliability Available

microservices

25 Architecture Styles and Domain Design Available

Mockito

19 Testing and Quality Available

modular monolith

25 Architecture Styles and Domain Design Available

money state machine

27 Fintech and Payments Engineering Available

MongoDB

06 Databases and Durable State Available

monolith

25 Architecture Styles and Domain Design Available

mTLS

08 Security, Identity and Data Protection Available

MuleSoft

13 Data Integration and iPaaS Available

multi-leader

22 Distributed Systems Theory Available

multi-region

10 System Design Foundations Available

multi-stage build

14 Docker and Container Internals Available

multi-tenancy

10 System Design Foundations Available

MVCC

06 Databases and Durable State Available

MySQL

06 Databases and Durable State Available

N+1

04 Hibernate and JPA Available

namespaces

14 Docker and Container Internals Available

NAT

21 Networking and Operating Systems Available

Netty

21 Networking and Operating Systems Available

NIO

21 Networking and Operating Systems Available

notification service

11 Object Design, Patterns and LLD Available

notification system

12 Twelve System Design Case Studies Available

OAuth 2.0

08 Security, Identity and Data Protection Available

Observer

11 Object Design, Patterns and LLD Available

OIDC

08 Security, Identity and Data Protection Available

Okta

08 Security, Identity and Data Protection Available

OLAP

26 Data Platform Basics Available

OLTP

26 Data Platform Basics Available

OOMKilled

15 Kubernetes and Production Workloads Available

OOP

11 Object Design, Patterns and LLD Available

OpenAPI

05 APIs and Real-Time Communication Available

OpenFeign

03 Spring and Spring Boot Internals Available

OpenTelemetry

18 Observability and Reliability Available

optimistic locking

04 Hibernate and JPA Available

ordering

07 Messaging, Batch and Streams Available

outbox

07 Messaging, Batch and Streams Available

OWASP

08 Security, Identity and Data Protection Available

ownership boundary

00 The System Before the Details Available

PACELC

22 Distributed Systems Theory Available

page cache

21 Networking and Operating Systems Available

pagination

05 APIs and Real-Time Communication Available

parking lot

11 Object Design, Patterns and LLD Available

partitioning

06 Databases and Durable State Available

pattern matching

01 Core Java and the JVM Available

Paxos

22 Distributed Systems Theory Available

payment gateway

27 Fintech and Payments Engineering Available

payment/wallet

12 Twelve System Design Case Studies Available

PCI DSS

08 Security, Identity and Data Protection Available

PCI scope

27 Fintech and Payments Engineering Available

Pending

15 Kubernetes and Production Workloads Available

performance testing

19 Testing and Quality Available

persistence context

04 Hibernate and JPA Available

pessimistic locking

04 Hibernate and JPA Available

phi accrual

22 Distributed Systems Theory Available

PII

08 Security, Identity and Data Protection Available

PITR

06 Databases and Durable State Available

PKCE

08 Security, Identity and Data Protection Available

PKI

08 Security, Identity and Data Protection Available

Playwright

19 Testing and Quality Available

Pod

15 Kubernetes and Production Workloads Available

polling

05 APIs and Real-Time Communication Available

pool sizing

23 JVM Performance Engineering Available

ports

21 Networking and Operating Systems Available

PostgreSQL

06 Databases and Durable State Available

postmortem

18 Observability and Reliability Available

Power BI

13 Data Integration and iPaaS Available

probes

15 Kubernetes and Production Workloads Available

processes

21 Networking and Operating Systems Available

Prometheus

18 Observability and Reliability Available

proxies

03 Spring and Spring Boot Internals Available

Proxy

11 Object Design, Patterns and LLD Available

proxy buffering

05 APIs and Real-Time Communication Available

pub/sub

11 Object Design, Patterns and LLD Available

Python

13 Data Integration and iPaaS Available

quality gates

19 Testing and Quality Available

quarantine

13 Data Integration and iPaaS Available

queues

10 System Design Foundations Available

QUIC

21 Networking and Operating Systems Available

quorum

22 Distributed Systems Theory Available

RabbitMQ

07 Messaging, Batch and Streams Available

Raft

22 Distributed Systems Theory Available

rate limiter

11 Object Design, Patterns and LLD Available

rate limiting

10 System Design Foundations Available

rate-limiter service

12 Twelve System Design Case Studies Available

RBAC

15 Kubernetes and Production Workloads Available

RCA

18 Observability and Reliability Available

RDS

17 Cloud and Infrastructure Available

React

09 The Browser and the Java Backend Available

Reactor

05 APIs and Real-Time Communication Available

read timeout

21 Networking and Operating Systems Available

readiness

15 Kubernetes and Production Workloads Available

reconciliation

27 Fintech and Payments Engineering Available

reconnect

05 APIs and Real-Time Communication Available

records

01 Core Java and the JVM Available

Redis

06 Databases and Durable State Available

Redlock

22 Distributed Systems Theory Available

refresh token

08 Security, Identity and Data Protection Available

refund

27 Fintech and Payments Engineering Available

rendering

09 The Browser and the Java Backend Available

repartitioning

07 Messaging, Batch and Streams Available

replication

10 System Design Foundations Available

request journey

00 The System Before the Details Available

Resilience4j

10 System Design Foundations Available

resource limits

15 Kubernetes and Production Workloads Available

REST

05 APIs and Real-Time Communication Available

restartability

07 Messaging, Batch and Streams Available

reverse proxy

21 Networking and Operating Systems Available

rollback

16 Build, Ship and Roll Back Available

rolling deployment

16 Build, Ship and Roll Back Available

rollout

15 Kubernetes and Production Workloads Available

rotation

08 Security, Identity and Data Protection Available

RPO

18 Observability and Reliability Available

RTO

18 Observability and Reliability Available

runbook

18 Observability and Reliability Available

S3

17 Cloud and Infrastructure Available

saga

07 Messaging, Batch and Streams Available

SAML

08 Security, Identity and Data Protection Available

scalability

10 System Design Foundations Available

schema mapping

13 Data Integration and iPaaS Available

SCIM

08 Security, Identity and Data Protection Available

sealed types

01 Core Java and the JVM Available

search service

12 Twelve System Design Case Studies Available

Secret

15 Kubernetes and Production Workloads Available

secrets

08 Security, Identity and Data Protection Available

security story

24 IntegrationHub in Motion Available

select/poll/epoll

21 Networking and Operating Systems Available

sequential consistency

22 Distributed Systems Theory Available

Service

15 Kubernetes and Production Workloads Available

service discovery

03 Spring and Spring Boot Internals Available

service mesh

15 Kubernetes and Production Workloads Available

settlement

27 Fintech and Payments Engineering Available

sharding

06 Databases and Durable State Available

shared libraries

16 Build, Ship and Roll Back Available

Singleton

11 Object Design, Patterns and LLD Available

skip/retry

07 Messaging, Batch and Streams Available

SLA

18 Observability and Reliability Available

SLI

18 Observability and Reliability Available

SLO

18 Observability and Reliability Available

SNI

21 Networking and Operating Systems Available

SNS

17 Cloud and Infrastructure Available

SOLID

11 Object Design, Patterns and LLD Available

source of truth

00 The System Before the Details Available

SPA authentication

09 The Browser and the Java Backend Available

split brain

22 Distributed Systems Theory Available

spoken script

24 IntegrationHub in Motion Available

Spring Batch

07 Messaging, Batch and Streams Available

Spring Cloud Gateway

03 Spring and Spring Boot Internals Available

Spring Data

04 Hibernate and JPA Available

Spring MVC

03 Spring and Spring Boot Internals Available

Spring Security

03 Spring and Spring Boot Internals Available

SQL

06 Databases and Durable State Available

SQS

17 Cloud and Infrastructure Available

ss

21 Networking and Operating Systems Available

SSE

05 APIs and Real-Time Communication Available

star schema

26 Data Platform Basics Available

state

09 The Browser and the Java Backend Available / 11 Object Design, Patterns and LLD Available

state store

07 Messaging, Batch and Streams Available

StatefulSet

15 Kubernetes and Production Workloads Available

STOMP

05 APIs and Real-Time Communication Available

strace

21 Networking and Operating Systems Available

strangler fig

25 Architecture Styles and Domain Design Available

Strategy

11 Object Design, Patterns and LLD Available

streaming replication

06 Databases and Durable State Available

Streams

01 Core Java and the JVM Available

supply chain

16 Build, Ship and Roll Back Available

Swagger

05 APIs and Real-Time Communication Available

synchronized

02 Concurrency and the Memory Model Available

system whiteboard

24 IntegrationHub in Motion Available

task scheduler

11 Object Design, Patterns and LLD Available

TCP

21 Networking and Operating Systems Available

tcpdump

21 Networking and Operating Systems Available

tech debt

25 Architecture Styles and Domain Design Available

technology poster

24 IntegrationHub in Motion Available

Template Method

11 Object Design, Patterns and LLD Available

Terraform

17 Cloud and Infrastructure Available

test pyramid

19 Testing and Quality Available

test slices

19 Testing and Quality Available

Testcontainers

19 Testing and Quality Available

thread dump

23 JVM Performance Engineering Available

ThreadPoolExecutor

02 Concurrency and the Memory Model Available

threads

21 Networking and Operating Systems Available

TIME_WAIT

21 Networking and Operating Systems Available

timeouts

10 System Design Foundations Available

TLS

08 Security, Identity and Data Protection Available

TLS handshake

21 Networking and Operating Systems Available

token service

12 Twelve System Design Case Studies Available

tokenization

08 Security, Identity and Data Protection Available

top

21 Networking and Operating Systems Available

traces

18 Observability and Reliability Available

Transactional

03 Spring and Spring Boot Internals Available

transactions

06 Databases and Durable State Available

TTL store

11 Object Design, Patterns and LLD Available

TypeScript

09 The Browser and the Java Backend Available

ubiquitous language

25 Architecture Styles and Domain Design Available

UPI

27 Fintech and Payments Engineering Available

URL shortener

12 Twelve System Design Case Studies Available

vacuum

06 Databases and Durable State Available

validation

03 Spring and Spring Boot Internals Available

value object

25 Architecture Styles and Domain Design Available

Vault

08 Security, Identity and Data Protection Available

vector clock

22 Distributed Systems Theory Available

versioning

05 APIs and Real-Time Communication Available

virtual memory

21 Networking and Operating Systems Available

virtual threads

01 Core Java and the JVM Available / 02 Concurrency and the Memory Model Available

vmstat

21 Networking and Operating Systems Available

volatile

02 Concurrency and the Memory Model Available

VPA

15 Kubernetes and Production Workloads Available

VPC

17 Cloud and Infrastructure Available

WAL

06 Databases and Durable State Available

wallet

27 Fintech and Payments Engineering Available

warehouse

26 Data Platform Basics Available

warm-up

23 JVM Performance Engineering Available

WebFlux

05 APIs and Real-Time Communication Available

webhook delivery

12 Twelve System Design Case Studies Available

webhook dispatcher

11 Object Design, Patterns and LLD Available

webhooks

05 APIs and Real-Time Communication Available

WebSocket

05 APIs and Real-Time Communication Available

windowing

07 Messaging, Batch and Streams Available

X.509

08 Security, Identity and Data Protection Available

zero-copy

21 Networking and Operating Systems Available

ZGC

23 JVM Performance Engineering Available

ZooKeeper

22 Distributed Systems Theory Available

The Java Full-Stack Guide / IntegrationHub is fictional / Chapters 00, 01, 02, 21, 03, 04, 06, 05, 07, 26, 08, 09, 10, 25, 22, 11, 12, 27, 13, 14, 15, 16, 17, 18, 23, 19, 24, 20, 2026-10-05